

Fusion Compiler™ Design-for-Test User Guide

Version X-2025.06-SP3, March 2026



Copyright and Proprietary Information Notice

© 2026 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys, Ansys, the Synopsys and Ansys logos, and other Synopsys trademarks are available at <https://www.synopsys.com/company/legal/trademarks-brands.html>. Other company or product names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	24
Related Products, Publications, and Trademarks	24
Conventions	25
Customer Support	25

1. DFT Insertion in the Fusion Compiler Tool	27
The DFT Post-Compile Flow	28
Setting DFT Optimization Goals	29

2. Basic DFT Configuration	31
Defining DFT Signals	31
Defining Scan Signals	31
Defining Test Clocks	32
Specifying a Hookup Pin for DFT-Inserted Clock Connections	33
Specifying a Location for DFT Logic Insertion	34

3. Scan Architecture Concepts	35
Architecting Your Test Design	35
Configuring DFT Architecture	37
Defining Scan Architecture	37
Specifying Individual Scan Paths	38
Architecting Scan Chains	39
Controlling the Scan Chain Length	40
Determining the Scan Chain Count	40
Defining Individual Scan Chain Characteristics	41
Balancing Scan Chains	41
Controlling the Routing Order	45
Retiming Scan-Ins and Scan-Outs to the Leading Clock Edge	46
Reporting Scan Path Information on the Current Design	47
Architecting Scan Signals	47
Specifying Scan Signals for the Current Design	48
Selecting Test Ports	51
Controlling Scan-Enable Connections to DFT Logic	52

Keeping Specifications Consistent	57
Requirements for Valid Scan Chain Ordering	58
Lock-Up Latch Insertion Between Clock Domains	59
Automatically Creating Skew Subdomains Within Clock Domains	63
Manually Creating Skew Subdomains at Associated Internal Pins	65
Manually Creating Skew Subdomains With Scan Skew Groups	68
Enhanced Multimode Scan Architecting	69
Building Scan Chains From Base Chains	72
Retiming Registers	73
Multivoltage Support	74
Creating Separate Retention Scan Chains	75
Bypassing Scan Segments	75
Bypassing Soft-Macro Scan Segments	78
Bypassing SE/SI Controlled Hard-Macro Scan Segments	79
Scan and Stub Chains	80
Optimizing Scan Path Hold Time	81
Optimizing Scan Chain Stitching Direction	82
Limitations	83
Reducing Shift Power Using Functional Output Pin Gating	83
4. DFTMAX Compression	89
The DFTMAX Compression Architecture	89
The DFTMAX Codec	89
Decompressor Operation	91
Compressor Operation	91
Compressor X-Blocking	91
Codec Implementation	92
Optimizing DFTMAX Compression Codec	92
Configuring DFTMAX Compression	92
Reducing Power Consumption in DFTMAX Designs	95
The Shift Power Groups Architecture	95
Scan-Enable Signal Requirements for Shift Power Groups	97
Configuring Shift Power Groups	97
Integrating Cores With Shift Power Groups in Hierarchical Flows	99
Configuring the Control Chain for Shift Power Groups Cores	99

Connecting Core-Level Shift Power Disable Signals	100
Configuring Shift Power Groups for a Top-Level Codec	100
Configuring Shift Power Groups in TestMAX ATPG	101
Using Shift Power Groups With Other DFT Features	101
Per-Test-Mode Scan Compression Configuration Commands	102
Per-Partition Scan Compression Configuration Commands	103
Limitations of Shift Power Groups	103
Excluding Scan Chains From Scan Compression	105
Scan Compression and OCC Controllers	105
Using Compressed Clock Chains	106
Defining External Clock Chains	107
5. DFTMAX Ultra Compression	109
DFTMAX Ultra Compression Architecture	109
Input Shift Register and Decompression MUX	110
Control Register	111
Output XOR Compression Tree and Shift Register	111
Test Pattern Scan Procedure	112
Scan-Enable Signal Requirements for Codec Operation	113
Multiple-Input, Multiple-Output Architecture	113
Optimizing DFTMAX Ultra Compression Codec	115
Configuring DFTMAX Ultra Compression	115
Using OCC Controllers With DFTMAX Ultra Compression	117
6. DFT Partitions	118
Defining DFT Partitions	118
Configuring DFT Partitions	119
Per-Partition Scan Configuration Commands	121
set_scan_configuration	122
set_dft_signal	122
set_scan_path	122
set_wrapper_configuration	123
Known Issues of the DFT Partition Flow	123
7. Pipelined Scan Data	124

Introduction to Pipelined Scan Data	124
The Pipelined Scan Data Logic Structure	125
Configuring Pipelined Scan Data	126
Pipelined Scan Data Test Protocol Format	127
Pipelined Scan Data Limitations	127
8. Pipelined Scan-Enable Signals	129
The Pipelined Scan-Enable Architecture	129
Pipelined Scan-Enable Requirements	131
Excluding Elements From Pipelined Scan-Enable Configurations	132
Implementing Pipelined Scan-Enable Signals	134
Scannable Pipelined Scan-Enable Registers	135
Limitations	135
Reporting the Pipelined Scan-Enable Clusters	136
Implementation Considerations for Pipelined Scan-Enable Signals	136
Pipelined Scan Enable Limitations	138
PSE max Fanout per Clock Specification	138
Limitations	140
Adaptive Pipelined Scan-Enable Reclustering	140
9. Core Wrapping	142
Core Wrapping Concepts	142
Wrapper Cells and Wrapper Chains	143
Wrapper Test Modes	146
Dedicated Core Wrapping Flow	147
Dedicated Core Wrapper Cells	147
Dedicated Core Wrapper Chains	149
The Maximized Reuse Core Wrapping Flow	150
Maximized Reuse Core Wrapper Cells	151
Maximized Reuse Core Wrapper Chains	153
Inserting Hold Multiplexer (MUX) for Shared Core Wrapper Cells	155
Controlling Register Reuse	156
Special Cases for Register Reuse	159
Wrapper Shift Signals	162
Dedicated Wrapper Cell Insertion in UPF Designs	162

Contents

Optimizing UPF Aware Dedicated Core Wrapper Cell Insertion Near Core . . .	165
Wrapping a Core	166
Enabling Core Wrapping	167
Disabling Core Wrapping	168
Resetting Wrapper Settings	168
Defining Wrapper Shift Signals	168
Defining Dedicated Wrapper Clock Signals	169
Configuring Global Wrapper Settings	169
Configuring Wrapper Clock Signals	169
Configuring Dedicated Wrapper Cell Clocks	169
Controlling Wrapper Chain Count	169
Setting a Reuse Threshold	171
Configuring Port-Specific Wrapper Settings	171
Mixing Input and Output Wrapper Cells	172
Creating Wrapper Chains with Mixed Input and Output Wrapper Cells	172
Configuring Maximized Reuse Core Wrapping	172
Enabling Maximized Reuse Core Wrapping	173
Applying a Register Reuse Threshold	173
Applying a Combinational Depth Threshold	175
Specifying Port-Specific Maximized Reuse Behaviors	176
Special Cases for Register Reuse	177
Using Dedicated Wrapper Cells	179
Configuring Dedicated Wrapper Cell Clocks	179
Including Additional Scan Cells in Input and Output Wrapper Chains	180
Using the Pipelined Scan-Enable Feature	180
Low-Power Maximized Reuse Features	181
Hierarchical Core Wrapping	184
Limitations of the Maximized Reuse Flow	185
Determining Power Domains for Dedicated Wrapper Cells	186
Using the set_scan_path Command With Wrapper Chains	186
Enabling Soft Core Wrapping	188
Enabling Soft Core Wrapping for Abutted Flow	188
Enabling Soft Core Wrapping for Non-abutted Flow	190
Previewing the Wrapper Cells	193
Post DFT DRC Rule Checks	195
Creating an EXTEST-Only Core Netlist	195
Defining Core Wrapping Test Modes	196
Creating Compressed EXTEST Core Wrapping Test Modes	197
Wrapping Cores With DFT Partitions	199

Excluding Test Pins from Wrapping Across DFT Partitions	199
Limitations	200
Wrapping Cores With Multibit Registers	200
Top-Down Flat Testing With Transparent Wrapped Cores	201
Introduction to Transparent Test Modes	202
Defining Core-Level Transparent Test Modes	203
Defining Top-Level Flat Test Modes	204
Limitations	205
SCANDEF Generation for Wrapper Chains	205
Core Wrapping Scripts	206
Limitations of Core Wrapping	208

10. Clock Gating	209
Clock Gating Concepts	209
Clock-Gating Test Control Points	210
Clock-Gating Test Control Signals	213
Configuring Clock-Gating for DFT	214
Specifying Clock-Gating Control Signals	214
Excluding Clock-Gating Cells From Test-Pin Connection	216
Clock Gating for DFT Wrapper Cells	217
Clock Gating	217
Limitations	218

11. The Internal Pins Flow	219
Introduction to the Internal Pins Flow	219
Defining Signals on Internal Pins	220
Writing Out the Test Protocol	221
Limitations	221

12. Generating SCANDEF for Scan Reordering in Layout	222
Introduction to SCANDEF	222
SCANDEF Constructs	223
Writing Out the SCANDEF Information	225
Controlling START/STOP Points for SCANDEF Generation	225

Fixing the First and Last Sequential Element of the Logical Chain Using SCANDEF	226
Inserting Buffer at the Beginning and End of Logical Scan Chains	227
Limitations of SCANDEF Generation	228
Supporting SCANDEF for Pre-Stitched Scan Segments	228
Limitations	230
13. On-Chip Clocking (OCC)	231
Background	232
Clock Type Definitions	233
OCC Controller Structure and Operation	234
DFT-Inserted and User-Defined OCC Controllers	235
OCC-Controlled Clock Relationships	236
OCC Controller Signal Operation	237
Clock Chain Operation	238
Logic Representation of an OCC Controller and Clock Chain	239
Scan-Enable Signal Requirements for OCC Controller Operation	239
Enabling On-Chip Clocking Support	240
Specifying OCC Controllers	240
Specifying DFT-Inserted OCC Controllers	240
Defining Clocks	241
Defining Global Signals	242
Configuring the OCC Controller	243
Configuring the Clock Chain	244
Configuring the Clock-Gating Logic	246
Performing Timing Analysis	247
Script Example	247
Specifying Existing User-Defined OCC Controllers	248
Defining Clocks	249
Defining Global Signals	251
Specifying Clock Chains	252
Script Example	252
Specifying OCC Controllers for External Clock Sources	254
Reporting Clock Controller Information	255
DFT-Inserted OCC Controller Flow	255
Controlling the OCC Controller Bypass Path	256
Controlling the OCC/SPC Scan Chain With General Multimode	256

DFT-Inserted OCC Controller Configurations	258
Single OCC Controller Configurations	258
Example 1	258
Example 2	259
Example 3	259
Multiple DFT-Inserted OCC Controller Configurations	260
Example 1	260
Example 2	261
Waveform and Capture Cycle Example	262
Limitations	263
14. Inserting Test Points	265
Test Point Types	265
Force Test Points	266
Control Test Points	267
Observe Test Points	268
Multicycle Test Points	269
Test Point Structures	270
Test Point Components	270
Test Point Register Clocks	271
Sharing Test Point Registers	272
Automatically Inserted Test Points	274
Enabling Automatic Test Point Insertion	276
Configuring Global Test Point Insertion Settings	276
Configuring the Random-Resistant Test Point Target	277
Configuring the Untestable Logic Test Point Target	278
Configuring the ATPG Conflict Type Test Points	280
Configuring the X-Blocking Test Point Target	280
Configuring the Multicycle Path Test Point Target	281
Configuring the Shadow Wrapper Test Point Target	282
Configuring the User-Defined Test Point Target	283
Enabling Multiple Targets in a Single Command	284
Implementing Test Points From an External File	284
Customizing the Test Point Analysis	287
Running Test Point Analysis	288
Automatic Test Point Insertion Example Script	288
Limitations	289

Contents

User-Defined Test Points	289
Enabling User-Defined Test Point Insertion	290
Configuring User-Defined Test Points	290
Limitations	292
Previewing the Test Point Logic	292
Inserting the Test Point Logic	294
Inserting Power-Aware Test Points	295
Clock Gating Support for Test Point Inserted Logic	295
Limitations	296
Enhancing Test Point Physical Clustering	297
Specifying Fault Space Instances for Test Point Analysis	297
15. Previewing, Inserting, and Checking DFT Logic	299
Previewing the DFT IP	299
Inserting the DFT IP	300
Estimating Scan Chain Count	301
Pre-DFT DRC	302
Inserting Scan Structures	303
Scan Structure Insertion in UPF Designs	303
Post-DFT DRC	305
Customizing the DFT DRC Analysis	306
Using AutoFix	307
When to Use AutoFix	307
Uncontrollable Clock Signals	307
Uncontrollable Asynchronous Set and Reset Signals	308
The AutoFix Flow	309
Configuring AutoFix	311
Enabling AutoFix Capabilities	311
Configuring Clock AutoFixing	311
Configuring Set and Reset AutoFixing	312
Reporting AutoFix	313
Limitations	314
16. Running the Test DRC Debugger	315
Starting and Exiting the Graphical User Interface	315

Checking Scan Test Design Rules	316
Examining DRC Violations	317
Viewing Design Violations	318
Reporting DRC Violations	318
Inspecting DRC Violations	322
Viewing a Violation	322
Viewing Multiple Violations Together	325
Viewing test_setup Pin Data Waveforms	325
17. DRC Rules	329
Category B - Build Rules	329
DRC Rule B1	331
DRC Rule B2	331
DRC Rule B3	332
DRC Rule B4	332
DRC Rule B5	333
DRC Rule B6	333
DRC Rule B7	334
DRC Rule B8	334
DRC Rule B9	335
DRC Rule B10	336
DRC Rule B11	336
DRC Rule B12	337
DRC Rule B13	337
DRC Rule B14	338
DRC Rule B15	338
DRC Rule B16	339
DRC Rule B17	340
DRC Rule B18	341
DRC Rule B19	341
DRC Rule B20	342
DRC Rule B21	342
DRC Rule B22	343
DRC Rule B23	344
DRC Rule B24	345
DRC Rule B25	346
DRC Rule B26	346
DRC Rule B27	347

Contents

DRC Rule B28	347
DRC Rule B29	348
DRC Rule B30	348
DRC Rule B31	349
DRC Rule B32	350
DRC Rule B33	350
DRC Rule B34	351
Category C - Clock Rules	353
DRC Rule C1	355
DRC Rule C2	357
DRC Rule C3	358
DRC Rule C4	360
DRC Rule C5	362
DRC Rule C6	363
DRC Rule C7	364
DRC Rule C8	365
DRC Rule C9	367
DRC Rule C10	368
DRC Rule C11	369
DRC Rule C12	370
DRC Rule C13	371
DRC Rule C14	373
DRC Rule C15	374
DRC Rule C16	375
DRC Rule C17	376
DRC Rule C18	378
DRC Rule C19	378
DRC Rule C20	379
DRC Rule C21	381
DRC Rule C22	381
DRC Rule C23	383
DRC Rule C24	384
DRC Rule C25	385
DRC Rule C26	386
DRC Rule C27	387
DRC Rule C28	387
DRC Rule C29	388
DRC Rule C30	389

Contents

DRC Rule C31	389
DRC Rule C34	390
DRC Rule C35	391
DRC Rule C36	392
DRC Rule C37	392
DRC Rule C38	393
DRC Rule C39	393
DRC Rule C40	396
DRC Rule C41	397
DRC Rule C43	398
DRC Rule C44	398
Category D - DFT Rules	399
DRC Rule D1	400
DRC Rule D2	401
DRC Rule D3	402
DRC Rule D4	403
DRC Rule D5	404
DRC Rule D6	405
DRC Rule D7	406
DRC Rule D8	407
DRC Rule D9	408
DRC Rule D10	409
DRC Rule D11	410
DRC Rule D12	411
DRC Rule D13	412
DRC Rule D14	413
DRC Rule D15	414
DRC Rule D16	415
DRC Rule D17	416
DRC Rule D18	417
DRC Rule D20	417
DRC Rule D21	418
DRC Rule D22	419
DRC Rule D23	419
DRC Rule D39	420
Category K - LBIST Rules	420
DRC Rule K1	424
DRC Rule K2	426

Contents

DRC Rule K3	427
DRC Rule K4	429
DRC Rule K5	430
DRC Rule K6	432
DRC Rule K7	434
DRC Rule K8	435
DRC Rule K9	436
DRC Rule K10	439
DRC Rule K11	440
DRC Rule K12	442
DRC Rule K13	443
DRC Rule K14	444
DRC Rule K15	445
DRC Rule K16	446
DRC Rule K17	447
DRC Rule K18	449
DRC Rule K19	449
DRC Rule K20	450
DRC Rule K21	452
DRC Rule K22	453
DRC Rule K23	454
DRC Rule K24	455
DRC Rule K25	456
DRC Rule K26	457
DRC Rule K27	458
DRC Rule K28	460
DRC Rule K29	461
DRC Rule K30	462
DRC Rule K31	463
DRC Rule K32	464
DRC Rule K33	465
DRC Rule K34	466
DRC Rule K35	467
DRC Rule K36	468
DRC Rule K37	469
DRC Rule K38	471
DRC Rule K39	472
DRC Rule K40	473
DRC Rule K41	473

Contents

DRC Rule K42	475
DRC Rule K43	476
DRC Rule K44	476
DRC Rule K45	477
DRC Rule K46	478
DRC Rule K47	479
DRC Rule K48	480
DRC Rule K49	481
DRC Rule K50	482
DRC Rule K51	483
DRC Rule K52	484
DRC Rule K53	485
DRC Rule K54	486
DRC Rule K55	487
DRC Rule K56	488
DRC Rule K57	489
DRC Rule K58	490
DRC Rule K59	491
DRC Rule K60	492
DRC Rule K61	493
DRC Rule K62	494
DRC Rule K63	496
DRC Rule K64	497
DRC Rule K65	498
DRC Rule K66	499
DRC Rule K67	500
DRC Rule K68	501
DRC Rule K69	502
DRC Rule K70	504
DRC Rule K71	505
DRC Rule K72	506
DRC Rule K73	507
DRC Rule K74	509
DRC Rule K75	509
DRC Rule K76	511
DRC Rule K77	512
DRC Rule K78	513
DRC Rule K79	515
DRC Rule K80	516

Contents

DRC Rule K81	518
DRC Rule K82	519
DRC Rule K83	520
DRC Rule K84	521
DRC Rule K85	522
DRC Rule K86	522
DRC Rule K87	524
DRC Rule K88	525
DRC Rule K89	526
DRC Rule K90	527
DRC Rule K91	528
DRC Rule K92	528
DRC Rule K93	529
DRC Rule K94	530
DRC Rule K95	531
DRC Rule K96	532
DRC Rule K97	533
DRC Rule K98	535
DRC Rule K99	536
DRC Rule K100	537
DRC Rule K101	538
DRC Rule K102	539
DRC Rule K103	539
DRC Rule K104	540
DRC Rule K105	540
DRC Rule K106	541
DRC Rule K107	541
DRC Rule K108	542
DRC Rule K109	543
DRC Rule K110	544
DRC Rule K111	545
DRC Rule K112	546
DRC Rule K113	549
DRC Rule K114	550
DRC Rule K115	551
DRC Rule K116	552
DRC Rule K117	553
DRC Rule K118	554
DRC Rule K119	554

Contents

DRC Rule K120	555
Category N - Netlist Rules	557
DRC Rule N1	558
DRC Rule N2	558
DRC Rule N3	559
DRC Rule N4	559
DRC Rule N5	560
DRC Rule N6	560
DRC Rule N7	561
DRC Rule N8	561
DRC Rule N9	562
DRC Rule N10	562
DRC Rule N11	563
DRC Rule N12	563
DRC Rule N13	564
DRC Rule N14	565
DRC Rule N15	565
DRC Rule N16	566
DRC Rule N17	568
DRC Rule N18	568
DRC Rule N19	569
DRC Rule N20	569
DRC Rule N21	571
DRC Rule N22	572
DRC Rule N23	573
DRC Rule N24	574
DRC Rule N25	575
DRC Rule N26	576
DRC Rule N27	576
DRC Rule N28	577
DRC Rule N29	578
DRC Rule N30	578
DRC Rule N31	579
DRC Rule N32	580
DRC Rule N33	580
Category R - Compressor Rules	581
DRC Rule R1	584
DRC Rule R2	584

Contents

DRC Rule R3	584
DRC Rule R4	585
DRC Rule R5	586
DRC Rule R6	586
DRC Rule R7	586
DRC Rule R8	588
DRC Rule R9	589
DRC Rule R10	590
DRC Rule R11	592
DRC Rule R12	594
DRC Rule R13	594
DRC Rule R14	595
DRC Rule R15	596
DRC Rule R16	597
DRC Rule R17	597
DRC Rule R18	598
DRC Rule R19	599
DRC Rule R20	600
DRC Rule R21	600
DRC Rule R22	601
DRC Rule R23	602
DRC Rule R24	603
DRC Rule R25	605
DRC Rule R26	605
DRC Rule R27	606
DRC Rule R28	607
DRC Rule R29	607
DRC Rule R30	608
DRC Rule R31	609
DRC Rule R32	610
DRC Rule R33	611
DRC Rule R34	612
DRC Rule R35	613
DRC Rule R36	614
DRC Rule R37	616
DRC Rule R38	617
DRC Rule R39	618
DRC Rule R40	618
DRC Rule R41	619

Contents

DRC Rule R42	620
DRC Rule R43	620
DRC Rule R44	622
DRC Rule R45	622
DRC Rule R46	623
DRC Rule R47	624
DRC Rule R48	624
DRC Rule R49	625
DRC Rule R50	626
DRC Rule R51	627
DRC Rule R52	628
DRC Rule R53	629
DRC Rule R54	630
DRC Rule R55	630
DRC Rule R56	631
DRC Rule R57	631
DRC Rule R58	632
DRC Rule R59	632
DRC Rule R60	633
DRC Rule R61	634
DRC Rule R62	635
DRC Rule R63	636
DRC Rule R64	636
DRC Rule R65	637
Category S - Scan Chain Rules	637
DRC Rule S1	639
DRC Rule S2	641
DRC Rule S3	641
DRC Rule S4	642
DRC Rule S5	643
DRC Rule S6	643
DRC Rule S7	644
DRC Rule S8	644
DRC Rule S9	645
DRC Rule S10	646
DRC Rule S11	646
DRC Rule S12	647
DRC Rule S13	647

Contents

DRC Rule S14	648
DRC Rule S15	649
DRC Rule S16	650
DRC Rule S17	650
DRC Rule S18	651
DRC Rule S19	652
DRC Rule S20	654
DRC Rule S21	654
DRC Rule S22	655
DRC Rule S23	657
DRC Rule S24	657
DRC Rule S25	658
DRC Rule S26	659
DRC Rule S27	661
DRC Rule S28	661
DRC Rule S29	662
DRC Rule S30	663
DRC Rule S32	665
DRC Rule S34	666
DRC Rule S35	666
DRC Rule S36	667
DRC Rule S37	667
Category V - Vector Rules	668
DRC Rule V1	669
DRC Rule V2	670
DRC Rule V3	671
DRC Rule V4	671
DRC Rule V5	672
DRC Rule V6	673
DRC Rule V7	673
DRC Rule V8	674
DRC Rule V9	676
DRC Rule V10	676
DRC Rule V11	676
DRC Rule V12	677
DRC Rule V13	678
DRC Rule V14	679
DRC Rule V15	680

Contents

DRC Rule V16	681
DRC Rule V17	681
DRC Rule V18	683
DRC Rule V19	684
DRC Rule V20	685
DRC Rule V21	686
DRC Rule V22	687
DRC Rule V23	688
DRC Rule V24	689
DRC Rule V25	690
DRC Rule V26	691
DRC Rule V27	691
DRC Rule V28	692
DRC Rule V29	692
DRC Rule V31	693
DRC Rule V32	695
DRC Rule V33	696
DRC Rule V34	696
DRC Rule V35	698
DRC Rule V36	699
DRC Rule V37	700
DRC Rule V39	701
Category X - X-State Rules	701
DRC Rule X1	702
DRC Rule X2	703
Category Z - Tristate Rules	705
DRC Rule Z1	705
DRC Rule Z2	707
DRC Rule Z3	708
DRC Rule Z4	709
DRC Rule Z5	710
DRC Rule Z6	711
DRC Rule Z7	712
DRC Rule Z8	714
DRC Rule Z9	715
DRC Rule Z10	716
DRC Rule Z11	717
DRC Rule Z12	718

Contents

Category Others - All Other Rules 719

SEL-003 720

SEL-004 720

SEL-005 720

About This User Guide

The Synopsys Fusion Compiler tool provides a complete Design-for-Test (DFT) flow.

This guide describes the Fusion Compiler Design-for-Test flow. For more information about the Fusion Compiler tool, see the following companion volumes:

- *Fusion Compiler User Guide*
- *Fusion Compiler Graphical User Interface User Guide*

This user guide is for design engineers who use the Fusion Compiler tool to insert scan logic into a design.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the Fusion Compiler Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the Fusion Compiler tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

- TestMAXTM ATPG
- TestMAXTM Advisor

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code>
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

1

DFT Insertion in the Fusion Compiler Tool

The Fusion Compiler tool unifies RTL synthesis and DFT insertion, which provides improved QoR compared to traditional multistage synthesis and DFT insertion flows.

The Fusion Compiler tool supports the in-compile flow; it optimizes the location of DFT operations within the stages of the `compile_fusion` command.

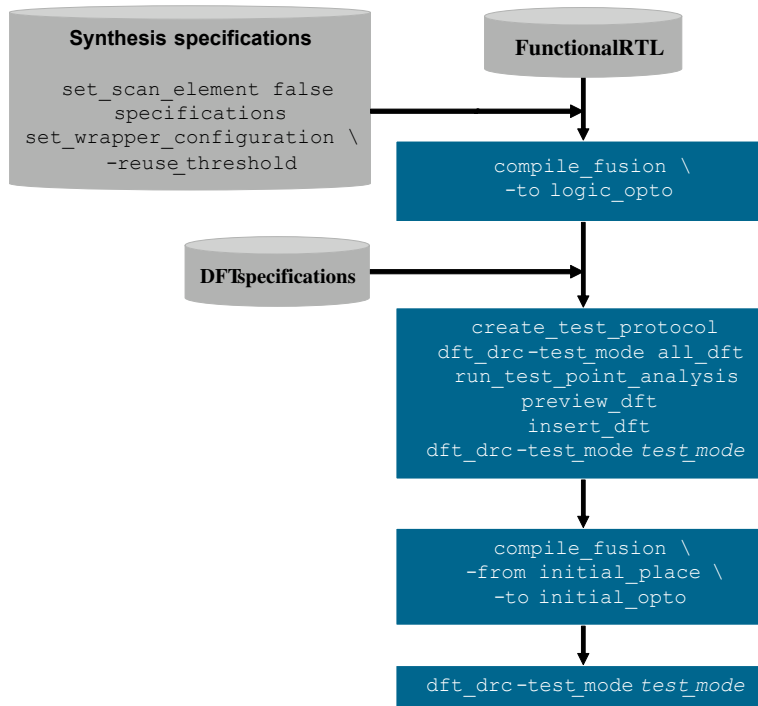
The DFT in-compile flow provides the flexibility to use the `preview_dft` command at the gate level. In addition, you can perform a second set of test DRC checks after the `insert_dft` command. During the final compile step (`compile_fusion -from initial_place`), the tool accounts for scan reordering during placement if the SCANDEF information is available.

In this flow, a few configuration steps must be performed before the first `compile_fusion` execution, which stops at the `logic_opto` stage, as follows:

- The `set_scan_element false` command controls scan replacement of sequential cells during sequential cell mapping.
- The `set_wrapper_configuration` command enables core wrapping analysis to prevent multibit banking of core wrapping registers of different types.

Other DFT configuration commands should be executed after the `compile_fusion -to logic_opto` command but before the `insert_dft` command. [Figure 1](#) illustrates the flow.

Figure 1 The DFT In-Compile Flow



Note:

The tool issues an NDMUI-441 warning message when you set the application option to enable the in-compile flow and issues a DFT-1158 warning message, when you issue any other DFT-related command.

To remove these warning messages, remove this application option from your script:

```
set_app_options -name dft.insertion_post_logic_opto -value true
```

The DFT Post-Compile Flow

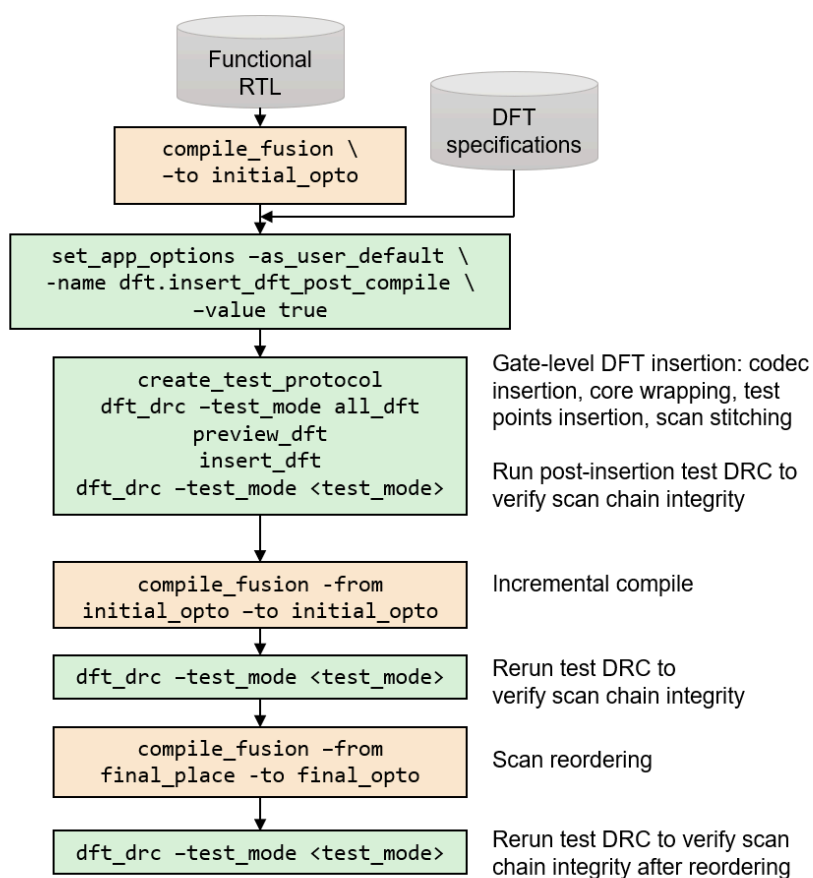
Some third-party tools require a mapped netlist to generate a DFT IP netlist to integrate LBIST, MBIST, or codec information.

In this case, the DFT post-compile flow allows you to insert third-party DFT codecs into a gate-level netlist after the `compile_fusion` command. However, you must subsequently execute an incremental compile step to incorporate the new logic by running the `compile_fusion -from initial_opto -to initial_opto` command.

Performing a standalone DFT insertion on an optimized gate-level netlist allows the tool to better consider the DFT configuration options. Debugging the DFT setup is simplified because it requires iterating only over the DFT insertion step. However, the incremental compile step requires additional runtime.

Figure 2 illustrates the steps in the post-compile flow.

Figure 2 The DFT Post-Compile Flow



Setting DFT Optimization Goals

The `optimize_dft` command performs physical DFT optimization and is automatically executed after coarse placement occurs in commands such as `create_placement`, `compile_fusion`, `place_opt`, and `clock_opt`.

You can optionally specify a priority for physical DFT optimization goals while running the `optimize_dft` command by using the `set_optimize_dft_options` command. By default, the DFT optimization goals, the `opt.dft.use_ng_engines` application option is set to `true`. To switch back to a legacy optimization engine, set the `opt.dft.use_ng_engines` application option to `false`.

For the in-compile flow, set the application option and the goal before executing the `compile_fusion -from_initial_place` command. For the post-compile flow, set the option and the goal before executing the `compile_fusion -from_initial_opto -to_initial_opto` command.

The DFT optimization goals are as follows:

- `total_interconnect_length` (default)

This goal minimizes the total scan data net length across all scan chains. In general, this setting improves routability by reducing the routed wire length.

- `max_interconnect_length`

This goal minimizes the length of the longest scan data net across all scan chains. This setting improves timing on the scan data paths. This is useful if the scan chains are shifted at a high frequency.

For example, the following command enables DFT optimization goals and sets the goal to `total_interconnect_length` by default:

```
fc_shell> set_app_options -list {opt.dft.use_ng_engines true}
```

The following commands enable DFT optimization goals and set the goal to `max_interconnect_length`:

```
fc_shell> set_app_options -list {opt.dft.use_ng_engines true}
fc_shell> set_optimize_dft_options {-goal max_interconnect_length}
```

2

Basic DFT Configuration

This chapter shows basic DFT configuration tasks that are common to all DFT insertion flows in Fusion Compiler.

It includes the following topics:

- [Defining DFT Signals](#)
- [Specifying a Hookup Pin for DFT-Inserted Clock Connections](#)
- [Specifying a Location for DFT Logic Insertion](#)

Defining DFT Signals

This topic describes how to define test clocks. It includes the following:

- [Defining Scan Signals](#)
- [Defining Test Clocks](#)

Defining Scan Signals

Fusion Compiler requires that you explicitly define all needed DFT-related signals on existing ports (or pins); it does not create new signals. Use the `set_dft_signal` command to define these signals for the current design. [Table 1](#) lists the basic scan-related signals.

Table 1 signal_type Attribute Values for Test Signals

Test I/O port signal	signal_type value	Port direction
Scan-in	ScanDataIn	Input
Scan-out	ScanDataOut	Output
Scan-enable	ScanEnable	Input
Scan clock	ScanClock	Input
Asynchronous sets/resets	Reset	Input

The following is an example of the `set_dft_signal` command specifying a scan-in port. If you enter

```
fc_shell> set_dft_signal -view spec -port SI -type ScanDataIn
```

the tool responds with the following:

```
Accepted dft signal specification for all_dft mode
```

In this example, the `-view spec` option indicates that the specified ports are to be used during DFT scan insertion and that Fusion Compiler is to perform the connections. In this example, `SI` is the name of the scan-in port that DFT insertion uses. (The other value of the `-view` argument is `-existing_dft`, which directs the tool to use the specified ports as is because they are already connected.)

Follow these guidelines when using the `set_dft_signal` command:

- Use the `set_dft_signal -view existing_dft` command if there are existing connections to this signal in your design.
- Use the `set_dft_signal -view spec` command if you expect Fusion Compiler to make connections to this signal for you.

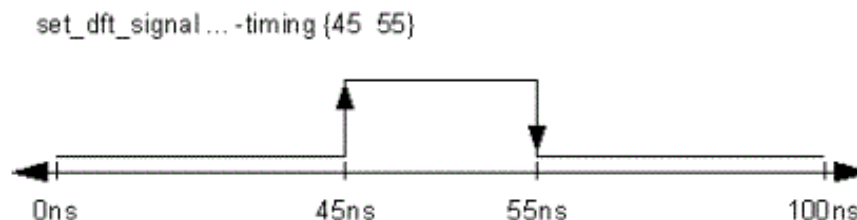
Defining Test Clocks

To explicitly define test clocks in your design, use the `set_dft_signal` command. For example,

```
fc_shell> set_dft_signal -view existing_dft -type ScanClock \  
-port CLK -timing {45 55}
```

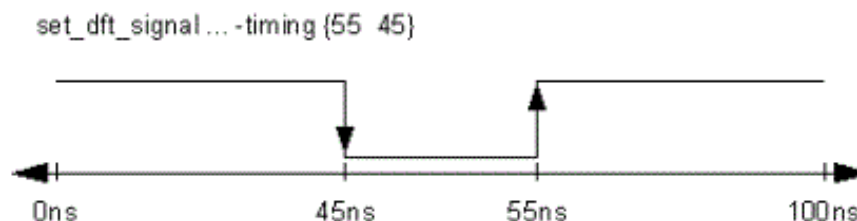
Define the test clock waveform with the `-timing` option. The waveform definition is a pair of values that specifies the rising-edge arrival time followed by the falling-edge arrival time. [Figure 3](#) shows a return-to-zero clock waveform definition.

Figure 3 Return-to-Zero Test Clock Waveform Definition



[Figure 4](#) shows a return-to-one clock waveform definition.

Figure 4 Return-to-One Test Clock Waveform Definition



For all test clocks, the clock period is the value defined by the `dft.test_default_period` application option, which defaults to 100 ns.

The rise and fall clock waveform values are the same as the values specified in the statements that make up the STIL waveform section. The rise argument becomes the value of the rise argument in the waveform statement in the test protocol clock group. The fall argument becomes the value of the fall argument in the waveform statement in the test protocol clock group.

Specifying a Hookup Pin for DFT-Inserted Clock Connections

In some cases, Fusion Compiler might need to make a connection to an existing scan clock network during DFT insertion. Some examples are

- Clock connections for DFT-inserted pipelined scan data registers
- Clock connections for test point registers
- ATE clock connections for DFT-inserted OCC controllers
- Codec clock connections for serialized and streaming scan compression
- Self-test clock connections to the LogicBIST self-test controller and codec

By default, Fusion Compiler makes the clock connection at the source port specified in the `-view existing_dft` signal definition. However, if you want the tool to make the clock connection at an internal pin, such as a pad cell or clock buffer output, you can specify it with the `-hookup_pin` option in a subsequent `-view spec` signal definition. For example,

```
fc_shell> set_dft_signal -view existing_dft -type ScanClock \  
           -port CLK -timing {45 55}  
fc_shell> set_dft_signal -view spec -type ScanClock \  
           -port CLK -hookup_pin UCLKBUF/Z
```

You do not specify the clock waveform timing for the `-view spec` signal definition, but you must specify the associated port with the `-port` option.

Specifying a Location for DFT Logic Insertion

By default, Fusion Compiler places global test logic, such as test-mode decode logic and compressed scan codecs, at the top level of the current design. Other test logic types, such as lock-up latches and reconfiguration MUXs, are placed at the local point of use.

You can specify alternative insertion locations for different types of test logic with the `set_dft_location` command:

```
set_dft_location dft_hier_name  
[-include test_logic_types]
```

The specified instance name must be a hierarchical cell. It cannot be a library cell, black box, or black-box CTL model.

If the specified hierarchical cell does not exist, the `insert_dft` command ignores the specification.

By default, all test logic is synthesized inside the specified hierarchical cell. To synthesize only some types of test logic at that location, use the `-include` option and list the test logic types to be included in the specified cell.

For example, to place the scan compression logic inside `my_cell`, and place the remaining test logic at the top level:

```
fc_shell> set_dft_location my_cell -include {CODEC}
```

The valid test logic types are:

- CODEC

This type includes the compressor and decompressor (codec) logic inserted by the tool for compressed scan, serialized compressed scan, and streaming compressed scan modes.

- WRAPPER

This type includes core wrapping cells and wrapper mode logic configured by the `set_wrapper_configuration` command. For more information, see [Chapter 9, Core Wrapping](#).

3

Scan Architecture Concepts

This chapter describes the various concepts of scan chain architecture in Fusion Compiler.

This chapter includes the following topics:

- [Architecting Your Test Design](#)
- [Requirements for Valid Scan Chain Ordering](#)
- [Lock-Up Latch Insertion Between Clock Domains](#)
- [Automatically Creating Skew Subdomains Within Clock Domains](#)
- [Manually Creating Skew Subdomains at Associated Internal Pins](#)
- [Manually Creating Skew Subdomains With Scan Skew Groups](#)
- [Enhanced Multimode Scan Architecting](#)
- [Building Scan Chains From Base Chains](#)
- [Retiming Registers](#)
- [Multivoltage Support](#)
- [Creating Separate Retention Scan Chains](#)
- [Bypassing Scan Segments](#)
- [Scan and Stub Chains](#)
- [Optimizing Scan Path Hold Time](#)
- [Optimizing Scan Chain Stitching Direction](#)
- [Reducing Shift Power Using Functional Output Pin Gating](#)

Architecting Your Test Design

This chapter describes the basic processes involved in configuring and architecting the test design for scan insertion.

This chapter includes the following topics:

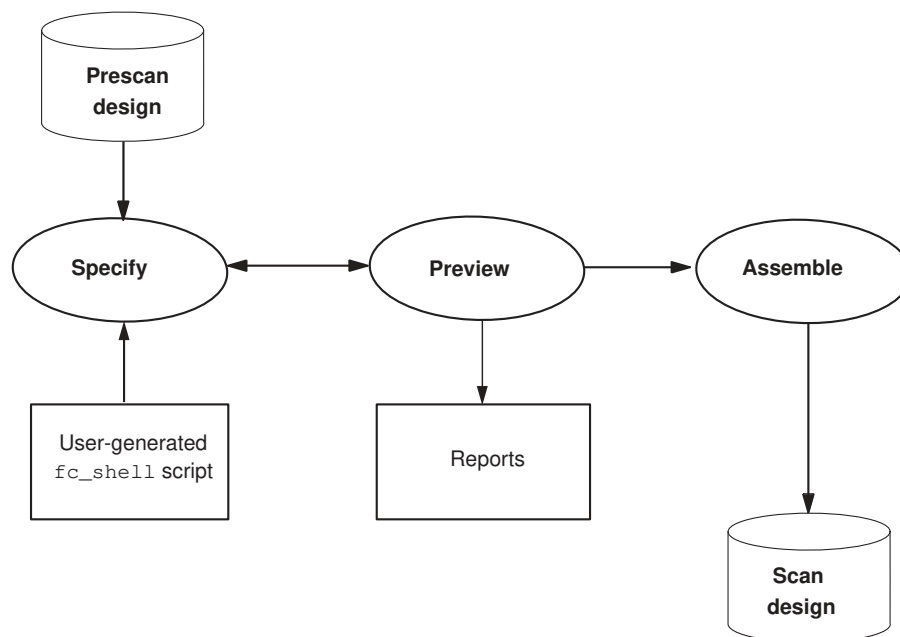
- [Configuring DFT Architecture](#)
- [Architecting Scan Chains](#)
- [Architecting Scan Signals](#)

The standard DFT architecture process consists of following steps:

- Configuring your architecture
- Building scan chains
- Connecting test signals
- Setting test clocks
- Analyzing your configurations before and after scan insertion

The following figure shows the basic flow of the scan chain generation process.

Figure 5 Scan Chain Generation Process



Configuring DFT Architecture

Before you run scan insertion, you need to configure your DFT architecture. This topic includes the following topics related to the configuration process:

- [Defining Scan Architecture](#)
- [Specifying Individual Scan Paths](#)

Defining Scan Architecture

To define your scan architecture, you need to set design constraints, define any test modes, specify test ports, and identify and mark any cells that you do not want to have scanned.

Use the following script for the basic scan assembly flow:

```
current_block top

# specify the scan architecture
set_scan_configuration -chain_count 4

# create the test protocol
create_test_protocol

# check pre-DFT DRC test design rules
dft_drc

# preview the scan structures
preview_dft

# assemble the scan structures
insert_dft

# check post-DFT DRC test design rules
dft_drc
```

Scan configuration is the specification of global scan properties for the current design. Use the `set_scan_configuration` command to specify global scan properties such as:

- Scan style and methodology
- Length and number of scan chains
- Handling of multiple clocks

Note:

This list of the `set_scan_configuration` command's options is not exhaustive. For a complete listing, and a description of each option's purpose, see the man page.

Fusion Compiler tool automatically generates a complete scan architecture from the global properties that you have defined.

Setting Design Constraints

DFT insertion uses constraints such as area and timing constraints, physical floorplan, and low power intent that were applied during the initial synthesis stage.

For more information about setting area and timing constraints on your design, see Synopsys Timing Constraints and Optimization User Guide.

Defining Constant Input Ports During Scan

If your design requires a signal to be held constant to enable DFT logic or satisfy test design rules, use the `set_dft_signal` command to define a constant or test-mode signal.

Specifying Test Ports

The `insert_dft` command adds scan signals that use existing ports. These ports are identified by using the `set_dft_signal` command.

Specifying Individual Scan Paths

Fusion Compiler supports detailed specification of individual scan paths. Use the following commands to specify the scan architecture:

- `set_scan_element`

Use this command to specify sequential elements that are to be excluded from scan replacement. You can specify leaf cells, hierarchical cells, references, library cells, and designs.

Use the `set_scan_element` command sparingly. For best results, use the command only on leaf or hierarchical cells.

- `set_scan_path`

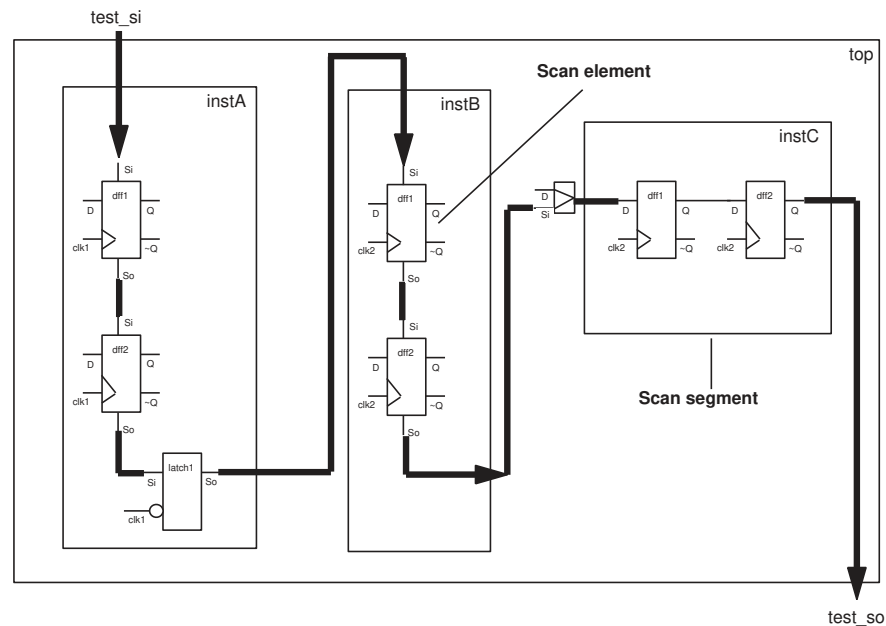
Use this command to specify properties specific to a scan chain, such as name, membership, chain length, clock association, and ordering.

- `set_dft_signal`

Use this command to specify required port connections and scan chain assignments for test signals.

In case you are unfamiliar with some of the scan path components used in the scan specification commands, see the following figure that illustrates the scan path components.

Figure 6 Scan Path Components



In the figure single scan path that starts at port `test_si`, which receives the `test_scan_in` scan signal, and ends at port `test_so`, which drives the `test_scan_out` scan signal. Cells `instA/dff1`, `instA/dff2`, `instB/dff1`, and `instB/dff2` are examples of scan elements. The shift register in `instC` is a defined scan segment. In the bottom-up flow, the scan chains in `instA` and `instB` are considered sub-chains or inferred scan segments.

Architecting Scan Chains

The `set_scan_configuration` command enables you to specify the scan chain design. This command controls almost all aspects of how the `insert_dft` command makes designs scannable. Exceptions are specific to particular scan chains and are specified in the `set_scan_path` command options.

The following covers the topics related to architecting scan chains:

- [Controlling the Scan Chain Length](#)
- [Determining the Scan Chain Count](#)
- [Defining Individual Scan Chain Characteristics](#)
- [Balancing Scan Chains](#)
- [Controlling the Routing Order](#)

- [Retiming Scan-Ins and Scan-Outs to the Leading Clock Edge](#)
- [Reporting Scan Path Information on the Current Design](#)

Controlling the Scan Chain Length

You can specify the length of scan chains for each test mode. Controlling the length of the scan chain can help to balance the scan configuration in a design that has bottom-up or system-on-a-chip (SoC) scan insertion.

Specifying the Scan Chain Length Limit

Setting the scan chain length limit helps with bottom-up scan insertion by balancing scan chains more efficiently at the top level. Setting a limit on the length of scan chains allows for design constraints related to pin availability or test vector depth.

Use the `set_scan_configuration -max_length` command to specify the length of a scan chain for each test mode:

```
fc_shell> set_scan_configuration -max_length 7 -test_mode <test mode name>
```

For example, if you set the scan chain length limit to 7 registers for a single-clock, single-edge design with 29 flip-flops, the `insert_dft` command creates five scan chains with lengths of 6, 6, 6, 6, and 5 registers. This scan chain allocation meets the scan chain length limit and also balances the scan chain lengths as closely as possible. The tool can ignore the length if there are `set_scan_group` or `set_scan_path` commands, where the length of the group or path exceeds the maximum length.

Determining the Scan Chain Count

You can specify the number of scan chains. Fusion Compiler attempts to create the specified number of scan chains while minimizing the longest scan chain length. Use these questions to decide how many scan chains to request:

- How many scan chains does your semiconductor vendor allow?

Many semiconductor vendors restrict the maximum number of scan chains due to application or tester limitations. Before performing scan specification, check with your semiconductor vendor for the maximum number of scan chains supported.

- How many clock domains exist in your design?

To prevent timing problems on the scan path in multiplexed flip-flop designs, allocate a scan chain for each clock domain (Fusion Compiler default behavior). Fusion Compiler

considers each edge of a clock a unique clock domain. Multiple clock domains do not affect the number of scan chains in scan styles other than multiplexed flip-flop.

- How much time does it take to test your design?

Because the test time is proportional to the length of the longest scan chain, increasing the number of scan chains reduces the test time for a design.

Use the `set_scan_configuration -chain_count` command to specify the number of scan chains.

```
fc_shell> set_scan_configuration-chain_count 7 -test_mode <test mode name>
```

Defining Individual Scan Chain Characteristics

Typically, scan chains are configured using mode specific chain count or chain length settings. Use the `set_scan_path` command to specify one or more additional requirements for individual scan chains in the current design.

The `set_scan_path` command enables you to

- Specify a name for a scan chain
- Allocate scan cells, scan segments, and subdesign scan chains to scan chains and specify the ordering of the scan chain
- Specify a dedicated scan-in and scan-out ports
- Limit a scan chain's elements to only those components you specify or enable Fusion Compiler balance scan chains by adding more elements
- Assign scan chains to clock domains

Scan chain elements cannot belong to more than one chain in a particular test mode. The command options are not incremental. Where `set_scan_path` commands conflict, the `preview` command (`preview_dft`) and scan insertion command (`insert_dft`) apply the most recent command.

Balancing Scan Chains

The `insert_dft` command always attempts to balance the number of cells in each scan chain. However, some scan chain requirements can limit or disable balancing, such as

- Disabling clock mixing or clock edge mixing
- Defining scan chains with the `set_scan_path` command
- Defining scan groups with the `set_scan_group` command
- Using test models

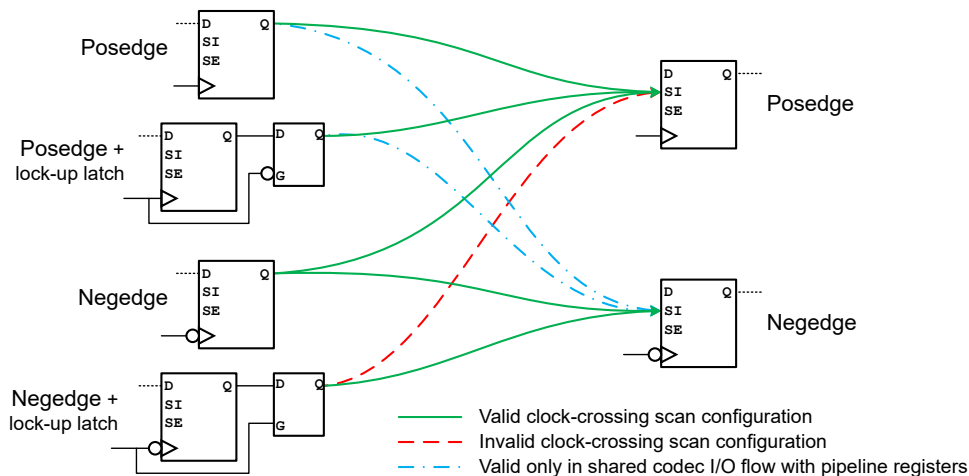
- Using a hierarchical DFT insertion flow
- Specifying a global scan chain exact length

When overriding the default behavior, always use the `preview_dft` command to verify that the result meets your requirements.

Concatenating Scan Cells and Segments

When concatenating scan cells or segments to form scan chains, the `insert_dft` command can join them using only the scan cell sequences shown in the following figure. The tool might insert additional synchronization elements, such as lock-up latches or retiming flip-flops (not shown), to create the sequences.

Figure 7 Valid Scan Cell and Segment Concatenation Sequences



This figure is relevant within a single clock domain or between clock domains with the same test clock waveforms. Between test clocks with differing waveforms, the tool determines the valid scan cell sequences using the waveform timing.

Multiple Clock Domains

The *clock edge* of a scan cell represents both the clock identity and the active clock edge of the cell. For multiplexed flip-flop designs, Fusion Compiler allocates cells to scan chains based on clock edges by default. You can override this default behavior by using the `set_scan_configuration -clock_mixing` command.

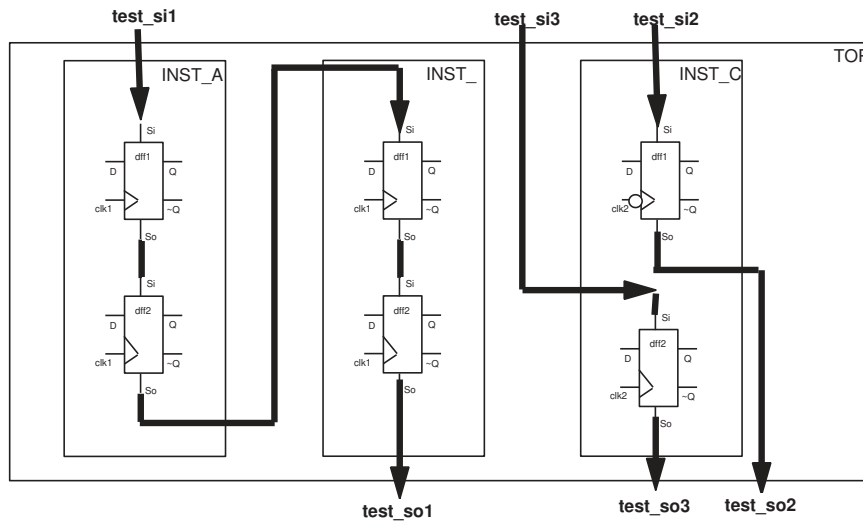
For example, assume that you have a design with three clock domains and your required scan architecture contains two balanced scan chains.

```
fc_shell> set_dft_signal -view existing_dft \
-type ScanClock -timing [list 45 55] \
-port {clk1 clk2}
```

```
fc_shell> set_scan_configuration -chain_count 2
```

In the default case shown in the following figure, the tool overrides your request for two chains and generates three scan chains, one for each clock edge (clk1, positive-edge clk2, negative-edge clk2). Because the clock domains contain unequal numbers of cells, the tool generates unbalanced scan chains.

Figure 8 Unbalanced Scan Chains Due to Multiple Clock Domains



You can reduce the number of scan chains and achieve slightly better balancing by mixing clock edges within a single chain.

```
fc_shell> set_dft_signal -view existing_dft \
-type ScanClock -timing [list 45 55] \
-port {clk1 clk2}
```

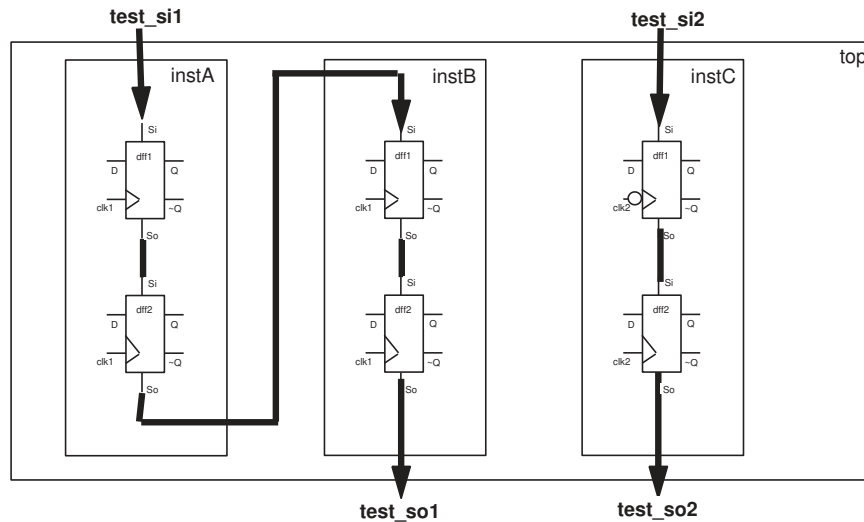
```
fc_shell> set_scan_configuration -chain_count 2
```

```
fc_shell> set_scan_configuration -clock_mixing mix_edges
```

Mixing clock edges in a single scan chain produces a small timing risk. Fusion Compiler automatically orders the cells within the scan chain so the cells clocked later in the cycle appear earlier in the scan chain, resulting in a functional scan chain. This is also known as the "NICE" rule, where the Next Instance must be clocked Concurrently or Earlier in the Tester Cycle.

The following figure shows the scan architecture when you allow edge mixing.

Figure 9 Better Balancing With Mixed Clock Edges



You can balance the scan chains by mixing clocks as follows:

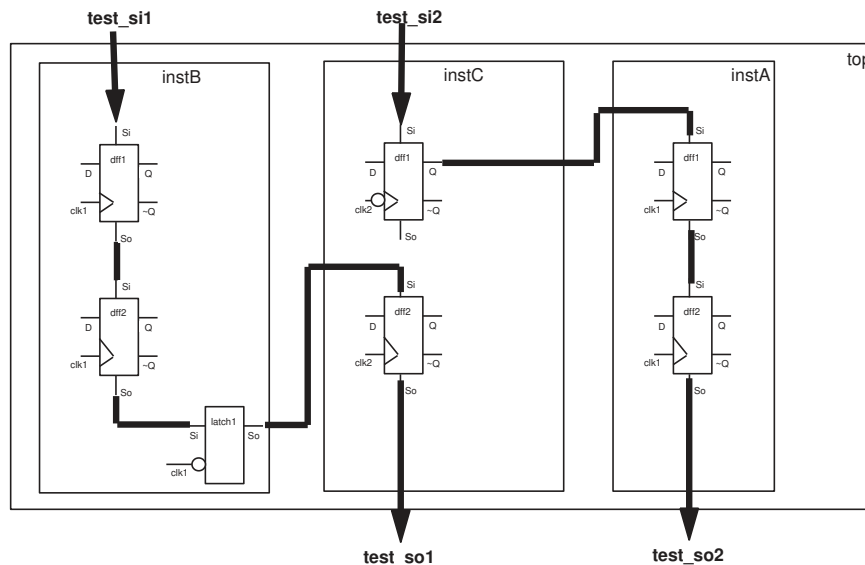
```
fc_shell> set_dft_signal -view existing_dft \
               -type ScanClock -timing [list 45 55] \
               -port {clk1 clk2}
```

```
fc_shell> set_scan_configuration -chain_count 2
```

```
fc_shell> set_scan_configuration -clock_mixing mix_clocks
```

Directly mixing clock in a single scan chain could produce a large timing risk. To reduce this risk, Fusion Compiler adds lock-up latches to the scan path wherever clock skew issues might occur. The following figure shows the resulting scan architecture.

Figure 10 Balanced Scan Chains With Mixed Clocks



Controlling the Routing Order

Use the `set_scan_path` command to control the routing order explicitly. Each `set_scan_path` command generates a scan chain; Fusion Compiler uses the first command argument as the scan chain name. If you enter multiple `set_scan_path` commands with the same scan chain name, it uses only the last command entered.

You can provide partial or complete scan ordering specifications. Use the `-complete true` option to indicate that you have completely specified a scan chain. The tool does not add cells to a completely specified scan chain. If you provide a partial scan-ordering specification, it might add cells to the scan chain. The cells are placed specified in a partial ordering at the end of the scan chain.

Validates the specified scan ordering. Following is the check performed:

- Clock mixing

The tool verifies that all cells on a scan path have the same clock unless you have specifically requested clock mixing. A violation triggers the following warning message during execution of the `preview_dft` command:

```
Warning: Chain '%s' has elements clocked by different clocks
(DFT-2803)
```

Creates the requested scan chain. Unless you have disabled scan lock-up latch insertion, the tool inserts a scan lock-up latch between clock domains.

Retiming Scan-Ins and Scan-Outs to the Leading Clock Edge

In some cases, hierarchical blocks can contain scan chains with a mix of positive edge-triggered and negative edge-triggered flip-flops. When these block-level scan chains are combined at a higher level in the design hierarchy to form longer scan chains, half-cycle paths across clock edges might be created between the scan chains. Meeting timing for these half-cycle scan chain paths can be challenging at higher frequencies, especially for chips with long top-level routes between blocks.

To avoid these half-cycle paths when block-level scan chains are combined, use the `-add_test_retiming_flops` option of the `set_scan_configuration` command. For example,

```
fc_shell> set_scan_configuration -add_test_retiming_flops begin_and_end
```

When this option is specified for block-level scan chain insertion, flip-flops triggering on the leading edge of the test clock are added as needed to any scan chains that begin or end with trailing-edge-triggered flip-flops. For return-to-zero clocks, rising-edge flip-flops are used to retime to the leading edge. For return-to-one clocks, falling-edge flip-flops are used to retime to the leading edge. The tool automatically chooses the edge-triggered retiming cell from the target library.

Valid keywords for the `-add_test_retiming_flops` option are `begin_and_end`, `begin_only`, `end_only`, and `none`. The default is `none`, which disables retiming register insertion.

The following table shows the retiming behaviors provided by the `-add_test_retiming_flops` option.

Table 2 *Retiming Behaviors Provided by the -add_test_retiming_flops Option*

If the scan chain	<code>begin_only</code>	<code>end_only</code>	<code>begin_and_end</code>
Begins with a leading-edge flip-flop			
Begins with a trailing-edge flip-flop	Adds retiming flip-flop to beginning of scan chain		Adds retiming flip-flop to beginning of scan chain
Ends with a leading-edge flip-flop			
Ends with a trailing-edge flip-flop		Adds retiming flip-flop to end of scan chain	Adds retiming flip-flop to end of scan chain

To report the locations where these retiming flip-flops are to be added, use the `preview_dft` command.

In addition, when this feature is enabled (with any value other than `none`), the following are clocked on the leading edge:

- Head pipeline registers
- DFT-inserted clock chains
- Shift-power control (SPC) chains
- DFTMAX Ultra decompressor registers
- DFTMAX serializer decompressor registers

If you generate a SCANDEF file for a design with retiming flip-flops, the retiming flip-flops are not included between the START and STOP points in the SCANDEF file. Only the original design scan cells are included between the START and STOP points. This feature controls only one aspect of retiming flip-flop insertion.

Reporting Scan Path Information on the Current Design

Use the `report_scan_path` command to display scan path information for the current design.

Note:

To show changes caused by running the `insert_dft` command, you must run the `dft_drc` command before the `report_scan_path` command. Running an incremental compile or any other command that changes the database causes the `dft_drc` results to be discarded. In such a case, you need to run `dft_drc` again before you use `report_scan_path` command.

[Example 1](#) shows the type of information displayed by the `report_scan_path -chain all` command.

Example 1 Scan Path Information Displayed by the `report_scan_path` Command

```
=====
AS BUILT BY insert_dft
=====
```

Scan_path	Len	ScanDataIn	ScanDataOut	ScanEnable	MasterClock	SlaveClock
I 1	22	test_si1	test_so1	test_se	CLK	-
I 2	21	test_si2	test_so2	test_se	CLK	-
I 3	21	test_si3	test_so3	test_se	CLK	-

For more information, see man page for the `report_scan_path` command.

Architecting Scan Signals

For test design rule check, to recognize test ports in your design, your scan-inserted design must have appropriate `signal_type` attributes on the test ports. If you are using

your own placeholder test ports, you must set these attributes with the `set_dft_signal` command.

The following topics discuss the process for architecting scan signals:

- [Specifying Scan Signals for the Current Design](#)
- [Selecting Test Ports](#)
- [Keeping Specifications Consistent](#)

Specifying Scan Signals for the Current Design

Use the `set_dft_signal` command to specify one or more scan signals for the current design.

[Table 3](#) provides a list of `signal_type` attribute values.

Table 3 *signal_type Attribute Values for Test Signals*

Test I/O port signal	signal_type value	Valid on input	Valid on output
Scan-in	ScanDataIn	Yes	No
Scan-out	ScanDataOut	No	Yes
Scan-enable	ScanEnable	Yes	No
Bidirectional enables	InOutControl	Yes	No
Asynchronous control ports	Reset	Yes	No

The following example specifies a scan-in port of the `set_dft_signal` command. If you enter

```
fc_shell> set_dft_signal -view spec -port scan_in -type ScanDataIn
```

The tool responds with the following message:

```
Accepted dft signal specification for modes: all_dft
```

In the preceding example, the `-view spec` option indicates that the specified ports are to be used during DFT scan insertion and that the tool must perform the connections. In this example, `scan_in` is the name of the scan-in port that the `insert_dft` command uses. (The other value of the `-view` argument is `-existing_dft`, which directs the tool to use the specified ports available because they are already connected.

Follow these guidelines when using the `set_dft_signal` command:

- Use the `set_dft_signal` command for scan insertion and for design rule checking. The `set_dft_signal` command indicates I/O ports that are to be used as scan ports. After the `insert_dft` command connects these ports, it places the necessary `signal_type` attributes on the ports for design rule checking after insertion.
- Use the `set_dft_signal -view existing_dft` command if you read an ASCII netlist and you need to perform design rule checking. Before you use the `set_dft_signal` command, the ASCII netlist does not contain the `signal_type` attributes annotated by scan insertion. Without these attributes, `dft_drc` does not know which ports are scan ports and therefore reports that the design is untestable.
- Use the `set_dft_signal -view existing_dft` command if the ports in your design are already connected and no connection is made.
- Use the `set_dft_signal -view spec` command if the connections do not exist in your design and you expect the tool to make the connections for you.

Using the `-view spec` and `-view existing_dft` Arguments

Unlike other tools used in the implementation flow, Fusion Compiler changes the functionality of your design such that the design can operate in either functional (“mission”) mode or test mode.

To construct this dual modality, it needs to know what information already exists in the design. You can use the `-view existing_dft` option with the `set_dft_signal` command to provide such information. The tool then uses this information to perform pre-insertion design rule checking (DRC) to determine the elements that can be incorporated into scan chains.

Examples that use the `-view existing_dft` option include:

- Clock signals:

```
set_dft_signal -view existing_dft -type ScanClock -port \
  clk -timing {45 55}
```

- Asynchronous set and reset signals:

```
set_dft_signal -view existing_dft -type Reset -port rst \
  -active_state 0
```

The `-view spec` option is used to specify ports that are to function as scan-in and scan-out ports (either dedicated scan-in and scan-out ports or shared functional ports used also as scan-in and scan-out ports), such as

```
set_dft_signal -view spec -type ScanDataIn -port scan_in_1
set_dft_signal -view spec -type ScanDataOut -port scan_out_1
```

and

```
set_dft_signal -view spec -type ScanDataIn -port data_in_bus_2
set_dft_signal -view spec -type ScanDataIn -port data_out_bus_2
```

As a general rule,

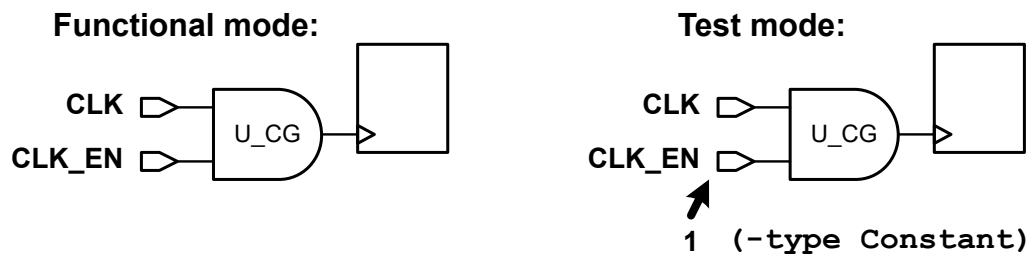
- If the information is needed for pre insertion DRC, then it should be specified by using the `-view existing_dft` option.
- If the information is needed to build DFT structures, then it should be specified by using the `-view spec` option.

Using `-type Constant` versus Using `-type TestMode`

The difference between `Constant` and `TestMode` signal definitions is as follows:

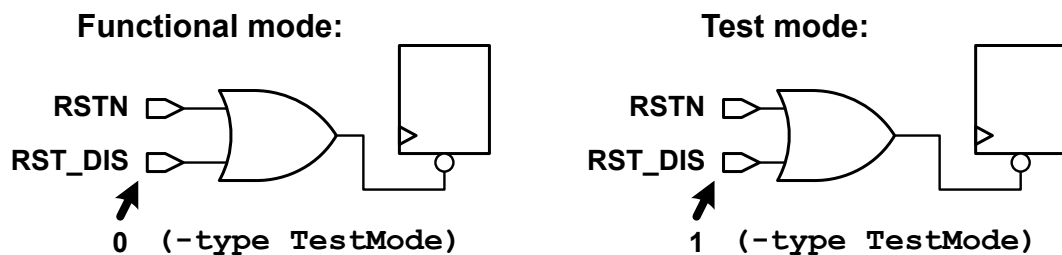
- `Constant` signal definitions specify the value of the signal in test mode, but they do not specify anything about the signal's value in functional mode. [Figure 11](#) shows the behavior of a `Constant` signal in the functional and test modes.

Figure 11 Constant Signal Specification



- `TestMode` signal definitions specify the value of the signal in both test mode and functional mode. [Figure 12](#) shows the behavior of a `TestMode` signal in the functional and test modes.

Figure 12 Design With a Controlled Clock Signal



This difference affects the generation of verification setup files, which are specified with the `set_svf` command. Verification setup files contain information about the functional mode values for any defined DFT signal. This allows Formality equivalence checking to

disable the test logic added by the tool when the `synopsys_auto_setup` variable is set to `true`.

This difference affects verification setup file generation for the preceding examples as follows:

- The `Constant` signal definition does not result in any verification setup file directives because it contains no information about the functional mode value of `CLK_EN`.
- The `TestMode` signal definition specifies that the functional mode value for the `RST_DIS` signal is 1. This results in the following verification setup file directive:

```
guide_scan_input \  
  -design { top } \  
  -disable_value 0 \  
  -ports { RST_DIS }
```

Selecting Test Ports

You can share ports between test and normal operation, which minimizes the number of dedicated test ports required for internal scan. If there is no support to this configuration from the vendor, you can request dedicated scan output ports. Always use dedicated ports for scan-enable and test clock signals.

The following topics describe how to select and define existing ports in your design as test ports:

- [Defining Existing Unconnected Ports as Scan Ports](#)
- [Sharing a Scan Input With a Functional Port](#)
- [Sharing a Scan Output With a Functional Port](#)
-

Defining Existing Unconnected Ports as Scan Ports

You can define existing unconnected ports in your RTL description for use as test ports. These are known as placeholder scan ports. This approach allows you to use the same testbench for the RTL and gate-level implementations of your design.

Use the `set_dft_signal` command to instruct the tool to use these ports:

```
fc_shell> set_dft_signal -type ScanDataIn -view spec \  
-port SI1
```

```
fc_shell> set_dft_signal -type ScanEnable -view spec \  
-port SE \  
-active_state 1
```

```
fc_shell> set_dft_signal -type ScanDataOut -view spec \  
-port SO
```

Sharing a Scan Input With a Functional Port

To share a scan input port with a specified existing functional port, use the `set_dft_signal` command.

```
fc_shell> set_dft_signal -type ScanDataIn -view spec \  
-port DATA_in[0].
```

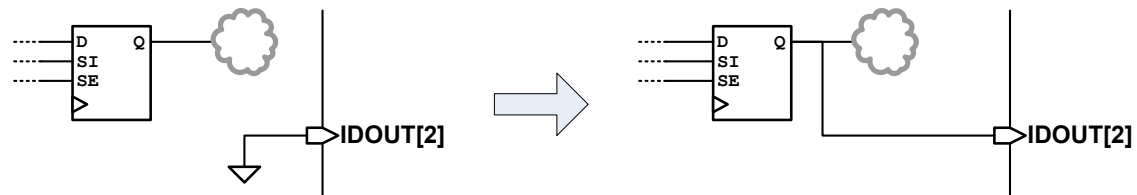
Sharing a Scan Output With a Functional Port

To select the functional port to be used as a scan output port, use the `set_dft_signal` command.

```
fc_shell> set_dft_signal -type ScanDataOut -view spec \  
-port DATA_out[0]
```

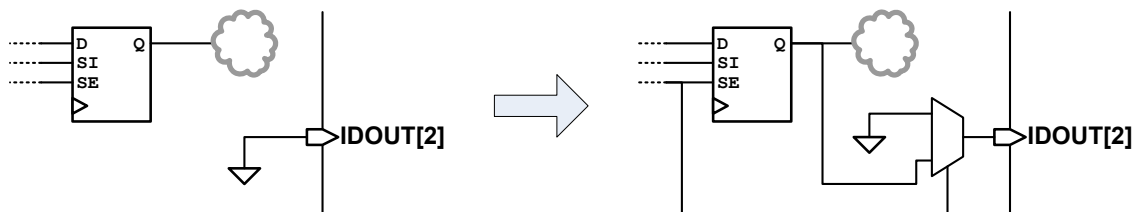
If a scannable sequential cell drives the specified output port, the tool places that cell last in the scan chain. Otherwise, the tool automatically adds the control or multiplexing logic required to share the scan output port with the functional output port.

Figure 13 Scan Output Port With Constant MUXing Disabled



If the existing constant value driving the port is required for proper operation in functional mode, set the `dft.scan.test_mux_constant_so` application option to `true`. In this case, the tool multiplexes the scan-out signal with the constant value, using the scan-enable signal to control the multiplexer, as shown in the following figure.

Figure 14 Scan Output Port With Constant MUXing Enabled



Controlling Scan-Enable Connections to DFT Logic

By default, Fusion Compiler uses a global scan-enable signal for DFT logic connections. In some cases, you might want to create multiple scan-enable signals and control how they connect to the DFT logic.

The following topics describe how to control scan-enable connections to DFT logic:

- [Associating Scan-Enable Ports With Specific Scan Chains](#)
- [Defining Dedicated Scan-Enable Signals for Scan Cells](#)
- [Connecting the Scan-Enable Signal in Hierarchical Flows](#)
- [Preserving Existing Scan-Enable Pin Connections](#)

Associating Scan-Enable Ports With Specific Scan Chains

To associate a specific port with specific scan chains, use the `set_dft_signal` and `set_scan_path` commands, as follows:

```
fc_shell> set_dft_signal -type ScanEnable -view spec \  
-port port_name -active_state 1
```

```
fc_shell> set_scan_path {chain_names} -view spec \  
-scan_enable port_name
```

If the condition set with these commands cannot be met, a warning message is issued during scan preview and scan insertion.

Defining Dedicated Scan-Enable Signals for Scan Cells

By default, Fusion Compiler chooses an available ScanEnable signal to connect to the scan-enable pins of scan cells. However, you can also define a dedicated ScanEnable signal to use for these scan-enable pin connections.

Specifying a Global Scan-Enable Signal

You can define a global ScanEnable signal to use for the scan-enable pins of scan cells by using the `-usage scan` option when defining the signal with the `set_dft_signal` command:

```
set_dft_signal  
-type ScanEnable  
-view spec  
-usage scan  
-port port_list
```

To define a signal with the `scan` usage, the `-view` option must be set to `spec`.

When you define a ScanEnable signal with the `scan` usage, the `insert_dft` command is limited to using only that signal to connect to the scan-enable pins of scan cells.

You can use the `report_dft_signal` and `remove_dft_signal` commands for reporting and removing the specification, respectively.

Specifying Object-Specific Scan-Enable Signals

You can also define dedicated ScanEnable signals for specific parts of the design by using the `-connect_to` option and associated options of the `set_dft_signal` command:

```
set_dft_signal
  -type ScanEnable
  -view spec
  -usage scan
  -port port_list
  [-connect_to object_list]
```

The `-connect_to` option specifies a list of design objects that use the specified ScanEnable signal. The supported object types are

- Scan cells
- Hierarchical cells
- Designs
- Test clock ports

This allows you to make clock-domain-based signal connections. It includes scan cells clocked by the specified test clocks. The functional clock behavior is not considered.

- Scan-enable pins of CTL-modeled cores

The following example defines two ScanEnable signals, named `SE_CLK1` and `SE_CLK2`, to connect to the scan-enable pins of test clock domains `CLK1` and `CLK2`, respectively:

```
fc_shell> set_dft_signal -type ScanClock -view existing_dft \
               -port {CLK1 CLK2} -timing {45 55}
fc_shell> set_dft_signal -type ScanEnable -view spec -usage scan \
               -port SE_CLK1 -connect_to {CLK1}
fc_shell> set_dft_signal -type ScanEnable -view spec -usage scan \
               -port SE_CLK2 -connect_to {CLK2}
```

Note the following limitation:

- You cannot specify object-specific scan-enable specifications for pipelined scan-enable signals.

Connecting the Scan-Enable Signal in Hierarchical Flows

When you insert DFT at a top level that contains cores, which are DFT-inserted blocks represented by CTL models, the cores already contain complete scan-enable networks. Instead of connecting the top-level ScanEnable signal to target pins inside the core, Fusion Compiler must connect to ScanEnable signal pins at the core boundary.

When ScanEnable signals at the core and top level are defined with the `-usage` option of the `set_dft_signal` command, Fusion Compiler attempts to determine which top-level signal should drive each core-level signal.

You can override the default connection behaviors for cores by using object-specific signal definitions at the top level, applied using the `set_dft_signal -connect_to` command and associated options:

```
set_dft_signal
  -type ScanEnable
  -view spec
  -usage scan | clock_gating
  -port port_list
  [-connect_to object_list]
```

All object types accepted by the `object_list` argument apply to cores. The object types that apply to cores are:

- Test clock ports

This allows you to make clock-domain-based signal connections to cores. It includes core-level scan-enable pins associated with the specified test clocks. The functional clock behavior is not considered.

- Scan-enable pins of CTL-modeled cores

This allows you to make direct pin-to-pin connections from top-level signal sources to core-level pins.

Specifying Domain-Based Connections to Core Scan-Enable Pins

When you specify a top-level domain-based signal connection, Fusion Compiler uses information inside a core's CTL model to determine the core scan-enable pins associated with each core clock. To ensure that this information is present in the model, the following requirements must be observed during core creation:

- Core-level ScanEnable signals must be defined using the `-usage` option of the `set_dft_signal` command. This ensures that the core's internal scan-enable signal is not used outside its intended usage.
- Core-level ScanEnable signals defined with an usage of `clock_gating` must also be defined as domain-specific signals using the `-connect_to clock_list` option of the `set_dft_signal` command. This ensures that clock-specific clock-gating annotations are included in the CTL model.

[Example 2](#) shows part of a core-level ASCII CTL model that contains clock domain information for a scan-enable signal defined with an usage of `scan`.

Example 2 Scan Chain Clock Information in an ASCII CTL Model

```
CTL all_dft {
  ...
  Internal {
    "SE_SCAN" {
      CaptureClock "CLK" {
        LeadingEdge;
      }
      DataType User "ScanEnableForScan" {
        ActiveState ForceUp;
      }
    }
  }
}
```

Example 3 shows part of a core-level ASCII CTL model that contains clock domain information for a scan-enable signal defined with an usage of `clock_gating`. The domain-based signal specification causes the `CaptureClock` constructs to be included.

Example 3 Clock-Gating Clock Information in an ASCII CTL Model

```
CTL all_dft {
  ...
  Internal {
    "SE_CG" {
      CaptureClock "CLK" {
        LeadingEdge;
      }
      DataType User "ScanEnableForClockGating" {
        ActiveState ForceUp;
      }
    }
  }
}
```

The CTL model information for a signal can contain multiple `DataType` constructs (for signals defined with multiple usages) and multiple `CaptureClock` constructs (for signals associated with multiple clocks).

Consider a core with two scan-enable pins, where pin SE1 is associated with CLK1 and pin SE2 is associated with CLK2. The following top-level commands connect corresponding top-level scan-enable signals TOP_SE1 and TOP_SE2 to these core-level pins indirectly using domain-based specifications:

```
fc_shell> set_dft_signal -type ScanClock -view existing_dft \
    -port {CLK1 CLK2} -timing {45 55}
fc_shell> set_dft_signal -type ScanEnable -view spec -usage scan \
    -port TOP_SE1 -connect_to {CLK1}
fc_shell> set_dft_signal -type ScanEnable -view spec -usage scan \
    -port TOP_SE2 -connect_to {CLK2}
```


Specifying Connections Directly to Core Scan-Enable Pins

You can specify pin-based signal connection specifications that connect any top-level ScanEnable signal to any core-level ScanEnable pin. These pin-based connection specifications override the default connection behavior, and there is no requirement for the top-level and core-level signal usages to match.

Consider a core with two scan-enable pins, where pin SE1 is associated with CLK1 and pin SE2 is associated with CLK2. The following top-level commands connect top-level scan-enable signals to corresponding core-level pins using direct core pin specifications:

```
fc_shell> set_dft_signal -type ScanEnable -view spec -usage scan \  
               -port TOP_SE1 -connect_to {CORE/SE_SCAN1}  
fc_shell> set_dft_signal -type ScanEnable -view spec -usage scan \  
               -port TOP_SE2 -connect_to {CORE/SE_SCAN2}
```

Preserving Existing Scan-Enable Pin Connections

During DFT insertion, Fusion Compiler identifies the scan-enable pins of scan cells and scan cores that should be connected to the global scan-enable signal. These are known as scan-enable target pins.

By default, if a scan-enable target pin already has a connection, the tool disconnects it to make the connection to a scan-enable signal.

To preserve existing connections to scan-enable target pins during DFT insertion, enable the following application option:

```
fc_shell> set_app_option -name dft.scan.test_keep_connected_scan_en  
               -value true
```

For more information, see the man page. For an example application, see [SolvNet article 034774](#), “How To Connect DFT Signals to Hierarchical Pins of Verilog Wrappers.”

Keeping Specifications Consistent

The set of user specifications contributing to the definition of the scan design must be consistent. User-supplied specification commands forming part of a consistent specification have the following characteristics:

- Each specification command is self-consistent. It cannot contain mutually exclusive requirements. For example, a command specifying the routing order of a scan chain cannot specify the same element in more than one place in the chain.
- All specification commands are mutually consistent. Two specification commands must not impose mutually exclusive conditions on the scan design. For example, two specification commands that place the same element in two different scan chains are mutually incompatible.

- All specification commands yield a functional scan design. You cannot impose a specification that leads to a nonfunctional scan design. For example, a specification that mandates fewer scan chains than the number of incompatible clock domains is not permitted.

The number of clock domains in your design, together with your clock-mixing specification, determines the minimum number of scan chains in your design. If you specify an exact number of scan chains smaller than this minimum, the `insert_dft` command issues a warning message and implements the minimum number of scan chains.

Requirements for Valid Scan Chain Ordering

This topic describes the requirements for valid scan chain ordering in the multiplexed flip-flop scan style.

Fusion Compiler generates valid mixed-clock scan chains based on the ideal test clock timing. Scan chain cells are ordered by the ideal test clock edge times, as defined with the `-timing` option of the `set_dft_signal` command. Cells clocked by later clock edges are placed before cells clocked by earlier clock edges. This guarantees that all cells in the scan chain get the expected data during scan shift.

Figure 15 shows the ideal test clock waveforms for two test clocks. The clock edges are numbered by their edge timing order, with the latest clock edge indicated by (1).

Figure 15 Ideal Test Clock Waveforms for Two Test Clocks

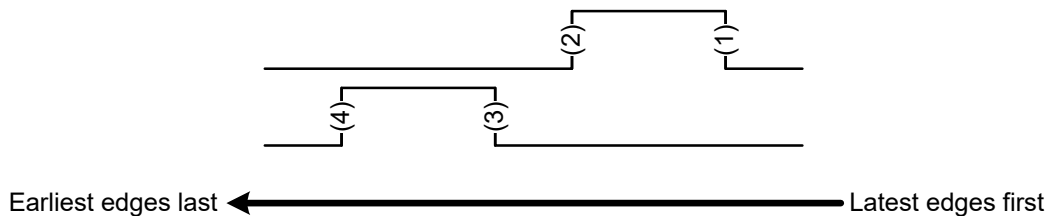
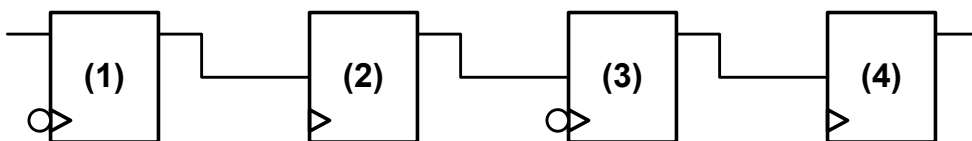


Figure 16 shows how Fusion Compiler constructs a scan chain containing a scan cell clocked by each clock edge. The scan cells are ordered with the cells clocked by the latest clock edges coming first.

Figure 16 Scan Chain Cells for Two Test Clocks



To maintain the validity of your scan chains, do not change the test clock timing after assembling the scan structures.

Although Fusion Compiler chooses an order that ensures correct shift function under ideal clock timing, it cannot guarantee that capture problems will not occur. Capture problems are caused by your logic functionality; modify your design to correct capture problems.

By default, when you request clock mixing within a multiplexed flip-flop scan chain, Fusion Compiler inserts lock-up latches to prevent timing problems. For more information, see [Lock-Up Latch Insertion Between Clock Domains](#).

Lock-Up Latch Insertion Between Clock Domains

A *scan lock-up latch* is a retiming sequential cell on a scan path that can address skew problems between adjacent scan cells when clock mixing or clock-edge mixing is enabled. Fusion Compiler inserts them to prevent skew problems that might occur.

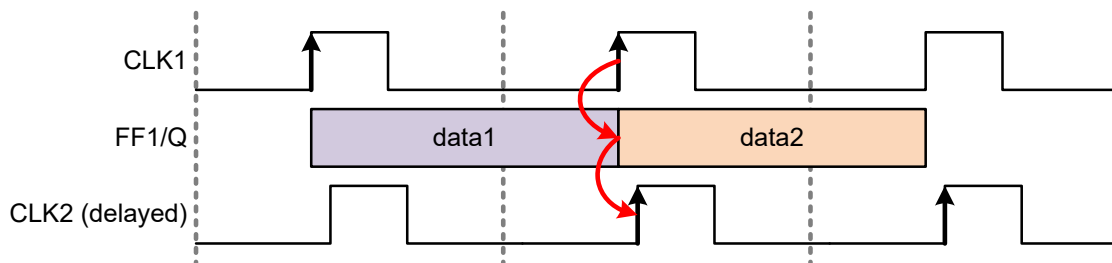
Consider the scan structure in [Figure 17](#), where a scan cell clocked by CLK1 feeds a scan cell clocked by CLK2, and both clocks are defined with the same ideal waveform definition.

Figure 17 Two Scan Cells Clocked by Two Different Clocks



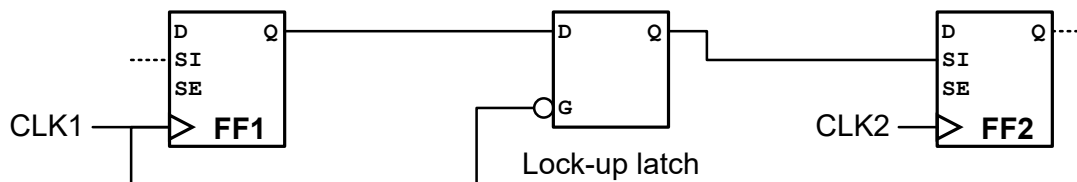
If both scan cells receive a clock edge at the same time, no timing violations occur. However, if the CLK2 waveform at FF2 is delayed, perhaps due to higher clock tree latency, a hold violation might result where FF2 incorrectly captures the current cycle's data instead of the previous cycle's data. [Figure 18](#) shows this hold violation for leading-edge scan cells.

Figure 18 Timing for Two Leading-Edge Scan Cells Clocked by Two Different Clocks



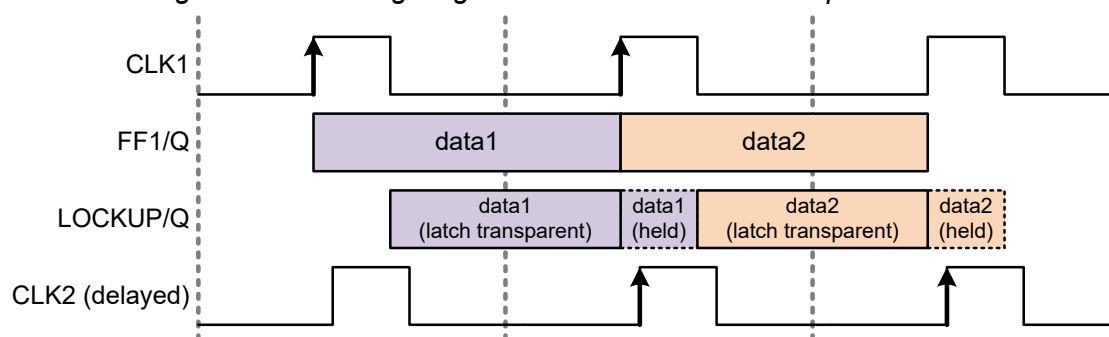
A lock-up latch prevents hold violations for scan cells that might capture data using a skewed clock edge. It is a latch cell that is inserted between two scan cells and clocked by the inversion of the previous scan cell's clock. [Figure 19](#) shows the same two scan cells with a lock-up latch added.

Figure 19 Two Scan Cells With a Lock-Up Latch



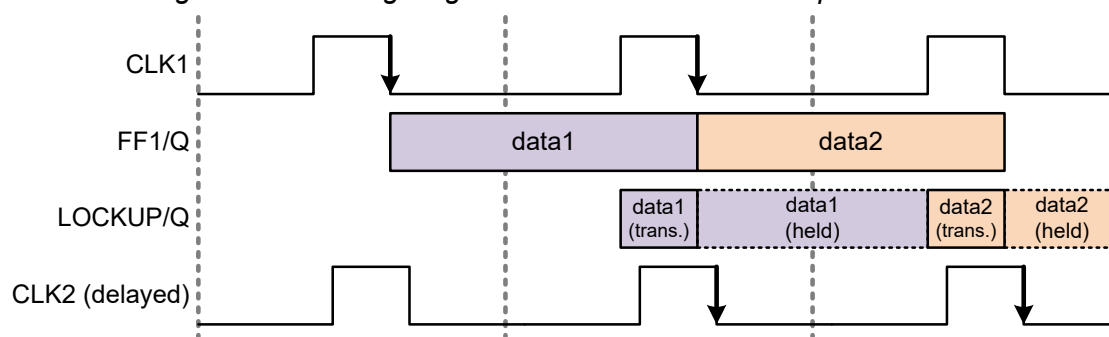
The lock-up latch cell works by holding the previous cycle's scan data while the current cycle's scan data is captured, effectively delaying the output data transition to the next edge of the source clock. Figure 20 shows the lock-up timing behavior for the example. Although this example uses return-to-zero clock waveforms, lock-up latch operation is similar for return-to-one clock waveforms.

Figure 20 Timing for Two Leading-Edge Scan Cells With a Lock-Up Latch



Lock-up latch operation for trailing-edge scan cells is similar to that of leading-edge scan cells, except that the data is held into the next clock cycle as shown in Figure 21.

Figure 21 Timing for Two Trailing-Edge Scan Cells With a Lock-Up Latch



By default, Fusion Compiler adds scan lock-up latches as needed to multiplexed flip-flop scan chains. Scan chain cells are ordered by the ideal test clock edge times, as defined with the `-timing` option of the `set_dft_signal` command. Cells clocked by later clock

edges are placed before cells clocked by earlier clock edges. Adjacent scan chain cells clocked by different clock edges are handled as follows:

- When the scan cells are clocked by clock edges with different ideal clock edge timing, Fusion Compiler does not insert a lock-up latch. The second scan cell captures data using an earlier clock edge, and the tool assumes this difference in the ideal clock edge timing is sufficient to avoid a hold time violation.
- When the scan cells are clocked by clock edges with identical ideal clock edge timing, Fusion Compiler inserts a lock-up latch to avoid a potential hold violation due to clock skew.

Fusion Compiler builds scan paths that meet zero-delay timing (without clock propagation delay or uncertainty). In Figure 20, if CLK2 is skewed later than CLK1 by more than the active-high pulse width of CLK1, a hold violation can still occur.

Figure 22 shows a set of ideal test clock waveforms for a set of overlapping test clocks. The clock edges are numbered by their edge timing order, with the latest clock edge indicated by (1). Clock edges with identical ideal clock edge timing are highlighted.

Figure 22 Ideal Test Clock Waveforms for Overlapping Test Clocks

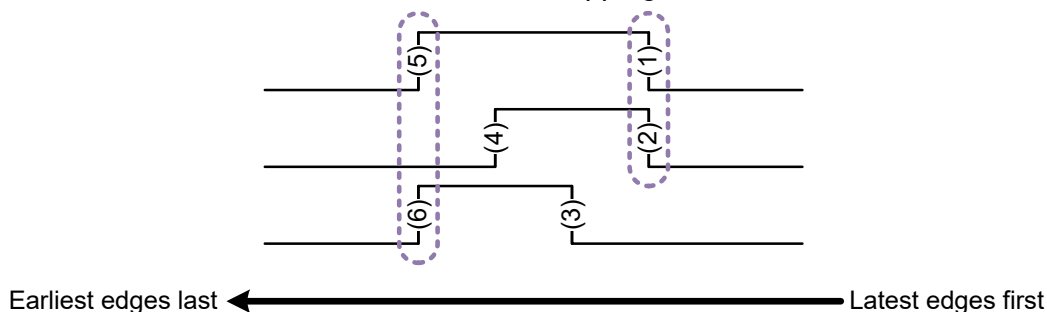
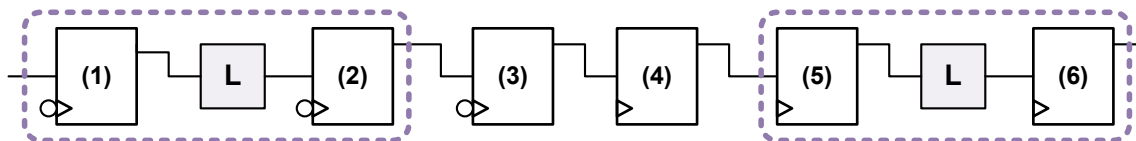


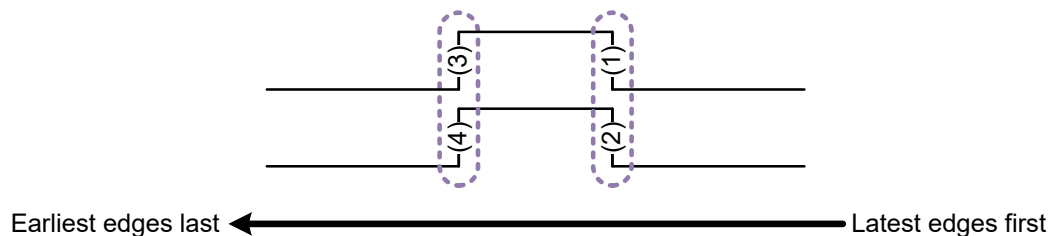
Figure 23 shows how Fusion Compiler constructs a scan chain containing a scan cell clocked by each clock edge. The scan cells are ordered with the cells clocked by the latest clock edges coming first. Lock-up latches are inserted between scan cells clocked by the clock edges with identical ideal clock edge timing.

Figure 23 Scan Chain Lock-Up Latches for Overlapping Test Clocks



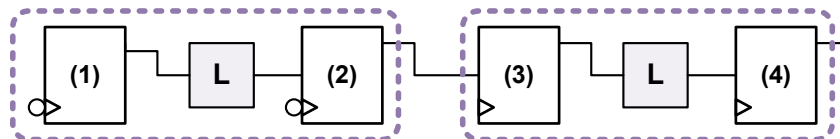
However, in most designs, all test clocks share identical return-to-zero test clock waveforms, as shown in Figure 24.

Figure 24 Ideal Test Clock Waveforms for Simple Test Clocks



In this case, the ordering behavior is simplified. The scan cells are ordered with all falling-edge scan cells first and all rising-edge scan cells last, as shown in Figure 25. Lock-up latches are inserted between differently-clocked scan cells within the rising-edge and falling-edge sections of the scan chain.

Figure 25 Scan Chain Lock-Up Latches for Simple Test Clocks



The `set_scan_configuration` command provides options to control lock-up latch insertion. To add lock-up latches at the end of each scan chain to assist with potential block-to-block timing issues during core integration, use the `-insert_terminal_lockup` option of the `set_scan_configuration` command:

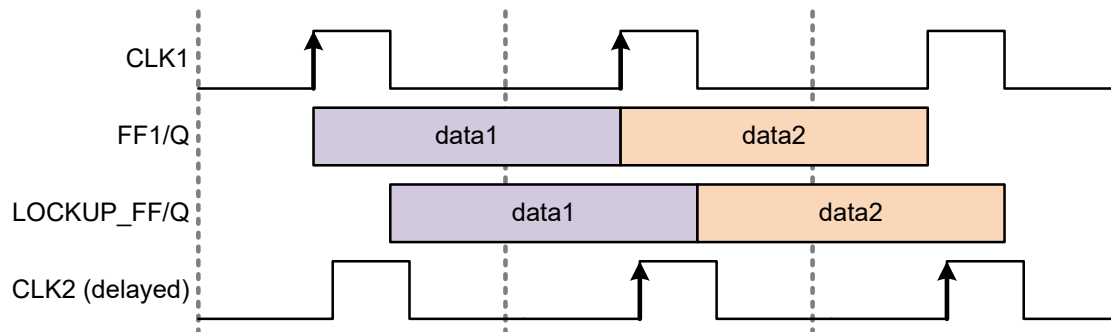
```
fc_shell> set_scan_configuration -insert_terminal_lockup true
```

The default lock-up element type is a level-sensitive lock-up latch. To use a lock-up flip-flop instead, use the `-lockup_type` option of the `set_scan_configuration` command:

```
fc_shell> set_scan_configuration -lockup_type flip_flop
```

When a lock-up flip-flop is used, the data is held as shown in Figure 26.

Figure 26 Timing for Two Scan Cells With a Lock-Up Flip-Flop



For successful lock-up operation, the falling edge of the current scan cell must occur after or concurrent with the rising edge of the next scan cell. This requirement is always inherently met when Fusion Compiler inserts lock-up elements between scan chain cells. However, when you are manually inserting lock-up elements with the `set_scan_path` command, you must ensure that this requirement is met.

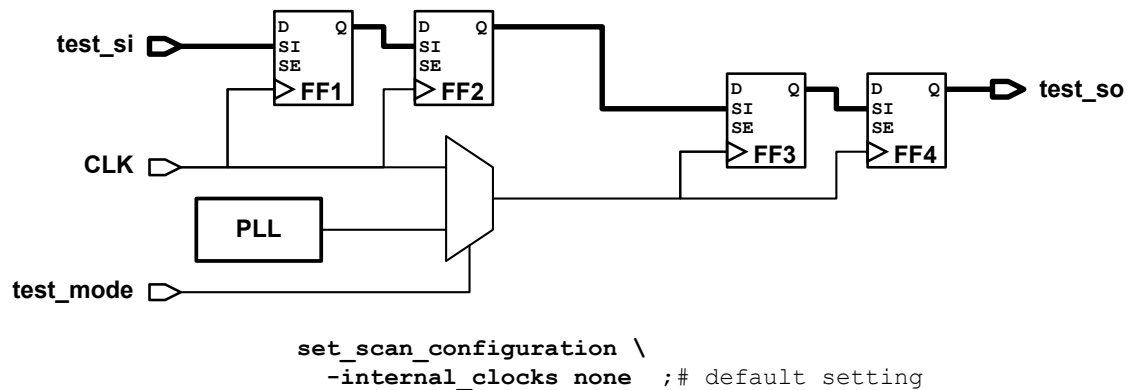
Automatically Creating Skew Subdomains Within Clock Domains

This topic describes how the tool can add lock-up latches within a clock domain between subdomains that might have higher skew between them.

For the purpose of building scan chains, DFT insertion, by default, treats the entire clock network driven by a given clock source as the same-skew clock signal.

Consider the netlist shown in [Figure 27](#), which shows a clock network structure before clock tree synthesis. By default, DFT insertion treats all four flip-flops as belonging to the same-skew top-level clock signal, CLK.

Figure 27 Circuit With Same Top-Level Clock Driving Internal Clock Signals



Note that the MUX cell introduces a delay in the clock network. If clock tree synthesis balances the test mode clock latency equally to all flip-flops, the MUX cell should not cause any timing problems. However, because clock tree synthesis might not consider the test mode clock tree latencies used for scan shift, a potential scan path hold violation could occur at FF3/SI.

To avoid creating this potential hold time violation, you can treat the scan cells downstream from any multi-input cell as a different *skew subdomain* within the clock domain, driven by their own internal clock pin (such as the MUX output pin).

To do this, use the following command:

```
fc_shell> set_scan_configuration -internal_clocks multi
```

This command instructs DFT insertion to

- Identify all multiple-input gates (such as MUX cells) in each clock network.

Note:

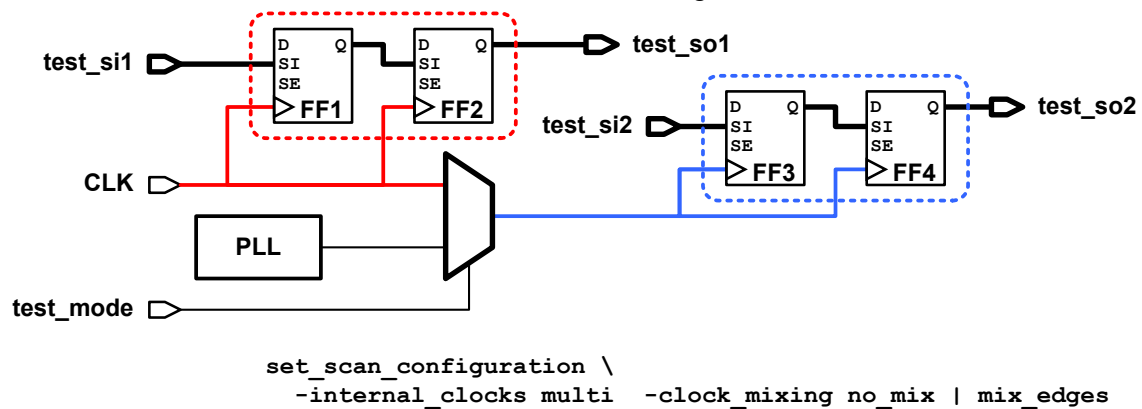
Integrated clock-gating cells, which have the `clock_gating_integrated_cell` attribute defined, are not considered; they are transparent for the determination of skew subdomains.

- Create an internal clock at the output pins of these gates.
- Treat these internal clocks as skew subdomains of the parent clock.

This feature affects only scan chain architecture; the clock network is still a single test clock domain for all other DFT operations, including writing out the test protocol.

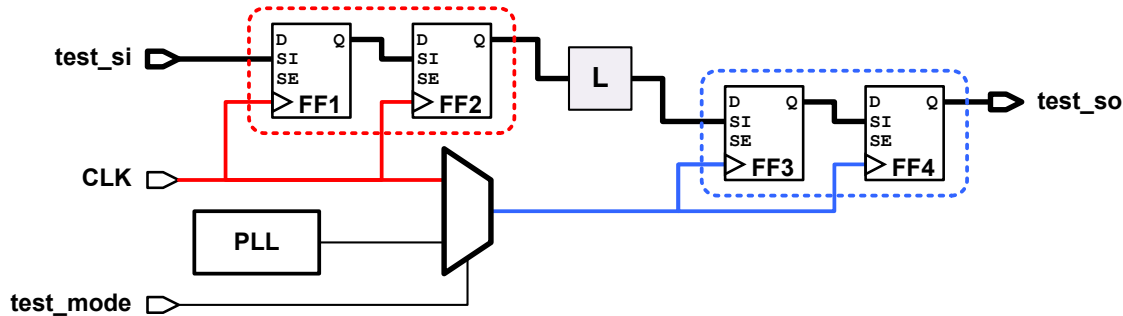
The resulting scan chains depend on the current clock-mixing setting, which is controlled by the `-clock_mixing` option of the `set_scan_configuration` command. If clock mixing is disabled by specifying the `no_mix` or `mix_edges` clock-mixing mode, DFT insertion creates separate scan chains for each internal clock, as shown in [Figure 28](#).

Figure 28 Circuit With Internal Clocks With Clock Mixing Disabled



If clock mixing is enabled by specifying the `mix_clocks` or `mix_clocks_not_edges` clock-mixing mode, DFT insertion can use lock-up latches to keep the scan cells from different internal clocks on the same scan chain, as shown in [Figure 29](#).

Figure 29 Circuit With Internal Clocks With Clock Mixing Enabled



```
set_scan_configuration \
  -internal_clocks multi -clock_mixing mix_clocks | mix_clocks_not_edges
```

You can create skew subdomains within specific clock domains by using the `-internal_clocks` option of the `set_dft_signal` command. The following command tells DFT insertion to create internal clocks at multiple-input cells only for the CLK domain:

```
fc_shell> set_dft_signal -view existing_dft \
  -type ScanClock -timing [list 45 55] \
  -internal_clocks multi -port CLK
```

If you set different `-internal_clocks` values using the `set_scan_configuration` and `set_dft_signal` commands, the more specific setting applied with the `set_dft_signal` command takes precedence. For example, assume that you set the following opposing `-internal_clocks` values by using these two commands:

```
fc_shell> set_scan_configuration -internal_clocks none

fc_shell> set_dft_signal -view existing_dft \
  -type ScanClock -timing [list 45 55] \
  -internal_clocks multi -port CLK
```

Because the value set by the `set_dft_signal` command takes precedence, signals driven by CLK via MUX cells or other multiple-input gates are treated as separate clocks. All other clocks in the design are treated according to the default configuration.

This feature is similar to the `-associated_internal_clocks` feature described in [Manually Creating Skew Subdomains at Associated Internal Pins](#), except that the internal clocks are created at all multi-input cell output pins instead of only user-specified pins.

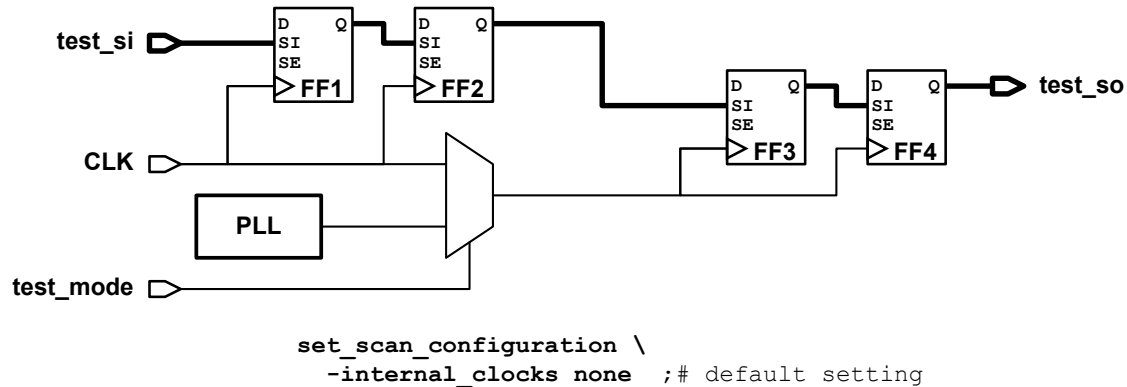
Manually Creating Skew Subdomains at Associated Internal Pins

This topic describes how you can manually define subdomains of a parent clock network that might have higher skew between them.

For the purpose of building scan chains, DFT insertion, by default, treats the entire clock network driven by a given clock source as the same-skew clock signal.

Consider the netlist shown in [Figure 30](#), which shows a clock network structure before clock tree synthesis. By default, DFT insertion treats all four flip-flops as belonging to the same-skew top-level clock signal, CLK.

Figure 30 Circuit With Same Top-Level Clock Driving Internal Clock Signals



Note that the MUX cell introduces a delay in the clock network. If clock tree synthesis balances the test mode clock latency equally to all flip-flops, the MUX cell should not cause any timing problems. However, because clock tree synthesis might not consider the test mode clock tree latencies used for scan shift, a potential scan path hold violation could occur at FF3/SI.

To avoid creating this potential hold time violation, you can treat the scan cells downstream from the MUX cell as a different *skew subdomain* within the clock domain, driven by their own internal clock pin (the MUX output pin).

To do this, define the scan clock as follows:

```
fc_shell> set_dft_signal -view existing_dft -type ScanClock \
    -timing {45 55} -associated_internal_clocks {UMUX/Z}
```

This command instructs DFT insertion to

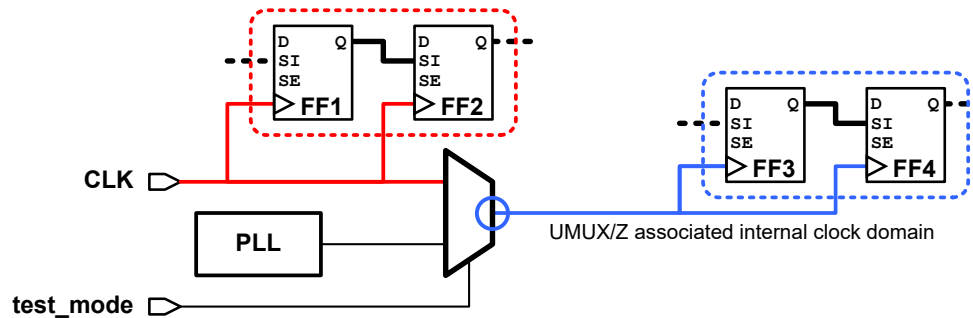
- Create an internal clock at each associated internal pin in the list
- Treat these internal clocks as skew subdomains of the parent clock

This feature affects only scan chain architecture; the clock network is still a single test clock domain for all other DFT operations, including writing out the test protocol.

In the previous example, the clock network contains a skew subdomain driven by UMUX/Z, plus the remainder of the parent clock domain, as shown in [Figure 31](#). These

are treated as separate clock domains according to the `set_scan_configuration -clock_mixing` setting applied to the design.

Figure 31 Circuit With Top-Level Clock Source and Associated Internal Clock Pin



```
set_dft_signal -view existing_dft -type ScanClock -timing {45 55} \
-port CLK -associated_internal_clocks {UMUX/Z}
```

This feature is similar to the `-internal_clocks` feature described in [Automatically Creating Skew Subdomains Within Clock Domains](#), except that the internal clocks are created only at the user-specified pins instead of all multi-input cell output pins.

Associated internal clocks take precedence over any `-internal_clocks` specifications.

Limitations

Note the following requirements and limitations:

- Associated internal clocks can be defined only on leaf pins, not hierarchical pins.
- Pre-DFT DRC drives the clock signal directly at both the clock source and the internal pins, allowing the clock signal to bypass cells that are black boxes in synthesis. However, post-DFT DRC drives the clock signal only at the clock source.

To propagate the clock through blockages during post-DFT DRC, use a custom `test_setup` procedure .

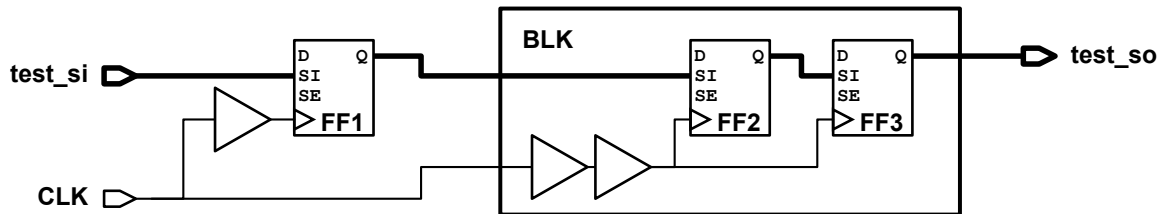
- This feature is used only with `-view existing_dft` clock signal definitions.
- The association is valid only when `-type` is `MasterClock`, `ScanMasterClock`, `ScanSlaveClock`, or `ScanClock`.
- The `report_dft_signal` command shows the associated internal pins as separate master clock.

Manually Creating Skew Subdomains With Scan Skew Groups

This topic describes how to define a skew subdomain on any arbitrary set of scan cells. Such a set of scan cells is called a *scan skew group*.

In some cases, you might want to provide manual guidance for lock-up latch insertion between areas of the design with potentially differing clock tree latencies. Consider [Figure 32](#), in which block BLK has a different clock latency than the top-level logic.

Figure 32 Circuit With Clock Tree Containing Multiple Latency Regions



A lock-up latch at the scan input pin of BLK would prevent hold violations along the scan path. However, the scan chain is entirely within the same scan clock domain, and there are no multi-input cells that allow the `-internal_clocks` option to be used.

You can use *scan skew groups* to provide manual guidance for lock-up latch insertion. A scan skew group is a group of scan cells that might have a different clock latency characteristic than other parts of the design. The DFT architect treats the scan skew group as a unique *skew subdomain*.

To define a scan skew group, use the `set_scan_skew_group` command:

```
set_scan_skew_group
  group_name
  -include_elements {include_list}
```

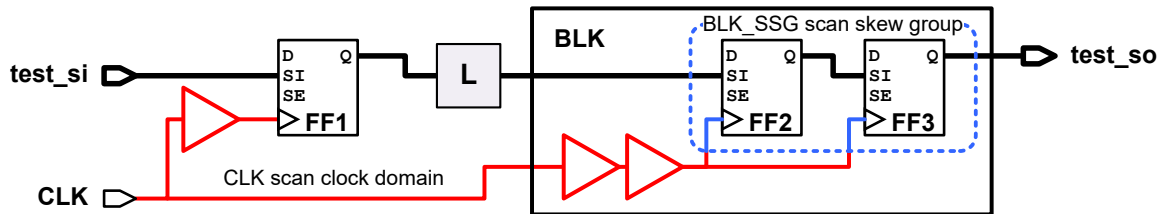
Each scan skew group has a unique name for identification. The include list can contain leaf cells, hierarchical cells, and CTL-modeled cells. Wildcards and collections are supported. You can define as many scan skew groups in your design as needed. You cannot include the same scan cell in multiple scan skew groups.

For the previous example, consider the following scan skew group definition:

```
fc_shell> set_scan_skew_group BLK_SSG -include_elements {BLK/FF*}
```

If clock mixing is enabled, DFT insertion adds a lock-up latch between the top-level and block-level scan cells, as shown in [Figure 33](#). (If clock mixing is disabled, DFT insertion keeps the top-level and block-level scan cells in separate scan chains, not shown.)

Figure 33 Circuit With Lock-Up Latch Due to Scan Skew Group Definition

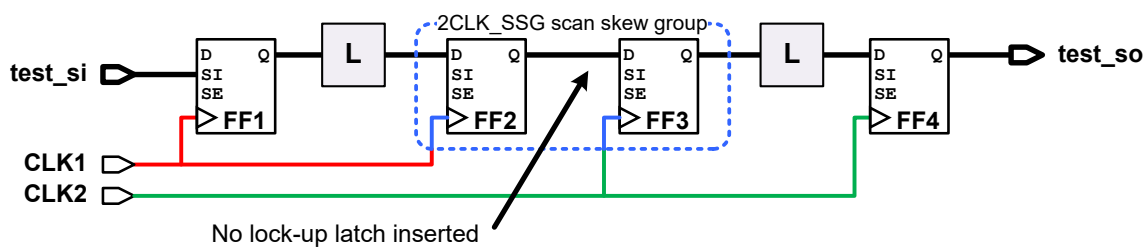


Scan skew groups override the normal scan clock domain identification behaviors such as

- Scan clock name
- The `-internal_clocks` option of the `set_scan_configuration` command

The DFT architect treats all cells in the scan skew group as if they are in the same clock domain. No lock-up latches are inserted between them, as shown in Figure 34. This can occur even if clock mixing is disabled because the scan cells in the scan skew group are no longer treated as belonging to different clock domains.

Figure 34 Circuit With Scan Skew Group Spanning Multiple Clock Domains



Scan skew groups override only clock identity considerations. They do not override the clock timing considerations of scan chain architecture such as

- Scan clock waveform timing
- Scan clock edge polarity

Enhanced Multimode Scan Architecting

In the default scan insertion flow, Fusion Compiler builds each test mode independently of each other that enables the following:

- You can control clock/voltage/power domains mixing for test modes with longer chains.
- You can control scan cell exclusion per test mode.
- Balance the scan chain in all the test modes.

Note the following tips for using enhanced multimode flow:

- Define the test modes before `set_scan_configuration`, `set_scan_compression_configuration`, `ScanDataIn`, and `ScanDataOut` specifications.
- The `scan_configuration`, `scan_compression_configuration` specifications must be specified with the `-test_mode {test_model test_mode2 ...}` option to indicate the test mode they are applied.
- Similarly, `ScanDataIn` and `ScanDataOut` specifications need to be specified with the `-test_mode {test_model test_mode2 ...}` option to indicate which test mode is applied.
- `ScanClocks`, resets, and constant signals do not need the `-test_mode` option specification.
- Also, `set_boundary_cell` configuration does not need a `-test_mode` specification.

Example 4 Script Example for Enhanced Multi Mode Scan Architecting

```
<Library creation steps>
<Analyze RTL and elaboration>
...
compile_fusion -to logic_opto
#Enable internal pins flow to use hookup pins when necessary
set_dft_drc_configuration -internal_pins enable
# Clocks and Asynchronous Resets
set_dft_signal -type ScanClock -port clk -timing {45 55}
set_dft_signal -type Reset -port reset -active_state 0
set_dft_signal -type Constant -port const -active_state 0
#Scan enable and clock gating scan enable
set_dft_signal -type ScanEnable -hookup_pin scan_interface/SE
set_dft_signal -type ScanEnable -hookup_pin scan_interface/CSE \
    -usage clock_gating
# Test mode signal definition
set_dft_signal -type TestMode -hookup_pin scan_interface/tm1
set_dft_signal -type TestMode -hookup_pin scan_interface/tm2
#Define the test modes
define_test_mode scomp_mode -usage scan_compression
define_test_mode uncompress -usage scan
define_test_mode scandump -usage scan
reset_test_mode

set_scan_compression_configuration \
    -inputs 16 -outputs 16 \
    -max_length 400 \
    -test_mode scomp_mode
set_scan_configuration -clock_mixing mix_clocks \
    -add_test_retiming_flops begin_and_end \
    -insert_terminal_lockup true \
    -test_mode Scomp_mode
```

```
# Scan Configuration
set_scan_configuration -exclude_elements [get_cell -hier \
    -filter "<filter_criteria>"]
set_scan_configuration -chain_count 16 -test_mode uncompress \
    -clock_mixing no_mix
set_scan_configuration -chain_count 1 -test_mode scandump \
    -clock_mixing mix_clocks
set_scan_configuration -clock_mixing no_mix -test_mode Scomp_mode
set_scan_configuration -add_test_retiming_flops begin_and_end \
    -insert_terminal_lockup true \
        -test_mode {Scomp_mode uncompress scandump}
# Scan in, scan out
for {set i 0} {$i < 16} {incr i} {
    set_dft_signal -view spec -type ScanDataIn -hookup_pin \
        scan_interface/si_[$i] -test_mode {scomp_mode uncompress}
    set_dft_signal -view spec -type ScanDataOut -hookup_pin \
        scan_interface/so_[$i] -test_mode {scomp_mode uncompress}
}
set_dft_signal -view spec -type ScanDataIn -port SI -test_mode scandump
set_dft_signal -view spec -type ScanDataOut -port SO -test_mode scandump
create_test_protocol
#pre-dft_drc
dft_drc -test_mode all_dft
preview_dft
insert_dft
# Perform post dft_drc
dft_drc -test_mode scomp_mode
dft_drc -test_mode uncompress
dft_drc -test_mode scandump
#scan reordering happens in the following compile steps
compile_fusion -from initial_place -to initial_opto
# Perform post dft_drc again
dft_drc -test_mode scomp_mode
dft_drc -test_mode uncompress
dft_drc -test_mode scandump
# DFT Reports
report_dft_drc_violations -test_mode scomp_mode -rule all
report_scan_path > scan_path.rpt

# Write out scandef and Test model
write_scan_def -o <design>.scan.def
write_test_model -output <design>.ctl

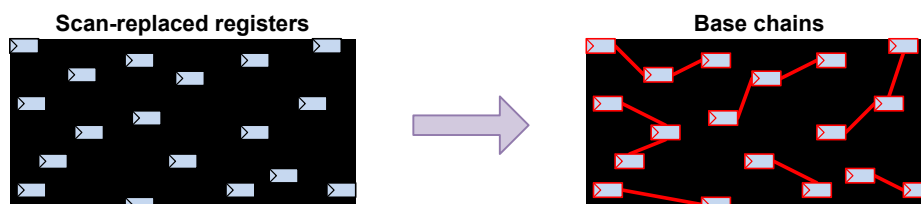
# Save test protocols for TestMAX ATPG
write_test_protocol -output scomp_mode.spf \
    -test_mode Scomp_mode
write_test_protocol -output uncompress.spf \
    -test_mode uncompress
write_test_protocol -output scandump.spf \
    -test_mode scandump
write_verilog -hierarchy all <design>.v
```

Building Scan Chains From Base Chains

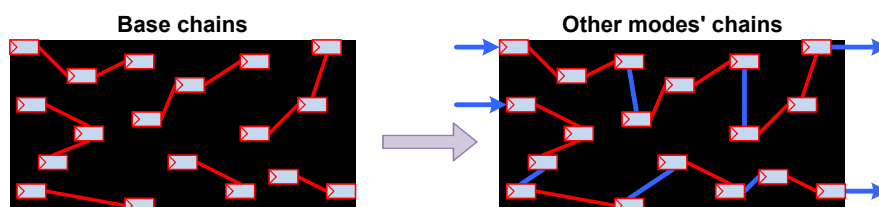
Fusion Compiler allows building the test mode with the shortest chains and then concatenating them to build longer chains using the following application option:

```
fc_shell > set_app_option -name dft.scan.enable_general_multimode_support  
-value false
```

To build scan chains for one or more test modes, Fusion Compiler first builds a set of physically compact *base chains* in the test mode that has the shortest chains (for example scan compression mode):



These base chains are then assembled to form the scan chains for other test modes:



Base chains are the building blocks for architecting scan chains. They can be concatenated together, but they cannot be divided.

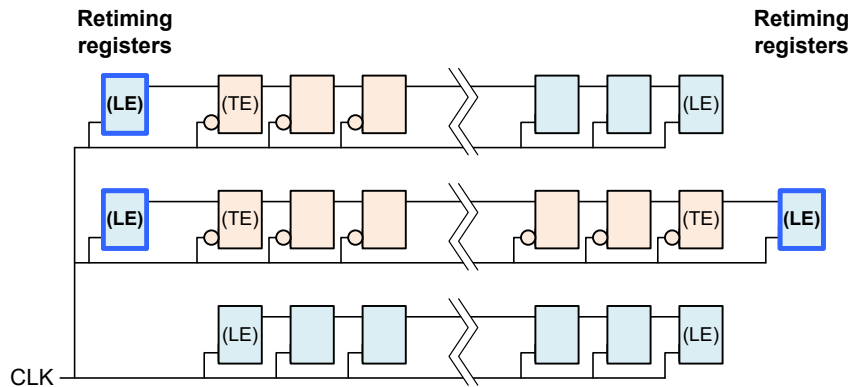
The following are the limitations for this flow:

- You cannot control clock/voltage/power domain mixing for test modes with longer chains
- You cannot control scan cell exclusion per test mode
- Balancing the scan chain on the longer chains is not possible

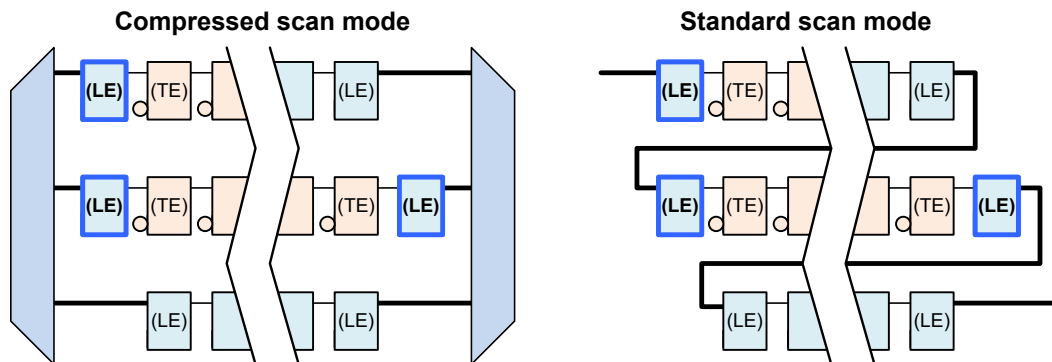
You can overcome these limitations by using the default [Enhanced Multimode Scan Architecting](#) flow.

Retiming Registers

In designs with scan compression, Fusion Compiler automatically inserts leading-edge *retiming registers* at any trailing-edge base chain beginnings and ends to improve quality of results.

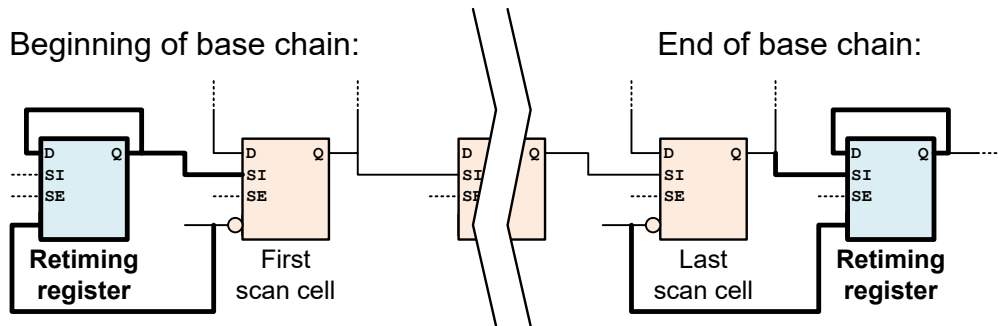


By retiming both ends of the base chains to the leading edge, the tool can concatenate them without being restricted by scan cell clocking edges.



Note that retiming registers are the only case where the tool allows a leading-edge register (blue) to be followed by a trailing-edge register (red).

Retiming registers are implemented using scan registers with a functional loopback path, as follows. This prevents timing violations when the clock runs at functional frequencies. The clock of the associated scan cell is used.



Retiming registers are not enabled by default for standard-scan-only designs. However, you can manually enable it by using the `-add_test_retiming_flops` option of the `set_scan_configuration` command.

Multivoltage Support

In multivoltage designs, scan chains that include cells from different power domains or voltage areas require special cells to avoid power-related problems. By default, the tool does not place cells from different domains in the same scan chain. However, some power domains or voltage areas have a small number of cells, which results in short scan chains.

You can allow power domain and voltage area mixing for scan chains by using the `set_scan_configuration` command, as follows:

- Use the `-voltage_mixing true` option to build scan chains that can cross voltage areas (the default is `false`). The tool attempts to minimize voltage crossings to reduce the number of level shifters added.
- Use the `-power_domain_mixing true` option to build scan chains that can cross power domains (the default is `false`). The tool attempts to minimize power domain crossings to reduce the number of isolation cells added.
- You can use the `-voltage_mixing true` and `-power_domain_mixing true` options together.

You must also use the `-test_mode all` option to apply power domain or voltage area mixing.

If you use any commands that violate the power domain or voltage area mixing specifications, such as the `set_scan_path` command, the tool issues a warning message at the `preview_dft` or `insert_dft` commands and continues with scan insertion.

You can specify power domain and voltage area mixing only for post-compile DFT flows. After you execute the `insert_dft` command, use the `check_mv_design` command to check for voltage violations introduced by the `insert_dft` and `create_mv_cells` commands. Add missing multivoltage cells as needed.

Creating Separate Retention Scan Chains

Scan synthesis creates separate chains consisting only retention cells. This helps retention testing strategies that requires retention and non-retention cells to be built into separate scan chains.

To create chains that consists only retention cells, set the `set_scan_configuration -mix_retention_cells` option to `false`. The default is `true`. Scan synthesis builds chains that mix retention and non-retention cells in the same chain.

This option applies to both scan and wrapper chains. To report which scan chains containing only the retention cells, you can use the `preview_dft` and `scan_dft.rpt` reports. For example,

```
Scan chain 'chain_1' (DFT_SI1 --> DFT_SO1) contains 276 cells (Partition
'default_partition'):
...
Scan chain 'chain_8' (DFT_SI8 --> DFT_SO8) contains 133 retention cells
(Partition 'default_partition')
```

Note:

This option applies to all test modes and DFT partitions.

When the retention mixing is disabled, if the `set_scan_path` specification has a mix of retention and non-retention cells, `preview_dft` and `insert_dft` commands generate the following error message:

```
Error: Can not honor set_scan_path specification for scan
path scan_chain1 since it includes a mix of retention and non-retention
registers, and retention cell mixing is turned off by the user.
(DFT-2126)
```

Bypassing Scan Segments

The `set_scan_group` command with `-class bypass` option defines the set of elements that the user wants to bypass with the optional values of the clock for the head and for the tail of the bypass. This command also enables the configuration of the bypass clocks. If

a design instance is specified, all sequential cells and segments within it are treated as a group and are logically ordered.

The syntax for this command is as follows:

```
set_scan_group scan_group_name
    -class bypass [-bypass_head_clock CLK1] [-bypass_tail_clock CLK2]
    -include_elements {list_of_cells_or_segments}
    [-access {list_of_access_pins}]
    [-serial_routed true|false]
```

scan_group_name

Specifies a unique group name.

```
-class bypass [-bypass_head_clock CLK1] [-bypass_tail_clock CLK2]
```

Specify the set of elements you want to bypass with the optional values of the clock for the head and for the tail of the bypass.

You can set a global `bypass_se` signal that can be used for all the segments in the mode where it is defined. It can be either a port or a `hookup_pin`. For example,

```
- set_dft_signal -type bypass_se -port bypass_se
- set_dft_signal -type bypass_se -hookup_pin internal_pin
```

```
-include_elements {list_of_cells_or_segments}
```

Specifies a list of cell names or segment names that are included in the group.

```
-access {list_of_access_pins}
```

Specifies a list of all access pins. Note that these access pins represent only a serially routed scan group specification. If the `-serial_routed` option is set to `false`, all specified access pins are ignored.

You can specify a specific `bypass_se` signal for this particular bypass segment that takes precedence over the global `bypass_se` signal if specified in the `set_dft_signal` command.

Specifies the `bypass_se` signal by using `-access {BypassSe bypass_se_port}`. Here, the `-serial_routed true` option is mandatory.

```
-serial_routed [true | false]
```

Specifies whether the scan group is serially routed (`true`) or not (`false`). The default is `false`.

Some specific scenarios are mentioned in the following table:

Scenario	Head Flop	Head Lockup	Tail Lockup	Tail Flop	Scan chain lockups
Bypass a CTL core segment clocked by a positive edge clock. Head and tail clocks specified.	Yes (pos-edge on head bypass clk)	Latch	Latch	Yes (pos-edge on tail bypass clk)	Insert if required (clock domain crossing)
Bypass a CTL core segment clocked by a positive edge clock. Segment ends with a terminal lock-up latch. Head and tail clocks specified	Yes (pos-edge on head bypass clk)	Latch	Not required	Yes (pos-edge on tail bypass clk)	Insert if required (clock domain crossing)
Bypass a CTL core segment clocked by a negative edge clock. Head and tail clocks specified	Yes (pos-edge on head bypass clk)	Not required	Not required	Yes (pos-edge on tail bypass clk)	Insert if required (clock domain crossing)
Bypass a CTL core segment clocked by a negative edge clock. Segment ends with a terminal lock-up latch. Head and tail clocks specified	Yes (pos-edge on head bypass clk)	Not required	Neg-edge flip-flop with feedback loop	Yes (pos-edge on tail bypass clk)	Insert if required (clock domain crossing)
Bypass a CTL core segment clocked by one clock; first elements are negative edge clocked and last elements are positive edge clocked. Head and tail clocks specified	Yes (pos-edge on head bypass clk)	Not required	Latch	Yes (pos-edge on tail bypass clk)	Insert if required (clock domain crossing)
Bypass a CTL core segment clocked by one clock; first elements are negative edge clocked and last elements are positive edge clocked. Segment ends with a terminal lock-up latch. Head and tail clocks specified	Yes (pos-edge on head bypass clk)	Not required	Not required	Yes (pos-edge on tail bypass clk)	Insert if required (clock domain crossing)

You can use the `preview_dft` command to report the bypass segment and head/tail bypass elements along with the required lockup latch markers. For example,

```
# Preview_dft:
Scan chain '8' (SI[7] --> SO[7]) contains 49 cells
```

```
grey_flop (l) (grep_flop_clock, 45.0, rising)
Head_Bypass (l) (head_bypass_clock, 45.0, rising)
Segment (s) (l) <macro_CLK, 45.0 rising>
Tail_Bypass (l) (tail_bypass_clk, 45, rising)
grey_flop_2 (grep_flop_2_clock, 45.0, rising)
```

Note the following:

- There is no `-view` option to the `set_scan_group` command, because the options only works in the specification view.
- All scan group specifications are applied across all test modes. You cannot specify a scan group to be applied on a particular test mode.
- An element specified as part of a scan group cannot be specified as part of a scan path, and vice versa.
- Scan group specifications are not cumulative. If you specify the same group name, the last scan group specification before the `insert_dft` command is used.
- Grouping elements implies that they must be adjacent in a scan chain.

Bypassing Soft-Macro Scan Segments

Bypassing soft-macro scan segments enhances the bypassing scan segments feature by allowing specification of multiple `set_scan_group` with automatic distribution of elements into specific groups.

You can use the `set_soft_macro_bypass_segments` command to create bypass segments from a list of elements with a specific maximum length parameter for each of these segments.

The elements supported are regular cells, but TCL collections, wildcards, and hierarchical cells. Created bypass segments consists of cells with the same domain properties in terms of the clock, edge, power domains, and so on.

The cells are assigned to bypass segments according to a specified list of registers. Thus, if there is a change of domain in the list of registers, shorter segments are created. The order of registers is kept according to the specified list.

To select the clocks for the segments, you can use the `bypass_head_clock`, `bypass_tail_clock` options, and the scan enable signal `bypass_scan_enable` similar to the `set_scan_group -class bypass` command. For example,

```
set_soft_macro_bypass_segments  name \  
    -include_elements {<elements_list>} \  
    -max_length <L> \  
    -bypass_head_clock <bypass_clk1> \  
    -bypass_tail_clock <bypass_clk2> \  
    -bypass_scan_enable <bypass_se>
```

The created scan groups get names with specified name parameters and attached suffixes like NAME_0, NAME_1 and so on. Reporting is based on these automatically assigned group names.

You can use the `report_soft_macro_bypass_segments` command to report configuration done by the `set_soft_macro_bypass_segments` command.

Bypassing SE/SI Controlled Hard-Macro Scan Segments

Bypassing SE/SI controlled hard-macro scan segments enhances the bypassing scan segments feature by deciding how SE and SI pins are controlled during the bypassed mode of hard macro segments.

You can use the new `se_pin_logic` and `si_pin_logic` options of the `set_scan_group` command to control the extra logic introduced for the scan enable and scan input pins of the elements inside a bypass segment. Each of these options inserts extra logic to the SE/SI signals connected to the elements of the bypassed group, when the `insert_dft` command is run.

The `se_pin_logic` option controls the extra logic (AND/OR gate) that is inserted at the scan enable pin, for all the elements of the bypass segment.

The `si_pin_logic` option controls the extra logic (AND/OR gate) that is inserted at the scan input pin, only for the first element of the bypass segment.

The options have three possible values: default (not used), zero, and one. For example,

```
set_scan_group  name -class bypass \
    -se_pin_logic <none | zero | one> \
    -si_pin_logic <none | zero | one>
```

The extra logic inserted for each option is summarized in the following table:

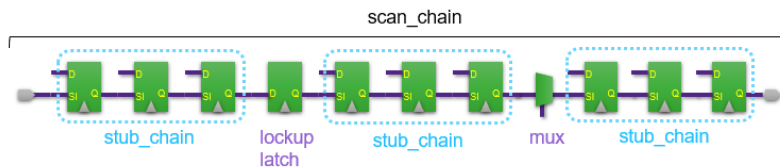
Option	Default	Zero value	One value
<code>se_pin_logic</code>	None	SE input of flops connected to AND gate output corresponding to <code>bypass_se</code> & Scan Enable	SE input of flops connected to OR gate output corresponding to <code>bypass_se</code> & Scan Enable
<code>si_pin_logic</code>	None	SI input of first flip-flop connected to AND gate output corresponding to <code>~bypass_se</code> & Scan In	SI input of first flip-flop connected to OR gate output corresponding to <code>bypass_se</code> & Scan In

To report the options, use the `report_scan_group` command and `preview_dft` report.

Scan and Stub Chains

SCANDEF files specify scan elements and their pins used for the scan path along with reordering constraints. A chain obtained from a SCANDEF file is called a *stub chain*. The term *scan chain* refers to the complete logical structure added during the DFT insertion step. A scan chain can contain stub chains, as shown in Figure 35. However, a scan chain might not contain any stub chains. A stub chain can belong to multiple scan chains.

Figure 35 Scan Chain Containing Stub Chains



Use the following commands to examine or report scan chains and stub chains:

- The `get_scan_chains` command creates a collection of scan chains.
- The `report_scan_chains` command displays information about the specified scan chains
- The `get_stub_chains` command creates a collection of stub chains.
- The `report_stub_chains` command displays information about the specified stub chains

The tool classifies scan chains and stub chains as `scan_chain` and `stub_chain` design objects, respectively. You can examine the attributes of these design objects by using the `get_attribute`, `list_attributes`, and `report_attributes` commands. To see a list of attributes, use the `man scan_chain_attributes` or `man stub_chain_attributes` commands.

You can work directly with scan and stub chains by using the following commands:

- The `create_scan_chain` command creates one scan chain from a collection of stub chains.
- The `remove_scan_chain` command removes scan chains.
- The `add_to_scan_chain` command adds stub chains to a scan chain.
- The `remove_from_scan_chain` command removes stub chains from a scan chain.
- The `create_stub_chain` command creates one stub chain from individual elements.

- The `remove_stub_chain` command removes stub chains.
- The `set_stub_chain` command modifies an existing stub chain.

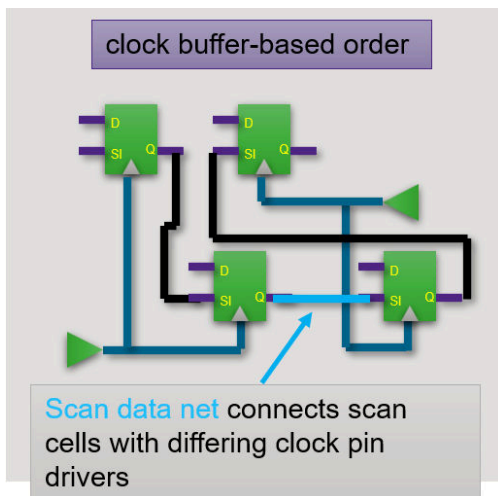
The `-include` and `-exclude` options of the `write_script` command accept an argument value of `scan_chain` to control scan chain output to the `design.tcl` file in the output directory.

The `-of_objects` options of the `get_ports`, `get_pins`, and `get_cells` commands accept scan chains and stub chains.

Optimizing Scan Path Hold Time

During the `final_opto` stage of `clock_opt`, the `optimize_dft` command performs repartitioning and reordering to minimize the clock buffer crossings, by default.

For more information, refer to [Controlling DFT Optimization for the place_opt Command](#).



The optimizing scan path hold time feature introduces a hold time aware algorithm that minimizes the clock skew and improves scan path hold timing by considering both clock and data path delays. This overcomes the default heuristic-based optimization that relies solely on netlist connectivity.

A timer scenario for hold time of scan paths must exist and the scenario with the highest scan path hold Total Negative Slack (TNS) is used for the optimization. The tool attempts to optimize the scan path hold time and the wirelength without degrading any other setup or hold time.

Note:

- Scan wirelength will increase over that of the pre-CTS optimization
- Scan wirelength is the secondary optimization goal after hold time

To enable this feature, use the `set_app_options -name opt.dft.use_ng_engines -value true` and `set_app_options -name opt.dft.ng_hold_aware -value true` application options.

To optimize the wirelength in the stages before CTS, you can leave the `opt.dft.ng_hold_aware` at its default, which is `false`.

Optimizing Scan Chain Stitching Direction

The scan chain stitching direction preference feature allows you to guide the `optimize_dft` command to favor horizontal or vertical routing directions when connecting scan cells.

This feature is used in designs with non-uniform routing resources, such as "tall and thin" or "short and wide" floorplans, where congestion is more severe in one direction. By biasing scan path connections to use the more available routing tracks, you can reduce congestion and improve overall design routability.

Designs with congestion primarily in one direction can benefit by setting the direction preference to the opposite value.

To enable scan chain stitching direction preference, use the `set_optimize_dft_options -direction_preference < none | horizontal | vertical >` command option. The default is `none` indicating no stitching direction preference.

To specify the stitching direction, you must enable the next generation DFT optimization engines with the application option `set_app_options -name opt.dft.use_ng_engines -value true`.

Both the `total_interconnect_length` and `max_interconnect_length` values of the `set_optimize_dft_options -goal < total_interconnect_length | max_interconnect_length >` command option are supported.

The `report_optimize_dft_options` command displays the current setting for the direction preference.

To view horizontal and vertical wirelength statistics before and after optimization, use the `set_app_options -name opt.dft.optimize_dft_detailed_report -value true` application option. An example output is as follows:

```
DFT: pre-opt wirelength: total: 378275 horizontal: 205653 vertical: 172622
```

```
DFT: post-opt wirelength: total: 229568 horizontal: 44459 vertical:
185108
DFT: post-opt wirelength difference: total: -148707 (ratio: -39.311902 %)
DFT: post-opt wirelength difference: horizontal: -161193 (ratio:
-78.381256 %)
DFT: post-opt wirelength difference: vertical: 12486 (ratio: 7.233298 %)
```

Limitations

Note the following limitations:

- The stitching direction preference only applies to wirelength-based scan chain optimization. Timing or clock buffer-based optimizations do not honor this setting.
 - Setting a direction preference may increase total scan wirelength compared to the default `none` setting.
-

Reducing Shift Power Using Functional Output Pin Gating

During scan testing, as scan data shifts through scan chains, the functional logic driven by the scan flip-flops also toggles. This can cause increased power dissipation during testing, which could damage the design under test.

You can use the `set_scan_suppress_toggling` command to enable functional output gating. When this command is used, the tool inserts gating logic to suppress toggling on the functional output of scan flip-flops that either you specify or the tool automatically selects. The tool uses either AND-gating or OR-gating logic, depending on which constant value most reduces toggling from other ungated signals entering the fanout logic cone.

Note:

This feature only works with designs that use the multiplexed flip-flop scan style.

[Figure 36](#) shows the design after DFT insertion, but without any gating logic inserted on the functional output.

Figure 36 Design After DFT Insertion Without Gating Logic on the Functional Output

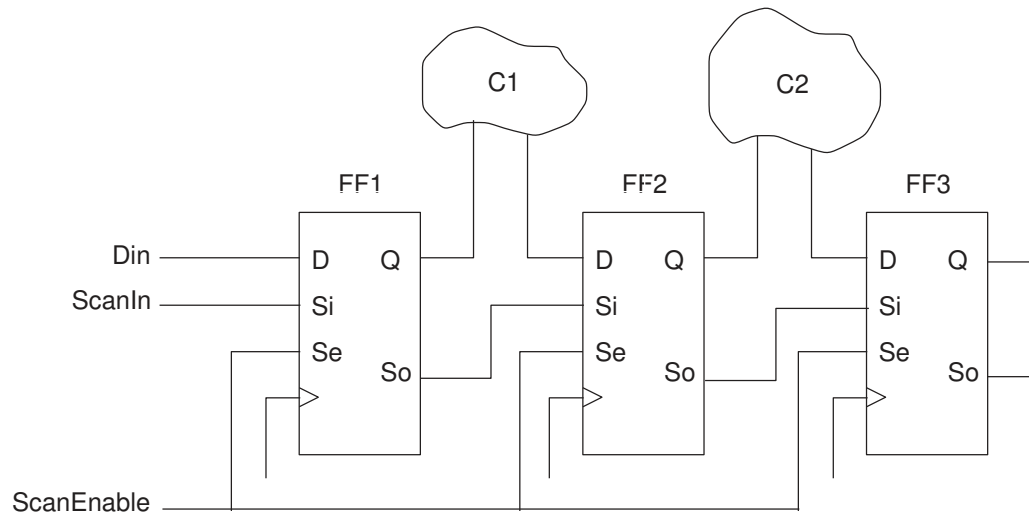
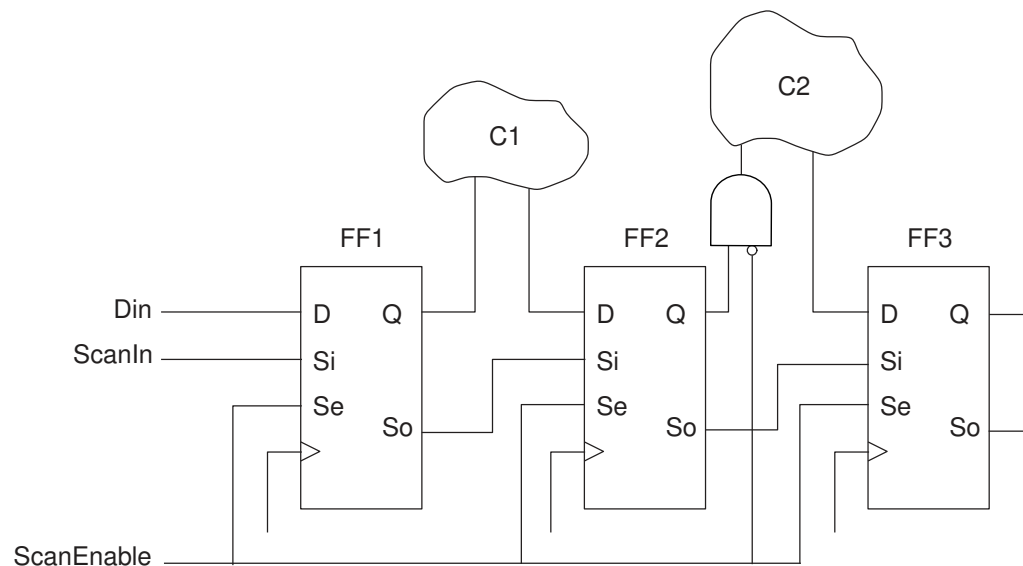


Figure 37 shows the design after the tool has inserted AND-gating logic on the output pin of the FF2 flip-flop.

Figure 37 Design After DFT Insertion With AND Clock Gating Inserted on the FF2 Functional Output



In the [Figure 37](#) example, the logic cone C2 does not toggle during scan shifting because the Q output of FF2 is gated by the extra AND gate, which is disabled by the scan-enable signal.

If your scan flip-flops have only a single output pin that is shared between functional and scan output, the logic path going from the flip-flop output pin to the functional logic is gated. [Figure 38](#) and [Figure 39](#) show this case for both AND-gating logic and OR-gating logic, respectively.

Figure 38 Design With AND-Gating Logic Inserted on the Functional Output of FF2, Where FF2 Has a Single Output Pin Shared Between Functional and Scan Output

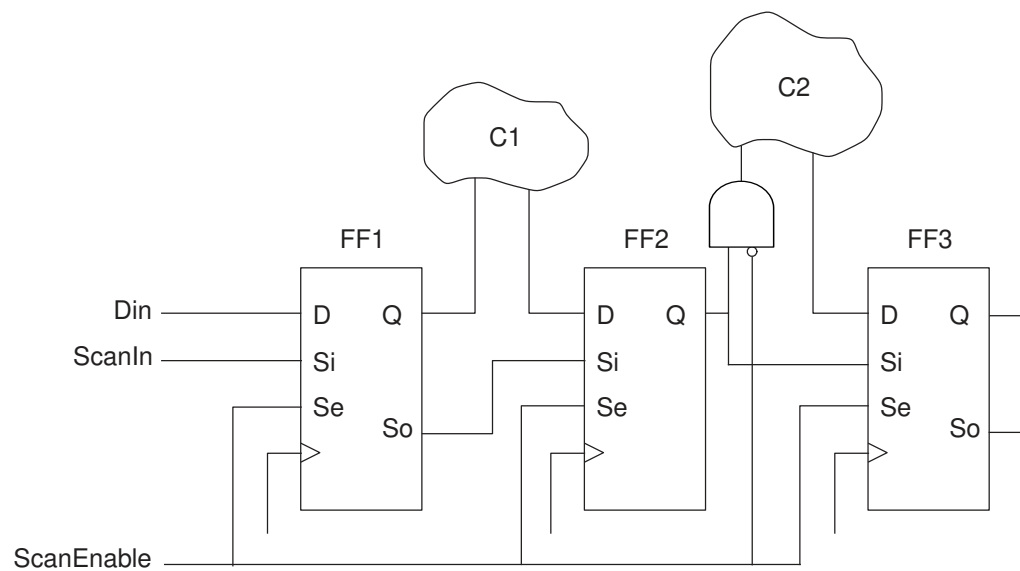
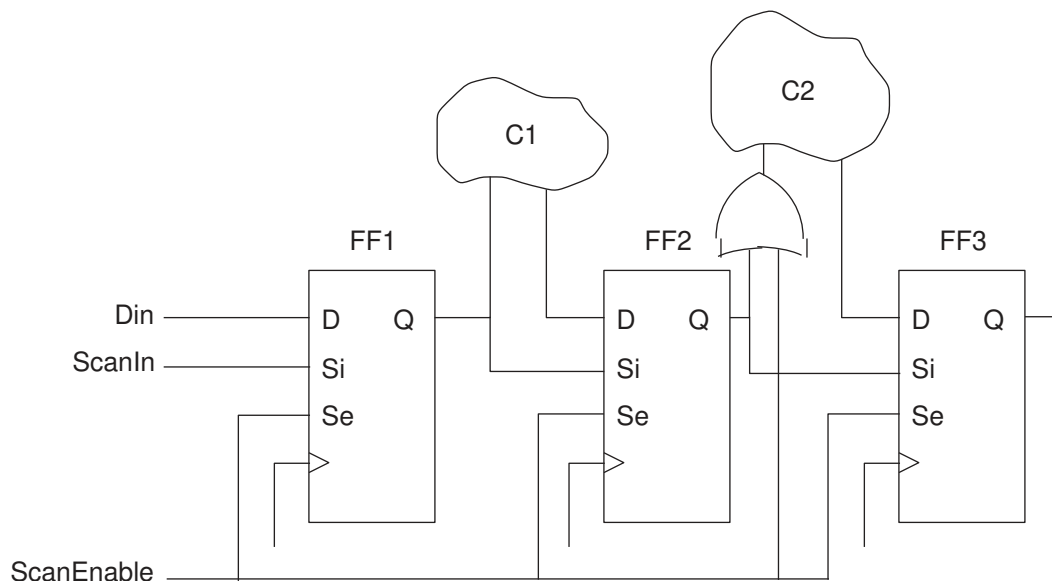


Figure 39 Design With OR-Gating Logic Inserted on the Functional Output of FF2, Where FF2 Has a Single Output Pin Shared Between Functional and Scan Output



To use this feature, run the `set_scan_suppress_toggling` command as part of your DFT specifications before you run the `insert_dft` command.

The `set_scan_suppress_toggling` command has the following syntax:

```
set_scan_suppress_toggling
  -selection_method [manual|auto|mixed]
  -include_elements collection_of_design_objects
  -exclude_elements collection_of_design_objects
  -ignore_timing_impact [true|false]
  -min_slack [0_to_1000]
  -total_percentage_gating [0.001_to_100]
```

Argument	Description
<code>-selection_method</code> manual auto mixed	This option defaults to <code>manual</code> , which applies gating only for objects manually specified with the <code>-include_elements</code> option. The <code>auto</code> value enables automatic selection of flip-flops for gating using power-based heuristics. The <code>mixed</code> value enables a combined approach, allowing the manual specification to be supplemented with automatic selection by the tool.

Argument	Description
<code>-include_elements</code> <code>collection_of_design_objects</code>	This option is used to manually specify a collection of design objects for gating. Flip-flop instance names, hierarchical cell names, and flip-flop and design references can be specified. A hierarchical cell specification includes all scan flip-flops in the cell.
<code>-exclude_elements</code> <code>collection_of_design_objects</code>	This option is used to manually specify a collection of design objects that should be excluded from gating. Flip-flop instance names, hierarchical cell names, and flip-flop and design references can be specified. A hierarchical cell specification includes all scan flip-flops in the cell.
<code>-ignore_timing_impact</code> <code>true</code> <code>false</code>	This option is <code>false</code> by default. When this option is set to <code>true</code> , the minimum slack requirement specified by the <code>-min_slack</code> option is ignored. When set to <code>true</code> , the <code>-exclude_elements</code> option might be useful for guiding automatic selection.
<code>-min_slack</code> <code>num_0_to_1000</code>	This option specifies the required minimum slack value after gating has been added. The default is 0, which means the slack should be nonnegative after gating.
<code>-total_percentage_gating</code> <code>num_0.001_to_100</code>	This option specifies the target percentage of scan flip-flops to be automatically selected for gating when the <code>-selection_method</code> option is set to <code>auto</code> or <code>mixed</code> . The default for this option is 5.

Functional gating logic can be inserted if the scan flip-flop is part of a scan chain. A flip-flop is excluded from gating consideration if it meets any of the following criteria:

- The flip-flop is manually excluded with the `-exclude_elements` option.
- The flip-flop is part of a shift register segment identified by the `compile_fusion` command.
- In a bottom-up flow, the flip-flop is within a block where test models are used or you have specified a `set_dont_touch` attribute on the block.
- The Q or QN pin of the flip-flop connects only to the scan-in signal. (They can be gated if they are functionally connected).
- The logic does not meet the minimum slack limit or would not meet the minimum slack limit if gating is inserted.

When the `-selection_method` option is set to `auto` or `mixed`, flip-flops are automatically selected for gating using propagated switching power analysis. A percentage value between 5 and 30 percent is typically specified with the `-total_percentage_gating` option. Consider the trade-offs between test-mode power consumption, functional power

consumption, and functional timing when choosing a gating percentage value. As the gating percentage value is increased, leakage power increases, and the potential for layout congestion and timing closure difficulty increases. While considering timing and power trade-offs, you should also consider that TestMAX ATPG has several power-aware algorithms that seek to reduce shift and capture flip-flop toggle rates during the ATPG process.

When the `-selection_method` option is set to `manual` or `mixed`, all flip-flops specified with the `-include_elements` option are gated, except those meeting the exclusion criteria defined earlier.

When the `-selection_method` option is set to `mixed`, all flip-flops specified with the `-include_elements` option are gated, even if the `-total_percentage_gating` target is exceeded. If the `-total_percentage_gating` target is not met by the manually specified flip-flops, additional flip-flops are selected automatically to meet the goal.

Use the `report_scan_suppress_toggling` command to confirm the option settings you specified with the `set_scan_suppress_toggling` command. Use the `remove_scan_suppress_toggling` command to remove the previous toggling suppression settings applied.

You can use the `preview_dft` command before DFT insertion to see which scan cells are gated. When scan output gating is enabled, the preview report annotates the gated scan cells with the `(q)` attribute. For example:

```
fc_shell> preview_dft
...
(q) shows cell scan-out drives a toggle suppressing gate
Scan chain '1' (SI1 --> SO1) contains 48 cells
CORE/Dstrobe_reg (CLK, 45.0, rising)
CORE/R1_reg[0] (q)
CORE/
```

When the `-selection_method` option is set to `auto` or `mixed`, the `preview_dft` command reports the number of scan flip-flops automatically selected for gating, the number considered for gating, the number that you manually included, and the number rejected due to timing considerations.

During DFT insertion, the combinational gating cell instances are named using the `compile.flow.user_instance_name_prefix` application option, by default.

After DFT insertion completes, you can use the `test_scan_suppress_toggling` cell attribute to find the gating cells. For example,

```
fc_shell> set cells [get_cells -hierarchical * \
                  -filter {test_scan_suppress_toggling == true}]
{CORE/U181 CORE/U182 U224 U225 U226}
```


4

DFTMAX Compression

DFTMAX scan compression provides synthesis-based scan data compression technology to lower the cost of testing larger designs. DFTMAX compression reduces manufacturing test costs by delivering a significant test data and test time reduction with very low silicon area overhead.

The following topics describe DFTMAX scan compression:

- [The DFTMAX Compression Architecture](#)
- [Configuring DFTMAX Compression](#)
- [Reducing Power Consumption in DFTMAX Designs](#)
- [Excluding Scan Chains From Scan Compression](#)
- [Scan Compression and OCC Controllers](#)
- [Using Compressed Clock Chains](#)
- [Defining External Clock Chains](#)

The DFTMAX Compression Architecture

The DFTMAX compression architecture is described in the following topics:

- [The DFTMAX Codec](#)
- [Decompressor Operation](#)
- [Compressor Operation](#)
- [Compressor X-Blocking](#)
- [Codec Implementation](#)

The DFTMAX Codec

DFTMAX compressed scan appears similar to standard scan at the chip-level interface, but it contains combinational compression logic and uses many more scan chains of shorter lengths within the chip core. As scan input values are shifted in, the decompressor

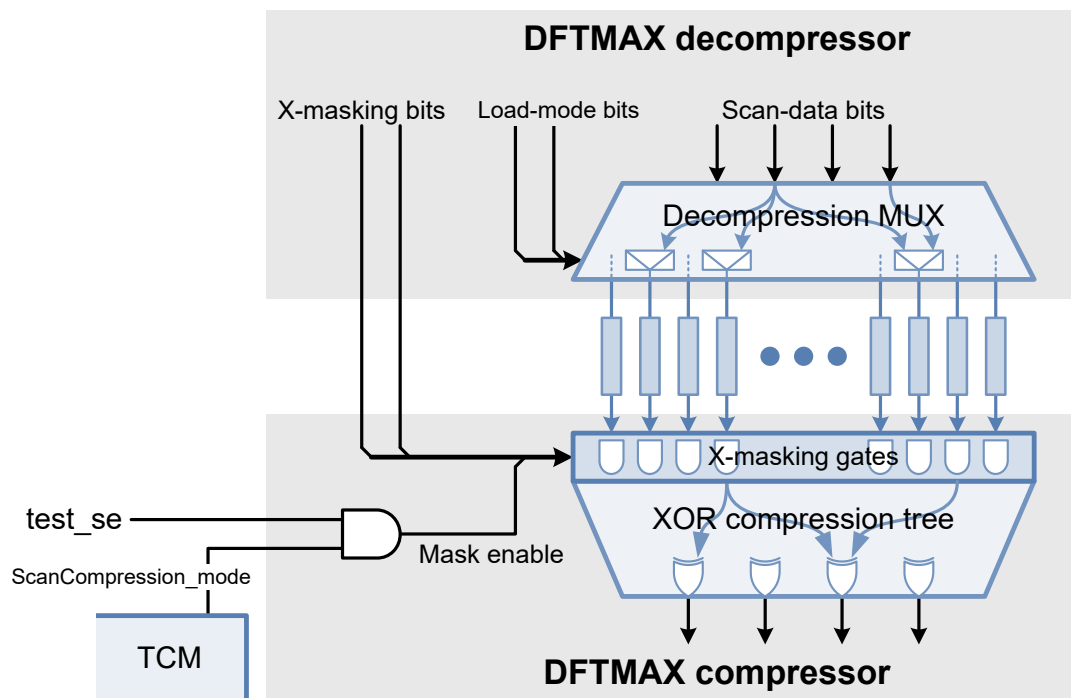
distributes them across numerous scan chains. The distribution method is accomplished with a patented hardware scheme and a process that allows chip-level scan input values to be placed into various scan chains. To maximize test coverage and minimize pattern count, the decompressor adapts to the needs of automatic pattern generation to supply the required values in the scan chain cells.

DFTMAX compression provides the following key benefits and features:

- A significant test time and test data reduction compared to standard scan
- Similar ease-of-use as standard scan
- Concurrent optimization of area, power, timing, physical constraints, and test constraints through a synthesis-based implementation
- Unknown logic value (X) handling
- Flexible scan channel configurations to support multisite testing and wafer-level burn-in

Figure 40 shows the DFTMAX compression architecture.

Figure 40 DFTMAX Compression Architecture



DFTMAX compression uses a larger number of shorter chains, called *compressed scan chains*, which reduces tester time. The decompressor controls the flow of scan data into the scan chains. The compressor reduces the captured data from the larger number of compressed scan chains so that it can be observed through the scan-out ports. The

combination of the decompressor and compressor wrapped around the scan chains is called the *codec*, which is short for compressor-decompressor.

The codec significantly reduces the amount of test data needed to comprehensively test the chip. In turn, this lowers automatic test equipment (ATE) memory requirements.

Decompressor Operation

The decompressor uses MUX logic to drive the scan chains with different combinations of scan-data pins. One or more scan-in pins, called *load-mode* pins, are dedicated as the MUX select signals.

This logic structure takes advantage of the fact that not every scan cell must be uniquely controllable in every pattern. Typically, only a sparse set of scan cells are required to be controlled in a pattern. In each shift clock cycle, ATPG can choose load-mode and scan data values that steer these required values into the compressed scan chains.

Some designs might have complex logic that requires more scan cells to be controlled in each pattern. As the decompressor input width increases, the number of load-mode and scan-data pins increases to provide additional controllability at the decompressor outputs.

Compressor Operation

Each compressor output is driven by a different combination of compressed scan chains, combined using XOR logic. Thus, an incorrect data value (fault) from a single compressed scan chain results in a specific signature of incorrect values at the compressor outputs.

X-masking gates prevent X (unknown) values from propagating through the XOR logic and destroying other chain observations. See [Compressor X-Blocking](#).

To reduce power, all compressor inputs are masked whenever the codec is not active and shifting.

The compressor input width is equal to the number of compressed scan chains. As the compressed chain count increases, more XOR configurations are needed. If the chain count is increased too high, the XOR configurations (and therefore the compressed chain signatures) must repeat, which can affect the diagnosability of the design.

As the compressor output width increases, the number of fault signatures observed at each output port decreases. This can improve the diagnosability of the design, especially when multiple faults must be simultaneously diagnosed.

Compressor X-Blocking

The X-masking logic in the compressor provides the following observe modes:

- A fully unmasked observe mode, which observes all chains
- Additional masked observe modes, each of which masks and observes a different pattern of chains

Information about the X-blocking observe modes is contained in the SPF (in the `CompressorStructures` section, then in the `Compressor "U_compressor_modename"` definition).

Codec Implementation

When scan compression is used, the compressed chains are architected, then used as the base chains for other test modes. This reduces the number of reconfiguration MUXs needed to implement the other modes.

The decompressor and compressor use congestion-reduction logic structures to efficiently connect to the physically distributed base chains.

The X-masking logic introduces a combinational path between the scan-in and scan-out ports (through the mask signal and XOR reduction logic). This path can potentially contain long routes as well as combinational logic. You can use the pipelined scan data feature to relax these timing requirements. For more information, see [Chapter 7, Pipelined Scan Data](#).

Optimizing DFTMAX Compression Codec

The scan chains distributed across the chip have connections to the DFTMAX codec that is physically grouped together in a large block. This might lead to routing congestion and QoR degradation issues.

This enhancement to the tool addresses the congestion issues by splitting the DFTMAX codec block into sub blocks and assigning a cluster of scan chains physically clustered to a sub block. This helps the codec logic to be placed as small blocks throughout the chip.

Use the `dft.enable_distributed_codec` application option and set it to `true` to enable the distributed codec feature for DFTMAX compression which enables the following:

- Implements the DFTMAX codec with multiple sub blocks
- Assigns physically clustered scan chain groups to the sub blocks
- The DFTMAX codec sub blocks are placed near the physically clustered scan chains

Configuring DFTMAX Compression

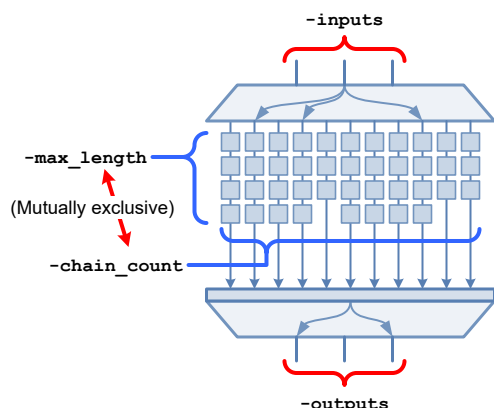
To enable and configure DFTMAX compression, use the following command:

```
set_scan_compression_configuration
-inputs num_scanins
-outputs num_scanouts
-chain_count num_chains | -max_length chain_length
[-test_mode mode_name]
```

The `-inputs` and `-outputs` options are required. There must be enough scan-in and scan-out signals defined to satisfy the specification.

The `-chain_count` and `-max_length` options are mutually exclusive; one is required.

These options apply to the compression architecture as follows:



Scan compression is automatically enabled when you configure it.

By default, the command creates and configures a mode named `ScanCompression_mode`. If you are using user-defined test modes, you specify the mode to configure with the `-test_mode` option.

If you are also implementing a standard scan mode, keep in mind that these compressed chains become the base chains used to create other modes.

If the specified compressed scan chain count cannot be satisfied, the tool issues a message that contains information about how many compressed scan chains were requested and how many compressed scan chains can be built for the current scan pin configuration. [Example 5](#) shows the message issued when 20 compressed scan chains are requested, but only 12 scan chains can be built.

Example 5 High X-Tolerance Message for Insufficient Scan-In Pins

```
Information: Architecting DFT Scan IP with '3' scan chains in mode
'Internal_scan'
Information: Architecting DFT Compressor IP with '3' inputs / '3'
outputs / '20' internal chains in mode 'ScanCompression_mode'
Information: Compressor will have 100% x-tolerance
Warning: Reducing the number of chains for 100% x-tolerant compressor
from 20 to 12
Warning: Architecting unload compressor with no direct diagnosis support
Error: Architecting of Load/Unload compressor failed with the given set
of parameters
Error: Unsuccessful DFT Insertion (DFT-1027)
Error: Unsuccessful DFT Architect (DFT-2021)
```

You can also use the TestMAX ATPG `analyze_compressors` command to determine if a codec can be built for a given set of parameters. For more information, see TestMAX ATPG and TestMAX Diagnosis Online Help.

Scan-In and Scan-Out Requirements

The codec architecture imposes a limit on the number of compressed scan chains you can build with a given number of scan-in and scan-out pins. [Table 4](#) shows the maximum number of compressed scan chains that can be built for a given set of scan-in and scan-out pins. The listed maximum number of chains assumes that a single decompressor input is dedicated to OCC clock chains. Additional dedicated clock chain decompressor inputs will reduce the limit.

Table 4 *Compressed Scan Chain Limits*

Number of scan-in and scan-out pins	Maximum number of chains without OCC controller	Maximum number of chains with OCC controller
2	4	
3	12	6
4	32	16
5	80	40
6	192	96
7	448	224
8	1024	512
9	2304	1152
10	5120	2560
11	11264	5632
12	24576	12288
13	32000	26624
14 and higher	32000	32000

If the on-chip clocking (OCC) feature is used with compressed clock chains, the dedicated decompressor scan input lowers the limit. For more information about compressed clock chains, see [Scan Compression and OCC Controllers](#).

[Table 5](#) shows the maximum compressed scan chain count when high X-tolerance is used with some asymmetrical low-pin-count configurations:

Table 5 *Compressed Scan Chain Limits for Asymmetrical Scan Pins*

Asymmetrical scan-in, scan-out pin configuration	Maximum number of chains without OCC controller	Maximum number of chains with OCC controller
2 scan-ins, 1 scan-out	2	
3 scan-ins, 1 scan-out	4	
3 scan-ins, 2 scan-outs	8	4
4 scan-ins, 3 scan-outs	24	12
5 scan-ins, 4 scan-outs	64	32
6 scan-ins, 5 scan-outs	160	80

Reducing Power Consumption in DFTMAX Designs

You can use shift power groups to reduce power consumption during scan shift. This feature is described in the following topics:

This chapter includes the following topics:

- [The Shift Power Groups Architecture](#)
- [Scan-Enable Signal Requirements for Shift Power Groups](#)
- [Configuring Shift Power Groups](#)
- [Integrating Cores With Shift Power Groups in Hierarchical Flows](#)
- [Configuring Shift Power Groups in TestMAX ATPG](#)
- [Using Shift Power Groups With Other DFT Features](#)
- [Limitations of Shift Power Groups](#)

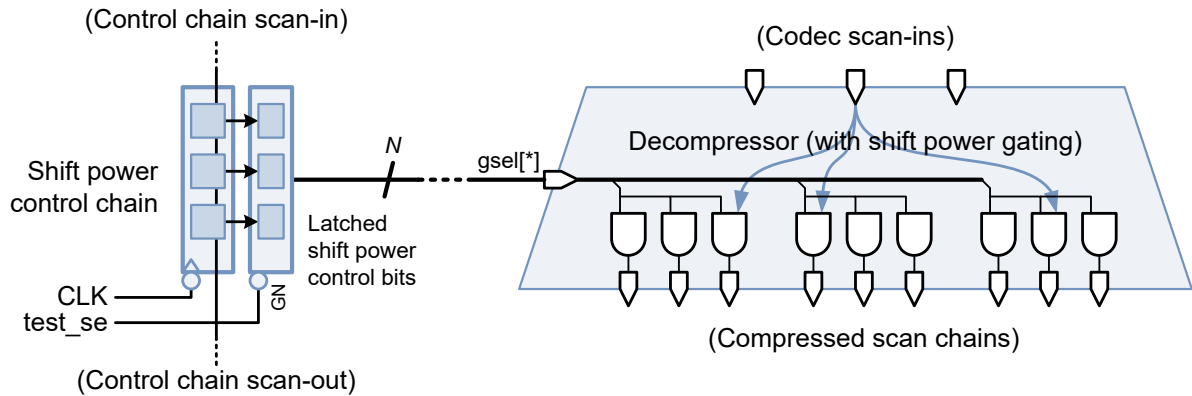
The Shift Power Groups Architecture

During scan shift, there is significant toggle activity in the scan chains. At high scan shift frequencies, this can result in higher-than-desired shift power consumption.

The shift power groups feature helps reduce power consumption during scan shift in DFTMAX compressed scan modes. This feature inserts AND gates at the decompressor

outputs before each compressed scan chain. The chains are gated in groups that are controlled by a shift power control (SPC) chain, as shown in [Figure 41](#).

Figure 41 Shift Power Groups Decompressor Architecture



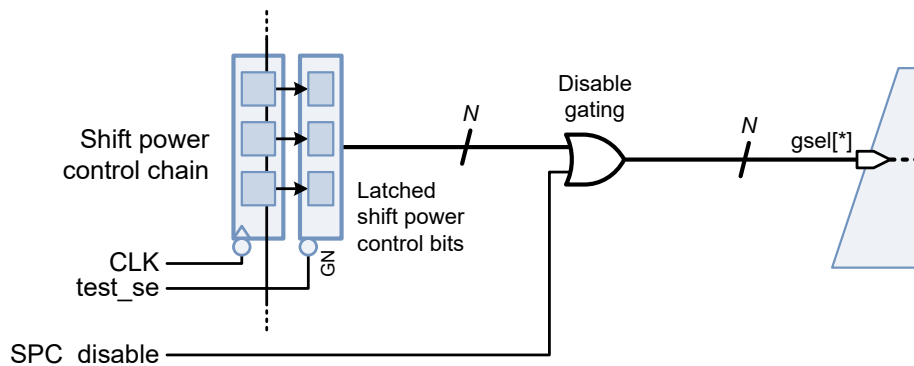
The SPC chain is an external (uncompressed) chain outside the DFTMAX codec. When scan-in completes, the SPC registers contain the group mask values for the next pattern. The de-asserted scan-enable signal, `test_se`, latches these bits into shadow latches that retain the mask values for scan-in of the next pattern.

TestMAX ATPG configures the group masking in each pattern, depending on the power constraints and the number of care bits in each chain group. The larger number of short chains inherent to scan compression provide finer granularity for this control. Masked groups load constant values into their chains, which reduces overall toggle activity.

SPC chains must be external chains because a compressed SPC chain would gate itself, preventing it from reliably loading in each pattern.

The shift power logic also includes a hardware disable signal that, when asserted, disables the shift power logic by enabling all compressed chains, as shown in [Figure 42](#). This signal must be de-asserted or asserted prior to DRC, depending on whether the shift power groups feature is enabled in TestMAX ATPG or not, respectively.

Figure 42 Shift Power Disabling Logic



Scan-Enable Signal Requirements for Shift Power Groups

When the DFTMAX shift-power codec scan-enable signal is de-asserted, the shift power logic latches the values from the control chain. Therefore, for proper operation, *this scan-enable signal must be held in the inactive state in all capture procedures.*

The STIL protocol file created by the tool does not apply this constraint. As a result, you must manually constrain any scan-enable signals used by DFTMAX shift-power codecs, as described in [Configuring Shift Power Groups in TestMAX ATPG](#).

Alternatively, if you use a custom STIL protocol file, you can update it to constrain the scan-enable signals in all capture procedures.

Configuring Shift Power Groups

To configure the shift power groups feature, do the following:

1. Enable the shift power groups feature.

```
fc_shell> set_scan_compression_configuration \
           -shift_power_groups true
```

2. Specify the configuration of the compressed chain groups.

- To directly specify the number of compressed chain groups, and therefore the length of the SPC chain, use the `-shift_power_chain_length` option:

```
fc_shell> set_scan_compression_configuration \
           -shift_power_chain_length 16
```

- To specify the number of compressed chains in each group, which makes the SPC chain length a function of the compressed chain count, use the `-shift_power_chain_ratio` option:

```
fc_shell> set_scan_compression_configuration \  
          -shift_power_chain_ratio 12
```

These options are mutually exclusive.

3. Define the shift power groups disable signal.

```
fc_shell> set_dft_signal -view spec -type TestControl \  
                  -port SPC_DISABLE  
  
fc_shell> set_scan_compression_configuration \  
          -shift_power_disable SPC_DISABLE
```

You can define the disable signal using the `-port` *and* or *or* `-hookup_pin` options of the `set_dft_signal` command. For an “internal pins” hookup pin, you must use a `test_setup` protocol that de-asserts the disable signal.

4. Configure the shift power control chain.

- If no OCC controllers (DFT-inserted or user-defined) are configured in the current design, you must configure an external SPC chain.

Specify the clock, scan-in, and scan-out signals to use for the SPC chain:

```
fc_shell> set_scan_compression_configuration \  
          -shift_power_clock CLK  
  
fc_shell> set_scan_path SPC -class spc \  
          -scan_data_in SPC_IN \  
          -scan_data_out SPC_OUT \  
          -test_mode all
```

You do not need to specify SPC scan path elements; the SPC chain is automatically included in the specification.

- If OCC controllers (DFT-inserted or user-defined) are configured in the current design, configure an external clock chain:

```
fc_shell> set_scan_path OCC -class occ \  
          -scan_data_in OCC_IN \  
          -scan_data_out OCC_OUT \  
          -test_mode all ;# includes the SPC chain too
```

An external clock chain is required because a compressed clock chain would be gated, preventing it from reliably loading in each pattern.

By default, the tool automatically appends the SPC chain to the clock chain. It is clocked by the ATE clock unless specified otherwise with the `-shift_power_clock` option of the `set_scan_compression_configuration` command.

Alternatively, you can explicitly define a separate external SPC chain as previously described, which provides independent access to the OCC and SPC chains when the core is integrated.

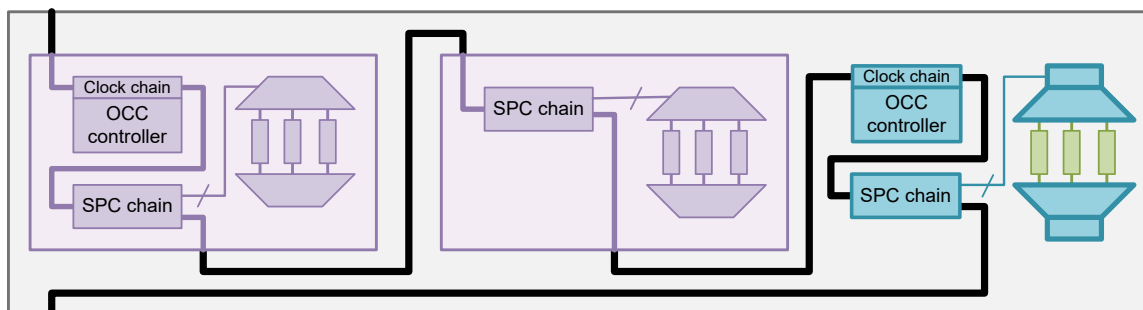
Integrating Cores With Shift Power Groups in Hierarchical Flows

This topic describes how to integrate cores with shift power groups.

Configuring the Control Chain for Shift Power Groups Cores

When you integrate cores that use shift power groups, you must define a top-level external control chain that includes all core-level and top-level clock chains *and* or SPC chains, as shown in [Figure 43](#).

Figure 43 External Control Chain in a Shift Power Groups Design



Use the `set_scan_path` command to define the top-level external control chain as follows:

- If any core-level or top-level clock chains exist or is inserted, then define the external chain using the `-class occ` option.
- If only core-level or top-level SPC chains exist or is inserted, then define the external chain using the `-class spc` option.
- All core-level clock chains and SPC chains *must be explicitly included in the specification* using the `-include_elements` option. They are not automatically included.
- All top-level clock chains *must be explicitly included in the specification* using the `-include_elements` option. They are not automatically included.
- Top-level SPC chains *are* automatically included in the external chain.

The following example includes core-level clock chains and SPC chains along with top-level clock chains and SPC chains:

```
set_scan_path clock_chain -class occ \
  -include_elements { \
    core1/SPC \
    core2/SPC \
    coreOCC1/OCC \
    coreOCC2/OCC \
    snps_clk_chain_2/clock_chain} \
  -complete true \
  -scan_data_in OCC_SI \
  -scan_data_out OCC_SO \
  -test_mode all
# (the top-level SPC chain is automatically included)
```

If you concatenate external control chains from pipelined cores, those cores must be created with beginning and ending retiming registers to avoid edge-related concatenation issues at the top-level. See the retiming register information in [Using Shift Power Groups With Other DFT Features on page 101](#).

Connecting Core-Level Shift Power Disable Signals

When integrating cores that contain shift power groups, you must manually hook up core-level shift power disable signals to a top-level disable signal.

You can use one of the following methods:

- Include preexisting connections to the cores in your top-level RTL.
- Use ECO commands, such as `disconnect_net` and `connect_pin`, to make the connections to the cores.

You can share a single disable signal or use multiple disable signals.

All shift power disable signals must be *de-asserted* (set to logic 0) to *enable* the shift power logic. The DFT-created disable signal for a top-level codec is already de-asserted in the SPF. Additional disable signals must be manually de-asserted by defining constant signals on them. For example,

```
fc_shell> set_dft_signal -view existing_dft -type Constant \
  -port SPC_CORE_DISABLE* -active_state 0
```

Configuring Shift Power Groups for a Top-Level Codec

If you are implementing a top-level codec, you must configure shift power groups for that codec using the pertinent options of the `set_scan_compression_configuration` command. For more information, see [Configuring Shift Power Groups on page 97](#).

Configuring Shift Power Groups in TestMAX ATPG

Use the following commands in TestMAX ATPG to configure ATPG use of the shift power groups hardware:

```
DRC_T> set_drc -spc_chain SPC_chain_name
DRC_T> set_atpg -shift_controller_peak probability_value
```

SPC_chain_name is the name of the scan path that contains the SPC chain.
probability_value is the maximum percentage of scan cells that can switch in a shift cycle. TestMAX ATPG rejects patterns that exceed this switching percentage.

As described in [Scan-Enable Signal Requirements for Shift Power Groups on page 97](#), you must also constrain the scan-enable signal used by the shift-power logic to be de-asserted during capture. For example,

```
DRC_T> add_pi_constraints 0 SE_port ;# signal is active-high
```

The STIL protocol file (SPF) created by the TestMAX DFT tool enables shift power groups by default. When enabled, you must configure the feature with the preceding commands, otherwise the compressed scan chains will fail DRC due to chain blockages.

Alternatively, you can assert the shift power disable signal, in which case the DFTMAX codec degenerates to a non-shift-power codec and no shift power configuration commands are needed.

Using Shift Power Groups With Other DFT Features

The shift power groups feature interacts with other DFT features as follows:

- Multiple test modes

You can use shift power groups with multiple test modes, including multiple DFTMAX compression modes. Configure the SPC chain in each DFTMAX compression mode. See [Per Test Mode Scan Compression Configuration Commands](#) for supported options.

The control chain must be external only in DFTMAX compression modes. If required, you can use the `-test_mode` option of the `set_scan_path` specification to limit the external chain specification to those modes (instead of `all`); the control chains are incorporated into regular scan chains in other modes.

If shift power groups are used, they must be used in all DFTMAX test modes. You cannot mix codecs with and without shift power groups across test modes.

- DFT partitions

You can use shift power groups with DFT partitions. Configure the SPC chain in each partition that contains a DFTMAX codec. See [Per-Partition Scan Compression Configuration Commands on page 103](#) for supported options.

Note that although SPC chains can be created for multiple partitions, they are all stitched into the single external control chain specified by the `set_scan_path` command.

If shift power groups are used, they must be used in all partitions that contain a DFTMAX codec. You cannot mix codecs with and without shift power groups across partitions.

- Retiming registers

When you enable beginning *and* or *or* ending retiming registers, SPC chains (and clock chains) are clocked on the leading clock edge instead of the trailing clock edge. This facilitates control chain concatenation at the top-level.

Per-Test-Mode Scan Compression Configuration Commands

This topic lists the commands and options you can use to configure compressed scan insertion for specific test modes. Additional commands are available to configure other aspects of DFT insertion for multiple test modes.

For information about other DFT commands that can be applied to specific test modes, see [Supported Test Specification Commands for Test Modes on page 102](#).

For information on how to order global and mode-specific configuration commands in your scripts, see [Recommended Ordering of Global and Mode-Specific Commands on page 102](#).

Supported Test Specification Commands for Test Modes

This topic lists the commands and options you can use to configure DFT insertion for specific test modes. These commands and options honor the `-test_mode` option or the current test-mode focus. Other DFT configuration commands and options apply to all test modes.

Recommended Ordering of Global and Mode-Specific Commands

When applying scan specifications in a multiple test-mode environment, perform these steps:

1. Define TestMode signals with the `set_dft_signal -test_mode all` command.
2. Define all test modes, their usage, and optional encodings with the `define_test_mode` command.

3. Define clock and asynchronous DFT signals and constants that are common to all test modes with the `set_dft_signal -test_mode all` command.
4. Define any mode-specific DFT signals with the `set_dft_signal -test_mode test_mode_name` command.
5. Specify any scan specifications to be used in all test modes using the `-test_mode all` option of the `set_scan_configuration` and `set_scan_path` commands.
6. Specify any scan specifications to be used in specific test modes using the `-test_mode test_mode_name` option of the `set_scan_configuration` and `set_scan_path` commands.

Note:

If you reference any test modes using the `-test_mode` option of any DFT configuration commands, you must define those test modes with the `define_test_mode` command before referencing them.

Per-Partition Scan Compression Configuration Commands

You can use to configure DFTMAX scan compression on a per-partition basis as follows:

- `set_scan_compression_configuration`
- `set_dft_location`
- `set_dft_signal`

Note:

Commands not listed in this section should be applied as part of the global DFT configuration settings.

Limitations of Shift Power Groups

Note the following limitations of shift power groups:

- This feature applies only to the compressed scan chains it is configured for. Standard scan modes are unaffected.
- When shift power groups are used, they must be used
 - In all DFTMAX test modes
 - In all codecs in the design (across both cores and DFT partitions)

You cannot mix DFTMAX codecs with and without shift power groups within the same design.

- The shift power control (SPC) chain must be an external (uncompressed) chain, which you explicitly define using the `set_scan_path` command.
- When using OCC controllers, you must use external (uncompressed) clock chains.
- When integrating cores that contains shift power groups,
 - You must manually hook up the core-level shift power disable signal to a top-level shift power disable signal.
 - In the Hybrid integration flow, you must explicitly configure the top-level codec using the `-inputs` and `-outputs` options of the `set_scan_compression_configuration` command. Otherwise, incorrect SPF might be generated.
- The `report_scan_path` command does not report SPC chain information.

In the TestMAX ATPG tool, the following requirement applies:

- You must use the `add_pi_constraints` command to constrain the scan-enable signal to be de-asserted during scan capture.

In the TestMAX Diagnosis tool, the following requirement applies:

- Diagnosis capability is limited. High-resolution diagnostics are not supported when shift power groups are used. Assert the shift power disable signal to generate patterns for high-resolution diagnostics.

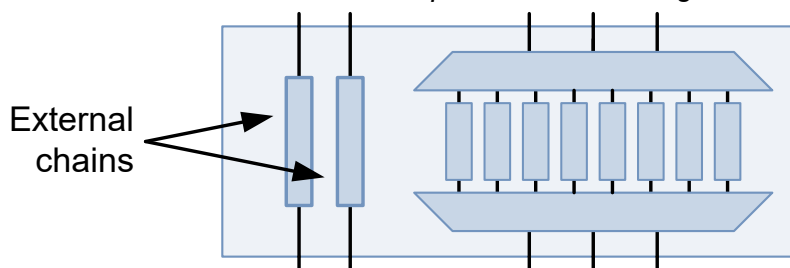
During ATPG, the following tasks are not supported when using shift power groups:

- Analyzing X effects or X sources performed during a TestMAX ATPG simulation
- Comparing simulation results from either a VCD simulation file, the internal patterns from the fast-sequential simulator, or the internal patterns from the full-sequential simulator
- Reporting total (cumulative) power data with the `report_power` command after performing multiple (incremental) ATPG runs
- Saving patterns and fault lists to files at a specified checkpoint interval during ATPG pattern generation
- Saving a GZIP-compressed parallel pattern set that can be simulated during the ATPG process
- Assigning ATPG constraints during an IDDQ measure strobe when the IDDQ fault model is selected

Excluding Scan Chains From Scan Compression

In some cases, you might need to exclude specific scan cells from scan compression by keeping them on a separate uncompressed scan chain. Such a scan chain is called an *external chain*. Figure 44 shows two external chains in a compressed scan design.

Figure 44 External Chains in a Compressed Scan Design



To define an external chain, use the `-scan_data_in` and `-scan_data_out` options of the `set_scan_path` command:

```
set_scan_path scan_chain_name
  -test_mode all
  -complete true
  -include_elements element_list
  -scan_data_in port_name -scan_data_out port_name
```

Note the following aspects of external chain definitions:

- The test mode must be previously defined with the `define_test_mode` command. The `-test_mode all` option applies the external chain specification to all test modes defined previously. To limit the specification to a specific scan or compression mode, specify it with the `-test_mode` option.
- The scan input and output ports must be previously defined with the `set_dft_signal` command.
- If you are using the pipelined scan data feature, external chains are treated the same as other scan chains. This means, in the automatically inserted pipelined scan data flow, the tool inserts pipeline registers around the external scan chains the same way it does with other scan chains.

Scan Compression and OCC Controllers

On-chip clocking (OCC) controllers allow on-chip clock sources to be used for at-speed capture during device testing. In an OCC controller flow, the *clock chain* is a special scan segment that provides control over the at-speed capture pulse sequence generated by the OCC controller.

In a scan compression flow, the clock chain can be compressed or external (uncompressed), as described in the following topics:

- [Using Compressed Clock Chains](#)
- [Defining External Clock Chains](#)

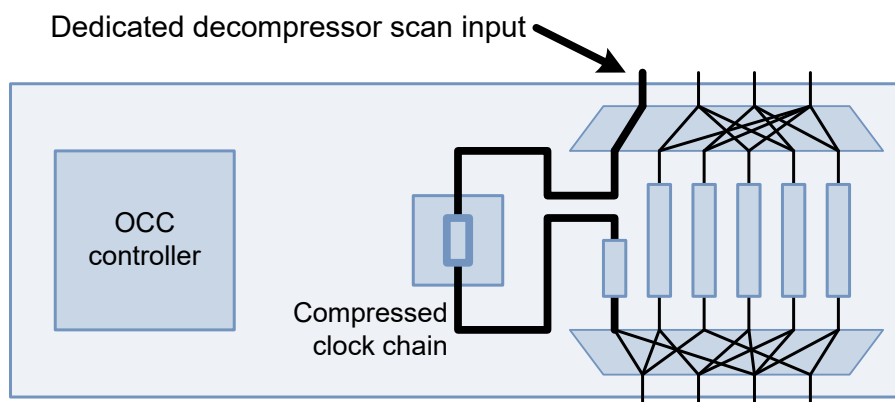
See Also

- [On-Chip Clocking \(OCC\)](#)

Using Compressed Clock Chains

In a DFTMAX flow, when you insert scan compression in a DFT-inserted or user-defined OCC controller flow, the clock chain is placed between the decompressor and compressor by default. The decompressor drives the clock chain along with the other compressed scan chains, but it dedicates a decompressor scan input to the clock chain as shown in [Figure 45](#).

Figure 45 *Compressed Clock Chain in a Compressed Scan Design*



The decompressor scan input path passes through the decompressor to the clock chain input. This allows the clock chain values to be controlled without imposing constraints on other scan cells. Such a clock chain is called a *compressed clock chain* because it exists between the decompressor and compressor, even though it is driven by a dedicated scan-in signal as if it was uncompressed.

For codecs with few scan inputs and high compression ratios, DFTMAX compression might be forced to share the decompressor scan input with other compressed chains. This happens if the codec would otherwise not be implementable.

The dedicated scan-in signal reduces the number of scan-in signals available for data decompression into the remaining compressed chains. You should consider this

when determining compression architecture parameters such as scan input count and compressed chain count.

The clock chain, which is clocked on the trailing edge, is always placed at the beginning of its compressed scan chain. Additional scan cells can follow the clock chain for length-balancing purposes, as allowed by the clock-mixing settings applied to the current design. The compressed scan chain then proceeds into the compressor in the normal way.

The compressed clock chain reduces the maximum compressed scan chain limit that can be created for a given number of scan-in signals. For more information, see [Scan-In and Scan-Out Requirements](#).

To see which compressed scan chain contains the clock chain, use the `preview_dft` command. The clock chain is marked with a clock chain segment attribute (s):

```
*****
Current mode: ScanCompression_mode
*****
Number of chains: 6

(l) shows cell scan-out drives a lockup latch
(s) shows cell is a scan segment
(m) shows cell scan-out drives a multi-mode multiplexer
(q) shows cell scan-out is q-gated
...

Scan chain 'occ_chain' contains 10 cells:

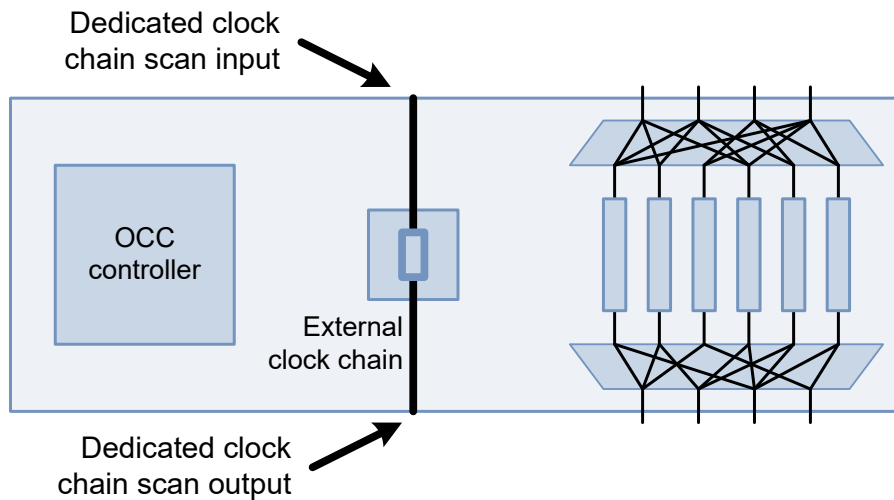
  occ_ctrl/1 (s) (voltage 0) (pwr domain
'DEFAULT_POWER_DOMAIN') (supply net 'VDD') (occ_ctrl/clk_0[0], 45,
rising)

Scan signals:
  test_scan_in: U_DFT_TOP_IP_0 (hookup pin:
U_DFT_TOP_IP_0/si_ScanCompression_mode[0])
  test_scan_out: U_DFT_TOP_IP_0 (hookup pin:
U_DFT_TOP_IP_0/so_ScanCompression_mode[0])
```

Defining External Clock Chains

External clock chains are uncompressed and exist outside the codec as shown in [Figure 46](#).

Figure 46 External Clock Chain in a Compressed Scan Design



External clock chains are normally used only with the following features:

To manually define the complete external clock chain, use the `set_scan_path` command. This method allows you to use specific scan-in and scan-out signals for the clock chain.

For example,

```
set_dft_signal -view spec -type ScanDataIn -port OCC_SI
set_dft_signal -view spec -type ScanDataOut -port OCC_SO
set_scan_path \
  MY_clock_chain -class occ \
    -include_elements {BLK1/OCC} \
    -scan_data_in OCC_SI -scan_data_out OCC_SO \
    -test_mode all
```

The `-class occ` option indicates that the scan path specification defines a clock chain. Use the `-include_elements` option to list the OCC chain segment. The `-test_mode` option must be specified.

You can define multiple external clock chains, if needed.

For more information on configuring clock chains, see [Configuring the Clock Chain](#).

5

DFTMAX Ultra Compression

DFTMAX Ultra compression uses a shift-register scan-data architecture and a single scan clock to deliver very high compression without restriction on the number of I/O pins. The input shift register feeds the decompression logic that provides data to many internal scan chains. The output shift register compresses the scan-out data using XOR logic. This architecture delivers high scan compression levels while providing a scan-compatible interface that retains the simplicity of a basic scan design.

The following topics describe the DFTMAX Ultra compression architecture:

- [DFTMAX Ultra Compression Architecture](#)
- [Optimizing DFTMAX Ultra Compression Codec](#)
- [Configuring DFTMAX Ultra Compression](#)
- [Using OCC Controllers With DFTMAX Ultra Compression](#)

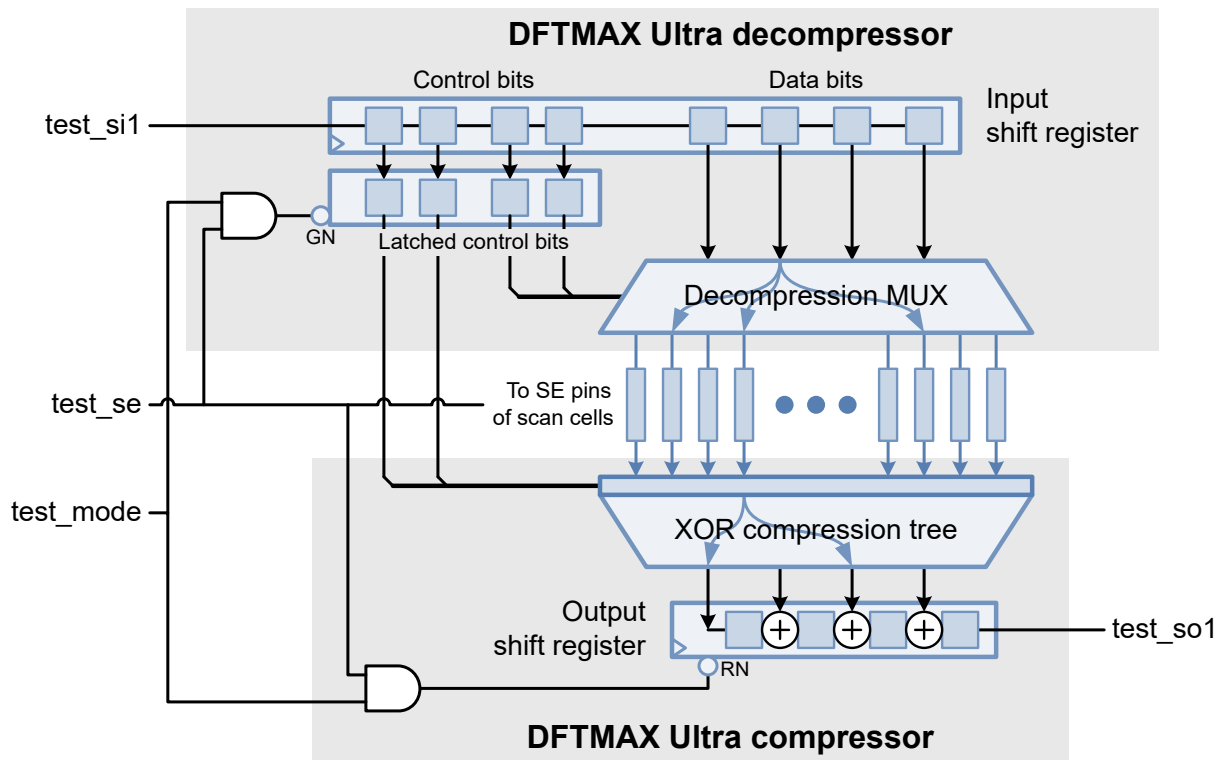
DFTMAX Ultra Compression Architecture

DFTMAX Ultra compression supports high levels of compression, high fault coverage, short scan chains, and low pin count. The scan architecture uses shift-register structures to feed in and read out the scan data streams, which enables high shift frequencies. All scan circuits operate at the same frequency; no codec clock controller circuit is needed for scan operations.

To insert DFTMAX Ultra scan compression, you specify the number of scan inputs and scan outputs, and the target number of compressed scan chains. The tool then implements an architecture based on your configuration. The DFTMAX Ultra architecture supports high compression levels using as few as one scan input and one scan output, even for a large number of internal scan chains.

[Figure 47](#) shows a block diagram for a single-input, single-output DFTMAX Ultra codec. Clock and control signals are active-high. The codec logic uses an existing scan clock; for simplicity, clock connections are not shown.

Figure 47 DFTMAX Ultra Compression Architecture, Single Scan-In and Scan-Out Pins



The features and function of the DFTMAX Ultra architecture are covered in the following topics:

- [Input Shift Register and Decompression MUX](#)
- [Control Register](#)
- [Output XOR Compression Tree and Shift Register](#)
- [Test Pattern Scan Procedure](#)
- [Scan-Enable Signal Requirements for Codec Operation](#)
- [Multiple-Input, Multiple-Output Architecture](#)

Input Shift Register and Decompression MUX

The input decompressor circuit uses a shift register and a decompression multiplexer (MUX). The input scan data at the scan-in pin feeds into the shift register, which is clocked on the leading clock edge at the normal scan clock rate. In this implementation example, the register has eight bits. The four register bits farthest from the scan-in pin feed into the decompression MUX.

The decompression MUX is a combinational logic block that causes each of the four scan data bits to fan out to multiple scan chains. The mapping of the four bits from the shift register to the scan chains remains constant for a particular pattern. However, the mapping can change from one pattern to the next. The mapping is controlled by bits in the control register. (For more details, see [Control Register](#).)

The shift-register structure that feeds into the decompression MUX causes the input data to stream into the scan chains multiple times. Therefore, the timing is shifted by one clock cycle from successive register bits in the shift register. This architecture allows the TestMAX ATPG tool to disperse the input data stream both in space and time – in space by fanning out to multiple chains under the control of the decompression MUX, and in time by controlling the stream of bits feeding the shift register. Although a single data stream enters the design, different chains receive different data by this time shifting.

The four last-arriving bits in the shift register are latched into the control register when the scan enable signal, `test_se`, is de-asserted. This de-assertion occurs once per pattern. Other than this once-per-pattern latching function, the input shift register bits feeding the control register operate only as a time-delay pipeline for the input data stream.

Control Register

The control register is a bank of latch cells that stores the configuration of the scan circuit for a given pattern. Some of the register bits control the mapping of input shift-register bits to scan chains through the decompression MUX, while others control the X-masking logic at the ends of the scan chains. In this example, two bits control the decompression MUX and two bits control the X-masking logic. The control register latches remain constant during scan shifting, so the scan configuration stays the same within a given pattern.

To program the control register, TestMAX ATPG appends the desired string of control bits to the end of the previous pattern's data stream. When scan-in completes, the control bits occupy the register positions that feed into the control register. The de-asserted scan-enable signal, `test_se`, latches these bits into the control register. Thus, the final bits of the scan-in pattern control X-masking for the current pattern to be scanned out and the decompression MUX mapping for the next pattern to be scanned in.

Output XOR Compression Tree and Shift Register

The output compressor circuit uses a combinational XOR compression tree and a sequential XOR output shift register.

The XOR compression tree is a multilevel combinational network of XOR gates that compresses the output bits from the scan chains into a smaller number of bits. Each scan chain feeds into multiple XOR tree outputs; this redundant logic helps to minimize the propagation of X values. In this example, the scan chain outputs are compressed into four bits that feed the output shift register.

If TestMAX ATPG determines that there are too many X values for the redundant XOR tree to isolate the X values, it invokes X-masking to block one or more scan chains during scan-out. The X-masking bits from the control register specify the chain or chains to mask for the current pattern and also specify the order of the signals feeding into the XOR compression logic.

The compressed bits feed into a chain of flip-flops that operate as an output shift register, which is clocked on the leading clock edge at the normal scan clock rate. During scan capture, the register is reset by the scan-enable signal. During scan shift, the XOR gate between each stage of the shift register merges scan data from an XOR compressor output into the scan data already moving through the output shift register. This architecture further compresses the scan data outputs from the XOR compression tree into a single data stream.

Test Pattern Scan Procedure

The ATE equipment performs the scan-in, scan-out procedure as specified by TestMAX ATPG. In this example, the scan procedure uses 15 extra clock cycles to flush out the extra bits from the input and output shift registers. For example, if the longest scan chain is 10,000 bits long, then the scan-in, scan-out procedure takes 10,015 scan clock cycles.

Consider two consecutive test patterns, 1 and 2, starting from the point at which pattern 1 has just been scanned in:

1. The first four bits of the input shift register (shifted in at the end of pattern 1) contain the desired scan control bits to be latched into the control register. The output XOR shift register contains leftover data from the previous pattern.
2. The scan-enable signal `test_se` changes from high to low, transitioning the device from scan shift mode to scan capture mode. The de-assertion of the `test_se` pin performs the following:
 - It latches the four control bits from the input shift register into the control register latches. This configures the X-masking circuit to scan out the data from pattern 1, and it configures the decompression MUX to decompress the data for incoming pattern 2.
 - It resets the output XOR shift register to known zero values.
3. The ATE equipment applies the test vector to the primary inputs of the device and reads the output vector from the primary outputs.
4. A clock pulse applied to the clock input causes the capture event, which changes the contents of the scan flip-flops.
5. The scan-enable signal `test_se` is asserted, which transitions the device from scan capture mode back into scan shift mode.

6. The ATE equipment applies a sequence of clock pulses at the scan clock rate. This reads out the captured scan data for pattern 1 through the test_so1 output and, at the same time, scans in the data for pattern 2 through the test_si1 input.
7. Scan-in and scan-out continue until the scan chains are filled with the data for pattern 2 (and the first four bits of the input shift register are filled with the control bits for the next pattern).

This same sequence is repeated for each pattern until the device is fully tested.

Before the initial scan-in of test pattern data, the MUX control bits of the control register must be programmed with values for proper decompression of the first test pattern. Therefore, the first test cycle uses an abbreviated padding pattern that contains only the MUX control bits and no actual scan data. For more information about padding patterns, see “Optimizing Padding Patterns” in TestMAX ATPG and TestMAX Diagnosis Online Help.

Scan-Enable Signal Requirements for Codec Operation

When the streaming codec scan-enable signal is de-asserted, the control register latches new control bit values, and the output shift register resets to a known state. Therefore, for proper operation, *this scan-enable signal must be held in the inactive state in all capture procedures.*

If you use the STIL protocol file created by the tool, the protocol already meets this requirement. In the capture procedures, the tool constrains all scan-enable signals that drive streaming codecs to the inactive state.

In some flows, streaming codecs cannot use signals defined with the `-usage` option of the `set_dft_signal` command.

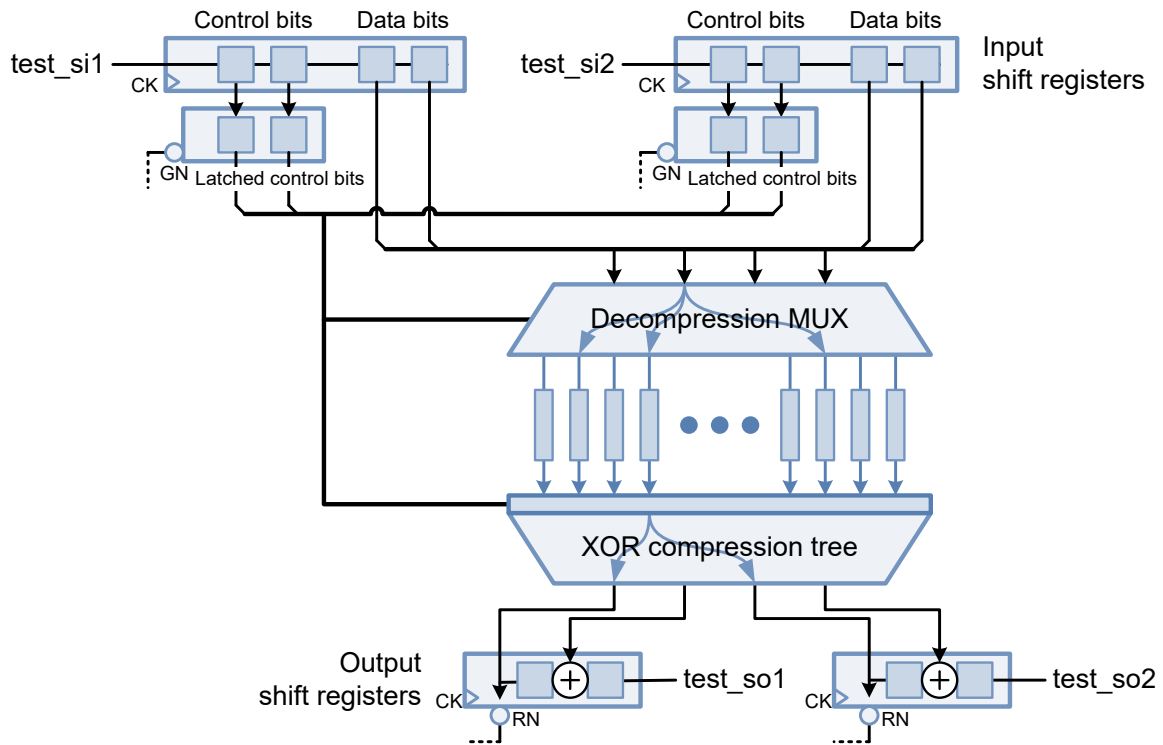
If you use a custom STIL protocol file, make sure that all scan-enable signals used by DFTMAX Ultra codecs are constrained to the inactive state in all capture procedures.

Multiple-Input, Multiple-Output Architecture

If the codec is configured to use multiple scan-in and scan-out connections, the tool synthesizes the scan circuitry in a manner similar to the single-pin circuit, but it splits the input and output shift registers into smaller segments and connects them to the available input and output pins. By using more scan I/O pins, you get shorter shift registers, improved controllability, and improved observability into the design.

Figure 48 shows a DFTMAX Ultra codec with two scan data inputs and two scan data outputs. The codec has four input shift-register scan data bits feeding into the decompression MUX, four control register bits that control the decompression and compression logic, and four output shift-register bits.

Figure 48 DFTMAX Ultra Compression Architecture, Multiple Scan-In and Scan-Out Pins



The tool determines the total input and output shift register lengths based on the number of compressed scan chains, then it then splits these shift register lengths across the scan inputs and outputs (rounding up shorter registers as needed). Therefore, as you increase the number of scan inputs and outputs, the scan shift overhead of the shift registers is reduced.

This bit distribution is more flexible than the single-input, single-output design because there are multiple independent data streams rather than one. This flexibility might allow the same fault coverage to be achieved with fewer patterns, but at the cost of using more device pins.

In this example, the distribution of bits in the input shift registers allows TestMAX ATPG to generate two independent data streams at the same time, one each for test_si1 and test_si2. Each input shift register provides its own scan data bits for multiplexing to the scan chains. For each compressed scan chain, TestMAX ATPG has a choice of up to four different bit streams: two from test_si1 and two from test_si2.

The control register bits are the last data bits shifted into the device for a pattern. Therefore, the bits of the shift register used for loading the control register are located closest to the input pin, whereas the bits of the shift register that are available to the decompression MUX are located farthest from the input pin.

On the output side, the output XOR shift register is divided into two smaller shift registers, one each for the output pins `test_so1` and `test_so2`. This reduces the amount of data compression performed in the shift register, which reduces the propagation of X values and provides greater observability into the design.

Optimizing DFTMAX Ultra Compression Codec

The scan chains distributed across the chip have connections to the DFTMAX Ultra codec that is physically grouped together in a large block. This might lead to routing congestion and QoR degradation issues.

This enhancement to the tool addresses the congestion issues by splitting the DFTMAX Ultra codec block into sub blocks and assigning a cluster of scan chains physically clustered to a sub block. This helps the codec logic to be placed as small blocks throughout the chip.

Use the `dft.enable_distributed_codec` application option and set it to `true` to enable the distributed codec feature for DFTMAX Ultra compression which enables the following:

- Implements the DFTMAX Ultra codec with multiple sub blocks
- Hierarchical XOR is inserted at the compressor outputs for DFTMax Ultra codec
- Assigns physically clustered scan chain groups to the sub blocks
- The DFTMAX Ultra codec sub blocks are placed near the physically clustered scan chains

Configuring DFTMAX Ultra Compression

To enable and configure DFTMAX Ultra compression, do the following:

1. Enable compression with the following command:

```
fc_shell> set_dft_configuration -scan_compression enable
```

2. Configure streaming compression by using the following command:

```
set_scan_compression_configuration
-streaming true
-inputs num_scanins
-outputs num_scanouts
-chain_count num_chains
  | -max_length chain_length
-clock codec_clock
[-test_mode mode_name]
```

The `-streaming` option indicates a DFTMAX Ultra streaming compression specification.

The `-inputs` and `-outputs` options are required. There must be enough scan-in and scan-out signals defined to satisfy the specification.

The `-chain_count` and `-max_length` options are mutually exclusive; one is required.

The `-clock` option specifies which clock to use for the streaming codec. By default, the tool selects clocks for the decompressor and compressor that minimize the number of lock-up latches at the decompressor outputs and compressor inputs. The selection rules are as follows:

- The decompressor uses a clock whose leading edge is as late or later than all head scan elements.
- The compressor uses a clock whose leading edge is as early or earlier than all tail scan elements.
- If multiple clocks meet these criteria for the decompressor or compressor, the dominant clock across the head or tail scan elements is used, respectively.

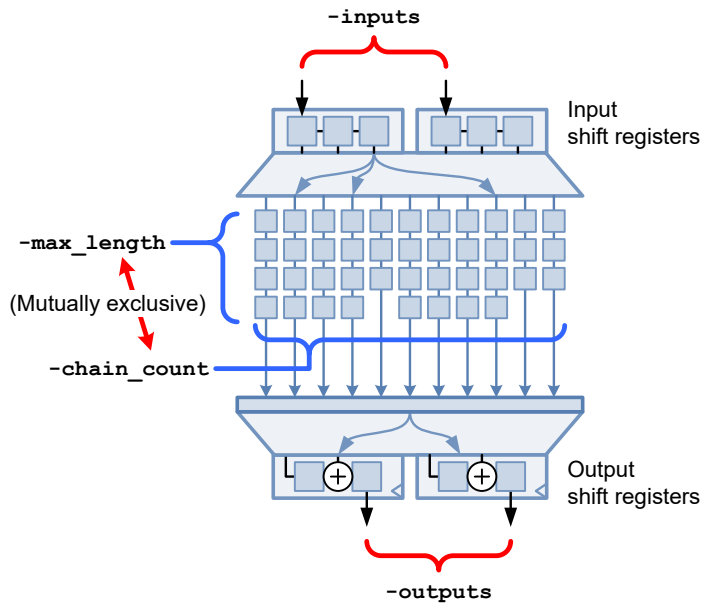
By default, the command creates and configures a mode named `ScanCompression_mode`. If you are using user-defined test modes, you specify the mode to configure with the `-test_mode` option.

3. Prevent QN pins of registers from being used by the DFTMAX Ultra IP:

```
fc_shell> set_app_options -name compile.seqmap.enable_output_inversion  
-value false
```

4. If the head or tail register of compressed chains are trailing edge triggered, the tool inserts retiming registers at the beginning or end of compressed chains.
5. You can specify the application option `set_app_options -name dft.test_ultra_put_load_ser_lul -value true` to insert lockup latches between shift registers of decompressor and the head registers of compressed chains which are leading edge triggered.
6. You can specify the application option `set_app_options -name dft.put_negedge_head_pipeline_flops -value true` for the shift register of decompressor to be trailing edge triggered.

The configuration options apply to the compression architecture as follows:



DFTMAX Ultra compression automatically computes the input and output shift register lengths that are optimal for the number of scan inputs and outputs and the number of compressed scan chains.

If you are also implementing a standard scan mode, keep in mind that these compressed chains become the base chains used to create other modes.

Using OCC Controllers With DFTMAX Ultra Compression

To define an external clock chain, use the `-scan_data_in` and `-scan_data_out` options of the `set_scan_path` to specify the input and output ports of the external chain:

```
set_dft_clock_controller -cell_name BLK/OCC ...
set_scan_path OCC_CHAIN -class occ \
  -include_elements {BLK/OCC} \
  -scan_data_in SI_OCC -scan_data_out SO_OCC
```

For details on defining external clock chains, see [Configuring the Clock Chain](#).

6

DFT Partitions

In some cases, you might want to apply different DFT configuration to different parts of the design. For example, you might want to use different scan-enable signals for different blocks, or you might want to enable clock-mixing for some blocks but not others.

You can use *DFT partitions* to do this. They allow you to divide up your design logic into multiple partitions, then you apply DFT configuration commands to each partition.

The following topics describe how to use DFT partitions:

- [Defining DFT Partitions](#)
- [Configuring DFT Partitions](#)
- [Per-Partition Scan Configuration Commands](#)
- [Known Issues of the DFT Partition Flow](#)

Defining DFT Partitions

To use DFT partitions, you must first define them. You can use the following commands to define and manage partition definitions in your top-level run:

- `define_dft_partition`
- `report_dft_partition`
- `remove_dft_partition`

You can use the `define_dft_partition` command to define a partition. The most commonly used options are:

```
define_dft_partition
  partition_name
  [-include list_of_cells_or_references]
  [-clocks list_of_clocks]
```

You must provide a unique name for each partition definition. This name is used to reference the partition when providing codec information, as well as to identify the partition in subsequent DFT reports.

A partition definition can include design references, hierarchical cells, scan cells, or clock domains. Although partitions are usually defined along physical or logical hierarchy boundaries, it is not a requirement.

To specify a set of cells, design references, or core scan segments, use the `-include` option. Leaf cells can be specified, although only sequential cells are relevant to the partition definition. Design references are converted to the set of all hierarchical instances of those designs. A particular cell or design reference can exist in only one partition definition. In [Example 6](#), two partitions are defined using hierarchical cells, and a third partition is defined using a design reference.

Example 6 Defining Three Partitions Using Cells and References

```
define_dft_partition P1 -include [get_cells {U_SMALL_BLK1 U_SMALL_BLK2}]
define_dft_partition P2 -include [get_cells {U_BIG_BLK}]
define_dft_partition P3 -include [get_references {my_ip_design}]
define_dft_partition P1 -clocks {CLK1 CLK2}
define_dft_partition P2 -clocks {CLK3}
```

The tool creates a default partition named `default_partition` that includes the flip-flops not included in any user-defined partitions. You cannot use the name `default_partition` when defining a partition.

You can use the `report_dft -partitions` command to see what partitions have been defined. [Example 7](#) shows the output for the three partitions previously defined in [Example 6](#). Note that the design reference has been converted to the corresponding set of hierarchical instances of that design.

Example 7 Example of Output From the `report_dft -partitions` Command

Cell Name	Partition Name
U_SMALL_BLK1/	P1
U_SMALL_BLK2/	P1
U_BIG_BLK/	P2
U_MY_IP_BLK/	P3

Configuring DFT Partitions

After the DFT partitions are defined, you can configure the scan configuration for each partition. Use the `current_dft_partition` command to set the current partition, then apply one or more supported test configuration commands to configure scan for that partition.

All DFT partitions share a common global configuration. Partition-specific configuration commands are applied incrementally on top of the global configuration.

In a DFT partition flow, the sequence of configuration commands is:

- Apply global DFT configuration settings
- Define DFT partitions with `define_dft_partition`
- Apply partition-specific DFT configuration settings to each partition with `current_dft_partition`

[Example 8](#) shows an example of global and partition-specific configuration commands.

Example 8 *Configuring Two DFT Partitions*

```
# apply global DFT configuration settings
set_scan_configuration -clock_mixing mix_clocks
set_dft_signal -view existing_dft \
    -type ScanClock -timing [list 45 55] -port CLK
set_dft_signal -view spec -type TestMode -port TM

# define DFT partitions
define_dft_partition P1 -include {BLK1}
define_dft_partition P2 -include {BLK2}

# configure DFT partition P1
current_dft_partition P1
set_dft_signal -view spec -type ScanEnable -port SE1
set_dft_signal -view spec -type ScanDataIn -port {SI1 SI2}
set_dft_signal -view spec -type ScanDataOut -port {SO1 SO2}
set_scan_configuration -chain_count 2

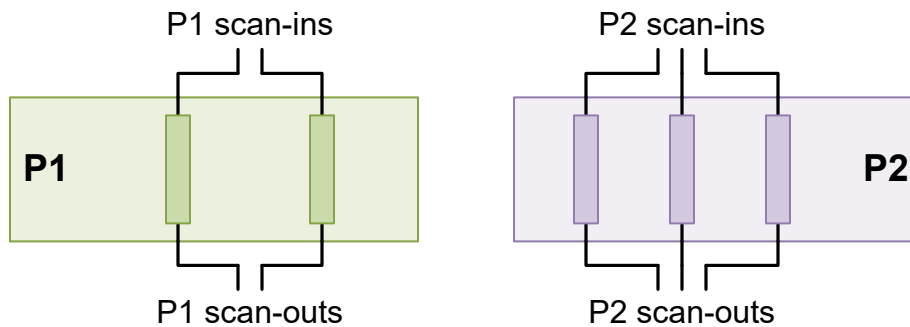
# configure DFT partition P2
current_dft_partition P2
set_dft_signal -view spec -type ScanEnable -port SE2
set_dft_signal -view spec -type ScanDataIn -port {SI3 SI4 SI5}
set_dft_signal -view spec -type ScanDataOut -port {SO3 SO4 SO5}
set_scan_configuration -chain_count 3
```

If you are defining scan-in or scan-out signals using the `set_dft_signal` command, you must define them as a part of each partition's scan configuration. Scan-enable signals can be defined globally or on a per-partition basis. However, if a scan-enable signal is defined for only one partition, it is automatically applied to the remaining partitions. Other signal types, such as reset, clock, and test-mode signals, must be defined globally before any partitions are defined, or as part of the default partition configuration.

For the entire design, the total scan chain count is the sum of the scan chain counts across all partitions. Each scan chain requires its own scan-in and scan-out pin pair, just as a scan chain does in an unpartitioned flow. Scan chains are not combined or rebalanced across the partitions.

[Figure 49](#) shows the scan mode chain connections for [Example 8](#). A total of five scan-in and scan-out pins are used, two for partition P1 and three for partition P2.

Figure 49 DFT Partition Scan Chains



After configuring the partitions, you can use the `preview_dft` command to display the scan chain distribution across partitions. [Example 9](#) shows a report example.

Example 9 Output From the `preview_dft` Command for Multiple DFT Partitions

```
*****
Current mode: Internal_scan
*****
Number of chains: 5
Scan methodology: full_scan
Scan style: multiplexed_flip_flop
Clock domain: no_mix
Scan chain '1' (SI1 --> SO1) contains 32 cells (Partition 'P1')
Active in modes: Internal_scan
Scan chain '2' (SI2 --> SO2) contains 32 cells (Partition 'P1')
Active in modes: Internal_scan
Scan chain '3' (SI3 --> SO3) contains 22 cells (Partition 'P2')
Active in modes: Internal_scan
Scan chain '4' (SI4 --> SO4) contains 21 cells (Partition 'P2')
Active in modes: Internal_scan
Scan chain '5' (SI5 --> SO5) contains 21 cells (Partition 'P2')
Active in modes: Internal_scan
```

Per-Partition Scan Configuration Commands

This topic lists the commands you can use to configure DFT insertion on a per-partition basis. Commands not listed in this section should be applied as part of the global DFT configuration.

set_scan_configuration

The following `set_scan_configuration` options can be specified on a per-partition basis:

- `-chain_count`
- `-max_length`
- `-clock_mixing`
- `-insert_terminal_lockup`
- `-test_mode`
- `-exclude_elements`

set_dft_signal

The following `set_dft_signal` options can be specified on a per-partition basis:

- `-view`
- `-type ScanDataIn | ScanDataOut | ScanEnable`
- `-port`
- `-hookup_pin`
- `-hookup_sense`
- `-active_state`

For other test logic types, you can only specify the location for the default partition.

set_scan_path

The following `set_scan_path` options can be specified on a per-partition basis:

- `-include_elements`
- `-ordered_elements`
- `-complete`
- `-scan_enable`
- `-scan_data_in`
- `-scan_data_out`

set_wrapper_configuration

The following `set_wrapper_configuration` options can be specified on a per-partition basis:

- `-chain_count`
- `-mix_cells`

Known Issues of the DFT Partition Flow

The following known issues apply to the partition flow:

- If you define a scan-enable signal for a partition, that scan-enable signal is reused for other partitions for which the signal has not been defined. This reuse avoids having to create new scan-enable signals.
- The `define_dft_partition` command does not perform duplicate object checking between cell instances and design modules. The overlapping objects will be included only in one of the partitions.
- If you apply commands or options that do not support per-partition specification to a DFT partition, they are ignored with no warning.

7

Pipelined Scan Data

You can use the pipelined scan data feature to resolve delay problems associated with long routes in compressed scan chain logic.

The following topics provide more detail:

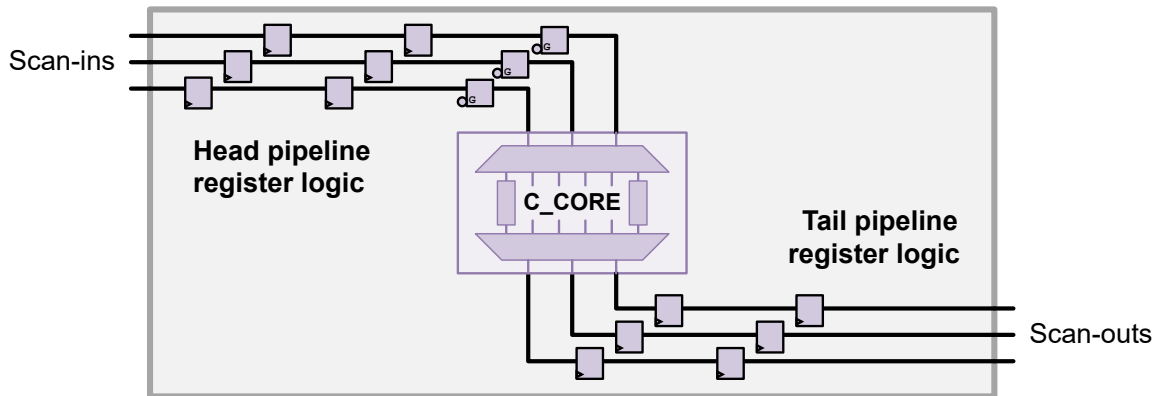
- [Introduction to Pipelined Scan Data](#)
- [The Pipelined Scan Data Logic Structure](#)
- [Configuring Pipelined Scan Data](#)
- [Pipelined Scan Data Test Protocol Format](#)
- [Pipelined Scan Data Limitations](#)

Introduction to Pipelined Scan Data

In typical scan flows, long wires between the scan chain input and the first flip-flop and between the last flip-flop and the scan chain output can cause delay problems. Scan compression logic and higher scan frequencies further amplify the problem. These delays are reduced by placing pipeline registers at the beginning and end of the scan chains. They divide the long routes between the scan chain terminals into smaller wires between the registers and therefore help reduce the path delay.

[Figure 50](#) shows an example of a compressed scan design with pipeline registers. The head pipeline registers are placed between the scan inputs and the decompressor, and the tail pipeline registers are placed between the compressor and the scan outputs.

Figure 50 Head and Tail Pipeline Register Logic



RTL DFT insertion adds these pipeline registers to the scan paths. Synthesis places them along the scan paths using physical information. Post-DFT DRC verifies their correct operation.

The Pipelined Scan Data Logic Structure

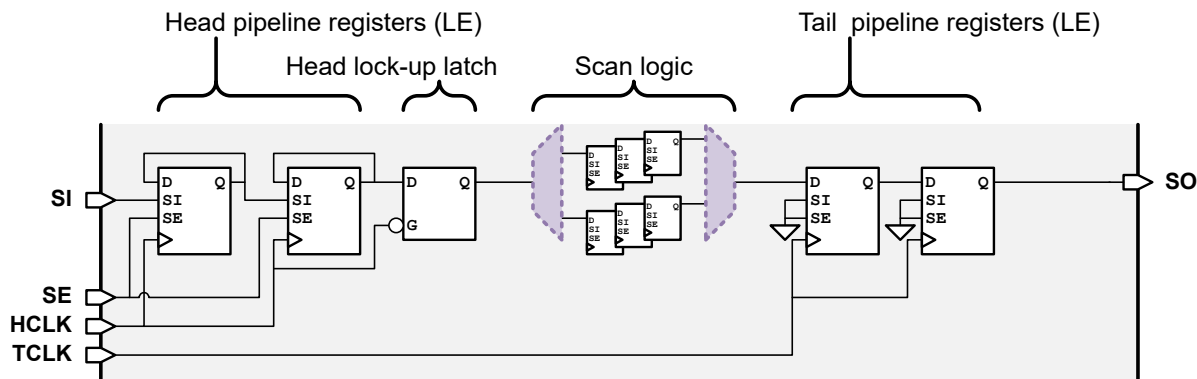
The head pipeline registers are implemented with state-holding scan cells. This prevents unknown X values from being captured, which would then get propagated through the decompressor into the compressed chains.

The tail pipeline registers are implemented using nonscan cells for minimal area. (There is no state-holding requirement.)

At the end of each head, a lockup latch is inserted to provide additional timing margin. The head lockup latches exist outside the codec, and thus serve all fanout registers (through reconfiguration MUXs, decompressor logic, etc.).

To insert terminal lockup between the tail pipeline registers and scan output ports, use the following application options `set_app_options -name dft.scan.enable_general_multimode_support -value true` and `set_scan_configuration -insert_terminal_lockup true -test_mode all_dft`.

Figure 51 Pipeline Registers (Depth == 2) in a Compressed Scan Design



Post-DFT DRC verifies whether the head pipeline registers hold their values during capture and issues an R-18 violation message if the check is unsuccessful.

Configuring Pipelined Scan Data

To configure pipelined scan data, do the following:

1. Enable the pipelined scan data feature (optional):

```
fc_shell> set_dft_configuration -pipeline_scan_data enable
```

2. Specify the head and tail clock and pipeline depths using the following options:

```
fc_shell> set_pipeline_scan_data_configuration \
    -head_pipeline_clock clock_name \
    -tail_pipeline_clock clock_name \
    -head_pipeline_stages integer \
    -tail_pipeline_stages integer
```

All options are required.

After RTL DFT insertion, the newly inserted pipeline registers and lock-up latches have names of the form

```
DFT_block/U_dft_head_p/pipe_stage_reg[signum]
DFT_block/U_dft_head_p/lul_out_reg[signum]
...
DFT_block/U_dft_tail_p/pipe_stage_reg[signum]
DFT_block/U_dft_head_p/lul_out_reg[signum]
```

where *DFT_block* is the DFT logic block name, *stage* is the stage depth, and *signum* is the ordinal number of the scan-in or scan-out signal.

You can use the `report_pipeline_scan_data_configuration` or `report_dft -pipeline` commands to report the current pipelined scan data configuration.

Note:

To use falling-edge triggered registers as head pipeline registers, specify the application option `set_app_options -name dft.put_negedge_head_pipeline_flops -value true`. Furthermore, the serial registers of DFTMAX ultra decompressor are implemented using falling-edge triggered registers when the application option is specified.

Pipelined Scan Data Test Protocol Format

The test protocol file generated for uncompressed scan modes does not contain any pipeline register information, as it is not needed for these modes.

The test protocol file generated for compressed scan modes does contain information about the pipeline registers. The test protocol file has the following additional information:

- The number of head pipeline stages is indicated by the `LoadPipelineStages` keyword.
- The number of tail pipeline stages is indicated by the `UnloadPipelineStages` keyword.

For example,

```
CompressorStructures {  
    Compressor "U_decompressor_ScanCompression_mode" {  
        LoadPipelineStages 2;  
        ModeGroup mode_group;  
        LoadGroup load_group;  
        CoreGroup core_group;  
    ... }  
    Compressor "U_compressor_ScanCompression_mode" {  
        UnloadPipelineStages 2;  
        ModeGroup mode_group;  
        UnloadGroup unload_group;  
        UnloadModeGroup unload_mode_group0 unload_mode_group1;  
    ... } }  
}
```

Pipelined Scan Data Limitations

These are the requirements and limitations for implementing pipeline registers:

- The tool does not insert pipeline scan data registers to external scan chains created by the `set_scan_path` command. However, external clock chains are pipelined.
- Return-to-one clocks are not supported. (The pipeline registers are not inserted with the correct polarity.)
- Designs without scan compression are not supported.

- When scan clocks of differing timing waveforms are used, the timing relationship between the head pipeline registers, scan chains, and tail pipeline registers is not checked and must be correct by construction.
- If you are using external (uncompressed) chains, such as external clock chains or other user-defined external chains, their pipeline depths must match other scan chains. For more information, see [Excluding Scan Chains From Scan Compression](#).
- When using internal pins for the scan-in and scan-out signals, any combinational logic between the scan ports and the pipeline registers must be sensitized to a known state.
- An ATE clock defined with the `set_dft_signal -type AteClock` command cannot be used to directly clock either the head or tail pipelined scan data registers. If the DFT-inserted OCC controller flow is used together with pipeline register insertion, specification of the ATE clock as the pipeline clock results in the OCC-controlled clock being used by default. If the ATE clock is manually connected to the pipeline registers, no violations are reported, but incorrect ATPG patterns might be generated.

8

Pipelined Scan-Enable Signals

You can use pipelined scan-enable signals to provide launch-on-extra-shift (LOES) transition-delay timing in TestMAX ATPG, which improves ATPG efficiency and reduces pattern count.

This is described in the following topics:

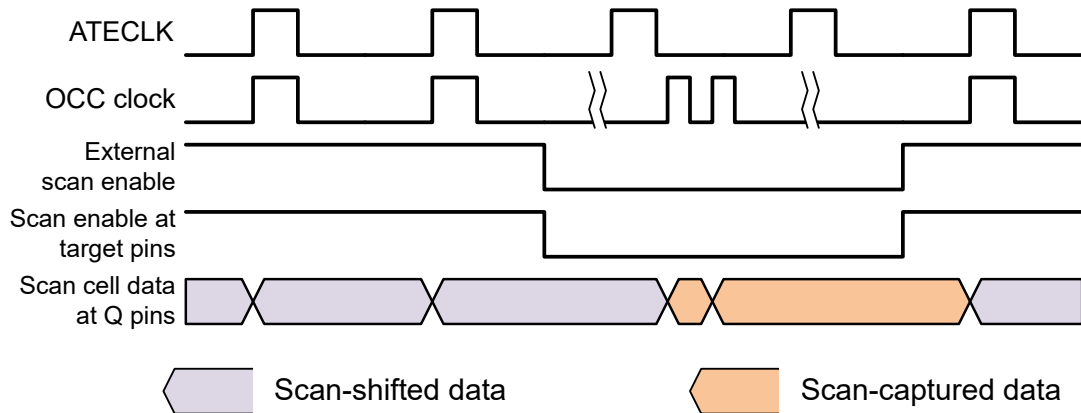
- [The Pipelined Scan-Enable Architecture](#)
- [Pipelined Scan-Enable Requirements](#)
- [Excluding Elements From Pipelined Scan-Enable Configurations](#)
- [Implementing Pipelined Scan-Enable Signals](#)
- [Scannable Pipelined Scan-Enable Registers](#)
- [Reporting the Pipelined Scan-Enable Clusters](#)
- [Implementation Considerations for Pipelined Scan-Enable Signals](#)
- [Pipelined Scan Enable Limitations](#)
- [PSE max Fanout per Clock Specification](#)
- [Adaptive Pipelined Scan-Enable Reclustering](#)

The Pipelined Scan-Enable Architecture

Transition-delay fault testing requires two at-speed clock cycles in a row—one to launch and one to capture—so that the launched data must propagate through the logic cones at-speed to be captured properly. These at-speed clock pulses are typically provided by an on-chip clocking (OCC) controller, as described in [Chapter 13, On-Chip Clocking \(OCC\)](#).

By default, Fusion Compiler implements a simple scan-enable signal, which is externally controlled at test clock frequencies. This requires the use of Launch-on-Capture (LOC) transition-delay timing, shown in [Figure 52](#), where scan enable is de-asserted *before* both the launch and capture clock pulses.

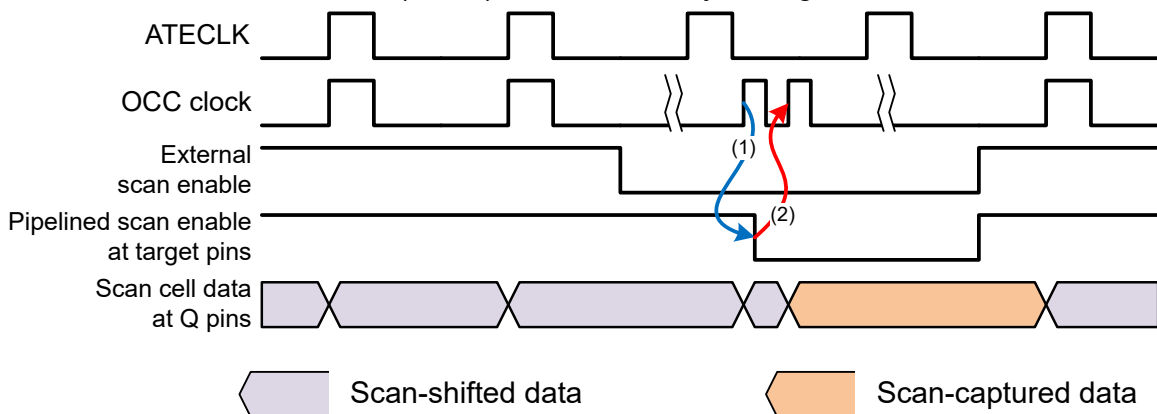
Figure 52 Launch-on-Capture (LOC) Transition-Delay Timing



Because scan enable is de-asserted for the launch clock pulse, the launch data must be controlled indirectly (captured through the logic cones) by the data from the previous scan-shift cycle. This requires fast-sequential ATPG, which imposes additional constraints and overhead on ATPG.

To avoid this limitation, you can implement *pipelined scan-enable signals*, which can generate the scan-enable transition at-speed. This allows launch-on-extra-shift (LOES) transition-delay timing, as shown in Figure 53, where TestMAX ATPG directly controls the launch data.

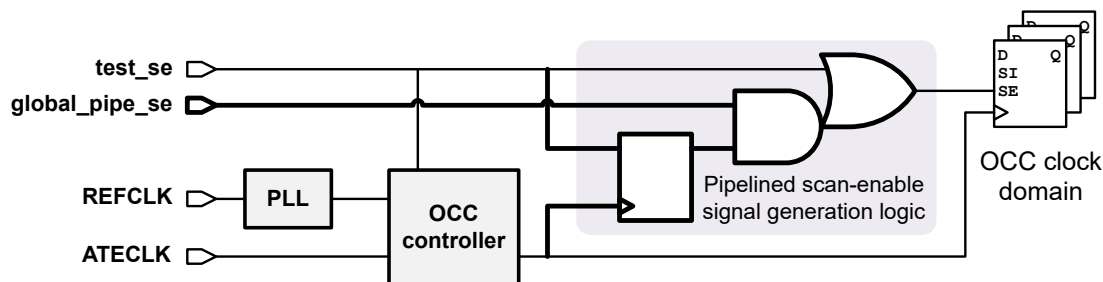
Figure 53 Launch-On-Extra-Shift (LOES) Transition-Delay Timing



The waveforms for the external signals are identical between LOC and LOES, but the scan-enable signal that reaches the scan cells is different.

The pipelined scan-enable feature works by creating a registered scan-enable transition on the chip, just as an OCC controller generates controlled at-speed clock pulses on the chip. Figure 54 shows the pipelined scan-enable logic structure.

Figure 54 Pipelined Scan-Enable Logic



The `global_pipe_se` signal controls whether the output scan-enable signal operates in nonpipelined or pipelined mode:

- When the `global_pipe_se` signal is de-asserted, the register is bypassed and the output scan-enable signal is a simple nonpipelined scan-enable signal.
- When the `global_pipe_se` signal is asserted,
 - When the input scan-enable signal is asserted, the output scan-enable signal is immediately asserted.
 - When the input scan-enable signal is de-asserted, the register holds the output scan-enable asserted until the next leading clock edge of the *at-speed* clock.

Each clock domain and clock edge has its own pipelined scan-enable logic construct that de-asserts the scan-enable signal synchronized to that clock edge.

The pipelined scan-enable feature and the pipelined scan data feature are independent and unrelated. Either can be used separately, or they can be used together, but the commands, limitations, and messages are specific to one or the other and must not be confused.

See also “Transition-Delay Fault ATPG Timing Modes” in TestMAX ATPG and TestMAX Diagnosis Online Help for more information about launch-on-capture (LOC) and launch-on-extra-shift (LOES) timing.

Pipelined Scan-Enable Requirements

Launch-on-extra-shift (LOES) with pipelined scan-enable signals has the following requirements:

- All scan-enable signals must have the same pipelined scan-enable depth after DFT insertion. Valid depths are zero (not pipelined) and one (pipelined).
- Each clock domain and clock edge must have its own pipelined scan-enable signal, which all scan flip-flops clocked by that domain and edge must use.

- Clock-gating cells should also use the pipelined scan enable for clock domains and edges.

It is possible to use LOES when clock-gating cells use the nonpipelined scan enable. However, you should use the pipelined scan enable because it maximizes the amount of data that can be launched in the extra shift, which usually results in a smaller pattern set. It also allows the use of the legacy ATPG method Launch-On-Last-Shift, where the last scan shift doubles as the transition launch cycle (which might be important in the case of design reuse).

- The nonpipelined scan enable must be used for OCC controllers. This allows the OCC controller to generate the extra-shift launch clock followed by the capture clock as an at-speed clock pair.
- The nonpipelined scan enable must be used for clock chains used by OCC controllers. This holds the clock chain data steady during capture. If the pipelined scan enable is used, the clock chain bits are corrupted by the launch clock.
- When clock mixing is used, lock-up latches might be ineffective because the clock timing in the extra shift uses the launch waveform table rather than shift timing. As a result, DRC in the TestMAX ATPG tool marks the first flip-flop following the clock domain crossing as disturbed, which might reduce coverage slightly (although an incremental LOC run can detect these faults).
- All on-chip scan-enable signals (nonpipelined and pipelined for each clock edge) must be derived from a single scan-enable signal source.

Excluding Elements From Pipelined Scan-Enable Configurations

When you enable the pipelined scan enable feature, all scan-enable connections are subject to pipelining behavior. However, in some designs, you might want to exclude specific scan elements from pipelining. You can exclude specific elements or entire classes of elements.

When a scan-enable connection is excluded, the tool routes that connection as if the pipelined scan-enable is not enabled. In other words, the tool tries to connect directly from the port or hookup pin to the scan cell's scan-enable pin.

To exclude specific elements, use the following command:

```
fc_shell> set_pipeline_scan_enable_configuration \
    -exclude element_list
```

The argument list can contain any of the following elements:

- Instance name

The tool excludes any scan-enable connection to that cell.

- Pin

The tool excludes only that pin, which must be a scan-enable pin. If the cell is not mapped, the pin name might change when the tool maps the cell. To avoid this, exclude the entire register instead of the pin.

- Hierarchy

The tool excludes all scan-enable connections to instances contained in that hierarchy and its child hierarchies.

- Scan clock port

The tool excludes all elements that belong to that clock domain.

- Scan clock hookup port

The tool excludes all elements that belong to that clock domain.

If multiple clock pins share the same scan enable, all of the clocks must share the same configuration, either pipelined or not pipelined.

To exclude entire classes of elements, use the following command:

```
fc_shell> set_pipeline_scan_enable_configuration \  
-exclude_types category_list
```

The argument list can contain any of the following categories:

- shared_wrapper
- dedicated_wrapper
- input_wrapper
- output_wrapper
- input_shared_wrapper
- output_shared_wrapper
- input_dedicated_wrapper
- output_dedicated_wrapper
- testpoint
- clock_gate
- repeater

Implementing Pipelined Scan-Enable Signals

To implement pipelined scan-enable signals, do the following:

1. Enable the pipelined scan-enable feature:

```
fc_shell> set_scan_configuration -pipeline_scan_enable true
```

2. Define the enable signal:

```
fc_shell> set_dft_signal -view spec -type LOSPipelineEnable \  
-port PSE_EN -active_state 1 -test_mode all
```

This enable signal selects launch-on-extra-shift (LOES) when asserted and Launch-on-Capture (LOC) operation when de-asserted.

You can drive the enable signal from an on-chip configuration register by using the internal pins flow. For more information, see [Chapter 11, The Internal Pins Flow](#).

3. Specify the maximum number of scan cells to be driven by each pipeline register:

```
fc_shell> set_scan_configuration -pipeline_fanout_limit 64
```

4. (Optional) Exclude specific elements from implementation:

```
fc_shell> set_pipeline_scan_enable_configuration \  
-exclude element_list
```

5. (Optional) Exclude specific categories of scan chain elements from implementation:

```
fc_shell> set_pipeline_scan_enable_configuration \  
-exclude_types category_list
```

The tool creates multiple physically compact clusters of scan cells, each with their own local pipelined scan-enable (PSE) construct. This improves the critical path delay between the pipeline registers and their scan cells.

The tool creates as many clusters as needed, each with exactly the specified number of scan cells. (The last-created cluster in each clock domain might have fewer scan cells.) The pipeline registers have the `size_only` attribute set, which prevents duplicate register merging by the `compile_fusion` command.

See [Implementation Considerations for Pipelined Scan-Enable Signals](#) for details on clustering and timing.

The default is not to apply a fanout limit.

If you are using a DFT-inserted OCC controller, the clock connection might be incorrect. See [Pipelined Scan Enable Limitations](#) for details.

The test protocol created by the tool does not constrain the pipeline enable signal, so post-DFT DRC checks both the LOC and LOES modes. The protocol written out by the `write_test_protocol` command also does not constrain the enable signal. Use the `add_pi_constraint` command in TestMAX ATPG to assert or de-assert the pipeline enable signal. See “Using Launch-On Extra-Shift Timing” in TestMAX ATPG and TestMAX Diagnosis Online Help for more information.

Scannable Pipelined Scan-Enable Registers

The Pipelined non-scan registers inserted during `insert_dft` can be converted into scan registers to improve the test coverage after DFT insertion, improve pattern counts during ATPG, and enable finer control on the Launch-on-Capture (LOC) logic.

This feature enables you to modify the top-level PSE registers inserted during `insert_dft`, which are scannable and stitched into top-level scan chains. These scan PSE registers need to be stitched into scan chains, without any restriction other than not having them all stitched together in one chain or in sequential order. That is, they should be spread out between chains and within the chain.

To enable this feature set the `set_pipeline_scan_enable_configuration -scan_pse_registers true` command which has a `false` by default. The `report_pipeline_scan_enable_configuration` command reports the value set for the application option.

```
fc_shell> report_pipeline_scan_enable_configuration
*****
Report : Pipeline Scan Enable Configuration
Design : TOP
Version: W-2024.09
Date   : Wed Jun  5 16:03:29 2024
*****
Scan PSE registers: True
```

When you have enabled the scannable PSE registers during the DFT insertion, the tool prints the following information message:

```
Information: Scannable PSE registers feature is enabled.
PSE registers will be stitched into chains. (DFT-3893)
```

Limitations

When scannable PSE registers are used, the tool falls back to the legacy PSE reclustering. Adaptive PSE reclustering can add or remove the PSE cells. Scannable

PSE registers are stitched to the scan chains, therefore incompatible with adaptive PSE reclustering. The tool prints the following information message:

```
Information: The design has PSE registers stitched into scan chains.
Adaptive PSE reclustering cannot be used, and Legacy PSE reclustering
will be used instead. (DFT-3892)
```

Reporting the Pipelined Scan-Enable Clusters

When the pipelined scan-enable feature is enabled, the `insert_dft` command creates a `scan_dft.rpt` file that reports the cluster information:

```
=====
Pipelining Scan Enable Report :
=====
Number of pipelining levels : 1
-----
Clock/Scan Enable      Stage 1      Leaf flops/elems
-----+-----+-----
"pll_1/CLK_4X"/SE      |      PSE1179 | dd_decrypt/ein_reg
                        |              | dd_decrypt/kdin_reg[107]
                        |              | dd_decrypt/kdin_reg[111]
...                      |              |
                        |              | dd_a/o_tval_reg
                        |              | dd_a/state_reg
Global pipeline scan enable signal: ctmi_1183/Y
-----
Number of pipelining levels : 1
Number of Flops driven per stage: 48
-----
Clock/Scan Enable      Stage 1      Leaf flops/elems
-----+-----+-----
"pll_1/CLK_4X"/SE      |      PSE1184 | dd_decrypt/kdin_reg[100]
                        |              | dd_decrypt/kdin_reg[101]
                        |              | dd_decrypt/kdin_reg[103]
...                      |              |
                        |              | dd_b/o_data_reg[7]
                        |              | dd_b/o_data_reg[8]
Global pipeline scan enable signal: ctmi_1188/Y
-----
```

Implementation Considerations for Pipelined Scan-Enable Signals

Note the following considerations when implementing pipelined scan-enable signals.

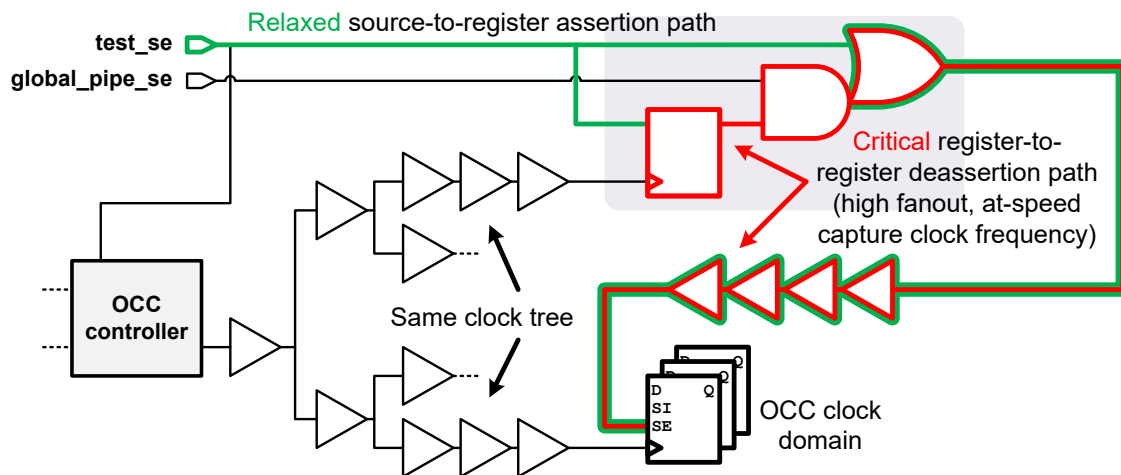
Timing Considerations

For a regular scan-enable signal, the signal path is a single-cycle path at the shift clock frequency, which is likely to be fairly slow—and even that requirement can be relaxed by adding extra cycles in TestMAX ATPG using the `-use_delay_capture_start` and `-use_delay_capture_end` options of the `write_patterns` command.

For a pipelined scan-enable signal, this relaxed timing applies only to the external signal source. The de-assertion event from the pipeline register must reach *all* downstream flip-flop scan-enable pins in a single cycle at the capture clock frequency, which is much faster than shift. This path cannot be relaxed because a multicycle path would result in bad patterns, and slowing down the clock makes the test less-than-at-speed.

Figure 55 shows the relaxed externally driven assertion path in green and the critical register-driven de-assertion path in red.

Figure 55 Pipelined Scan-Enable Pipeline Register Timing Path



The pipeline register and its fanout scan cells are all typically driven by the same skew-balanced clock tree. As a result, the scan-enable buffer tree paths are standard register-to-register timing paths, with allowable delays between nearly zero and almost a full clock cycle. The buffer tree does not need to be skew-balanced.

In addition,

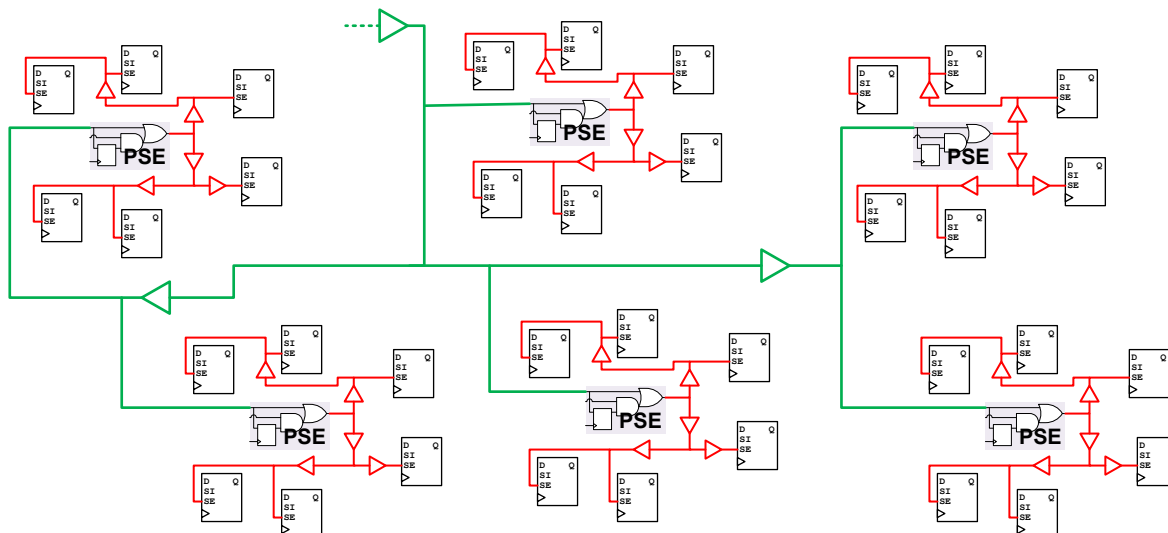
- For high-frequency clocks driving large clock domains, the buffer tree delay might be greater than the clock period. In this case, you can move the pipeline register's clock connection earlier in the clock tree, as long as hold constraints are met in all operating conditions.

Physical Implementation Considerations

In Fusion Compiler, the tool creates physically aware buffer trees on the pipelined scan-enable nets to avoid synthesis design rule violations.

If you set a fanout limit to implement pipelined scan-enable clusters, the tool analyzes scan cell locations to create compact clusters. Then, it builds physically aware buffer trees to all cluster pipeline registers (green) and within each cluster from the pipeline register to the scan cells (red), as shown in Figure 56.

Figure 56 Pipelined Scan-Enable Signal With Fanout-Limited Scan Cell Clusters



Pipelined Scan Enable Limitations

Note the following requirement and limitation of pipelined scan-enable signals:

When using DFT-inserted OCC controllers, the clock connection to the pipeline scan-enable registers might be wrong.

This usually means that the tool uses the clock coming from the uncontrolled PLL source. In this case, TestMAX ATPG will generate patterns, but those patterns will fail Verilog simulation.

The correct clock connection to the pipeline scan enable register is the output of the OCC controller, at the leaf level of the clock tree. If the register is in a module with scan flip-flops that it controls, the same clock signal that drives the scan flip-flops should also drive the pipeline scan-enable register.

The register instance names begin with “PSE”. There might be more than one for each clock, and they might be buried in the design hierarchy, depending on the settings.

PSE max Fanout per Clock Specification

The Pipeline Scan-Enable (PSE) logic is inserted during DFT scan insertion to provide at-speed transition of scan enable signals to allow launch-on-extra-shift (LOES) transition-delay timing.

To enable PSE max fanout per clock specification, use the `set_pipeline_scan_enable_configuration-clock_based_fanout_limit` command.

You must define the new input using the `set_pipeline_scan_enable_configuration` command before calling `preview_dft` or `insert_dft`.

For example, `set_pipeline_scan_enable_configuration`
`-clock_based_fanout_limit {clock1 100 clock2 500}`

In the preceding command, PSE groups driven by `clock1` have a max fanout limit of 100, whereas PSE groups driven by `clock2` have a fanout of 500. If other PSE groups driven by different clocks are not affected by these two values, then they must use the global PSE fanout limit, if defined.

The clocks used in the command must be defined as scan clocks. The valid scan clocks are ports or hookup pins.

For a scan clock specification with both `port` and `hookup_pin` options, the fanout limit for `hookup_pin` gets precedence if different values are specified for the options. In the following example, for all the PSE groups, a max fanout limit of 100 is applied, as `hookup_pin` gets precedence.

```
set_dft_signal -type ScanClock -port clk -hookup_pin U1/clkout  
  
set_pipeline_scan_enable_configuration -clock_based_fanout_limit {U1/  
clkout 100 clk 500 }
```

For the clocks that do not have PSE max fanout specified, specify the value from the global PSE max fanout specification using the `set_scan_configuration` `-pse_fanout_limit` command. If a specific value for the PSE fanout limit is set for a clock, it overrides the global PSE fanout limit, if any.

To report the per-clock fanout specifications, use the `report_pipeline_scan_enable_configuration` command.

Limitations

The PSE max fanout per clock specification feature has the following limitations:

- Synopsys-inserted OCC output pins cannot be specified because they do not exist at the time of specification.
- For Synopsys OCC flows:
 - Specifying the fanout limit is allowed on PLL clocks. The PLL clock fanout specification translates to the associated OCC output pins.
 - Specifying the fanout limit is not allowed on a port defined as an ATE clock. You can use the global specification, `set_scan_configuration -pse_fanout_limit`.
- The fanout limit specification is not stored in the NDM database. You must specify the fanout limit in the same session of the `insert_dft` step.

Adaptive Pipelined Scan-Enable Reclustering

The Fusion Compiler tool creates physically-close clusters of Pipelined Scan Enable(PSE) logic and their loads during `insert_dft`. Further, the placement steps can move the loads and disturb the close-proximity of PSE drivers and the respective loads. This feature improves PSE clustering and reduces the PSE wire length.

The PSE reclustering runs as part of each of the `optimize_dft` sub-step in the steps of `compile_fusion` command, following the `insert_dft` command.

To improve the Quality of Results(QoR), a maximum box size for each PSE cluster can be specified. The PSE cluster includes the elements placed within the user-defined box size while still considering the value of the option `pipeline_fanout_limit`.

You can define the box size using the following application options:

- `set_app_options -name dft.scan.pse_cluster_box_x_size -value integer>`
- `set_app_options -name dft.scan.pse_cluster_box_y_size -value integer>`

Values are expressed on user length units, which can be queried using the command `get_user_units -input -type length`.

The default adaptive reclustering algorithm achieves tighter PSE clusters by creating new PSE cells and destroying old PSE cells when required. To disable the adaptive PSE reclustering algorithm, you can use the `set_app_options -name dft.scan.adaptive_pse_reclustering -value false` application option before the `insert_dft` command.

Note:

- When adaptive PSE reclustering algorithm is disabled, the tool uses the legacy reclustering algorithm that is unable to create new PSE cells or destroy the old PSE cells and might not always honor the PSE cluster box application options.
- When stitching PSE registers to scan chains feature is enabled by using the `set_pipeline_scan_enable_configuration -scan_pse_registers true` option, the tool automatically switches to the legacy PSE reclustering algorithm.

An example script is as follows:

```
compile_fusion -to logic_opto
DFT setup>
#pipelined scan enable setup
set_scan_configuration -pipeline_scan_enable true \
-pipeline_fanout_limit integer>
#specify the maximum box size of the PSE clusters
set_app_options -name dft.scan.pse_cluster_box_x_size -value integer>
set_app_options -name dft.scan.pse_cluster_box_y_size -value integer>
set_dft_signal -view spec -type LOSPipelineEnable -port string>
create_test_protocol
dft_drc -test_mode all_dft
insert_dft #initial PSE logic implementation happens here
compile_fusion -from initial_place -to initial_opto #PSE reclustering
happens here
```

9

Core Wrapping

This chapter shows you how to add a test wrapper to a design, which creates a *wrapped core*. A wrapped core provides both test access and test isolation during hierarchical testing.

The following topics provide more information:

- [Core Wrapping Concepts](#)
- [Wrapping a Core](#)
- [Defining Core Wrapping Test Modes](#)
- [Creating Compressed EXTEST Core Wrapping Test Modes](#)
- [Wrapping Cores With DFT Partitions](#)
- [Excluding Test Pins from Wrapping Across DFT Partitions](#)
- [Wrapping Cores With Multibit Registers](#)
- [Top-Down Flat Testing With Transparent Wrapped Cores](#)
- [SCANDEF Generation for Wrapper Chains](#)
- [Core Wrapping Scripts](#)
- [Limitations of Core Wrapping](#)

Core Wrapping Concepts

When you integrate a DFT-inserted core into a top-level design, the core-level scan structures are integrated into the top-level scan structures. However, to test the core-level logic separately from the top-level logic, the core must be a *wrapped core*.

The default core wrapping flow in Fusion Compiler is called a *maximized reuse* core wrapping flow because it reuses existing functional registers to minimize the area and timing impact.

Core wrapping is described in the following topics:

- [Wrapper Cells and Wrapper Chains](#)
- [Wrapper Test Modes](#)
- [Dedicated Core Wrapping Flow](#)
- [The Maximized Reuse Core Wrapping Flow](#)
- [Controlling Register Reuse](#)
- [Special Cases for Register Reuse](#)
- [Wrapper Shift Signals](#)
- [Dedicated Wrapper Cell Insertion in UPF Designs](#)
- [Optimizing UPF Aware Dedicated Core Wrapper Cell Insertion Near Core](#)

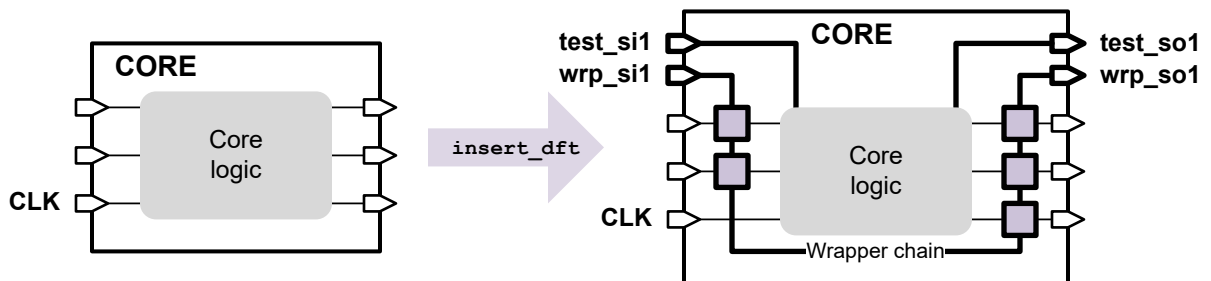
See Also

- [SolvNetPlus article 1918995, “How Do Wrapper Chains and Wrapper Cells Work in Detail?”](#) for additional reference information about wrapper chains and wrapper cells

Wrapper Cells and Wrapper Chains

A wrapped core has a *wrapper chain* that allows the core to be isolated from the surrounding logic. A wrapper chain is composed of *wrapper cells* inserted between the I/O ports and the core logic of the design. [Figure 57](#) shows an example of a wrapped core.

Figure 57 A Wrapped Core



A wrapper cell consists of a scan cell and MUX logic. It can transparently pass the I/O signal through, or it can capture values at its input and launch values at its output. Wrapper chains are shift chains (separate from regular scan chains) that allow known values to be scanned into the wrapper cells and captured values to be scanned out.

Core wrapping is primarily intended to wrap core data ports. The following ports are excluded from wrapping:

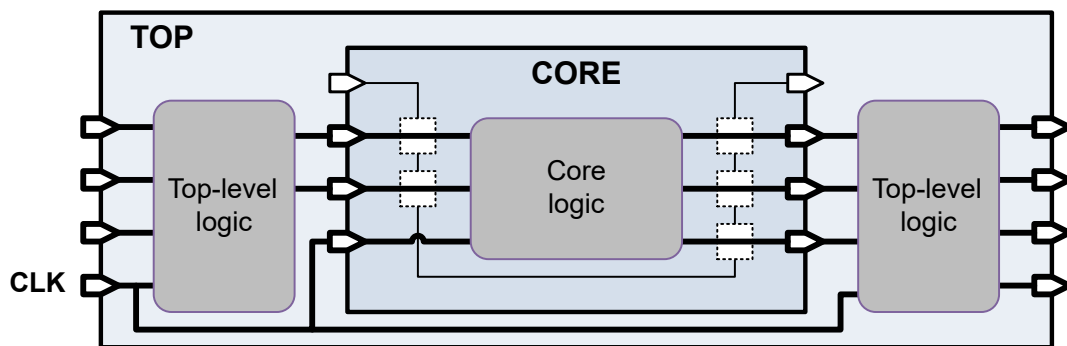
- Functional and test clock ports
- Asynchronous set or reset signal ports
- Scan-input, scan-output, scan-enable, and other global test signal ports
- Wrapper signal ports
- Any port with a constant test signal value defined

The wrapper chain operates in one of four modes—inactive, inward-facing, outward-facing, or safe. These wrapper operation modes behave as follows:

- **Inactive mode**

The wrapper chain is inactive and I/O signals pass through it. This is the behavior used in mission mode and all non-wrapper test modes.

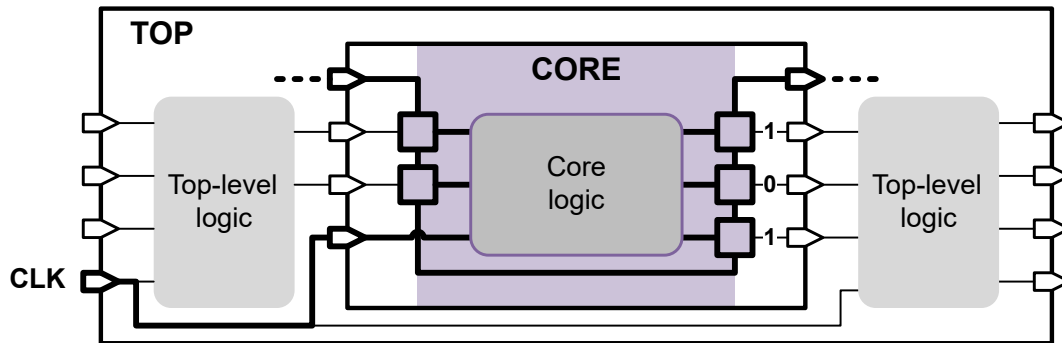
Figure 58 Inactive Mode of Wrapper Chain



- **Inward-facing mode (INTEST)**

This mode is used to test the core in isolation of the surrounding logic. It includes the wrapper chain and internal chains. The input wrapper cells provide controllability, and the output wrapper cells provide observability. If safe values are specified to protect the surrounding fanout logic from the core output response, they are driven from the output wrapper cells.

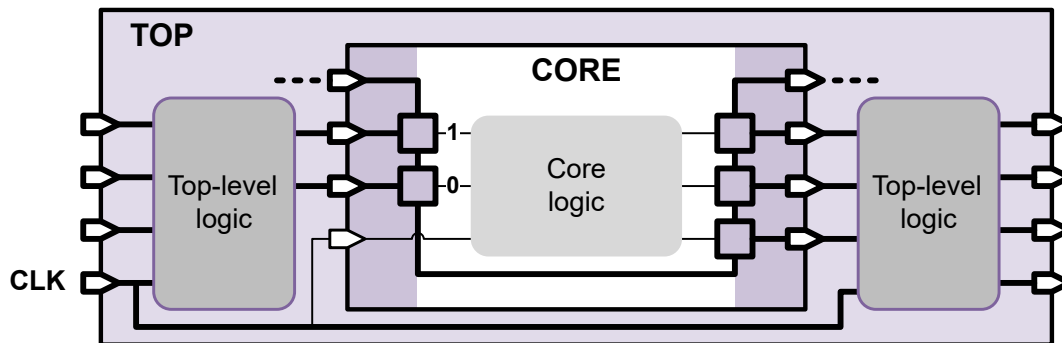
Figure 59 Inward-Facing Mode of Wrapper Chain



- **Outward-facing mode (EXTEST)**

This mode tests the logic surrounding the core in isolation from the core itself. It includes only the wrapper chain. The input wrapper cells provide observability, and the output wrapper cells provide controllability. If safe values are specified to protect the core inputs from the surrounding fanin logic responses, they are driven from the input wrapper cells. Clock inputs to the core remain unaffected.

Figure 60 Outward-Facing Mode of Wrapper Chain

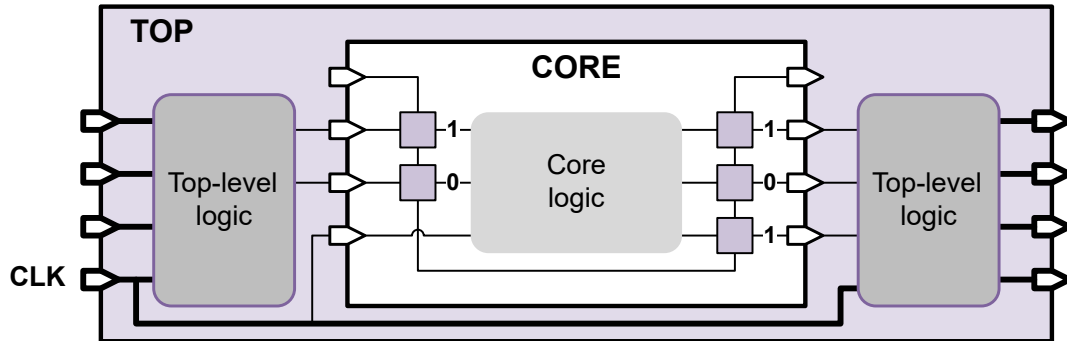


- **Safe mode**

This optional mode drives safe values from all wrapper cells that have a safe value specified. Safe values are driven into core inputs by any such input wrapper cells, and safe values are driven into the surrounding fanout logic by any such output wrapper

cells. There are no scan chains (internal or wrapper). Clock inputs to the core remain unaffected.

Figure 61 Safe Mode of Wrapper Chain



During core creation, DFT insertion does not mix wrapper chains and regular scan chains (although they can be compressed by the same codec).

See Also

- [SolvNetPlus article 3017081, “How Does Core Wrapping Identify Clock Ports for Exclusion?”](#) for more information about how clock ports are identified

Wrapper Test Modes

When core wrapping is enabled, you can define the following test mode types:

- ScanCompression_mode

This is an inward-facing compressed scan mode. The wrapper chain is placed in the INTEST mode of operation. Both wrapper chains and internal core chains are active and compressed by the scan compression codec.

This mode is created only if scan compression is also enabled.

- Internal_scan

This is an inward-facing uncompressed scan mode. The wrapper chain is placed in the INTEST mode of operation. Both wrapper chains and internal core chains are active.

This mode is created if the `set_scan_configuration -chain_count` command is used to configure an uncompressed scan chain architecture.

- wrp_of

This is an outward-facing uncompressed scan mode. Only the wrapper chain is active; it is placed in the EXTEST mode of operation.

This mode is always created when core wrapping is enabled.

These test modes must be explicitly defined. You can use your own test mode names, if desired. For more information, see [Defining Core Wrapping Test Modes](#).

Dedicated Core Wrapping Flow

The dedicated core wrapping flow provides basic core wrapping functionality, as described in the following topics:

- [Dedicated Core Wrapper Cells](#)
- [Dedicated Core Wrapper Chains](#)

Dedicated Core Wrapper Cells

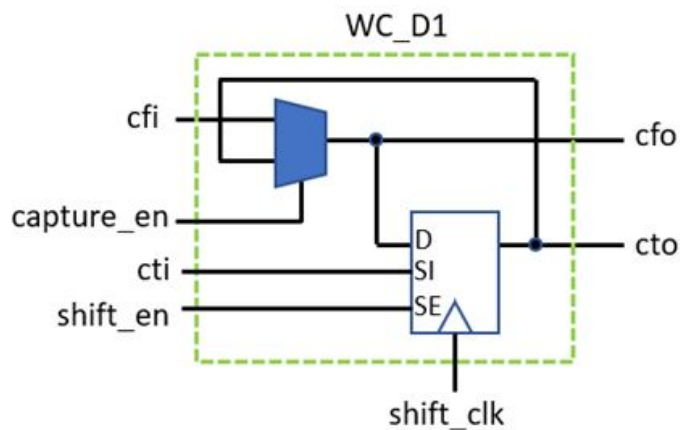
The dedicated core wrapping flow uses the following wrapper cell types:

- [Dedicated Wrapper Cell](#)
- [Dedicated Safe-State Wrapper Cells](#)

Dedicated Wrapper Cell

By default, the dedicated core wrapping flow uses the WC_D1 *dedicated wrapper cell* for core wrapping. A dedicated wrapper cell is a wrapper cell that uses its own internal dedicated flip-flop to provide controllability, observability, and shift capabilities. The following figure shows the internal logic of the WC_D1 dedicated wrapper cell.

Figure 62 WC_D1 Dedicated Wrapper Cell



The interface to the WC_D1 wrapper cell consists of the following signals:

cti – Core test input

This is the test input to the wrapper cell. It can come from either a primary input (if the cell is the first cell in the wrapper chain) or the cto signal of the previous wrapper cell in the chain.

cto – Core test output

This is the test output of the wrapper cell. It can drive either a primary output (if the cell is the last cell in the wrapper chain) or the cti signal of the next wrapper cell in the chain.

cfi – Core functional input

For input wrapper cells, this input is fed from the logic surrounding the core. For output wrapper cells, this input is fed from the core.

cfo – Core functional output

For input wrapper cells, this input drives the core. For output wrapper cells, this output drives the logic surrounding the core.

shift_clk – Wrapper clock

This is usually driven by the `wrp_clock` signal in the core. It clocks the flip-flop in the wrapper cell.

shift_en – Shift enable

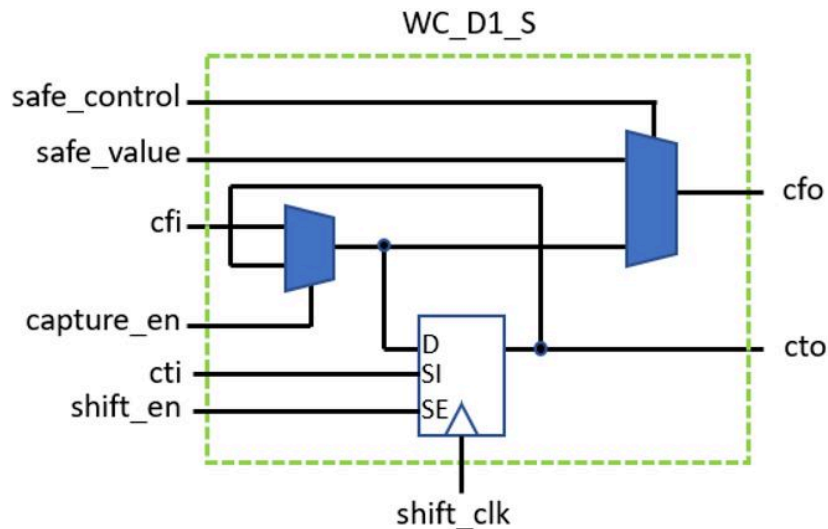
This is like a scan-enable signal for wrapper cells. When the signal is high, the wrapper clock shifts data through the cti and cto scan data pins. When the signal is low, the wrapper clock captures the functional input value or holds the current state, depending on the value of the capture_en signal. The shift enable signal can be controlled differently for input and output wrapper cells.

Dedicated Safe-State Wrapper Cells

A wrapper cell provides observability at its input, and controllability at its output. The same WC_D1 wrapper cell is used for both core inputs and core outputs. However, the controlled output of the wrapper cell can toggle as data is shifted through the wrapper chain. In some cases, if edge-triggered or level-sensitive logic exists in the fanout of the wrapper cell, unintended circuit operation can occur.

To avoid this, you can specify a *safe value* for a wrapper cell. The tool uses the WC_D1_S wrapper cell to implement the safe value capability. It contains an additional multiplexer at its output to drive a static safe logic value, enabled by a safe value control signal. The following figure shows the internal logic of the WC_D1_S wrapper cell.

Figure 63 WC_D1_S Dedicated Wrapper Cell



The interface to the WC_D1_S wrapper cell consists of the same signals as the WC_D1 wrapper cell, plus the following additional signals:

safe_control – Control signal

This signal determines when the safe state value is driven at the output.

safe_value – Logic value

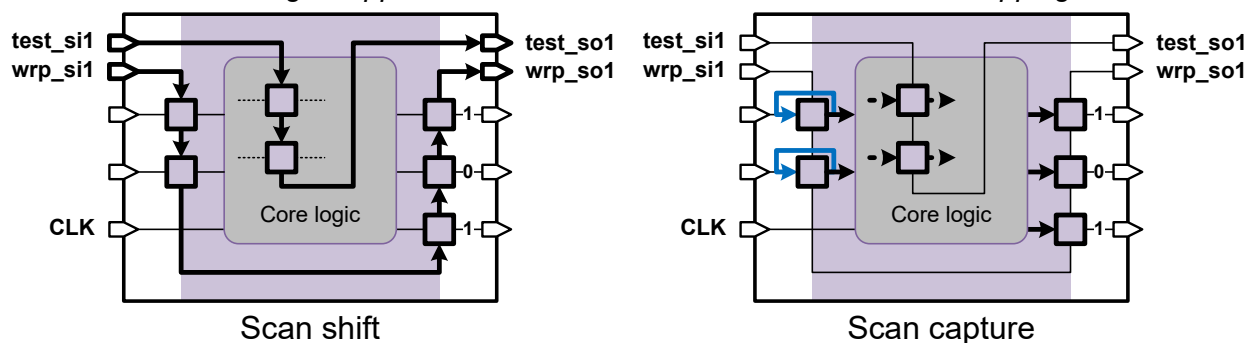
This signal specifies the safe state logic value.

Dedicated Core Wrapper Chains

In the dedicated core wrapping flow, input and output wrapper cells can be placed in the same wrapper chain. The design can have a single wrapper chain or multiple wrapper chains.

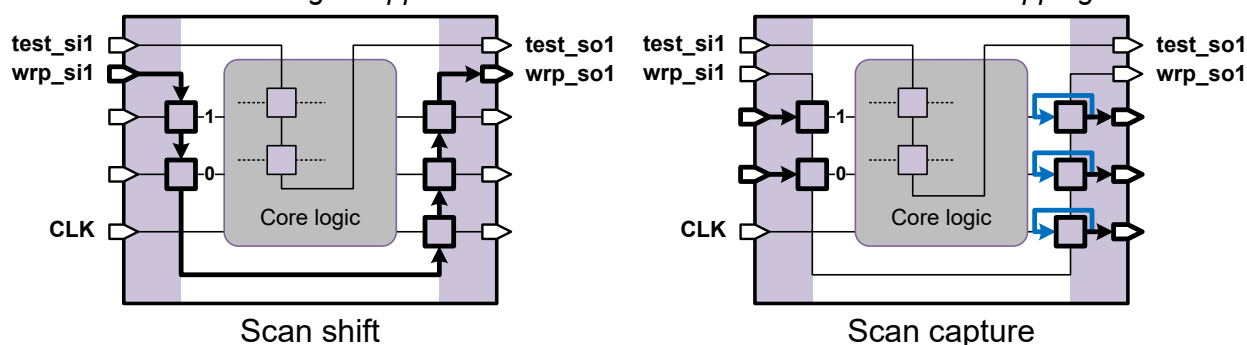
The following figure shows the shift and capture behaviors used for inward-facing operation. In scan capture, the wrapper cell state-holding loops (shown in blue) are used to block external values at core inputs.

Figure 64 Inward-Facing Wrapper Chain Behaviors in the Dedicated Core Wrapping Flow



The following figure shows the shift and capture behaviors used for outward-facing operation. In scan capture, the wrapper cell state-holding loops (shown in blue) are used to block core-driven values at core outputs.

Figure 65 Outward-Facing Wrapper Chain Behaviors in the Dedicated Core Wrapping Flow



The Maximized Reuse Core Wrapping Flow

The dedicated core wrapping flow adds dedicated wrapper cells when functional I/O registers are not directly connected to I/O ports through simple buffering or inverting logic. To reduce the timing and area impact of core wrapping, Fusion Compiler also provides a *maximized reuse* mode that can share I/O registers that are connected to I/O ports through combinational logic.

The maximized reuse flow is described in the following topics:

- [Maximized Reuse Core Wrapper Cells](#)
- [Maximized Reuse Core Wrapper Chains](#)

Maximized Reuse Core Wrapper Cells

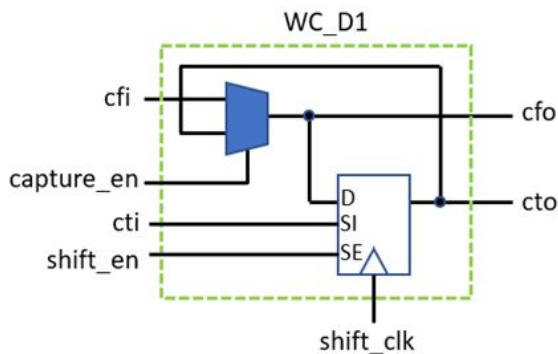
The maximized reuse core wrapping flow uses the following wrapper cell types:

- [Dedicated Wrapper Cells](#)
- [Shared-Register Wrapper Cells](#)

Dedicated Wrapper Cells

Dedicated wrapper cells are used for I/O ports to be wrapped that exceed the sharing thresholds. The following figure shows the internal logic for the WC_D1 dedicated wrapper cell.

Figure 66 WC_D1 Wrapper Cell



Shared-Register Wrapper Cells

If your design has existing boundary I/O registers by the ports, you can share these functional registers with the wrapper cell logic to reduce the core wrapping area overhead. *Shared wrapper cells* reuse the existing functional scan-register, providing equivalent functionality in functional mode. The following figure shows the internal logic for the WC_S1 and WC_S1_S shared wrapper cells. The signals are identical to the signals used for dedicated wrapper cells.

Figure 67 WC_S1 Shared Wrapper Cells

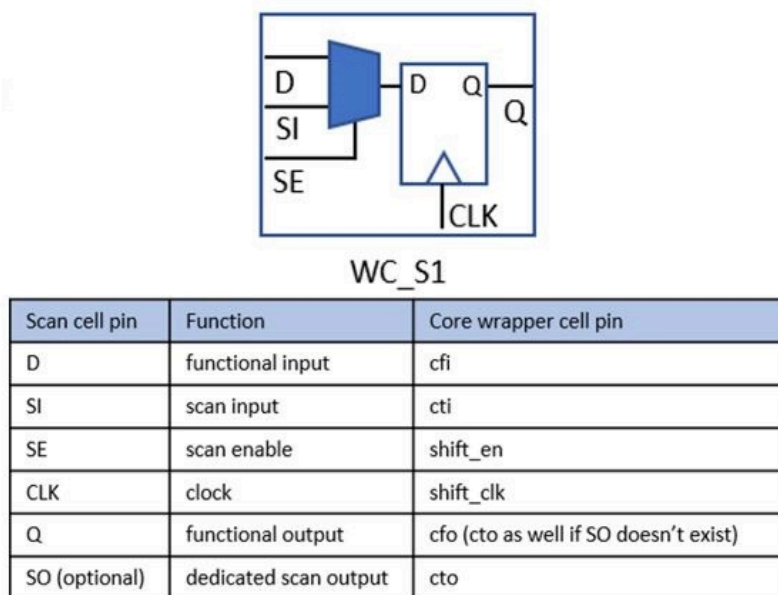
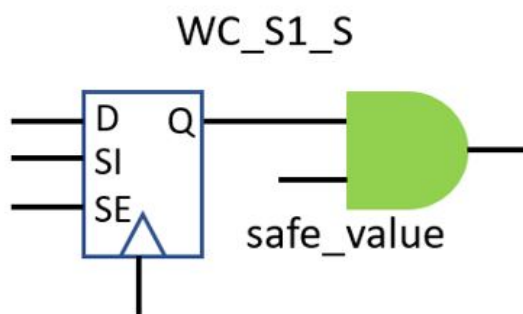


Figure 68 WC_S1_S Shared Wrapper Cells



Note:

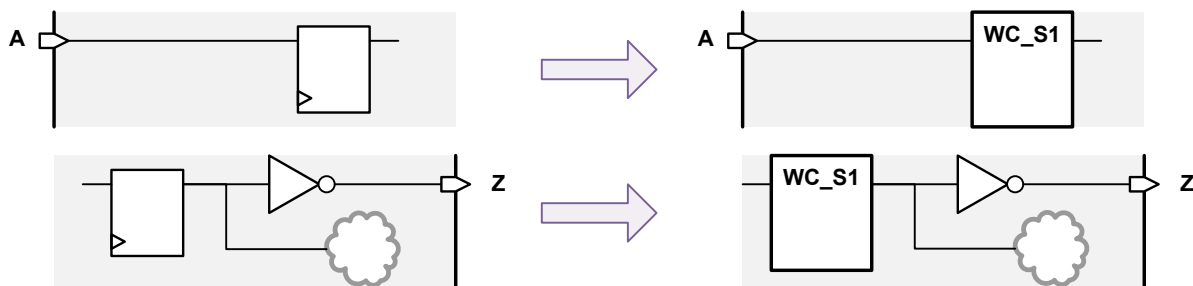
WC_S1_S is implemented by adding AND or OR gate to the output of WC_S1 wrapper cell depending on the safe value.

In order for the core wrapping feature to share the existing functional register in the maximized reuse flow, the I/O register and the port must meet the following conditions:

- The register's data input or output must be connected to a boundary port with a wire or logic path.
- The shared registers must be clocked by a functional clock defined as ScanClock.

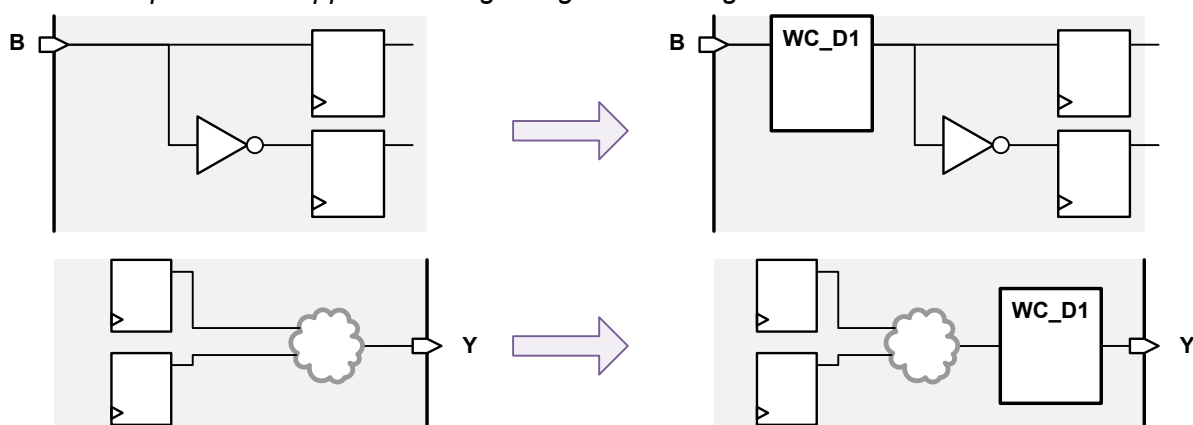
The following figure illustrates some cases where shared wrapper cells can be used.

Figure 69 Examples of Supported Design Register Sharing



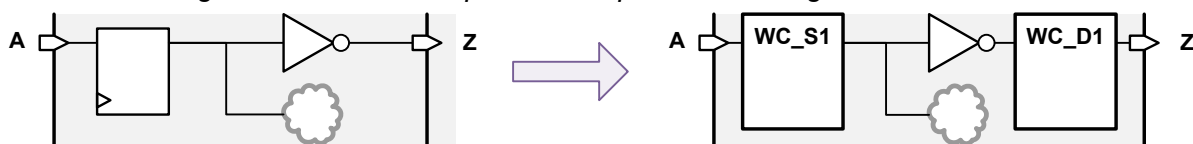
However, if a register does not meet the requirements, a dedicated wrapper cell is used instead. The following figure illustrates some cases where shared wrapper cells cannot be used.

Figure 70 Examples of Unsupported Design Register Sharing



If a register qualifies as both an input shared register and an output shared register, the register becomes an input shared register, and a dedicated wrapper cell is placed at the output port as shown in the following figure.

Figure 71 Same Register Qualified as Input and Output Shared Register



Maximized Reuse Core Wrapper Chains

Input and output wrapper cells are placed in separate wrapper chains with separate input and output wrapper shift-enable signals. During capture, wrapper chains are kept in scan shift mode as needed to block values from being captured by wrapper cells.

Figure 72 shows the shift and capture behaviors used for inward-facing operation. In scan capture, input wrapper chains are kept in scan shift (highlighted in blue) to block external values at core inputs.

Figure 72 Inward-Facing Wrapper Chain Behavior

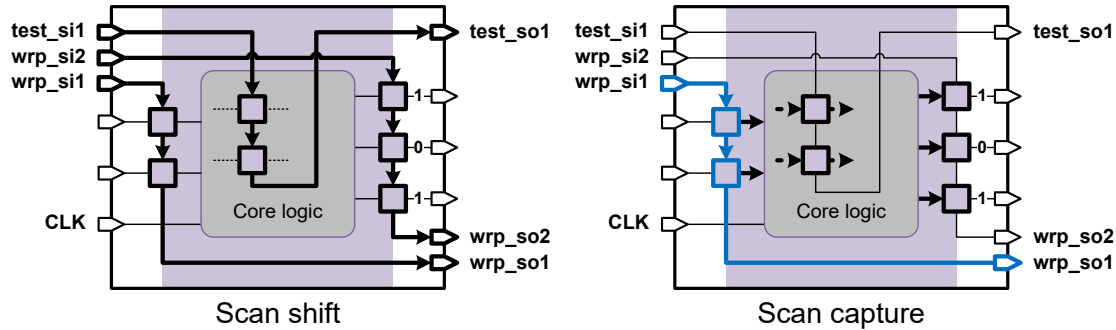
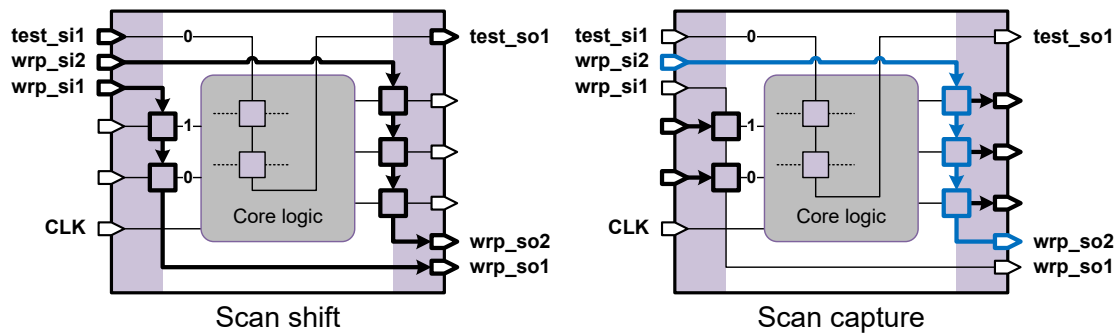


Figure 73 shows the shift and capture behaviors used for outward-facing operation. In scan capture, output wrapper chains are kept in scan shift (highlighted in blue) to block core-driven values at core outputs. Core wrapper chain scan-ins are driven with logic 0 to reduce power consumption.

Figure 73 Outward-Facing Wrapper Chain Behaviors



These shift and capture behaviors apply to all wrapper cells, shared and dedicated, in the input and output wrapper chains.

In transition-delay ATPG, the highlighted wrapper chains that are kept in scan shift generate transitions by shifting the opposite value into a wrapper cell from the preceding wrapper cell.

Note:

Shared register core wrapper might introduce unknown value in transition-delay test. Inserting hold mux for shared core wrapper cells enables holding test data during capture like dedicated core wrapper cells and prevents introducing unknown value. You can enable hold mux feature by using

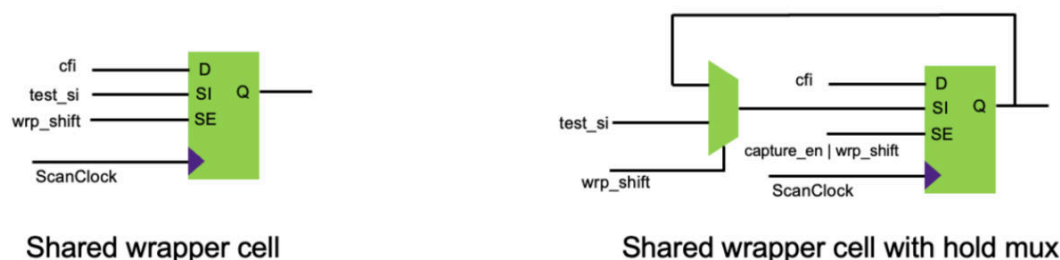
```
set_wrapper_configuration -hold_mux_for_shared_wrapper_cells
enable.
```

Inserting Hold Multiplexer (MUX) for Shared Core Wrapper Cells

Shared register core wrapper cells might introduce unknown values (X) during transition-delay tests. This occurs because, unlike dedicated wrapper cells, shared wrapper cells may not hold the test data during the capture phase, potentially leading to unpredictable test results.

To address this, Fusion Compiler supports the insertion of a hold multiplexer (MUX) for shared core wrapper cells. During the capture phase, the hold MUX enables these shared wrapper cells to retain the test data, similar to the behavior of dedicated wrapper cells. This prevents the introduction of unknown values and improves the reliability and coverage of transition-delay tests. The following figure explains the insertion of hold multiplexer (MUX) for the shared core wrapper cells.

Figure 74 Inserting Hold Multiplexer (MUX) for Shared Core Wrapper Cells

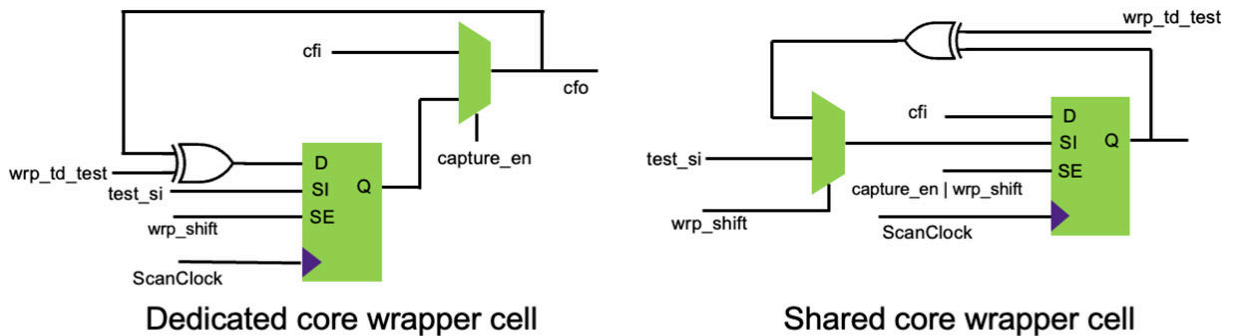


You can enable the hold Multiplexer feature for shared core wrapper cells using the `set_wrapper_configuration -hold_mux_for_shared_wrapper_cells enable` command.

For transition delay testing, you can specify the `wrp_td_test` signal with hold multiplexer feature using the `set_dft_signal -type wrp_td_test -port <port>` command. Based on the signal value, the cell can either hold the state or toggle during the capture phase.

To achieve the toggle functionality, the tool implements an XOR gate in the loop back path, as shown in the following figure:

Figure 75 Inserting Hold Multiplexer (MUX) for Shared Core Wrapper Cells



Controlling Register Reuse

By default, the tool reuses all registers associated with a port as shared wrapper cells, regardless of number. However, if some ports have many I/O registers in their fanin or fanout,

- The number of shared wrapper cells increases accordingly.
- The amount of external combinational logic between the ports and I/O registers increases (which then becomes tested in outward-facing mode instead of inward-facing mode).

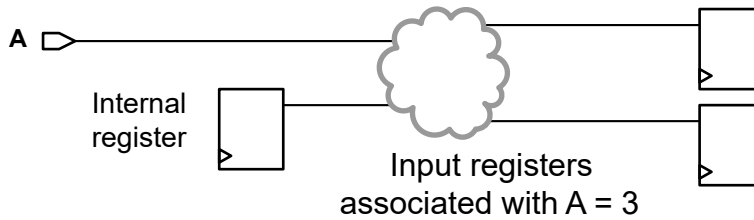
The maximized reuse feature considers how many registers exist in the fanout from an input port or the fanin to an output port. A single port might have many registers in its fanin or fanout, which would require many shared wrapper cells to fully wrap the port. If the number of fanin or fanout registers for a port exceeds a reuse threshold value, a single dedicated wrapper cell is used for that port.

To limit register reuse for ports with large numbers of I/O registers, you can apply a *reuse threshold* to the design. If the number of registers exceeds the reuse threshold, a dedicated wrapper cell is placed at the port.

Applying Reuse Thresholds to Input Ports

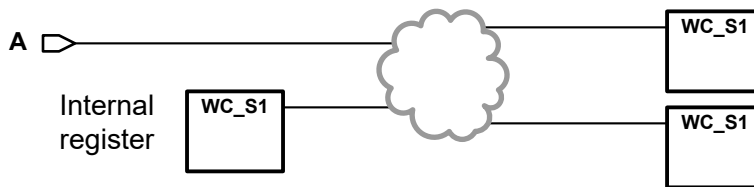
For an input port, the tool determines the number of registers in the fanout of the port. In addition, it includes any internal registers that feed data pins or synchronous set/reset pins of the fanout registers. Figure 76 shows an input port with an associated register count of three.

Figure 76 Input Port Register Count Computation Example



In this example, if the reuse threshold is set to a value of three or higher, the tool replaces the registers with shared wrapper cells, as shown in [Figure 77](#).

Figure 77 Input Port Registers After Wrapper Cell Replacement



The tool places the shared wrapper cells for the fanout registers in the input wrapper chain. Because any internal registers feeding these fanout registers capture values from the core logic instead of input ports, the tool places the shared wrapper cells for these internal registers in the output wrapper chain.

Dedicated wrapper cells are added to input ports according to the following rules:

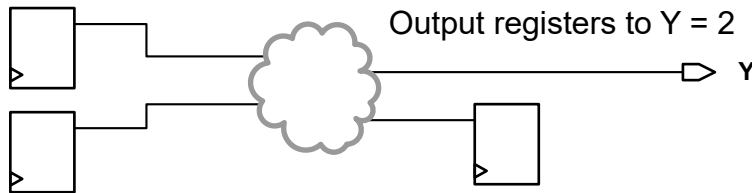
- If the sum of the input fanout registers and the internal registers feeding them exceeds the reuse threshold, then a dedicated wrapper cell is added to the input port.
- When the input register cells exceed the reuse threshold for a bidirectional port, dedicated wrapper cells are added to the data-out, enable, and data-in paths.
- If an input port is associated with a CTL-modeled cell, then a dedicated wrapper cell is added to the port. See [Wrapping Ports Associated With CTL-Modeled Cells](#) for details.

A warning is issued if all registers associated with an input port do not use the same clock.

Applying Reuse Thresholds to Output Ports

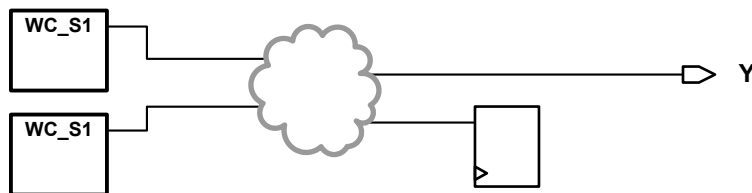
For an output port, the tool determines the number of registers in the fanin of the port. It does not include any additional surrounding registers. [Figure 78](#) shows an output port with an associated register count of two.

Figure 78 Output Port Register Count Computation Example



In this example, if the reuse threshold is set to a value of two or higher, the tool replaces the registers with shared wrapper cells, as shown in [Figure 79](#).

Figure 79 Output Port Registers After Wrapper Cell Replacement



The tool places the shared wrapper cells for the fanin registers in the output wrapper chain.

Dedicated wrapper cells are added to output ports according to the following rules:

- If the sum of the output fanin registers exceeds the reuse threshold, then a dedicated wrapper cell is added to the output port.
- When the number of the output register cells exceeds the reuse threshold value for a three-state output port, dedicated wrapper cells are added to both data-out and enable paths.
- If an output port is associated with a CTL-modeled cell, then a dedicated wrapper cell is added to the port. See [Wrapping Ports Associated With CTL-Modeled Cells](#) for details.
- For three-state and bidirectional ports, only a functional register that directly controls the pad is considered as a shared wrapper cell for the enable path of the pad. There can be inverting or noninverting buffers between the register and the pad enable pin. If no such register is found, a dedicated wrapper cell is added at the driver cell enable pin to control the port.

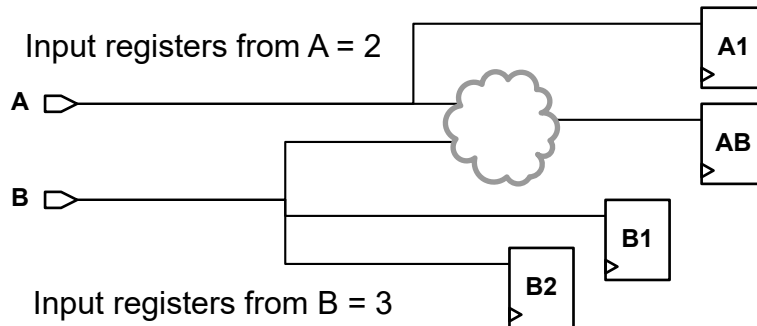
A warning is issued if all registers associated with an output port do not use the same clock.

Registers Associated With Multiple Ports

If the same register is associated with multiple ports, the register is replaced with a shared wrapper cell if *any* port meets the threshold. In [Figure 80](#), register AB is in the fanout

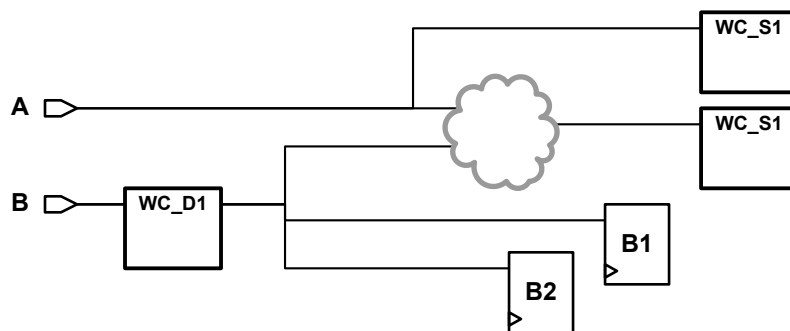
of ports A and B. Input port A has two associated registers, and input port B has three associated registers.

Figure 80 Register in Fanout of Multiple Ports



In this example, if the reuse threshold is set to a value of two, the tool replaces all registers for input port A with shared wrapper cells, and it inserts a dedicated wrapper cell at input port B, as shown in [Figure 81](#). Register AB is replaced with a shared wrapper cell to completely wrap port A, even though it is also in the fanout of the dedicated wrapper cell for input port B.

Figure 81 Register in Fanout of Multiple Ports After Wrapper Cell Replacement



Registers that become shared wrapper cells for one port do not affect the associated register count for other ports. In [Figure 80](#), the associated register count for input port B is three even though register AB is wrapped for input port A.

If a register is classified as both an input shared register and an output shared register, the register becomes an input shared register and is removed from the output shared register list. No dedicated wrapper cell is placed at the output port.

Special Cases for Register Reuse

In some cases, ports are associated with logic constructs that require special-case handling for register reuse.

An information message is issued if I/O registers are not detected for a port and no dedicated wrapper cell is manually specified for the port:

```
Information: Port 'B' has no I/O Registers associated with it;  
not adding any dedicated wrapper cells to this port. (DFT-2184)  
Information: Port 'Z' has no I/O Registers associated with it;  
not adding any dedicated wrapper cells to this port. (DFT-2184)
```

Wrapping Ports Associated With CTL-Modeled Cells

CTL-modeled cells, such as memories or DFT-inserted cores, cannot be used as shared wrapper cells. When a port is associated with a CTL-modeled cell, as shown in the examples in [Figure 82](#) and [Figure 83](#), a dedicated wrapper cell is added to the port.

Figure 82 Core Wrapping of Input Ports Associated With CTL-Modeled Cells

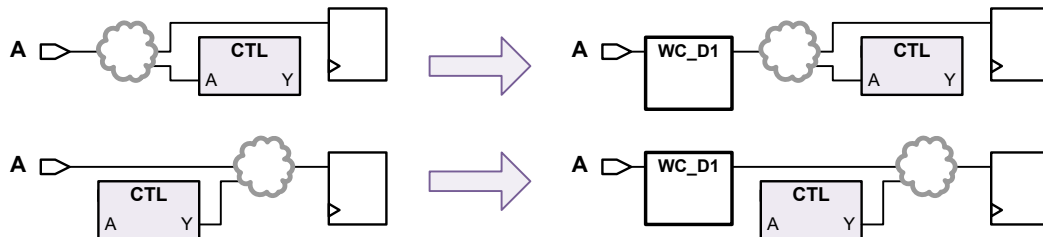
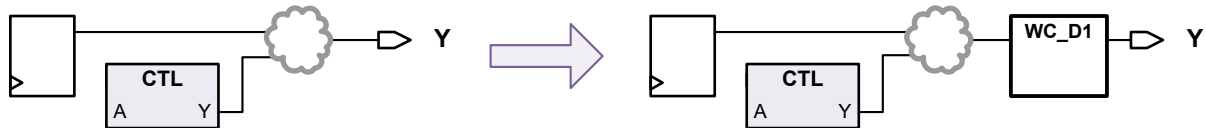


Figure 83 Core Wrapping of Output Ports Associated With CTL-Modeled Cells



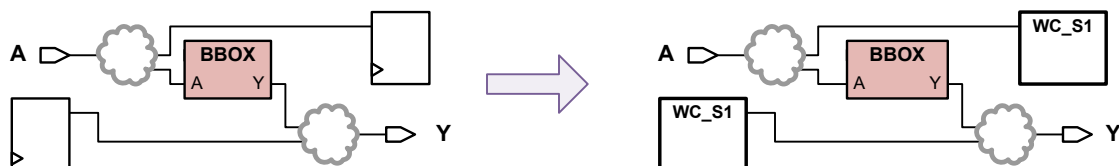
In addition, a warning message is issued:

```
Warning: Port 'DACK' is connected to cell 'IP_CORE', which is represented  
by a CTL model; a dedicated wrapper cell is added to the port. (DFT-3109)
```

Port Wrapping and Other Black-Box Cells

Black-box cells without CTL models do not affect core wrapping. Wrapper cells are not added unless needed by other design logic. See [Figure 84](#).

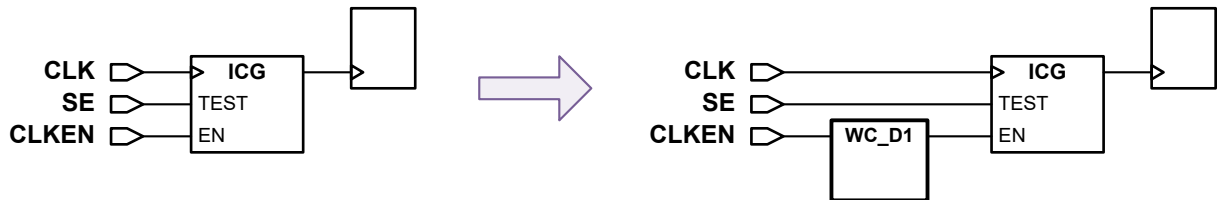
Figure 84 Black-Box Cell With No CTL Model



Wrapping Ports Associated With Clock-Gating Cells

A dedicated wrapper cell is added to an input port that drives the enable signal of a clock-gating cell. In [Figure 85](#), an integrated clock-gating cell has both a functional enable pin and a test-mode pin. A dedicated wrapper cell is added for the functional enable signal driven by input port CLKEN. No wrapper cells are inserted for the clock signal or the global test-mode signal.

Figure 85 Integrated Clock Gating Cell Enable Signals



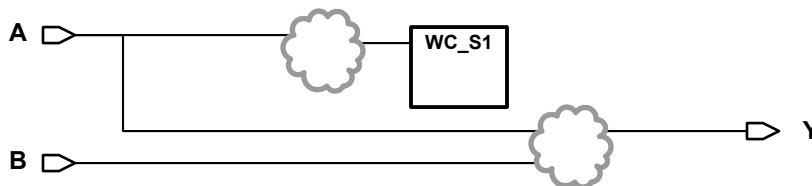
A warning message is issued if such a clock-gating enable signal is detected:

```
Warning: Port 'CLKEN' is connected to a clock gating cell 'UICG';
a dedicated wrapper cell is added to the port. (DFT-3111)
```

Port Wrapping and Feedthrough Paths

No wrapper cell is added to a port that is bounded only by combinational logic unless you explicitly specify a dedicated wrapper cell for that port. See [Figure 86](#).

Figure 86 Combinational Feedthrough Paths



An information message is issued if a feedthrough port is detected and no dedicated wrapper cell is manually specified for the port: Information:

```
Information: Port 'B' has no I/O Registers associated with it;
not adding any dedicated wrapper cells to this port. (DFT-2184)
Information: Port 'Y' has no I/O Registers associated with it;
not adding any dedicated wrapper cells to this port. (DFT-2184)
```

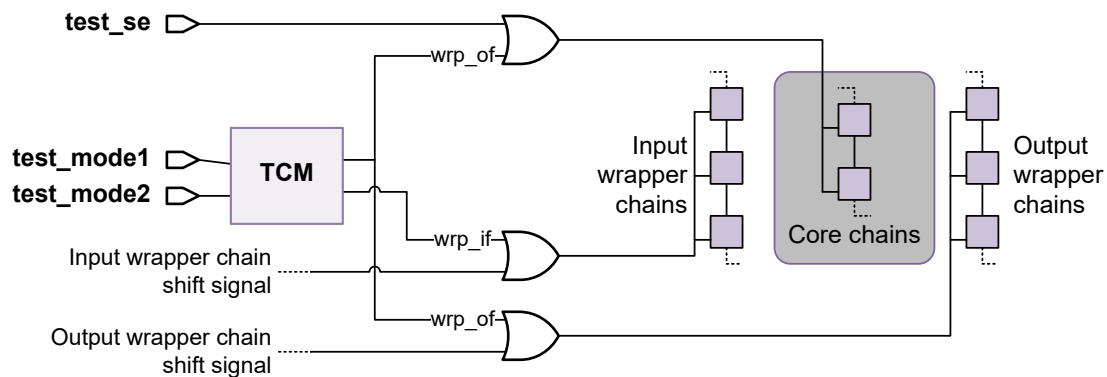
Wrapper Shift Signals

The scan-enable and wrapper shift signals are conditioned by the currently active test mode as follows:

- In inward-facing wrapper modes, the input wrapper shift signal is always asserted.
- In outward-facing wrapper modes, the output wrapper shift signal is always asserted.
- In outward-facing wrapper modes, the scan-enable signal for the internal core scan chains is always asserted (to load constant values into all scan chain elements).

Figure 87 shows an example of the conditioning logic.

Figure 87 Scan-Enable and Wrapper Shift Signals



The wrapper shift signals going to the input and output wrapper chains are conditioned separately as shown in the diagram, even when a single scan-enable signal is used for all three shift signals (scan, input wrapper, and output wrapper).

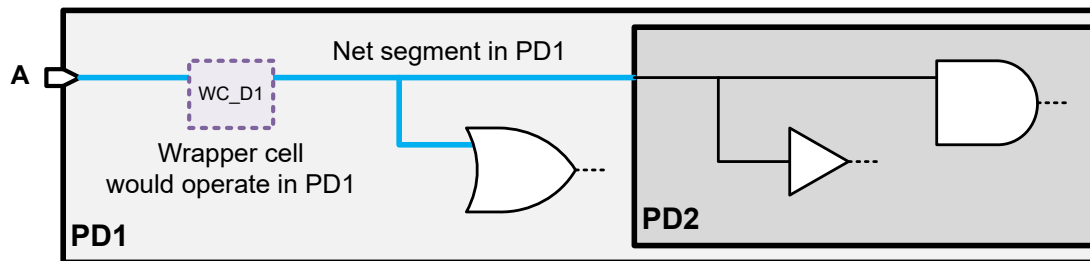
See Also

- [Defining Wrapper Shift Signals](#)

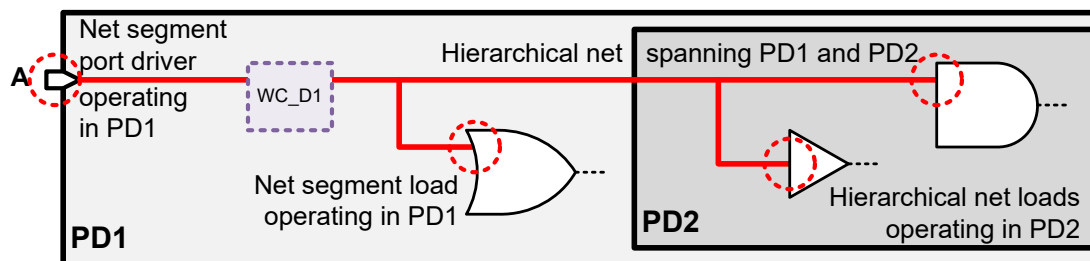
Dedicated Wrapper Cell Insertion in UPF Designs

When the tool inserts dedicated wrapper cells in a design that contains UPF information, it prefers the highest possible hierarchical nets that do not result in multivoltage violations.

When a dedicated wrapper cell is inserted on a net segment, its logic inherits—and thus operates in—the power domain containing that particular net segment:



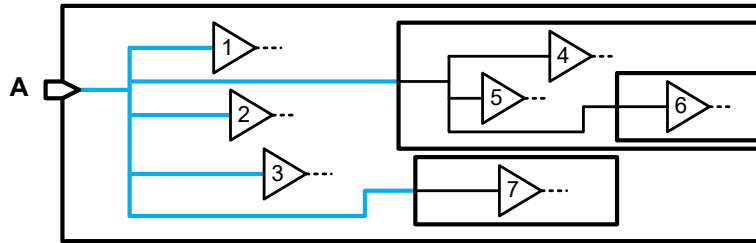
But if the net segment is part of a hierarchical net that crosses power domains, the wrapper cell power domain must be compatible with the driver and load power domains of the *entire* hierarchical net:



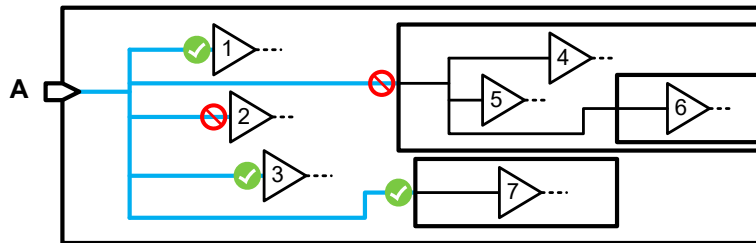
Dedicated Wrapper Cell Insertion Algorithm

To meet these requirements, the tool inserts dedicated wrapper cells for input ports as follows:

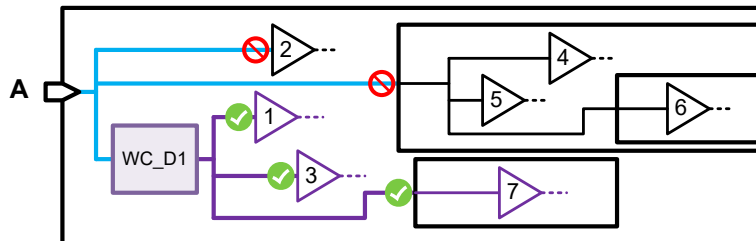
1. Starting at the port, the tool searches for the highest hierarchical net segment (closest to the port) where a dedicated wrapper cell could be inserted.



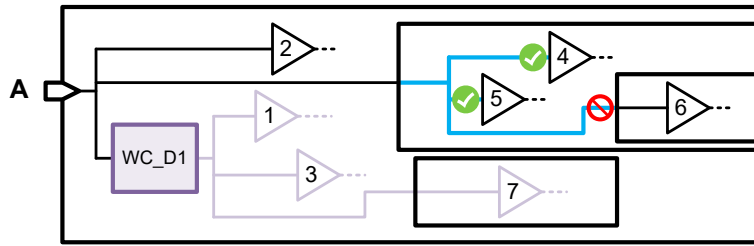
2. At each net segment, the tool determines which segment load pins—hierarchical and leaf—can be driven by a dedicated wrapper cell without introducing a multivoltage violation. A hierarchical load pin violates if any downstream leaf load pin would violate.



3. If any segment load pins can be driven without violation, the tool inserts a dedicated wrapper cell to drive them. Their downstream logic is now wrapped.



4. The search then continues forward through any unwrapped load pins.



The search can also cross buffers, inverters, isolation cells, and level-shifter cells; it stops at all other cells.

5. When the search completes, if any unwrapped load pins remain, the tool repeats the search but this time allowing *fixable* multivoltage violations to occur.

Caution:

The first search (for nonviolating locations) considers the existing isolation strategies and power state tables. The second search (for fixable locations) considers a location fixable if an isolation strategy *could* be applied to insert the isolation cell, even if no such strategy currently exists.

The process works similarly for output ports, except that the search proceeds backward through each net segment driver pin instead of forward through the load pins. Only one dedicated wrapper cell can be inserted for an output port.

A dedicated wrapper cell cannot be inserted in the following locations:

- Between an isolation cell and the upper boundary of the power domain where that isolation strategy is applied (using the `-domain` option)
- Between a zero-pin retention cell and its clamp cell
- Between an isolation, retention, or switch control signal and the power management cells that it controls

Optimizing UPF Aware Dedicated Core Wrapper Cell Insertion Near Core

Dedicated core wrapper cells are added in the design for ports, where shared wrapper cells identification is not possible to core wrap the port. This insertion of dedicated core wrapper cells is UPF aware such that the new DFT logic does not break the UPF intention of the design.

The Fusion Compiler tool supports two flows for this insertion as follows:

1. Preference is given to insert the dedicated core wrappers near the top level ports, the default flow
2. Preference is given to insert dedicated core wrapper away from the top level ports, near the core

In both the flows, the algorithm enables the tool to first find the location without skipping PMK (Power Management Kit) cells. If it is unable to find then it is continued to skip PMK cells to find UPF legal location for the wrapper cells.

The second flow can increase the overall number of dedicated core wrapper cells since the input ports fanout to multiple loads and the wrapper cells are added to every load.

This feature now enables the tool to optimize the number of dedicated core wrapper cells as follows:

- From a port, trace the signal nets until it reaches the core cells by skipping power management (PMK) cells, inverters, and buffers
- Then, insert dedicated core wrapper cells to each net of the core cells
- If a common signal net drives multiple core cells, insert a dedicated core wrapper cell on the common net

Firstly, to enable UPF aware dedicated core wrapper cell insertion flow, use the `set_wrapper_configuration -add_wrapper_cells_to_power_domain enable` command.

Then, to enable the UPF aware dedicated core wrapper cell insertion flow near the core, you can consecutively set the following application options:

- `set_app_options -name dft.test_power_aware_dw_insertion_near_port -value false`
- `set_app_options -name dft.test_power_aware_dw_insertion_near_multi_input_cells -value true`

Wrapping a Core

Core wrapping is performed during RTL DFT insertion. Core wrapping configuration is described in the following topics:

- [Enabling Core Wrapping](#)
- [Disabling Core Wrapping](#)
- [Resetting Wrapper Settings](#)

- [Defining Wrapper Shift Signals](#)
- [Defining Dedicated Wrapper Clock Signals](#)
- [Configuring Global Wrapper Settings](#)
- [Configuring Wrapper Clock Signals](#)
- [Configuring Dedicated Wrapper Cell Clocks](#)
- [Controlling Wrapper Chain Count](#)
- [Setting a Reuse Threshold](#)
- [Configuring Port-Specific Wrapper Settings](#)
- [Mixing Input and Output Wrapper Cells](#)
- [Creating Wrapper Chains with Mixed Input and Output Wrapper Cells](#)
- [Configuring Maximized Reuse Core Wrapping](#)
- [Determining Power Domains for Dedicated Wrapper Cells](#)
- [Using the `set\_scan\_path` Command With Wrapper Chains](#)
- [Enabling Soft Core Wrapping](#)
- [Previewing the Wrapper Cells](#)
- [Post DFT DRC Rule Checks](#)
- [Creating an EXTEST-Only Core Netlist](#)

Enabling Core Wrapping

You can enable automatic core wrapping by using the command `set_wrapper_configuration` specified after loading and linking the design.

You can enable dedicated core wrapping flow by setting `reuse_threshold` to -1 to. No functional cell is used as a wrapper cell in this flow.

You can enable shared core wrapping flow by setting `reuse_threshold` to 0 or any positive integer. This is also the maximized use core wrapping flow.

Note:

`reuse_threshold` value '0' sets the reuse threshold limit to infinite.

Also, in the shared wrapper flow, the tool still adds a dedicated wrapper cell such as ports driving clock gating cell enable pins, CTL-modeled cells and wherever required.

Disabling Core Wrapping

You can disable core wrapping by using the command `set_dft_configuration -wrapper disable` specified after loading and linking the design.

Resetting Wrapper Settings

You can reset wrapper configuration to default setting by using `reset_wrapper_configuration` specified after loading and linking the design.

You can reset wrapper configuration for a specific mode by using the command `reset_wrapper_configuration -test_mode <test mode>`.

Defining Wrapper Shift Signals

The wrapper shift signal enables scan data to shift through wrapper chains, just as a scan-enable signal does for scan chains. You can use any of the following wrapper shift signal configurations.

Using the Default Scan-Enable Signal

By default, Fusion Compiler uses the design's scan-enable signal as the wrapper shift signal:

```
fc_shell> set_dft_signal -view spec -type ScanEnable \  
    -port my_test_se ;# by default, this is used for wrapper cells
```

Unless your design requires otherwise, no further configuration is needed.

Defining a Single Dedicated Wrapper Shift Signal

To define a single dedicated wrapper shift signal, define the signal source as a `wrp_shift` signal:

```
fc_shell> set_dft_signal -view spec -type wrp_shift \  
    -port my_wrp_shift
```

Defining Separate Input and Output Wrapper Shift Signals

To use separate wrapper shift signals for input and output wrapper chains, define two signals as `input_wrp_shift` and `output_wrp_shift`, respectively:

```
fc_shell> set_dft_signal -view spec \  
    -type input_wrp_shift -port {wrp_ishift}  
fc_shell> set_dft_signal -view spec \  
    -type output_wrp_shift -port {wrp_oshift}
```

Defining Dedicated Wrapper Clock Signals

You can define a dedicated wrapper clock signal with the `set_dft_signal` command:

If you have an existing port to use for this wrapper clock signal, you can define it with the `set_dft_signal` command:

```
fc_shell> set_dft_signal -view spec -type wrp_clock \  
                  -port port_name
```

By default, this clock signal uses the default wrapper clock timing (10 percent duty cycle). You can also specify signal-specific timing by defining an `existing_dft` view of the wrapper clock signal:

```
fc_shell> set_dft_signal -view existing_dft -type wrp_clock \  
                  -timing {45 55} -port port_name
```

Configuring Global Wrapper Settings

The `set_wrapper_configuration` command allows you to specify global configuration parameters that apply to the entire wrapper chain.

By default, the tool does not create safe state wrapper cells. To specify a safe state value for all wrapper cells in the wrapper chain, use the `-safe_state` option of the `set_wrapper_configuration` command.

Configuring Wrapper Clock Signals

Shared wrapper cells reuse an existing, clocked functional register; no clock configuration is needed for them.

Configuring Dedicated Wrapper Cell Clocks

By default, dedicated core wrapping uses dedicated wrapper cells that use a dedicated wrapper clock. However, you can use system clocks for dedicated wrapper cells by setting the following option:

```
fc_shell> set_wrapper_configuration \  
                  -use_system_clock_for_dedicated_wrp_cells enable.
```

Controlling Wrapper Chain Count

In Fusion Compiler, the compressed wrapper chains are architected first, then the other modes are architected using the compressed wrapper chains as segments. These compressed wrapper chain segments cannot be further subdivided.

Wrapper cells and internal scan cells cannot be mixed on the same chain. Also, input and output wrapper cells cannot be mixed on the same chain unless enabled as described in [Mixing Input and Output Wrapper Cells](#).

Inward-Facing Compressed Mode

In this mode, Fusion Compiler always balances the wrapper chains and scan chains together to meet the compressed chain count specification *IC* specified by

```
fc_shell> set_scan_compression_configuration -streaming true \
  -chain_count IC ;# compressed DFTMAX chain count
fc_shell> # ...or...
fc_shell> set_scan_compression_configuration \
  -chain_count IC ;# compressed DFTMAX Ultra chain count
```

This mode is created only when scan compression is enabled.

Inward-Facing Uncompressed Mode

In this mode, the tool creates wrapper chains by concatenating the smaller compressed wrapper chain segments. The tool balances the compressed wrapper chain segments and scan cells together to meet the overall chain count target *IU* specified by

```
fc_shell> set_scan_configuration \
  -chain_count IU ;# inward-facing uncompressed chain count
```

This mode is created only when an uncompressed scan mode is configured using the preceding command.

Outward-Facing Uncompressed Mode

In this mode, the tool creates wrapper chains using the smaller compressed wrapper chain segments as building blocks. The default wrapper chain count *O* is chosen to match the scan I/O resources of other modes in the design (highest priority first):

- Uncompressed inward-facing mode (Internal_scan)
(set_scan_configuration -chain_count *IU*)
- Compressed inward-facing mode (ScanCompression_mode)
set_scan_compression_configuration -inputs *N*-outputs *N*

To specify a particular chain count for the outward-facing mode, use the following command:

```
fc_shell> set_wrapper_configuration \
  -chain_count W ;# outward-facing mode wrapper chain count
```

This specification does not affect the architecture of the compressed wrapper chains. If there are not enough wrapper chain segments to satisfy this constraint, the tool issues a warning message during DFT preview and insertion:

```
Warning: Only '2' wrapper chains can be architected in wrp_of mode.  
Cannot honor -chain_count specification of '5'.
```

Setting a Reuse Threshold

By default, the tool reuses all registers associated with a port as shared wrapper cells, regardless of number. However, you can set a *reuse threshold* to control this, as described in [Controlling Register Reuse](#).

To set a reuse threshold, use the following command:

```
fc_shell> set_wrapper_configuration -reuse_threshold N
```

The default is no limit (infinity).

Configuring Port-Specific Wrapper Settings

To specify the wrapper cell characteristics of specific ports, you can use the `set_boundary_cell` command.

To prevent the insertion of wrapper cells for a specific list of ports, use the following command:

```
fc_shell> set_boundary_cell -ports port_list -type none
```

This might be needed for special input or output port signals that should not be controlled during test. Since excluding ports from the wrapper chain reduces test coverage, you should use this capability only when necessary.

To specify a dedicated wrapper cell for ports that would otherwise use a shared wrapper cell, specify a WC_D1 wrapper cell with the `set_boundary_cell` command:

```
fc_shell> set_boundary_cell -ports port_list -type WC_D1
```

You cannot use the `set_boundary_cell` command to force a shared wrapper cell type to be used for a port if the I/O register does not meet the requirements for a shared wrapper cell.

Mixing Input and Output Wrapper Cells

By default, input and output wrapper cells cannot be mixed on the same scan chain. To allow this mixing, use the following command:

```
fc_shell> set_wrapper_configuration \  
-mix_cells true
```

Creating Wrapper Chains with Mixed Input and Output Wrapper Cells

To prevent input and output wrapper cells from being mixed on the same scan chains, use the `-mix_cells false` option of the `set_wrapper_configuration` command:

```
fc_shell> set_wrapper_configuration \  
-mix_cells false
```

If you use a single wrapper shift signal, it is used for both the input and output wrapper chains. You can also define separate input and output wrapper chain shift signals, as described in [Defining Separate Input and Output Wrapper Shift Signals on page 168](#).

Configuring Maximized Reuse Core Wrapping

The maximized reuse feature considers how many registers exist in the fanout from an input port or the fanin to an output port. A single port might have many registers in its fanin or fanout, which would require many shared wrapper cells to fully wrap the port. If the number of fanin or fanout registers for a port exceeds a *reuse threshold* value, a single dedicated wrapper cell is used for that port.

Configuration of the maximized reuse core wrapping functionality is described in the following topics:

- [Enabling Maximized Reuse Core Wrapping](#)
- [Applying a Register Reuse Threshold](#)
- [Applying a Combinational Depth Threshold](#)
- [Specifying Port-Specific Maximized Reuse Behaviors](#)
- [Special Cases for Register Reuse](#)
- [Using Dedicated Wrapper Cells](#)
- [Configuring Dedicated Wrapper Cell Clocks](#)
- [Including Additional Scan Cells in Input and Output Wrapper Chains](#)

- [Using the Pipelined Scan-Enable Feature](#)
- [Low-Power Maximized Reuse Features](#)
- [Hierarchical Core Wrapping](#)
- [Limitations of the Maximized Reuse Flow](#)

Enabling Maximized Reuse Core Wrapping

To enable the maximized reuse feature, use the following command:

```
fc_shell> set_wrapper_configuration \
           -reuse_threshold
```

where the value of N defines the reuse threshold. The default of N is 0.

When maximized reuse flow is enabled and the number of registers encountered from an input or to an output exceeds the reuse threshold, a dedicated wrapper cell is added to the port. If the number of registers is less than this threshold, the tool can use these registers for wrapping the core. If a reuse threshold value of zero is specified, the tool converts all I/O registers to shared wrapper cells and no dedicated wrapper cells are used unless specified by the user. For more information about how the reuse threshold is computed, see [Applying a Register Reuse Threshold on page 173](#).

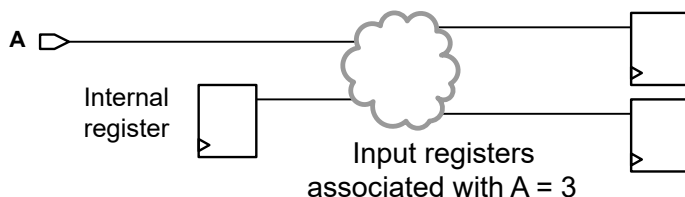
Applying a Register Reuse Threshold

When applying the reuse threshold value to a port, the tool determines the number of registers associated with that port. If the number of registers for a port does not exceed the reuse threshold, the registers are replaced with shared wrapper cells. If the number of registers exceeds the reuse threshold, a dedicated wrapper cell is placed at the port.

Computing Reuse Thresholds for Input Ports

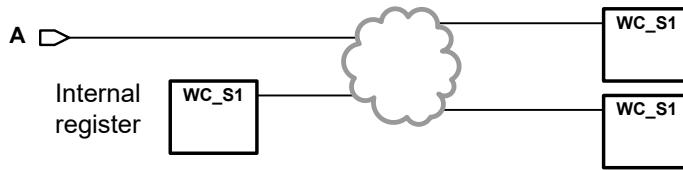
For an input port, the tool determines the number of registers in the fanout of the port. In addition, *it includes any internal registers that feed data pins or synchronous set/reset pins of the fanout registers*. [Figure 88](#) shows an input port with an associated register count of three.

Figure 88 Input Port Register Count Computation Example



In this example, if the reuse threshold is set to a value of three or higher, the tool replaces the registers with shared wrapper cells, as shown in [Figure 89](#).

Figure 89 Input Port Registers After Wrapper Cell Replacement

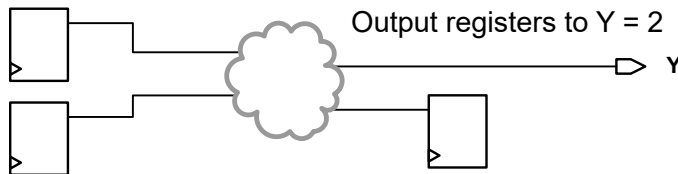


The tool places the shared wrapper cells for the fanout registers in the input wrapper chain. Because any internal registers feeding these fanout registers capture values from the core logic instead of input ports, the tool places the shared wrapper cells for these internal registers in the output wrapper chain.

Computing Reuse Thresholds for Output Ports

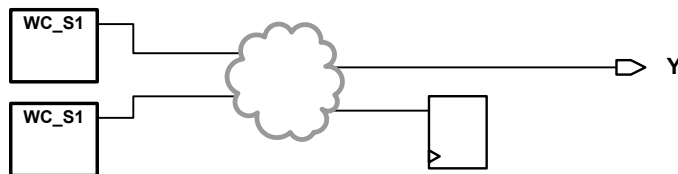
For an output port, the tool determines the number of registers in the fanin of the port. It does not include any additional surrounding registers. [Figure 90](#) shows an output port with an associated register count of two.

Figure 90 Output Port Register Count Computation Example



In this example, if the reuse threshold is set to a value of two or higher, the tool replaces the registers with shared wrapper cells, as shown in [Figure 91](#).

Figure 91 Output Port Registers After Wrapper Cell Replacement

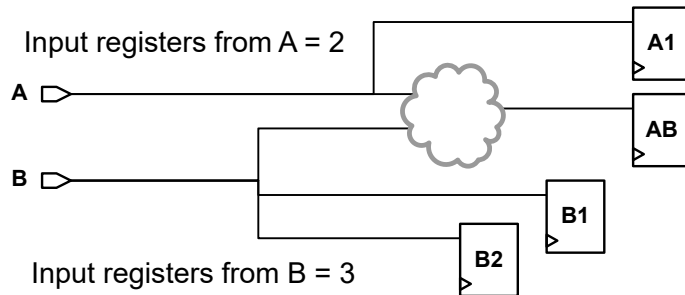


The tool places the shared wrapper cells for the fanin registers in the output wrapper chain.

Registers Associated With Multiple Ports

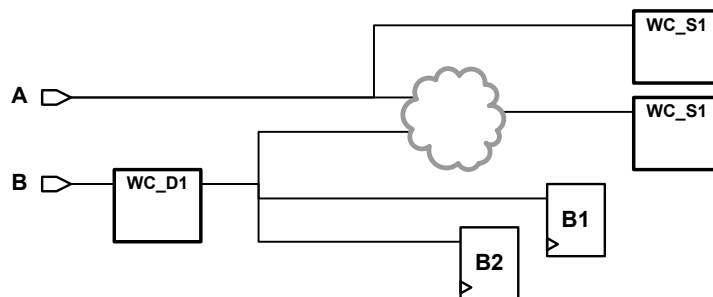
If the same register is associated with multiple ports, the register is replaced with a shared wrapper cell if *any* port meets the threshold. In [Figure 92](#), register AB is in the fanout of ports A and B. Input port A has two associated registers, and input port B has three associated registers.

Figure 92 Register in Fanout of Multiple Ports



In this example, if the reuse threshold is set to a value of two, the tool replaces all registers for input port A with shared wrapper cells, and it inserts a dedicated wrapper cell at input port B, as shown in [Figure 93](#). Register AB is replaced with a shared wrapper cell to completely wrap port A, even though it is also in the fanout of the dedicated wrapper cell for input port B.

Figure 93 Register in Fanout of Multiple Ports After Wrapper Cell Replacement



Registers that become shared wrapper cells for one port do not affect the associated register count for other ports. In [Figure 93](#), the associated register count for input port B is three even though register AB is wrapped for input port A.

If a register is classified as both an input shared register and an output shared register, the register becomes an input shared register and is removed from the output shared register list. No dedicated wrapper cell is placed at the output port.

Applying a Combinational Depth Threshold

For a core-wrapped design, any logic that exists between the wrapper chain and the I/O ports becomes logic that is external to the block for testing purposes. The `-reuse_threshold` option of the `set_wrapper_configuration` command applies a maximum fanout-breadth or fanin-breadth threshold limit for input and output shared wrapper cells, respectively. However, this reuse threshold only indirectly limits the logic depth that can exist outside the wrapper chain.

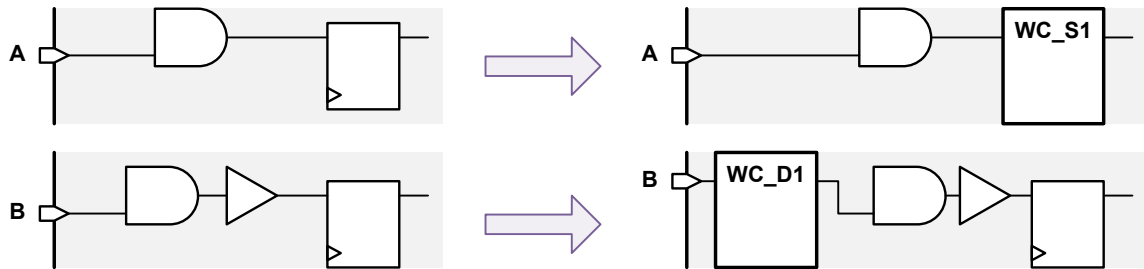
You can use the `-depth_threshold` option of the `set_wrapper_configuration` command to directly specify the maximum number of combinational cells, including buffers and inverters, that can exist between a port and its associated registers. For example,

```
fc_shell> set_wrapper_configuration \
        -depth_threshold 2
```

If this depth is exceeded, a dedicated wrapper cell is used for that port. This can be used to prevent too much logic from being placed on the external side of the wrapper chain.

Figure 94 shows the wrapper cell insertion behavior when the `-depth_threshold` value is set to 1.

Figure 94 Core Wrapping With a Combinational Depth Threshold Value of 1



Specifying Port-Specific Maximized Reuse Behaviors

The reuse threshold can be set on a port-by-port basis with the `set_boundary_cell` command:

```
fc_shell> set_boundary_cell -class wrapper \
        -reuse_threshold N -ports {port_list}
```

This command sets the reuse threshold value to N (where $N \geq 0$) for all ports in the `port_list` specification.

To ignore the reuse threshold and force the use of a dedicated wrapper cell for one or more ports, use the following command:

```
fc_shell> set_boundary_cell -class wrapper \
        -ports port_list -type WC_D1
```

If a port meets the reuse threshold and you want to manually exclude some port-associated registers as shared wrapper cells, use the following command:

```
fc_shell> set_boundary_cell -class wrapper
        -exclude {IO_register_cell_names} -ports {port_name}
```

This command causes the registers listed in `IO_register_cell_names` to be excluded for the port `port_name`. If a register is excluded for the specified port but qualifies to be wrapped for another port, the register is still wrapped.

To manually exclude some registers as shared wrapper cells for all associated ports that meet the reuse threshold, omit the `-ports` option:

```
fc_shell> set_boundary_cell -class wrapper
               -exclude {IO_register_cell_names}
```

Note:

The `-exclude` option of the `set_boundary_cell` command can cause a port to become partially wrapped.

Special Cases for Register Reuse

In some cases, ports are associated with logic constructs that require special-case handling for register reuse.

Wrapping Ports Associated With CTL-Modeled Cells

If your design contains CTL-modeled synchronizer registers, they can be reused as shared wrapper cells.

Other CTL-modeled cells, such as memories or DFT-inserted cores, cannot be used as shared wrapper cells. When a port is associated with a CTL-modeled cell, as shown in the examples in [Figure 95](#) and [Figure 96](#), a dedicated wrapper cell is added to the port.

Figure 95 Core Wrapping of Input Ports Associated With CTL-Modeled Cells

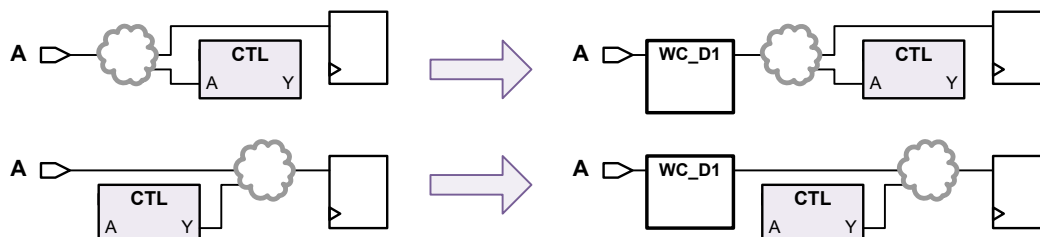
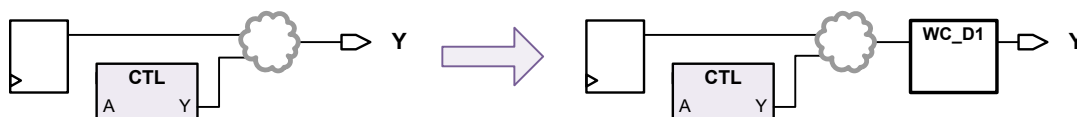


Figure 96 Core Wrapping of Output Ports Associated With CTL-Modeled Cells



In addition, a warning message is issued:

```
Information: Port 'DACK' is connected to cell 'IP_CORE', which is
             represented
             by a CTL model; a dedicated wrapper cell is added to the port.
             (DFT-3109)
```

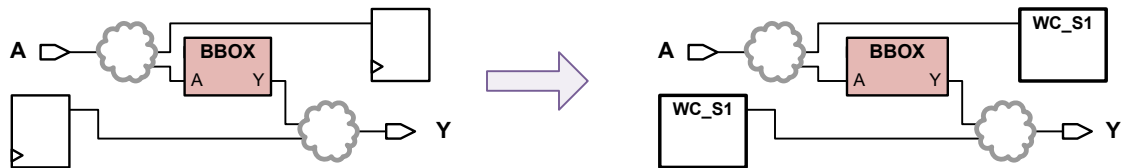
The scan clocks of the CTL-modeled cell are considered when determining the dominant system clock to use for the dedicated wrapper cell.

If the CTL-modeled cell has a netlist and there is no path from the port to a register inside the CTL-modeled cell, this behavior does not apply. In other words, connections to clock, reset, or DFT-related pins of a CTL-modeled cell do not force a dedicated wrapper cell.

Port Wrapping and Other Black-Box Cells

Black-box cells without CTL models do not affect core wrapping. Wrapper cells are not added unless needed by other design logic. See [Figure 97](#).

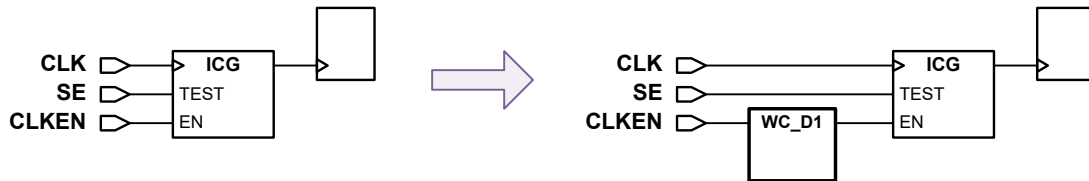
Figure 97 Black-Box Cell With No CTL Model



Wrapping Ports Associated With Clock-Gating Cells

A dedicated wrapper cell is added to an input port that drives the enable signal of a clock-gating cell. In [Figure 98](#), an integrated clock-gating cell has both a functional enable pin and a test-mode pin. A dedicated wrapper cell is added for the functional enable signal driven by input port CLKEN. No wrapper cells are inserted for the clock signal or the global test-mode signal.

Figure 98 Integrated Clock Gating Cell Enable Signals



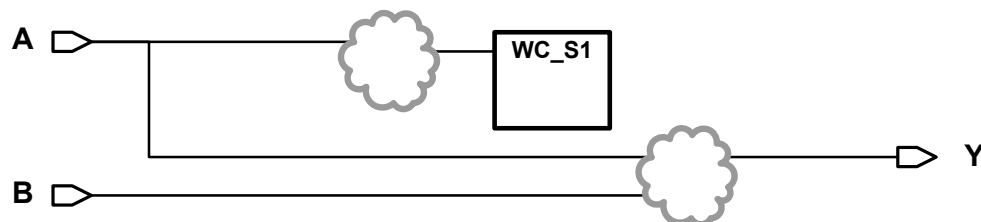
A warning message is issued if such a clock-gating enable signal is detected:

```
Information: Port 'CLKEN' is connected to a clock gating cell 'UICG';
a dedicated wrapper cell is added to the port. (DFT-3111)
```

Port Wrapping and Feedthrough Paths

Feedthrough paths do not affect core wrapping. Wrapper cells are not added unless needed by other design logic. See [Figure 99](#).

Figure 99 Combinational Feedthrough Paths



An information message is issued if a feedthrough port is detected and no dedicated wrapper cell is manually specified for the port:

Using Dedicated Wrapper Cells

Dedicated wrapper cells are used for any ports that exceed the reuse or combinational depth thresholds. You can also manually force the use of a dedicated wrapper cell with the following command:

```
fc_shell> set_boundary_cell -class wrapper \
  -ports port_list -type WC_D1
```

The following rules guide the selection of a clock for dedicated wrapper cells in the maximized reuse flow:

- If a wrapper clock is specified using the command `set_dft_signal -view spec -type wrp_clock`, then it is used for dedicated wrapper cells added due to port connected to a clock gating enable signal or CTL instance.
- A port's dedicated wrapper cell added due to crossing reuse or depth threshold, uses the same clock as the flip-flops associated with that port.

Configuring Dedicated Wrapper Cell Clocks

There are two ways to override automatic clock selection for dedicated wrapper cells:

- To specify a particular wrapper clock to be used for the dedicated wrapper cells of specific ports, use the following command:

```
fc_shell> set_boundary_cell -class wrapper -type WC_D1 \
  -shift_clk clock_name -ports port_name
```

- To specify that a dedicated wrapper clock is to be used for all dedicated wrapper cells, use the following command:

```
fc_shell> set_wrapper_configuration \
  -use_system_clock_for_dedicated_wrp_cells disable
```

See Also

- [Defining Dedicated Wrapper Clock Signals on page 169](#) for more information about defining dedicated wrapper clock signals

Including Additional Scan Cells in Input and Output Wrapper Chains

When wrapping a core, you can include additional scan cells in the input and output wrapper chains by using the `-input_wrapper_cells` and `-output_wrapper_cells` options, respectively. For example,

```
fc_shell> set_wrapper_configuration \
    -input_wrapper_cells \
        [get_object_name [get_cells IN_CFG*reg]] \
    -output_wrapper_cells \
        [get_object_name [get_cells OUT_CFG*reg]]
```

The specified cells are reclassified from internal scan cells to shared wrapper cells in all test modes.

Note:

Collections are not supported, although you can use the `get_object_name` command to convert a collection to a list.

Using the Pipelined Scan-Enable Feature

By default, two signals are used to enable shifts for designs using wrapper cells:

- `test_se` – Pipelined, used for internal flip-flops
- `wrp_shift` – Pipelined, used for wrapper flip-flops (shared or dedicated)

When the pipelined scan-enable feature is enabled, pipelined scan-enable structures are added to both of these signals. A pipelined scan-enable cell is used for each clock domain used by the input and output wrapper cells, with pipeline registers clocked by that clock. Pipelined scan-enable cells are not shared between input and output wrapper cells.

For example, if the design has two clock domains driving input wrapper cells and three clock domains driving output wrapper cells and if the number of wrapper cells within each of these domains meets the required pipeline fanout limit, a total of five pipelined scan-enable cells are created. Two of the pipelined scan-enable cells drive input wrapper cells; the remaining three pipelined scan-enable cells drive output wrapper cells. To create this pipelined structure, use the following commands:

```
fc_shell> set_dft_configuration -wrapper enable

fc_shell> set_scan_configuration \
    -pipeline_scan_enable true -pipeline_fanout_limit P
```

Pipelined scan-enable cells are not shared between internal cells, input wrapper cells, or output wrapper cells.

Note:

A single LOSPipelineEnable DFT signal is used for both internal and wrapper chains for all of these scenarios.

See Also

- [The Pipelined Scan-Enable Architecture on page 129](#) for more information about the pipelined scan-enable feature

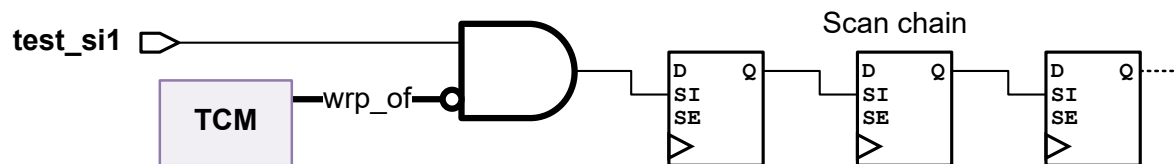
Low-Power Maximized Reuse Features

The following topics describe low-power features available in the maximized reuse flow.

Loading Constant Core Scan Data in EXTEST Mode

In the maximized reuse flow, to minimize power consumption in wrp_of (EXTEST) mode, all core scan chain cells are loaded with logic 0. This reduces toggle activity inside the block during outward-facing (EXTEST) modes. To accomplish this, the logic shown in [Figure 100](#) drives the first scan cell inputs of the internal scan chains.

Figure 100 Scan Data Gating Logic for Low-Power EXTEST



This feature is part of the wrapped core scan architecture; there is no option that controls it.

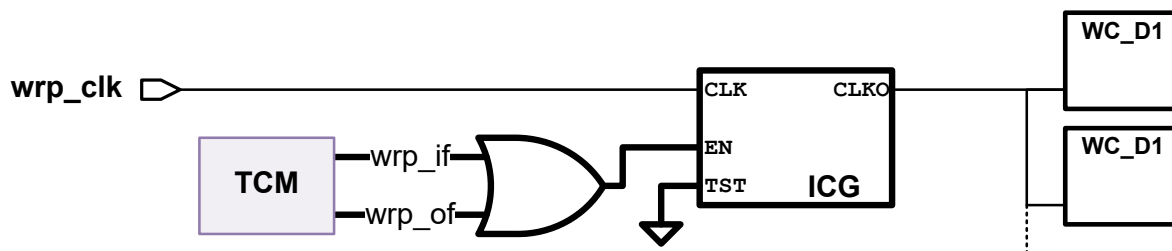
Gating Dedicated Wrapper Cell Clocks in Non-Wrapper Modes

The tool can insert clock-gating cells to disable the clock used for dedicated wrapper cells when a wrapper mode is not active. To do this, use the following command:

```
fc_shell> set_wrapper_configuration \  
-gate_dedicated_wrapper_cell_clk enable
```

The tool adds a clock-gating cell to each dedicated wrapper cell clock source, including any scan clocks used for dedicated wrapper cells, as shown in [Figure 101](#). Shared wrapper cells and scan cells are not affected, even if they use the same clock as a dedicated wrapper cell.

Figure 101 Clock Gating Logic for Low-Power Dedicated Clock Operation



By default, the clock-gating logic uses discrete latch cells. To use an integrated clock-gating cell instead, set the required library cell reference (without the library name) by using the `dft.test_icg_p_ref_for_dft -value <lib_cell_name>` application option.

Gating Scan and Wrapper Cell Clocks in Wrapper Modes

The tool can use integrated clock-gating cells to disable the clock to unused scan and wrapper cells in the `wrp_of` (EXTEST) and `wrp_safe` (SAFE) wrapper modes, as shown in Table 6.

Table 6 Gated-Clock Behaviors in Wrapper and Non-Wrapper Test Modes

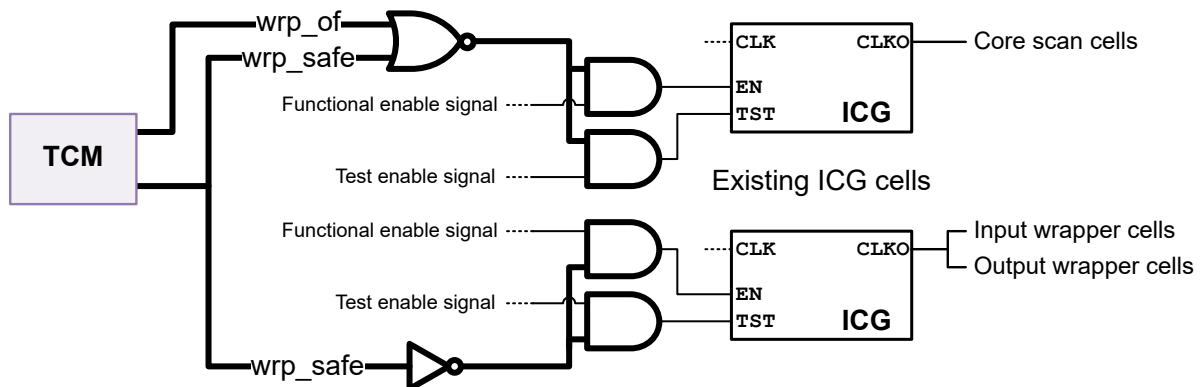
Test mode	Clock disabled for input wrapper cells?	Clock disabled for scan cells?	Clock disabled for output wrapper cells?
INTEST (<code>wrp_if</code>)	No	No	No
EXTEST (<code>wrp_of</code>)	No	Yes	No
SAFE (<code>wrp_safe</code>)	Yes	Yes	Yes
All other test modes	No	No	No
Mission mode	No	No	No

To modify only existing integrated clock-gating cells in the design, use the following command:

```
fc_shell> set_wrapper_configuration \
    -gate_cells existing_cg
```

Figure 102 highlights the logic added to existing clock-gating cells by DFT insertion. The functional enable and test enable signals are both negated to disable the clock. The `existing_cg` mode leaves the clock tree unchanged.

Figure 102 Clock-Disabling Logic for Existing Clock-Gating Cells



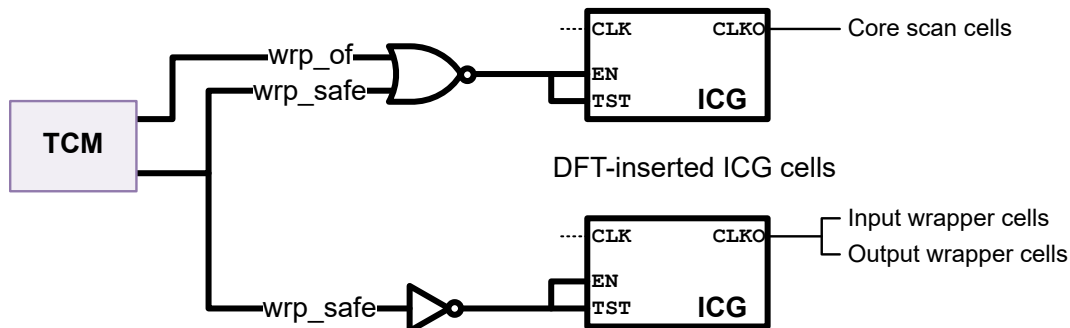
If an existing clock-gating cell gates multiple cell types, that gated clock signal is disabled only when all downstream clocked cells can be gated.

To apply clock-disabling logic to all wrapper and scan cells in the design for more aggressive power reduction, use the following command:

```
fc_shell> set_wrapper_configuration \
        -gate_cells all
```

Existing integrated clock-gating cells are modified as previously described. In addition, new integrated clock-gating cells are added to wrapper and scan cells which are not gated. Figure 103 highlights the clock-gating cells added by DFT insertion. The tool inserts a clock-gating cell for each group of same-edge cells clocked by a common hierarchical net. It does not trace backward through buffers and inverters to find logically identical nets.

Figure 103 Clock-Disabling Logic Added for Ungated Cells



You must specify the required integrated clock-gating library cell references (without the library name) using the `dft.test_icg_p_ref_for_dft` application option for rising-edge cells and the `dft.test_icg_n_ref_for_dft` application option for falling-edge cells (if present). Otherwise, DFT insertion does not add clock-gating logic.

Note the following limitation of low-power wrapper and scan cell clock gating:

- Falling-edge scan cells use the `test_icg_p_ref_for_dft` application option instead of the `test_icg_n_ref_for_dft` application option.

Hierarchical Core Wrapping

The hierarchical core wrapping feature can be used to build a core that contains only wrapper and core chains, but does not contain any test control module (TCM) or wrapper mode logic. This core can then be used at a higher level of integration with other wrapper chains or compression architectures.

To enable hierarchical wrapping, use the following commands at the core level:

```
fc_shell> set_dft_configuration -wrapper enable

fc_shell> set_wrapper_configuration \
           -hier_wrapping enable
```

By default, hierarchical wrapping is disabled.

When hierarchical wrapping is enabled, a single mode, `Internal_scan`, is created. This mode creates both wrapper chains and core internal chains. In addition, the following core-level input ports and signals are required:

- Four separate shift signals are required to control input wrapper chains, output wrapper chains, and core internal chains.
- If safe states are specified, separate input and output safe control signals are required for input and output wrapper cells.
- In the maximized reuse flow, shared wrapper cells do not use a capture signal. However, if there are dedicated wrapper cells in the input and output wrapper chains, separate input and output capture-enable signals are required for input and output wrapper cells.

You can define the signals as follows:

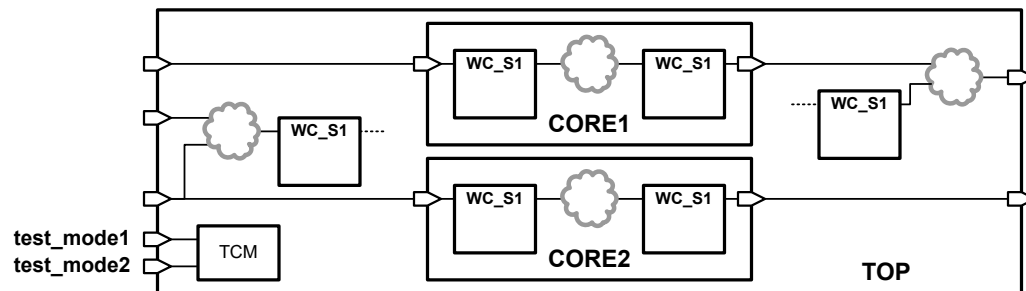
```
fc_shell> set_dft_signal -view spec -type input_wrp_shift -port port_name
fc_shell> set_dft_signal -view spec -type output_wrp_shift
           -port port_name
fc_shell> set_dft_signal -view spec -type wrp_ded_capture_in \
           -port port_name
fc_shell> set_dft_signal -view spec -type wrp_ded_capture_out \
           -port port_name
fc_shell> set_dft_signal -view spec -type wrp_safe_in -port port_name
fc_shell> set_dft_signal -view spec -type wrp_safe_out -port port_name
```


Note:

Multiple test modes are not allowed at the core level when using hierarchical wrapping to create the core.

You can guide the tool to include the core wrapper chains at the top level using the command `set_wrapper_configuration -input_wrapper_cells {} -output_wrapper_cells {}`. The tool creates the normal set of wrapper test modes, selected by a test control module, according to user specifications. As with typical wrapping flows, multiple test modes including scan compression are allowed at this level.

Figure 104 Integrating Wrapper-Only Cores at a Higher Level



Core-level input wrapper chains are mixed with integration-level input wrapper cells, and core-level output wrapper chains are mixed with integration-level output wrapper cells by using the command `set_wrapper_configuration -input_wrapper_cells {} -output_wrapper_cells {}`.

You can manually connect the core-level wrapper shift, capture, and safe control signals to appropriate integration-level wrapper logic.

Limitations of the Maximized Reuse Flow

Note the following limitations related to the behavior of the scan-enable signal in capture mode:

- Preexisting scan-enable control of clock-gating cells

If the design has I/O registers controlled by clock-gating cells that are controlled by the scan-enable port, these I/O registers might not shift during the capture mode operation of the INTEST and EXTEST modes because scan-enable might not be active in capture mode.

- Preexisting scan-enable control of set and reset

If the set or reset pins of the I/O registers are controlled by the scan-enable port, the I/O registers might not shift during the capture mode of operation of the INTEST and EXTEST modes because scan-enable might not be active in capture mode.

Determining Power Domains for Dedicated Wrapper Cells

By default, dedicated wrapper cells are added to the top-level power domain. To have dedicated wrapper cells added to power domains other than the top-level power domain, use the following command:

```
fc_shell> set_wrapper_configuration \
          -add_wrapper_cells_to_power_domains enable
```

When this option is enabled, wrapper cells are added according to the following rules:

- If the port is connected to an I/O register, the dedicated wrapper cell is added to the top-level power-domain hierarchy of the I/O register.
- If the port is connected to a CTL model, the top-level power-domain hierarchy of the instantiated CTL model is used as the location for the dedicated wrapper cell.
- If the port is connected to a gate, the top-level power-domain hierarchy of the gate is used as the location for the dedicated wrapper cell.
- If none of these conditions apply, the cell is added to the top-level hierarchy.

Using the set_scan_path Command With Wrapper Chains

You can use the `set_scan_path` command to control detailed aspects of wrapper chain construction. The following options are supported for wrapper chains:

```
set_scan_path
  -class wrapper scan_chain_name
  [-ordered_elements ordered_port_list]
  [-complete true | false]
  [-input_wrapper_cells_only enable | disable]
  [-output_wrapper_cells_only enable | disable]
  [-scan_enable se_port]
  [-test_mode test_mode_name]
  [-scan_data_in si_port]
  [-scan_data_out ordered_port_list]
```

You must always specify the `-class wrapper` option when using `set_scan_path` to configure wrapper chains.

You can use the `-ordered_elements` option to control the ordering of wrapper cells in the wrapper chain. Provide an ordered list of ports, and Fusion Compiler uses it to order the corresponding wrapper cells according to the specification:

```
fc_shell> set_scan_path WC1 -class wrapper \
          -ordered_elements {C B A Z Y}
```

By default, Fusion Compiler can add wrapper cells to the beginning of the specified chain. To prevent this, specify the `-complete true` option. You cannot use multiple ordered list specifications to add more wrapper cells to an already specified wrapper chain because the last specification for a chain overwrites any previous specification. Wrapper cells cannot belong to more than one chain, so if you specify a cell as belonging to more than one chain, the last specification takes precedence.

Multiple wrapper cells are inserted for three-state and bidirectional ports. You can reference these wrapper cells in the ordered list of ports by appending `/out`, `/en`, or `/in` to the name of the bidirectional or three-state port. For example, the following command specifies an ordering for ports named a, b, c, and d, where a is an input port, b is a three-state port, and c and d are bidirectional ports.

```
fc_shell> set_scan_path Wchain0 -class wrapper \
-ordered_elements [list A B/in B/out C/out \
D/out B/en C/en D/en] -complete true \
-test_mode test_mode_name
```

Note:

All of the wrapper cells for an individual three-state or bidirectional port must be in the same wrapper chain. For example, you cannot specify that the enable wrapper cell and output wrapper cell belong to different chains for a given port.

You can use the `-scan_data_in` and `-scan_data_out` options to specify the wrapper scan-in or wrapper scan-out signals for a chain:

```
fc_shell> set_dft_signal -type ScanInData -port wsi
fc_shell> set_dft_signal -type ScanDataOut -port wso
fc_shell> set_scan_path -class wrapper WC2 \
-ordered_elements [list A B C] \
-scan_data_in wsi -scan_data_out wso
```

The following commands implement separate input and output wrapper chains with separate wrapper shift signals using the `set_scan_path` command:

```
fc_shell> set_dft_signal -view spec \
-type wrp_shift -port {wrp_ishift wrp_oshift}

fc_shell> set_scan_path -class wrapper WC_inputs \
-input_wrapper_cells_only enable \
-scan_enable wrp_ishift

fc_shell> set_scan_path -class wrapper WC_outputs \
-output_wrapper_cells_only enable \
-scan_enable wrp_oshift
```

Note:

For hierarchical core wrapping, you must enable only the `-input_wrapper_cells_only` or `-output_wrapper_cells_only` option at a time.

A `set_scan_path` specification applied with the `-class wrapper` option and without the `-test_mode` option applies to all wrapper modes. You can also explicitly specify the `-test_mode all` option. To apply a `set_scan_path` specification to a specific wrapper mode, you must predefine the wrapper mode before referencing it. For more information, see .

For more information about the `set_scan_path` command, see the man page.

Enabling Soft Core Wrapping

Core wrapper insertion supports wrapping the entire design by adding wrapper cells to the design interface. For some of the sub hierarchies, you can add wrapper logic inside the hierarchies to isolate those hierarchies. Each user-specified soft hierarchy is considered as a standalone core to be tested with all ports wrapped.

Soft core wrapping such as DFT core wrapping allows you to wrap top-level ports along with soft core hierarchies. You can use partition-based core wrapping to insert core wrappers on user-specified soft hierarchies.

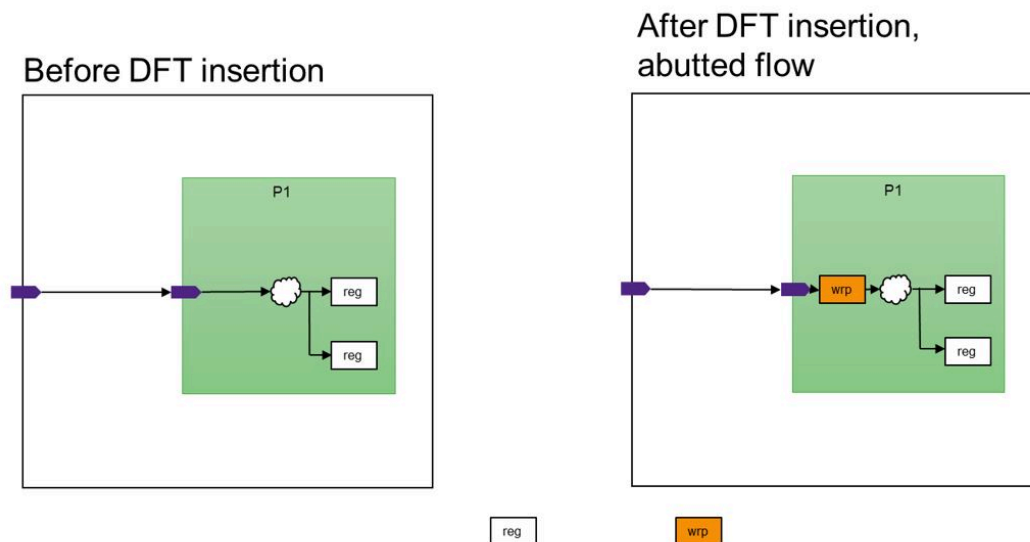
Soft core wrapping can be enabled by defining partitions with wrapper enabled, and then specifying wrapper configurations for each partition.

Enabling Soft Core Wrapping for Abutted Flow

Abutted flow is used to enable wrapping only at the soft-core level and disable wrapping at the parent core level. Scan stitching is still supported at the parent level.

If a port is directly connected to a soft core, you can insert a shared or dedicated wrapper cell for the soft cores inside the partition hierarchy and skip core wrapping at the top-level as shown in the following figure.

Figure 105 Soft Core Wrapping for Abutted Flow



Abutted soft core wrapping is enabled using the following application option:

```
set_app_options -name dft.test_wrp_soft_core_abutted_flow -value true.
```

The following example script is created for a design which has 3 instances, des0, des1, and des2. Based on this you can create partitions as default partition (des0), P1(des1), and P2(des2).

```
# To generate wrapper report
set_app_options -name dft.test_core_wrap_report_file -value
  compile_wrp_report.rpt

# not to consider top-level ports while soft-core wrapping is enabled
set_app_options -name dft.test_wrp_soft_core_abutted_flow -value true

#specify all DFT signals
set_dft_signal -view exist -type ScanClock -port clk_st_u0 -timing [list
  45 55]
set_dft_signal -view exist -type ScanClock -port clk_st_u1 -timing [list
  45 55]
set_dft_signal -view exist -type Reset -port rst_st_u0 -active 0
set_dft_signal -view exist -type Reset -port rst_st_u1 -active 0
set_dft_configuration -wrapper enable

# Define DFT Partitions
define_dft_partition P1 -include [list u0] -wrapper enable
define_dft_partition P2 -include [list u1] -wrapper enable

# With hierarchical core wrapping, wrapper control signals per partition
# for default partition
current_dft_partition default_partition
```

```

set_wrapper_configuration -hier_wrapping enable

# For partition P1
current_dft_partition P1

set_dft_signal -view spec -type ScanEnable -port se_des1
set_dft_signal -view spec -type input_wrp_shift -port i_shift_des1
set_dft_signal -view spec -type output_wrp_shift -port o_shift_des1
set_dft_signal -view spec -type wrp_ded_capture_in -port i_cap_en_des1
    -active 1
set_dft_signal -view spec -type wrp_ded_capture_out -port o_cap_en_des1
    -active 1
set_scan_configuration -chain_count $chains -clock_mixing mix_clocks

set chains 10
for { set i 0 } { $i < $chains } { incr i } {
    set_dft_signal -view spec -type ScanDataIn -port P1_SI$i
    set_dft_signal -view spec -type ScanDataOut -port P1_SO$i
}

set_wrapper_configuration -reuse_threshold 6 -mix_cells false
    -chain_count 4
# For partition P2
current_dft_partition P2

set_dft_signal -view spec -type ScanEnable -port se_des2
set_dft_signal -view spec -type input_wrp_shift -port i_shift_des2
set_dft_signal -view spec -type output_wrp_shift -port o_shift_des2
set_dft_signal -view spec -type wrp_ded_capture_in -port i_cap_en_des2
    -active 1
set_dft_signal -view spec -type wrp_ded_capture_out -port o_cap_en_des2
    -active 1
set_scan_configuration -chain_count $chains -clock_mixing mix_clocks
set_wrapper_configuration -reuse_threshold 6 -mix_cells false
    -chain_count 4

set chains 10
for { set i 0 } { $i < $chains } { incr i } {
    set_dft_signal -view spec -type ScanDataIn -port P2_SI$i
    set_dft_signal -view spec -type ScanDataOut -port P2_SO$i
}

create_test_protocol
dft_drc -test_mode all_dft
preview_dft
insert_dft
dft_drc -test_mode Internal_scan

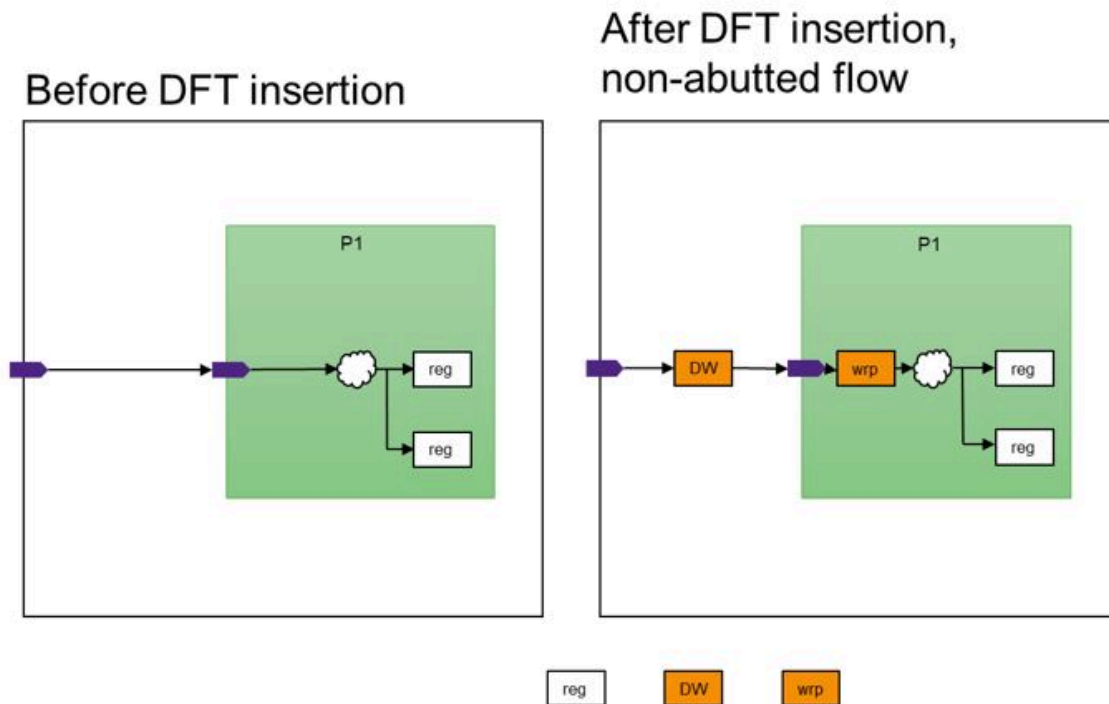
```

Enabling Soft Core Wrapping for Non-abutted Flow

Soft core wrapping for non-abutted flow allows ports at the top-level design to be wrapped along with soft cores.

If a port is directly connected to a soft core, you can insert a dedicated wrapper cell at the top-level design and a shared or dedicated wrapper cell for the soft cores inside the partition hierarchy as shown in the following figure.

Figure 106 Soft Core Wrapping for Non-abutted Flow



Non-abutted soft core wrapping is enabled using the following application option:

```
set_app_options -name dft.test_wrp_soft_core_abutted_flow -value false.
```

```
# To generate wrapper report
```

```
set_app_options -name dft.test_core_wrap_report_file -value  
compile_wrp_report.rpt
```

```
# not to consider top-level ports while soft-core wrapping is enabled  
set_app_options -name dft.test_wrp_soft_core_abutted_flow -value false
```

```
#specify all DFT signals
```

```
set_dft_signal -view exist -type ScanClock -port clk_st_u0 -timing [list  
45 55]
```

```
set_dft_signal -view exist -type ScanClock -port clk_st_u1 -timing [list  
45 55]
```

```
set_dft_signal -view exist -type Reset -port rst_st_u0 -active 0
```

```
set_dft_signal -view exist -type Reset -port rst_st_ul -active 0
set_dft_configuration -wrapper enable

# Define DFT Partitions
define_dft_partition P1 -include [list u0] -wrapper enable
define_dft_partition P2 -include [list u1] -wrapper enable

# With hierarchical core wrapping, wrapper control signals per partition
# for default partition
current_dft_partition default_partition

set_dft_signal -view spec -type ScanEnable -port se
set_dft_signal -view spec -type input_wrp_shift -port i_shift
set_dft_signal -view spec -type output_wrp_shift -port o_shift
set_dft_signal -view spec -type wrp_ded_capture_in -port i_cap_en -active
1
set_dft_signal -view spec -type wrp_ded_capture_out -port o_cap_en
-active 1

set_scan_configuration -chain_count $chains -clock_mixing mix_clocks
set_wrapper_configuration -hier_wrapping enable
set_wrapper_configuration -reuse_threshold -1 -mix_cells false
-chain_count 4

set chains 4
for { set i 0 } { $i < $chains } { incr i } {
    set_dft_signal -view spec -type ScanDataIn -port SI$i
    set_dft_signal -view spec -type ScanDataOut -port SO$i
}

# For partition P1
current_dft_partition P1

set_dft_signal -view spec -type ScanEnable -port se_des1
set_dft_signal -view spec -type input_wrp_shift -port i_shift_des1
set_dft_signal -view spec -type output_wrp_shift -port o_shift_des1
set_dft_signal -view spec -type wrp_ded_capture_in -port i_cap_en_des1
-active 1
set_dft_signal -view spec -type wrp_ded_capture_out -port o_cap_en_des1
-active 1
set_scan_configuration -chain_count $chains -clock_mixing mix_clocks

set chains 10
for { set i 0 } { $i < $chains } { incr i } {
    set_dft_signal -view spec -type ScanDataIn -port P1_SI$i
    set_dft_signal -view spec -type ScanDataOut -port P1_SO$i
}

set_wrapper_configuration -reuse_threshold 6 -mix_cells false
-chain_count 4

# For partition P2
current_dft_partition P2
```



```
set_dft_signal -view spec -type ScanEnable -port se_des2
set_dft_signal -view spec -type input_wrp_shift -port i_shift_des2
set_dft_signal -view spec -type output_wrp_shift -port o_shift_des2
set_dft_signal -view spec -type wrp_ded_capture_in -port i_cap_en_des2
    -active 1
set_dft_signal -view spec -type wrp_ded_capture_out -port o_cap_en_des2
    -active 1
set_scan_configuration -chain_count $chains -clock_mixing mix_clocks
set_wrapper_configuration -reuse_threshold 6 -mix_cells false
    -chain_count 4

set chains 10
for { set i 0 } { $i < $chains } { incr i } {
    set_dft_signal -view spec -type ScanDataIn -port P2_SI$i
    set_dft_signal -view spec -type ScanDataOut -port P2_SO$i
}

create_test_protocol
dft_drc -test_mode all_dft
preview_dft
insert_dft
dft_drc -test_mode Internal_scan
```

Previewing the Wrapper Cells

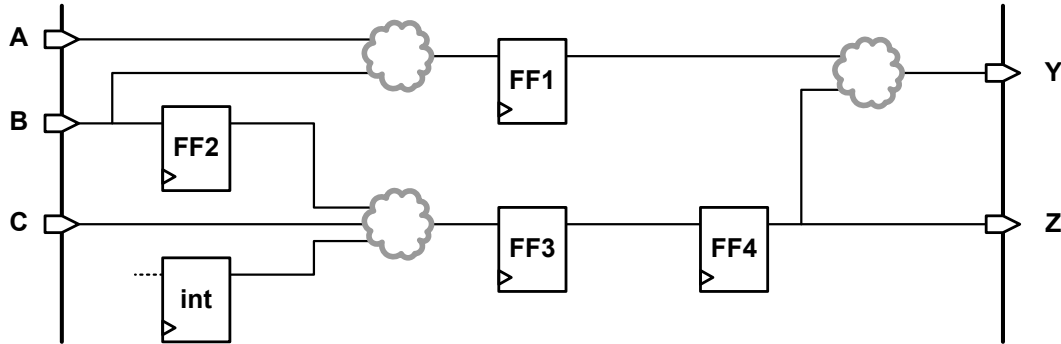
After you have configured your wrapper chain and wrapper cells, use the `preview_dft` command to preview the core wrapping configuration.

For all ports that have wrapper cells, the following information is reported:

- Index number of wrapper cell (used to indicate duplicate input/output wrapper cells and to reference control cells)
- Port name
- Wrapper cell type: dedicated or shared
- Function of wrapper cell: input, output, three-state, or control
- Wrapper cell implementation: swapped-in (SWP) or in-place (INP)
- Safe value
- Wrapper cell clock
- Wrapper cell instance name

Consider the following example design:

Figure 107 Example Design for Core Wrapping



Example 10 shows the wrapper chain preview report for this design.

Example 10 Core Wrapping Report

```

****Test wrapper plan report
Design : top
*****
Reuse Threshold : 5
Wrapper Length : 5

```

Index	Port	Wrapper Function	Cell Type	Control Cell	Impl	Safe Value	Wrapper Clock	Cell Name
4	C	input	WC_S1	-	INP	0	CLK	FF3_reg
3	A	input	WC_S1	-	INP	0	CLK	FF1_reg (*)
3	B	input	WC_S1	-	INP	0	CLK	FF1_reg (*)
3	Y	output	WC_S1	-	INP	0	CLK	FF1_reg (*)
2	Y	output	WC_S1	-	INP	0	CLK	FF4_reg (d)
2	Z	output	WC_S1	-	INP	0	CLK	FF4_reg (d)
1	B	input	WC_S1	-	INP	0	CLK	FF2_reg (*)
1	C	output	WC_S1	-	INP	0	CLK	FF2_reg (*)
0	C	output	WC_S1	-	INP	0	CLK	int_reg

Number of ports not wrapped : 26

```

ATECLK
CLK
OCC_bypass
OCC_reset
SE
...

```

The wrapper report uses annotations to indicate when a shared wrapper cell is associated with multiple ports. Note the following reporting conventions:

- Shared registers that are common to multiple input ports and multiple output ports are shown with the letter d (d) for all subsequent entries after the first register entry; these marked entries do not count against the total wrapped register count.
- Internal registers in the fanin to an input shared register are classified as output registers (because they drive values that are captured during outward-facing test).
- Shared registers that are common to both an input port and an output port but are used only as input wrapper cells are classified as input registers and shown with an asterisk (*) for the output port.
- Shared input registers that are in the fanin to another shared input register are classified as input registers and shown with the annotation (*) (i) for the fanin to the shared input register.

Post DFT DRC Rule Checks

You can perform post DFT DRC for the wrapper modes. To verify that the input wrapper chains shift in wrp_if (INTEST) mode, use the following commands:

```
fc_shell> dft_drc-test_mode wrp_if
```

To verify that the output wrapper chains shift in wrp_of (EXTEST) mode, run the following commands:

```
fc_shell> dft_drc -test_mode wrp_of
```

Creating an EXTEST-Only Core Netlist

During core creation, you can create and write out an additional version of the core netlist that contains only the logic needed for operation in outward-facing (EXTEST) test modes. This EXTEST-only core netlist significantly reduces the memory requirements for pattern generation of top-level test modes in TestMAX ATPG and for pattern simulation.

After inserting DFT and writing the full netlist files, protocol files, and other design files, execute the following command:

```
fc_shell> create_dft_netlist -type extest
```

This command removes all logic in the design except for the following:

- Wrapper chains
- Interface logic between wrapper chains and I/O ports
- Wrapper chain control logic

- Test-mode decode logic
- IEEE 1500 controller logic
- Any other logic required for EXTEST mode operation

After removing the unnecessary logic with the `create_dft_netlist` command, write out an EXTEST-only Verilog netlist file using the `write_verilog` command. Because the `create_dft_netlist` command removes logic from the design in memory, these should be the last steps in your core creation script.

You can use this EXTEST-only core netlist in any TestMAX ATPG pattern generation run where the core operates in its outward-facing test mode. All necessary logic is retained so that the core operates properly when exercised by the test protocols.

Note the following limitations:

- The `create_dft_netlist` command can be run only after DFT is inserted.
- The `create_dft_netlist` command uses the EXTEST mode CTL stored in the NDM for preserving the required cells. Any structural design modifications made after DFT insertion are not considered during netlist processing.
- Cores with multiple EXTEST test modes are not supported.

Defining Core Wrapping Test Modes

In Fusion Compiler, when core wrapping is enabled, you must define the list of test modes using the `define_test_mode` command; the wrapper modes are not automatically created.

[Table 7](#) shows the test mode usage types you can specify with the `-usage` option of the `define_test_mode` command. You can define one or more test modes for each usage.

Table 7

Test mode usage	Test mode description	Default test mode name
wrp_if	Inward-facing uncompressed scan mode	Internal_scan
scan_compression	Inward-facing compressed scan mode	ScanCompression_mode
wrp_of	Outward-facing uncompressed scan mode	wrp_of
scan	Unwrapped uncompressed scan mode	

You can configure individual user-defined test modes using the `-test_mode` option of DFT configuration commands. The `-test_mode` option cannot reference a default test mode unless that mode name was explicitly defined with the `define_test_mode` command.

Example 11 defines a set of user-defined core wrapping test modes. The inward-facing compressed scan mode, defined with `-usage scan_compression`, specifies the base mode as the inward-facing uncompressed scan mode, defined with `-usage wrp_if`.

Example 11 User-Defined Core Wrapping Test Modes

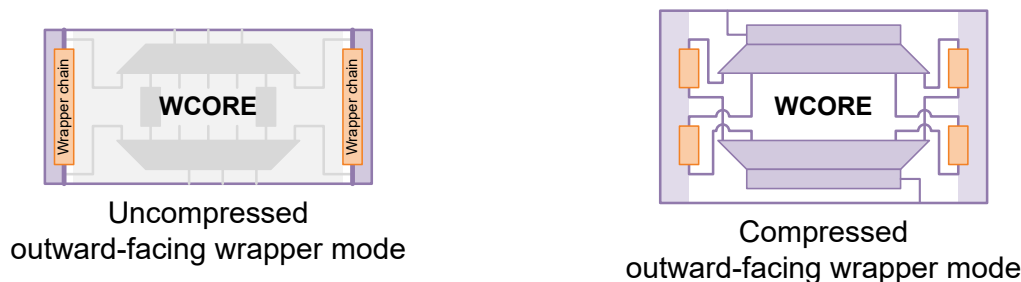
```
# define test modes
define_test_mode MY_INTEST      -usage wrp_if
define_test_mode MY_INTEST_COMP -usage scan_compression
define_test_mode MY_EXTEST      -usage wrp_of

# configure scan modes
set_scan_compression_configuration \
  -test_mode MY_INTEST_COMP \
  -input 4 -output 4 \
  -chain_count 12
set_scan_configuration \
  -test_mode MY_INTEST \
  -chain_count 4
set_wrapper_configuration \
  -test_mode MY_EXTEST \ -chain_count 4
```

Creating Compressed EXTEST Core Wrapping Test Modes

When you use DFTMAX Ultra streaming compression, you can compress both your inward-facing (INTEST) and outward-facing (EXTEST) test modes. This helps reduce scan I/O requirements when you have many wrapped cores instantiated at a higher level.

Figure 108 Uncompressed and Compressed Outward-Facing (EXTEST) Wrapper Modes



To do this, define a streaming compression mode that references an uncompressed outward-facing mode as its base mode. For example,

```
# define test modes
define_test_mode wrp_if -usage wrp_if
```

```
define_test_mode wrp_if_comp -usage streaming_compression
define_test_mode wrp_of -usage wrp_of
define_test_mode wrp_of_comp -usage {wrp_of streaming_compression}

# configure inward-facing standard and compressed modes
set_scan_configuration \
  -test_mode wrp_if \
  -chain_count 2
set_scan_compression_configuration \
  -test_mode wrp_if_comp -streaming true \
  -chain_count 4

# configure outward-facing standard and compressed modes
set_wrapper_configuration \
  -test_mode wrp_of \
  -chain_count 2
set_scan_compression_configuration \
  -test_mode wrp_of_comp -streaming true \
  -chain_count 4 -inputs 1 -outputs 1
```

Wrapper chain count and length is configured as follows:

- For the compressed EXTEST mode, wrapper chains are always length-balanced using the configuration specified by the `set_scan_compression_configuration` command. The `set_wrapper_configuration` command should not be applied to this mode.
- For the uncompressed EXTEST mode and all other wrapper modes, wrapper chains follow the rules described in [Controlling Wrapper Chain Count on page 169](#).

Keep in mind that unlike uncompressed EXTEST chains, compressed EXTEST chains cannot be concatenated with other chains for rebalancing at higher levels. Just as with other compression modes, compressed EXTEST modes require dedicated codec scan I/O connections during core integration.

Alternatively, you can create many uncompressed wrapper chains at the core level, then concatenate or compress them along with other chains at a higher level in standard or compressed scan modes, respectively.

Note:

Because this feature is intended for few scan I/O pins, it requires DFTMAX Ultra streaming compression. Other compression technologies are not supported.

Wrapping Cores With DFT Partitions

If you are using DFT partitions, the tool creates wrapper chains within each partition and assigns wrapper cells to partitions as follows:

- Shared wrapper cells are functional scan cells. They inherently belong to a DFT partition, as specified by the DFT partition configuration.
- Dedicated wrapper cells are added to the default partition.

The following example shows per-partition wrapper chain configuration:

```
# enable core wrapping
set_dft_configuration -wrapper enable
set_wrapper_configuration

# define DFT partitions
define_dft_partition P1 -include ...
define_dft_partition P2 -include ...

# configure DFT partition P1
current_dft_partition P1
set_scan_configuration -chain_count 4
set_wrapper_configuration -chain_count 2

# configure DFT partition P2
current_dft_partition P2
set_scan_configuration -chain_count 4
set_wrapper_configuration -chain_count 2
```

Note the following limitation:

- When you use DFT partitions, separate wrapper chains are created within each partition. Wrapper chains cannot span across partitions.

Excluding Test Pins from Wrapping Across DFT Partitions

This feature enables you to automatically exclude specific test pins such as scan clock (MasterClock), reset, constants, scanEnable, and TestMode — from core wrapping across DFT partitions in a soft core wrapping flow.

In complex designs, test signals may cross partition boundaries or connect to multiple soft cores. Without this feature, wrapper cells may be incorrectly inserted on test pins, leading to unnecessary manual exclusions and potential errors.

By automating the exclusion of these test pins, the feature streamlines the DFT flow and reduces manual intervention ensuring consistent and predictable wrapper analysis that supports complex signal paths, including those with combinational logic and clock-gating cells.

To enable the automatic exclusion of test pins from core wrapping across DFT partitions, use the `dft.test_core_wrap_exclude_soft_core_pin_wrap` application option. The default is `false`.

The exclusion logic works as follows:

- The tool traverses the fanout cone of each DFT test signal port or pin within every partition.
- It skips over combinational logic (buffers, inverters, multiplexers), clock-gating cells, and isolation/level shifter cells.
- It stops traversal at sequential logic (flip-flops, latches), memory, pad, BAM block, or CTL cells.
- If a test signal fans out to a pin within a soft core hierarchy, that pin is automatically excluded from wrapper insertion.
- The logic is generic and applies to all supported DFT test signals.

Limitations

Note the following limitations:

- The automatic test pin exclusion feature is available only for soft cores.
- The traversal logic only bypasses combinational logic, clock-gating cells, and isolation/level shifter cells. It does not bypass sequential logic, memory, pad, BAM block, or CTL cells.

Wrapping Cores With Multibit Registers

You can core-wrap designs that use multibit registers.

Multibit registers have multiple state elements (bits) that are individually addressable but share clock and scan signals. This reduces power consumption and routing congestion.

The Maximized Reuse Core Wrapper Flow

In the maximized reuse flow, multibit registers associated with ports can be shared wrapper cells. For best results, enable and configure maximized reuse core wrapping *before* the first `compile_fusion` command.

This informs the multibit banking algorithms that you need to perform maximized reuse core wrapping during DFT insertion. The tool builds each multibit register from single-bit

registers of the same type—input registers, output registers, or core registers—so they can be stitched into the corresponding wrapper or core scan chains.

[Table 8](#) lists the wrapper configuration commands used by multibit banking.

Table 8 *Wrapper Configuration Commands Used by Multibit Banking*

Wrapper configuration command	Required?
<code>set_dft_configuration -wrapper enable</code>	Required
<code>set_wrapper_configuration -reuse_threshold</code>	Required
<code>set_boundary_cell</code>	Required
<code>set_wrapper_configuration \</code> <code>-chain_count/mix_cells/all other options</code>	Not Required

Specifying the threshold values allows multibit banking to determine any ports that get dedicated wrapper cells by exceeding the thresholds. This removes the multibit banking register-type restrictions for registers associated with those ports.

To prevent multibit registers from being used as shared wrapper cells, which forces dedicated wrapper cells on any ports associated with them, set the following application option:

```
fc_shell> set_app_options -name
dft.test_soc_core_wrap_allow_multibit_ioregs -value true
```

The Dedicated Core Wrapper Flow

In the dedicated core wrapper flow, multibit registers cannot be wrapper cells. Ports associated with multibit registers always get a dedicated wrapper cell.

Top-Down Flat Testing With Transparent Wrapped Cores

In some cases (such as for IDDQ testing), you might want to make core wrapper chains transparent to perform top-down flat testing of the full chip. This can be done by implementing and using a *transparent mode* in your wrapped cores.

Transparent modes are described in the following topics:

- [Introduction to Transparent Test Modes](#)
- [Defining Core-Level Transparent Test Modes](#)

- [Defining Top-Level Flat Test Modes](#)
- [Limitations](#)

Introduction to Transparent Test Modes

Wrapped cores allow a design to be tested hierarchically. Inward-facing test modes test the logic inside a core, and outward-facing test modes test the logic surrounding a core. In both types of modes, the wrapper chain is actively used to logically separate the core logic from the surrounding logic.

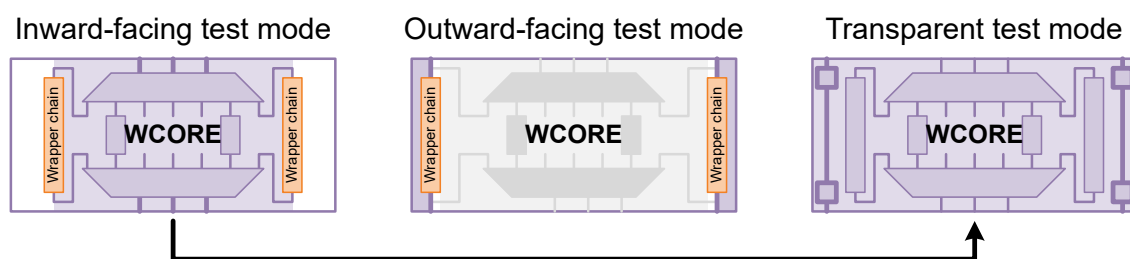
To perform top-down flat testing of a design with wrapped cores, wrapped cores must provide an additional test mode where the wrapper chain is logically transparent. This core-level test mode is called a *transparent* test mode. At the top level, wrapped cores can be placed into their transparent modes to perform top-down flat testing of the entire design. This top-level test mode is called a *flat* test mode.

At the core level, a transparent test mode is defined as an extension of an inward-facing standard scan or compressed scan test mode. A transparent mode is identical to its referenced inward-facing mode, except that

- Wrapper chains are treated as regular scan chains.
- Dedicated wrapper cells are logically transparent, although their flip-flops remain in the wrapper chains and act as observe test points on I/O paths.
- Any scan compression codecs from the referenced inward-facing mode are reused.
- Feedthrough chains drive the outward-facing wrapper scan I/Os in transparent mode.

Figure 109 shows how the transparent mode of a wrapped core relates to the inward-facing and outward-facing test modes. Standard scan modes are not shown.

Figure 109 The Three Test Mode Types of a Wrapped Core



Transparent modes require that dedicated wrapper scan I/Os be used in the core's outward-facing modes. The tool creates and stitches placeholder chains, called *feedthrough* chains, between these outward-facing I/Os in the transparent modes. Each

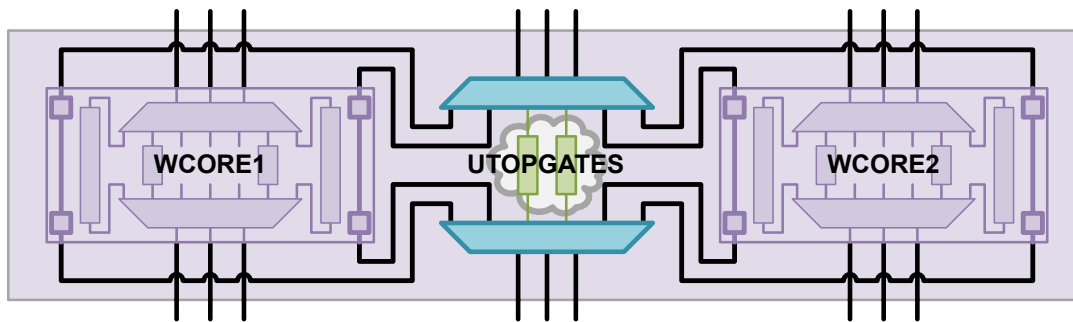
feedthrough chain consists of two scan cells that are clocked by the same head and tail clock and edge as the outward-facing wrapper chains they represent.

At the top level, a flat test mode is defined as an extension of a top-level-only mode, which tests only top-level logic with wrapped cores in outward-facing mode. A flat mode is identical to its underlying top-level-only mode, except that:

- You schedule wrapped cores in transparent mode instead of outward-facing mode.
- Any top-level scan compression codecs from the underlying top-level-only mode are reused.

Figure 110 shows how wrapped cores, placed in their transparent modes, are tested along with the top-level logic in a top-level flat test mode.

Figure 110 Performing Top-Down Flat Testing Using Wrapped Cores in Transparent Mode



The feedthrough chains allow the codec from the top-level-only mode to be reused in the flat test mode. Feedthrough chains are used only in flat test modes where they are compressed by a top-level codec reused from the underlying top-level-only mode.

Feedthrough chains are unused in

- Top-level flat standard scan modes
- Top-level flat compressed scan modes without top-level codecs that compress the core wrapper connections
- Flat modes where the top-level codec participates in codec I/O sharing

See [Limitations](#).

Defining Core-Level Transparent Test Modes

To define a transparent test mode at the core level, use the `-transparent_mode_of` option of the `define_test_mode` command to specify a previously defined inward-facing

standard scan or compressed scan test mode as the parent mode. The transparent mode is derived from this parent mode. For example,

```
# define inward-facing modes
define_test_mode INWARD_STD -usage wrp_if
define_test_mode INWARD_COMP -usage scan_compression

# define outward-facing mode
define_test_mode OUTWARD_STD -usage wrp_of

# define transparent modes
define_test_mode TRANS_STD -usage wrp_if \
    -transparent_mode_of INWARD_STD
define_test_mode TRANS_COMP -usage scan_compression \
    -transparent_mode_of INWARD_COMP
```

Specify the usage of a transparent mode to be the same as its parent mode: `wrp_if` for a standard scan mode and `scan_compression` for a compressed scan mode.

To define dedicated wrapper scan I/Os for the core's outward-facing modes, use the `-test_mode` option of the `set_dft_signal` command. For example,

```
# define dedicated scan I/Os for outward-facing mode
set_dft_signal -view spec -type ScanDataIn -port SI[*] \
    -test_mode {INWARD_STD TRANS_STD INWARD_COMP TRANS_COMP}
set_dft_signal -view spec -type ScanDataOut -port SO[*] \
    -test_mode {INWARD_STD TRANS_STD INWARD_COMP TRANS_COMP}

set_dft_signal -view spec -type ScanDataIn -port WSI[*] \
    -test_mode {OUTWARD_STD}
set_dft_signal -view spec -type ScanDataOut -port WSO[*] \
    -test_mode {OUTWARD_STD}
```

For more information, see the man page for this message.

Defining Top-Level Flat Test Modes

To define a flat test mode at the top level, define a test mode that schedules the cores in their transparent mode along with the top-level logic of the current design. For example,

```
# define modes that test only cores
define_test_mode CORES_INWARD_STD -usage scan \
    -target {core1:INWARD_STD core2:INWARD_STD}
define_test_mode CORES_INWARD_COMP -usage scan_compression \
    -target {core1:INWARD_COMP core2:INWARD_COMP}

# define modes that test only top-level logic
define_test_mode TOPONLY_STD -usage scan \
    -target {top}
define_test_mode TOPONLY_COMP -usage scan_compression \
    -target {top}
```

```
# define flat test modes that test the entire design
define_test_mode TOPFLAT_STD -usage scan \
  -target {core1:TRANS_STD core2:TRANS_STD top}
define_test_mode TOPFLAT_COMP -usage scan_compression \
  -target {core1:TRANS_COMP core2:TRANS_COMP top}
```

The tool automatically identifies the standard scan or compressed scan top-level-only mode associated with each flat mode.

Do not apply any DFT configuration commands specifically to the flat test modes; they are ignored.

Limitations

Note the following limitations of top-down flat testing using transparent core-level test modes:

- DFTMAX Ultra compression is not supported.
- You must define core-level transparent and top-level flat test modes with the `define_test_mode` command; they are not created by default.
- At the core level, to create transparent modes, you must use dedicated wrapper scan I/Os in the outward-facing modes.
- At the top level, you cannot mix transparent and nontransparent core test modes in the same test mode.
- At the top level, you cannot define multiple flat top-level scan compression modes.

SCANDEF Generation for Wrapper Chains

SCANDEF generation is supported for wrapper chains. The SCANDEF information is generated as follows:

- Wrapper cells can be reordered within wrapper chains.
- Wrapper chain cells cannot be repartitioned with regular scan chain cells.
- Input wrapper cells can be repartitioned between input wrapper chains, and output wrapper cells can be repartitioned between output wrapper chains.
- If input and output wrapper cell mixing is enabled, input wrapper chain cells can be repartitioned with output wrapper chain cells.

Input and output wrapper chain mixing is enabled by default in the simple wrapper flow and disabled by default in the maximized reuse flow. You can change this setting with the `-mix_cells` option of the `set_wrapper_configuration` command.

- Each wrapper cell is represented as an ORDERED construct that ensures all logic gates for that wrapper cell are kept together. However, a shared wrapper cell in the maximized reuse flow is simply a scan cell, so it is represented as an individual scan cell instead of an ORDERED construct.

See Also

- [Chapter 12, Generating SCANDEF for Scan Reordering in Layout](#) for more information about generating SCANDEF information

Core Wrapping Scripts

The following example script illustrate the core wrapping flow in Fusion Compiler.

```
<Library creation steps>
<Analyze RTL and elaboration>
...
# Enable and configure wrapper client, set reuse threshold to -1 for
  dedicated core wrapping
set_wrapper_configuration -reuse_threshold 0 \
-use_system_clock_for_dedicated_wrp_cells enable

# Wrapper cells for specific ports
set_boundary_cell -type none -port {i_trdy_dd}
set_boundary_cell -type WC_D1 \
-port {i_addr[0] i_addr[1] i_addr[2]}

#Compile till logic_opto
compile_fusion -to logic_opto

#Enable internal pins flow
set_dft_drc_configuration -internal_pins enable

# Clocks and Asynchronous Resets
set_dft_signal -type ScanClock -port clk -timing {45 55}
set_dft_signal -type Reset -port reset -active_state 0

#Scan enable and clock gating scan enable
set_dft_signal -type ScanEnable -hookup_pin scan_interface/SE
set_dft_signal -type ScanEnable -hookup_pin scan_interface/CSE \
-usage clock_gating

#wrapper shift and clock signals
set_dft_signal -type wrp_shift -port wrp_shift
set_dft_signal -type wrp_clock -port wrp_clock
```

```
# Test mode signal definition
set_dft_signal -type TestMode -hookup_pin scan_interface/tm1
set_dft_signal -type TestMode -hookup_pin scan_interface/tm2

#Define test modes
define_test_mode intest      -usage wrp_if
define_test_mode extest      -usage wrp_of
define_test_mode scandump -usage scan
reset_test_mode

set_wrapper_configuration -chain_count 10 -mix_cells true \
    -test_mode intest
set_wrapper_configuration -chain_count 10 -mix_cells true \
    -test_mode extest

# Scan Configuration
set_scan_configuration -exclude_elements [get_cell -hier \
    -filter "<filter_criteria>"]
set_scan_configuration -chain_count 20 -test_mode uncompress \
    -clock_mixing no_mix
set_scan_configuration -chain_count 1 -test_mode scandump \
    -clock_mixing mix_clocks

# Scan in, scan out
for {set i 0} {$i < 20} {incr i} {
    set_dft_signal -view spec -type ScanDataIn -hookup_pin \
        scan_interface/si_[$i] -test_mode {intest extest}
    set_dft_signal -view spec -type ScanDataOut -hookup_pin \
        scan_interface/so_[$i] -test_mode {intest extest}

    set_dft_signal -view spec -type ScanDataIn -port SI \
        -test_mode scandump
    set_dft_signal -view spec -type ScanDataOut -port SO \
        -test_mode scandump
}

#create temp directory to store internally generated tetramax files
during_dft_drc
file delete -force tmax
file mkdir tmax
set_dft_drc_configuration -log_dir tmax
create_test_protocol

#pre-dft_drc
dft_drc -test_mode all_dft
preview_dft
insert_dft

# Perform post dft_drc
dft_drc -test_mode intest
dft_drc -test_mode extest
dft_drc -test_mode scandump
```

```
report_dft_drc_violations -test_mode intest -rule all
report_dft_drc_violations -test_mode extest -rule all
report_dft_drc_violations -test_mode scandump -rule all

#scan reordering happens in the following compile steps
compile_fusion -from initial_place -to initial_opto

# Perform post dft_drc again
dft_drc -test_mode intest
dft_drc -test_mode extest
dft_drc -test_mode scandump

# Write out scandef
write_scan_def -o <design>.scan.def
report_scan_path > scan_path.rpt

# Write out test model
write_test_model -output <design>.ctl

# Save test protocols for TestMAX ATPG
write_test_protocol -output intest.spf \
    -test_mode intest
write_test_protocol -output extest.spf \
    -test_mode extest
write_test_protocol -output scandump.spf \
    -test_mode scandump
write_verilog -hierarchy all <design>.v
```

Limitations of Core Wrapping

Note the following limitations related to the behavior of the scan-enable signal in capture mode:

- Preexisting scan-enable control of clock-gating cells

If the design has I/O registers controlled by clock-gating cells that are controlled by the scan-enable port, these I/O registers might not shift during the capture mode operation of the INTTEST and EXTEST modes because scan-enable might not be active in capture mode.

- Preexisting scan-enable control of set and reset

If the set or reset pins of the I/O registers are controlled by the scan-enable port, the I/O registers might not shift during the capture mode of operation of the INTTEST and EXTEST modes because scan-enable might not be active in capture mode.

10

Clock Gating

To reduce switching power consumption, Fusion Compiler inserts *integrated clock-gating cells* for banks of functional registers that conditionally hold their state. These clock-gating cells have test pins that ensure proper clocking behavior during test.

The following topics provide more information:

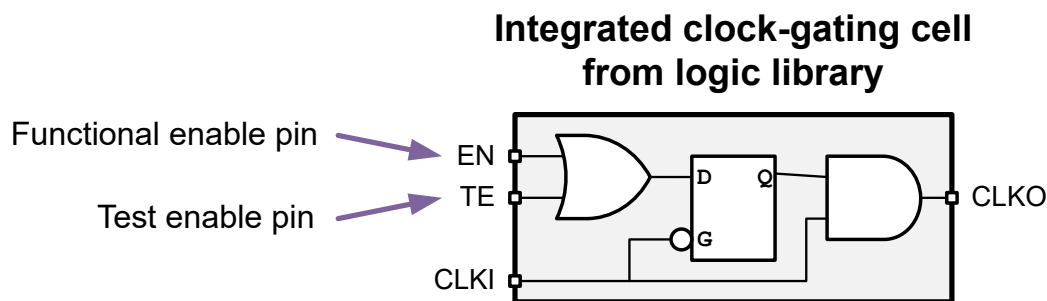
- [Clock Gating Concepts](#)
- [Configuring Clock-Gating for DFT](#)
- [Clock Gating for DFT Wrapper Cells](#)

Clock Gating Concepts

During scan shift, all registers must receive the clock signal. Correspondingly, integrated clock-gating cells have two enable pins:

- The functional enable pin is driven by functional logic; it enables the clock during functional operation.
- The test enable pin is driven by test logic; it enables the clock during test operation.

The signals are combined inside the clock-gating cell so that either one enables the clock:

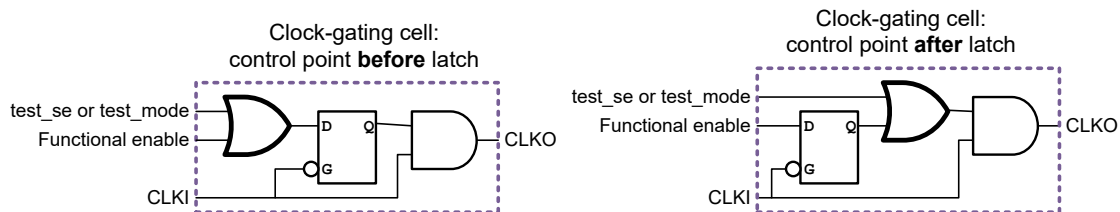


Clock-Gating Test Control Points

In a clock-gating cell, the *control point* is the logic gate that allows the clock to be enabled when its test pin is asserted. There are two degrees of freedom in control point implementation, also shown in [Figure 111](#):

- The control point can be placed before or after the latch element.
- The control signal can be driven by the scan-enable or test-mode signal.

Figure 111 Clock-Gating Control Points Before and After Latch



For most designs, the “before” control point style driven by the scan-enable signal is recommended for the following reasons:

- The “before” control point ensures that no combinational path exists from the control signal input port to the downstream clock pins of the scan cells.
- If the “after” control point is used and both phases of the clock are gated in the design, there is no time at which the control signal can cleanly toggle without truncating an active clock pulse.
- A scan-enable control signal ensures that the gated clocks are always-active during scan shift, but that the functional clock-gating paths can still be tested during scan capture.
- A test-mode control signal prevents the functional clock-gating paths from being tested, requiring additional testability logic to be inserted in the design.

Most integrated clock-gating cells are implemented using control points before the latch.

Choosing a Clock-Gating Control Point Configuration

[Table 9](#) shows the possible combinations of clock gating, clock waveforms, control signals, and control point location you can use.

Table 9 *Latched-Based Clock-Gating Configurations*

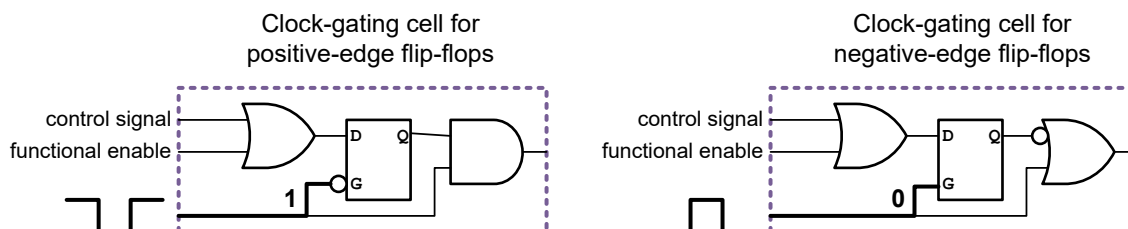
Clock-gating style	Clock waveform	Control signal	Control point location	Gated register can be scanned?
Latch-based gating for positive-edge flip-flops		test_mode	Before latch	Yes
			After latch	Yes
		scan_enable	Before latch	Yes
			After latch	Yes
		test_mode	Before latch	Yes, but requires extra initialization cycles to be manually specified for the test protocol.
			After latch	Yes
		scan_enable	Before latch	Yes, but supports only one level of clock gating.
			After latch	No
Latch-based gating for negative-edge flip-flops		test_mode	Before latch	Yes
			After latch	Yes
		scan_enable	Before latch	Yes
			After latch	Yes
		test_mode	Before latch	Yes, but requires extra initialization cycles to be manually specified for the test protocol.
			After latch	Yes
		scan_enable	Before latch	Yes, but supports only one level of clock gating.
			After latch	No

For each combination, the last column indicates whether the gated register can be included in scan chains. Four special cases, marked with footnotes, require additional consideration as described in the following section.

Initialization for Special Cases of Before-Latch Control Points

In the four special cases marked by footnotes in , the gating latch is inactive at the beginning of the test program (time = 0). This unknown latch state causes an X value to reach the gated flip-flops, which would normally prevent them from being included in scan chains. Figure 112 shows the clock waveform and clock-gating logic for these cases.

Figure 112 Special Clock-Gating Cases With Inactive Latch at Beginning of Test Clock Cycle



If you are using a scan-enable control signal, which asserts and de-asserts during every test pattern, the DRC engine performs an analysis of the clock-gating logic to verify a known state in the latch. This analysis supports only one such level of special-case clock gating, although you can have additional levels of non-special-case clock gating.

If you are using a test-mode control signal, to achieve a known state in the latch, you must add a clock pulse to the `test_setup` section of the test protocol. Use the following `set_dft_drc_configuration` command to update the `test_setup` section with the clock pulse:

```
fc_shell> set_dft_drc_configuration -clock_gating_init_cycles 1
```

If you have multiple cascaded latch-based clock-gating cells and the first latch is loaded with the test-mode signal, use the following `set_dft_drc_configuration` command to update the `test_setup` section with the specified number of clock pulses:

```
fc_shell> set_dft_drc_configuration -clock_gating_init_cycles n
```

Here, *n* equals the number of clock pulses required to initialize clock-gating latches.

Note the following:

- The set of clock cycles should equal the depth of the chain of clock-gating latches. Make sure this is the case.
- Violations still occur when there are multiple cascaded latches and the scan-enable and control point location are used as before, with mixed active-high and active-low latches.

You can use either the scan-enable or test-mode signal as the clock-gating control signal. They differ in the following ways:

- A scan-enable signal forces the clock to be enabled only during scan shift; the functional enable controls the clock during scan capture.
- A test-mode signal forces the clock to be enabled during the entire test (scan shift and scan capture).

Using a scan-enable signal as the clock-gating control signal typically provides higher fault coverage than the test-mode signal because the functional clock-gating path is exercised in scan capture mode. Fault coverage with the scan-enable signal is comparable to a circuit without clock gating, as shown in [Figure 113](#).

0 during scan capture

test_se

CLK

0

Clock-gating cell

D SI SE Q

D SI SE Q

D SI SE Q

Legend:

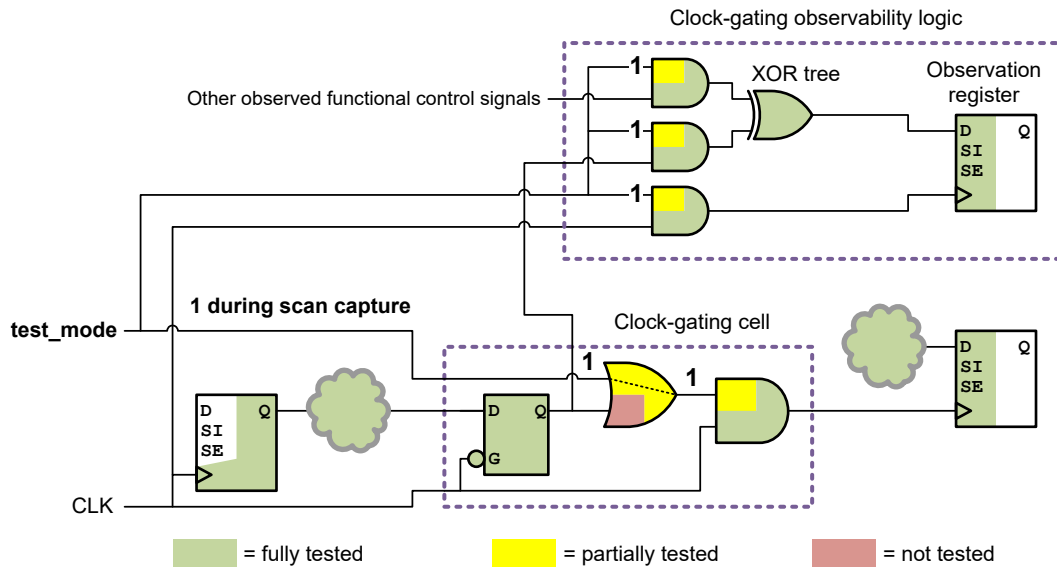
- Green box = fully tested
- Yellow box = partially tested
- Red box = not tested

If you use a test-mode signal as the clock-gating control signal, the test-mode signal is asserted during both scan shift and scan capture. This bypasses the functional clock-gating control logic completely and prevents it from being tested in scan capture mode, as shown in [Figure 114](#). In addition, the clock-gating enable signal asserted by the test-mode signal can be tested only for stuck-at-0 faults (assuming an active-high test-mode signal).

[illegible]

If your clock-gating cell has an observe pin, Fusion Compiler inserts observability logic to regain some of this coverage. [Figure 115](#) shows an observability logic example.

Figure 115 Observability Logic for Clock-Gating Cells



In test mode, an XOR tree observes the functional gating control signals from one or more clock-gating cells; the output of the XOR tree is captured by a dedicated observability register. This observability register is included in the scan chains, but its output is not otherwise connected functionally in the design. AND or NAND gates (depending on synthesis) prevent the observability logic from consuming power during mission mode.

Note that placing the control point after the latch allows the latch to be tested by the observability logic.

Configuring Clock-Gating for DFT

In Fusion Compiler, clock gating is always enabled. During compile, conditional-capture register banks in the RTL are identified and gated. In addition, the test pins of clock-gating cells (both tool-inserted and instantiated) are connected to the scan-enable signal. No further configuration is needed.

However, the DFT implementation for clock gating can be customized as follows.

Specifying Clock-Gating Control Signals

By default, Fusion Compiler chooses an available ScanEnable signal to use for the clock-gating control signal. If this is sufficient, no further configuration is needed.

However, you can also define particular clock-gating control signals to meet your own design requirements.

Specifying a Particular Global Clock-Gating Control Signal

To specify a particular scan-enable or test-mode signal to use as the clock-gating control signal, define that signal with the `clock_gating` usage:

```
set_dft_signal
  -type ScanEnable | TestMode
  -view spec
  -usage clock_gating
  -port port_list
```

The `-type` option specifies the type of clock-gating control signal you want to define.

Remember that when you define a DFT signal with a usage, the tool is limited to using that signal only for that usage.

Specifying Object-Specific Clock-Gating Control Signals

You can also define clock-gating control signals for specific parts of the design by using the `-connect_to` option of the `set_dft_signal` command:

```
set_dft_signal
  -type ScanEnable | TestMode
  -view spec
  -usage clock_gating
  -port port_list
  [-connect_to object_list]
```

The `-connect_to` option specifies a list of design objects that are to use the specified clock-gating control signal. The supported object types are

- Clock-gating cells (instantiated)
- Hierarchical cells
- Designs
- Test clock ports

This allows you to make clock-domain-based signal connections. It includes clock-gating cells that gate the specified test clocks. The functional clock behavior is not considered.

This specification requires that a functional clock also be defined on the test clock port.

- Scan-enable or test-mode pins of CTL-modeled cores

The following example defines a ScanEnable signal named SE_CG to connect to the test pins of existing clock-gating cells ICG_CLK100 and ICG_CLK200:

```
fc_shell> set_dft_signal \  
-type ScanEnable -view spec -port SE_CG \  
-usage clock_gating -connect_to {ICG_CLK100 ICG_CLK200}
```

Excluding Clock-Gating Cells From Test-Pin Connection

You might have a portion of the design that is excluded from scan testing, and you do not want Fusion Compiler to connect the test pins of those clock-gating cells to test-mode or scan-enable signals. To prevent the tool from connecting the test pins of some clock-gating cells, use the `-exclude_elements` option of the `set_dft_clock_gating_configuration` command:

```
fc_shell> set_dft_clock_gating_configuration  
-exclude_elements object_list
```

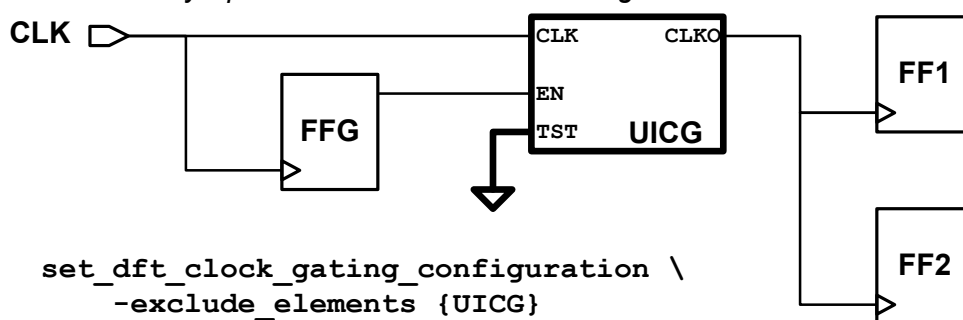
The *object_list* can include the following object types:

- Clock-gating cell leaf instances (instantiated in RTL)
- Hierarchical cell instances – All clock-gating cells within the instances are included in the specification.
- Clock-gating library cell – All instances of that cell are included in the specification.
- Clocks – All clock-gating cells in the clock domains are included in the specification.

For tool-inserted clock-gating cells, the tool ties the test pin to a de-assertion constant. For RTL-instantiated clock-gating cells, the tool leaves the existing RTL connection in place.

Figure 116 shows an example of an excluded clock-gating cell.

Figure 116 Directly Specified Excluded Clock-Gating Cell



Clock Gating for DFT Wrapper Cells

Clock gating enables the clock gating engine to gate wrapper cells and internal cells with separate clock gates. The power consumption in DFT EXTEST mode can be reduced by turning off the clock to the internal cells.

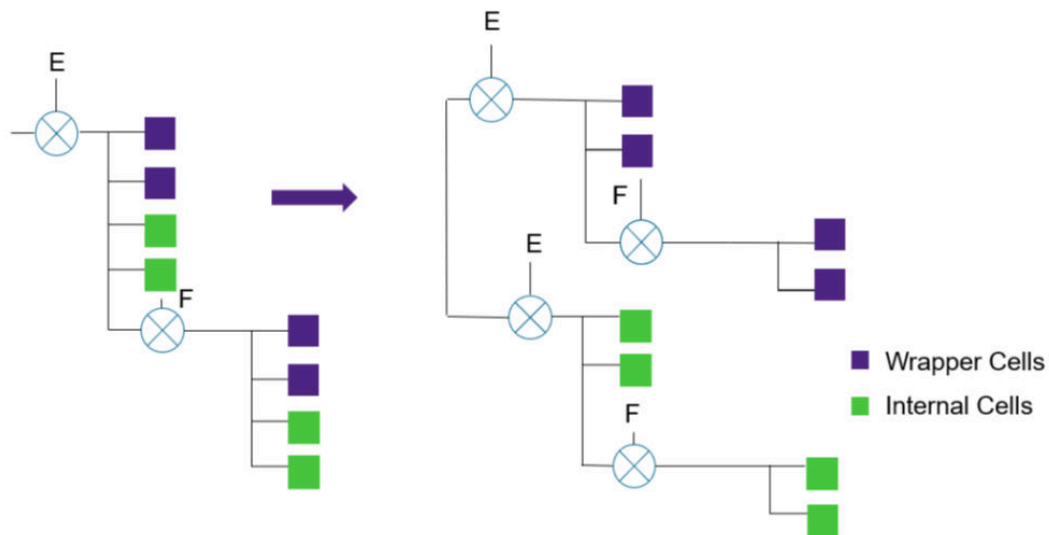
The following topics provide more information on clock gating:

- [Clock Gating](#)
- [Limitations](#)

Clock Gating

When you do not specify the `max_fanout` option, the Fusion Compiler tool inserts one clock gate for every unique enable condition, by default. In DFT wrapper clock gating, the clock gate is split into two. One clock gate is used for the wrapper cells, and the other is used for the internal cells, as shown in the following figure.

Figure 117 DFT wrapper clock gating



When you split the clock gating signal to gate wrapper cells and internal cells separately, the resultant clock gates and the wrapper cells that were split away are marked such that they can be queried independently.

To enable DFT wrapper clock gating, set the `clockgate.enable_dft_wrapper_cell_support` application option to `true`.

Limitations

If the `depth_threshold` option is used, then clock gating might not work after using the `insert_dft` command.

11

The Internal Pins Flow

The internal pins flow allows you to define certain DFT signals on internal pins instead of top-level ports. This allows more complicated test logic to be inserted, but it also comes with some caveats that must be considered.

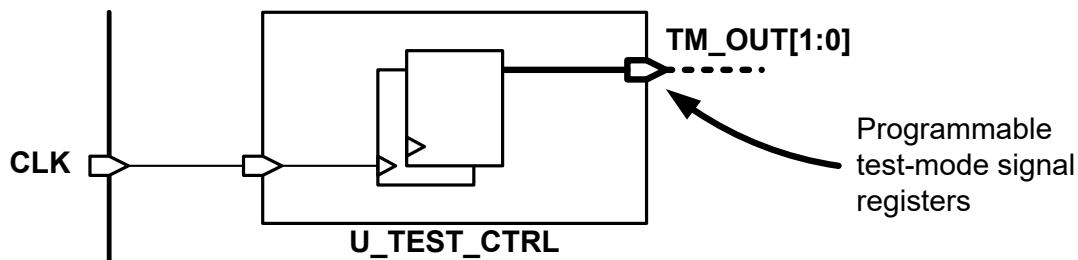
The following topics provide more information:

- [Introduction to the Internal Pins Flow](#)
- [Defining Signals on Internal Pins](#)
- [Writing Out the Test Protocol](#)
- [Limitations](#)

Introduction to the Internal Pins Flow

Normally, DFT signal (such as clocks, resets, scan-ins, and scan-outs) are defined on ports of the current design. Even when defined with a hookup pin, these signals are ultimately connected to a port.

However, advanced DFT architectures might require some DFT signals to connect to an internal pin, with no straightforward path to a port. The following figure shows programmable registers intended to drive TestMode signals:



In these cases, you can use the *internal pins* flow to define such signals. It is called a flow because it requires consideration both before and after DFT insertion

Defining Signals on Internal Pins

To define signals on internal pins, do the following:

1. In your global DFT configuration, enable the internal pins flow:

```
set_dft_drc_configuration -internal_pins enable
```

2. When defining your DFT signals, define each internal-pins signal by using the `-hookup_pin` option *without* the `-port` option:

```
set_dft_signal -view spec -type TestMode \  
-hookup_pin U_TEST_CTRL/TM_OUT[1]  
set_dft_signal -view spec -type TestMode \  
-hookup_pin U_TEST_CTRL/TM_OUT[0]
```

The `-hookup_pin` option accepts only a single pin object; use multiple commands for multiple pins.

You can define a mix of port-driven and internal-pins signals as needed.

The following signal types can be defined as internal-pins signals:

Table 10 *Supported Internal-Pins Signal Types*

ScanMasterClock
MasterClock
ScanClock
Reset
Constant
TestMode
ScanEnable
ScanDataIn
ScanDataOut
PLLClock
occ_reset (or pll_reset)
occ_bypass (or pll_bypass)
LOSPipelineEnable

Writing Out the Test Protocol

After DFT insertion, the CTL model for the design contains information about internal pins. This information is understood by Fusion Compiler, but it cannot be used by TestMAX ATPG.

When you write out a test protocol using the `write_test_protocol` command, the resulting SPF references these internal pins. Before using the protocol in TestMAX ATPG, you must remove the internal pins references, then make any modifications needed for the assumptions in the test protocol to be true.

Limitations

Note the following limitations of the internal pins flow:

- Post-DFT DRC is not supported for compressed scan designs with internal-pins signals.
- In most cases, the output protocol must be modified before use in the TestMAX ATPG tool.
- The core wrapper flow does not support internal-pins signals.

See Also

- [SolvNetPlus article 040136](#), “Using the Internal Pins Flow With Internal Test Registers” for an example that drives DFT signals from design registers

12

Generating SCANDEF for Scan Reordering in Layout

Fusion Compiler can generate SCANDEF information that describes how scan cells in the design can be reordered and repartitioned. You can use this SCANDEF information in the Fusion Compiler tool to optimize scan chains and to fix timing violations using physical information. You can also use this information in other place-and-route tools.

This topic covers the following:

- [Introduction to SCANDEF](#)
 - [SCANDEF Constructs](#)
 - [Writing Out the SCANDEF Information](#)
 - [Controlling START/STOP Points for SCANDEF Generation](#)
 - [Limitations of SCANDEF Generation](#)
 - [Supporting SCANDEF for Pre-Stitched Scan Segments](#)
-

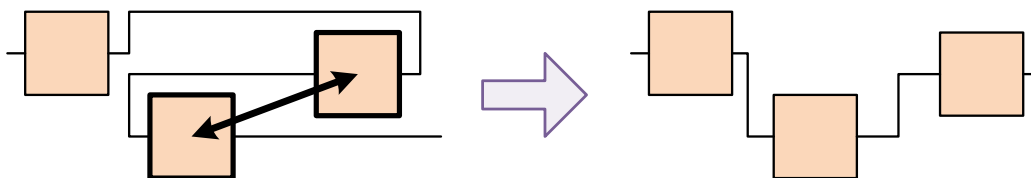
Introduction to SCANDEF

Scan-inserted designs require additional routing compared to nonscan designs. To meet die size and timing requirements, you should reduce the routing overhead as much as possible. One way to do this is to optimize scan chains based on physical information.

There are two types of scan optimization operations:

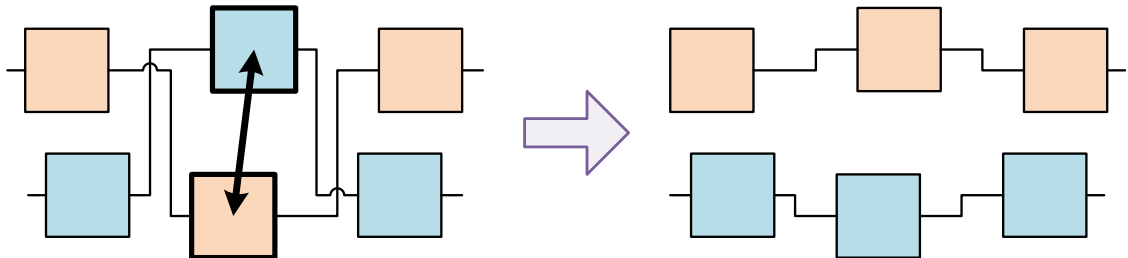
- *Scan reordering* changes the scan chain position of two (or more) scan cells within the same scan chain. [Figure 118](#) shows a scan reordering example.

Figure 118 Scan Reordering of Two Scan Cells



- *Scan repartitioning* swaps two (or more) scan cells between different scan chains, such that the original chain lengths are preserved. [Figure 119](#) shows a scan repartitioning example.

Figure 119 Scan Repartitioning of Two Scan Cells



During scan reordering and repartitioning, the layout tool must honor DFT constraints such as clock mixing, DFT partitions, multivoltage regions, and multiple test modes. However, communicating this information directly to the layout tool would be complex and error-prone.

Instead, SCANDEF communicates what reordering and repartitioning operations are possible given the DFT constraints. As a result, the layout tool does not need to understand DFT constraints; it simply optimizes as allowed by the SCANDEF.

SCANDEF Constructs

A SCANDEF file is a DEF file that uses a set of scan-specific constructs to describe scan chain information. A *stub chain* definition describes a portion of a scan chain that can be reordered within itself. A stub chain consists of a START point, a STOP point, and one or more scan elements between them. [Example 12](#) shows the first three stub chain definitions in a SCANDEF file.

Example 12 SCANDEF Example

```
VERSION 5.5 ;
NAMESCASESENSITIVE ON ;
DIVIDERCHAR "/" ;
BUSBITCHARS "[]" ;
DESIGN top ;
SCANCHAINS 8 ;
- 1_clk_st_45_55_DEFAULT_POWER_DOMAIN_VDD_internal
+ PARTITION clk_st_45_55_DEFAULT_POWER_DOMAIN_VDD_internal
  MAXBITS 8
+ START U131 Y
+ FLOATING ZN_reg[0] ( IN SI ) ( OUT Q )
              ZN_reg[1] ( IN SI ) ( OUT Q )
              ZN_reg[2] ( IN SI ) ( OUT Q )
              ZN_reg[3] ( IN SI ) ( OUT Q )
              ZN_reg[4] ( IN SI ) ( OUT Q )
```

Chapter 12: Generating SCANDEF for Scan Reordering in Layout

SCANDEF Constructs

```
+ PARTITION CLK_45_45
+ STOP ZN_reg[4] SI ;
- 2_clk_st_45_55_DEFAULT_POWER_DOMAIN_VDD_internal
+ PARTITION clk_st_45_55_DEFAULT_POWER_DOMAIN_VDD_internal
MAXBITS 8
+ START U132 Y
+ FLOATING ZN_reg[5] ( IN SI ) ( OUT Q )
+ ORDERED SR_reg[3] ( IN SI ) ( OUT Q )
          SR_reg[2] ( IN D ) ( OUT Q )
          SR_reg[1] ( IN D ) ( OUT Q )
          SR_reg[0] ( IN D ) ( OUT Q )
+ PARTITION CLK_45_45
+ STOP ZN_reg[4] SI ;
- 3_clk_st_45_55_DEFAULT_POWER_DOMAIN_VDD_internal
+ PARTITION clk_st_45_55_DEFAULT_POWER_DOMAIN_VDD_internal
  MAXBITS 8
+ START test_si2
+ FLOATING IP_inst ( IN test_si1 ) ( OUT test_so1 ) (BITS 5)
          IPglue_logic_cell1 ( IN TI ) ( OUT SO )
          IPglue_logic_cell2 ( IN TI ) ( OUT SO )
+ PARTITION IPCLK_45_45
+ STOP test_so2 ;
...
```

Note the following constructs:

- The START and STOP points specify the stub chain boundaries. They can be a variety of scan chain constructs such as scan I/O ports, codec logic gates, lockup latches, reconfiguration MUXs, or buffer/inverter pins. Therefore, stub chains are not usually identical to scan chains, and the number of stub chains defined in the SCANDEF information does not necessarily match the number of scan chains in the design.
- A FLOATING section is a list of unordered cells that can be reordered freely by the layout tool. Because the cells are described as an unordered list, the order in the SCANDEF file might not match the order in the design.
- An ORDERED section describes a group of scan cells that cannot be reordered within that group, but can be reordered as a group within a stub chain. Common causes of ORDERED sections are shift registers identified by the `compile_fusion` command, scan segments defined with the `set_scan_path -ordered_elements` command, and buffers or inverters between scan cells.
- The BITS attribute indicates a scan element that represent multiple scan bits. This allows complex scan cells, such as DFT-inserted cores, to be represented in abstract form. By default, each individual scan element represents a single scan bit.
- A PARTITION name indicates that the stub chain elements can be repartitioned with those of another stub chain with the same partition name. The tool constructs

partition names so that identical names indicate stub chains that are compatible for repartitioning. Partition names are made unique or omitted for stub chains whose elements cannot be repartitioned.

A stub chain can include zero or more ORDERED sections. However, it can only contain zero or one FLOATING section, as having multiple FLOATING sections within the same stub chain is meaningless.

A SCANDEF file does not necessarily contain all scan cells in the design. It contains information only about scan cells in the design that *can* be reordered or repartitioned. Scan cells or scan segments that cannot be optimized are omitted from the file.

The layout tool can reorder *and* or *or* repartition many scan cells at a time. For example, several compatible scan chains in a geographic region can be completely reconstructed, if the SCANDEF information is honored and the original scan chain lengths are preserved.

Writing Out the SCANDEF Information

To generate SCANDEF information, perform the following steps after reading in your design and applying the DFT configuration:

1. Execute the `insert_dft` command.
2. Generate the SCANDEF information with the `write_scan_def` command:

```
fc_shell> write_scan_def -output filename.scandef
```

This command generates and writes out the SCANDEF information to the specified file name.

You can use the resulting SCANDEF information in your place-and-route tool to optimize scan chain connection order. The SCANDEF file is not necessary when the implementation is continued in the Fusion Compiler tool. To read in the SCANDEF file generated by a different tool, use the following command in the Fusion Compiler tool:

```
read_def filename.scandef
```

When you read SCANDEF information into the Fusion Compiler tool, it checks the integrity of the information against the design netlist before using it.

Controlling START/STOP Points for SCANDEF Generation

START and STOP points are automatically chosen by the Fusion Compiler tool for each SCANDEF chain such that the original DFT requirements are preserved, and potential clock and power domain violations are avoided during DFT optimizations.

The START and STOP points of the logical scan chains defined could end up being the SI/SO ports or the hookup pins of the scan chains, or the input/output of a combinational cell, a multiplexer, or even a lookup latch.

There are scenarios where you can make use of specific controls that give the flexibility to choose START and STOP points for potential optimizations explained in the following topics:

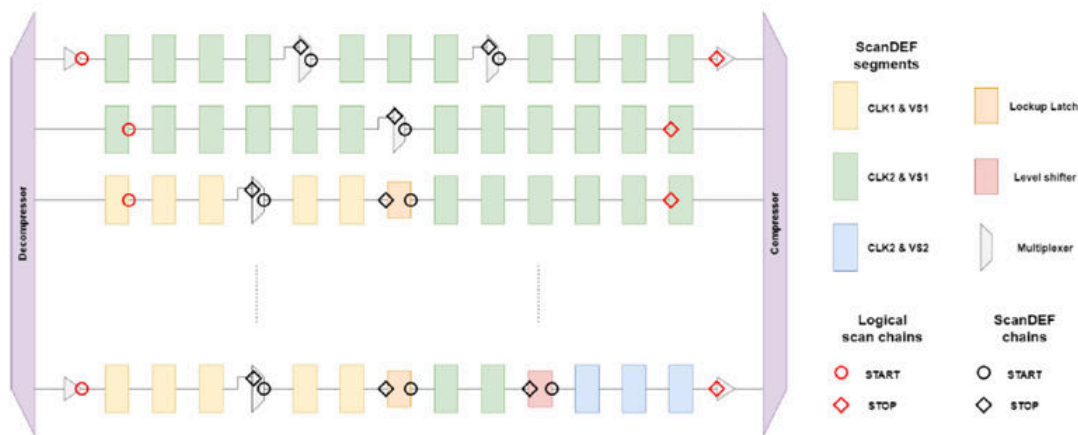
This topic covers the following:

- [Fixing the First and Last Sequential Element of the Logical Chain Using SCANDEF](#)
- [Inserting Buffer at the Beginning and End of Logical Scan Chains](#)

Fixing the First and Last Sequential Element of the Logical Chain Using SCANDEF

Consider the following figure where the START and STOP points of each SCANDEF chain are highlighted with a circle and rhombus respectively and with color red for logical scan chains.

Figure 120 SCANDEF with automatically chosen START/STOP Points



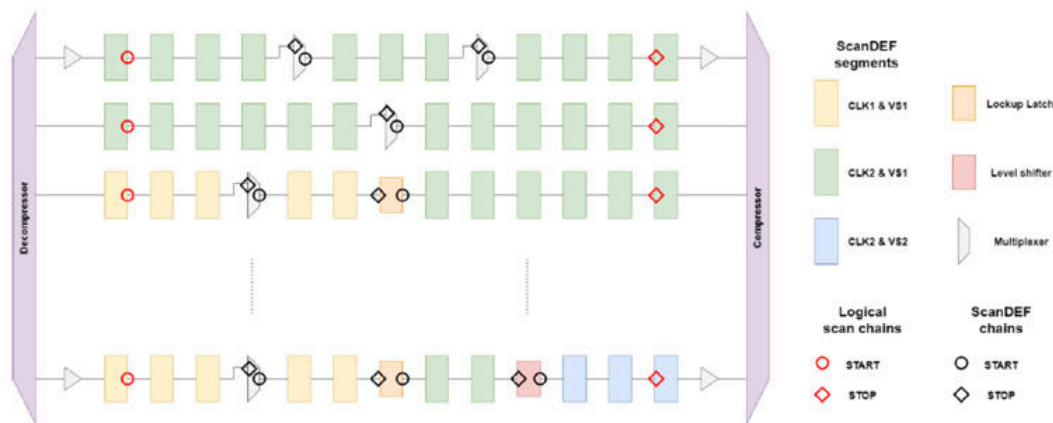
You can choose the START/STOP points of the logical scan chains so that the first and last sequential elements of the logical scan chain remains intact throughout the flow.

You can set the `dft.scan.skip_scandef_segment_at_logical_chain_endpoints` application option to `true` that enables the following tool behavior:

The tool uses the Q(output) of the first scan cell of the logical chain as the START point of the corresponding stub-chain instead of the SI/SO pins. Similarly, it uses the SI pin of the last cell of the logical chain as the STOP point of the corresponding stub-chain.

The resulting SCANDEF START/STOP points are as shown in the following figure:

Figure 121 Resulting SCANDEF START/STOP Points



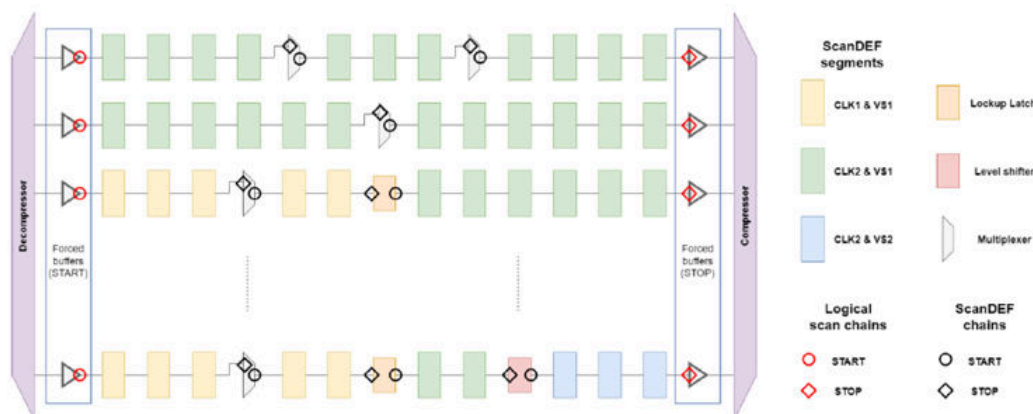
Inserting Buffer at the Beginning and End of Logical Scan Chains

For the same design example as in [Figure](#) , you can insert buffers at the beginning and end of logical scan chains such that they can be used as START/STOP points.

To insert the buffers at the beginning and end of logical chains, set the `dft.scan.force_buffers_at_logical_chain_endpoints` application option to `true`.

This SCANDEF implementation allows the complete reordering of the sequential elements appearing at the start and end of the logical scan chains, as shown in the following figure:

Figure 122 Buffer Inserted and Used as START/STOP Points



Limitations of SCANDEF Generation

Note the following limitations of SCANDEF generation:

- Manual post-DFT modifications to the scan structure or scan element naming are not supported.

Manual post-DFT design modifications that affect the scan architecture or scan element naming are not supported. Examples include:

- Hierarchy uniquification or ungrouping
- ECO modification to scan chains or logic that affects DFT operation
- Modifying IEEE 1801 Standard (UPF) power intent to insert isolation *and* or *or* level-shifter cells after DFT insertion
- For stub chains that terminate at a reconfiguration MUX, the STOP pin is the scan-in pin of the last scan cell instead of the input pin of the reconfiguration MUX. This prevents the last scan cell in the stub chain from being reordered or repartitioned.
- Some `set_scan_group` specifications might not be represented properly.
- Some `set_scan_path` specifications, when applied to a specific test mode other than the first-defined test mode, are not represented properly.

For details, see [SolvNetPlus article 2314593](#), “SCANDEF Generation Limitations for Multiple Test Modes.”

Supporting SCANDEF for Pre-Stitched Scan Segments

Pre-stitched scan segments are defined using `set_scan_group -serial_routed true` so that the `insert_dft` command can include these scan segments in the top-level scan chains.

However, these scan segments do not appear in the SCANDEF. This disables the reordering of their endpoints. This feature enables scan chain optimization during reordering by allowing the pre-stitched scan segments information available in the SCANDEF.

The scan input and output connections of the scan group are reordered with other eligible scan elements sharing same clocks and power domains while keeping scan group together.

You can use the `set_app_option -name dft.scan.scandef_analyze_scangroups -value true` application option to enable the feature.

For the `-serial_routed true` option, you should define the access pins of the group using the `-access` option. These access pins can be hierarchical pins without physical location and therefore cannot be added to the SCANDEF. The tool needs to find the leaf pin of the nearest scan cell to add the segment to the SCANDEF.

To add the pre-stitched scan segments to the SCANDEF, you can use the following two cases:

Case 1: No scan elements specified, segment length specified.

Only segments with scan cells and buffers/inverters in the scan path are added to the SCANDEF using this method. You can use the `set_scan_group -serial_routed true -interface {} -segment_length N` command. For example,

```
set_scan_group sgl \
  -access [list \
    ScanDataIn sgl_inst1/din \
    ScanDataOut sgl_inst1/sgl_out \
    ScanEnable sgl_inst1/SE \
    ScanClock sgl_inst1/clk \
  ] \
  -segment_length 3 \
  -edge rising \
  -serial_routed true \
  -lockup_exist true \
  -clock clk_st
set_scan_configuration exclude_elements [get_cell -hier sgl_inst1/*]
```

Note:

If there is combinational logic in the scan path that needs to be set to a certain value to trace the chain, then that segment is not added to the SCANDEF as there is no simulation based tracing.

Case 2: Scan segment mentioned with `-include_elements {}`

You can use this case if there are multiple-input combinational logic such as muxes in the scan path, that fails to add the segment to the SCANDEF in *Case 1*. You must specify the list of cells in the order they exist in the scan chain.

This enables the tool to use the elements mentioned in the order. If more than one cell is encountered when tracing from one cell to the next cell, the tool uses the adjacent cell mentioned in the `-include_elements` section.

Note:

In this use case, you need to mention both combinational and sequential cells of the scan segment in the order they appear in the netlist. Without the application option `dft.scan.scandef_analyze_scangroups`, combinational cells are not allowed in the `-include_elements` option of `set_scan_group` command.

You can use the `set_scan_group -serial_routed true -interface {} -include_elements {}` command. For example,

```
set_scan_group sgl \
  -access [list \
    ScanDataIn sgl_inst1/din \
    ScanDataOut sgl_inst1/sgl_out \
    ScanEnable sgl_inst1/SE \
  ] \
  -serial_routed true \
  -lockup_exist true \
  -include_elements {sgl_inst1/sg2_buf sgl_inst1/sg1_mux \
sgl_inst1/sg1_reg1 sgl_inst1/sg1_reg2 sgl_inst1/sg1_reg3 }
```

Limitations

Note the following limitations of supporting SCANDEF for pre-stitched scan segments:

- If a segment is not traceable, then the segment is not added to the SCANDEF.
- When the `-segment_length` is specified, you must exclude the scan cells of the segment from scan stitching to avoid stitching them again.
- When the `-include_elements` is specified, you must not exclude the cells from scan configuration. The tool is aware of the actual cells of the scan group.

13

On-Chip Clocking (OCC)

On-chip clocking (OCC) support is common to all scan ATPG (basic-scan and fast-sequential) and compressed scan environments. This implementation is intended for designs that require ATPG in the presence of phase-locked loop (PLL) and clock controller circuitry.

OCC support includes phase-locked loops, clock shapers, clock dividers and multipliers and so on. In the scan-ATPG environment, scan chain `load_unload` is controlled through an automatic test equipment (ATE) clock. However, internal clock signals that reach state elements during capture are PLL-related.

OCC flows can use either the user-defined clock controller and clock chains or the DFT-inserted OCC clock controller. If you use an existing user-defined clock controller, you would need a set of user-defined commands to identify the existing clock controller outputs with their corresponding clock chain control bits.

This chapter includes the following topics:

- [Background](#)
- [Clock Type Definitions](#)
- [OCC Controller Structure and Operation](#)
- [Enabling On-Chip Clocking Support](#)
- [Specifying OCC Controllers](#)
- [Reporting Clock Controller Information](#)
- [Controlling the OCC Controller Bypass Path](#)
- [Controlling the OCC/SPC Scan Chain With General Multimode](#)
- [DFT-Inserted OCC Controller Configurations](#)
- [Waveform and Capture Cycle Example](#)
- [Limitations](#)

Background

At-speed testing for deep-submicron defects requires not only more complex fault models for ATPG and fault simulation, such as transition faults and path delay faults, but also requires the accurate application of two high-speed clock pulses to apply the tests for these fault models. The time delay between these two clock pulses, referred to as the launch clock and the capture clock, is the effective cycle time at which the circuit will be tested.

A key benefit of scan-based at-speed testing is that only the launch clock and the capture clock need to operate at the full frequency of the device under test. Scan shift clocks and shift data can operate at a much slower speed, thus reducing the performance requirements of the test equipment. However, complex designs often have many different high-frequency clock domains, and the requirement to deliver a precise launch and capture clock for each of these from the tester can add significant or prohibitive costs to the test equipment. Furthermore, special tuning is often required for properly controlling the clock skew to the device under test.

One common alternative for at-speed testing is to leverage existing on-chip clock generation circuitry. This approach uses the active controller, rather than off-chip clocks from the tester, to generate the high-speed launch and capture clock pulses. This type of approach generally reduces tester requirements and cost and can also provide high-speed clock pulses from the same source as the device in its normal operating mode without additional skews from the test equipment or test fixtures.

When using this approach, additional on-chip controller circuitry is inserted to control the on-chip clocks in test mode. The on-chip clock control is then verified, and at-speed test patterns are generated that apply clocks through proper control sequences to the on-chip clock circuitry and test-mode controls. The Fusion Compiler and TestMAX ATPG tools support a comprehensive set of features to ensure that

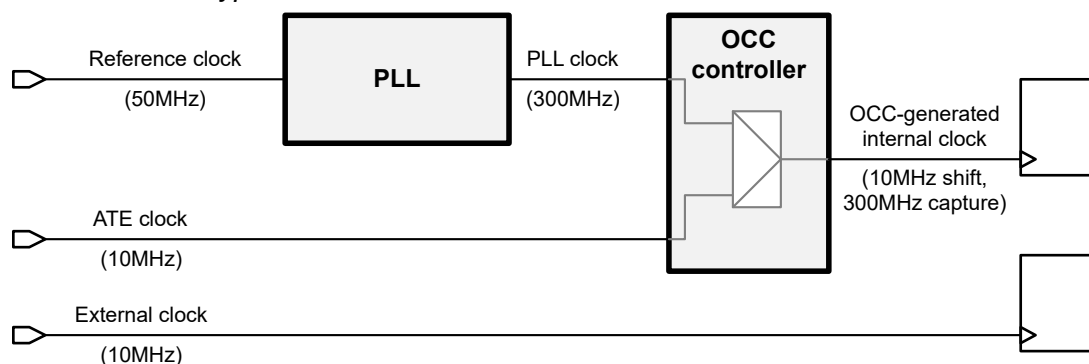
- The test-mode control logic for the OCC controller operates correctly and has been connected properly
- Test-mode clocks from the OCC circuitry can be efficiently used by TestMAX ATPG for at-speed test generation
- OCC circuitry can operate asynchronously to other shift clocks from the tester
- TestMAX ATPG patterns do not require additional modifications to use the OCC and to run properly on the tester

Clock Type Definitions

Note the following clock definitions as they apply to OCC controller clocks in this chapter. [Figure 123](#) shows an example of each clock type.

- **Reference clock** – The frequency reference to the phase-locked loop (PLL). It must be maintained as a constantly pulsing and free-running oscillator, or the circuitry will lose synchronization.
- **PLL clock** – The output of the PLL. It is also a free-running source that runs at a constant frequency that might or might not be the same as the reference clock.
- **ATE clock** – Shifts the scan chain, typically more slowly than a reference clock. This signal must preexist, or you must manually add this signal (that is, port) when inserting the OCC. The period for this clock is determined by the `dft.test_default_period` application option. Usually the ATE clock is not used as a reference clock, but it must be treated as a free-running oscillator so that it does not capture predictable data while the OCC controller generates at-speed clock pulses. The ATE clock is called a dual clock signal when the same port drives both the ATE clock and the reference clock.
- **Internal clock** – The OCC controller is responsible for gating and selecting between the PLL and ATE clocks, thus creating the internal clock signal to satisfy ATPG requirements.
- **External clock** – A primary clock input of a design that directly clocks flip-flops through the combinational logic.

Figure 123 Clock Types Used in the OCC Controller Flow



OCC Controller Structure and Operation

OCC controller types and operation is covered in the following topics:

- [DFT-Inserted and User-Defined OCC Controllers](#)
- [OCC-Controlled Clock Relationships](#)
- [OCC Controller Signal Operation](#)
- [Clock Chain Operation](#)
- [Logic Representation of an OCC Controller and Clock Chain](#)
- [Scan-Enable Signal Requirements for OCC Controller Operation](#)

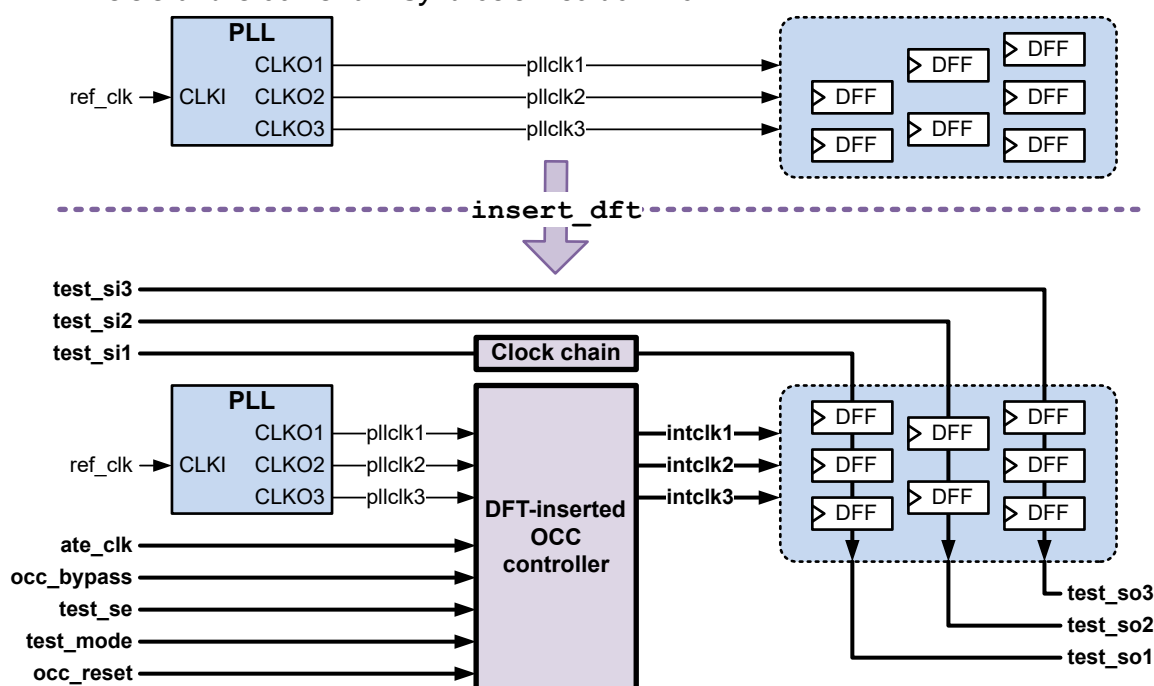
DFT-Inserted and User-Defined OCC Controllers

You can use DFT-inserted or user-defined OCC controllers, as described in the following flows:

- [Specifying DFT-Inserted OCC Controllers](#)

The `insert_dft` command inserts a DFT-inserted OCC controller and clock chain, making control signal connections and modifying the clock signal connections as needed. The OCC controller design is validated and incorporated into the resulting test protocol. This flow is shown in [Figure 124](#).

Figure 124 OCC and Clock Chain Synthesis Insertion Flow

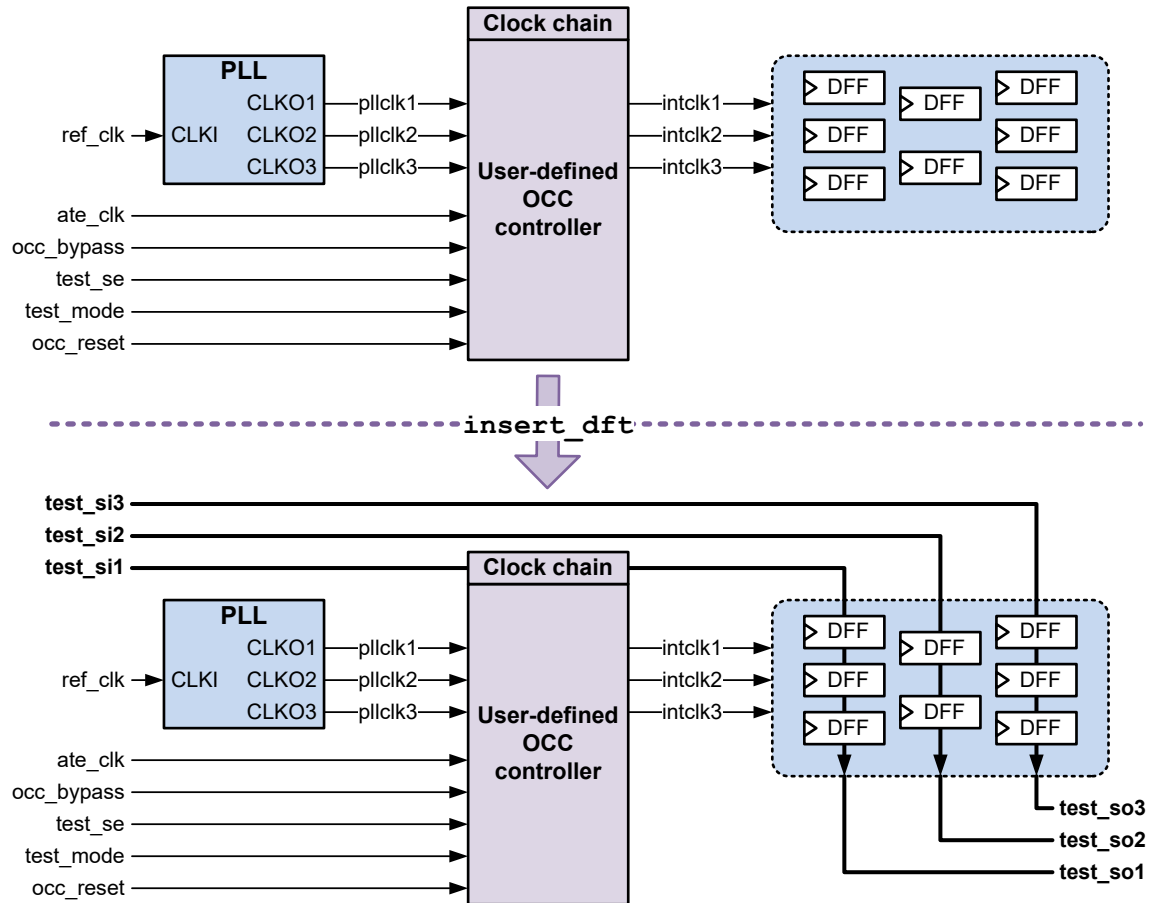


- [Specifying Existing User-Defined OCC Controllers](#)

The RTL contains the OCC controller and clock chain logic, all control signal and clock signal connections, and the connections from the clock chain to the OCC controller. Before DFT insertion, this existing user-defined OCC controller and clock chain logic is described to the tool using the `set_dft_signal` and `set_scan_group` commands. The

OCC controller design is validated and incorporated into the resulting DFT logic and test protocol. This flow is shown in [Figure 125](#).

Figure 125 User-Defined Clock Controller and Clock Chain Flow

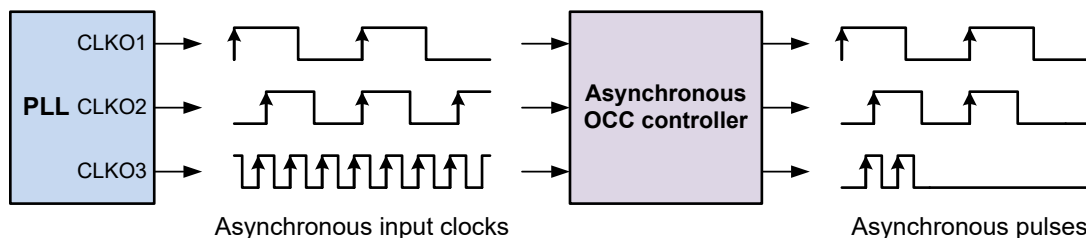


Note that in this flow, the tool does not make any signal connections to the OCC controller or clock chain logic, except for scan data connections to the clock chain segments. You must ensure that all clock and control signal connections exist prior to DFT insertion.

OCC-Controlled Clock Relationships

Each controlled clock uses dedicated clock control logic that generates clock pulses with no regard to alignment with other controlled clocks, as shown in [Figure 126](#). The input clocks being controlled can be synchronous or asynchronous with each other.

Figure 126 Asynchronous OCC Controller Generating Two Clock Pulses



ATPG treats the controlled clock domains as asynchronous; faults between them are not tested. You can use an OCC controller to control synchronous clocks, but their synchronous relationships are lost; there is no guarantee that their pulse sequences will initiate on the same rising edge.

OCC Controller Signal Operation

Note the following:

- The reference clock (refclk) is always free-running. It is used as a test default frequency input to the PLL.
- The PLL clocks (pllclk1, pllclk2, and pllclk3) are free-running clock outputs from the on-chip clock generator; they can be divided, shaped, or multiplied. They are used for the launch and capture of internal scanable elements that become internal clocks.
- The ATE clock (ate_clk) shifts the scan chain per tester specifications. Each PLL might have its own ATE clock.

See [Waveform and Capture Cycle Example](#) for a waveform diagram that demonstrates the relationship between the various clocks.

- The OCC controller serves as an interface between the on-chip clock generator and internal scan chains. This logic typically contains clock multiplexing logic that allows internal clocks to switch from a slow ATE clock during shift to a fast PLL clock during capture.
- Internal clocks (intclk1, intclk2, and intclk3) are outputs of the PLL control logic driving the scan cells. Each internal clock is controlled or enabled by the clock chain and is connected to the sequential elements within the design.
- The OCC bypass signal (occ_bypass) allows the ATE clock signal to connect directly to the internal clock signals, thus bypassing the PLL clocks.
- The ScanEnable signal (test_se) enables switching between the ATE shift clock and output PLL clock signals. ScanEnable must be inactive during every capture procedure, as described in [Scan-Enable Signal Requirements for OCC Controller Operation](#). You can use individual ScanEnable signals for each PLL clock signal.

- The OCC TestMode signal (test_mode) must be asserted in order for the clock controller to operate.
- The OCC reset signal (occ_reset) is asserted during test setup to reset the OCC controller flip-flops to their initial states.

Clock Chain Operation

The clock chain provides a per-pattern clock selection mechanism for ATPG. It is implemented as a scan chain segment of one or more scan cells. Clock selection values are loaded into the clock chain as part of the regular scan load process.

A clock chain operates as follows:

- During scan shift, the clock chain shifts in new values when clocked by a scan clock.

The clock chain can be clocked by either the rising or falling clock edge, depending on what best fits into the overall DFT architecture.

- During scan capture, the clock chain holds its value.

The value scanned into the clock chain must be scanned out, undisturbed, after capture. The clock controller inserted by Fusion Compiler meets this requirement. If you provide your own clock controller, ensure that it meets this requirement.

Note the following scan architecture aspects of clock chains:

- For standard scan designs, the clock chain can be a dedicated scan chain or a segment within a scan chain.
- For DFTMAX compressed scan designs, the clock chain can be an uncompressed (external) scan chain or a special segment within a compressed scan chain. For more information, see [Scan Compression and OCC Controllers](#).
- For DFTMAX Ultra compressed scan designs, the clock chain must be an uncompressed (external) scan chain. For more information, see [Using OCC Controllers With DFTMAX Ultra Compression](#).
- Clock chains of the same type (compressed or external) can be concatenated together.

Compressed clock chains are concatenated into a single chain and placed inside the compressor where a regular single chain would be placed.

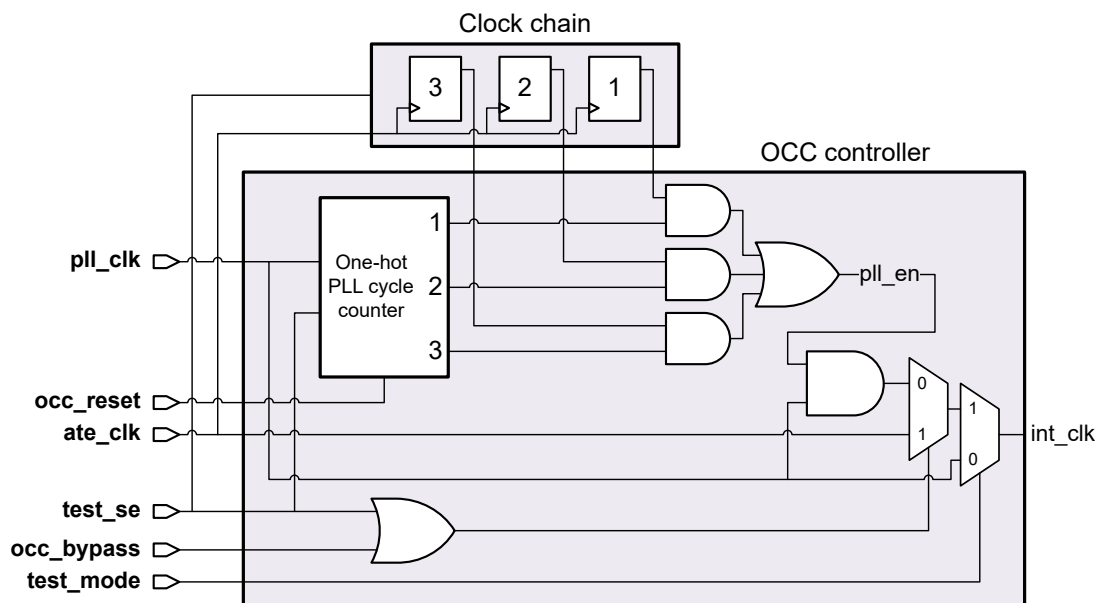
In the DFT-inserted OCC controller flow, the tool inserts a clock chain block that is separate from the OCC controller block.

In the user-defined OCC controller flow, the clock chain can be a part of the OCC controller design or it can be a separate design.

Logic Representation of an OCC Controller and Clock Chain

Figure 127 shows the logic structure of an OCC controller and clock chain.

Figure 127 Logic Representation of an OCC Controller



An OCC controller is designed to deliver up to a user-specified number N of at-speed clock pulse cycles during capture. A PLL cycle counter generates N successive one-hot enable signals, each of which is gated by the output of a clock chain scan cell. This logic structure provides ATPG with the flexibility to control which cycles deliver an at-speed clock pulse. For an OCC controller that handles multiple OCC generators, the clock chain contains a set of N scan cells for each clock.

Note that the figure shows the conceptual operation of a single-clock OCC controller. Implementation details, such as cleanly switching between the PLL clock and ATE clock and providing synchronous or asynchronous control of multiple clocks, are not shown.

See Also

- [SolvNetPlus article 034274, "DFT-inserted OCC Controller Data Sheet"](#) for more information about the logic structure and operation of the DFT-inserted OCC controller

Scan-Enable Signal Requirements for OCC Controller Operation

The scan-enable signal switches the OCC controller between the ATE shift clock and output PLL clock signals. Therefore, for proper operation, *the scan-enable signal must be held in the inactive state in all capture procedures.*

If you use the STIL protocol file created by the tool, the protocol already meets this requirement. The tool constrains all scan-enable signals to the inactive state in the capture procedures, excluding any scan-enable signals defined with the `-usage {clock_gating}` option of the `set_dft_signal` command. (Signals with multiple usages that include `clock_gating` are still constrained.)

If you use a custom STIL protocol file, make sure that all scan-enable signals used by OCC controllers are constrained to the inactive state in all capture procedures.

Enabling On-Chip Clocking Support

To enable OCC support for a design that contains or will contain OCC controllers, use the `-clock_controller` option of the `set_dft_configuration` command:

```
fc_shell> set_dft_configuration -clock_controller enable
```

Specifying OCC Controllers

This topic covers the different methods of specifying OCC controllers in Fusion Compiler:

- [Specifying DFT-Inserted OCC Controllers](#)
- [Specifying Existing User-Defined OCC Controllers](#)
- [Specifying OCC Controllers for External Clock Sources](#)

Specifying DFT-Inserted OCC Controllers

If you have a design that contains an OCC generator, such as a PLL, but not an OCC controller and clock chain, Fusion Compiler can insert both the OCC controller and clock chain. Note that this clock controller design supports only one ATE clock per OCC controller. This topic describes the flow associated with this type of implementation.

The PLL clock is expected to already be connected in the design being run through this flow. Fusion Compiler will disconnect this PLL clock at the hookup location and insert the newly synthesized clock controller at this location.

This topic covers the following:

- [Defining Clocks](#)
- [Defining Global Signals](#)
- [Configuring the OCC Controller](#)
- [Configuring the Clock Chain](#)

- [Configuring the Clock-Gating Logic](#)
- [Performing Timing Analysis](#)
- [Script Example](#)

Defining Clocks

You need to define the reference, PLL, and ATE clocks by using the `set_dft_signal` command. Note that this command does not require you to specify the primary inputs.

This topic covers the following:

- [Reference Clocks](#)
- [PLL-Generated Clocks](#)
- [ATE Clocks](#)

```
fc_shell> set_dft_signal -view existing_dft \  
-type RefClock -port refclk1 \  
-period 10 -timing [list 3 8]
```

Also note the following caveats when defining a reference clock:

- If the reference clock period is an integer divisor of the test default period, then patterns can be written in a variety of formats, including STIL, STIL99, and WGL.
- If the reference clock is not an integer divisor to the test default period, the only format that can be written in a completely correct way is STIL. Other formats, including STIL99, cannot include the reference clock pulses, and a warning is printed, indicating that these pulses must be added back to the patterns manually.
- Do not define a reference clock period or timings with resolution finer than 1 picosecond. The TestMAX ATPG tool cannot work with finer timing resolutions.

```
fc_shell> set_dft_signal -view existing_dft \  
-type PLLClock \  
-hookup_pin PLL/pllclk1  
fc_shell> set_dft_signal -view existing_dft \  
-type PLLClock \  
-hookup_pin PLL/pllclk2  
fc_shell> set_dft_signal -view existing_dft \  
-type PLLClock \  
-hookup_pin PLL/pllclk3  
  
fc_shell> set_dft_signal -view existing_dft \  
-type ATEClock \  
-port ATEclk \  
-timing [list 45 55]
```

By default, Fusion Compiler makes the ATE clock connection at the source port specified in the `-view existing_dft` signal definition. To specify a hookup pin to be used for the clock connection, use the `-hookup_pin` option in a subsequent `-view spec` scan clock signal definition. For example,

```
fc_shell> set_dft_signal -view spec \  
-type ATEClock \  
-port ATEclk \  
-hookup_pin PAD_ateclk/Z
```

You can use the same clock port as both the ATE clock and PLL reference clock. However, caveats apply. For more information, see [SolvNetPlus article 037838, "How Can I Use the Same Clock Port for the ATE and PLL Reference Clocks?"](#)

Reference Clocks

Reference clock signals are always defined in the existing DFT view. The `insert_dft` command does not connect them because they are considered to be functional signals rather than test signals. The only effect of defining them is that they are defined in the test protocol for use by DRC in the TetraMAX tool. For some special cases, a reference clock signal might not be needed.

See [Script Example](#) to see how to define a PLL reference clock.

PLL-Generated Clocks

For Fusion Compiler to correctly insert the OCC, you must define the PLL-generated clocks as well as the point at which they are generated. See [Script Example](#) to see how to define a set of launch and capture clocks for internal scannable elements controlled by the OCC controller.

ATE Clocks

See [Script Example](#) to see how to define the signal behavior of the ATE-provided clock required for shifting scan elements.

Defining Global Signals

You must identify the top-level interface signals that control the OCC controller. This includes the OCC bypass, OCC reset, and ScanEnable signals. You must also define a dedicated TestMode signal that activates the OCC controller logic. In the OCC controller insertion flow, these signals are defined with the `-view spec` option because they will be implemented and connected by the `insert_dft` command.

The following examples show how to define a set of OCC controller interface signals for the design example:

```
fc_shell> set_dft_signal -view spec \  
-type occ_reset \  
-port OCC_reset
```

```
fc_shell> set_dft_signal -view spec \
    -type occ_bypass \
    -port OCC_bypass
fc_shell> set_dft_signal -view spec \
    -type ScanEnable \
    -port SE
fc_shell> set_dft_signal -view spec \
    -type TestMode -usage occ \
    -port TM_OCC
```

The TestMode signal must be a dedicated signal for the OCC controller. It must be active in all test modes and inactive in mission mode. It cannot be shared with TestMode signals used for other purposes, such as test-mode selection.

In the internal pins flow, you can specify internal hookup pins for these OCC control signals by using the `-hookup_pin` option of the `set_dft_signal` command. However, you cannot specify internal hookup pins for ATE clocks or reference clocks.

Configuring the OCC Controller

To specify where to insert a DFT-inserted OCC controller, use the `set_dft_clock_controller` command. Note the following syntax and descriptions:

```
set_dft_clock_controller
-cell_name cell_name
-ateclock clock_name
-test_mode_port port_name
-pllclocks ordered_list
[-chain_count integer]
[-cycles_per_clock integer]
```

Table 11 *set_dft_clock_controller Command Syntax*

Option	Description
<code>-cell_name <i>cell_name</i></code>	Specifies the hierarchical name of the clock controller cell.
<code>-ateclock <i>clock_name</i></code>	Specifies the ATE clock (port) you want to connect to the OCC controller. Note: You cannot specify multiple clocks per controller.
<code>-test_mode_port <i>port_name</i></code>	Specifies the test-mode port used to enable the clock controller. Use this option if you have multiple test-mode ports and you want to use a specific port to enable the clock controller. The specified port must be defined as a TestMode signal using the <code>set_dft_signal</code> command.
<code>-pllclocks <i>ordered_list</i></code>	For asynchronous OCC controllers, specifies the ordered list of PLL output clock pins to control.

Table 11 *set_dft_clock_controller Command Syntax (Continued)*

Option	Description
<code>-chain_count integer</code>	Specifies the number of clock chains. The default number of clock chains is one.
<code>-cycles_per_clock integer</code>	Specifies the maximum number of capture cycles per clock. You should specify a value of two or greater. Capture cycles are cycles during capture when capture clocks are pulsed. Typically, for at-speed transition testing, there are two capture cycles: one is used for launching a transition and the other for capturing the effect of that transition.

Use the `-pllclocks` option to specify the list of clock source pins to be controlled.

The following example inserts an OCC controller that controls three clocks:

```
fc_shell> set_dft_clock_controller \
-cell_name occ_int \
-ateclock { ATEclk } \
-test_mode_port { occ_test_mode } \
-pllclocks { pll/pllclk1 pll/pllclk2 pll/pllclk3 } \
-cycles_per_clock 2
```

To insert multiple OCC controllers, use multiple `set_dft_clock_controller` commands.

Configuring the Clock Chain

By default, the clock chain is constructed as follows:

- In scan compression modes, the clock chain registers exist on their own compressed chain.
- In standard scan modes, the clock chains can be combined with other compressed chains to form scan chains.

You can use the `set_scan_path` command with the `-class occ` option to define a particular structure. Specify the cell name of an OCC controller to reference its clock chain.

For example, to create an external clock chain for two OCC controllers,

```
set_dft_clock_controller -cell_name BLK1/OCC1 ...
set_dft_clock_controller -cell_name BLK2/OCC2 ...
set_scan_path OCC_CHAIN -class occ \
-include_elements {BLK1/OCC1 BLK2/OCC2} \
-scan_data_in SI_OCC -scan_data_out SO_OCC
```

```
set_dft_clock_controller -cell_name BIG_OCC -chain_count 2 \
  -pll_clocks {(many clocks)} ...
```

In `set_scan_path` specifications, the OCC controller cell name references all clock chains for that OCC controller. To reference individual clock chains, append an index number using the colon (:) character:

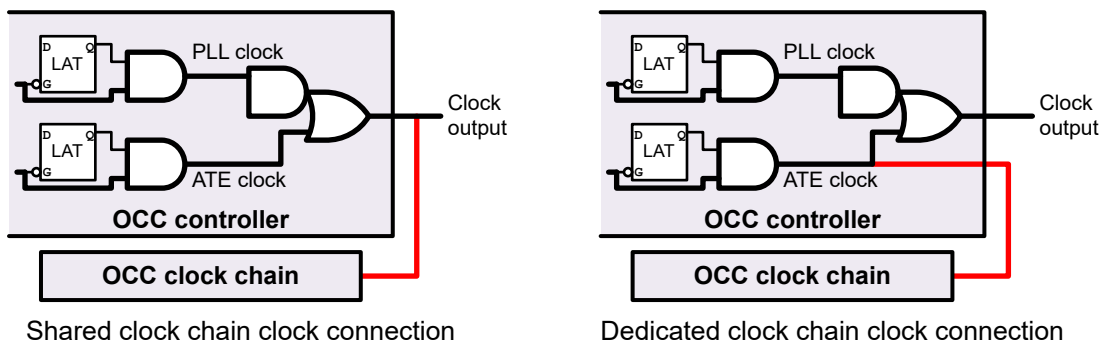
```
set_dft_clock_controller -cell_name BIG_OCC -chain_count 2 \
  -pll_clocks {(many clocks)} ...
set_scan_path OCC_CHAIN1 -class occ \
  -include_elements {BIG_OCC:0} \
  -scan_data_in SI_OCC1 -scan_data_out SO_OCC1
set_scan_path OCC_CHAIN2 -class occ \
  -include_elements {BIG_OCC:1} \
  -scan_data_in SI_OCC2 -scan_data_out SO_OCC2

fc_shell> set_app_options -name dft.test_dedicated_clock_chain_clock
  -value true
```

This creates a dedicated OCC controller clock output for the clock chain that places it in only the ATE clock path, which prevents it from affecting the high-speed PLL clock path.

Figure 128 shows both types of clock-chain clock connections.

Figure 128 Shared and Dedicated Clock Chain Clock Connections



In both cases, you should consider how the clock-chain clock connection interacts with clock tree synthesis (CTS).

Handling Long Clock Chains

The default is to create a single clock chain. However, if you have many clocks to control, the clock chain might become longer than other scan chains.

To split the clock control bits up across multiple clock chains, use the `-chain_count` option.

Configuring the Clock-Chain Clock Connection

By default, the clock-chain clock connection shares the first functional clock output of the OCC controller. This places the clock chain in both the PLL and ATE clock paths.

To use a dedicated clock-chain clock connection from the OCC controller design, set the following application option:

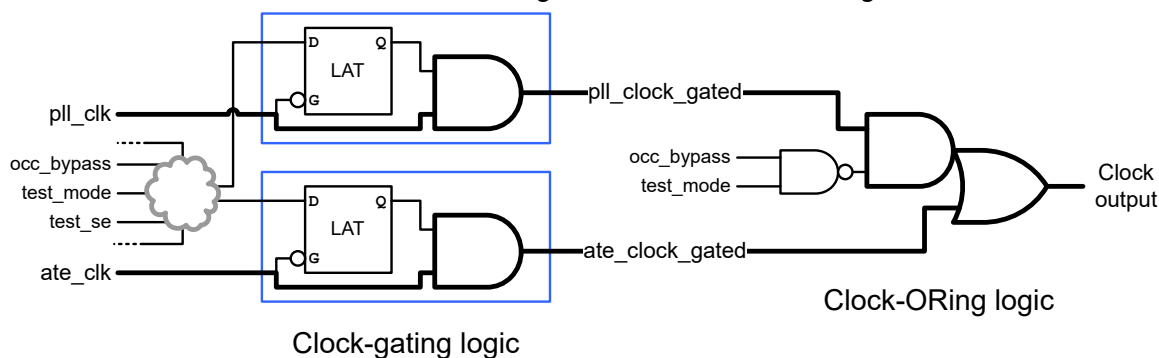
```
fc_shell> set_app_options -name dft.test_dedicated_clock_chain_clock
-value true
```

Configuring the Clock-Gating Logic

The DFT-inserted OCC controller uses latch-based clock-gating logic for glitch-free selection between the fast and slow clocks. For the fast and slow clocks, the combinational gating-enable signals are combined and latched for each clock; the latched enable signal is then used to gate the clock.

By default, the tool builds the clock-gating logic using discrete latch and combinational library cells, as shown in [Figure 129](#). Clock paths are shown in bold, and rectangles indicate hierarchy created inside the OCC controller block by DFT insertion.

Figure 129 Default Latch-Based Clock-Gating and Clock Selection Logic Structure

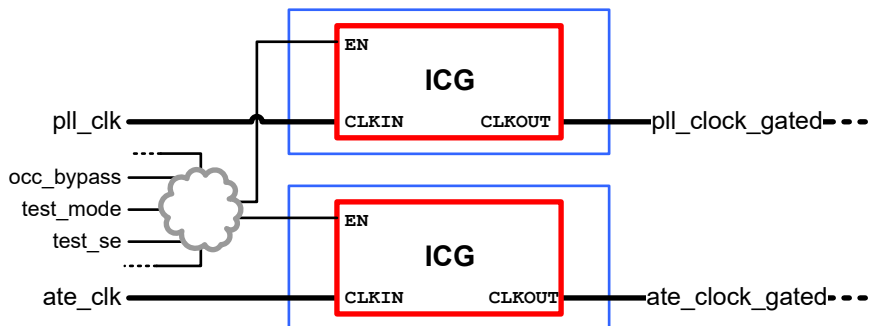


To use an integrated clock-gating cell instead, as shown in [Figure 130](#), set the targeted library cell reference (without the library name) as follows:

```
fc_shell> set_app_options -name dft.test_occ_insert_clock_gating_cells
-value true
```

```
fc_shell> set_app_options -name dft.test_icg_p_ref_for_dft
-value CGLPPRX2_HVT
```

Figure 130 Specifying an Integrated Clock-Gating Cell for the Clock-Gating Logic



For more information about this application option, see the man page.

See Also

- [SolvNetPlus article 034274, “DFT-inserted OCC Controller Data Sheet”](#) for more information about the logic structure and operation of the DFT-inserted OCC controller

Performing Timing Analysis

After DFT insertion completes, you must ensure that the OCC controller logic is properly constrained for timing analysis.

See Also

- [SolvNetPlus article 022490, “Static Timing Analysis Constraints for On-Chip Clocking Support”](#) for more information about performing timing analysis in a DFT-inserted OCC controller flow
- [SolvNetPlus article 034274, “DFT-Inserted OCC Controller Data Sheet”](#) for information about synthesizing a DFT-inserted OCC controller (two sections titled “Special Considerations”)

Script Example

[Example 13](#) shows a script that performs pre-DFT DRC, scan chain stitching, and post-DFT DRC. The STIL protocol file generated at the end of the DFT insertion process contains PLL clock details suitable for the TestMAX ATPG tool.

Example 13 Flow Example for DFT-Inserted OCC Controller and Clock Chain

```
# Define the PLL reference clock
# top level free running clock
set_dft_signal -view existing_dft -type RefClock \
    -port refclk1 -period 100 -timing {45 55}

# Define the ATE clock
# the ATE-provided clock for shift of scan elements
```

```
set_dft_signal -view existing_dft -type ATEClock \
    -port ATEclk -timing {45 55}

# Define the PLL generated clocks --
# these are the launch/capture clocks for internal scannable
# elements and are controlled by occ controller
set_dft_signal -view existing_dft -type PLLClock \
    -hookup_pin pll/pllclk1
set_dft_signal -view existing_dft -type PLLClock \
    -hookup_pin pll/pllclk2
set_dft_signal -view existing_dft -type PLLClock \
    -hookup_pin pll/pllclk3

# Define the OCC TestMode signal
set_dft_signal -type TestMode -usage occ -port TM_OCC -view existing_dft

# Enable on-chip clocking control
set_dft_configuration -clock_controller enable

# The following command specifies the OCC controller
# design to be instantiated
set_dft_clock_controller \
    -cell_name snps_occ_controller \
    -ateclock { ATEclk } \
    -test_mode_port { TM_OCC } \
    -pllclocks { pll/pllclk1 pll/pllclk2 pll/pllclk3 } \
    -cycles_per_clock 2
set_scan_path OCC_CHAIN -include_elements { snps_occ_controller }
-set_scan_data_in ... -scan_data_out ...
set_scan_configuration -chain_count 30 -clock_mixing no_mix
report_dft_clock_controller
create_test_protocol
preview_dft
insert_dft

# Run DRC with external clocks enabled during capture
# (PLL bypassed)
set_dft_drc_configuration -occ_bypass enable
dft_drc -test_mode ...
change_names -rules verilog -hierarchy
write_verilog -hierarchy all top.scan.v

# Write out combined PLL enabled/bypassed test protocol:
write_test_protocol -output scan_pll.spf
```

Specifying Existing User-Defined OCC Controllers

If you have a design that contains an OCC generator and it already instantiates an existing user-defined OCC controller and clock chain, Fusion Compiler can analyze the OCC controller, validate the functionality, and incorporate it into the test protocol.

This topic covers the following:

- [Defining Clocks](#)
- [Defining Global Signals](#)
- [Specifying Clock Chains](#)
- [Script Example](#)

Defining Clocks

Use the `set_dft_signal` command to define the following clock signals:

- [Reference Clocks](#)
- [PLL-Generated Clocks](#)
- [ATE Clocks](#)
- [Clock Chain Configuration and Control-Per-Pattern Information](#)

```
fc_shell> set_dft_signal -view existing_dft \  
                  -type RefClock -port refclk1 \  
                  -period 10 -timing [list 3 8]
```

Also note the following caveats associated when defining a reference clock:

- If the reference clock period is an integer divisor of the test default period, then patterns can be written in a variety of formats, including STIL, STIL99, and WGL.
- If the reference clock is not an integer divisor of the test default period, the only format that can be written in a completely correct way is STIL. Other formats (including STIL99) cannot include the reference clock pulses. The tool issues a warning to indicate that these pulses must be added back to the patterns manually.
- Do not define a reference clock period or timings with resolution finer than 1 picosecond. The TestMAX ATPG tool cannot work with finer timing resolutions.

```
fc_shell> set_dft_signal -view existing_dft \  
                  -type PLLClock \  
                  -hookup_pin PLL/pllclk1  
fc_shell> set_dft_signal -view existing_dft \  
                  -type PLLClock \  
                  -hookup_pin PLL/pllclk2  
fc_shell> set_dft_signal -view existing_dft \  
                  -type PLLClock \  
                  -hookup_pin PLL/pllclk3  
  
fc_shell> set_dft_signal -view existing_dft \  
                  -type ATEClock \
```

```
-port ATEclk \  
-timing [list 45 55]
```

You can use the same clock port as both the ATE clock and PLL reference clock. However, caveats apply. For more information, see [SolvNetPlus article 037838, “How Can I Use the Same Clock Port for the ATE and PLL Reference Clocks?”](#)

```
fc_shell> set_dft_signal -view existing_dft \  
-type MasterClock -usage occ \  
-hookup_pin occ_int/intclk1 \  
-ate_clock ATEclk \  
-pll_clock PLL/pllclk1 \  
-ctrl_bits [list 0 occ_int/FF_cyc1/Q 1 \  
                1 occ_int/FF_cyc2/Q 1 \  
                2 occ_int/FF_cyc3/Q 1]
```

The `-ctrl_bits` option is used to provide a list of triplets that specify the sequence of bits needed to enable the propagation of the clock generator outputs. The first element of each triplet is the cycle number (integer) indicating the cycle where the clock signal will be propagated. The second element is the pin name (a valid design hierarchical pin name) of the clock chain control bit. The third element is the active state (0 or 1) of the control bit signal. For more information about this option, see the `set_dft_signal` man page.

The `-view existing_dft` option is used because connections already exist between the referenced port and the clock controller.

Reference Clocks

A reference clock definition is used primarily as an informational device. The only effect of defining them is that they are defined in the test protocol for use by DRC. For some special cases, a reference clock signal might not be needed.

The following example shows how to define a PLL reference clock:

PLL-Generated Clocks

PLL clocks are the output of the PLL. This output is a free-running source that also runs at a constant frequency, which might not be the same as the reference clock's. This information is forwarded to the TestMAX ATPG tool through the protocol file to allow the verification of the clock controller logic.

The following commands show how to define the PLL clocks for the design example:

ATE Clocks

An ATE clock signal can be pulsed several times before and after scan shift (scan-enable signal inactive) to synchronize the clock controller logic in the capture phase and back into the shift phase.

The following commands show how to define ATE clocks for the design example:

Clock Chain Configuration and Control-Per-Pattern Information

You must specify the correlation between the internal clock signals driven from the clock controller outputs, the signals driven from the clock generator (PLL) outputs, and the signals provided by the user-defined clock chain. This information indicates how the clock signal is enabled by the clock chain control bits in each clock cycle. The correspondence between the controlled internal clock signals and clock chain control bits is identified in the protocol file for TestMAX ATPG to generate patterns.

You must specify

- All user-defined clock controller outputs referencing the internal clocks
- A corresponding set of clock chain control bits, ATE clock, and clock generator output (PLL) for each clock controller output

The following example specifies a user-defined OCC controller with a three-bit clock chain:

Defining Global Signals

You must identify the top-level interface signals that control the OCC controller. This includes the OCC bypass, OCC reset, and ScanEnable signals. You must also define a dedicated TestMode signal that activates the OCC controller logic. In the user-defined OCC controller flow, these signals are defined with the `-view existing_dft` option because they already exist and must be described to the tool.

The following examples show how to define a set of OCC controller interface signals for the design example:

```
fc_shell> set_dft_signal -view existing_dft \  
    -type occ_reset \  
    -port OCC_reset  
fc_shell> set_dft_signal -view existing_dft \  
    -type occ_bypass \  
    -port OCC_bypass  
fc_shell> set_dft_signal -view existing_dft \  
    -type ScanEnable \  
    -port SE  
fc_shell> set_dft_signal -view existing_dft \  
    -type TestMode -usage occ \  
    -port TM_OCC
```

The TestMode signal must be a dedicated signal for the OCC controller. It must be active in all test modes and inactive in mission mode. It cannot be shared with TestMode signals used for other purposes, such as AutoFix or multiple test-mode selection.

You can specify an internal hookup pin for any of these OCC controller interface signals by using the `-hookup_pin` option of the `set_dft_signal` command. You cannot specify internal hookup pins for ATE clocks or reference clocks.

Specifying Clock Chains

The clock chain is a shift register with an enable, typically clocked on the rising edge. It is always defined with the `set_scan_path` command and must be defined with the `-class occ` option to mark it as the clock chain.

In your OCC controller netlist, use the `set_scan_group` command to define the cell order and access pins of the shift register:

```
fc_shell> set_scan_group my_cc_group \
-include_elements { \
  my_occ/my_clock_chain/qmask_reg[0] \
  my_occ/my_clock_chain/qmask_reg[1] \
  my_occ/my_clock_chain/qmask_reg[2]} \
-access { \
  ScanDataIn my_occ/my_clock_chain/si \
  ScanDataOut my_occ/my_clock_chain/so \
  ScanEnable my_occ/my_clock_chain/se} \
-serial_routed true
```

Then, use the `set_scan_path` command with the `-class occ` option to mark this shift group as a clock chain:

```
fc_shell> set_scan_path my_clock_chain -class occ \
-include_elements {my_cc_group}
```

Script Example

When you run a design that contains a user-defined OCC controller and clock chains, a STIL protocol file is generated, as shown in [Example 14](#).

Example 14 Flow Example for Existing OCC Controller and Clock Chain

```
# Define the PLL reference clock
# (top-level free running clock)
set_dft_signal -view existing_dft -type RefClock -port ref_clk \
-period 100 -timing {45 55}

# Define the ATE clock
set_dft_signal -type AteClock -port ate_clk -timing [list 45 55]

# Specify global controller signal
set_dft_signal -type occ_reset -port occ_reset
set_dft_signal -type occ_bypass -port occ_bypass

# Define the OCC TestMode signal
set_dft_signal -type TestMode -port occ_test_mode -usage occ
```

```
#-----
# # existing clock controller specification
#-----
# Define the PLL generated clocks
set_dft_signal -type PLLClock -hookup_pin pll_1/pll_clk -usage occ
-timing [list 45 55]
set_dft_signal -type PLLClock -hookup_pin pll_2/pll_clk -usage occ
-timing [list 45 55]

# Enable PLL capability
set_dft_configuration -clock_controller enable

# Specify clock controller output and control-per-pattern information
set_dft_signal -type MasterClock -usage occ \
  -hookup_pin my_occ_ctrl/clk[0] \
  -pll_clock pll_1/pll_clk \
  -ate_clock ate_clk \
  -ctrl_bits [ list 0 my_cc/U_dftShiftReg_0_0/ff3/q_reg/Q 1 \
                  1 my_cc/U_dftShiftReg_0_0/ff2/q_reg/Q 1 ] \
  -active_state 1 \
  -view existing -timing [list 45 55]

set_dft_signal -type MasterClock -usage occ \
  -hookup_pin my_occ_ctrl/clk[1] \
  -pll_clock pll_2/pll_clk \
  -ate_clock ate_clk \
  -ctrl_bits [ list 0 my_cc/U_dftShiftReg_0_0/ff1/q_reg/Q 1 \
                  1 my_cc/U_dftShiftReg_0_0/ff0/q_reg/Q 1 ] \
  -active_state 1 \
  -view existing -timing [list 45 55]

set_scan_configuration -chain_count 4
set_scan_compression_configuration -chain_count 70 -inputs 8 -outputs 8

# Define the existing clock chain segments
set_scan_group my_cc_group \
  -include_elements [list my_cc/U_dftShiftReg_0_0/ff3/q_reg \
                        my_cc/U_dftShiftReg_0_0/ff2/q_reg \
                        my_cc/U_dftShiftReg_0_0/ff1/q_reg \
                        my_cc/U_dftShiftReg_0_0/ff0/q_reg ] \
  -access [ list ScanDataIn my_cc/si[0] \
                ScanDataOut my_cc/so[0] \
                ScanEnable my_cc/se ] \
  -serial_routed true

set_scan_path my_cc_path -include_elements { my_cc_group } -class occ

report_dft_clock_controller

create_test_protocol
dft_drc -test_mode all_dft
```

```

preview_dft
insert_dft

dft_drc -test_mode Internal_scan
dft_drc -test_mode ScanCompression_mode

# (PLL bypassed)
set_dft_drc_configuration -occ_bypass enable
dft_drc -test_mode Internal_scan
dft_drc -test_mode ScanCompression_mode

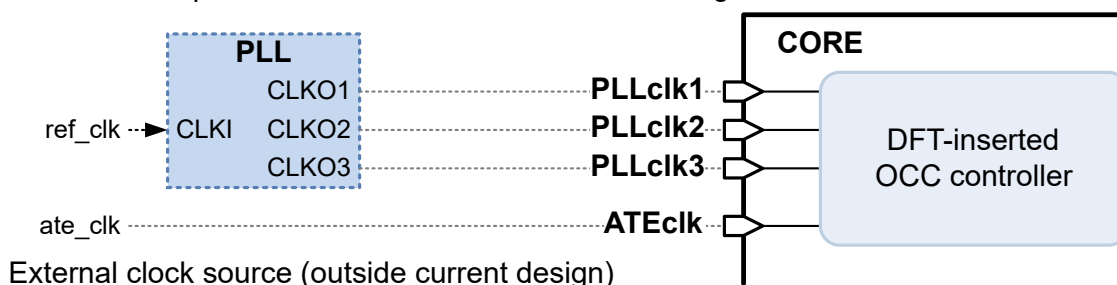
# Test protocol generation
write_test_protocol -test_mode Internal_scan -output scan.occ.spf
write_test_protocol -test_mode ScanCompression_mode -output comp.occ.spf

```

Specifying OCC Controllers for External Clock Sources

In some cases, the on-chip clocking source might be external to the current design, as shown in [Figure 131](#), so that the PLL clocks enter the design through input ports.

Figure 131 On-Chip Clock Source External to Current Design



Define such external clock sources as follows:

- Define the port-driven PLL-generated clocks at the ports, and define them as PLLClock:

```

fc_shell> set_dft_signal -view existing_dft \
  -type PLLClock \
  -port {PLLclk1 PLLclk2 PLLclk3} \
  -timing {45 55}

```

Specify typical scan clock timing, even though the clock is a PLL-generated clock instead of a tester-driven clock. This timing discrepancy does not affect OCC architecture or operation.

- If the reference clock does not enter the design, you do not need to define it.
- Include the PLL-generated input ports in the PLL clock sources list specified with the `-pllclocks` option:

```
fc_shell> set_dft_clock_controller \
-cell_name occ_int \
-test_mode_port TM_OCC \
-pllclocks {PLLclk1 PLLclk2 PLLclk3} \
-ateclock {ATEclk} \
-cycles_per_clock 2
```

The resulting SPF does not pulse the clock ports during capture, which is incorrect. As a workaround, define the clock as a reference clock in the TestMAX ATPG tool. For example,

```
DRC-T> add_clock 0 {PLLclk1 PLLclk2 PLLclk3} -refclock
```

External clock sources are not supported for existing user-defined OCC controllers.

Reporting Clock Controller Information

Use the `report_dft_clock_controller` or `report_dft -clock_controller` command to generate reports.

DFT-Inserted OCC Controller Flow

For DFT-inserted OCC controllers and clock chains, use the `report_dft_clock_controller` or `report_dft -clock_controller` command to output a report. This report displays the options that you set for the `set_dft_clock_controller` command.

[Example 15](#) shows a clock controller report for the DFT-inserted OCC controller flow.

Example 15 Report Example from the report_dft_clock_controller Command

```
*****
Report : OCC Controllers Specification
Design : des_unit
Version: U-2022.12-SP5-VAL
Date   : Tue Aug 1 17:43:20 2023
*****

-----
TEST MODE: all_dft
VIEW      : Specification
-----

Cell name:                occ_ctrl1
Chain count:              1
```

Number of bits per clock:	10
Fast clock pins:	U_PLL/CLK_4X
Slow clock ports/pins:	ate_clk

Controlling the OCC Controller Bypass Path

Use the `set_dft_drc_configuration` and `write_test_protocol` commands to enable the OCC controller bypass configuration for design rule checking. The `-occ_bypass` option of the `set_dft_drc_configuration` command configures post-DFT DRC to place the OCC clock controller in bypass configuration.

The syntax is as follows:

```
set_dft_drc_configuration -occ_bypass enable | disable
```

The default setting is `disable`.

The following example performs DRC of both bypass configurations for a scan compression mode:

```
insert_dft
set_dft_drc_configuration -occ_bypass disable ;# already the default
dft_drc -test_mode ScanCompression_mode
set_dft_drc_configuration -occ_bypass enable
dft_drc -test_mode ScanCompression_mode
write_test_protocol -test_mode Scan_Compression_mode -output
my_design.spf
```

The test protocol written by the `write_test_protocol` command contains information for PLL bypass as well as for PLL enabled. In the TestMAX ATPG tool, use the `run_drc -patternexec` command to select the operating mode to use.

Controlling the OCC/SPC Scan Chain With General Multimode

Architecting OCC/SPC chains in the Fusion Compiler tool for the current flow has limitations for complex combinations of OCC/SPC segments in different test modes as follows:

- You cannot exclude OCC and SPC in segments for the particular test modes.
- The `-complete false` option of the `set_scan_path` command is not supported for the OCC/SPC chains. The elements that are not OCC/SPC cannot be added to the OCC/SPC chains.
- When multiple SPC segments are available, all the SPC segments are handled as a group. Handling of individual segments is not possible.

This feature enables you to have the flexibility to control each test mode independently with different combinations of OCC/SPC segments in any test modes overcoming the mentioned limitations.

To enable this feature, you can set the `dft.scan.occ_spc_chain_arch_flat_flow` application option to `true`.

OCC segments in the `set_scan_path` command are defined based on the cell name used in the controller specification as follows:

```
set_dft_clock_controller -cell_name occ_ctrl -chain_count 1
set_dft_clock_controller -cell_name occ_2chain_ctrl -chain_count 2
set_scan_path occ_chain -class occ -include [list occ_ctrl \
    occ_2chain_ctrl:0 \
    occ_2chain_ctrl:1 ]
```

During the specification given for the SPC in the `set_scan_compression_configuration` command, the feature enables an SPC segment to be created and its name to be printed. You need to explicitly specify this name as part of the `set_scan_path` command.

```
set_scan_compression_configuration \
    -input 5 \
    -output 5 \
    -chain_count 20 \
    -test_mode scan2 \
    -shift_power_groups true \
    -shift_power_chain_length 10 \
    -shift_power_clock clk_st
```

Accepted scan compression specification for design `des_unit` in partition `default_partition`

Created user scangroup `SNPS_PWR_CHAIN_GRP1` for SPC segment, use `SNPS_PWR_CHAIN_GRP1` as spc segment for `set_scan_path`

```
set_scan_path occ_chain -class occ
    -include [list SNPS_PWR_CHAIN_GRP1 \
        occ_ctrl \
        occ_2chain_ctrl:0 \
        occ_2chain_ctrl:1 ]
```

Note:

- Before enabling this feature, you must set the `dft.scan.enable_general_multimode_support` general multi-mode application option to `true`.
- This feature works for the integration of scan-only cores and is not applicable to the integration of compressed cores.
- All the configurations for OCC/SPC chains are done using the `set_scan_path -class OCC/SPC` commands.

- The SPC segment names (printed as part of the output of `set_scan_compression_configuration` command) need to be explicitly specified in the `set_scan_path -class occ/spc -include_elements{}` command.

DFT-Inserted OCC Controller Configurations

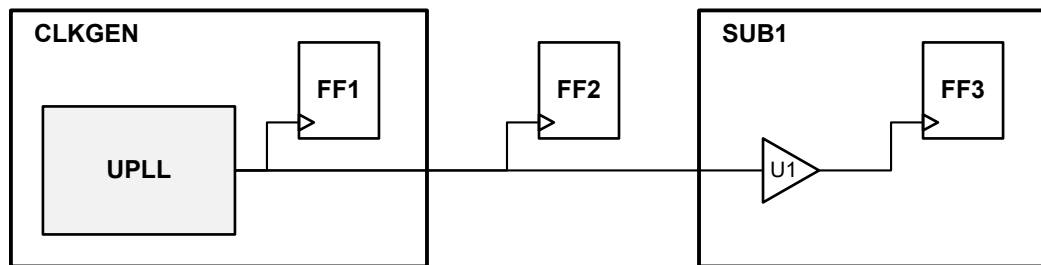
This topic shows DFT-inserted OCC controller configurations and the associated configuration commands, as described in the following topics:

- [Single OCC Controller Configurations](#)
- [Multiple DFT-Inserted OCC Controller Configurations](#)

Single OCC Controller Configurations

This topic shows the results of using various configurations of the `set_dft_clock_controller` command on the design example shown in [Figure 132](#).

Figure 132 Design Example for Single OCC Controller Insertion



The following configuration examples are applied to this design:

- [Example 1](#) – Controller inserted at the output of UPLL, within the CLKGEN block.
- [Example 2](#) – Controller inserted at the output of the CLKGEN block.
- [Example 3](#) – Controller inserted at the output of the buffer.

Example 1

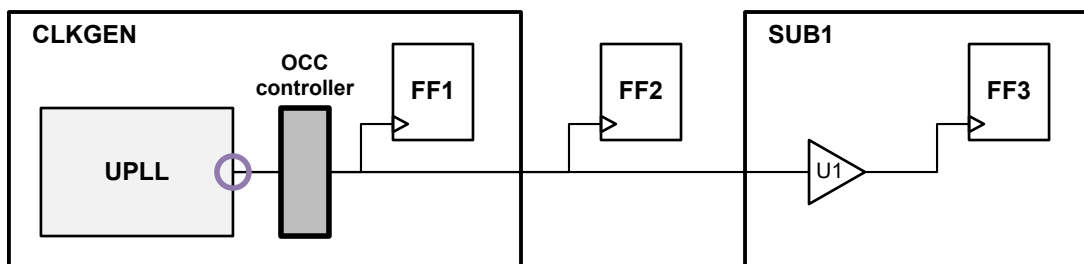
The first example, shown in [Figure 133](#), uses the following configuration:

```
fc_shell> set_dft_clock_controller \  
-pllclks {CLKGEN/UPLL/clkout}
```

In this case, the following occurs:

- The controller is inserted at the output of PLL, within the clkgen1 block.
- The clocks of all flip-flops are controllable.

Figure 133 Controller Inserted at Output of PLL Within CLKGEN Block



Example 2

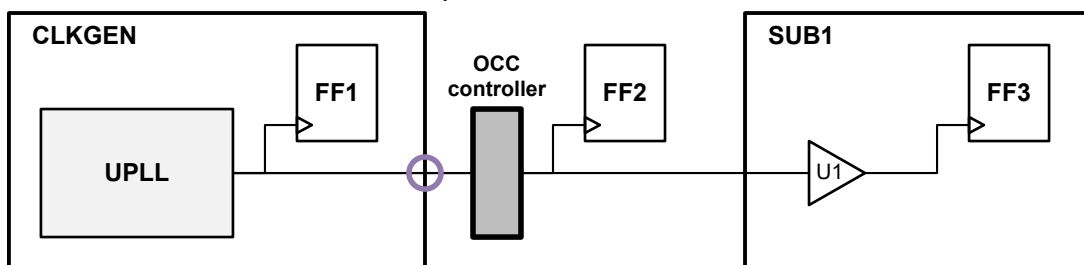
The second example, shown in Figure 134, uses the following configuration:

```
fc_shell> set_dft_clock_controller \
  -pllclocks {CLKGEN/clkout}
```

In this case, the following occurs:

- The controller is inserted at the output of the CLKGEN block.
- The FF1 clock remains uncontrollable.

Figure 134 Controller Inserted at Output of CLKGEN Block



Example 3

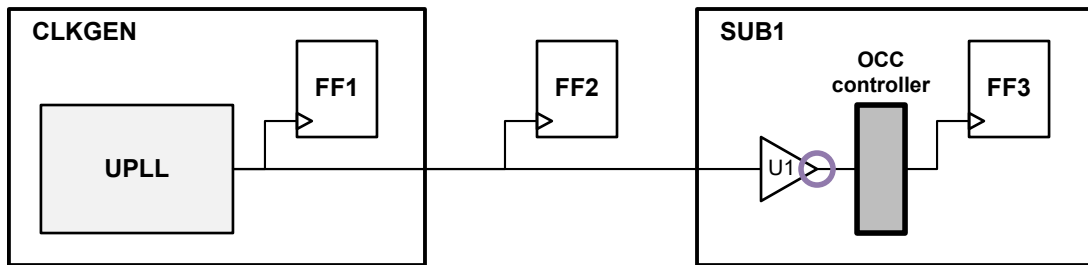
The third example, shown in Figure 135, uses the following configuration:

```
fc_shell> set_dft_clock_controller \
  -pllclocks {SUB1/U1/Z}
```

In this case, the following occurs:

- The controller is inserted at the output of buffer U1.
- The FF1 and FF2 clocks remain uncontrollable.

Figure 135 Controller Inserted at Output of the Buffer



Multiple DFT-Inserted OCC Controller Configurations

This topic shows the results of configuring multiple DFT-inserted OCC controllers for the design example shown in [Figure 132](#).

When multiple PLLs exist in a design, the reference clock input to each PLL cell must be a free-running clock in test mode. Care must be taken to insure that an OCC controller is not inserted at a location that would block a free-running clock to a downstream PLL cell.

In this design example, the primary PLL named UPLL1 receives the incoming reference clock and generates a PLL output clock. This PLL output clock then feeds either a second PLL or clock divider cell, creating a second cascaded PLL output clock.

The following configuration examples are applied to this design:

- [Example 1](#) – Controller incorrectly inserted, at the output of UPLL1.
- [Example 2](#) – Controller correctly inserted, at the output of a buffer driven by UPLL1.

Example 1

The first example, shown in [Figure 136](#), uses the following configuration:

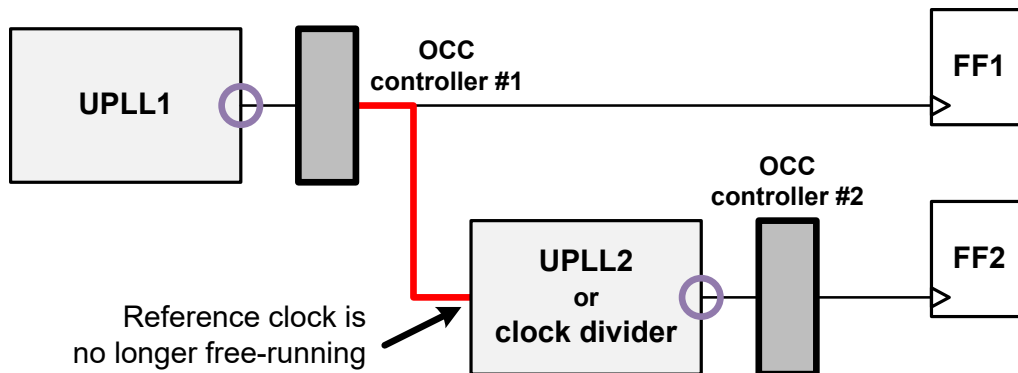
```
fc_shell> set_dft_clock_controller \
    -pllclocks {UPLL1/clkout}
fc_shell> set_dft_clock_controller \
    -pllclocks {UPLL2/clkout}
```

In this case, the following occurs:

- The controller is inserted at the output of UPLL1.
- As a result, the free-running clock to UPLL2 is blocked by the first OCC controller, causing incorrect operation of UPLL2.

The incorrect operation of UPLL2 might only be detectable during Verilog simulation of the resulting netlist.

Figure 136 Free-Running Clock Blocked to UPLL2



Example 2

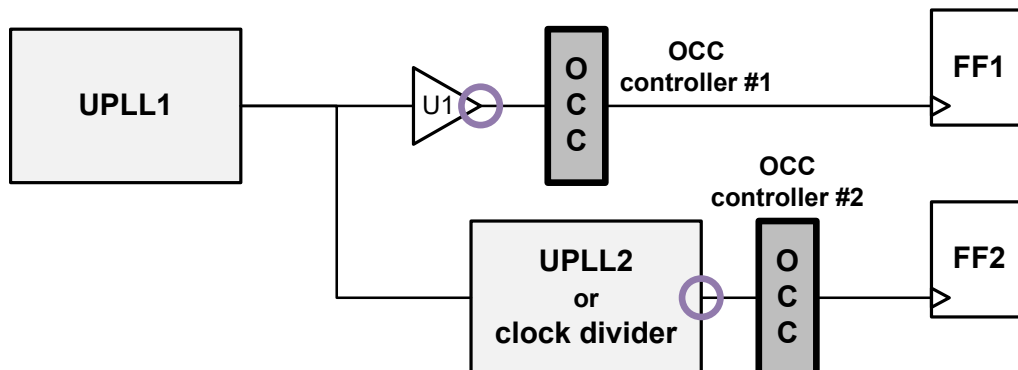
The second example, shown in [Figure 137](#), uses the following configuration:

```
fc_shell> set_dft_clock_controller \  
-pllclocks {U1/Z}  
fc_shell> set_dft_clock_controller \  
-pllclocks {UPLL2/clkout}
```

This example uses a buffer to isolate the downstream fanout that the first OCC controller should drive. In this case, the following occurs:

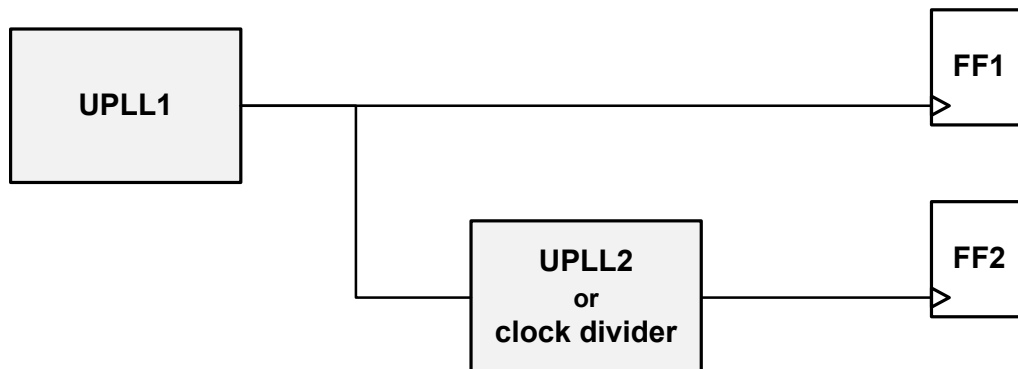
- The controller is inserted at the output of buffer U1 driven by UPLL1.
- As a result, the free-running clock from UPLL1 propagates to UPLL2, allowing correct operation of UPLL2.

Figure 137 Free-Running Clock Propagates to UPLL2



You must ensure that the isolation buffer is not optimized away by applying a `set_dont_touch` command, applying a `set_size_only` command, or using hierarchy. After the clock controller is inserted, the resulting test protocol references the specified pin as a PLL pin.

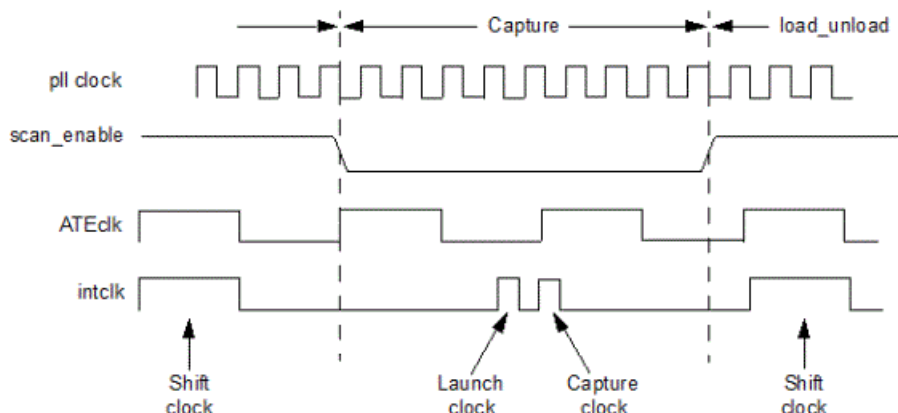
Figure 138 Design Example for Multiple OCC Controller Insertion



Waveform and Capture Cycle Example

Figure 139 shows an example of the relationship between various clocks when the design contains an OCC generator and an OCC controller.

Figure 139 Desired Clock Launch Waveform Example



For information about `pll clock`, `ATEclk`, and `intclk`, see [Clock Type Definitions](#).

Limitations

Note the following limitations:

- In Fusion Compiler,
 - Clock chains must be defined with the `set_scan_path -class occ` command; they are not automatically created.
- Clock chains are reported by the `report_scan_path` or `report_dft -scan_path` commands after `insert_dft` command.
- Inferencing internal PLL or any reference clocks is not supported. For pre-DFT DRC, you must explicitly define your reference clock, ATE clock, and PLL clocks.
- You cannot mix the DFT-inserted and user-defined OCC controller types in the same DFT insertion run. However, this restriction does not apply to cores that already contain OCC controllers.
- The only supported scan style is multiplexed flip-flop.
- Fast-sequential patterns with OCC support cannot measure the primary outputs between system pulses. The measure primary output is placed before the first system pulse and measures only Xs. You have to use pre-clock-measure, with the strobe being placed before the clock.
- External clocks, which have a direct connection to scan flip-flops, cannot serve as ATE clocks for the OCC controller.

- End-of-cycle measures cannot be used when an OCC controller is used to control the clock.
- When a pipelined scan-enable signal is used with OCC flows, the `insert_dft` command fails to make some connections properly. To use these features together, you must check and correct the connections so that the following requirements are met:
 - The scan-enable connections to the OCC controller and clock chain must use the unpipelined scan-enable signal. That is, use the input to the scan-enable pipeline register instead of its output.
 - The clock connection to the scan-enable pipeline register in OCC controller clock domains must be connected to the internal clock output of the OCC controller block.
- External (port-driven) clock sources are not supported for existing user-defined OCC controllers.
- The existing user-defined OCC feature works with DFTMAX/DFTMAX Ultra compression.

14

Inserting Test Points

Test points are points in the design where Fusion Compiler inserts logic to improve the testability of the design. The tool can automatically determine where to insert test points to improve test coverage and reduce pattern count. You can also manually define where test points are to be inserted.

The test point capabilities are described in the following topics:

- [Test Point Types](#)
- [Test Point Structures](#)
- [Automatically Inserted Test Points](#)
- [User-Defined Test Points](#)
- [Previewing the Test Point Logic](#)
- [Inserting the Test Point Logic](#)
- [Inserting Power-Aware Test Points](#)
- [Clock Gating Support for Test Point Inserted Logic](#)
- [Enhancing Test Point Physical Clustering](#)
- [Specifying Fault Space Instances for Test Point Analysis](#)

Test Point Types

The available test point types are:

- [Force Test Points](#)
- [Control Test Points](#)
- [Observe Test Points](#)
- [Multicycle Test Points](#)

The test point schematics in these topics show the functional operation of the test points. During synthesis, constant logic is simplified, and the test point logic might be optimized into the surrounding logic.

Force Test Points

Force test points allow a signal to be overridden (always force) throughout the entire test program. They are typically used to block some other value (such as an X value) from propagating.

The following force test point types are available:

- `force_0`
- `force_1`
- `force_01`

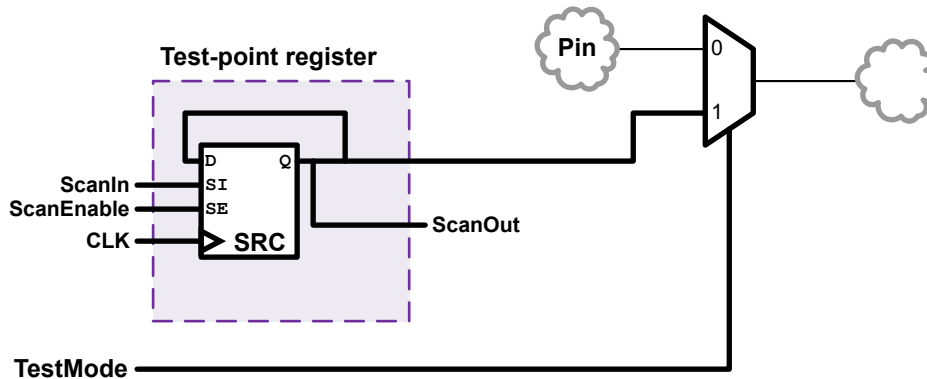
The `force_0` and `force_1` test point types allow a signal to be replaced with a constant 0 or constant 1 value throughout the entire test session. These test point types are useful when a particular signal must be forced to a known value for testability purposes. A logic gate is used to replace the original signal with a fixed constant 0 or 1 value when the TestMode signal is asserted. See [Figure 140](#).

Figure 140 Example of a `force_0` or `force_1` Test Point



The `force_01` test point type allows a signal to be replaced with a scan-selected value throughout the entire test session. A multiplexer is used to replace the original signal with the output of this scan register when the TestMode signal is asserted. See [Figure 141](#).

Figure 141 Example of a force_01 Test Point



The forced value can vary per-pattern (as the scan register reloads with each pattern), but it remains constant for all capture cycles within a given pattern.

Control Test Points

Control test points allow a hard-to-control signal to be controllable (selectively forced) for some test vectors but not others. They are typically inserted to increase the fault coverage of the design. They provide some control while still allowing some observation of upstream logic.

The following control test point types are available:

- `control_0`
- `control_1`

A `control_0` or `control_1` test point is built with a controlling logic gate, an enabling AND gate, and a source scan register. When `TestMode` is not asserted, the signal always retains its original value. When `TestMode` is asserted, the signal is forced with a fixed constant 0 or 1 value only when the output of the scan register selects the constant value. This allows the test program to select either the original signal behavior or the constant-forced behavior on a per-pattern basis. See [Figure 142](#) and [Figure 143](#).

Figure 142 Example of a control_1 Test Point

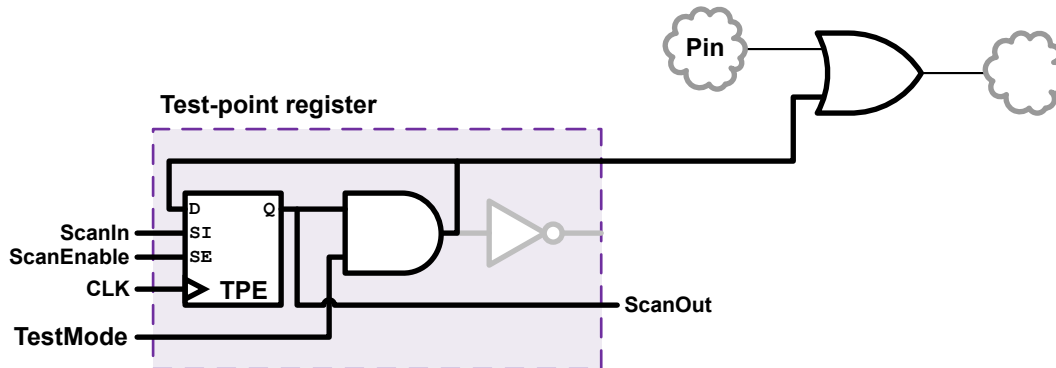
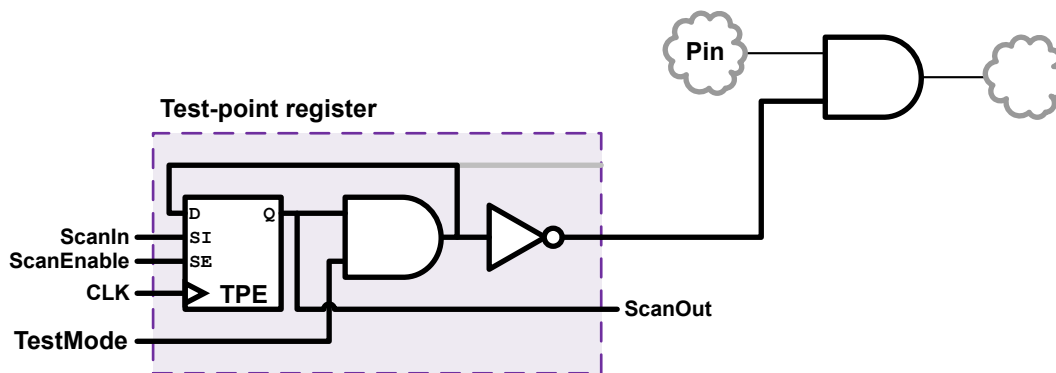


Figure 143 Example of a control_0 Test Point



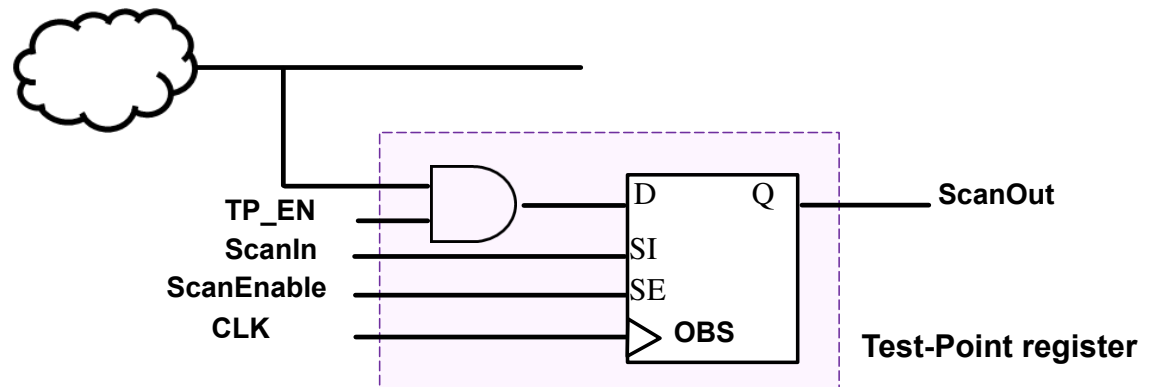
A control point can be enabled or disabled per-pattern, but its assertion behavior remains constant across capture cycles within a given pattern.

Observe Test Points

The `observe` test point type is typically inserted at hard-to-observe signals in a design to reduce test data volume or to increase coverage.

An `observe` test point is a scan register with its data input connected to the signal to be observed. See [Figure 144](#).

Figure 144 Example of an observe Test Point



Multicycle Test Points

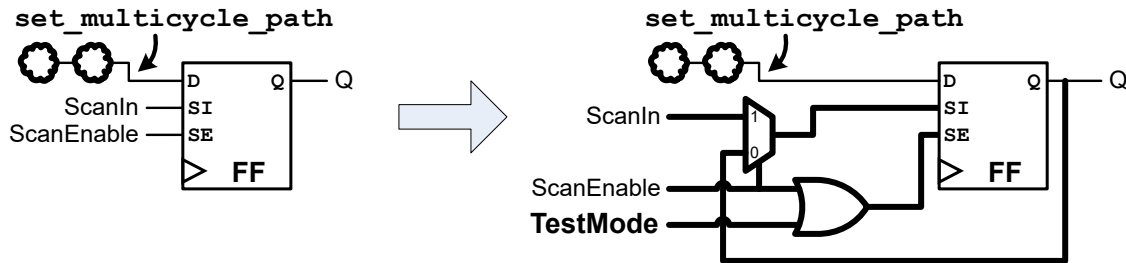
During ATPG, when a scan cell captures a value from a logic path constrained by a multicycle path exception, it captures a dynamic X value because the multicycle logic might not be stable by the capturing clock edge. These captured X values can affect other captured values in compressed designs, and they can propagate through the scan cell into other areas of the design when fast sequential ATPG is used.

The multicycle test point prevents these X values from being captured. It implements the following scan capture behavior:

- When the TestMode signal is asserted, the register holds state instead of capturing.
- When the TestMode signal is deasserted, the register captures normally.

Scan shift operation and functional operation are unaffected. This test point uses reconfigured scan path logic to avoid inserting logic along the functional path. See [Figure 145](#).

Figure 145 Example of a Multicycle Test Point



The multicycle test point does not provide coverage for the blocked capture path. However, it does prevent multicycle X values from propagating into scan compression logic or self-test logic.

Multicycle test points can only be inserted using automatic test point insertion.

Test Point Structures

The following topics describe how test point logic is structured:

- [Test Point Components](#)
- [Test Point Register Clocks](#)
- [Sharing Test Point Registers](#)

Test Point Components

Test points are constructed from (up to) three primary components:

- **Pin**

The functional pin where the test point is inserted to assert its behavior. This is the pin where the test point is located. Every test point has a corresponding insertion pin.

- **Register**

A scan-controllable register that provides source (driving) or sink (capturing) capability to the test point logic. A single register can be shared by multiple test points. Some test points do not use a register.

- **Control signal**

The TestMode or IbistEnable signal that activates the test point logic.

[Table 12](#) summarizes which components are used by each type of test point.

Table 12 Components Used by Each Type of Test Point

Test point type	Source register?	Sink register?	Control signal?
force_0, force_1			X
force_01	X		X
control_0, control_1	X		X
observe		X	X

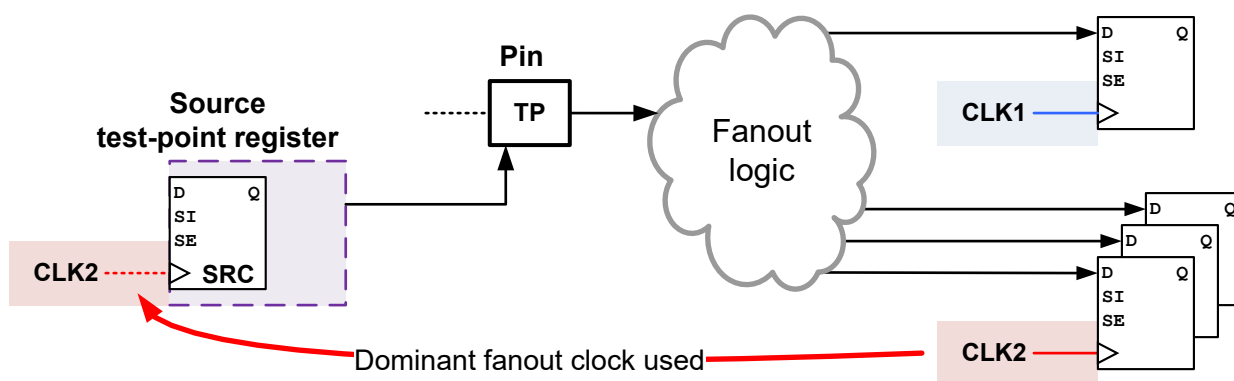
Test point registers do not have a reset signal; their value is set by scan shift when used and blocked by the control signal when not used.

Test Point Register Clocks

As described in [Test Point Components](#), some test point types use a register to drive (source) or capture (sink) data. By default, the tool clocks this register with the same clock as the surrounding logic.

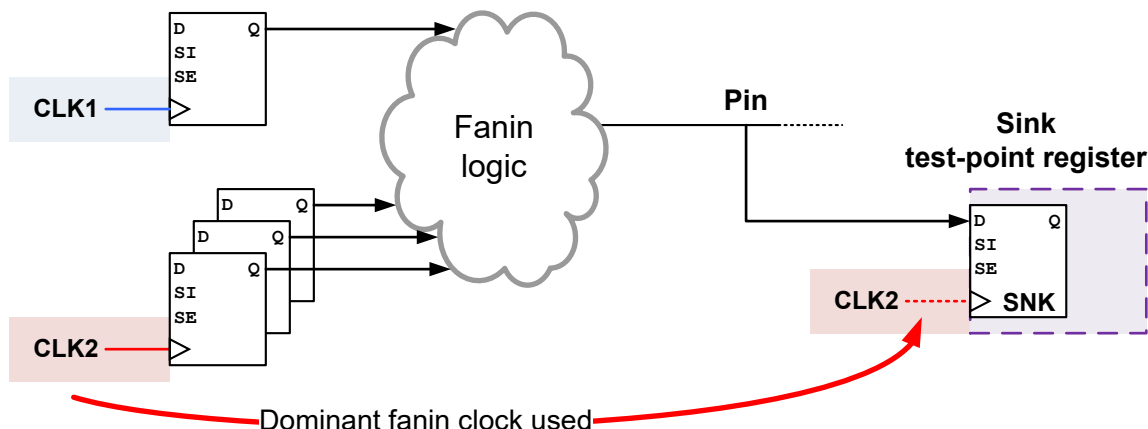
For source registers, the tool uses the dominant clock of the fanout registers, which capture data propagating from the test point:

Figure 146 Using Dominant Fanout Clock



For sink registers, the tool uses the dominant clock of the fanin registers, which drive data that propagates to the test point:

Figure 147 Using Dominant Fanin Clock



Or, you can specify a dedicated test point clock signal to be used for all test point registers:

- You can specify the name of a scan clock signal, defined as a `ScanClock` signal type with the `set_dft_signal` command.
- In a DFT-inserted OCC controller flow, you can specify the name of a PLL output pin. In this case, the tool maps the test point clock to the output pin of the corresponding OCC controller during DFT insertion.
- In a user-defined OCC controller flow, you can directly specify the name of an output pin of an existing OCC controller.

For more information about OCC controller flows, see [Chapter 13, On-Chip Clocking \(OCC\)](#).

For multivoltage designs, the register is associated with the power domain of the block in which the test point is inserted.

Sharing Test Point Registers

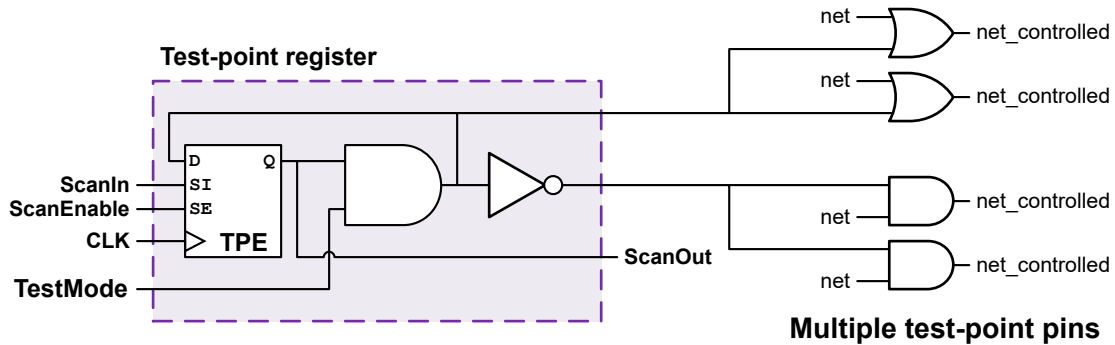
By default, to reduce the area overhead of test point logic from TestMAX Advisor analysis, Fusion Compiler shares each test point register with multiple test points.

Sharing Source and Enable Registers

A source or enable test point register can be shared with multiple force or control test point pins. No additional logic gates are required; the register outputs are tied to multiple test

point logic gates. Figure 148 shows the logic for multiple `control_0` and `control_1` test points that share the same enable register.

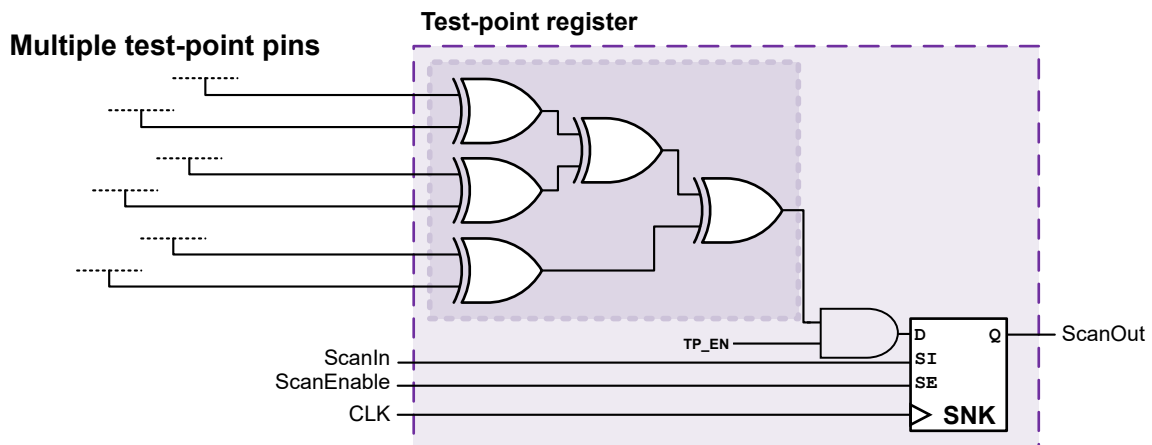
Figure 148 Shared Source Register for Multiple `control_0/1` Test Points



Sharing Sink Registers

A sink test point register can be shared with multiple observe test point pins. The tool builds an XOR reduction tree which collapses multiple observed signals down to a single sink signal connected to the data input of the sink register. See Figure 149.

Figure 149 Shared Sink Register For Multiple observe Test Points



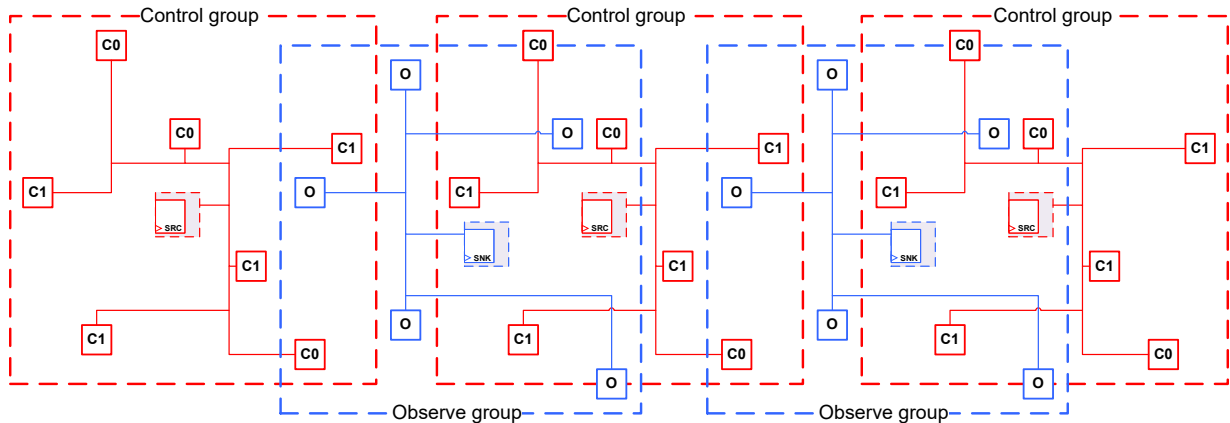
Sharing Rules

The test point pins within each sharing group share the same test point type, clock domain, and power domain. The pins are chosen to be in close physical or logical proximity. The maximum number of pins in a group is set by the `-test_points_per_scan_cell` option of the relevant test point configuration command. The register is inserted in the lowest hierarchy level common to all pins.

A register cannot be shared as both a source (data) and enable (control) register.

In Fusion Compiler, the tool groups pins that are in close physical proximity, then creates the test point register within the group. Test point groups of differing type, clock domain, or power domain are created independently and thus might overlap.

Figure 150 Shared Source Registers (Red) and Sink Registers (Blue)



Automatically Inserted Test Points

You can automatically insert test points in your design to improve its testability. With this feature, Fusion Compiler calls the TestMAX Advisor tool to analyze the design and determine an optimal set of test points, then it implements them during synthesis.

You can use one or more test point *targets*, each focusing on a different aspect of testability:

- `random_resistant`

This target inserts test points that improve random-pattern coverage. This improves the coverage for a given pattern count. It can also improve the maximum coverage obtainable for the design.

- `untestable_logic`

This target inserts test points that make untestable logic testable. This improves the maximum coverage obtainable for a design. It also improves the coverage for a given pattern count.

- `atpg_conflict`

This target inserts test points to reduce conflict between test patterns to achieve test pattern reduction.

- `x_blocking`

This target inserts test points to block X values at their sources so they cannot propagate into downstream logic and be captured.

- `multicycle_paths`

This target inserts test points to prevent multicycle-path-constrained logic from being captured by scan cells (which ATPG treats as an X value).

- `shadow_wrapper`

This target inserts *shadow wrappers* around untestable blocks or macrocells so that surrounding logic can be tested. The outputs are forced to known values and the inputs are observed to ensure coverage.

- `user`

This target inserts test points whose types and locations are provided by the user.

The following topics describe how to configure automatic test point insertion:

- [Enabling Automatic Test Point Insertion](#)
- [Configuring Global Test Point Insertion Settings](#)
- [Configuring the Random-Resistant Test Point Target](#)
- [Configuring the Untestable Logic Test Point Target](#)
- [Configuring the ATPG Conflict Type Test Points](#)
- [Configuring the X-Blocking Test Point Target](#)
- [Configuring the Multicycle Path Test Point Target](#)
- [Configuring the Shadow Wrapper Test Point Target](#)
- [Configuring the User-Defined Test Point Target](#)
- [Enabling Multiple Targets in a Single Command](#)
- [Implementing Test Points From an External File](#)
- [Customizing the Test Point Analysis](#)
- [Running Test Point Analysis](#)
- [Automatic Test Point Insertion Example Script](#)
- [Limitations](#)

Enabling Automatic Test Point Insertion

To enable automatic test point insertion, issue the following command before pre-DFT DRC:

```
fc_shell> set_dft_configuration -testability enable
```

Note:

A TestMAX Advisor or Spyglass® DFT ADV license is required to use the automatic test point insertion feature.

Then, use the `set_testability_configuration` command to configure one or more automatic test point targets:

- To configure global settings, which are shared by all targets, omit the `-target` option.
- To enable a particular target (and to optionally configure any target-specific options it supports), specify that target with the `-target` option.

Configuring Global Test Point Insertion Settings

To configure global aspects of automatic test point insertion, use the `set_testability_configuration` command without the `-target` option:

```
set_testability_configuration
[-clock_signal clock_name]
[-control_signal control_name]
[-test_points_per_scan_cell n]
[-user_options file_name]
[-max_test_points n]
```

Table 13 shows the global configuration options.

Table 13 *Global set_testability_configuration Options*

To do this	Use this option
Use a single dedicated clock for all test point registers in the design	<code>-clock_signal <i>clock_name</i></code> (default is the dominant clock)
Use a particular TestMode signal to enable the test points	<code>-control_signal <i>control_name</i></code> (default is the first available TestMode signal)
Specify the number of force, control, or observe points that can share a test point register	<code>-test_points_per_scan_cell <i>n</i></code> (default is 8)
Customize the test point analysis with user-provided TestMAX Advisor Tcl commands	<code>-user_options <i>file_name</i></code>

Table 13 *Global set_testability_configuration Options (Continued)*

To do this	Use this option
Set the maximum number of <code>random_resistant</code> and <code>untestable_logic</code> test points (combined) that can be inserted	<code>-max_test_points n</code> (default is 5% of the total register count)

You can also use the `-clock_signal`, `-control_signal`, and `-test_points_per_scan_cell` options on a per-target basis to override the global values.

The `-max_test_points` option applies to the `random_resistant` and `untestable_logic` targets. When both targets are enabled, the analysis automatically allocates the global limit across them. Any per-target `-max_test_points` limits apply in addition to, not in place of, the global limit. For details, see [SolvNetPlus article 3021459](#), “How Does The `-max_test_points` Option Work?”

Configuring the Random-Resistant Test Point Target

You can use the `random_resistant` test point target to improve the testability of hard-to-test logic in your design. This improves the coverage for a given pattern count. It can also improve the maximum coverage obtainable and ATPG effectiveness for the design.

The random-resistant target invokes a TestMAX Advisor algorithm that determines an optimal set of test points that improves random-pattern test coverage for a given pattern count. It provides the following benefits:

- High capacity—No random pattern simulation is used; instead, the pattern count is a parameter in a mathematical probabilistic fault-detection analysis.
- Easy to use—The algorithm finds the optimum set of `control_0`, `control_1`, and observe test points; there is no need to guess at per-type limits.

To enable and configure the random-resistant target, use the `set_testability_configuration -target random_resistant` command. The remaining options, which are described in [Table 14](#), can be specified to override their defaults.

Table 14 *Random-Resistant set_testability_configuration Options*

To do this	Use this option
Override the corresponding global value of a parameter	<code>-clock_signal clock_name</code> <code>-control_signal control_name</code> <code>-test_points_per_scan_cell n</code>

Table 14 *Random-Resistant set_testability_configuration Options (Continued)*

To do this	Use this option
Control the runtime-versus-accuracy tradeoff of the analysis	<code>-effort low medium high</code> (default is <code>medium</code>)
Set the number of random-resistant test points at which the analysis completes. Random resistant analysis stops when either the test point or test coverage limit is met.	<code>-max_test_points n</code> (default is no target-specific limit)
Set the random-pattern test coverage value at which the analysis completes. Random resistant analysis stops when either the test point or test coverage limit is met.	<code>-target_test_coverage coverage_value</code> (default is <code>100</code>)
Specifies the random pattern count used for the analysis	<code>-random_pattern_count n</code> (default is <code>64000</code>)
Restrict test point insertion to only particular hierarchical cells or logic cones	<code>-include_elements cell_list</code> <code>-include_fanin_cone pin_port_list</code> <code>-include_fanout_cone pin_port_list</code> (default is to consider the entire design)
Exclude particular hierarchical cells or logic cones from test point insertion	<code>-exclude_elements cell_list</code> <code>-exclude_fanin_cone pin_port_list</code> <code>-exclude_fanout_cone pin_port_list</code> (default is not to exclude anything)

If you are inserting LogicBIST self-test, for best results, set the `-random_pattern_count` option to the number of self-test patterns.

See Also

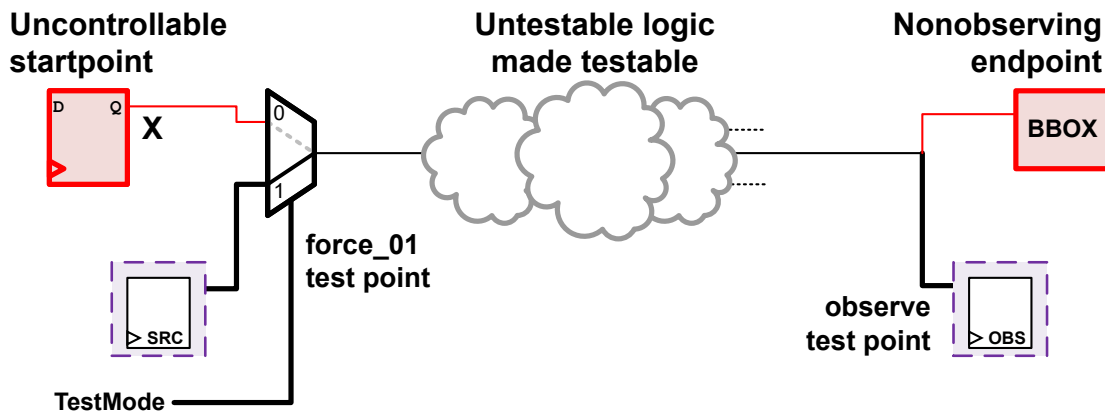
- [Info_random_resistance rule](#) documentation in the TestMAX Advisor documentation for details on the random-resistant analysis algorithm and its parameters

Configuring the Untestable Logic Test Point Target

You can use the `untestable_logic` test point target to make untestable logic testable. This improves the maximum coverage obtainable for a design. It also improves the coverage for a given pattern count.

The untestable logic target invokes a TestMAX Advisor algorithm that determines an optimal set of force_01 and observe test points to control uncontrollable nets and observe unobservable logic, respectively.

Figure 151 Untestable Logic Made Testable



The untestable logic target differs from the X-blocking target as follows:

- It inserts observe test points as well as force_01 test points.
- It performs gain-based analysis, which prefers test points with the highest testability improvement first.
- It considers the `-max_test_points` option.

To enable and configure the untestable logic target, use the `set_testability_configuration -target untestable_logic` command. The remaining options, which are described in [Table 15](#), can be specified to override their defaults.

Table 15 Untestable Logic `set_testability_configuration` Options

To do this	Use this option
Override the corresponding global value of a parameter	<code>-clock_signal clock_name</code> <code>-control_signal control_name</code> <code>-test_points_per_scan_cell n</code>
Set the number of untestable logic test points at which the analysis completes	<code>-max_test_points n</code> (default is no target-specific limit)
Restrict test point insertion to only particular hierarchical cells or logic cones	<code>-include_elements cell_list</code> <code>-include_fanin_cone pin_port_list</code> <code>-include_fanout_cone pin_port_list</code> (default is to consider the entire design)

Table 15 *Untestable Logic set_testability_configuration Options (Continued)*

To do this	Use this option
Exclude particular hierarchical cells or logic cones from test point insertion	<code>-exclude_elements cell_list</code> <code>-exclude_fanin_cone pin_port_list</code> <code>-exclude_fanout_cone pin_port_list</code> (default is not to exclude anything)

This target is highly recommended for designs with LogicBIST self-test.

See Also

- [TA_10 rule](#) documentation in the TestMAX Advisor documentation for details on the untestable logic analysis algorithm

Configuring the ATPG Conflict Type Test Points

You can use the `atpg_conflict` test point target to reduce conflict between test patterns to achieve test pattern reduction. The `atpg_conflict` target invokes a TestMAX Advisor algorithm that determines an optimal set of control test points by identifying nets that might cause conflict between test patterns

To enable the `atpg` conflict target, use the `set_testability_configuration -target atpg_conflict` command. The remaining options, which can be specified to override their defaults are described in the following table:

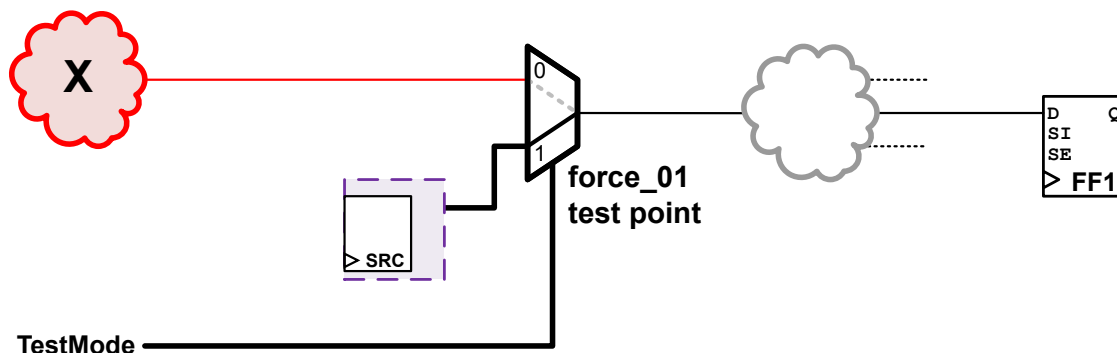
Table 16 *ATPG Conflict type set_testability_configuration Options*

To do this	Use this option
Override the corresponding global value of a parameter	<code>-clock_signal clock_name</code> <code>-control_signal control_name</code> <code>-test_points_per_scan_cell n</code>

Configuring the X-Blocking Test Point Target

You can use the `x_blocking` test point target to identify and block X-value sources from black-box cells. This target inserts force_01 test points at the sources to force scan-controllable values in place of the X values.

Figure 152 X-Value Source Blocked by force_01 Test Point



The X-blocking target differs from the untestable logic target as follows:

- It blocks all X sources in the design regardless of gain improvement, which is important for designs with LogicBIST self-test.
- It does not consider the `-max_test_points` option.

To enable the X-blocking target, use the `set_testability_configuration -target x_blocking` command. The remaining options, which are described in Table 17, can be specified to override their defaults.

Table 17 X-Blocking `set_testability_configuration` Options

To do this	Use this option
Override the corresponding global value of a parameter	<code>-clock_signal clock_name</code> <code>-control_signal control_name</code> <code>-test_points_per_scan_cell n</code>

This target blocks X values from black-box cells. It does not block values from registers with uncontrolled clock or asynchronous set/reset signals.

This target is highly recommended for designs with LogicBIST self-test.

Configuring the Multicycle Path Test Point Target

You can use the `multicycle_paths` test point target to prevent multicycle-path-constrained logic from being captured by scan cells (which ATPG treats as an X value). This target inserts multicycle path test points at multicycle-constrained data input pins of scan cells.

To enable the multicycle paths target, use the `set_testability_configuration -target multicycle_paths` command. The remaining options, which are described in [Table 18](#), can be specified to override their defaults.

Table 18 Multicycle Paths `set_testability_configuration` Options

To do this	Use this option
Override the corresponding global value of the parameter	<code>-control_signal control_name</code>

You must apply all multicycle constraints before test point analysis is run.

See Also

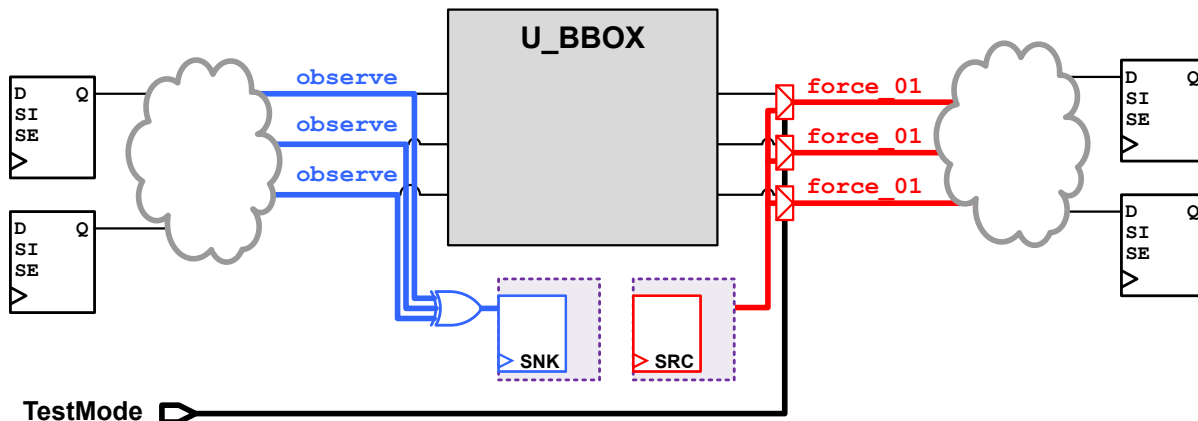
- [Atspeed_05 rule](#) documentation in the TestMAX Advisor documentation for details on the multicycle paths analysis rule
- [Multicycle Test Points](#) for information on the multicycle path test point

Configuring the Shadow Wrapper Test Point Target

You can use the `shadow_wrapper` test point target to allow logic around untestable blocks or macrocells to be tested.

This target inserts `force_01` test points at data output pins to drive known values, and it inserts `observe` test points at data input pins to ensure coverage. If the functional logic around a pin already allows for testability, the analysis detects this and does not insert a test point.

Figure 153 Shadow Wrapper Around a Black-Box Cell



To enable the shadow wrapper target, use the `set_testability_configuration -target shadow_wrapper -isolate_elements cell_list` command.

The `-target shadow_wrapper` option enables the shadow wrapper target and the `-isolate_elements` option specifies the list of cells to isolate; both are required. The remaining options, described in [Table 19](#), can be specified to override their defaults.

Table 19 Shadow Wrapper `set_testability_configuration` Options

To do this	Use this option
Override the corresponding global value of a parameter	<code>-clock_signal clock_name</code> <code>-control_signal control_name</code> <code>-test_points_per_scan_cell n</code>

Only hierarchical, black-box, and CTL-modeled cells can be isolated.

Configuring the User-Defined Test Point Target

The `user` test point target controls the implementation of user-defined test points.

This target applies to the following:

- User-defined test points specified by the `-test_point_file` option
Test points not from TestMAX Advisor in an external file are assigned to the `user` test point target. User-defined test points from an external file are not implemented unless the `user` target is enabled.
- The `set_test_point_element` command
User-defined test points defined with the `set_test_point_element` command are always implemented, whether the `user` target is enabled or not. However, the `user` target allows you to specify implementation defaults for them.

To enable the user-defined target, use the `set_testability_configuration -target user` command.

The `-target user` option is required to use test points from an external file. The remaining options, which are described in [Table 20](#), can be specified to override their defaults.

Table 20 User-Defined `set_testability_configuration` Options

To do this	Use this option
Override the corresponding global value of a parameter	<code>-clock_signal clock_name</code> <code>-control_signal control_name</code> <code>-test_points_per_scan_cell n</code>

Note:

If you specify the tool to insert user-defined test points on some registers, but then change the names of those registers using the `change_names` command, then the tool does not insert the user-defined test points.

See Also

- [Implementing Test Points From an External File](#) for details on TestMAX Advisor and user-defined test point files

Enabling Multiple Targets in a Single Command

You can enable multiple targets in a single `set_testability_command` by specifying a list with the `-target` option:

```
fc_shell> set_testability_configuration \
          -target {random_resistant x_blocking}
Information: Creating testability configuration for target
'random_resistant'.
Information: Creating testability configuration for target 'x_blocking'.
Accepted testability configuration specification for design 'top'.
1
```

Note:

The tool prefers to enable a target per command.

Implementing Test Points From an External File

You can implement test points defined in an external file by using the `-test_point_file` option of the `set_testability_configuration` command:

```
set_testability_configuration
  -test_point_file file_name
```

The `-test_point_file` option is a global option that specifies the name of the test point file to implement.

The test point file must be formatted as follows:

- A test point is described by a line containing three fields, separated by whitespace:
 - A user-defined label (not used by the tool; can be any string)
 - The test point type keyword
 - The net where the test point should be inserted
- For net names, the hierarchy separator character can be "/" (used by the synthesis tools) or "." (used by the TestMAX Advisor tool).
- Text after "#" is a comment and is ignored.
- Blank lines are ignored.

Note that file-based test points are not implemented unless their targets are enabled (just as with analysis-based test points). Test points are assigned to targets as described in the following sections.

Using User-Defined Test Point Files

Test points from user-defined test point files are assigned to the `user` test point target. [Example 16](#) shows an example user-defined test point file.

Example 16 Example User-Defined Test Point File

```
# Put these memories into test mode
* force_1 MEM0_0/TSTMODE
* force_1 MEM0_1/TSTMODE
* force_1 MEM1_0/TSTMODE
* force_1 MEM1_1/TSTMODE

# these test points come from in-house Perl script
TP1 control_1 core0/U37641/NET1 # my_algorithm.pl result: Q=478 V=479
TP2 control_0 core4/U73087/NET2 # my_algorithm.pl result: Q=861 V=22
TP3 control_0 core0/U64599/NET3 # my_algorithm.pl result: Q=227 V=964
TP4 control_1 core4/U99749/NETN # my_algorithm.pl result: Q=841 V=9
```

Using TestMAX Advisor Test Point Files

Test points from a TestMAX Advisor test point file (with in-line comments) are routed to their corresponding targets. [Example 17](#) shows an example test point file generated by the TestMAX Advisor tool that contains `random_resistant` and `x_blocking` test points.

Example 17 Example TestMAX Advisor Test Point File

```
# Moment : "TP analysis start " # Time : 2017- 1-30 9:52:19
# Design : "top"
# Initial Random Pattern Test Coverage : 80.68602
# Stuck At Test Coverage : 99.44180
```

```
# Random Pattern Count : 10
# Effort Level : medium
# Requested Test Points : 1
# Cut-Off Gain : 0.00000, (cumulative gain of last '1' test points)
# Thread Count : 8

# Index Test_Point_Type Net Comment
1 observe sub3.N50 #Gain : 0.04860 Cov : 80.73462 S@TC : 99.4
# Search completed : 1 (dft_rrf_tp_count(1 - 1)) test points identified.
# BENCHMARK : "COMPLETE TP ANALYSIS " # Time : 5.7086
# Moment : "TP analysis End " # Time : 2017- 1-30 9:52:24

# X Source Section
# Index force_01 net
* force_01 sub2.nZ[0] # -xsource -cell_name sub2.BBOX
* force_01 sub2.nZ[1] # -xsource -cell_name sub2.BBOX
* force_01 sub2.nZ[2] # -xsource -cell_name sub2.BBOX
* force_01 sub2.nZ[3] # -xsource -cell_name sub2.BBOX
```

Using TestMAX Advisor Test Point Files Without Rerunning Analysis

By default, analysis is performed for all enabled targets. However, when implementing test points from a TestMAX Advisor file, you can prevent analysis from rerunning for its targets by specifying the `-only_from_file true` option for those targets. Analysis is still performed for any targets enabled without this option.

The following command implements `random_resistant` and `x_blocking` test points from a TestMAX Advisor file, then also implements shadow-wrapper test points that are not described in the file:

```
# global options - specify test point file
set_testability_configuration -test_point_file my_rrf_xblocking_TPs.txt

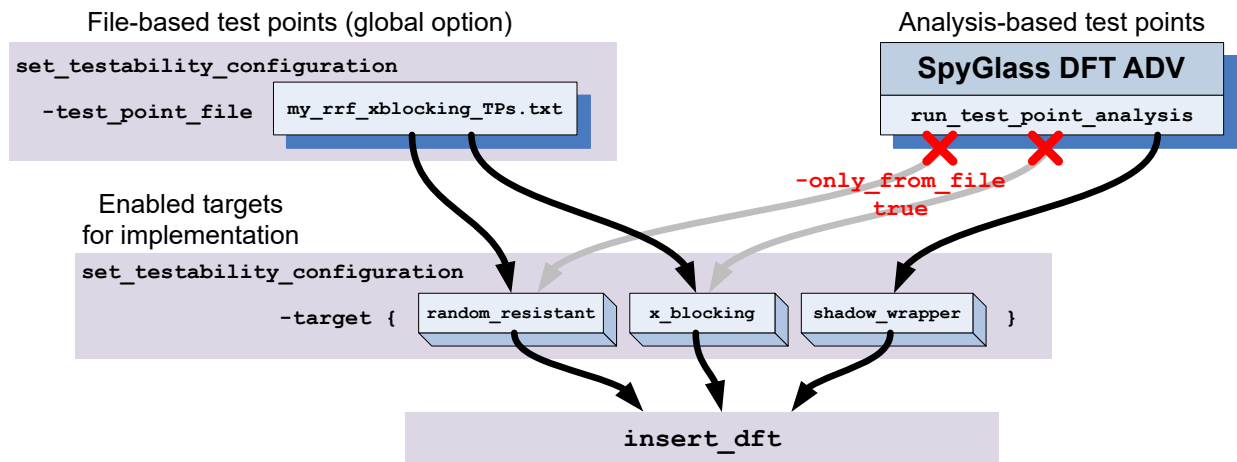
# enable targets for implementation
## implement these from the existing TestMAX Advisor file:

set_testability_configuration -only_from_file true -test_point_file
my_rrf_xblocking_TPs.txt
set_testability_configuration -target {random_resistant x_blocking}

## derive these in the current run (during run_test_point_analysis)
set_testability_configuration -target {shadow_wrapper} \
-isolate_elements {IP_BLOCK1 IP_BLOCK2}
```

Figure 154 shows how the test points are populated in each target for this example.

Figure 154 Combining File-Based and Analysis-Based Test Points



If you implement TestMAX Advisor test points from a file by enabling its target without specifying the `-only_from_file true` option, both the file-based and analysis-based test points are implemented for that target.

Customizing the Test Point Analysis

To customize the test point analysis, you can provide a user file containing one or more TestMAX Advisor Tcl commands to be included:

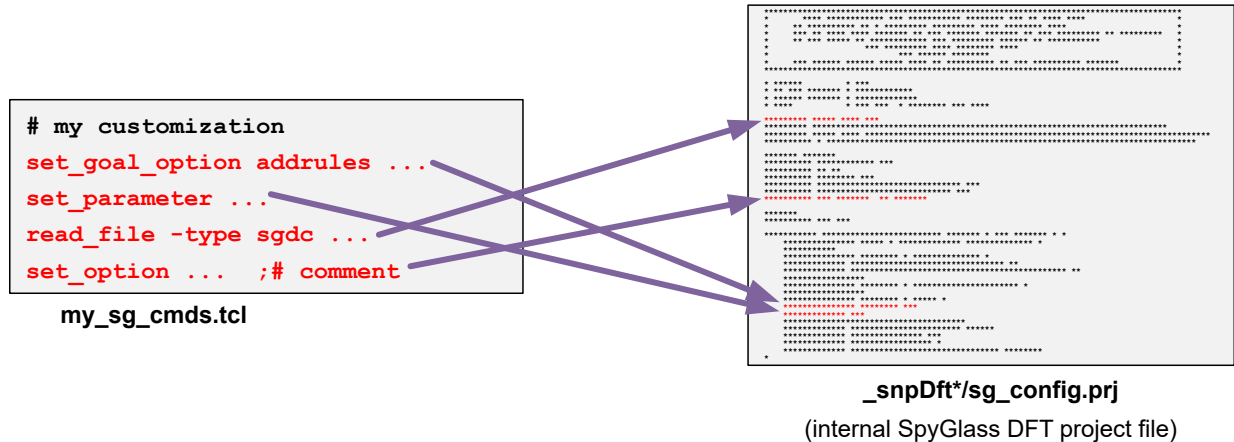
```
fc_shell> set_testability_configuration -sg_command_file my_sg_cmds.tcl
```

This user file is an unordered list of one or more of the following commands:

```
read_file
set_parameter
set_option
set_goal_option
```

The TestMAX Advisor analysis automatically places the commands at the appropriate points in the internal analysis script (also known as the TestMAX Advisor project file):

Figure 155 Including User Commands in the TestMAX Advisor Test Point Analysis



Note:

TestMAX Advisor design constraint (*.sgdc) commands cannot be placed directly in the user file, but they can be placed in a separate file and applied by a `read_file -type sgdc` command in the user file.

Comments after commands are preserved; full-line comments are not.

After the TestMAX Advisor analysis completes, you can examine the TestMAX Advisor project file by looking at the `sg_config.prj` file in the newly created `_snpDft*` directory.

Running Test Point Analysis

If you have configured testability analysis features that require TestMAX Advisor analysis, Fusion Compiler automatically invokes SpyGlass and performs the analysis during the `compile_fusion` command.

The `SPYGLASS_HOME` environmental variable (not Tcl application variable) must be set to find the `spyglass` binary. You can check this as follows:

```
fc_shell> echo $env(SPYGLASS_HOME)
/global/apps/spyglass_2016.06-SP2
```

Automatic Test Point Insertion Example Script

The following script inserts several kinds of testability test points, using a test-mode control signal named `TM_TESTPOINTS`.


```
# define DFT signals
set_dft_signal -view existing_dft -type ScanClock \
  -port CLK -timing [list 45 55]
set_dft_signal -view spec -type TestMode \
  -port TM_TESTPOINTS

# enable automatic test point insertion
set_dft_configuration -testability enable

# configure global automatic test point insertion settings
# (shared by all targets)
set_testability_configuration \
  -control_signal TM_TESTPOINTS \
  -test_points_per_scan_cell 16

# enable and configure test point targets
set_testability_configuration \
  -target random_resistant \
  -random_pattern_count 1024

Remove # preview test point
create_test_protocol
dft_drc -test_mode all_dft
# run TestMAX Advisor to compute test points
run_test_point_analysis
preview_dft

# insert DFT logic
insert_dft
```

Limitations

Automatic test point insertion has the following limitations:

- The `-control_signal` option *must* be specified, otherwise test point insertion is skipped.
- Test point registers are always positive-edge, even if the dominant clocking around a test point is negative-edge.
- When automatic test point insertion is enabled, do not run the `change_names` command between the `run_test_point_analysis` and the `preview_dft` commands, the `run_test_point_analysis` and `insert_dft` commands, or the `preview_dft` and `insert_dft` commands.

User-Defined Test Points

User-defined test points provide you with the flexibility to insert control and observe test points at user-specified locations in the design. User-defined test points can be used for

a variety of purposes, including the ability to fix uncontrollable clocks and asynchronous signals, increase the coverage of the design, and reduce the pattern count.

The following topics describe how to implement user-defined test points:

- [Enabling User-Defined Test Point Insertion](#)
- [Configuring User-Defined Test Points](#)
- [Limitations](#)

Enabling User-Defined Test Point Insertion

To enable user-defined test point insertion, you must first issue the following command before pre-DFT DRC:

```
fc_shell> set_dft_configuration -testability enable
```

After you have enabled user-defined test point insertion, you can enable and configure one or more user-defined test point specifications with the `set_test_point_element` command, as described in the following topics.

Configuring User-Defined Test Points

You can use the `set_test_point_element` command to specify the location and type of user-defined test points to insert in the design during DFT insertion, as well as other aspects of test point construction. User-defined test points can be defined at leaf pins, hierarchy pins, and ports. These test points are then inserted by the `insert_dft` command.

To define a user-defined test point, use the following command:

```
set_test_point_element  
  -type force_0 | force_1 | force_01  
    | control_0 | control_1 | observe  
  [-control_signal control_name]  
  [-clock_signal clock_name]  
  [-test_points_per_source_or_sink n]  
  pin_port_list
```

Specify the test point type and the list of signal pins or ports to be forced, controlled, or observed. For more information on test point types, see [Test Point Types on page 265](#).

The remaining options, described in [Table 21](#), can be specified to override their defaults.

Table 21 *set_test_point_element Options*

To do this	Use this option
Use a particular test clock for any test point registers needed by these test points	<code>-clock_signal clock_name</code> (default is the dominant clock)
Use a particular TestMode, ScanEnable, or IbiStEnable signal to enable these test points	<code>-control_signal control_name</code> (default is the first available TestMode signal)
Specify the number of force, control, or observe points (in this specification only) that can share a test point register	<code>-test_points_per_source_or_sink n</code> (default is 8)

The test points are implemented using clock, control signal, and test points per scan cell settings as follows, highest precedence first:

- From the `set_test_point_element` specification
- From the `set_testability_configuration` global specification
- Using the global testability defaults

Note:

The `user` testability target does not need to be enabled to implement test points using the `set_test_point_element` command, and the `user` target settings do not affect test points implemented by the `set_test_point_element` command.

By default, any needed source or sink registers are created by the `insert_dft` command.

Registers are not shared across multiple `set_test_point_element` specifications. If test points within a limited physical region should share registers, they should all be provided in a single `set_test_point_element` command.

If the specified pin list spans multiple clock or power domain configurations, the tool creates separate test point registers for each configuration.

After specifying test point definitions with the `set_test_point_element` command, you can report them with the `report_test_point_element` command, or remove them before DFT insertion with the `remove_test_point_element` command. For more information about these commands, see the man pages.

See Also

- [Sharing Test Point Registers on page 272](#) for details on test point registers sharing
- [Configuring the User-Defined Test Point Target on page 283](#) for details on the user testability target
- [Configuring Global Test Point Insertion Settings on page 276](#) for details on the global testability configuration

Limitations

User-defined test point insertion has the following limitations:

- The `-control_signal` option *must* be specified, otherwise the control signal for the test point logic is tied to logic 0.
- All specifications must use the same explicitly specified control signal; you cannot use multiple control signals across `set_test_point_element` specifications.
- Test point registers are always positive-edge, even if the dominant clocking around a test point is negative-edge.
- User-defined test points do not inherit the `-test_points_per_scan_cell` value from the global or user target `set_testability_configuration` value.

Previewing the Test Point Logic

To preview the test point logic that the tool implements according to your specifications, use the following command after running test point analysis.

```
fc_shell> preview_dft
```

You can also review the separate report of `testpoint.rpt` generated by `preview_dft` command.

Note:

If you are using TestMAX Advisor testability features, you must first run analysis as described in [Running Test Point Analysis on page 288](#).

The test point preview report is organized as follows:

- The legend at the beginning defines attributes used in the report.
 - Test point registers and test point pins use separate attributes.
- In the table, each line with values in the “Test Point Type” and “Pin” columns indicates a test point of that type at that pin.

- For each test point, the “Test Point Register” column indicates the source or sink register used for that test point.
 - An empty value means this test point shares the same register as the previous test point.
 - A value of “-” means no test point register is needed for that test point type.
- The summary at the end reports indicates how many test points and test point registers will be implemented.

Example 18 shows an example test point preview report.

Example 18 Example Test Point Preview Report

***** Test Point Plan Report *****

Test point register attributes:

d - dedicated (DFT-inserted) test point register
f - reused (functional) test point register
tpe - test point enable signal
src - test point source signal
snk - test point sink signal

Test point pin attributes:

r - random-resistant test point pin
x - X-blocking test point pin
m - multicycle path test point pin
w - core wrapper test point pin
s - shadow wrapper test point pin
g - self-gating test point pin
l - untestable logic test point pin
a - atpg conflict test point pin
u - user-defined test point pin

Index	Test Point Register	Test Point Type	Pins
----	-----	-----	----
1	U_dft_tp_sdtc_ip_0/dtc_reg (d, src) (CLK1)		
	force_01		IP_BLOCK/RX0 (u)
	force_01		IP_BLOCK/RX1 (u)
2	U_dft_tp_sdtc_ip_1/dtc_reg (d, snk) (CLK1)		
	observe		IP_BLOCK/TX0 (u)
	observe		IP_BLOCK/TX1 (u)
3	sub3/mult_60/U_dft_tp_sdtc_ip_2/dtc_reg (d, tpe) (CLK3)		
	control_0		sub3/mult_60/U1057/Z (r)
	control_1		sub3/mult_60/U1156/Z (r)
	control_1		sub3/mult_60/U1262/Z (r)
	control_0		sub3/mult_60/U1343/Z (r)
4	sub3/mult_60/U_dft_tp_sdtc_ip_3/dtc_reg (d, snk) (CLK3)		
	observe		sub3/mult_60/U4479/Z (r)
	observe		sub3/mult_60/U4483/Z (r)

```

                                observe          sub3/mult_60/U4487/Z (r)
                                observe          sub3/mult_60/U4596/Z (r)
5      sub2/U_dft_tp_sdte_ip_4/dtc_reg ( d, src ) ( CLK2 )
                                force_01        sub2/BBOX/Z[0] (x)
                                force_01        sub2/BBOX/Z[1] (x)
6      sub2/U_dft_tp_sdte_ip_5/dtc_reg ( d, snk ) ( CLK2 )
                                observe          sub2/BBOX/A[0] (x)
                                observe          sub2/BBOX/A[1] (x)
7      -                          multicycle    MULT_reg[3]/D (m)
8      -                          multicycle    MULT_reg[2]/D (m)
9      -                          multicycle    MULT_reg[1]/D (m)
10     -                          multicycle    MULT_reg[0]/D (m)

***** Test Point Summary *****
Number of testability force_01 test points:      2
Number of testability control_0 test points:     2
Number of testability control_1 test points:     2
Number of testability observe test points:       6
Number of testability multicycle path test points: 4
Number of user-defined force_01 test points      2
Number of user-defined observe test points:      2

Total number of test points:                     20
Total number of DFT-inserted test point registers: 6
*****
```

Inserting the Test Point Logic

When the test point feature is enabled, the `insert_dft` command does the following:

- Inserts test point logic in design
- Creates a test point report file

Test point scan registers are placed in the lowest level of hierarchy common to all test points for that register.

The test point report file, `testpoint.rpt`, shows the implemented test points, test point registers, and register sharing characteristics:

```

***** Test Point Plan Report *****
Test point register attributes:
  d - dedicated (DFT-inserted) test point register
  f - reused (functional) test point register
  tpe - test point enable signal
  src - test point source signal
  snk - test point sink signal

Test point pin attributes:
  r - random_resistant test point pin
  x - X-blocking test point pin
```

```
m - multicycle path test point pin
w - core wrapper test point pin
s - shadow wrapper test point pin
g - self-gating test point pin
a - AutoFix test point pin
u - user-defined test point pin
```

Index	Test Point Register	Test Point Type	Pins
1	sub3/mult_60/U_dft_tp_sdtc_ip_2/dtc_reg	(d, tpe) (CLK3)	
	control_0		sub3/mult_60/U1057/Z (r)
	control_1		sub3/mult_60/U1156/Z (r)
	control_1		sub3/mult_60/U1262/Z (r)
	control_0		sub3/mult_60/U1343/Z (r)
2	sub3/mult_60/U_dft_tp_sdtc_ip_3/dtc_reg	(d, snk) (CLK3)	
	observe		sub3/mult_60/U4479/Z (r)
	observe		sub3/mult_60/U4483/Z (r)
	observe		sub3/mult_60/U4487/Z (r)
	observe		sub3/mult_60/U4596/Z (r)

```
***** Test Point Summary *****
Number of testability control_0 test points:      2
Number of testability control_1 test points:      2
Number of testability observe test points:        4
Total number of test points:                      8
Total number of DFT-inserted test point registers: 2
*****
```

Inserting Power-Aware Test Points

Test points are points in the design where the Fusion Compiler tool inserts logic to improve the testability of the design.

This feature enables UPF checks for the test point location and test point controller logic.

To enable power-aware test point insertion, use the `set_app_options -name dft.tp_enable_mv_checks` command. UPF checks are done on the nets between the existing driver and its loads. The test points are rejected if the UPF check fails on the nets.

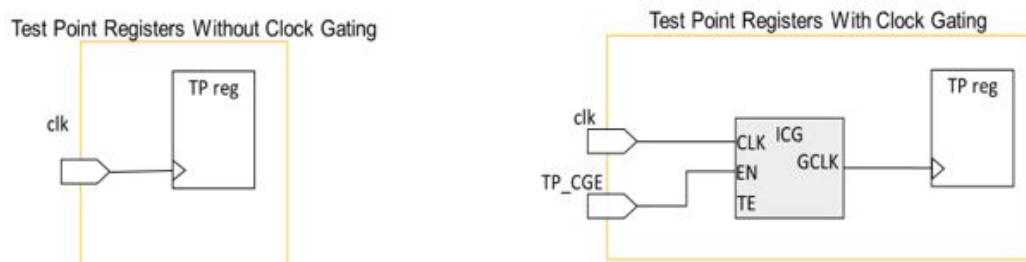
Clock Gating Support for Test Point Inserted Logic

The inserted test points require a register that acts as a test point data controller. This register in the data controller is connected to the top-level clock. As a result, all the test point registers get clocks for every cycle. This leads to unnecessary toggling in functional mode and increased power consumption.

This feature enables the tool to insert new clock gating cells to gate the scan clock and drive the test point registers. Clock gating cells are shared in the same hierarchy as test point registers with a minimum bit-width of 1, ensuring all test point registers are gated.

The top-level control signal is connected to the clock gating functional enable signal, as shown in the following figure.

Figure 156 Clock Gating Support for Test Points



To enable clock gating support for test points, use the `dft.tp_enable_clock_gating -value true` application option as part of the test point DFT specifications.

You can control these clock gating cells by specifying a TestMode DFT signal `set_testability_configuration -cg_enable TP_CGE`. If the TestMode signal is not specified, the tool uses a TestMode signal specified with `-control_signal`.

Limitations

Clock gating support for test points has the following limitations:

- Clock gating of test point registers is not supported for the following targets:
 - Multicycle_path
 - User-defined
- The tool does not support separate control signals for different test point types. The Top-level control signal is based on test point targets instead of test point types.

Enhancing Test Point Physical Clustering

When test points are inserted, the tool uses coarse placement information for physical clustering of test points, shared by a test point register. This implementation of physical clustering for test points has limitations in handling the outlier test points as follows:

- Having a long wire net between the outlier test points and its test point register.
- Introducing a chain of inverter or buffer to meet the timing
- These test points cannot be used for at-speed testing

These limitations can lead to timing closure issues, affecting the QoR.

This enhancement allows you to enhance test point clustering to reduce outliers with consideration of physical location of test points reducing the distance between clusters and their respective test points.

To enable this, you can set the `dft.tp_improve_clustering_to_reduce_outliers` application option to `true`.

This enhancement might result in some test point registers getting shared with less test points than specified through `set_testability_configuration -test_points_per_scan_cell n` and introduce more test point registers to reduce the outliers.

Specifying Fault Space Instances for Test Point Analysis

The ability to specify a list of design instances as the fault space for test point analysis enhances the flexibility and precision of the test point insertion flow. You can focus test point analysis on specific areas of the design, rather than the entire design, which improves analysis efficiency and target fault coverage effectively.

This feature enables targeted test point analysis on user-specified design instances and reduces the analysis time by limiting the fault space to relevant elements.

You can use the following command to enable testability features before running `create_test_protocol`:

```
fc_shell> set_dft_configuration -testability enable
```

Use the `-add_fault_list` option with the `set_testability_configuration` command to define the list of elements (instances) to be included in the fault space. This option is accepted only at the global scope.

```
fc_shell> set_testability_configuration -add_fault_list {inst1 inst2  
inst3}
```

Chapter 14: Inserting Test Points

Specifying Fault Space Instances for Test Point Analysis

If you do not specify the `-add_fault_list` option, the tool assumes the complete design as the fault space by default.

After configuring the fault space, continue with the standard test point analysis flow after `create_test_protocol`, as follows:

```
fc_shell> run_test_point_analysis
```

15

Previewing, Inserting, and Checking DFT Logic

This chapter describes how to preview and insert your DFT IP, insert optimized scan structures during synthesis, and check the resulting DFT logic for correct operation.

This chapter includes the following topics:

- [Previewing the DFT IP](#)
- [Inserting the DFT IP](#)
- [Estimating Scan Chain Count](#)
- [Pre-DFT DRC](#)
- [Inserting Scan Structures](#)
- [Post-DFT DRC](#)
- [Customizing the DFT DRC Analysis](#)
- [Using AutoFix](#)

Previewing the DFT IP

If you use any of the following elaborated-RTL DFT features, you can use the `preview_dft` command to preview the DFT IP before inserting it with the `insert_dft` command:

- On-chip clocking (OCC) controllers
- Scan compression

The `preview_dft` command also shows the configuration for the following DFT features (although these alone do not enable the `preview_dft` command):

- Core wrapping
- Pipelined scan data
- DFT partitions
- Test modes

For example,

```
fc_shell> preview_dft
- Partition: 'default_partition'
Information: Architecting DFT Compressor IP with '2' inputs / '2' outputs
/ '4' internal chains in mode 'ScanCompression_mode'
Information: Compressor will have 100% x-tolerance
Warning: Architecting unload compressor with no direct diagnosis support
ScanDataIn:
    "SI" -> U_DFT_TOP_IP_0/si[0]
    "SI2" -> U_DFT_TOP_IP_0/si[1]
ScanDataOut:
    "SO" -> U_DFT_TOP_IP_0/so[0]
    "SO2" -> U_DFT_TOP_IP_0/so[1]
ScanEnable:
    "SE" -> U_DFT_TOP_IP_0/test_se
    "SE" -> myOCC/test_se
OCC Bypass:
    "OCC_bypass" -> myOCC/pll_bypass
OCC Reset:
    "OCC_reset" -> myOCC/pll_reset
OCC ATE Clocks:
    "ATECLK" -> myOCC/slow_clk_0
OCC TestMode:
    "TM_OCC" -> myOCC/test_mode_0
OCC PLL Clocks:
    "CLK" -> myOCC/fast_clk_0[0] -> myOCC/clk_0[0]
OCC PLL Chains:
    "SI_OCC" -> myOCC/cc_si_0[0]
    myOCC/cc_so_0[0] -> "SO_OCC"
Clock Controller 0 : myOCC
    Number of bits per clock: 4
    Clock chain count: 1
    Fast clock pins: "CLK"
    Slow clock: "ATECLK"
    Test Mode: "TM_OCC"
Scan path: OCC_CHAIN (o)
    "SI_OCC" -> "SO_OCC" { myOCC:0 }
1
```

If no DFT features are enabled that require elaborated-RTL insertion, the command issues the following message:

```
fc_shell> preview_dft
Warning: There is no DFT IP specification to be reported (DFT-2003)
1
```

Inserting the DFT IP

You can use the `insert_dft` command to insert and connect the DFT structures and stitch the scan chains.

Estimating Scan Chain Count

To determine the number of scan chains (scan_in/scan_out pairs) required for a particular combination of the `-max_length` option and scan/wrapper configuration setup, use the `preview_dft -estimate_chains_with_max_length` command option.

This enables an estimation-only flow and prints the number of scan chains required per test mode in each DFT partition based on the specified `-max_length` option and other scan/wrapper configuration constraints.

Note:

- For each DFT partition, at least one `-max_length` specification per test mode must be defined.
- For accurate calculation of required `-chain_count` in each test mode, specify the scan and wrapper specifications that affects the scan chain architecting.

An example of a scan-chain estimation flow is as follows:

```
fc_shell> define_test_mode intest_mode -usage wrp_if -encoding [list TM0
0 TM1 1 TM2 0]
fc_shell> define_test_mode extest_mode -usage wrp_of -encoding [list TM0
0 TM1 1 TM2 1]
fc_shell> define_test_mode scomp -usage streaming_compression -encoding
[list TM0 0 TM1 0 TM2 1]
fc_shell> reset_test_mode
fc_shell> set_scan_configuration -max_length 30 -clock_mixing mix_clocks
-test_mode intest_mode
fc_shell> set_scan_compression_configuration -inputs 10 -outputs 10 \
-max_length 10 -test_mode scomp -streaming true -clock clk_st
fc_shell> set_wrapper_configuration -reuse_threshold 0
-use_system_clock_for_dedicated_wrp_cells enable

fc_shell> set_wrapper_configuration -hold_mux enable -hier_wrapping
disable
fc_shell> set_wrapper_configuration -max_length 20 -test_mode extest_mode
-mix_cells true
fc_shell> set_wrapper_configuration -max_length 20 -test_mode intest_mode
-mix_cells true
fc_shell> create_test_protocol
fc_shell> dft_drc -test_mode all_dft
fc_shell> preview_dft -estimate_chains_with_max_length
<preview_dft information messages>
...
Estimations for partition default_partition, mode intest_tmode:
Total chain count          = 35
wrapper chain count        = 10
Estimations for partition default_partition, mode extest_tmode:
Total chain count          = 10
```

```
wrapper chain count          = 10
Estimations for partition default_partition, mode scomp:
Total chain count            = 99
input wrapper chain count    = 18
output wrapper chain count    = 3
```

Pre-DFT DRC

Pre-DFT DRC (design rule checking) is the process of checking the suitability of the design for test operation before the scan structures are inserted.

If the `dft_drc` command is run before the `insert_dft` command, the tool performs *Pre-DFT DRC*. If `dft_drc` command is not run explicitly before `preview_dft` or `insert_dft` commands, then the tool runs *Pre-DFT DRC* as part of `preview_dft` or `insert_dft` processing.

For example,

```
fc_shell> dft_drc
...
-----
Begin Pre-DFT violations...
Warning: Clock input ZC_reg[3]/CLK of DFF ZC_reg[3] was not controlled. (D1-1)
Information: There is 3 other cell(s) with the same violation. (DFT-3019)
Warning: Clock input ZD_reg[7]/CLK of DFF ZD_reg[7] cannot capture data. (D17-1)
Information: There is 3 other cell(s) with the same violation. (DFT-3019)
Warning: ZC_reg[7]/RSTB input of DFF ZC_reg[7] was not controlled. (D3-1)
Information: There is 3 other cell(s) with the same violation. (DFT-3019)
Warning: Clock input ZD_reg[7]/CLK of DFF ZD_reg[7] not active when clocks are
set on. (D9-1)
Information: There is 3 other cell(s) with the same violation. (DFT-3019)
Pre-DFT violations completed...
-----
Violation Report Summary
Total Violations: 17
-----
1 VECTOR VIOLATIONS
  1 missing state (V14)
16 PRE-DFT VIOLATIONS
  4 DFF clock line not controlled (D1)
  4 Clock_port not capable of capturing data (D17)
  4 DFF reset line not controlled (D3)
  4 Clock_port not active when clocks set to on (D9)

Warning: Violation(s) occurred during test design rule checking. (DFT-2031)
-----

-----
Sequential Cell Report:      Sequential      Core      Core
                             Cells        Segments   Segment Cells
-----
Sequential Elements Detected: 34              0              0
Clock Gating Cells          : 1
Synchronizing Cells         : 0
Non-Scan Elements           : 0
Excluded Scan Elements      : 0              0              0
```

```
Violated Scan Elements      : 12          0          0
(Traced) Scan Elements     : 22 ( 58.8%) 0 ( 0.0%) 0 ( 0.0%)
```

```
-----
Sequential Cell Report
Total Sequential Cells: 34
-----
```

```
12 SEQUENTIAL CELLS WITH VIOLATIONS
    12 cells have test design rule violations
22 SEQUENTIAL CELLS WITHOUT VIOLATIONS
    20 cells are valid scan cells
    1 cell is clock gating cell
    1 cell is transparent latch
-----
```

```
Information: Test design rule checking completed. (DFT-3018)
Information: Test DRC reports 12 violating cells. (DFT-1086)
...
```

Inserting Scan Structures

In the Fusion Compiler tool, the scan structures are inserted using the `insert_dft` command.

The scan structures are inserted into the placed and mapped design to minimize the disturbance. For example,

- Scan chains are routed to be physically compact.
- Existing functional multivoltage cells are used where possible.
- Buffering for test-mode and scan-enable signals is able to utilize free space around functional logic cells.

Scan Structure Insertion in UPF Designs

When the Fusion Compiler tool inserts scan structures in a design that contains UPF information, it considers the multivoltage characteristics of the design when creating new nets and adding connections to existing nets.

In particular, it considers the following:

- The supply of the *source* (driver) port or leaf pin
- The supplies of the *sink* (load) ports or leaf pins
- The domain crossings between the source and sink(s)

When all sinks of a net have the same supply, the net has a *homogeneous* fanout. When the sinks have multiple supplies, the net has a *heterogeneous* fanout.

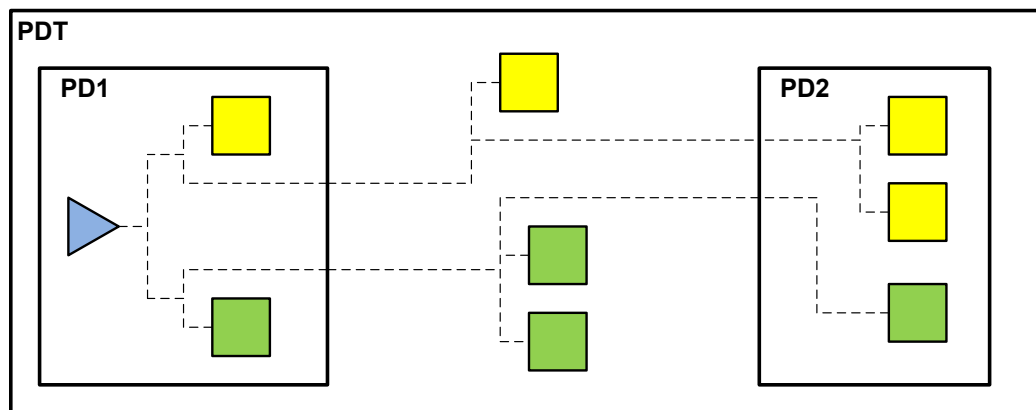
Note:

The tool considers sinks to have the same supply if their supplies are either connected or defined as equivalent with the `set_equivalent` command, even if their power domains are different.

Creating New Nets

When the tool creates a new net,

- Connections to sinks with the *same* supply are branched as close to the sinks as possible, which minimizes port-punching.
- Connections to sinks with *different* supplies are branched as close to the source as possible, which minimizes the potential for unfixable multivoltage violations to the sinks.



Connections to Existing Nets

When the tool adds connections to an existing net, it uses the same rules as described in [Creating New Nets](#). In addition, it applies the following rules:

- The analysis can cross existing isolation and level-shifter cells to find a connection point.
 - However, connections cannot be added on the net segment between an isolation or level-shifter cell and the power domain boundary its strategy was applied to.
- Connections to existing net segments with heterogeneous fanout are not allowed.

Dedicated Wrapper Cell Insertion

When the tool inserts dedicated wrapper cells in a design that contains UPF information, it prefers the highest possible hierarchical nets (closest to the ports) that do not result in multivoltage violations.

For details on the algorithm, see [Dedicated Wrapper Cell Insertion in UPF Designs](#).

Post-DFT DRC

Post-DFT DRC (design rule checking) is the process of checking the scan structures in a design for proper operation.

In the Fusion Compiler tool, post-DFT DRC must be explicitly run after compile by running the `dft_drc` command. For example,

```
fc_shell> dft_drc
Information: Running Post-DFT DRC for Internal_scan mode
-----
Begin scan design rule checking...
-----
Begin reading test protocol file /prj/dft/tmax/top_31512.3293_0.spf...
End parsing STIL file /prj/dft/tmax/top_31512.3293_0.spf with 0 errors.
Test protocol file reading completed, CPU time=0.00 sec.
-----
Begin simulating test protocol procedures...
Nonscan cell constant value results: #constant0 = 1, #constant1 = 0
Nonscan cell load value results      : #load0 = 1, #load1 = 0
Test protocol simulation completed, CPU time=0.00 sec.
-----
Begin scan chain operation checking...
Chain 1 successfully traced with 20 scan_cells.
Warning: Rule S19 (nonscan cell disturb) was violated 8 times.
Scan chain operation checking completed, CPU time=0.00 sec.
-----
Begin clock-gating analysis...
1 ATPG controllable clock-gating cells were found
Setting CTRL3(id=4) to 1 allows 20.00% of all scan cells to toggle.
Setting SE(id=39) to 1 allows 20.00% of all scan cells to toggle.
Clock-gating analysis completed, CPU time=0.00 sec.
-----
Begin clock rules checking...
Warning: Rule C2 (unstable nonscan DFF when clocks off) was violated 8 times.
Warning: Rule C16 (nonscan cell port unable to capture) was violated 4 times.
Warning: Rule C24 (nonclock PI connected to unstable cell clock input) was
violated 2 times.
Clock rules checking completed, CPU time=0.00 sec.
Clock grouping results: #pairs=0, #groups=0, #serial_pairs=0,
#disturbed_pairs=1, CPU time=0.00 sec.
-----
Begin nonscan rules checking...
Nonscan cell summary: #DFF=12 #DLAT=2 #RAM_outs=0 tla_usage_type=hot_clock_tla
Nonscan behavior: #CU=8 #C0=1 #TLA=1 #LE=4
Nonscan rules checking completed, CPU time=0.00 sec.
-----
Begin DRC dependent learning...
Fast-sequential depth results: control=1(226), observe=1(74), detect=1(226),
CPU time=0.00 sec
DRC dependent learning completed, CPU time=0.00 sec.
-----
DRC Summary Report
-----
Warning: Rule S19 (nonscan cell disturb) was violated 8 times.
Warning: Rule C2 (unstable nonscan DFF when clocks off) was violated 8 times.
Warning: Rule C16 (nonscan cell port unable to capture) was violated 4 times.
Warning: Rule C24 (nonclock PI connected to unstable cell clock input) was
```

```
violated 2 times.  
There were 22 violations that occurred during DRC process.  
Design rule checking was successful, total CPU time=0.00 sec.  
-----
```

1

If you have multiple test modes, use the `-test_mode` option to specify which mode to analyze:

```
fc_shell> dft_drc -test_mode Internal_scan  
...  
fc_shell> dft_drc -test_mode ScanCompression_mode  
...
```

You can use the `list_test_modes` command to list the test modes in the current design.

Customizing the DFT DRC Analysis

This topic describes how to customize the DFT DRC (design rule checking) analysis. It applies to both pre-DFT DRC (before scan insertion) and post-DFT DRC (after scan insertion).

Saving the DFT DRC Temporary Files

DFT DRC uses the TestMAX ATPG DRC engine to perform its checking. By default, the log files and other temporary files from this analysis are deleted. To preserve these files, use the `-log_dir` option of the `set_dft_drc_configuration` command.

For example,

```
file delete -force pre  
file mkdir pre  
set_dft_drc_configuration -log_dir pre  
compile_fusion -to initial_opto  
  
file delete -force post  
file mkdir post  
set_dft_drc_configuration -log_dir post  
dft_drc
```

The directory contains the DRC log file, along with the Tcl script, netlist, and test protocol files used to generate it.

Providing DRC Simulation Models

In some cases, the synthesis libraries used by the tool might model a cell as a black box, while the simulation libraries used by the TestMAX ATPG analysis might provide functional information for the same cell.

To improve the accuracy of the analysis, you can provide the TestMAX ATPG simulation models for these black-box cells to DFT DRC:

```
fc_shell> set_dft_drc_configuration -libs {mem32x4.v mem16x8.v}
```

Note:

You should use this option only to replace leaf cells that do not have functional models. Do not use it to replace any arbitrary module in the design.

Using AutoFix

The AutoFix feature automatically fixes scan rule violations resulting from the following types of uncontrollable signals:

- Clock signals
- Asynchronous set signals
- Asynchronous reset signals

You can enable fixing for one or more additional signal types to fix testability problems in your design. You can specify AutoFix configurations globally or on particular design objects. AutoFix is supported in both the multiplexed flip-flop and LSSD scan styles.

When enabled, AutoFix automatically fixes all violations of the specified types found by the `dft_drc` command. If there are no violations, AutoFix makes no changes to the design.

This topic covers the following:

- [When to Use AutoFix](#)
- [The AutoFix Flow](#)
- [Configuring AutoFix](#)

When to Use AutoFix

Use AutoFix to resolve testability problems caused by uncontrollable signals, as described in the following topics:

- [Uncontrollable Clock Signals](#)
- [Uncontrollable Asynchronous Set and Reset Signals](#)

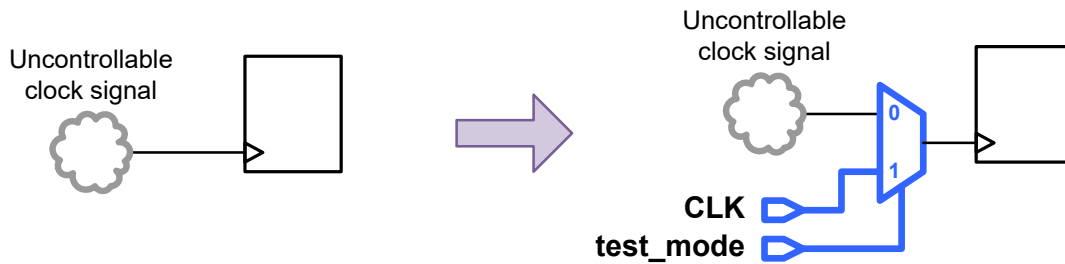
Uncontrollable Clock Signals

Each scan flip-flop in a design must be clocked by a signal controlled by a primary input port. Otherwise, the clocking of data into the flip-flop cannot be controlled during

test. Uncontrollable clock signals are flagged by the `dft_drc` command as design rule violations. If you do not fix these violations, the associated flip-flops are not included in scan chains and faults downstream from the flip-flop outputs may not be detectable.

When AutoFix is enabled for uncontrollable clock signals, it inserts a multiplexer test point to select a controllable clock signal during test, as shown in [Figure 157](#). A test-mode signal controls the multiplexer. For mission-mode operation, the test-mode signal is inactive and the circuit operation is unchanged. During the test, the signal is asserted and the controllable primary input signal clocks the flip-flop.

Figure 157 AutoFix Controllability Logic for an Uncontrollable Clock Signal

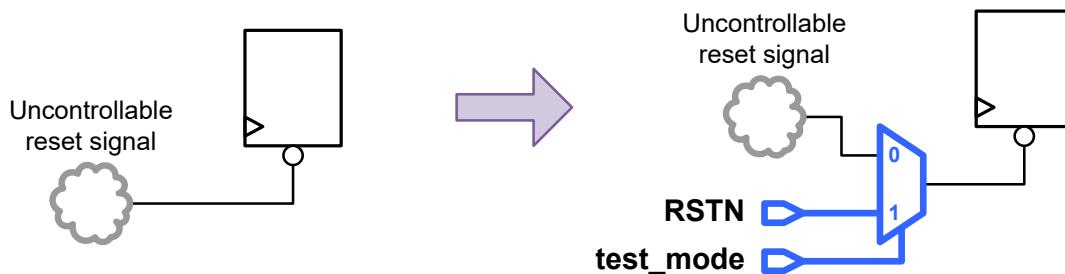


Uncontrollable Asynchronous Set and Reset Signals

The asynchronous set and reset inputs of each flip-flop must be inactive during test. Otherwise, the data in the flip-flop can be set or cleared at any time, leaving unknown data in the flip-flop.

When AutoFix is enabled for an uncontrollable asynchronous set or reset signals, by default, it inserts a multiplexer test point to select a controllable set or reset signal during the test, as shown in [Figure 158](#). A test-mode signal controls the multiplexer. For mission-mode operation, the test-mode signal is inactive and the circuit operation is unchanged. During test, the asynchronous signal is driven by the controllable primary input signal.

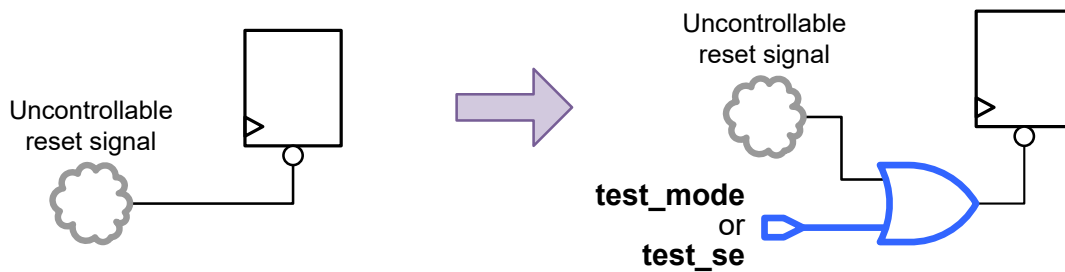
Figure 158 AutoFix MUX-Based Controllability Logic for an Uncontrollable Reset Signal



AutoFix can also insert gating logic to de-assert the uncontrollable asynchronous set or reset signal, as shown in [Figure 159](#). The gating logic is controlled by a test-mode or scan-enable control signal. For mission-mode operation, the control signal is inactive and the

circuit operation is unchanged. When controlled by a test-mode signal, the asynchronous signal is held inactive throughout the entire test program. When controlled by a scan-enable signal, the asynchronous signal is held inactive during scan shift but remains controlled by the functional logic during scan capture.

Figure 159 AutoFix Gating-Based Controllability Logic for an Uncontrollable Reset Signal



Typically, one of the following configurations is used:

- The `mux` method with a test-mode signal

This method directly controls of asynchronous resets during scan shift and scan capture. However, it blocks the functional reset path. This can cause faults in the reset logic to become untestable.

- The `gate` method with a scan-enable signal

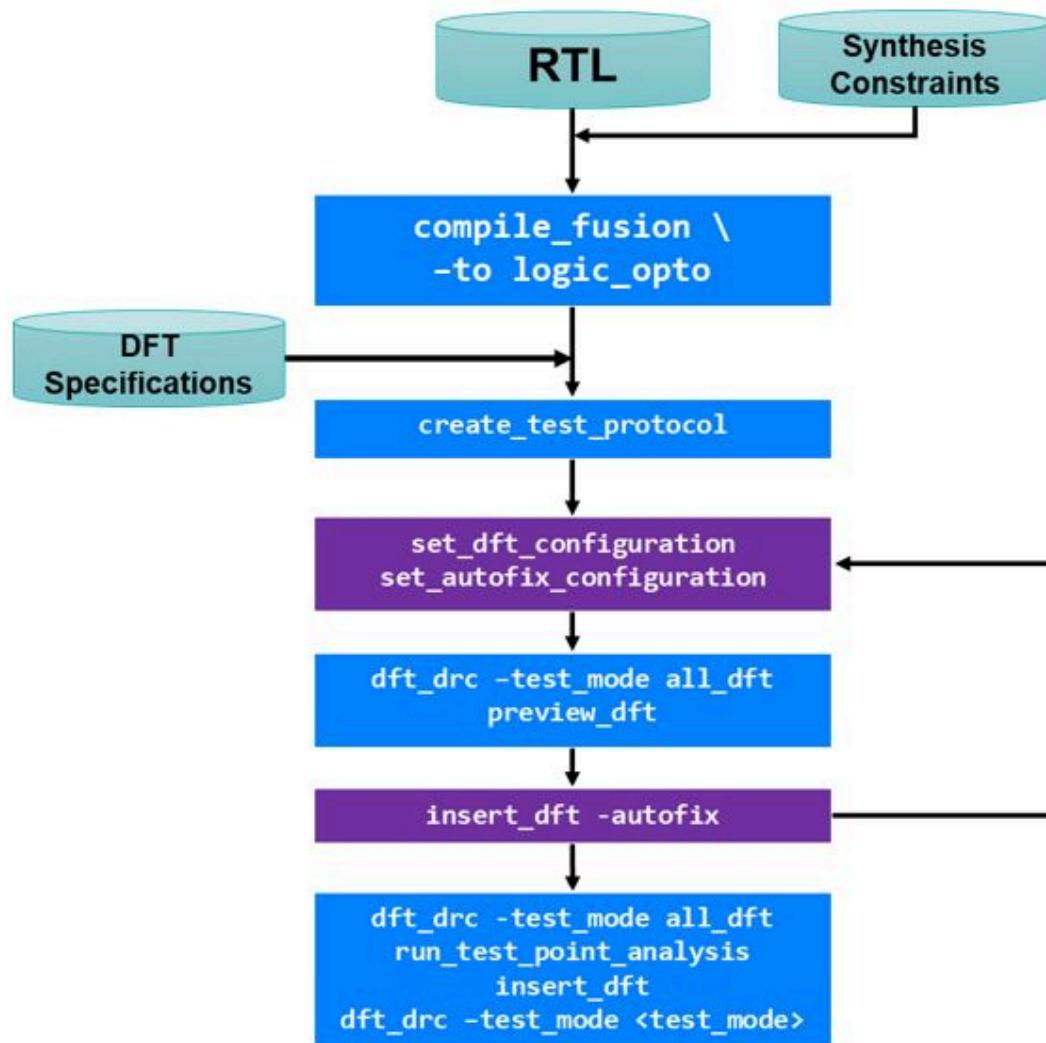
This method disables asynchronous resets during scan shift only. During scan capture, the functional reset logic controls the reset line. This does not limit the testability of the reset logic.

The AutoFix Flow

To enable AutoFix logic, use the `insert_dft -autofix` application option without inserting other DFT structures. You can iterate until pre-scan DRC gives clean results.

The AutoFix design flow fixes the violations automatically during `insert_dft` as per the in-compile flow illustrated in the following figure.

Figure 160 AutoFix Design Flow



Configuring AutoFix

The following topics explain how to use AutoFix:

- [Enabling AutoFix Capabilities](#)
- [Configuring Clock AutoFixing](#)
- [Configuring Set and Reset AutoFixing](#)
- [Reporting AutoFix](#)

Enabling AutoFix Capabilities

You can enable or disable individual AutoFix capabilities using `set_dft_configuration` command options, as shown in the following table.

Table 22 Options of the `set_dft_configuration` Command to Enable AutoFix

Signal type to AutoFix	Enabling option of the <code>set_dft_configuration</code> command	Default
Clock signals	<code>-fix_clock disable enable</code>	disable
Asynchronous set signals	<code>-fix_set disable enable</code>	disable
Asynchronous reset signals	<code>-fix_reset disable enable</code>	disable

Configuring Clock AutoFixing

Use the following options of the `set_autofix_configuration` command to configure clock AutoFixing:

```
set_autofix_configuration
  -type clock
  [-test_data clock_signal]
  [-control_signal test_mode_signal]
```

To specify an existing scan clock signal to use, define it as a `TestData` signal as well as a `ScanClock` signal, then specify it with the `-test_data` option of the `set_autofix_configuration` command:

```
set_dft_signal -view existing_dft -type ScanClock -port CLK \
  -timing [list 45 55]
set_dft_signal -view spec -type TestData -port CLK

set_autofix_configuration -type clock -test_data CLK
```

If no clock signal is specified, AutoFix chooses an available scan clock signal that is also defined as a `TestData` signal.

To specify an existing test-mode signal to use for AutoFixed clock test points, use the `-control_signal` option of the `set_autofix_configuration` command:

```
set_dft_signal -view spec -type TestMode -port AUTOFIX_TM

set_autofix_configuration -type clock -control_signal AUTOFIX_TM
```

If no control signal is specified, AutoFix chooses an available test-mode signal that is not already used for another purpose.

Configuring Set and Reset AutoFixing

Use the following options of the `set_autofix_configuration` command to configure set and reset AutoFixing:

```
set_autofix_configuration
  -type set | reset
  [-method mux | gate]
  [-test_data set_reset_signal]
  [-control_signal control_signal]
  [-fix_latch disable | enable]
```

By default, AutoFix uses MUXs to fix uncontrollable asynchronous set or reset signals. To specify an existing controllable set or reset signal to be selected by the MUXs, define it as a TestData signal as well as a Set or Reset signal, then specify it with the `-test_data` option of the `set_autofix_configuration` command:

```
set_dft_signal -view existing_dft -type Reset -port RSTN -active_state 0
set_dft_signal -view spec -type TestData -port RSTN -active_state 0

set_autofix_configuration -type set -test_data RSTN
set_autofix_configuration -type reset -test_data RSTN
```

If no such signal is specified, AutoFix chooses a set or reset signal that is also defined as a TestData signal.

Use the `-type set` or `-type reset` option of the `set_autofix_configuration` command to configure set or reset AutoFixing, respectively. You can use a single existing asynchronous set or reset signal to AutoFix both uncontrollable set and reset signals. However, the same asynchronous signal cannot be used to AutoFix both the set and reset pins of the same cell. If this occurs in your design, you must specify separate set and reset signals.

The `mux` fixing method can use only a test-mode control signal. To specify an existing test-mode signal to control the MUXs, use the `-control_signal` option of the `set_autofix_configuration` command:

```
set_dft_signal -view spec -type TestMode -port AUTOFIX_TM

set_autofix_configuration -type set -control_signal AUTOFIX_TM
set_autofix_configuration -type reset -control_signal AUTOFIX_TM
```


If no control signal is specified, AutoFix chooses an available test-mode signal that is not already used for another purpose.

To fix uncontrollable sets and resets using gating logic instead of a MUX, use the `-method` option to specify the `gate` fixing method:

```
set_autofix_configuration -type set -method gate
set_autofix_configuration -type reset -method gate
```

The `gate` fixing method can use either a scan-enable or test-mode control signal.

To specify a scan-enable or test-mode signal to control the gating logic, use the

`-control_signal` option of the `set_autofix_configuration` command:

```
set_autofix_configuration -type set -method gate -control_signal SE_FIX
set_autofix_configuration -type reset -method gate -control_signal SE_FIX
```

If no control signal is specified, AutoFix chooses an available test-mode signal that is not already used for another purpose.

By default, AutoFix does not consider set or reset pins of latch cells. To consider latch set and reset pins, use the `-fix_latch enable` option of the `set_autofix_configuration` command.

Note:

If you use the `gate` fixing method along with a scan-enable control signal, you must also allow unstable reset signals in both DFT DRC and TestMAX ATPG DRC. For more information, see [SolvNet article 021644, "How Can I Improve Testability for Internally Generated Asynchronous Set and Reset Signals?"](#)

Reporting AutoFix

After you enable and configure the AutoFix capabilities, you can use

`report_dft_configuration` command that reports AutoFix features that are enabled.

You can use `report_autofix_configuration` command to report detailed AutoFix specifications. For example,

```
fc_shell> report_dft_configuration
*****
Report : DFT Configuration
Design : des_unit
Version: U-2022.12-SP3-DEV
Date   : Fri Feb 24 11:00:14 2023
*****

DFT Structures      Status
-----
Scan:               enable
Testability:        disable
Wrapper:            disable
Scan Compression:   disable
Pipeline Scan Data: disable
Mode Decoding Style: binary
IEEE1500 Control:   disable
Fix Clock:          enable
Fix Set:            enable
Fix Reset:          enable

1
```

```
fc_shell> report_autofix_configuration
*****
Report : AutoFix Configuration
Design : des_unit
Version: U-2022.12-SP3-DEV
Date   : Fri Feb 24 11:00:41 2023
*****

Autofix Structures      Status
-----
Fix Type                Clock
Test Data:              "AUTOFIX_CK"
Control Signal:         "AUTOFIX_TM_CK"
Method:                 Mux
Fix Type                Set
Test Data:              "RSTN"
Control Signal:         "AUTOFIX_TM_RST"
Method:                 Gate
Fix Latch:              disable
Fix Type                Reset
Test Data:              "RSTN"
Control Signal:         "AUTOFIX_TM_RST"
Method:                 Gate
Fix Latch:              disable

1
```

Limitations

Using AutoFix has the following limitations:

- If you choose to use the gate method and use Scan Enable as your gating signal, the tool does not insert Pipeline stages on this Scan Enable signal
- You must run `insert_dft` command in two steps: AutoFix followed by scan synthesis

16

Running the Test DRC Debugger

This chapter describes debugging design rule checking (DRC) violations by using the Fusion Compiler graphical user interface (GUI).

Fusion Compiler GUI provides analysis tools for viewing and analyzing your design. It allows you to view the design violations, and it can provide an early warning of test-related issues. The GUI provides the debug environment for pre-DFT DRC violations, post-DFT DRC violations.

This chapter includes the following topics:

- [Starting and Exiting the Graphical User Interface](#)
- [Checking Scan Test Design Rules](#)
- [Viewing Design Violations](#)

Starting and Exiting the Graphical User Interface

To invoke Fusion Compiler GUI and view test DRC results, you need to

- Execute the `fc_shell -gui` command. Or,
- Choose File > Execute Script to run `fc_shell` script.
- Enter `dft_drc -gui` to check the design for DRC violations. This brings up the message browser

To exit Fusion Compiler GUI,

- Choose File > Exit

You can also enter `exit` or `quit` on the command line or press Control-c three times in the Linux shell.

To invoke Fusion Compiler GUI directly from `fc_shell`, enter

```
fc_shell> gui_start
```

To use options with this command, see the *Fusion Compiler Graphical User Interface User Guide* for further information.

Note:

Before invoking Fusion Compiler GUI, make sure you have correctly set your display environment variable. See the *Fusion Compiler Graphical User Interface User Guide* for information on setting this variable.

Checking Scan Test Design Rules

Before you perform other DFT operations such as inserting scan cells, check the current design for DFT DRC violations in your scan test implementation. You can use the message browser and the violation inspector to examine and debug the DRC violations reported.

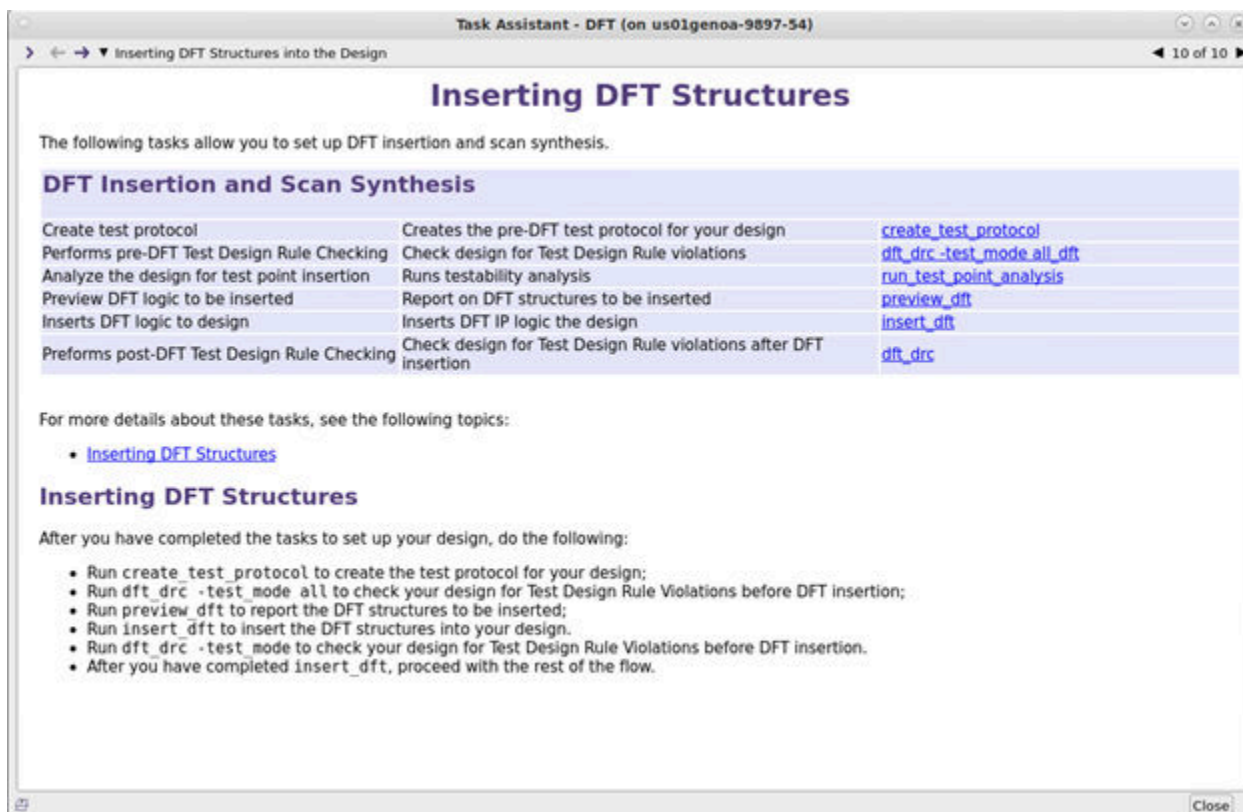
To view the DRC violations, you can:

- Use the `dft_drc -gui` command.

Or,

- From Task Assistant menu, you can choose Task Assistant -> DFT - -> Inserting DFT Structures into the design> Performs pre-DFT Test Design Rule Checking or Performs post-DFT Test Design Rule Checking.

Figure 161 Task Assistant Menu



The Fusion Compiler tool checks the design for DFT DRC violations and displays messages in the console log view. If violations exist, Fusion Compiler automatically opens a new message browser window and displays the violation messages.

Examining DRC Violations

You can use the message browser to search for and view information about DRC violations in the current design. The message browser window appears in a new message browser window

For more information about viewing and debugging DRC violations in the GUI, refer to [Viewing Design Violations](#) topic.

Viewing Design Violations

This topic covers the following:

- [Reporting DRC Violations](#)
- [Inspecting DRC Violations](#)

Reporting DRC Violations

The message browser lets you examine detailed information about violations and also provides a variety of tools for viewing the different aspects of a violation. The message browser groups the rule violations that help you find the problems you are concerned about.

To report the DRC violations, you can:

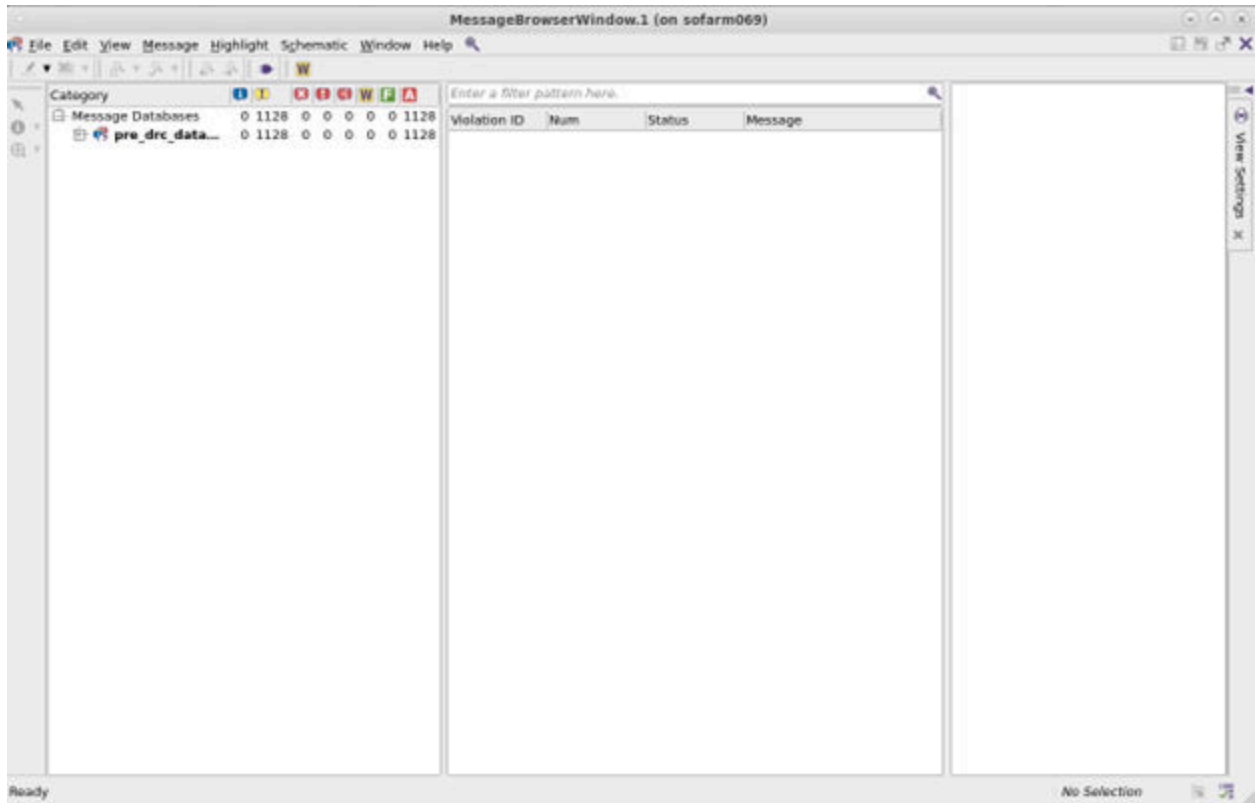
- Use the `dft_drc -gui` command.

Or,

- From Task Assistant menu, you can choose Task Assistant -> DFT - -> Inserting DFT Structures into the design> Performs pre-DFT Test Design Rule Checking or Performs post-DFT Test Design Rule Checking.

This opens the message browser view window, as shown in the following figure:

Figure 162 Message Browser View Window



Note:

The `dft_drc` command prints the rules with name `<alphabet><number>`, while the message browser and ManPages list the rules with `<alphabet> - <number>`. The `<alphabet>-<number>` convention is used in the following descriptions.

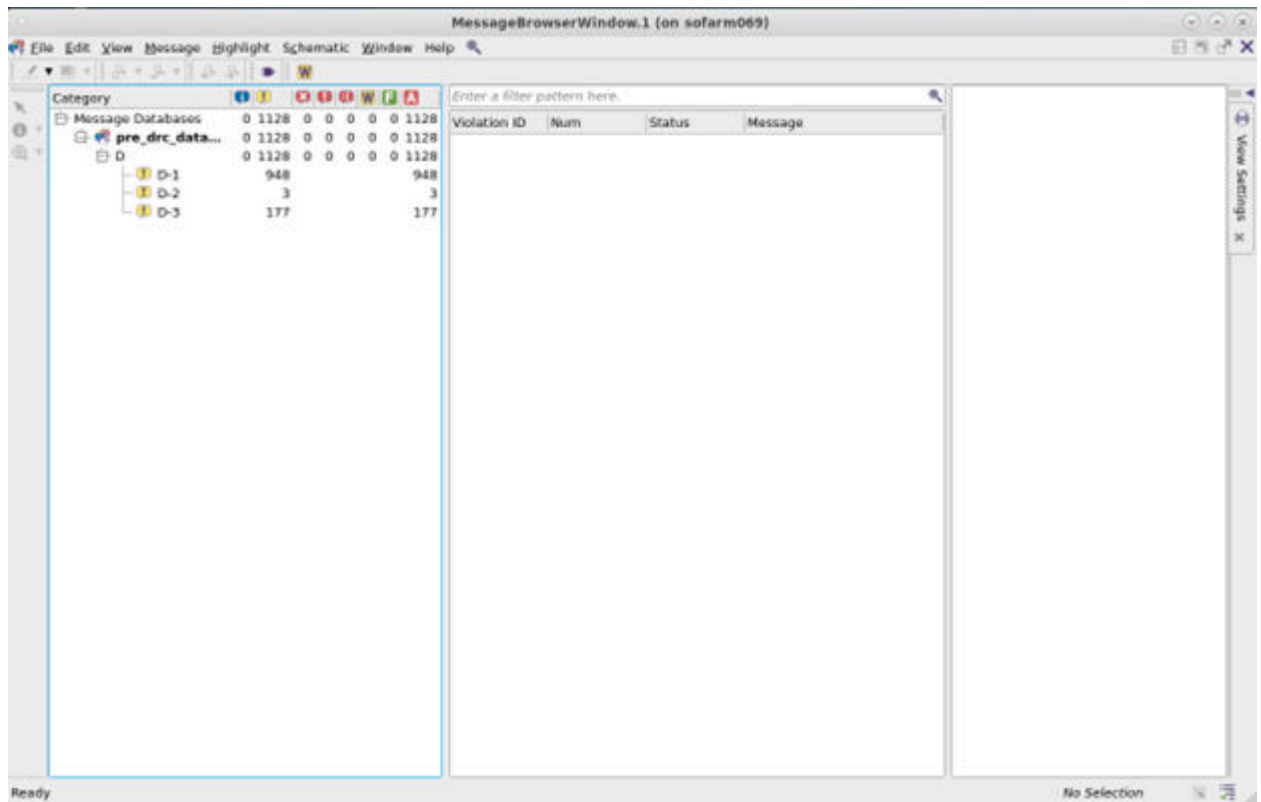
When the violation message is selected, the violation browser window consists of following three panes:

- A violation category tree on the left
- A violation messages in the middle
- A violation explanation on the right

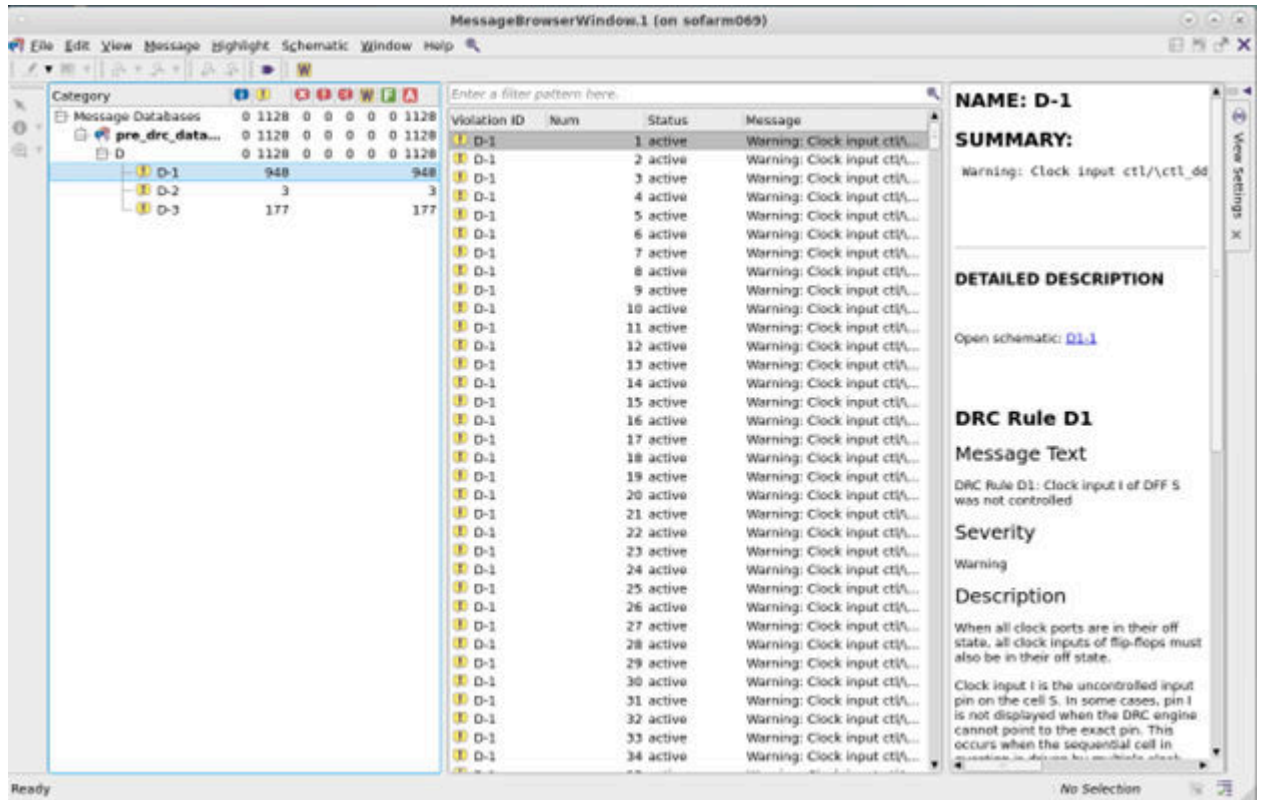
The Violation Categories pane lists different categories of violations. For example, Pre-DFT and Post-DFT.

To see the violations, perform the following steps:

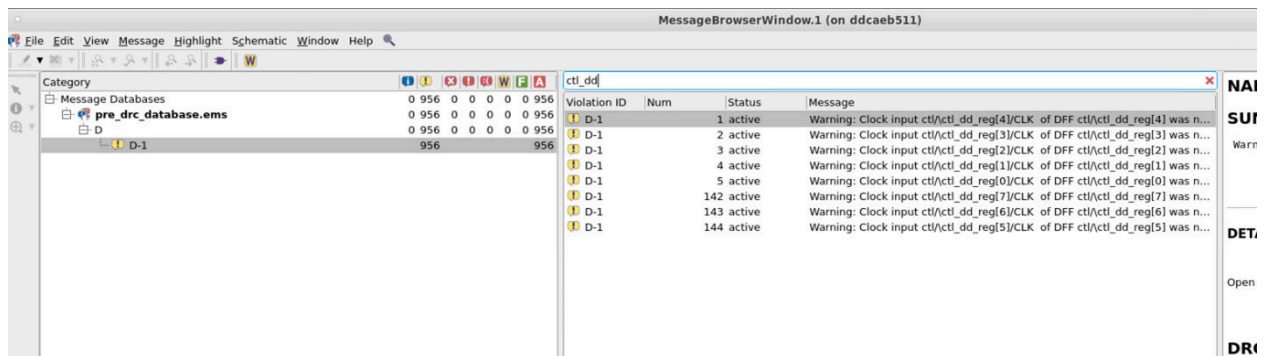
1. Click the expansion button (plus sign) of the violation category to display the violations of that group. The expanded view displays the types and number of violations as follows:



2. Select a DRC violation type in the left pane. A list of violating messages appears in the middle pane as follows:



3. To filter the violations list, you can enter name filter patterns in the search box at the top of the violation message pane as follows:



This is an optional step.

For example,

- Expand the `pre_drc_database`
- Expand the D violation tree
- Select violation D-1

The resulting D1 violations are shown in the middle pane.

- If you click a specific violation pin or a violation ID, the corresponding description is displayed in the description pane.
- If you click the link next to *open schematic* the Violation Inspector opens to view the violation schematic. For more details, see [Inspecting DRC Violations](#).

Inspecting DRC Violations

You can analyze and debug DRC violations by using the violation inspector window. You can inspect multiple violations of the same type together. The violation browser displays the schematic view that allows you to inspect the logic structure of the DRC violation, including the pin data. You can also display waveforms for `test_setup` pin data.

This topic covers the following:

- [Viewing a Violation](#)
- [Viewing Multiple Violations Together](#)
- [Viewing test_setup Pin Data Waveforms](#)

Viewing a Violation

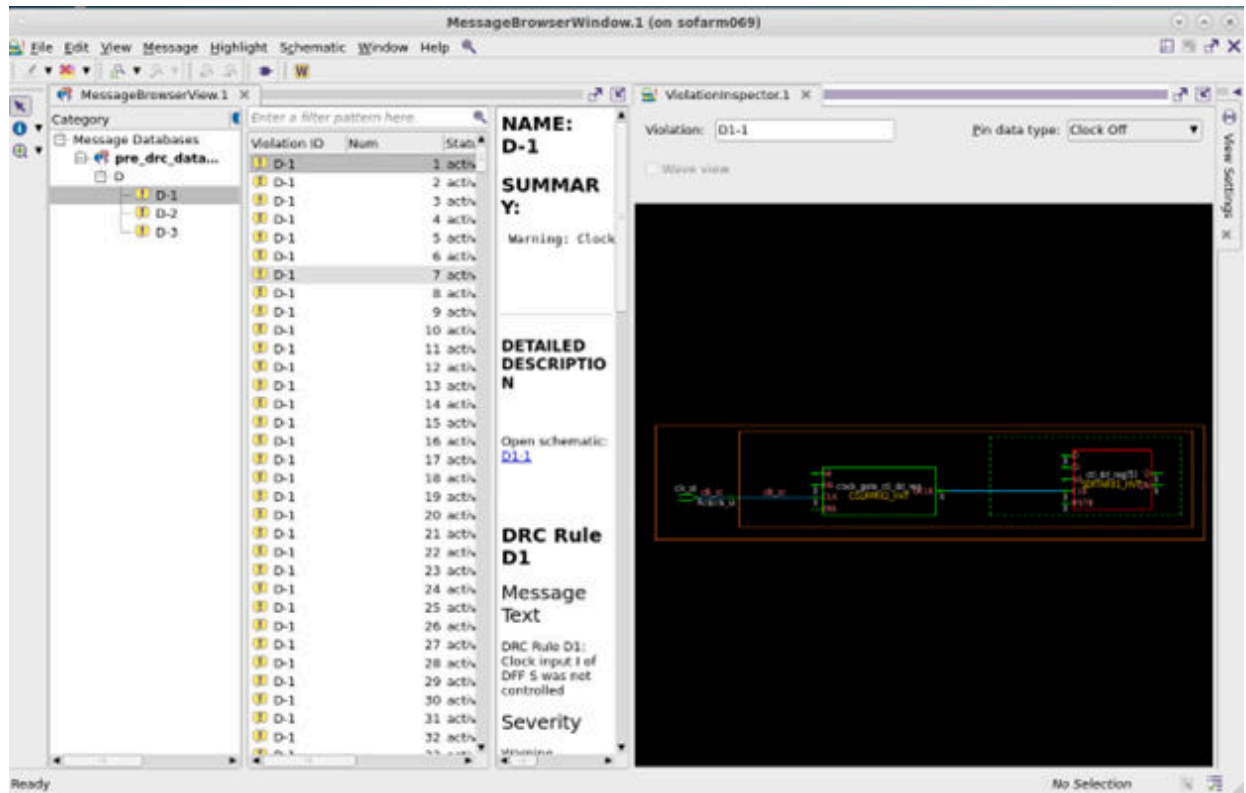
When you select a violation in the message browser, you can see a detailed explanation of the violation. You can click a link at the top of the detailed explanation to open the path schematic in the violation inspector which allows you to investigate the violations. A path schematic contains cells, pins, ports, and nets.

- Click the Inspect Violation tab at the bottom of the console.

This opens the violation inspector window, as shown in [Figure 163](#).

This window displays the path schematic for visually examining any violations and the violation source. A path schematic provides contextual information and details about the path and its components. Red-colored cells indicate pins with violations.

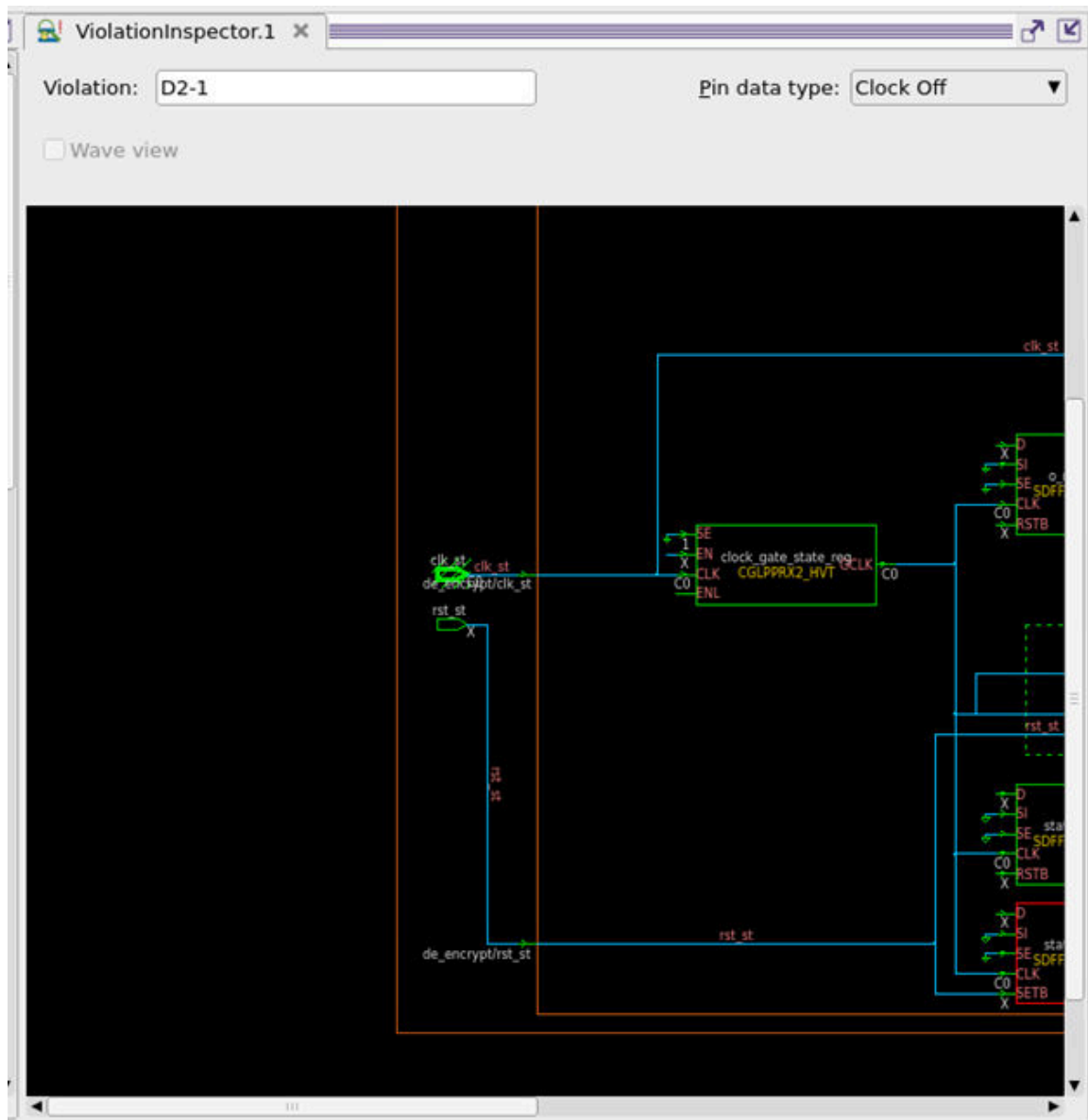
Figure 163 *Violation Inspector: Viewing a Violation*



- The Violation Inspector will open the appropriate pin data type to help you debugging the violation.
- (Optional) Select the data type name in the “Pin data type” menu to display a different pin data type.
- Display object information in an InfoTip by moving the pointer over a pin, cell, net, or other type of object.
- Pin data simulation values are annotated on the schematic.

- If you define a DFT signal with the `set_dft_signal` command, the signal source is highlighted with a hatched fill pattern, as shown in [Figure 164](#).

Figure 164 Highlighted DFT Signal Sources



Viewing Multiple Violations Together

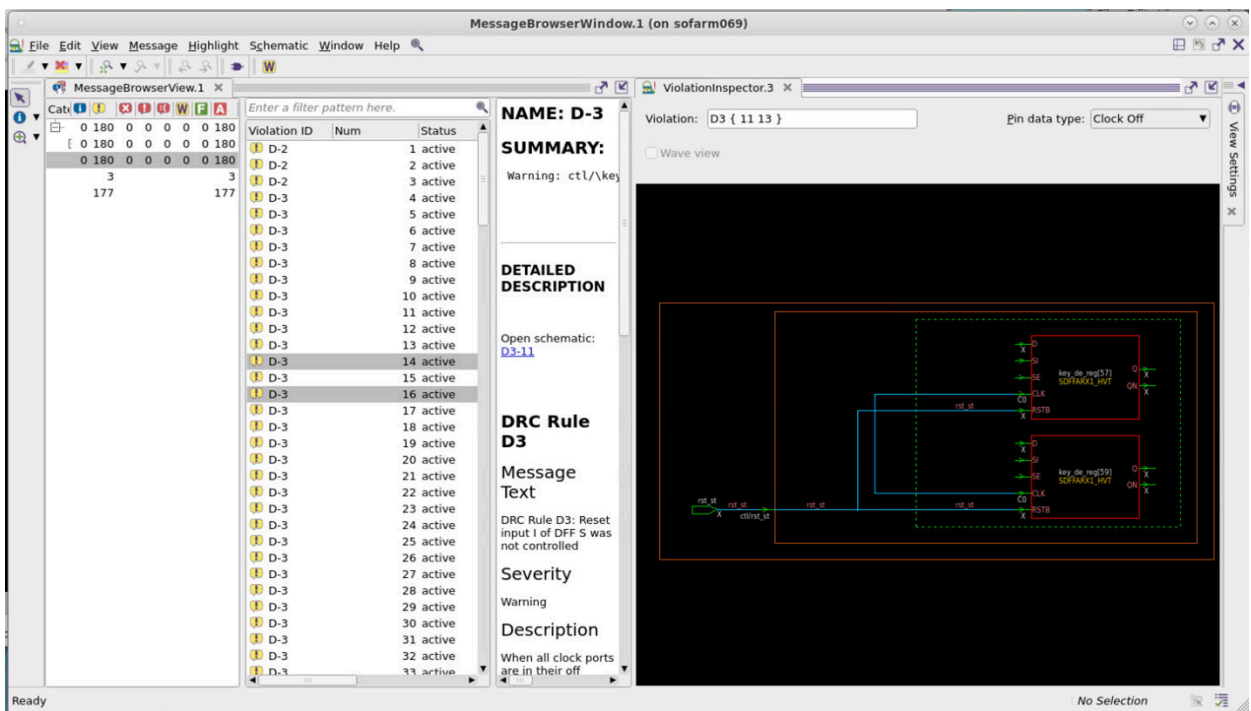
To view the path schematics for multiple violations,

1. Shift-click to select multiple violation IDs in the violation browser.
2. Right click, then select Inspect Violation button .

The schematics of the selected violations are displayed in the schematic viewer of the violation inspector.

Figure 165 shows the selection of multiple violations.

Figure 165 Viewing Multiple Violations



Viewing test_setup Pin Data Waveforms

In the Violation Inspector window, the *Pin data type* menu lets you choose what data to annotate on pins in the design schematic. Most pin data types are one or three characters (test cycles) long. However, pin data from the `test_setup` procedure is arbitrarily long and cannot be annotated on the schematic.

When you select `Test setup` in the *Pin data type* list, a waveform viewer appears that displays pin data waveforms for one or more pins. You can add, remove, and group pins together. The waveform viewer is integrated with the schematic display, so you can explore the logic cone and add additional pins of interest as needed.

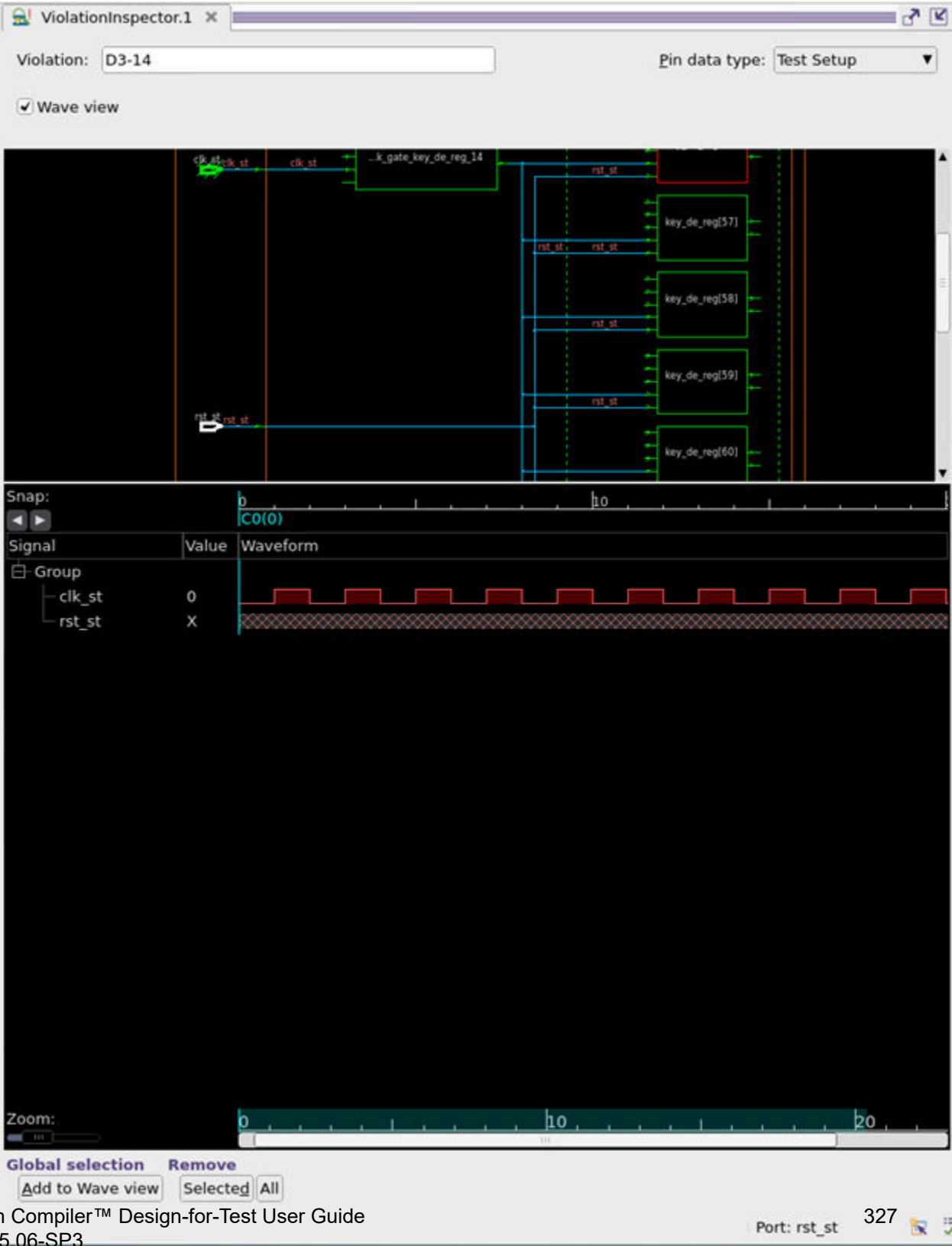
To display `test_setup` waveforms for pins,

1. Select a Violation from the Message Browser
2. Click on the link under *Open schematic*
3. In the Violation Inspector schematic window, select `test_setup` under *Pin data type*
4. Select the signal that you want to examine the pin data waveform, then click the `Add to Wave View` button
5. To add more signals to the waveform window, repeat step 4

The waveform view appears below the schematic view in the violation inspector window, as shown in [Figure 166](#). You can adjust the relative heights of these views by dragging the split bar up or down.

The waveform view consists of two panes: an expandable signal list on the left and the waveform viewer on the right. You can adjust the relative widths of the panes by dragging the split bars left or right.

Figure 166 Waveform Viewer



Use the Zoom dial at the bottom left corner of the Waveform view to adjust the zoom level.

Use the scroll bar at the bottom of the Waveform view to move the timeline.

You can remove signals by either click the *remove all* button to remove all signals, or select the signal and click the *remove selected* button to remove the selected signal.

17

DRC Rules

This chapter describes how to preview and insert your DFT IP, insert optimized scan structures during synthesis, and check the resulting DFT logic for correct operation.

This chapter includes the following topics:

- [Category B - Build Rules](#)
- [Category C - Clock Rules](#)
- [Category D - DFT Rules](#)
- [Category K - LBIST Rules](#)
- [Category N - Netlist Rules](#)
- [Category R - Compressor Rules](#)
- [Category S - Scan Chain Rules](#)
- [Category V - Vector Rules](#)
- [Category X - X-State Rules](#)
- [Category Z - Tristate Rules](#)
- [Category Others - All Other Rules](#)

Category B - Build Rules

See Also

- [B1 - Invalid top module](#)
- [B2 - Multiple choices for top module](#)
- [B3 - Invalid number of net entries for submodule](#)
- [B4 - Inout used unidirectionally](#)
- [B5 - Undefined module referenced](#)

- B6 - Undriven module inout pin
- B7 - Undriven module output pin
- B8 - Unconnected module input pin
- B9 - Undriven module internal net
- B10 - Unconnected module internal net
- B11 - Unsupported library primitive
- B12 - Undriven instance input pin
- B13 - Undriven instance inout pin
- B14 - Unconnected instance output pin
- B15 - Invalid instance pin connection
- B16 - Invalid name
- B17 - Driven tied signal net
- B18 - Tristate and non-tristate drivers combined
- B19 - Tristate driver on wired gate
- B20 - Non-tristate driver on buskeeper
- B21 - PI drives tristate and non-tristate gates
- B22 - Dropped design view
- B23 - Feedback path
- B24 - Memory never written
- B25 - No memory init file
- B26 - Invalid memory init file
- B27 - Incorrect memory init file size
- B28 - Invalid memory
- B29 - Invalid net connection
- B30 - Lost connection strength
- B31 - Module references itself
- B32 - Illegal buskeeper net

- [B33 - Cascaded BUSES](#)
- [B34 - Illegal wired net](#)

DRC Rule B1

Message Text

Build Rule B1: Invalid top module name (M).

Severity

Fatal Error

Description

The module name that you specified either does not exist or does not match the design because of case-sensitivity problems.

What Next

Make sure that you have read in a netlist. An empty netlist produces this error. Use the `report_module-summary` command to see a summary of modules defined.

Check the spelling of the top module name that you used. Compare it to the module name in netlists that TestMAX ATPG has read in. A `report_module-all` command lists the names of all modules known.

DRC Rule B2

Message Text

Build Rule B2 There are multiple choices for top module (M selected).

Severity

Warning

Description

There are multiple unreferenced modules in a netlist that can be chosen as the default top module. If the DRC severity is not set to error, the last unreferenced netlist module is chosen as the top module.

M is the module name of the selected top module.

What Next

If the reported top module is not correct, specify the correct top module name in the `run_build_model` command.

DRC Rule B3

Message Text

Build Rule B3: Invalid number of pins (N1/N2) for module primitive (M/I).

Severity

Fatal error

Description

The number of net entries given for an instantiation of a module that uses an ordered list of pins must be consistent with the number of external pins for the module.

N1 is the number of net entries of the instance, N2 is the number of external pins for the instantiated module, M is the name of the module that contains the instance, and I is the name of the instance.

What Next

The netlist of the module in error must be corrected and read in again before a successful model build can be done.

DRC Rule B4

Message Text

Build Rule B4 I/O used unidirectionally (M).

Severity

Warning

Description

A module port pin declared as bidirectional is connected to internal gate pins that are only inputs or only outputs.

The presence of this violation has no effect on ATPG pattern generation or test coverage.

What Next

Consider changing the module's port declaration from bidirectional to input or output.

The DRC severity can be changed to ignore using the `set_rules b` command.

DRC Rule B5

Message Text

Build Rule B5: Module (M) referenced undefined module (U).

Severity

Error

Description

When using the `run_build_model` command, all modules referenced by the top module must be defined or specified as a black box or empty box. In this case, a module referenced by the top module was not defined.

`M` is the name of the module that referenced the undefined module and `U` is the name of the undefined module.

What Next

You can review a list of missing modules using the `-undefined` option of the `report_modules` command. You must provide definitions for this list of missing modules or mark them as black box or empty box models.

Some Verilog primitives are not supported. When these primitives are referenced by the design or by library cells, they appear as undefined modules. The primitives are: `rtran`, `tranif`, `rtranif0`, `rtranif1`, `tranif0`, `tranif1`. In this case, you need to create a custom ATPG model as a replacement. For more information, see ATPG Modeling Primitive Summary.

You can use the `-black_box` and `-empty_box` options of the `set_build` command to explicitly name modules that are missing and for which you would like black box or empty box behavior. To determine the list of missing modules, adjust the severity of the B5 rule to a warning and repeat the `run_build_model` command. Next, report all missing modules using either the `report_violationsb5` command or the `report_modules-undefined` command. Use this list to construct an explicit series of `set_build` commands with the missing modules identified as black box or empty box. Then repeat the build and continue.

DRC Rule B6

Message Text

Build Rule B6: Undriven module inout pin (M/D).

Severity

Warning

Description

An inout pin is not driven and is treated as inout (default), input, or output.

What Next

The selection of how to treat an undriven pin can be made using the `set_build-undriven_bidi` command.

DRC Rule B7

Message Text

Build Rule B7: Undriven module output pin (M/P).

Severity

Warning

Description

An output pin of a module must be driven by circuitry inside the module. A module was encountered with a port declared as an output, which did not have a connection to an internal gate output. The output port is then undriven.

M is the name of the module and *P* is the name of the pin.

This DRC is performed at the *module* level, and does not consider the effect of any `add_net_connections` commands which can affect the final in-memory image after run build completes.

What Next

Review the module definition and make changes to the module.

If needed, you can lower the DRC severity to warning or ignore, and the output port is set to TIEZ. All connected pins are also set to TIEZ. ATPG pattern generation can then proceed but overall coverage is affected.

DRC Rule B8

Message Text

Build Rule B8: Unconnected module input pin (M/P).

Severity

Warning

Description

An input pin of a module should drive circuitry inside the module. When a module port list is declared with an input pin and this input pin has no connection to internal gates, then this DRC violation occurs.

M is the name of the module and P is the name of the pin.

Note: This DRC is performed at the module level, and does not consider the effect of any `add_net_connections` commands which can affect the final in-memory image after run build completes.

What Next

After investigating the source module, if no change to the module is made, change the severity of the DRC to warning or ignore. This should have no affect on ATPG pattern generation.

DRC Rule B9

Message Text

Build Rule B9: Undriven module internal net (M/N).

Severity

Warning

Description

An internal net of a module must be driven by circuitry inside the module.

M is the name of the module and N is the name of the net.

This message is generated when an internal net is defined or implied by instance connections and the net is not connected to an input port, a bidirectional port, or an internal instance output pin. When this happens, there is no signal driver for this net.

Note: This DRC is performed at the module level, and does not consider the effect of any `add_net_connections` commands which can affect the final in-memory image after run build completes.

What Next

Frequently this message occurs when a net/wire is defined within the module but not actually used.

Review the module definition to determine if this problem is harmless or serious.

If needed, you can use the `set_rules` command to set the DRC severity to warning or ignore and any input pins attached to the undriven net is connected to TIEX for ATPG pattern generation purposes.

DRC Rule B10

Message Text

Build Rule B10: Unconnected module internal net (M/N).

Severity

Warning

Description

An internal net of a module must drive circuitry inside the module. The net might be connected to a driving gate output, but has no connection to another gate input or to a module output or inout port.

M is the name of the module and *N* is the name of the net.

This type of violation is common when designs change and sections of circuitry are deleted but a few nets become unused and are not removed. This violation poses no danger to pattern generation and will have no effect on test coverage. It can be ignored.

Note: This DRC is performed at the *module* level, and does not consider the effect of any `add_net_connections` commands which can affect the final in-memory image after run build completes.

What Next

If the DRC severity is set to warning or ignore, the net is left dangling and will automatically be removed if default `set_build-delete_unused_gates` options are in effect. This will have no affect on test coverage or ATPG pattern generation.

If there are also B9 violations in the same module you might want to investigate as there might be a net name mismatch error or typo which is causing an output pin that goes nowhere (B10) and an associated input without a driver (B9).

DRC Rule B11

Message Text

Build Rule B11: Unmodeled netlist simulation primitive (P) modeled as TIEX.

Severity

Warning

Description

A netlist simulation primitive must be able to be modeled as an ATPG primitive.

P is the name of the netlist primitive.

What Next

If the DRC severity is set to warning or ignore, the netlist instance are modeled as a TIEX gate. The outputs is constant X values. If this is not desirable, provide an alternate ATPG model.

DRC Rule B12

Message Text

Build Rule B12: Undriven instance input pin (P).

Severity

Error

Description

An instantiation of a module has a floating input pin.

What Next

Investigate the floating input. A floating input is very often a netlist error and should be corrected. If you determine that a floating input is not an error, then use the `set_rules` command to adjust the severity of this rule to warning or ignore and then continue with another `run_build_model` command. All floating inputs is connected to TIEX sources during the build operation.

DRC Rule B13

Message Text

Build Rule B13: Undriven instance inout pin (P).

Severity

Warning

Description

An instantiation of a module has a floating bidirectional pin.

P is the pathname for the pin.

What Next

You might want to investigate the floating bidirectional pin. If you had intended the pin be used as only an output there will not be a problem. However, if you had intended that the pin be used as an input there is no connection to another driving gate in the design and this might be a netlist error.

DRC Rule B14

Message Text

Build Rule B14: Unconnected instance output pin (P).

Severity

Ignored

Description

An instance output pin should be connected to some other pins or primary outputs or faults cannot be detected upon the output.

P is the pathname for the pin.

What Next

If the DRC severity is set to warning or ignore, the unused output pin is left dangling and results in unused circuitry. ATPG pattern generation can still be performed. The unused outputs will result in UU faults.

Unused pins can be removed during unused gate optimization using the `set_build-delete_unused_gates` command.

DRC Rule B15

Message Text

Build Rule B15: Invalid instance pin usage (P).

Severity

fatal error

Description

The design usage of an instance pin must be consistent with the declared direction from the defining module.

P is the pin pathname within the design where the violation occurs.

This violation will occur if a module's pin direction is in conflict with how the pin has been connected in the design. For example, a pin defined as an output is connected to a TIE1, TIE0, or TIEX source within the netlist.

What Next

Check the list of ports used when the module is instantiated in the design against the list of ports in the module definition. Some mismatch can have occurred, especially if the port connections are made "by order", rather than "by name".

This error can occur when module **A** is defined which has vectored ports in its port list, then another module **B** which references **A** has been defined, and then module **A** is read again or a different version of module **A** is read which redefines the port list or has the potential of redefining the port list. You should check the list of **N5** violations to see if any modules have been redefined. If so, and one of the redefined modules corresponds to the module involved with this violation message, then try to eliminate the duplicate module definitions. You might also try changing the order in which modules are read, as this will affect which module definition is used when there are duplicates.

If there has been no duplication of module definitions, and the module in question is correct, then the instantiation of this module is in error and the design netlist must be corrected and the design re-read before a successful model build can be completed.

DRC Rule B16

Message Text

Build Rule B16: Invalid instance pin name (P).

Severity

Fatal error

Description

The name of a pin of an instantiated module with a named pin list must be consistent with its associated module pin. When the module was instantiated in the design netlist one pin name was used, but the definition of the module itself called out a different pin name.

P is the instance pathname of the pin.

Failure to satisfy this rule is a fatal error and the flattening process is halted. Without a flattened model you cannot proceed in the ATPG pattern generation flow.

What Next

Review the names of the pins as defined in the module versus the names of the pins called out when the module is instantiated in the design. The two usages must match or the error will continue to occur. After the mismatch has been corrected, re-read the design and libraries before attempting another `run_build_model` command.

Sometimes the error can occur because the design has been flattened and the original instance path names retained using an escape mechanism such as:

```
\\I$1/ABC/I$31/U3
```

You can try adjusting the default hierarchy character (which is "/") to overcome this problem. The `-hierarchy_delimiter` option of the `set_build` command is used for this. Try using a period ".".

DRC Rule B17

Message Text

```
Build Rule B17: Tied signal net N (nsource=#, nioconns=#) is driven by  
<list of drivers>.
```

Severity

Fatal Error

Description

A net which is connected to a `supply0` or `supply1` statement in Verilog or is tied to a zero or one by instantiating a module and then placing a `1'b0` or `1'b1` or other constant in its port list is considered a tied net. A tied net connected to a module port which is declared to be either output or bidirectional will produce this violation.

The tied net is identified by `N` along with the number of driver sources as well as bidirectional sources attached to this net in addition to the tied value. The additional drivers are listed by their pin pathnames.

Failure to satisfy this rule is a fatal error and the flattening process is halted. Without a flattened model you cannot proceed in the ATPG pattern generation flow.

What Next

A common cause of this error is the definition inside of a module of a port direction of `inout` or `output` which should be `input`.

The netlist of the module in error must be corrected and read in again before repeating a `run_build_model` command.

DRC Rule B18

Message Text

Build Rule B18: Three-state and nonthree-state drivers on T gate I (G).

Severity

Warning

Description

A net is driven by at least one three-state and at least one nonthree-stateable driver, which results in the net not being able to be three-stated.

`T` is the type of the driven gate (typically a BUS gate), `I` is the instance name and `G` is the primitive ID number.

What Next

You should investigate the design. If left uncorrected, some test coverage loss occurs.

There is an option to `run_build_model`, called `-weakgates`, which treats all non tristate drivers on the net with a tristate driver as having a weak drive strength. If your design matches this condition, you might want to rebuild the in memory image using the `-weakgates` option.

DRC Rule B19

Message Text

Build Rule B19: Three-state driver on wired T gate I (G).

Severity

Warning

Description

A wired-OR or wired-AND gate cannot be driven by a three-state driver.

`T` is the type of the driven gate, `I` is the instance name and `G` is the primitive ID number.

What Next

Investigate the design. If left uncorrected, some test coverage loss occurs.

DRC Rule B20

Message Text

Build Rule B20: Nonthree-state drivers on T gate I (G) .

Severity

Warning

Description

A buskeeper primitive is driven by a non tristate driver, which renders the buskeeper useless.

T is the type of the driven gate, I is the instance name and G is the primitive ID number.

What Next

Investigate the design. If left uncorrected, some test coverage loss occurs because the buskeeper gate never has any influence on the propagated value.

DRC Rule B21

Message Text

Build Rule B21: T gate I (ID) drives three-state and nonthree-state gates.

Severity

Ignored

Description

A primary input fans out to both three-state and non three-state gates.

T is the type of the gate (PI), I is the instance name and ID is the primitive ID number.

What Next

No action is required and no loss of test coverage occurs. For some design styles, this is an informative message that might suggest design flaws.

DRC Rule B22

Message Text

Build Rule B22: Message variations:

#1 - Pin *P* of internal BUS could not be placed anywhere, dropped design view of *I* with *N* pins.

#2 - Pin *P* of removed equivalent DLAT could not be placed anywhere, dropped design view with *N* pins.

#3 - Dropped design view of *I* with *N* pins, merged with *I2*.

#4 - Dropped design view of *I* with *N* pins.

#5 - Pullup/downs removed from PIOs: *N* pins lost.

Severity

Warning

Description

For the following discussion, *P* is a pin name, *I* is an instance path, and *N* is a count.

For message type #1, when multiple BUS or WIRE primitives are directly connected in series they are merged into a single primitive. This is done for efficiency of the ATPG algorithm, for bus analysis, and so forth. When multiple BUS or WIRE gates are merged there are pins in the middle which disappear. An attempt is made to move these named pins (only named pins can hold faults) to equivalent positions. If this is not possible the B22 violation is issued. After a pin of an instance is lost, the instance is not viewable at the design-level, only at the primitive level.

Message type #2 is similar to #1 except that you have two DLAT or DFF primitives in parallel which are being merged into a single gate.

For message type #3, the situation is again similar to #1 except that pins are dropping the from an entire instance, *I*, as it is merged with another instance, *I2*. A design cell might be a candidate for message type #1 and #3 and which message is generated is somewhat random.

For message type #4, the situation is again similar to #1 and #3. This is a catch-all message for the cases where pins could be moved and instances are not merged yet you cannot keep the design-level representation. An example would be a BUS primitive that was created by merging several other BUS primitives for which one of the original BUS primitives had a pin with no name. The final BUS is missing at least one pin name and cannot be shown at the design level, only the primitive level.

For #5, pullup/downs were removed from all bidis as a result of specifying `run_build_model-remove_pio_pull`.

Failure to satisfy the B22 rule can result in a slight drop in fault population. There is no danger that patterns are created which do not simulate.

What Next

For most users, no action is required.

For the test purist you can count up the number of pins dropped due to B22 violations and use this information to manually adjust the final test coverage or fault coverage. For most users this number is negligible.

DRC Rule B23

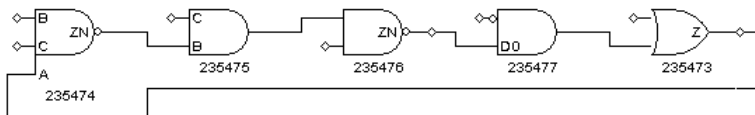
Message Text

Build Rule B23: Feedback path#ID contains N1 gates and N2 sources.

Severity

Warning

One example, many variations are possible:



Description

A combinational feedback path was identified. As each combinational feedback path is identified, it is recorded as a B23 violation. `ID` is the identification number assigned to this feedback path, `N1` is the number of gates in the path, and `N2` is the number of sources in the path. Feedback path sources are a subset of the gates in the feedback path that, if removed, would eliminate the feedback. The sources subset might not be unique. For example, in the picture above, any one gate can be considered a source. The more sources a feedback path has, the more "branches and loops" it has.

What Next

A few feedback loops are not uncommon and should not greatly affect performance or test coverage. However, an excessive number of feedback loops (> 50) or a loop with a high number of gates could increase CPU effort to create patterns and lead to lower test coverage. In addition, if the number of feedback loops is unexpected, this could be a sign of a library problem or the use of a library modeled at the transistor or pass-gate level.

You can also use the `report_feedback_paths` command to review the number of gates and sources in the feedback loop. You can also use it to get a detailed list of instance path names of gates in the loop. The "source" gates of the feedback loop are always listed first when the verbose option is used.

You can analyze a B23 violation using the graphical schematic viewer by selecting the ANALYZE button on the left, and then choosing the specific occurrence of the B23 violation from the menu. You can also perform an analysis by manually entering the `analyze_violation` command with the `-display` option. This sets the pin data reporting to constrain value data and graphically displays all the gates in the feedback path.

If you are experiencing ATPG performance problems (runtime is long) and you suspect it might be due to the presence of feedback loops, you might want to consider manually breaking the loop for ATPG efforts by disconnecting selected gates in the loop. The `add_net_connections` command along with the `TIEX` and `-disconnect` options might be used to disconnect all inputs from selected gates in the feedback path. However, the use of net connections can lead to performance issues during circuit flattening, so you might want to modify the netlist (used only in the ATPG environment) to break the loop rather than use the `add_net_connections` command.

DRC Rule B24

Message Text

Build Rule B24: Memory (M) has no write ports and no init file.

Severity

Error

Description

A RAM memory must have at least one write port and a ROM memory must have an initialization file. If neither is true, the memory is always at X.

M is the memory instance name.

What Next

Review the netlist or the model to make sure that the RAM has at least one write port.

If the device is a ROM, ensure that it has an initialization file and that the pathname in the model definition for the initialization file can be found.

You can also downgrade the severity of this rule to warning or ignore using the `set_rules` command. The memory with the violation is treated as a TIEX source for any outputs which can lower test coverage.

DRC Rule B25

Message Text

Build Rule B25: Cannot open memory init file (F).

Severity

Error

Description

The memory initialization file must be readable.

F is the file pathname that was attempted.

What Next

Check the spelling and file pathname specified for the memory initialization file.

Check the permissions and ownership of the file to make sure you have read access to it.

The `read_memory_file` command can be used to attempt to reread the memory initialization file associated with a particular memory instance.

TestMAX ATPG follows Verilog rules for locating specified initialization files from paths inside Verilog models by resolving file references in the same manner as a Verilog simulator. Since a design/module file contains hard-coded relative paths, make sure it has not moved to a different location; you can address this either by changing the path inside the Verilog model, or by replicating the path to the external file in the new location.

DRC Rule B26

Message Text

Build Rule B26: Line **L** (*filename*), invalid memory init file entry (D).

Severity

Error

Description

The memory initialization file must be syntactically correct.

L is the line in the file, and **D** is the text segment or address where the invalid syntax was encountered.

What Next

Correct the syntax problem in the file and try again. If you are in DRC mode you can use the `read_memory_file` command to interactively try re-reading the memory initialization file.

DRC Rule B27

Message Text

Build Rule B27: Line L (*filename*), D data values in memory init file.

Severity

Ignored

Description

The number of data values in the memory initialization file must be equal to the number of data bits in the memory.

What Next

Adjust the number of bits in the file or redefine the RAM model to match the data width of the file. Unspecified memory bits are considered to be X.

DRC Rule B28

Message Text

Build Rule B28: Line L (*filename*), invalid memory (<additional info>).

Severity

fatal error

Description

The memory described must be valid.

What Next

The netlist of the module in error must be corrected and read in again before a successful model build can be done.

DRC Rule B29

Message Text

Build Rule B29: Invalid net connection (<additional info>).

Severity

Error

Description

The defined net connection could not be found in the design during the design build process.

A net connection cannot be checked when it is first defined, because the complete design might not yet have been read in and the top-level module has not yet been selected. Only during the flattening of the design (which comes with `run_build_model` command) can the net connection be properly checked for a match. When no match is found, the B29 violation is issued.

What Next

Check your spelling and case sensitivity. Your defined net connection must match exactly the net pathname used in the design. If Verilog-escaped names are used, you might want to review faults located within the same design instance for an idea of the pin pathnames to the corresponding area. This can assist in forming a correct net pathname.

If the violation message indicates "Cannot add net connection on source- and sink-less net", then your target net is a floating net. This can occur when using the `-alloption` and some modules declare nets such as `GND`, but these nets are not used. This type of violation can be ignored as it causes no harm to the ATPG environment. However, you might want to edit the design and delete the unused net. This net will also show up as a B10 violation.

After investigating the B29 violations, if you find that they can be ignored, you might want to use the `set_rules` command to downgrade the rule to a severity of ignore, or just continue to ignore the warnings generated.

DRC Rule B30

Message Text

Build Rule B30: Transfer path from weak input of BUS I (G) to strong input of BUS I (G).

Severity

Warning

Description

The tool cannot properly model a purely transfer path that changes strength from weak to strong.

The message indicates the instance path of the two devices as "I" and the primitive ID as "G".

This violation occurs when there exists a transfer path (that is, a path through only BUS primitives and the data path of SW primitives) that starts with a weak input to a BUS and ends with a strong input to another BUS. If the value of the first BUS is given by its weak input (all other strong inputs being Z, and all other weak inputs being either Z or driving the same value) then this value will appear through at least one SW primitive as a *strong* input to the second BUS. This can cause mismatches against a simulator that accurately considers all strength levels.

Mismatches between the 2-strength ATPG simulator and a full strength-accurate simulator can result in several cases of multiple strengths designs, but this is the only case where a mismatch can result in a design with only two strengths (normal and weak).

What Next

You can use the ANALYZE button to see the source gates graphically. However, there is not much you can do about the issue. There is a potential that ATPG patterns cannot match simulation results.

DRC Rule B31

Message Text

```
Build Rule B31: Module (M) references itself.
```

Severity

fatal error

Description

A module must not directly or indirectly reference itself.

What Next

The netlist of the module in error must be corrected and read in again before a successful model build can be done.

DRC Rule B32

Message Text

Build Rule B32: Illegal Bus Keeper Net.

Severity

fatal error

Description

A bus keeper has been defined which cannot be initialized to a value because attachment to another driving gate is absent.

What Next

The netlist of the module in error must be corrected and read in again before a successful model build can be done.

One commonly occurring cause of this type of error is a bus keeper circuit attached to a net which is declared as a module output with no other internal gates which can drive the net. Changing the port definition to an inout is required to overcome the error.

DRC Rule B33

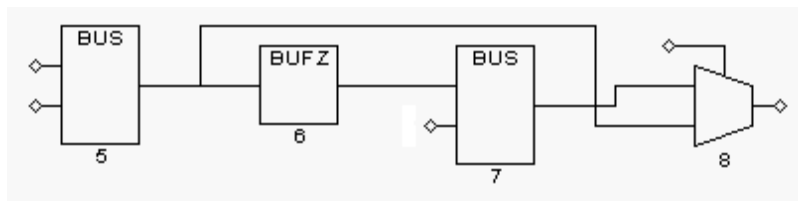
Message Text

Build Rule B33: Cascaded Buses.

Severity

Warning

One example, many variations are possible:



Description

The flattened model contained two cascaded BUS primitives, or two cascaded BUS primitives separated by a BUFZ primitive. BUS primitives are inserted during flattening to resolve the output of tristateable drivers and bus holds.

This condition could cause TestMAX ATPG to see contention and reject patterns due to contention that would otherwise be kept. This is because the contention detection algorithm requires that all gates driving a BUS primitive have an enable (TSD's), or be weak drives (BUSK's). When a BUFZ occurs between two BUS primitives it is regarded as a gate that can't be disabled and the BUS prim is marked as having contention.

What Next

The occurrence in the design of logic that would justify cascaded BUS devices is very rare and is most often an error in the netlist. Carefully check the source of the gates involved, as well as the defined directions of the ports in the parent module.

If the netlist has no error, you might want to disable contention checking to avoid discarding patterns due to this false contention condition. Unfortunately, the contention checking is globally disabled and this cannot be acceptable.

Sometimes insertion into the original design of a unidirectional gate can overcome this issue.

DRC Rule B34

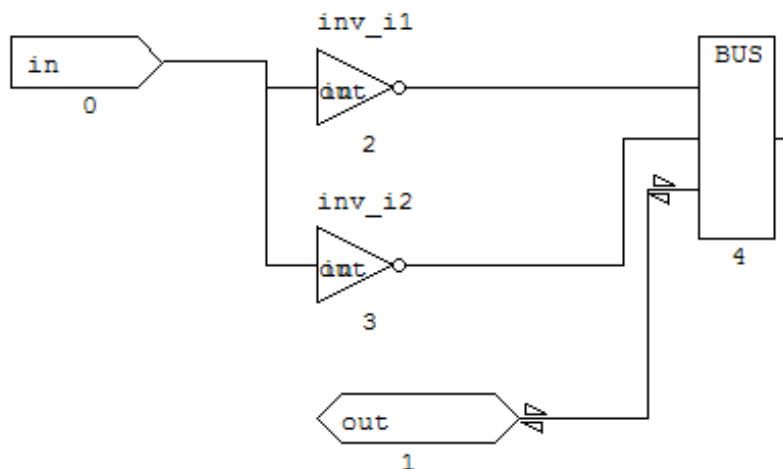
Message Text

Build Rule B34: Illegal wired net

Severity

Error

The following figure shows one example of the B34 build rule violation; many variations are possible:

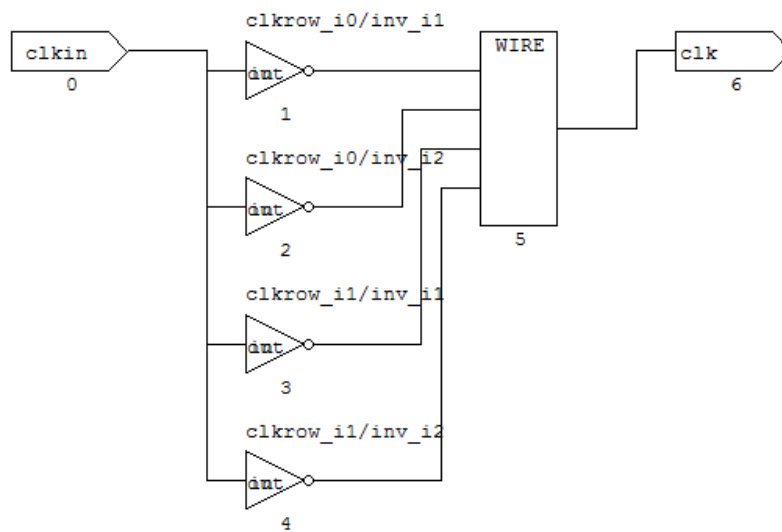


Description

TestMAX ATPG cannot properly model or represent wired_and or wired_or nets with bidirectional connections.

The flattened model contains two primitives in parallel. The BUS primitive is inserted during flattening to resolve the output of tristateable drivers and bus holds.

If instantiating two of the previous modules and wired_or the out this cannot be represented properly in TestMAX ATPG. When you downgrade the B34 build rule, TestMAX ATPG builds the model ignoring the wired statement.



```
module top (clkin, clk);
input clkin;
output clk;
wor clk;

clkrow clkrow_i0 ( .out ( clk ) , .in ( clkin ));
clkrow clkrow_i1 ( .out ( clk ) , .in ( clkin ));
endmodule

module clkrow ( out , in );
input in ;
inout out ;

    invert_it inv_i1 (.out ( out ) , .in ( in ));
    invert_it inv_i2 (.out ( out ) , .in ( in ));
endmodule

module invert_it (out, in);
input in;
output out;

```



```
    not inv1 (out,in);  
endmodule
```

What Next

When you downgrade the B34 build rule from its default rule violation, TestMAX ATPG builds the model ignoring the wired statement. The final netlist might not be the required representation.

A design with a logic that justifies the wired_and or wired_or devices with bidirectional output ports is very rare. It is most probably an error in the netlist. Carefully check the source of the gates involved as well as the defined directions of the ports in the parent module.

To overcome this issue, insert a unidirectional gate or port in the original design.

Category C - Clock Rules

See Also

- [C1 - Unstable scan cells when clocks off](#)
- [C2 - Unstable nonscan DFF when clocks off](#)
- [C3 - No latch transparency when clocks off](#)
- [C4 - Clock unable to capture](#)
- [C5 - LS port captured data affected by new capture](#)
- [C6 - TE port captured data affected by new capture](#)
- [C7 - LE port captured data affected by new capture](#)
- [C8 - LS port clock path affected by new capture](#)
- [C9 - TE port clock path affected by new capture](#)
- [C10 - LE port clock path affected by new capture](#)
- [C11 - LS port captured data affected by clock](#)
- [C12 - LE port captured data affected by clock](#)
- [C13 - TE port captured data affected by clock](#)
- [C14 - Clock connected to multiple ports of same cell](#)
- [C15 - Scan cell port unable to capture](#)

- C16 - Nonscan cell port unable to capture
- C17 - Clock connected to PO
- C18 - Clock connected to PO affected by new capture
- C19 - Clock connected to non-contention-free BUS
- C20 - Unstable RAM when clocks off
- C21 - RAM port unable to capture
- C22 - Clock as data for unstable cells
- C23 - State element connected to unstable cell clock input
- C24 - Nonclock PI connected to unstable cell clock input
- C25 - Unstable cell clock input connected from multiple sources
- C26 - Clock as data different from capture clock for stable cell
- C27 - Missing PLL conditioning data
- C28 - Invalid PLL source for internal clock
- C29 - Undefined PLL source for internal clock
- C30 - Scan PLL conditioning affected by nonscancells
- C31 - Scan PLL conditioning not stable during capture
- C34 - Unsensitized path between PLL source and internal clock
- C35 - Multiple sensitizations between PLL source and internal clock
- C36 - Mistimed sensitizations between PLL source and internal clock
- C37 - Cannot satisfy all internal clocks off for all cycles
- C38 - Bad off-conditioning between PLL source and internal clock
- C39 - Nonlogical clock C connects to scancell T
- C40 - Internal clock is restricted
- C41 - Equivalent clocks are not groupable
- C43 - Scan enable does not facilitate capture after N delay cycles.
- C44 - Scan enable facilitates capture at N1 delay cycles but expected after N2 delay cycles.

DRC Rule C1

Message Text

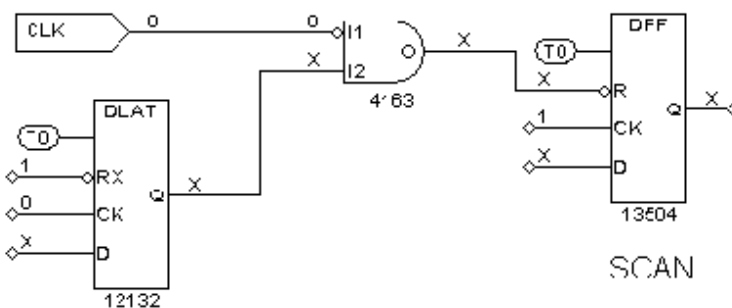
Clock Rule C1: Clock PIs off failed to force off clock input N of scan S I (G) .

Severity

Error

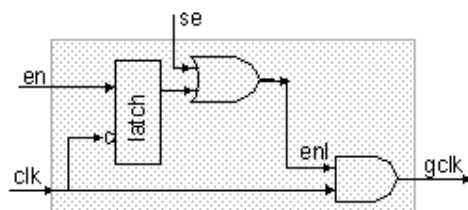
Examples

Example 1: Scan chain with a master-slave configuration



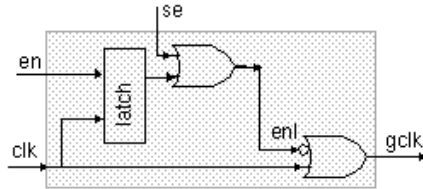
Example 2, below, displays an integrated cell using a latch-based gating style, rising-edge-triggered. The integrated cell contains test logic scan enable.

Example 2: Rising-Edge Latch-Based Integrated Cell With Post-Control (latch_posedge_postcontrol)



Example 3, below, displays an integrated cell using a latch-based gating style, falling-edge-triggered. The integrated cell contains test logic scan enable.

Example 3: Falling-Edge Latch-Based Integrated Cell With Post-Control (latch_negedge_postcontrol)



When all clocks ports are at their off state, all clock/set/reset inputs of scan state elements must be at their off state. The only exception to this would be if these inputs were causing a capture of data from within the scan cell in which they reside such as when a slave latch captures data from its master latch.

N is the input port number of the ATPG primitive (set=0, rst=1, clk1=2, data1=3...), S is the name of the scan chain, I is the instance pathname, and G is the gate ID number of the device.

Failure to satisfy this rule can result in scan cells failing to hold their value before capture, and there is a high risk that patterns generated would fail in simulation. Scan based simulation continues to assume that the values captured during a capture clock cycle are the result of state element values that existed at the end of the load operation. This assumption might be false if scan cells fail to hold their value and lead to patterns generated that fail simulation.

This check is performed by simulating the logic values that result when:

- clocks are at their off state

- constrained ports are set to their constrained values

- constant value nonscan cells are set to their constant state

nonscan cells which have constant values are determined automatically during circuit learning.

This rule violation occurs if any clock/set/reset input of any scan cell memory element is not at its off state.

What Next

A common cause of this violation is a missing clock definition or a clock defined with the wrong polarity.

Another common cause is a clock which passes through a MUX, and the select line of the MUX is not constant. If the MUX is controlled by some sort of test mode port, that port should be constrained with an `add_pi_constraint` command.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to clock-off data and displays the failing cell with all the gates in the backtrace cone of the failing input. This shows the failing clock/set/reset input and tracing back from this input can assist you in determining how to correct the problem.

DRC Rule C2

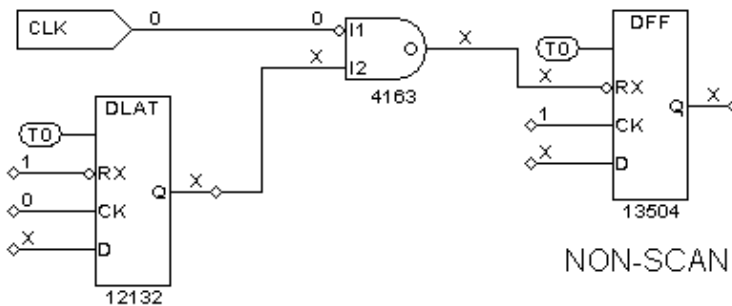
Message Text

Clock Rule C2: Clock PIs off failed to force off clock input N of nonscan DFF I (G).

Severity

Warning

One example, many variations are possible:



Description

When all clocks are at their off state, all clock/set/reset inputs of `nonscan` DFF primitives must be at their off state.

N is the input port number of the ATPG primitive (set=0, rst=1, clk1=2, data1=3...), I is the instance pathname, and G is the gate ID number of the device.

Failure to satisfy this rule results in the DFFs not being usable for Fast-Sequential ATPG algorithms. This is not harmful but can contribute to a lower test coverage.

This check is performed by simulating the logic values that result when:

- Clocks are at their off state
- Constant value nonscan cells are set to their constant state

- Constrained ports are set to their constrained values
- Constant value nonscan cells are set to their constant state

Nonscan cells that have constant values are determined automatically during circuit learning.

This rule violation occurs if any clock/set/reset input of any nonscan DFF is not at its off state.

What Next

A common cause of this violation is a missing clock definition or a clock defined with the wrong polarity.

Another common cause is a clock which passes through a MUX, and the select line of the MUX is not constant. If the MUX is controlled by some sort of test mode port, that port should be constrained with an `add_pi_constraint` command.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to clock-off data and displays the failing cell with all the gates in the back trace cone of the failing input. This shows the failing clock/set/reset input and tracing back from this input can assist you in determining how to correct the problem.

Use of the `-mask` option of the `set_rules` command can also be used to mask the C2 rule DRC violation. This allows use of `set_simulation -capture_mask_always_x` to mask nonscan cells with an active set/reset that also have a C2 violation when using threaded ATPG. The option `set_simulation -capture_mask_always_x` is not supported with non-threaded ATPG.

DRC Rule C3

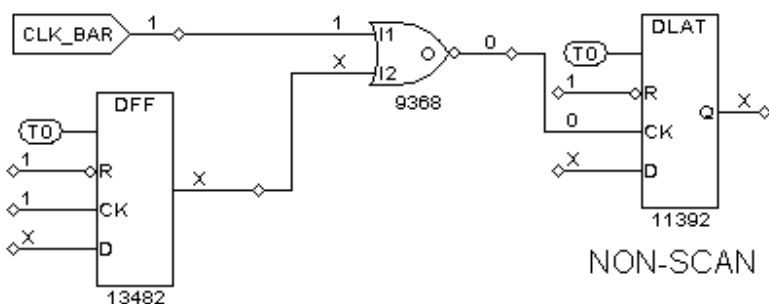
Message Text

Clock Rule C3: Clock PIs off failed to allow transparency of nonscan DLAT I (G) .

Severity

Warning

One example, many variations are possible:



Description

When all defined clocks are at their off state, at least one clock input of a nonscan DLAT (latch primitive) must be capable of being on.

I is the instance pathname, and **G** is the gate ID number of the device.

Failure to satisfy this rule means the DLAT primitive is not usable as a transparent latch. The Basic-Scan ATPG algorithm treats the DLAT as if it were a TIEX device and the coverage that uses this algorithm might be degraded. The Fast-Sequential or Full-Sequential algorithm might be required to increase coverage.

This check is performed by simulating the logic values that result when:

clocks are at their off state

constrained ports are set to their constrained values

constant value nonscan cells are set to their constant state

nonscan cells which have constant values are determined automatically during circuit learning.

The rule violation occurs if all clock/set/reset inputs of a nonscan latch are at their off state. DLATs which fail this rule are treated as if their outputs are at X.

What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This changes the pin data reporting to clock-off data and displays the failing cell with all the gates in the backtrace cone of the failing input. This shows the failing clock/set/reset input and tracing back from this input can assist you in determining why the latch cannot be placed in a transparent mode.

Enabling Fast-Sequential or Full-Sequential ATPG pattern generation might help to reduce test coverage loss from latches that cannot be made transparent.

DRC Rule C4

Message Text

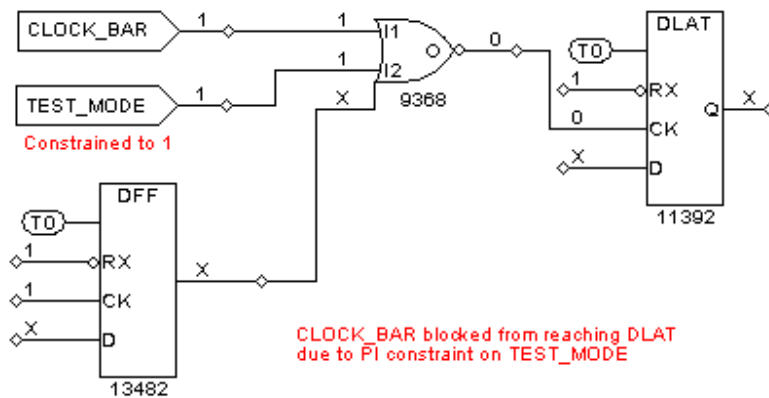
Clock Rule C4: Clock *port_name* cannot capture data with other clocks off.

Clock Rule C4: Multiple clocks or a single clock through multiple paths drives scan-cell *instance_name* during shift.

Severity

Error

One example, many variations are possible:



Description

There are two different variations of the C4 rule violation message.

The first variation of this rule violation message occurs if all clock/set/reset/write inputs of all sequential elements (including RAMs) are off during this condition. In this case, each defined clock port must turn on a clock, set, reset, or write_control input of at least one sequential device or memory element when all other clocks are off. Failure to satisfy this rule indicates a defined clock cannot perform a capture operation which can result in some loss of test coverage.

This check is performed by simulating the logic values that result when:

the specified clock (and any equivalent ports) are set to X

all other defined clocks are set to their off states

constrained ports are set to their constrained values

constant value nonscan cells are set to their constant state

The second variation of the C4 rule violation message occurs when the `set_drc -check_multiple_shift_clocks` command is specified. In this case, a C4 violation is reported for each scan cell whose shift clock comes from multiple clocks that are active during shift, or from a single clock which fans out through multiple paths that reconverge.

What Next

The explanation for the first variation of C4 rule violation message (Clock `port_name` cannot capture data with other clocks off.) is as follows:

The most common cause of the first variation of this violation is defining a clock that does not behave as a clock. Removing the invalid clock definition corrects this problem.

Another common cause of the first variation of this violation is circuitry blockage that is in the way of a clock pulse reaching the internal sequential device or memory element. This can occur when a clock passes through an AND gate and the control side is 0, or an OR gate with a control side of 1.

Another possibility when the instance is a nonscan device is that there is another pin on the device such as an asynchronous set or reset that is controlled from a primary input that has not been defined as a clock. For example, a CLK pin has been defined as a clock but a RESETB pin has not and so pulsing the CLK pin cannot capture data while RESETB=X. This can be corrected by defining the missing clock.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to constrain data and displays the failing clock with gates along a path to a potential capture cell.

You might also want to switch the display to Clock Off data for a different perspective of the problem.

After reviewing the cause of the violation you might want to continue by setting the severity of the C4 rule to a warning using the `set_rules` command, and then re-issuing the `run_drc` command.

There are certain signals identified as clocks that also generate C4 violations. These types of clock signals are not connected to clock inputs of sequential elements; they are only connected to data inputs. To review these clocks, use the `report_clocks-verbose` command, and determine if a clock uses any sequential elements. You should not define signals without clock usage as clocks; this affects the usage of these signals. If the design context requires these signals to be identified as clocks even though they have no clock usage, you can use the `set_rulesC4 -autofix` command to remove violations for these types of signals. This option does not remove the blocked clocks from generating C4 violations.

The explanation for the second variation of the C4 rule violation message (Multiple clocks or a single clock through multiple paths drives scan-cell *instance_name* during shift.) is as follows:

This variation of the violation is reported by the `set_drc -check_multiple_shift_clocks` command and is normally ignored. But it can cause problems in static timing analysis or simulations with full delays. Fixing it requires redesign or reconfiguration of the clock generation circuitry of the affected scan cells.

DRC Rule C5

Message Text

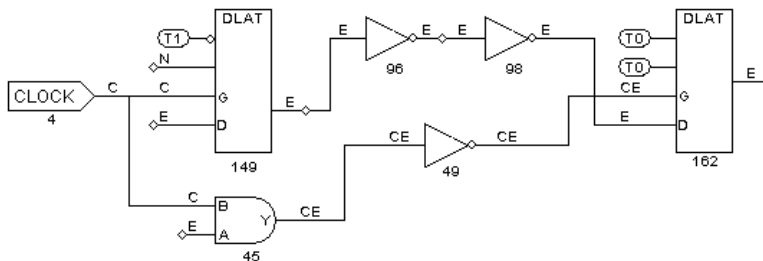
Clock Rule C5: Clock C can capture new data on LS input N of <type> I1 (G1).

Source of violation: input N of <type> I2 (G2)

Severity

Warning

One example, many variations are possible:



Description

A clock must not capture data into a level sensitive (LS) port (latch or RAM) if that data might be affected by new captured data.

C is the clock port; the <type> is either DLAT or DFF; I1 and I2 are instance pathnames; G1 and G2 are gate IDs. Input N refers to the ATPG gate's inputs following the scheme 0=set, 1=reset, 2=clk1, 3=data1, 4=clk2, 5=data2...

There is a risk that test generation will not correctly control the new value that is captured, which will result in a loss of test coverage. In this case, use the Fast-Sequential pattern operator and the command `set_atpg -lete_fastseq`.

A list of all potential source ports for the violation is identified by tracing back from the sink data line through gates in the effect cone. To be considered a violation, the source port must not be a trailing edge (TE) port, the source port clock must be capable of being at an on state when the sink port clock is at an on state, and the path from source to sink must be propagatable under this condition.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This changes the pin data reporting to clock cone data and displays all gates in the paths from the source port to the sink port, the clock to the source port, and the clock to the sink port.

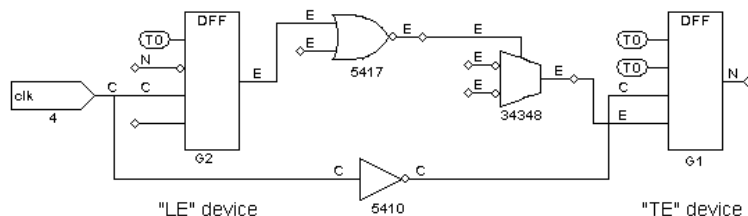
Use of the `-mask` option of the `set_rules` command can also be used to compensate for the DRC violation. This causes the ATPG algorithm to mask off the observe value in an attempt to ensure that any ATPG pattern generated will pass simulation. The tradeoff is that in doing this, there is a reduction of test coverage.

Message Text

Source of violation: input N of <type> I2 (G2).

Warning

One example, many variations are possible:



Description

A clock must not capture data into a trailing edge (TE) port if that data might be affected by new captured data. In the violation message,

`C` is the clock port; the `<type>` is either DLAT or DFF; `I1` and `I2` are instance pathnames; `G1` and `G2` are gate IDs.

There is a risk that test generation does not correctly control the new value that is captured. This results in some loss of test coverage and in some cases simulation mismatches. In such a scenario, proper masking can be applied using `set_rules C6 -mask` to avoid any mismatches.

This check is performed using clock cone data and a potential rule violation occurs on a clock if a clock input of a TE port is in the clock cone and its data input (including RAM address) is in the effect cone. This port is called the sink port of a violation. To be considered a violation, the source port must not be a TE port, the source port clock must be capable of being at 1 (0 if LE) when the sink port clock is at 1, and the path from source to sink must be propagatable under this condition.

Note that C6 violations are not issued on DSLAVE cells. This is because ATPG uses these cells only for controllability and does not observe data from them during scan chain unload.

What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to clock cone data and displays all gates in the paths from the source port to the sink port, the clock to the source port, and the clock to the sink port.

A list of all potential source ports for the violation is identified by tracing back from the sink clock line through gates in both the clock and its effect cone.

DRC Rule C7

Message Text

Clock Rule C7: Clock `C` can capture new data on LE input `I1` of `T1 N1` (`G1`).
Source of violation: input `I2` of `T2 N2` (`G2`).

Severity

Ignored

Description

A clock must not capture data into a leading edge (LE) port if that data might be affected by the newly captured data.

Failure to satisfy this rule is not likely to result in a race condition. However, if a race condition occurs this can lead to inaccurate simulation results and the creation of ATPG patterns which fail simulation. Scan based simulation assumes that the values captured during a capture clock cycle are the result of state element values that existed at the end of the load operation.

This check is performed using clock cone data and a potential rule violation occurs on a clock if a clock input of a LE port is in the clock cone and its data input (including RAM address) is in the effect cone. This port is called the sink port of a violation.

A list of all potential source ports for the violation is identified by tracing back from the sink data line through gates in the effect cone. To be considered a violation, the source port must be an LS port, the source port clock must be capable of being at 1 when the sink port clock is at 1, and the path from source to sink must be propagatable under this condition.

What Next

You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the clock cone data and displays all gates in the paths from the source port to the sink port, the clock to the source port, and the clock to the sink port.

Use the `-mask` option of the `set_rules` command to compensate for the DRC violation. By setting the rule severity to warning and enabling the `-mask` option, this causes the ATPG algorithm to mask off the control or observe functions, or both, in an attempt to ensure that any ATPG pattern generated will pass simulation. The tradeoff is that in doing this, there is a reduction of test coverage.

DRC Rule C8

Message Text

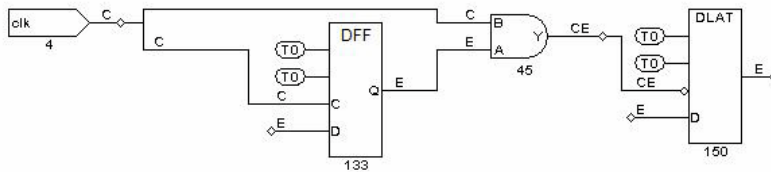
Clock Rule C8: C clock path affected by new capture on LS input N of <type> I1 (G1).

Source of violation: input N of <type> I2 (G2).

Severity

Warning

One example, many variations are possible:



Description

The path from a clock to a level sensitive (LS) port must not be affected by its new captured data.

`C` is the clock port; the `<type>` is either DLAT or DFF; `I1` and `I2` are instance pathnames; `G1` and `G2` are gate ID's.

Failure to satisfy this rule can result in a race condition which can create inaccurate ATPG simulation results and lead to patterns that fail simulation. Scan based simulation continues to assume that the values captured during a capture clock cycle are the result of state element values that existed at the end of the load operation. This assumption could be false in the presence of a race condition and produce patterns that fail simulation.

This check is performed using clock cone data and a potential rule violation occurs on a clock if a clock input of a level sensitive port is in both the clock cone and effect cone. This port is called the sink port of a violation. To be considered a violation, the source port must not be a TE port, the source port clock must be capable of being at 1 when the sink port clock is at 1, and the path from source to sink must be propagatable under this condition.

What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to clock cone data and displays all gates in the paths from the source port to the sink port, the clock to the source port, and the clock to the sink port.

A list of all potential source ports for the violation is identified by tracing back from the sink clock line through gates in both the clock and its effect cone.

Use the `-mask` option of the `set_rules` command to compensate for the DRC violation. By setting the rule severity to warning and enabling the `-mask` option, this causes the ATPG algorithm to mask off the observe value, in an attempt to ensure that any ATPG pattern generated will pass simulation. The tradeoff is a reduction of test coverage.

A list of all potential source ports for the violation is identified by tracing back from the sink clock line through gates in both the clock and its effect cone.

DRC Rule C10

Message Text

Clock Rule C10: C clock path affected by new capture on LE input N1 of T1 I1 (G1).

Source of violation: input N2 of T2 I2 (G2).

Severity

Ignored

Description

The path from a clock to a leading edge (LE) port must not be affected by its new captured data.

C is the clock port name; N1/N2 are the input numbers of the ATPG primitive (set=0, rst=1, clk1=2, data1=3...); T1/T2 are the primitive types of either DLAT or DFF; I1/I2 are the instance pathnames containing the primitive; G1/G2 are the gate ID's of the primitives;

Failure to satisfy this rule is not likely to result in a race condition. If there is a race condition, then inaccurate simulation results would be possible which would in turn lead to generation of ATPG patterns which can fail in simulation.

This check is performed using clock cone data, and a potential rule violation occurs on a clock if a clock input of a LE port is in both the clock cone and effect cone. This port is called the sink port of a violation. To be considered a violation, the source port must be a level sensitive (LS) port, the source port clock must be capable of being at 1 when the sink port clock is at 1, and the path from source to sink must be propagatable under this condition.

What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to the clock cone data and displays all gates in the paths from the source port to the sink port, the clock to the source port, and the clock to the sink port.

A list of all potential source ports for the violation is identified by tracing back from the sink clock line through gates in both the clock and its effect cone.

Use of the `-mask` option of the `set_rules` command to compensate for the DRC violation. By setting the rule severity to warning and enabling the mask option, this causes the ATPG algorithm to mask off the control or observe functions, or both, in an attempt to ensure that

any ATPG pattern generated will pass simulation. The tradeoff is that in doing this, there is a reduction of test coverage.

DRC Rule C11

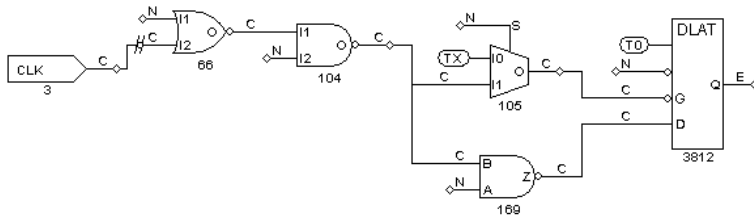
Message Text

Clock Rule C11: Clock C connects to LS clock/data inputs N1/N2 of I (G).

Severity

Warning

One example, many variations are possible:



Description

A clock has a combinational path to both the data and clock ports of a level sensitive (LS) device.

C is the clock port name; N1/N2 are the input numbers of the ATPG primitive (set=0, rst=1, clk1=2, data1=3...); I is the instance pathname containing the primitive; G is the gate ID of the primitive;

A clock must not capture data for a level sensitive (LS) port whose data line is affected by the clock.

Failure to satisfy this rule can result in a race condition, which can create inaccurate simulation results. Simulation is performed assuming a captured data value for the clock at an on state. If this assumption is not correct, patterns can fail simulation.

This check is performed by using clock cone data, and a rule violation occurs on a clock when a clock input of a level sensitive clock line and its data line are in the same clock cone.

What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This changes the pin data reporting to the clock cone data and displays all gates in the paths from the clock to both the clock input and data input of the failing port.

If the sequential element involved in the DRC violation is part of a scan chain, you might be able to define a Cell Constraint of OX (observe X) for this cell and reduce the possibility of generating ATPG patterns that fail simulation. This however, will result in a lowering of test coverage.

Use the `-mask` option of the `set_rules` command to compensate for the DRC violation. By setting the rule severity to warning and enabling the `-mask` option, this causes the ATPG algorithm to mask off the control or observe functions, or both, in an attempt to ensure that any ATPG pattern generated will pass simulation. The tradeoff is that in doing this, there is a reduction of test coverage.

DRC Rule C12

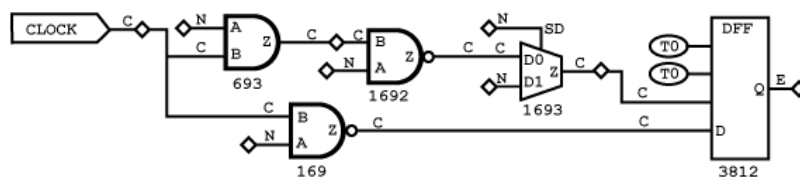
Message Text

Clock Rule C12: Clock C connects to LE clock/data inputs N1/N2 of I (G).

Severity

Warning

One example, many variations are possible:



Description

This check is performed using clock cone data. A violation occurs if a common path (a combinational gate) from a defined clock affects both the data and clock ports of a leading edge (LE) triggered sequential device.

The `C` in the diagram is the clock port name; `N1/N2` are the input numbers of the ATPG primitive (`set=0, rst=1, clk1=2, data1=3...`); `I` is the instance pathname containing the primitive; `G` is the gate ID number of the primitive.

This violation identifies a potential race condition between the changing data input and the clock capturing that data. TestMAX ATPG cannot determine the actual timing between the two paths, so it is not possible to determine which path is faster. By default, TestMAX ATPG assumes the clock path is faster for an LE device. When this assumption is incorrect, it is likely that any patterns generated are incorrect and will fail in simulation.

What Next

You can analyze this violation by clicking the ANALYZE button in the schematic viewer and selecting the violation ID. This changes the pin data reporting to clock cone data and

displays all gates in the paths from the clock to both the clock input and data input of the failing sequential device.

After reviewing the design gates and paths, you can fix the violation by doing one or more of the following:

C12 violations are masked by default, and ATPG and simulation do not mask every capture cycle. Determine if the cycle data remains stable and mask only unstable cycles. Also, if the data change occurs when another register is clocked, the effects of that change are considered to take place after the capture. In this case, do not mask any cycles.

A race can be caused by clock skew. In this case, use a Synopsys Design Constraints (SDC) hold-time exception to mask the capture. For more information on SDC, see Specifying Timing Exceptions from an SDC File.

Also note the following:

Only scan registers with C12 violations are masked. If you need to mask a nonscan register, use the `add_capture_masks` command.

You can use the `set_rulesc12 -nomask` command if coverage is too low. In this case, the flip-flop captures the old value in both the Basic and Fast-Sequential engines.

You can block the path from the clock to the data input as it passes through combinational logic by applying a constraint to a net connecting to the input of the gate. For example, holding the input of an AND gate to zero will block the path of the clock through the data input gate.

You can change your design so the race path is not in ATPG test mode.

DRC Rule C13

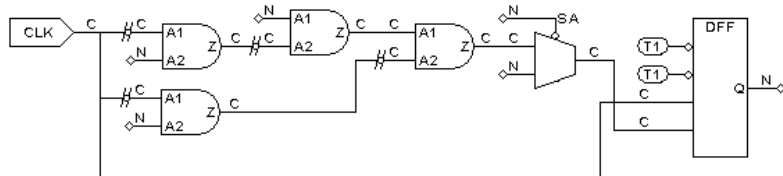
Message Text

Clock Rule C13: Clock C connects to TE clock/data inputs N1/N2 of I (G).

Severity

Warning

One example, many variations are possible:



Description

A common path (combinational gates) exists from a defined clock to both the data and clock ports of a trailing edge (TE) triggered sequential device.

`C` is the name of the clock, `N1/N2` are the input port numbers of the ATPG primitive (`set=0`, `rst=1`, `clk1=2`, `data1=3...`), `I` is the instance pathname to the device, and `G` is the gate ID number of the device.

Failure to pass this rule check indicates there is a potential race condition between the data input changing and the clock capturing that data. The ATPG tool does not have knowledge of the actual timing between the two paths so it is not possible to know which will win the race. By default, the ATPG tool assumes the clock path is faster for a TE device. In other words, the value captured is the value which results at the data input while the clock is asserted. When this assumption is wrong, it is highly likely that any patterns generated is wrong and will fail in simulation.

This check is performed by using data.

This rule violation occurs if a clock input and its data input are in the same clock cone of a trailing edge clock.

What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to clock cone data and displays all gates in the paths from the clock to both the clock input and data input of the failing sequential device.

After review of the design gates and paths involved a solution can involve one or more of the following:

If the clock path is indeed the faster path, you can ignore the violation by changing the rule to a warning using the `set_rules` command.

It might be possible to block the path from the clock to the data input as it passes through combinational logic by applying a constraint to a net connecting to the other input of one of these combinational gates. For example, holding the other input of an AND gate to zero will block the path of the clock through that gate.

If ATPG patterns generated fail simulation you might be able to sidestep the problem using the `add_cell_constraint` command to mask any data captured into the scan cell.

A design change might be required so that at least in ATPG test mode, this race path is not present.

Use the `-mask` option of the `set_rules` command to compensate for the DRC violation. By setting the rule severity to warning and enabling the `-mask` option, this causes the ATPG algorithm to mask off the control or observe functions, or both, in an attempt to

ensure that any ATPG pattern generated will pass simulation. The tradeoff is that in doing this, there is a reduction of test coverage.

DRC Rule C14

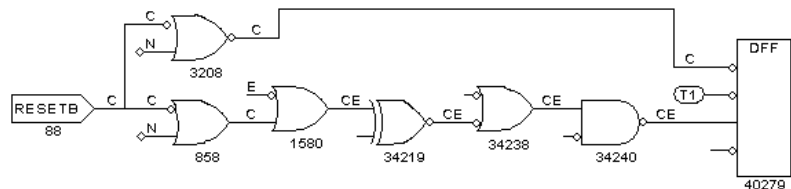
Message Text

Clock Rule C14: Clock C connects to clock/clock inputs N1/N2 of I (G).

Severity

Warning

One example, many variations are



possible:

Description

A common path (combinational gates) exists from a defined clock port to more than one clock input of a sequential device. A clock input includes any asynchronous set or reset control inputs as well as data clocks.

C is the name of the clock, N1/N2 are the input port numbers of the ATPG primitive (set=0, rst=1, clk1=2, data1=3...), I is the instance pathname to the device, and G is the gate ID number of the device.

Failure to pass this rule check indicates there is a potential race condition because a clock port has a combinational gate path to two different clock inputs of the same ATPG sequential primitive. The ATPG tool does not have knowledge of the actual timing between the two paths so it is not possible to know which input will control the stored sequential value. The data captured in the sequential device will come from the data input with the clock at an on state. When this chosen value is incorrect, then simulation failures can result from patterns generated.

This check is performed using clock cone data.

A rule violation occurs on a clock when multiple clock inputs of a state element are in the same clock cone and test generation has identified that it is possible that both clock inputs might be on at the same time.

What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to clock cone data and displays all gates in the paths from the clock to both clock inputs of the failing sequential device.

After review of the design gates and paths involved a solution can involve one or more of the following:

If the clock path is indeed the faster path, you can ignore the violation by changing the rule to a warning using the `set_rules` command.

If ATPG patterns generated fail simulation you might be able to sidestep the problem using the `add_cell_constraint` command to mask any data captured into the scan cell.

A design change might be required so that at least in ATPG test mode, this race path is not present.

Use the `-mask` option of the `set rules` command to compensate for the DRC violation. By setting the rule severity to warning and enabling the `-mask` option, this causes the ATPG algorithm to mask off the control or observe functions, or both, in an attempt to ensure that any ATPG pattern generated will pass simulation. The tradeoff is that in doing this, there is a reduction of test coverage.

DRC Rule C15

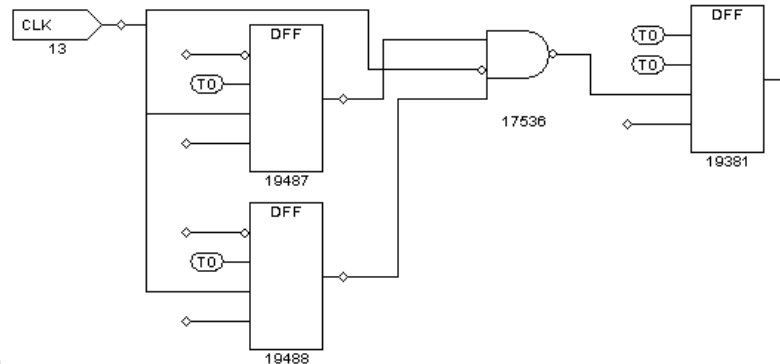
Message Text

Clock Rule C15: Clock input N of scan T I (G) cannot capture data.

Severity

Warning

One example, many variations are



possible:

Description

Each clock input of a scan cell state element must be capable of capturing data when a single clock is on and all other clocks are off. Clock inputs include asynchronous set/reset inputs.

N is the input number of the ATPG primitive (set=0, rst=1, clk1=2, data1=3...); T is a gate type of DFF or DLAT; I is the instance pathname to the device; G is the gate ID number of the device.

This check is performed using the simulated values that result when one clock is set to X, all other clocks are at their off state, the constrained pins are set to their constrained value, and the constant value nonscan cells are set to their constant state. The rule violation occurs when the clock input of a scan cell is off rather than X.

A violation of this rule introduces no danger of generating bad patterns but is a warning that a potential reduction of test coverage can result because the clock input cannot be used.

What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This changes the pin data reporting to "constrain" data and displays the failing cell with all of the gates in the back trace cone of the failing clock input.

DRC Rule C16

Message Text

Clock Rule C16: Clock input N of nonscan T I (G) cannot capture data.

Severity

Warning

Description

Each clock input of a nonscan cell must be capable of capturing data when a single clock is on and all other clocks are off. Clock inputs include asynchronous set/reset inputs.

N is the input number of the ATPG primitive (set=0, rst=1, clk1=2, data1=3...); T is a gate type of DFF or DLAT; I is the instance pathname to the device; G is the gate ID number of the device.

This check is performed using the simulated values that result when one clock is set to X, all other clocks are at their off state, the constrained pins are set to their constrained value, and the constant value nonscan cells are set to their constant state. The rule violation occurs when a clock input of a scan cell always remains off.

A violation of this rule introduces no danger of generating bad patterns but is a warning that a potential reduction of test coverage can result because the clock/set/reset input cannot be used when performing Fast-Sequential ATPG.

What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to "constrain" data and displays the failing cell with all of the gates in the back trace cone of the failing clock input.

DRC Rule C17

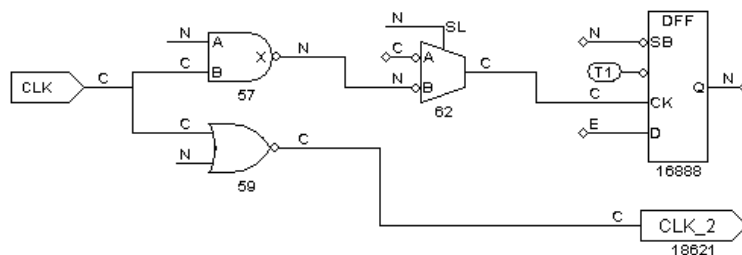
Message Text

Clock Rule C17: Clock C is connected to primary output P.

Severity

Warning

One example, many variations are



possible:

Description

A clock should not have a path to a primary output consisting only of combinational gates. This type of connection is referred to as a clock PO output. Faults along this path which require the clock to be asserted for detection require a special type of pattern called a "Clock ON" pattern which is, by default, not produced by TestMAX ATPG. However, patterns are generated for faults that can be detected with the clock off.

C is the clock port name; P is the primary output port name.

This check is performed by using the clock cone data, and a violation occurs when a primary output (PO) is in a clock cone.

A violation of this rule introduces no danger of generating bad patterns but is a warning that there might be a reduction in test coverage. The ATPG algorithm simulates an additional time frame, which occurs before the capture clock. This is used to determine the correct values for the primary outputs identified.

What Next

The generation of Clock ON patterns is enabled by using the `-allow_clockon_measures` of the `set_atpg` command and by supplying appropriate procedures in the DRC procedure file. Please Note that not all testers can support this type of pattern.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to "clock cone" data and displays all gates in the path from the clock to the failing primary output (PO).

One potential cause of this violation, in addition to the direct combinational path shown above, is a bidirectional port used as a clock. You can ignore C17's occurring on bidirectional clock ports.

See also Supporting Clock ON Patterns in STIL

DRC Rule C18

Message Text

Clock Rule C18: Path from clock C to PO P affected by new captured data.

Severity

Ignored

Description

A path from a clock to a primary output (PO) must not be affected by new data values captured by the clock.

C is the clock port name; P is the primary output port name.

Failure to satisfy this rule has no effect on current functionality.

This check is performed by using the clock cone data, and a violation occurs when a primary output is in both a clock and effect cone.

What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the [ANALYZE](#) button. This sets the pin data reporting to the clock cone data and displays all gates in the path from the clock to the affected primary output (PO).

DRC Rule C19

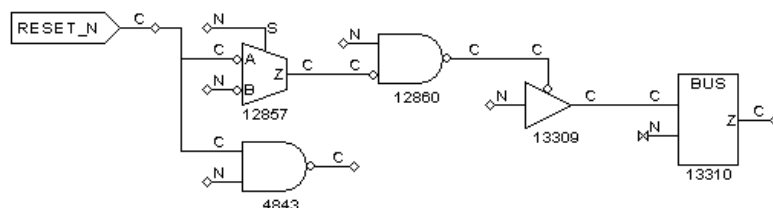
Message Text

Clock Rule C19: Clock C is connected to at least one non-contention-free BUS (G).

Severity

Warning

One example, many variations are possible:



Description

A clock must not have a combinational path to a BUS gate if that BUS gate is not contention free. A BUS gate is inserted into the flattened design for all multi driver nets and acts as a resolution point for the final driven value on this net. `C` is the clock port name; `G` is the gate ID of the BUS gate.

Failure to satisfy this rule creates a risk that bus contention can occur during precapture clock time. This contention condition will result in the pattern being discarded and other patterns is attempted which will increase pattern generation time. Finding a pattern which can avoid contention cannot be possible which will result in a lower test coverage.

This check is performed by using the clock cone data and a violation occurs when a BUS gate is in a clock cone.

What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This sets the pin data reporting to the clock cone data and displays all gates in the path from the clock to the failing BUS.

As a workaround, you might want to disable contention checking using the `set_contention` command. However, any patterns generated can contain contention and can either fail in simulation or cause stress or device failure when used on the actual silicon, or both.

DRC Rule C20

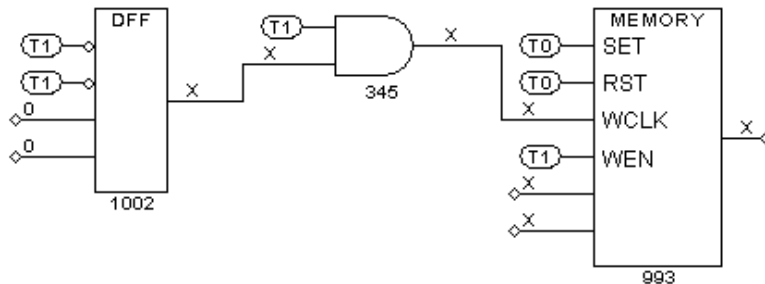
Message Text

Clock Rule C20: Clock PIs off failed to force off clock input N of RAM I (G) .

Severity

Warning

One example, many variations are possible:



Description

When all clocks are at their off state, all clock/set/reset inputs of a MEMORY gate must be in their inactive state.

N is an input starting with 0 at the top and counting down; I is an instance pathname to the MEMORY; G is the gate ID of the MEMORY primitive.

Failure to satisfy this rule will result in the MEMORY gate being the equivalent of a TIEX gate for Basic-Scan and Fast-Sequential pattern generation. A reduction in test coverage can occur but can be recovered by using Full-Sequential pattern generation.

This check is performed using the simulated values that result when clocks are at their off state, the constrained pins are set to their constrained value, and the constant value nonscan cells are set to their constant state. The rule violation occurs if any clock/set/reset input of a MEMORY gate is not at its inactive state.

What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This sets the pin data reporting to the simulation values of the clock-off pattern and displays the failing MEMORY gate with all the gates in the backtrace cone of the failing input. This identifies the failing clock input, and tracing back from this input can help you identify how to correct the problem.

A common cause of this violation is having a primary input which controls MEMORY reads or writes which is not defined as a clock.

If this violation cannot be corrected and the test coverage loss around the memory device is not acceptable, you might wish to explore the use of Full-Sequential pattern generation to attempt to create patterns which make use of the memory, despite this violation.

DRC Rule C21

Message Text

Clock Rule C21: Write input `N` of RAM `I (G)` cannot capture data.

Severity

Warning

Description

Each write control of a RAM must be capable of capturing data when a single clock is on and all other clocks are off.

`N` is an input of the MEMORY primitive starting with 0 at the top and counting down; `I` is an instance pathname to the MEMORY; `G` is the gate ID of the MEMORY primitive.

Failure to satisfy this rule does not cause any danger of generating patterns which fail in simulation but it does indicate the potential for a lower test coverage because the write port cannot be used.

This check is performed using the simulated values that result when one clock is set to X, all other clocks are at their off state, the constrained pins are set to their constrained values, and the constant value nonscan cells are set to their constant state. The rule violation occurs when the write input of a RAM is always off rather than X.

What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This sets the pin data reporting to the "constrain" data and displays the failing MEMORY primitive with all of the gates in the back trace cone of the failing write control input.

You might also wish to switch the viewed data to the `clock off` values in the schematic viewer.

A common cause of this violation is failure to declare the write control input a clock, or not having a top level primary input control for the RAM write control in ATPG test mode.

DRC Rule C22

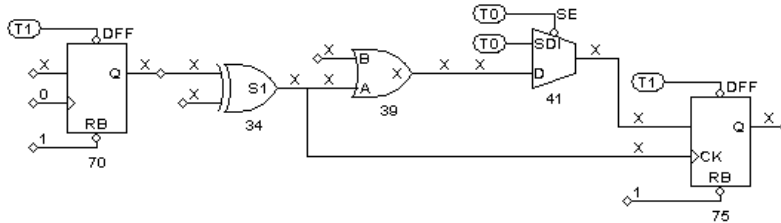
Message Text

Clock Rule C22: Clock as data connection between `G1` and inputs `N1/N2` of unstable `T (G2)`.

Severity

Warning

One example, many variations are possible:



Description

The clock input of an unstable nonscan DFF or DLAT was found to have a common connection with its data input and the source of this common connection is a state element or a primary input not declared as a clock.

G1 is the gate ID of the gate that is the common source of the connection; N1 is the input number of the clock input of the failing device; N2 is the input number of the data input of the failing device; T is the gate type (DLAT or DFF) of the failing device; G2 is the gate ID of the failing device.

Failure to satisfy this rule will hinder the ability to detect faults in nonscan areas using the Fast-Sequential or Full-Sequential ATPG pattern capability. This will reduce test coverage.

Failure to satisfy this rule indicates there is a potential race condition between the data input changing and the clock capturing data. Basic-scan and Fast-Sequential ATPG algorithms will treat unstable cells as an X source, so the danger of generating patterns which mismatch in simulation will exist only if Full-Sequential patterns are being generated.

What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button and changing the pin data displayed to "none". This will draw the trace back from the failing nonscan cell and show the local clock source common to both inputs.

If you are generating Full-Sequential patterns, then it is suggested that you use the `-mask` option of the `set_rules` command to compensate for the DRC violation. By selecting the `-mask` option you cause the ATPG algorithm to mask off the control or observe functions, or both, in an attempt to ensure that any ATPG pattern generated will pass simulation. The tradeoff is that in doing this, there is a reduction of test coverage.

DRC Rule C23

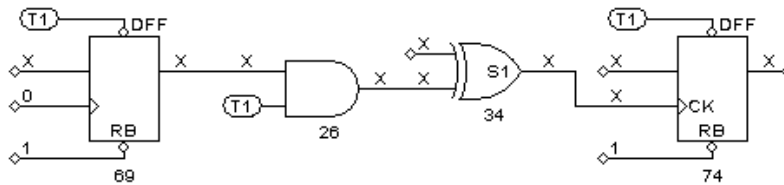
Message Text

Clock Rule C23: I1 (G1) connected to input N of unstable I2 (G2).

Severity

Ignored

One example, many variations are possible:



Description

The clock input of a state element (DFF/DLAT/RAM) has an unblocked path from an unstable cell (DFF/DLAT/RAM).

I1 is an instance pathname to the storage gate that connects to the clock input; G1 is its corresponding gate ID; N is the input of the unstable ATPG primitive starting with 0 at the top and counting down; I2 is an instance pathname to the unstable cell; G2 is its corresponding gate ID;

A violation of this rule indicates that the failing device drives an internally generated clock. Basic-scan and Fast-Sequential ATPG ignore the behavior of unstable cells driven by this internally generated clock, and unless the circuit is modified, Full-Sequential ATPG is required to improve test coverage around unstable cells. This will result in additional ATPG effort and simulation run time, but there is usually no additional risk of simulation mismatches.

This check is performed by identifying storage gate effects for clock inputs of unstable nonscan cells.

What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This sets the pin data reporting to the "none" and displays all gates in the path between the storage gate and the failing unstable nonscan cell.

DRC Rule C24

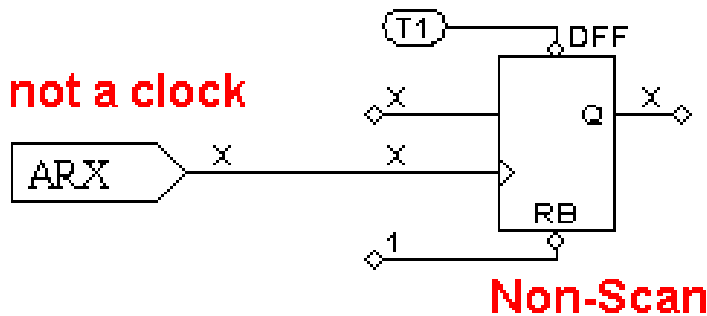
Message Text

Clock Rule C24: Nonclock PI connected to unstable cell clock input

Severity

Ignored

One example, many variations are possible:



Description

The clock input of an unstable state element (DFF/DLAT/RAM) was found to have an unblocked path from primary input which was not declared as a clock. This violation occurs if you neglect to declare a primary input to be a clock when the signal is used to clock an unstable cell.

What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This sets the pin data reporting to the "clock off" and displays all gates in the path between the primary input and the clock input pin of the unstable state element.

After identifying which Primary Input has a path to the state element, return to DRC mode and define the PI as a clock using the `add_clocks` command.

DRC Rule C25

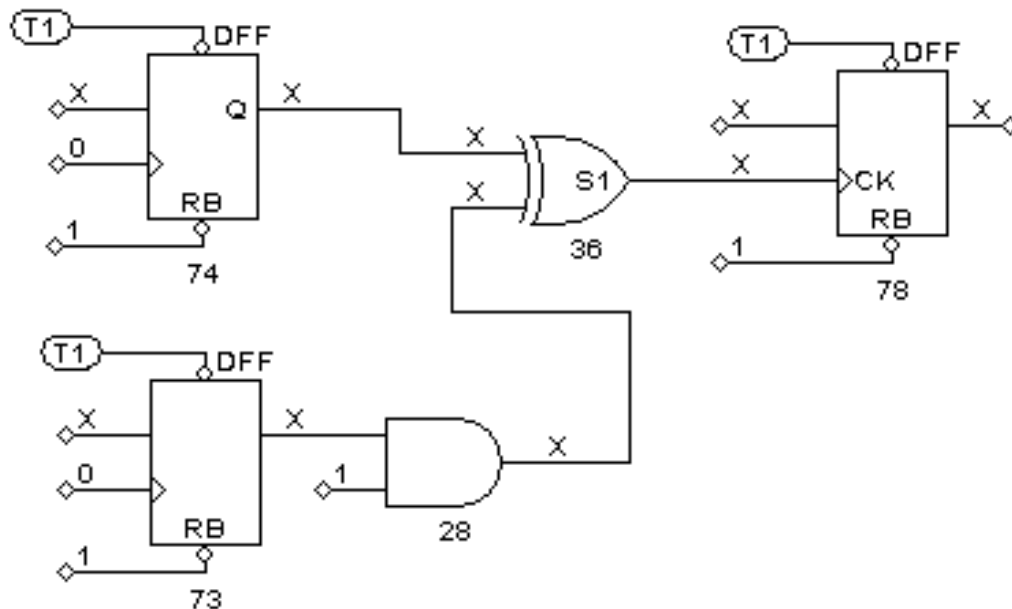
Message Text

Clock Rule C25: Unstable cell clock input connected from multiple sources.

Severity

Warning

One example, many variations are possible:



Description

The clock/set/reset input of an unstable cell (DFF/DLAT/RAM) was found to have an unblocked path from multiple sources which were either state elements or primary inputs not declared as clocks.

For example, the clock input might be the logical OR of two different state elements using the same clock polarity. The inputs to the gating must transition in the same simulation frame, so the combinations LE-TE, PI-LE, PI-TE, and PI-LS are allowed and do not result in C25 violations.

Failure to satisfy this rule indicates there is a potential for a clock glitch to occur. Latches that cause a C25 violation are considered to be X sources immediately after shift only. So

if the latch had a value stored in it during shift, this value would be lost. However, since a latch has the ability to become transparent during ATPG, non-X data can pass through the latch and be stored in it. This will override the initial X following the scan chain load and, in some cases, might allow TestMAX ATPG to test faults through the latch.

What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This sets the pin data reporting to the "clock off" and displays the unstable cell gate and the gates in the backtrace cone of the appropriate clock input.

The clock generation circuitry should be modified to block one of the paths in test mode, or to ensure in some other way that a glitch is impossible.

Use of the `-mask` option of the `set_rules` command can also be used to compensate for the DRC violation. By setting the rule severity to warning and enabling the `-mask` option, this causes the ATPG to mask off the control or observe functions, or both, to ensure that any ATPG pattern generated will pass simulation. The trade-off is that in doing this, there is a reduction of test coverage.

The command `set_rule C25 -mask` is considered only for clock ports and not for set or reset lines. Use the `set_simulation -capture_mask_always_x` command to enable the ability to mask active set/reset values on flops with a C25 and not just the "capture clock" values.

DRC Rule C26

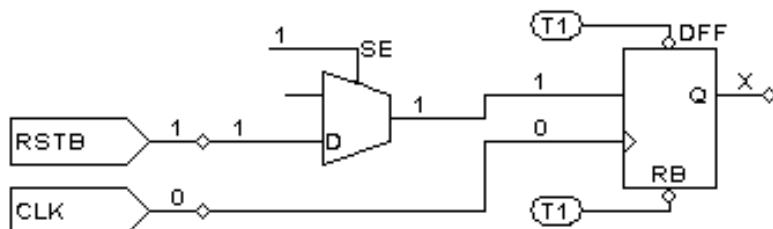
Message Text

Clock Rule C26: clock as data different from capture clock for stable cell.

Severity

Warning

One example, many variations are possible:



Description

The data input of a stable cell (DFF/DLAT/RAM) was found to have an unblocked path to a primary input declared as a clock, and this is a different clock than the one connected to the clock input of the stable cell. Violations of this rule do not create a danger of simulation mismatches, but can indicate a reduction in test coverage.

What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This sets the pin data reporting to the "constrain_data" and displays the gate of the stable cell and the backtrace path to primary input declared as a clock.

You should not define the clock going to the data pin as a clock input. Instead, use the `-allow_unstable` option of the `set_drc` command.

DRC Rule C27

Message Text

Clock Rule C27: Missing PLL conditioning data.

Severity

Error

Description

All internal clocks must have the same number of control clauses, corresponding to consecutive cycles, starting with 0.

What Next

Verify the number of capture cycles, defined by using a `set_drc-num_pll_cycles d` command or by using the ClockStructures block of a Stil Protocol File, and specify control clauses for all capture cycles. See Defining Internal Clocks for PLL Support for the syntax of the ClockStructures block.

DRC Rule C28

Message Text

Clock Rule C28: Invalid PLL source for internal clock.

Severity

Fatal Error

Description

The PLL source declared for an internal clock must be defined as a PLL clock.

What Next

Define the missing PLL clock by using the `-pllclock` option of the `add_clocks` command or by using the ClockStructures block of a Stil Protocol File. See Defining Internal Clocks for PLL Support for the syntax of a ClockStructures block.

DRC Rule C29

Message Text

Clock Rule C29: Undefined PLL source for internal clock. Default
Severity: warning

Severity

Warning

Description

For internal clocks, the PLL source must be specified to benefit from PLL clock rule checking; for example, C34 and so on. When defining internal clocks from the command line, specify the corresponding PLL clock source by using the `add_clocks` command.

Note:

In cases where the missing `pll_source` specifications in the SPF lead to C29 violations, it is recommended that the default setting of `set_simulation` be changed to `set_simulation -nosource_disturb` to prevent the patterns from failing in simulation. The missing accurate `pll_source` specification could lead to optimistic X masking of disturbed clock groups if this switch is not used. Another solution would be to add the missing `pll_source` to resolve the C29 violations and define a unique `pll_source` for each OCC defined in the SPF. This adds proper masking and still provides the benefit of using `source_disturb` simulation.

What Next

Use the `-pll_source` option of the `add_clocks` command to specify the missing PLL clock. Alternatively, this information can also be provided in the ClockStructures block of the Stil Protocol File. See Defining Internal Clocks for PLL Support for the syntax of a ClockStructures block.

DRC Rule C30

Message Text

Clock Rule C30: Scan PLL conditioning affected by nonscancells.

Severity

Error

Description

For scan-ATPG, internal clocks can only be controlled by scan cells, PIs, and PIOs. PIs and PIOs cannot change after the first "forcePI" in a multi cycle pattern.

What Next

Provide the required PI constraints or design modification that would allow valid conditioning with a scan cell, PI, or PIO.

This can be debugged in the GSV after downgrading C30 to a warning, so that DRC can finish its analysis. Do not attempt to generate patterns with C30 violations because the patterns will be rejected due to bad clock conditioning. After running `analyze_violations C30-1`, the nonscan cell is shown along with the conditioning node and internal clock node specified in the SPF ClockStructures block or the `add_clocks` command. Verify the behavior of the nonscan cell by right-clicking it and selecting Display Gate Info. Its behavior during scan chain operation can be seen by selecting Shift pin data. If the nonscan cell should be on the scan chain, correct the design or protocol so that it is scanned. Otherwise, correct the conditioning node specification, so that it refers to a scan cell.

If the violation is for a PI or a PIO, the `set_delay -nopi_changes` command is required to prevent the value from changing during the capture cycle. In addition, a PIO must be forced to the input direction so it cannot be overridden by output values.

DRC Rule C31

Message Text

Clock Rule C31: Scan PLL conditioning not stable during capture.

Severity

Error

Description

For scan ATPG, the scan cells that control internal clocks must be stable during capture.

What Next

Make the required constraint or design modifications to allow stability of the conditioning data through capture.

To make the conditioning data stable, it needs to either come from a stable PI or from a flop in the clock chain that retains its state during the capture clocks. This can be accomplished by wrapping the Q of these flops back to their respective D inputs.

DRC Rule C34

Message Text

Clock Rule C34: Unsensitized path between PLL source and internal clock.

Severity

Warning

Description

This rule checks that the conditions defined for each cycle of an internal clock must sensitize the path between the corresponding PLL source and the internal clock.

If a clock fails the C34 check, the [C35](#), [C36](#), and [C38](#) checks cannot be performed on the same clock. As a result, a C34 check might mask other violations. The other checks are still performed for clocks that do not fail the C34 check.

Note:

In cases where `pll_source` specifications are repeated in the SPF leading to C34 violations, it is recommended that the default setting of `set_simulation` be changed to `set_simulation -nosource_disturb` to prevent the patterns from failing in simulation. Missing accurate `pll_source` specification could lead to optimistic X masking of disturbed clock groups if this switch is not used. Another solution would be to add unique `pll_source` to resolve the C34 violations for each OCC defined in the SPF. This adds proper masking and still provides the benefit of using `source_disturb` simulation.

What Next

Use the graphical schematic viewer to trace the path between the PLL source and the internal clock to identify the cause of the sensitization problem.

Make sure the correct PLL clock location was specified for the violated internal clock. If the correspondence is not specified correctly, the C34 violation is the only consequence. The PLL clock location is specified either in the ClockStructures block of the STL procedure file, or in the `add_clocks` command.

DRC Rule C35

Message Text

Clock Rule C35: Multiple sensitizations between PLL source and internal clock.

Severity

Error

Description

The conditions defined for each cycle of an internal clock must sensitize the path between the corresponding PLL source and the internal clock for a single PLL clock pulse.

TestMAX ATPG expects control over the conditioning of internal clocks from one capture cycle to the next (cycle-specific conditioning).

This check cannot be performed on a clock that fails a [C34](#) check.

What Next

Some OCC controllers use one control bit to enable a double pulse. This type of controller violates a C35 check because it is impossible to control the pulses separately. This clock conditioning is suitable for two-clock transition-delay testing using common launch and capture clocks, but not for any other purpose. If this limitation is acceptable, you can downgrade the C35 rule using the command `set_rules C35 warning`.

You should verify that the PLL clock controller can accommodate cycle-specific conditioning of the internal clocks. A PLL cycle counter is a typical solution, in which the outputs of the counter are used to gate the internal clock conditioning values.

The following examples show how to define internal clocks:

```
add_clocks 0 intclk3 -intclock -pll_source u1/pllclk3 \  
-cycle { 0 clock_chain/cell[5]/Q 1 \  
1 clock_chain/cell[4]/Q 1 }
```

```
add_clocks 0 intclk2 -intclock -pll_source u1/pllclk2 \  
-cycle { 0 clock_chain/cell[3]/Q 1 \  
1 clock_chain/cell[2]/Q 1 }
```

```
add_clocks 0 intclk1 -intclock -pll_source u1/pllclk1 \  
-cycle { 0 clock_chain/cell[1]/Q 1 \  
1 clock_chain/cell[0]/Q 1 }
```

A counter keeps track of the number of clock pulses from the PLL since the start of the capture cycle. It needs to keep track of as many cycles as the maximum number defined in any one single `add_clocks` command. Each cycle number is ANDed with the

corresponding clock chain bit and the results are then ORed together to enable the clock pulse to pass through the clock MUX.

```
Count (value) 1 is ANDed with bit[1]
Count (value) 2 is ANDed with bit[0]
```

DRC Rule C36

Message Text

Clock Rule C36: Mistimed sensitizations between PLL source and internal clock.

Severity

Error

Description

This rule checks that the conditions defined for each cycle of an internal clock must sensitize the path between the corresponding PLL source and the internal clock in order of the cycles. That is, the conditions for cycle n must sensitize the path for a PLL pulse preceding the sensitization for cycle $n+1$.

This check cannot be performed on a clock that fails a [C34](#) check.

What Next

Verify that the clock controller provides for cycle-specific conditioning of the internal clocks. A typical solution is to implement a PLL cycle or pulse counter that provides one-hot values used to gate the internal clock conditioning value.

DRC Rule C37

Message Text

Clock Rule C37: Cannot satisfy all internal clocks off for all cycles.

Severity

Error

Description

It must be possible to condition all clocks off for all cycles; otherwise, no chain test can be created and some faults might be untestable.

What Next

Verify that internal clock conditioning allows for all internal clocks to be turned off by loading the appropriate conditioning values into the clock chain.

DRC Rule C38

Message Text

Clock Rule C38: Bad off-conditioning between PLL source and internal clock.

Severity

Error

Description

This rule checks that the setting of the first condition of the cycle in which the internal clock can be set to its opposite value results in the clock off, independent of other conditions.

This check cannot be performed on a clock that fails a [C34](#) check.

What Next

You should verify that the conditioning values provide exclusive control over the enabling of the internal clocks.

DRC Rule C39

Message Text

Clock Rule C39

Nonlogical clock *clock_name* connects to *number* scan cells (*gate_id*)

Clock Rule C39

Nonlogical clock *clock_name* connects to *number* non-state holding pipeline cell (*gate_id*)

Severity

Error

registers as well as the `add_capture_mask` command to mask captured value and prevent it from propagating in another cycle to another scan cell.

Also note that C39 violations that were downgraded to warnings are not saved in image files. Make sure that any violated scan cells are masked before saving the image file.

By default, TestMAX ATPG prints only one message for each clock that causes a C39 violation. You can use the `set_rules c39 warning -verbose` command to print out an expanded list of C39 clock rules violation information. In this case, every scan cell affected by a C39 violation is printed.

The following example shows the default output that result after specifying the `set_rules c39 warning` command:

```
set_rules c39 warning
report_violations C39

Warning: Nonlogical clock dft_clock connects to 4 scancells
(u1/ci_reg_3_ (827)). (C39-1)
Warning: Nonlogical clock clkref connects to 19 scancells (u8/LOCKUP
(812)). (C39-2)
```

The next example shows the output after using the `set_rules c39 warning -verbose` option. Note, in this case, that the `set_rules` command must be specified before the `run_drc` command or the violations will not be calculated using a verbose methodology.

```
set_rules c39 warning -verbose
...
run_drc
...
report_violations C39

Warning: Nonlogical clock dft_clock connects to scancell u1/ci_reg_3_
(827). (C39-1)
Warning: Nonlogical clock dft_clock connects to scancell u1/ci_reg_2_
(828). (C39-2)
Warning: Nonlogical clock dft_clock connects to scancell u1/ci_reg_0_
(829). (C39-3)
Warning: Nonlogical clock dft_clock connects to scancell u1/ci_reg_1_
(836). (C39-4)
Warning: Nonlogical clock clkref connects to scancell u8/LOCKUP (812).
(C39-5)
Warning: Nonlogical clock clkref connects to scancell u8/ai_reg_3_
(873). (C39-6)
.
.
.
```

If you see first variation of C39 rule violation in uncompressed design with non-state holding pipeline cells or second variation of C39 rule violation message, you can safely

downgrade the violation using `set_rules C39 warning` to pass DRC and VCS. You must mask the non-state holding pipeline cells by default to pass VCS.

DRC Rule C40

Message Text

Clock Rule C40

Internal clock `gate_id` is restricted due to `<reason>`.

Clock Rule C40-1 Internal clock `name` is restricted due to its `<time>` latency not aligning with its controller's `<time>` latency.

Severity

Warning

Description

The C40 rule applies only to synchronized internal clocks (declared in a SynchronizedClocks block of the STIL procedure file). TestMAX ATPG manipulates the clock chain data for synchronized clocks to compensate for different periods or start-up latencies. This warning indicates that DRC has encountered a configuration that prevents such manipulation, and the clock is flagged as restricted. A restricted clock cannot be activated in the same pattern as another clock of the same sync group, unless both clocks have the same period and latency. Internal clocks with the same period and latency do not require special clock chain manipulation.

The C40 warning is triggered by ATPG or cell constraints on the clock chain or PLL condition nets. It is also triggered by unsupported clock controller configurations in which the one-to-one relationship between clock chain bits and internal clock pulses is violated. Some examples of unsupported configurations are shared clock chain bits (either within or across clock controllers), or complex logic driving the PLL conditions.

What Next

You should evaluate the described constraints to see if they can be eliminated or moved away from the clock controller logic.

In most cases, you can ignore this warning and accept the clocking restrictions. This might result in pattern inflation or a reduction in fault coverage.

As a last resort, you can remove the clock from the SynchronizedClocks block of the STIL procedure file. This will eliminate all of the synchronized clock ATPG restrictions, as TestMAX ATPG is ignorant of the timing for this clock. In this case, you should be careful to simulate the patterns outside of TestMAX ATPG to validate the ATPG timing assumptions.

If you get a C40-1 message, you will need to adjust the latency for multiple synchronized clocks on the same controller, and should refer to the details specified in the corresponding M722 message.

DRC Rule C41

Message Text

Clock Rule C41:Internal clocks *gate_id* and *gate_id* are equivalent.

Severity

Warning

Description

The two clocks noted in this message have been identified as equivalent clocks, which means they will always both be active or inactive together. Equivalence markers for internal clocks are removed prior to DRC. Equivalence checks for internal clocks are now done before clock checks and It is now impossible to get ungroupable equivalent clocks.

During DRC it will show information on these clocks.

The following clocks were identified as groupable with clock CG/mux_clk500ma (357).

CG/mux_clk500mb (358).

What Next

If the clocks always pulse simultaneously and are skew-balanced in the design, the patterns generated by ATPG will be correct. However, if the clocks are not both simultaneous and skew-balanced, the patterns generated with [M285](#) messages will be incorrect and will fail at test or with full-timing simulation.

Review the grouping opportunities for these clocks using the command `report_clocks-matrix`. Consider design or constraint changes to allow these clocks to be grouped.

Review the conditions or constraints on these clocks that cause them to be identified as equivalent. For instance, "always on" conditions for internal clocks will cause these clocks to be equivalent. Consider reducing these constraints to allow the clocks to be independent.

As a last resort, consider removing the definition of one or the other clock, and running DRC without the combination of clocks present. This will require multiple DRC and ATPG passes to process each clock separately.

DRC Rule C43

Message Text

Scan enable (name = <scan_enable_name>, id = <G_id>, (got/exp) <E>/<G>) does not facilitate capture after <N> delay cycles.

Scan_enable_name indicates the instance name of the scan_enable gate.

G_id indicates gate id of the scan_enable gate

N indicates expected delay-capture cycles after load_unload before capture occurs(specified as se_delay in protocol file)

E is the expected value on scan_enable gate after N delay-capture cycles

G is the value on the scan_enable gate after N delay-capture cycles

Severity

Error

Description

After the load-unload procedure, the scan_enable gate (specified in the protocol file) is checked to have the expected value (specified in the protocol file) after se_delay (specified in the protocol file) number of delay-capture cycles. The C43 violation is displayed if the scan_enable gate does not have the expected value after specified number of delay-capture cycles.

What Next

To correct this problem, the netlist or protocol file needs to be changed. You can use the automated violation analysis to debug this problem. This analysis displays the scan_enable gate when the pindata is set to “set_pindata -se_delay” and contains the simulated values of the capture-delay cycles.

This rule can be downgraded but it is not recommended as the generated patterns may be incorrect.

DRC Rule C44

Message Text

Scan enable (name = <scan_enable_name >, id = <G_id>, (got/exp) <E>/<G>) facilitate capture at <N1> delay cycles but expected after <N2> delay cycles.

Scan_enable_name indicates the instance name of the scan_enable gate

G_id indicates gate id of the scan_enable gate

N1 indicates current delay-capture cycle at which capture occurs

N2 indicates expected delay-capture cycle at which capture should occur(specified as se_delay in protocol file)

E is the expected value on scan_enable gate at N1 delay-capture cycle

G is the value on the scan_enable gate at N1 delay-capture cycle

Severity

Error

Description

After the load-unload procedure, scan_enable gate (specified in the protocol file) is checked to have the expected value (inverted of the value specified in the protocol file) during all the delay-capture cycles except the last delay-capture cycle. C44 violation is thrown if scan_enable gate has incorrect value during these delay-capture cycles.

What Next

To correct this problem, netlist or protocol file needs to be changed. The automated violation analysis can be used to debug this problem. This analysis displays the scan_enable gate and pindata is set to "set_pindata -se_delay", which has the simulated values of the capture-delay cycles.

This rule can be downgraded but not recommended as the generated patterns may be incorrect.

Category D - DFT Rules

See Also

- [D1 - clock input was not controlled](#)
- [D2 - set input was not controlled](#)
- [D3 - reset input was not controlled](#)
- [D4 - clock input was not controlled](#)
- [D5 - set input was not controlled](#)
- [D6 - reset input was not controlled](#)
- [D7 - clock input RAM was not controlled](#)

- D8 - clock cannot capture data
- D9 - clock input not active when set on
- D10 - clock connects to data input
- D11 - clock connects to clock and data inputs
- D12 - clock connects to clock/set/reset inputs
- D13 - clock can capture new data on LS input
- D14 - clock can capture new data on TE input
- D15 - clock affected by new capture on LS input
- D16 - clock affected by new capture on TE input
- D17 - clock input couldn't capture data
- D18 - multiple clock/reset/set inputs (G1/G2) of DFF cell are active
- D20 - bus gate failed contention check
- D21 - bus gate failed state check
- D22 - wire gate failed contention check
- D23 - feedback path sensitizable through source gate
- D39 - probability of capture X (P) exceeds threshold for scancell

DRC Rule D1

Message Text

DRC Rule D1: Clock input I of DFF S was not controlled

Severity

Warning

Description

When all clock ports are in their off state, all clock inputs of flip-flops must also be in their off state.

Clock input I is the uncontrolled input pin on the cell S. In some cases, pin I is not displayed when the DRC engine cannot point to the exact pin. This occurs when the sequential cell in question is driven by multiple clock inputs.

This check is performed using the simulated values that result when:

Clocks are in their off state

Constrained ports are set to their constrained values

Constant-value nonscan cells are set to their constant state

This rule violation occurs if any clock input of a flip-flop is not in its off state.

If the scan style is a multiplexed flip-flop, violation of this rule has the following effects:

The cell is violated

The cell is not included in scan chains

DFT insertion unscans the cell if it has already been scan-replaced, depending on user specifications

If the scan style is other than multiplexed flip-flop, this violation is flagged only when the violation occurs on the functional clock, and will not have any effect even if it is flagged.

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

A common cause of this violation is an incorrect or missing clock definition. Another common cause is a clock signal that is blocked from reaching the input of a scan cell.

Check the signal type specifications, the protocol and the design.

DRC Rule D2

Message Text

DRC Rule D2: Set input I of DFF S was not controlled

Severity

Warning

Description

When all clock ports are in their off state, all set inputs of flip-flops must be in their off state.

I is the uncontrolled input pin to the cell S. The pin I might not be displayed in some cases in which the DRC engine cannot point to the exact pin. This will occur if the sequential cell in question is driven by multiple clock inputs.

This check is performed using the simulated values that result when:

Clocks are in their off state

Constrained ports are set to their constrained values

Constant value nonscan cells are set to their constant state

This rule violation occurs if any set input of any sequential cell is not in its off state.

Violation of this rule has the following effects:

The cell is violated

The cell is not included in scan chains

DFT insertion unscans the cell if it has already been scan-replaced, depending on user specifications

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

A common cause of this violation is an incorrect or missing clock definition. Another common cause is a clock signal that is blocked from reaching the input of a scan cell. Check the signal type specifications, the protocol and the design.

DRC Rule D3

Message Text

DRC Rule D3: Reset input I of DFF S was not controlled

Severity

Warning

Description

When all clock ports are in their off state, all reset inputs of scannable flip-flops must be in their off state.

S is the scan cell-name and I is the input pin that is not controlled. The pin I might not be displayed in some cases where the DRC engine cannot point to the exact pin. This will occur if the sequential cell in question is driven by multiple clock inputs.

This check is performed using the simulated values that result when:

Clocks are in their off state

Constrained ports are set to their constrained values

Constant value nonscan cells are set to their constant state

This rule violation occurs if any set input of any sequential cell is not in its off state.

Violation of this rule has the following effects:

The cell is violated

The cell is not included in scan chains

DFT insertion unscans the cell if it has already been scan-replaced, depending on user specifications

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

A common cause of this violation is an incorrect or missing clock definition. Another common cause is a clock signal that is blocked from reaching the input of a scan cell. Check the signal type specifications, the protocol and the design.

DRC Rule D4

Message Text

DRC Rule D4: Clock input I of DLAT S was not controlled

Severity

Warning

Description

When all clock ports are in their off state, all latch clock inputs must also be at their off state.

S is the scan cell-name and I is the input pin that is not controlled. The pin I might not be displayed in some cases where the DRC engine cannot point to the exact pin. This will occur if the sequential cell in question is driven by multiple clock inputs.

This check is performed by simulating the logic values that result when:

Clocks are in their off state

Constrained ports are set to their constrained values

Constant value nonscan cells are set to their constant state

This rule violation occurs if any clock input of a latch is not in its off state.

Violation of this rule has the following effects:

The cell is violated

The cell is not included in scan chains

DFT insertion unscans the cell if it has already been scan-replaced, depending on user specifications

If the scan style is other than multiplexed flip-flop, this violation is flagged only if the violation occurs on the functional clock. There is no effect from the flagging.

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

DRC Rule D5

Message Text

DRC Rule D5: Set input I of DLAT S was not controlled

Severity

Warning

Description

When all clock ports are in their off state, all latch clock inputs must be at their off state also.

S is the cell-name and I is the input pin that is not controlled. The pin I might not be displayed in some cases where the DRC engine cannot point to the exact pin. This will occur if the sequential cell in question is driven by multiple clock inputs.

This check is performed by simulating the logic values that result when:

Clocks are in their off state

Constrained ports are set to their constrained values

Constant value nonscan cells are set to their constant state

This rule violation occurs if any clock input of a latch is not in its off state.

Violation of this rule has the following effects:

The cell is violated

The cell is not included in scan chains

DFT insertion unscans the cell if it has already been scan-replaced, depending on user specifications

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

DRC Rule D6

Message Text

DRC Rule D6: Reset input I of DLAT S was not controlled

Severity

Warning

Description

When all clock ports are in their off state, all latch clock inputs must be in their off state also.

S is the cell-name and I in the input pin that is not controlled. Pin I might not be displayed in cases where the DRC engine cannot point to the exact pin. This occurs if the sequential cell in question is driven by multiple clock inputs.

This check is performed by simulating the logic values that result when:

Clocks are at their off state

Constrained ports are set to their constrained values

Constant value nonscan cells are set to their constant state

This rule violation occurs if any clock input of a latch is not at its off state.

Violation of this rule has the following effects:

The cell is violated

The cell is not included in scan chains

DFT insertion unscans the cell if it has already been scan-replaced, depending on user specifications

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

DRC Rule D7

Message Text

DRC Rule D7: Clock input I of RAM M was not controlled

Severity

Warning

Description

M is the name of the memory element and I is the input pin that is not controlled.

When all clock ports are in their off state, all clock/set/reset inputs of memory elements must be in their off state.

The off state of a clock PI is the value 0 or 1 in which the cell driven by the PI is stable.

This check is performed using the simulated values that result when:

Clocks are in their off state

Constrained ports are set to their constrained values

Constant value nonscan cells are set to their constant state

This rule violation occurs if any clock/set/reset input of a memory element is not in its off state.

Violation of this rule has the following effects:

The cell is violated

Memory cells are considered black boxes unless a simulation model is provided for them. Use the test_simulation_library variable to provide a simulation model.

Can lead to C20 violations in post-DFT DRC

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check your design, model, and specification. A common cause of this violation is a primary input that controls memory writes not defined as a clock.

DRC Rule D8

Message Text

DRC Rule D8: Clock C cannot capture data with other clocks off

Severity

Warning

Description

Each defined clock port must be capable of turning on a clock, set, reset, or write_control input of in least one sequential device or memory element when all other clocks are off. C is the name of the port.

This check is performed using the simulated values that result when:

The specified port (and any equivalent port) is set to X

All other defined clocks or asynchronous sets and resets are set to there off states

Constrained ports are set to their constrained values

Constant value nonscan cells are set to their constant state

The rule violation occurs if all clock/set/reset/write inputs of all sequential elements (including RAMs) are off during this condition. In addition, this violation can be caused by any of the following:

Incorrect or missing clock definition

A clock signal is blocked from reaching the input of the internal sequential cell or memory element. This can occur when the signal passes through an AND gate and the control size is 0, or an OR gate with a control size of 1.

If the instance is a nonscan device and there is another pin on the device such as an asynchronous set or reset that is controlled from a primary input that has not been correctly defined. For example, a CLK pin has been defined as a clock but a RESET pin has not been defined as a reset. Consequently, data cannot be captured when the CLK pin is pulsed and RESET=X. Correct this by defining the missing signal.

Violation of this rule has the following effects:

Failure to satisfy this rule indicates the specified port cannot perform a capture operation

It does not violate any cells but can lead to C4 violations is post-DFT DRC

Note the following:

Pipeline register clocks should have D8 violations if they are definitely dedicated clocks

Reference clocks are not subject to D8 violations

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check the signal-type specifications, the protocol and the design.

DRC Rule D9

Message Text

DRC Rule D9: Clock input I of <type> S not active when clocks are set on.

Severity

Warning

Description

When all clock ports are in their on state, none of the clock inputs of sequential elements must be inactive (X).

S is the cell name and I is the input pin that is not controlled. The <type> is either DLAT (latch) or DFF (flip-flop). The pin I might not be displayed in some cases in which the DRC engine cannot point to the exact pin.

This check is performed using the simulated values that result when:

Clocks are in their on state

Constrained ports are set to their constrained values

Constant value nonscan cells are set to their constant state

This rule violation occurs if any clock input of a flip-flop is not active. This rule checks the scan controllability of a sequential cell.

If the scan style is multiplexed flip-flop, violation of this rule has the following effects:

The cell is violated

The cell is not included in scan-chains

DFT insertion unscans the cell if it has already been scan-replaced, depending on user specifications

If the scan style is other than multiplexed flip-flop, this violation is flagged only if the violation occurs on the functional clock, and does not have any effect even if it is flagged.

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

A common cause of this violation is an incorrect or missing clock definition. Another common cause is a clock signal that is blocked from reaching the input of a scan cell. For example, the clock signal could be blocked by a clock-gating cell that does not have test mode connections that put it into a pass-through mode during test. Check the signal type specifications, the protocol, and the design.

A less common cause of this violation is if the registers are clocked by a clock that has been specified as an ATE clock for on-chip clocking (OCC) controller insertion using the DFT Compiler command `set_dft_signal -type Oscillator`. In this case, the ATE clock is free-running so that the clock for the registers is uncontrollable for scan ATPG purposes. This type of dual usage of the clock should be avoided. But if there are no available clocks or ports to use as an ATE clock, there is a workaround.

The workaround is to remove the `set_dft_signal -type Oscillator` specification for the ATE clock. This will result in post-DFT DRC failing with [S1](#) violations, but the resulting netlist and protocol are usable. The ATE clock must be specified in the TestMAX ATPG command file with the command `add_clocks 0 ate_clock -refclock`. The resulting S18 and C39 errors must be downgraded, and command `set_rules S18 warning -mask` must be used to prevent miscompares in Verilog resimulation.

Warning: Although this workaround enables stuck-at fault testing for these flip-flops, it causes a significant loss of transition delay fault coverage in ATPG.

DRC Rule D10

Message Text

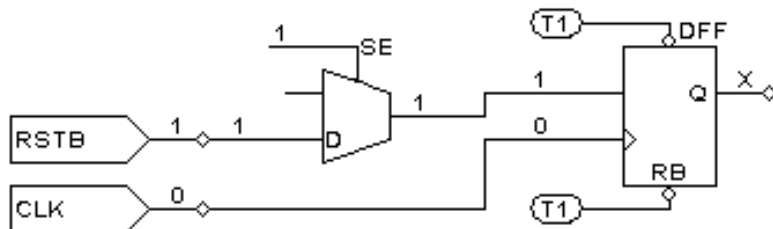
DRC Rule D10: Clock C connects to data input of <type> S

Severity

Warning

Example

One example, many variations are possible:



Description

A clock has a path that can be propagated to a data input of a DFF/DLAT when PI constraints and constant value cells are set. The data input of a sequential cell was found to have an unblocked path to a primary input declared as a clock, and this is a different clock than the one connected to the clock input of the cell. C is the clock port, <type> is either DFF (flip-flop) or DLAT (latch), and S is the cell-name.

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Also make sure the pindata that it uses the clock is on.

Check your design and specifications.

DRC Rule D11

Message Text

DRC Rule D11: Clock C connects to clock and data inputs I1/I2 of <type> S

Severity

Warning

Description

A clock has a combinational path to both the data and clock/set/reset inputs of a sequential cell. C is the clock port name; I1/I2 are the input pins; S is the sequential cell name. A clock must not capture data for a sequential cell whose data line is affected by the same clock.

This check is performed using clock cone data. A rule violation occurs when a clock/set/reset input and its data input are in the same clock cone. A clock cone is the cone of

combinational logic fanning out from a single clock input port and extending to one or more sequential device input pins or a primary output port.

Violation of this rule has the following effects:

Can result in a race condition

This does not violate a cell but it can lead to C11/C12/C13 violations in post-DFT DRC

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check the design and specifications. It can be possible to block the path from the clock to the data input as it passes through combinational logic by applying a constraint to a net connecting to the other input of one of these combinational gates. For example, holding the other input of an AND gate to zero will block the path of the clock through that gate.

DRC Rule D12

Message Text

DRC Rule D12: Clock C connects to clock/set/reset inputs (G1 / G2) of DFF I

Severity

Warning

Description

A common path (combinational gates) exists from a defined clock port to more than one clock input of a sequential device. A clock input includes any asynchronous set or reset control inputs as well as data clocks. C is the name of the clock, G1 and G2 are the clock or set or reset pins to the cell I, and I is the instance pathname to the device.

Failure to pass this rule check indicates there is a potential race condition because a clock port has a combinational gate path to two different clock inputs of the same ATPG sequential primitive.

If the scan style is a multiplexed flip-flop, violation of this rule has the following effects:

- The cell is violated.
- The cell is not included in scan-chains.
- DFT insertion unscans the cell if it has already been scan-replaced, depending on user specifications.

If the scan style is other than a multiplexed flip-flop, this violation is flagged only if the violation occurs on the functional clock, and does not have any effect even if it is flagged.

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

You might choose to ignore this rule. If you do, you might need to mask this instance during ATPG to avoid potential mismatches. A timing analysis or simulation will guide you with respect to this masking.

DRC Rule D13

Message Text

DRC Rule D13: Clock C can capture new data on LS input I1 of <type> S1

Source of violations: input I2 of <type> S2.

Severity

Warning

Description

A clock must not capture data into a level sensitive (LS) input (latch or RAM) if that data can be affected by new captured data. C is the clock port; the <type> is either DLAT (latch) or DFF (flip-flop); I1 and I2 are input pin names; S1 and S2 are cell names.

A potential rule violation occurs on a clock if a clock input of a LS pin is in the clock cone and its data input (including RAM address) is in the effect cone.

A clock cone is the cone of combinational logic fanning out from a single clock input port and extending to one or more sequential device input pins or a primary output port.

An effect cone is the cone of combinational logic fanning out from the output pin of a sequential device (clocked by CLK1 for example) and extending through all combinational logic until it reaches other sequential devices or primary output ports. An effect cone is always relative to a specific clock (CLK1) connected to the clock/set/reset pin of a sequential device.

Violation of this rule has the following effects:

This is a capture violation

It does not violate any cells but can lead to C5 violations in post-DFT DRC

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check the design and specifications.

DRC Rule D14

Message Text

DRC Rule D14: Clock C can capture new data on TE input I1 of <type> S1
Source of violations: input I2 of <type> S2.

Severity

Warning

Description

A clock must not capture data into a trailing edge (TE) input if that data can be affected by new captured data. C is the clock port; the <type> is either DLAT (latch) or DFF (flip-flop); I1 and I2 are input pin names; S1 and S2 are cell names.

A potential rule violation occurs on a clock if a clock input of a TE pin is in the clock cone and its data input (including RAM address) is in the effect cone. This input is called the sink of the violation.

To be considered a violation, all of the following are true:

The source must not be a TE pin

The source clock must be capable of being at 1 (0 if LE) when the sink clock is at 1

It must be possible to propagate the path from source to sink under this condition

A clock cone is the cone of combinational logic fanning out from a single clock input port and extending to one or more sequential device input pins or a primary output port.

An effect cone is the cone of combinational logic fanning out from the output pin of a sequential device (clocked by CLK1 for example) and extending through all combinational logic until it reaches other sequential devices or primary output ports. An effect cone is always relative to a specific clock (CLK1) connected to the clock/set/reset pin of a sequential device.

Failure to satisfy this rule does not violate any cells, but it will lead to C6 violations in post-DFT DRC. This is a capture violation.

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check the design and specifications.

DRC Rule D15

Message Text

DRC Rule D15: C clock path affected by new capture on LS input I1 of <type> S1

Source of violation: input I2 of <type> S2.

Severity

Warning

Description

The path from a clock to a level sensitive (LS) input must not be affected by its new captured data.

C is the clock port; the <type> is either DLAT (latch) or DFF (flip-flop); I1 and I2 are input pin names; S1 and S2 are cell names.

This check is performed using clock cone data and a potential rule violation occurs on a clock if a clock input of a level sensitive pin is in both the clock cone and effect cone. This input is called the sink of a violation. To be considered a violation, the source must not be a TE input, the source clock must be capable of being at 1 when the sink clock is at 1, and it must be possible to propagate the path from source to sink under this condition.

A clock cone is the cone of combinational logic fanning out from a single clock input port and extending to one or more sequential device input pins or a primary output port.

An effect cone is the cone of combinational logic fanning out from the output pin of a sequential device (clocked by CLK1 for example) and extending through all combinational logic until it reaches other sequential devices or primary output ports. An effect cone is always relative to a specific clock (CLK1) connected to the clock/set/reset pin of a sequential device.

Violation of this rule has the following effects:

This is capture violation

Failure to satisfy this rule can result in a race condition that can eventually lead to C8 violations in post-DFT DRC

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check the design and specifications.

DRC Rule D16

Message Text

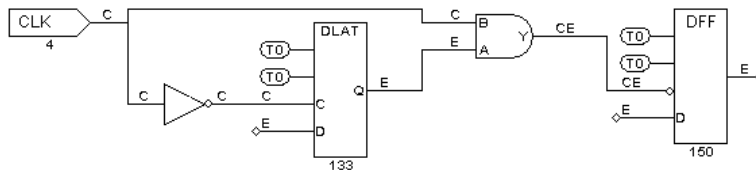
DRC Rule D16: C clock path affected by new capture on TE input I1 of <type> S1.

Source of violation: input I2 of <type> S2.

Severity

Warning

One example, many variations are possible:



Description

The path from a clock to a trailing edge input must not be affected by its new captured data.

C is the clock port; the <type> is either DLAT (latch) or DFF (flip-flop); I1 and I2 are input pin names; S1 and S2 are cell names.

Failure to satisfy this rule results in an additional simulation pass to allow the trailing edge (TE) port to capture new data from leading edge (LE) or level sensitive (LS) ports. There is also risk that test generation will not correctly control the new value that is captured, which will result in some loss of test coverage.

This check is performed using clock cone data and a rule violation occurs on a clock if a clock input of a TE port is in both the clock cone and effect cone. This port is called the sink port of a violation. To be considered a violation, the source port must not be a TE port, the source port clock must be capable of being at 1 (0 if LE) when the sink port clock is at 1, and the path from source to sink must be propagatable under this condition.

What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the ANALYZE button. This changes the pin data reporting to the clock cone data and displays all gates in the paths from the source port to the sink port, the clock to the source port, and the clock to the sink port.

A list of all potential source ports for the violation is identified by tracing back from the sink clock line through gates in both the clock and its effect cone.

Use the `-mask` option of the `set_rules` command to compensate for the DRC violation. By setting the rule severity to warning and enabling the `-mask` option, this causes the ATPG algorithm to mask off the observe values, in an attempt to ensure that any ATPG pattern generated will pass simulation. The tradeoff is that in doing this, there is a reduction of test coverage.

DRC Rule D17

Message Text

DRC Rule D17: Clock input I of <type> S couldn't capture data

Severity

Warning

Description

Each clock/set/reset input of a state element must be capable of capturing data when a single clock is on and all other clocks are off. I is the clock/set/reset input, <type> is either DLAT (latch) or DFF (flip-flop), and S is the name of the cell. The input I might not be displayed if DRC cannot identify the exact pin.

This check is performed using the simulated values that result when:

One clock is set to X

All other clocks are in their off state

The constrained pins are set to the constrained values

The constant-value nonscan cells are set to their constant values.

The violation occurs when the clock/set/reset input of a sequential cell is off rather than X.

The off state of a clock PI is the value 0 or 1 at which the sequential cell driven by the PI is stable.

If the scan style is multiplexed flip-flop, violation of this rule has the following effects:

The cell is violated

The cell is not included in scan chains

TestMAX DFT unscans the cell if it has already been scan replaced, depending on user specifications

If the scan style is other than multiplexed flip-flop, this violation is flagged only if the violation occurs on the functional clock, and will not have any effect even if it is flagged

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

A common cause of this violation is a clock pin tied to a constant. Check your design and specifications.

DRC Rule D18

Message Text

DRC Rule D18: Multiple clock/reset/set inputs (G1/G2) of DFF <cell_name> are active.

Severity

Warning

Description

More than one clock input of a sequential device is active at the same time. A clock input can include any asynchronous set or reset control inputs or data clocks. G1 and G2 are the clock or set or reset pins to the cell <cell_name>, which includes the instance pathname to the device. Failure to pass this rule check indicates there is a potential race condition.

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

DRC Rule D20

Message Text

DRC Rule D20: Bus gate N failed contention ability check for drivers G1 and G2

Severity

Warning

Description

A bus must not be capable of being in a contention state when constraints are set.

N is the instance name of the bus device, and G1 and G2 are two drivers that could contribute to a contention situation. The analysis will stop after identifying two nets in a possible contention state.

This check does not indicate that contention is occurring, only that it is possible. Therefore, monitor the bus closely.

There should be no affect on test coverage.

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check your design and eliminate the contention if possible.

DRC Rule D21

Message Text

DRC Rule D21: Bus gate N failed Z state ability check

Severity

Warning

Description

A bus must not be capable of being in a Z state when constraints are set.

Failure to satisfy this rule will not violate cells or affect test coverage. However, internal Z states become Xs as they pass through other gates and can lead to an increase in tester mask requirements if the Xs make their way into scan chains or outputs.

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check your design and eliminate the Z state if possible.

DRC Rule D22

Message Text

DRC Rule D22: Wire gate N failed contention ability check for drivers G1 and G2

Severity

Warning

Description

A wired-and net must not be capable of being at different state when constraints are set.

Violation of this rule has the following effects:

An X state on the wired-and output, which can lead to a loss of test coverage

The contending condition can also cause harmful device stress if the non-tristate drivers are allowed to be in contention

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check your design and eliminate the contention if possible. Remove the wired-and structures.

DRC Rule D23

Message Text

DRC Rule D23: Feedback path network X is sensitizable through source gate G

Severity

Warning

Description

Feedback paths must not be sensitizable. A combinational feedback path was also found to be sensitizable. In other words, there is a feedback loop that completes the path back on itself and results in a potential oscillation or ring driver.

Failure to satisfy this rule indicates a potential sensitizable feedback path. You must also consider the effect of nonscan cells with constant values or user-specified constraints. Investigate further to determine whether there is a truly sensitizable feedback path.

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check your design. Eliminate the feedback loop if necessary.

DRC Rule D39

Message Text

DRC Rule D39: Probability of capture X (P) exceeds threshold for scancell
<gate_type> *gate_id*.

Severity

Warning

Description

The probability of capture *x* exceeds the threshold for the scan cell. In the message, *P* is the percent probability (0 -100). This rule checks for scan cells with data input that has a probability of *X* that equals or exceeds the threshold value. If the `set_drc-xchain_threshold_probability` option is set to 0 (the default), or the severity is set to `ignore`, this rule will not be checked. The scan cells checked will all be DFFs which pass rules [D1](#), [D2](#), and [D3](#), or cells which have been selected by the `set_scan_ability` command.

What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check the design and specifications.

Category K - LBIST Rules

See Also

- [K1](#)
- [K2](#)
- [K3](#)
- [K4](#)
- [K5](#)

Chapter 17: DRC Rules
Category K - LBIST Rules

- [K6](#)
- [K7](#)
- [K8](#)
- [K9](#)
- [K10](#)
- [K11](#)
- [K12](#)
- [K13](#)
- [K14](#)
- [K15](#)
- [K16](#)
- [K17](#)
- [K18](#)
- [K19](#)
- [K20](#)
- [K21](#)
- [K22](#)
- [K23](#)
- [K24](#)
- [K25](#)
- [K26](#)
- [K27](#)
- [K28](#)
- [K29](#)
- [K30](#)
- [K31](#)
- [K32](#)

- [K33](#)
- [K34](#)
- [K35](#)
- [K36](#)
- [K37](#)
- [K38](#)
- [K39](#)
- [K40](#)
- [K41](#)
- [K42](#)
- [K43](#)
- [K44](#)
- [K45](#)
- [K46](#)
- [K47](#)
- [K48](#)
- [K49](#)
- [K50](#)
- [K51](#)
- [K52](#)
- [K53](#)
- [K54](#)
- [K55](#)
- [K56](#)
- [K57](#)
- [K58](#)
- [K59](#)

- [K60](#)
- [K61](#)
- [K62](#)
- [K63](#)
- [K64](#)
- [K65](#)
- [K66](#)
- [K67](#)
- [K68](#)
- [K69](#)
- [K70](#)
- [K71](#)
- [K72](#)
- [K73](#)
- [K74](#)
- [K75](#)
- [K76](#)
- [K77](#)
- [K78](#)
- [K79](#)
- [K80](#)
- [K81](#)
- [K82](#)
- [K83](#)
- [K84](#)
- [K85](#)
- [K86](#)

- [K87](#)
- [K88](#)
- [K89](#)
- [K90](#)
- [K91](#)
- [K92](#)
- [K93](#)
- [K94](#)
- [K96](#)
- [K97](#)
- [K98](#)
- [K99](#)
- [K100](#)
- [K101](#)
- [K102](#)
- [K103](#)
- [K104](#)
- [K105](#)
- [K106](#)
- [K107](#)
- [K112](#)
- [K119](#)
- [K120](#)

DRC Rule K1

Message Text

Seq compressor chain length L1 did not equal expected length L2.

Where:

L1 indicates calculated length of `codec_drc_chain` and `occ_chain`(optional)

L2 indicates expected length of `codec_drc_chain` and `occ_chain`(options)
from the STIL protocol file

Severity

Fatal Error

Description

Seq compressor chain includes `codec_drc_chain` and `occ_chain` (optional).
The `codec_drc_chain` is traced back from its defined chain output port using the simulated values of the `CODEC_DRC_LOAD_UNLOAD` and `CODEC_DRC_SHIFT` procedures. `occ_chain` is also traced back from its defined chain output port using the simulated values of `OCC_LOAD` and `OCC_SHIFT` procedures. If the number of scan cells identified by this tracing does not equal the expected number of cells, then a violation of this rule is issued. The expected number of cells in the particular `seq_compressor_chain(codec_drc_chain)` is the sum of the following which are defined in the STIL protocol file:

Length of the `occ_chain` register (optional - may be zero)
Lengths of the registers on `codec_drc_chain`:
Registers of CODEC:
lengths of the CarePRPG registers
lengths of the XTOLPRPG registers
lengths of the MISR registers
lengths of the PRPGShadow registers
lengths of the CARE_shadow registers
lengths of the XTOLShadow registers
lengths of the XTOL_enable registers (always 1)
length of the Power_enable register (1 if length is not specified)
length of the SERDESReg
length of the MaskEnable (always 1)
length of the XchainsRegister
Registers of IDU:
lengths of SERDESRegs
lengths of Address registers
lengths of Broadcast registers(always 1)
lengths of New_address_shadow registers(always 1)
lengths of Final_segment registers(always 1)
lengths of Shift_counter_shadow registers
lengths of Seed_type registers (always 1)
lengths of Capture_enable_shadow registers (always 1)
lengths of Current_address registers
lengths of First_address registers
lengths of PatternStart registers (always 1)
lengths of New address registers
lengths of NewAddressCyclesReg registers
lengths of IncrAddressCyclesReg registers

```
lengths of OCCCyclesReg registers
lengths of OCCLocalCyclesReg registers
lengths of MISRCyclesReg registers
lengths of PayloadCyclesCounter registers
lengths of LocalCyclesCounter registers
lengths of CumulativeCodecEnable registers
lengths of CodecEnable registers
lengths of Shift_counter registers
lengths of Capture_enable registers
lengths of FirstShift registers (always 1)
lengths of MaskAllChains registers (always 1)
```

What Next

The severity of this violation is a fatal error, so it must be corrected before an ATPG (Automatic Test Pattern Generation) can be performed. To correct the problem, identify which of the register lengths are incorrectly defined in the STIL protocol file and edit them to specify proper values. There is no automated violation analysis that can be used to help debug this problem. The `-codec_drc_chain_cells` option of the `report_compressors` command can be used to display an ordered list of all the cells in the `codec_drc_chain`. The `-cells` option of the `report_compressor` and `report_idus` can be used to display the cells and lengths of various registers of CODEC and IDU respectively. The `-occ_chain_load_segments` option of the `report_compressor` can be used to report the `occ_chain` cells.

DRC Rule K2

Message Text

```
DFTMAX LBIST Rule K2: Invalid clock capture edge for compressor cell
Invalid clock capture edge was used for T seq compressor cell G.
```

Severity

Fatal Error

Description

Note the following:

T indicates register type

G indicates gate ID of failing register cell

The `codec_drc_chain` is traced back from its defined chain output port using the simulated values of the `CODEC_DRC_LOAD_UNLOAD` and `CODEC_DRC_SHIFT` procedures. The edge on which each register type captures data during the `CODEC_DRC_SHIFT`

procedure is checked to ensure that it is correct. The following registers are checked for the given edge:

CarePRPG cells — must capture on leading edge

XTOLPRPG cells — must capture on leading edge

MISR cells — must capture on leading edge

PRPGShadow cells — must capture on leading edge

CARE_shadow cells — must capture on leading edge

XTOLShadow cells — must capture on leading edge

XTOL_enable cells — must capture on leading edge

occ_chain cells — not checked

Power_enable cells — not checked

What Next

To correct this problem, you must edit the netlist or the STL procedure file so that the failing cells capture data on the required edge. You can use automated violation analysis to debug this problem. This analysis displays the failing cell with the pin data set to the simulated values of the CODEC_DRC_SHIFT procedure.

You can use the `-cells` option of the `report_compressors` command to report the cells for a selected register type.

You can use the `codec_drc_shift` option of the `set_pindata` command to display the simulated values of the `codec_drc` procedure.

This rule violation can be downgraded to a warning. But this is not recommended because this violation is likely to cause the data transfer of the compression circuitry to be out of sync and corrupt any patterns created by TestMAX ATPG.

DRC Rule K3

Message Text

DFTMAX LBIST Rule K3: PRPG output could not be identified

Hold output(P) of CarePRPG in compressor S could not be identified.

Hold output of XTOLPRPG in compressor S could not be identified.

Output N of XTOLPRPG in compressor S could not be identified.

Severity

Fatal Error

Description

Note the following:

S indicates the name of failing DFTMAX LBIST CODEC

P is the Power Enable Register Value setting for which hold output is not identified

N is the failing output ID

The nodes corresponding to the CarePRPG hold output, the XTOLPRPG hold output, and the XTOLPRPG observe mode selection outputs are inferred based on defined connections to their associated PRPGs. This inference is accomplished by performing parallel (32) pattern simulation using the simulated values of the SHIFT procedure and by placing random values on the PRPG cells. Gates connected to the cone of influence of the PRPGs cells are inspected to identify any gate which is are the expected values or the inverse value. Expected values are calculated by exclusive-OR in the values assigned to the associated connecting PRPG cells. A violation is issued if a gate cannot be found at the expected value.

Note:

While inferring CAREPRPG hold output, all the `pwr_enable` settings are used to validate the inferred hold output.

For CAREPRPG, the expected values are calculated based on Low Power Hardware versions (detected and informed during DRC) as follows:

For V1:

```
HOLD =  
((Pwr_enable[0] && Pwr_ctrl[0]) ||  
(Pwr_enable[1] && (Pwr_ctrl[1] || Pwr_ctrl[2]))) ||  
(Pwr_enable[2] && (Pwr_ctrl[3] || Pwr_ctrl[4] || Pwr_ctrl[5] ||  
Pwr_ctrl[6])))
```

For V2:

```
Pwr_enable=0: HOLD=0  
Pwr_enable=1: HOLD=(Pwr_ctrl[0] && Pwr_ctrl[3]) && !shadow_to_careprpg  
Pwr_enable=2: HOLD=((Pwr_ctrl[0] || Pwr_ctrl[1]) && Pwr_ctrl[3])  
    && !shadow_to_careprpg  
Pwr_enable=3: HOLD=(Pwr_ctrl[0]) && !shadow_to_careprpg  
Pwr_enable=4:  
    HOLD=(Pwr_ctrl[0] || ! (Pwr_ctrl[1] || Pwr_ctrl[2] || Pwr_ctrl[3]))  
    && !shadow_to_careprpg  
Pwr_enable=5: HOLD=(Pwr_ctrl[0] || ! (Pwr_ctrl[1] || Pwr_ctrl[2]))  
    && !shadow_to_careprpg
```

```
Pwr_enable=6: HOLD=(Pwr_ctrl[1]||Pwr_ctrl[2]) && !shadow_to_careprpg  
Pwr_enable=7: HOLD=(Pwr_ctrl[0]||Pwr_ctrl[1]||Pwr_ctrl[2])  
    && !shadow_to_careprpg
```

What Next

To correct this problem, edit the netlist or the STL procedure file to identify the failing PRPG output. You can use automated violation analysis to help debug this problem. This analysis displays the PRPG cells that are connected to the failing PRPG output. The displayed pin data is set to the simulated values of the SHIFT procedure with binary values assigned to the connecting PRPG cells. Gates in the fanout cone of the PRPG cells that have binary values are also displayed.

You can use the `-hold_connections` option of the `report_compressors` command to report the connecting PRPG cells.

DRC Rule K4

Message Text

```
DFTMAX LBIST Rule K4: Chain output could not be identified  
  
Output of chain C in compressor S could not be identified.
```

Severity

Fatal Error

Description

Note the following:

c indicates name of failing chain

s indicates name of DFTMAX SEQ CODEC used for failing chain

The nodes corresponding to the outputs of internal chains are inferred based on their defined connections to their associated MISR during single observe mode conditions. This inference is done by performing simulation using the simulated values of the SHIFT procedure, and placing the XTOL_enable cells to their on state, and XTOLPRPG outputs to select the single observe mode of the given chain. A traceback is performed from the data input of the first connecting MISR cell through gates set to X identifying potential chain output cell candidates. If no candidates or more than one candidate can be found, a violation is issued.

What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that the failing chain output can be identified. The automated violation analysis can be used to help debug this problem. This analysis will display the first connecting MISR cell for the failing

chain. The displayed pin data is set to the simulated values of the SHIFT procedure with XTOL_enable cells set on and XTOLPRPG outputs set to the single observe mode of the failing chain. Gates in the traceback cone from the first connecting MISR cell which are set to X are also displayed.

After you pull a K4 violation up in the GSV, start tracing back from the MISR bit on the right. That bit should have an x on its D input (but from the scan chain, not from an X (unknown) driving the scan chain...). 0 and 1 are wrong (bad) values. Following back in the circuit from this bit, you will run into another MISR bit. That is the wrong path. Keep going back on the other path (of the XOR) until you get deep into the XOR tree of the compactor. These XORs lead back to AND or OR gates. These are the masking gates. MOST of these AND/OR gates will have a 0/1 on one leg and an X on the other. The 0s and 1s are BLOCKING the x. The x is from a scan chain. Find the AND/OR gate with a 1/0 on one leg and a 0/1/X on the other. X (as I mentioned) is “good” unless it is from a real UNKNOWN... 0 and 1 on the scan data path are likely from static values or reset signals.

Following the XOR tree back goes to a lot of AND and OR gates (the mask gates, as mentioned). You are looking for the non-XOR gates which have NO “x” on the inputs. Usually, that x is being blocked (a good thing) by the other gate inputs (0 on AND, 1 on OR). You will eventually find an XOR fed by a gate that has all 1s and 0s on the inputs. Or a passing value on most inputs, but an X on the last. This is an “unknown” X value.

In summary: `analyze_violation`; trace XOR tree until you hit some OTHER logic gates; see if these gates are set to BLOCK or PASS a value. If passing a value, that value should be x. If it IS an X (all the way to the initial MISR bit pulled up by `analyze_violation`) then why is it X? If a 1 or 0, why is it 1 or 0 and not x?

DRC Rule K5

Message Text

```
Invalid MISR capture value for no_observe mode
```

```
N MISR cells (P/G) in compressor S had invalid capture data for  
no_observe mode.
```

```
N MISR cells (P/G) in seq compressor S had invalid capture data for  
MASK_ENABLE no_observe mode.
```

Severity

Error

Description

Note the following:

N indicates the number of failing MISR cells

`P` indicates the position of the first failing cell in the MISR

`G` indicates the gate ID of the first failing cell in the MISR

`S` indicates the name of DFTMAX codec associated with the failing MISR

When the `XTOL_ENABLE` cells are set to their on state and the `XTOLPRPG` outputs are set to select the `no_observe` mode, the data inputs of MISR cells are checked to ensure they are successfully blocked (set to 0).

This check performs parallel (32) pattern simulation using the simulated values of the `SHIFT` procedure, and places MISR cells to 0, `XTOL_ENABLE` cells to their on state, and `XTOLPRPG` outputs to select the `no_observe` mode.

Data inputs of MISR cells which are not at 0 for all of the simulated parallel patterns cause a violation of the K5 rule. Only one violation is issued for a given MISR.

If `MASK_ENABLE` cells are used, this check is also performed with `MASK_ENABLE` set to the on state.

Data inputs of MISR cells which are not at their full observe mode value without `xchain` credit for all of the simulated parallel patterns cause a violation of this rule. Only one violation is issued for a given MISR.

If `MASK_ENABLE` cells are used, this check is also performed with `MASK_ENABLE` set to on. Data inputs of MISR cells which are not at their full observe mode value without `xchain` credit and blocked selected chains for all of the simulated parallel patterns cause a violation of this rule. Only one violation is issued for a given MISR.

What Next

To correct this problem, you must change the netlist or the STIL protocol file so that the data input of the failing MISR cell is properly blocked in `no_observe` mode.

You can use automated violation analysis to debug this problem. This analysis will display the first failing cell for the failing MISR.

The displayed pin data is set to the simulated values of the `SHIFT` procedure with MISR cells set to 0, `XTOL_ENABLE` cells set to their on state, and `XTOLPRPG` outputs set to select the `no observe` mode. Gates in the paths between `XTOLPRPG` outputs used to select the `no observe` mode and the failing MISR cell are also displayed.

You can use the `-cells` option of the `report_compressors` command to report the `XTOL_ENABLE` or MISR cells.

You can use the `-lfsr_connections` option of the `report_compressors` command to report the `XTOLPRPG` output values.

You can use the `-mode_data` option of the `report_compressors` command to report the `XTOLPRPG` output values for all observe modes.

You can downgrade this rule violation to a warning. But it is not recommended since it will likely cause incorrect BIST pattern intervals.

DRC Rule K6

Message Text

Invalid MISR capture value for single_observe mode unblocked cell N MISR unblocked cells (P/G1) in compressor S had invalid capture data for single_observe mode of chain C (G2).

N MISR unblocked cells (P/G1) in seq compressor S had invalid capture data for MASK_ENABLE single_observe mode of chain C (G2).

Severity

Error

Description

Note the following:

N indicates the number of failing MISR cells

P indicates the position of the first failing cell in the MISR

G1 indicates the gate ID of the first failing cell in the MISR

S indicates the name of the DFTMAX codec associated with the failing MISR

C indicates the `chain_id` of the chain observed with single observe mode

G2 indicates the gate ID of the chain output

When the `XTOL_ENABLE` cells are set to their `on` state and the `XTOLPRPG` outputs are set to select the single observe mode for a given chain, the data inputs of unblocked MISR cells are checked to ensure they successfully capture data from the chain output.

This check performs parallel (32) pattern simulation using the simulated values of the SHIFT procedure, and places the chain outputs to random values, MISR cells to 0, `XTOL_ENABLE` cells to their `on` state, and `XTOLPRPG` outputs to select the single observe mode for the chain.

The values of `'mask_all_chain_cells'` are set to 0, `'mask_enable'` to OFF and `'xchains_registers'` to X, which are displayed in the GUI.

The data inputs of any unblocked MISR cells which are not at the same value as the chain output (or inverted) for all of the simulated parallel patterns causes a violation of this rule. Only one violation is issued for a given MISR.

What Next

To correct this problem, you must change the netlist or the STIL procedures file so the data input of the failing MISR cell successfully captures data from the chain output in the single observe mode of the failing chain.

You can use automated violation analysis to debug this problem. This analysis displays the first failing cell for the failing MISR.

The displayed pin data is set to the simulated values of the `SHIFT` procedure with MISR cells set to 0, `XTOL_ENABLE` cells set to their on state, and `XTOLPRPG` outputs set to select the no observe mode. Gates in the paths between `XTOLPRPG` outputs used to select the no observe mode and the failing MISR cell are also displayed.

You can use the `-cells` option of the `report_compressors` command to report the `XTOL_ENABLE` or MISR cells.

You can use the `-lfsr_connections` option of the `report_compressors` command to report the `XTOLPRPG` output values.

You can use the `-mode_data` option of the `report_compressors` command to report the `XTOLPRPG` output values for all observe modes

Understanding K6 check With Passing Testcase

If you have passing testcase, you can perform the following steps with passing case to debug and fix the problems.

1. Before 'run_drc', use this command to save the simulation values of failing chain.

```
set_drc -store_k6_data <chain_id_of_failing_chain>
```

Note:

If you want to know the `chain_id` of any chain, you can use the following command

```
report_compressors -chain_data -all
```

2. To display the stored value of failed chain, use the following command.

```
set_pindata -debug_sim_data
```

Now , you have the simulation value of passing case, you can easily compare it with failing case by using following command to debug/fix the issue.

```
analyze_violation k6-<N>
```

`N` can be the error number in the failing message.

For example - `analyze_violation k6-1`

You can downgrade this rule violation to a warning. But this is not recommended since it likely causes incorrect BIST pattern intervals

You can use the `-mode_data` option of the `report_compressors` command to report the XTOLPRPG output values for all observe modes.

DRC Rule K7

Message Text

```
DFTMAX LBIST Rule K7: Invalid MISR capture value for single_observe mode  
blocked cell
```

```
N MISR blocked cells (P/G1) in compressor S had invalid capture data for  
single_observe mode of chain C (G2).
```

Severity

Error

Description

Note the following:

`N` indicates number of failing MISR cells

`P` indicates position of first failing cell in the MISR

`G1` indicates gate ID of first failing cell in the MISR

`S` indicates name of DFTMAX CODEC associated with failing MISR

`C` indicates name of chain observed with single observe mode

`G2` indicates gate ID of chain output

When the XTOL_enable cells are set to their on state and the XTOLPRPG outputs are set to select the single observe mode for a given chain, the data inputs of blocked MISR cells are checked to ensure that they are successfully blocked (set to 0). This checking is done by performing parallel (32) pattern simulation using the simulated values of the SHIFT procedure, and placing the chain outputs to random values, MISR cells to 0, XTOL_enable cells to their on state, and XTOLPRPG outputs to select the single observe mode for the desired chain. Data inputs of blocked MISR cells which are not at 0 for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given MISR.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that the data input of the failing MISR cell is properly blocked in the single observe mode of the

failing chain. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing MISR. The displayed pin data is set to the simulated values of the SHIFT procedure with chain outputs set to a random binary value, MISR cells set to 0, XTOL_enable cells set to their on state, and XTOLPRPG outputs set to select the single observe mode for the chain. Gates in the paths between XTOLPRPG outputs used to select the single observe mode and the failing MISR cell are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable or MISR cells.

The `-lfsr_connections` option of the `report_compressors` command can be used to report the XTOLPRPG outputs.

The `-mode_data` option of the `report_compressors` command can be used to report the XTOLPRPG output values for all observe modes.

The `-chain_modes` option of the `report_compressors` command can be used to report the XTOLPRPG output values for the single observe mode for chains.

The `-lfsr_connections` option of the `report_compressors` command can be used to report the connections between chains and the MISR.

DRC Rule K8

Message Text

DFTMAX LBIST Rule K8: Different chain output to MISR inversion parity

Cells P1/G1 and P2/G2 of MISR in compressor S had different inversion parity for single_observe mode of chain C (G3)

Severity

Error

Description

Note the following:

P1 indicates position of first cell of failing pair in the MISR

G1 indicates gate ID of first cell of failing pair in the MISR

P2 indicates position of second cell of failing pair in the MISR

G2 indicates gate ID of second cell of failing pair in the MISR

S indicates name of DFTMAX CODEC associated with failing MISR

C indicates name of chain observed with single observe mode

G3 indicates gate ID of chain output

When the XTOL_enable cells are set to their on state and the XTOLPRPG outputs are set to select the single observe mode for a given chain, the inversion parity between data inputs of unblocked MISR cells and the chain output are checked to ensure they are all the same. This checking is done by performing parallel (32) pattern simulation using the simulated values of the SHIFT procedure, and placing the chain outputs to random values, MISR cells to 0, XTOL_enable cells to their on state, and XTOLPRPG outputs to select the single observe mode for the desired chain. Any 2 data inputs of unblocked MISR cells which have differing inversion parity will result in a violation of this rule.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that the data inputs of the failing MISR cells have the same inversion parity in the single observe mode of the failing chain. The automated violation analysis can be used to help debug this problem. This analysis will display the 2 failing cells for the failing MISR. The displayed pin data is set to the simulated values of the SHIFT procedure with chain outputs set to a random binary value, MISR cells set to 0, XTOL_enable cells set to their on state, and XTOLPRPG outputs set to select the single observe mode for the chain. Gates in the paths between the chain output and the failing MISR cells are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable or MISR cells.

The `-lfsr_connections` option of the `report_compressors` command can be used to report the XTOLPRPG outputs.

The `-mode_data` option of the `report_compressors` command can be used to report the XTOLPRPG output values for all observe modes.

The `-chain_modes` option of the `report_compressors` command can be used to report the XTOLPRPG output values for the single observe mode for chains.

The `-lfsr_connections` option of the `report_compressors` command can be used to report the connections between chains and the MISR.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause incorrect patterns for all observe modes.

DRC Rule K9

Message Text

Invalid MISR capture value for full_observe mode

N MISR cells (P/G) in compressor S had invalid capture data for full_observe mode.

`N MISR cells (P/G) in seq compressor S had invalid capture data for MASK_ENABLE full_observe mode.`

`N MISR cells (P/G) in seq compressor S had invalid capture data for MASK_ENABLE xt看_enable_off full_observe mode.`

Severity

Error

Description

Note the following:

`N` indicates the number of failing MISR cells

`P` indicates the position of the first failing cell in the MISR

`G` indicates the gate ID of the first failing cell in the MISR

`S` indicates the name of the DFTMAX codec associated with the failing MISR

When the XTOL_ENABLE cells are set to their on state and the XTOLPRPG outputs are set to select the full observe mode, the data inputs of MISR cells are checked to ensure they successfully capture data from their connecting chains.

This check performs parallel (32) pattern simulation using the simulated values of the SHIFT procedure, and places the chain outputs to random values, MISR cells to 0, XTOL_ENABLE cells to their on state, and XTOLPRPG outputs to select the full observe mode.

Data inputs of MISR cells which are not at their expected value for all of the simulated parallel patterns cause a violation of this rule. The expected value is calculated by exclusively applying the OR operator to the values of all chain outputs connected to a MISR cell input with consideration given to their precalculated inversion parity. Only one violation is issued for a given MISR.

If MASK_ENABLE cells are used, this check is also performed with MASK_ENABLE set to on. Data inputs of MISR cells which are not at their full observe mode value with X-chain credit for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given MISR. In addition, this check is done with MASK_ENABLE set to on and XTOL_ENABLE set to off. Data inputs of MISR cells which are not at their full observe mode value without X-chain credit for all simulated parallel patterns cause a violation of this rule. Only one violation is issued for a given MISR.

If the previous check passes and the XTOLPRPG outputs are not all initialized to binary values during the TEST_SETUP procedure, the full_observe capture mode is also checked to ensure it works correctly when the XTOL_ENABLE cells are set to their off state and the XTOLPRPG outputs are set to X. When a failure occurs, the text of the K9 violation message is changed to indicate global full_observe mode.

If there are no failures due to either of the previous checks, the full-observe capture mode is also checked for each X-chain to ensure it works correctly when the XTOL_ENABLE cells are set to their on state and the XTOLPRPG outputs are set to select the single observe mode for that X-chain.

When a failure occurs, the text of the K9 violation message is changed to indicate global full_observe mode and also report the name of a failing X-chain.

What Next

To correct this problem, you must change the netlist or the STIL protocol file so that the data input of the failing MISR cell properly captures data from all of its chain connections in full_observe mode.

You can use automated violation analysis to debug this problem. This analysis displays the first failing cell for the failing MISR. Gates in the paths between XTOLPRPG outputs used to select the full observe mode and the failing MISR cell are also displayed.

The displayed pin data is set to one of the following:

Type 1 - The simulated values of the SHIFT procedure with chain outputs set to a random binary value, MISR cells set to 0, XTOL_ENABLE cells set to their on state, and XTOLPRPG outputs set to select the full observe mode.

Type 2 - The simulated values of the SHIFT procedure with chain outputs set to a random binary value, MISR cells set to 0, XTOL_ENABLE cells set to their off state, and XTOLPRPG outputs set to X. A frequent cause of this failure type is the usage of complex synthesized circuitry which exploit X and X-BAR effects to force the full observe mode when XTOL_ENABLE is set off. This can be corrected by either initializing the XTOLPRPG outputs in the TEST_SETUP procedure or running synthesis using options that avoid this problem.

Type 3 - The simulated values of the SHIFT procedure with chain outputs set to a random binary value, MISR cells set to 0, XTOL_ENABLE cells set to their off state, and XTOLPRPG outputs set to select the single observe mode of the failing X-chain.

You can use the `-cells` option of the `report_compressors` command to report the XTOL_ENABLE or MISR cells.

You can use the `-lfsr_connections` option of the `report_compressors` command to report the XTOLPRPG output values.

You can use the `-mode_data` option of the `report_compressors` command to report the XTOLPRPG output values for all observe modes.

You can downgrade this rule violation to a warning. But this is not recommended since it will likely cause incorrect BIST pattern intervals.

DRC Rule K10

Message Text

Invalid MISR capture value for multiple_observe mode

N MISR cells (P/G) in compressor S had invalid capture data for F multiple_observe mode.

N MISR cells (P/G) in compressor S had invalid capture data for MASK_ENABLE F multiple_observe mode.

Severity

Error

Description

Note the following:

N indicates the number of failing MISR cells

P indicates the position of the first failing cell in the MISR

G indicates the gate ID of the first failing cell in the MISR

S indicates the name of the DFTMAX codec associated with failing MISR

F indicates the fraction of scan cells observed by multiple_observe mode

When the XTOL_ENABLE cells are set to their on state and the XTOLPRPG outputs are set to select a multiple observe mode, the data inputs of MISR cells are checked to ensure that they successfully capture data from their connecting chains.

This check performs parallel (32) pattern simulation using the simulated values of the SHIFT procedure, and places the chain outputs to random values, MISR cells to 0, XTOL_ENABLE cells to their on state, and XTOLPRPG outputs to select the multiple_observe mode.

Data inputs of MISR cells which are not at their expected value for all of the simulated parallel patterns cause a violation of this rule. The expected value is calculated by exclusively applying the OR operator to the values of all chain outputs connected to a MISR cell input with consideration given to their precalculated inversion parity. Only one violation is issued for a given MISR.

If MASK_ENABLE cells are used, this check is also performed with MASK_ENABLE set to on. Data inputs of MISR cells which are not at their correct value, assuming all chains are observed except selected chains which are also dynamic X-chains for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given MISR.

What Next

To correct this problem, you must change the netlist or the STIL protocol file so that the data input of the failing MISR cell properly captures data from all of its chain connections in the multiple observe mode.

You can use automated violation analysis to debug this problem. This analysis displays the first failing cell for the failing MISR. Gates in the paths between XTOLPRPG outputs used to select the full observe mode and the failing MISR cell are also displayed.

You can use the `-cells` option of the `report_compressors` command to report the XTOL_ENABLE or MISR cells.

You can use the `-lfsr_connections` option of the `report_compressors` command to report the XTOLPRPG output values.

You can use the `-mode_data` option of the `report_compressors` command to report the XTOLPRPG output values for all observe modes.

You can downgrade this rule violation to a warning. But this is not recommended since it will likely cause incorrect BIST pattern intervals.

DRC Rule K11

Message Text

```
Invalid MISR capture value for complement multiple_observe mode
```

```
N MISR cells (P/G) in compressor S had invalid capture data for F  
multiple_observe mode.
```

```
N MISR cells (P/G) in seq compressor S had invalid capture data for  
MASK_ENABLE F complement multiple_observe mode.
```

Severity

Error

Description

Note the following:

N indicates the number of failing MISR cells

P indicates the position of the first failing cell in the MISR

G indicates the gate ID of the first failing cell in the MISR

S indicates the name of the DFTMAX codec associated with the failing MISR

F indicates the fraction of scan cells observed by multiple_observe mode

When the XTOL_ENABLE cells are set to their on state and the XTOLPRPG outputs are set to select a complement multiple observe mode, the data inputs of MISR cells are checked to ensure they successfully capture data from their connecting chains.

This check performs parallel (32) pattern simulation using the simulated values of the SHIFT procedure, and places the chain outputs to random values, MISR cells to 0, XTOL_ENABLE cells to their on state, and XTOLPRPG outputs to select the multiple observe mode.

Data inputs of MISR cells which are not at their expected value for all of the simulated parallel patterns cause a violation of this rule. The expected value is calculated by exclusively applying the OR operator to the values of all chain outputs connected to a MISR cell input with consideration given to their precalculated inversion parity. Only one violation is issued for a given MISR.

If MASK_ENABLE cells are used, this check is also performed with MASK_ENABLE set to on.

Data inputs of MISR cells which are not at their correct value, assuming all chains are observed except selected chains which are also dynamic X-chains for all of the simulated parallel patterns, cause a violation of this rule. Only one violation is issued for a given MISR.

What Next

To correct this problem, change the netlist or the STL protocol file so that the data input of the failing MISR cell properly captures data from all of its chain connections in the complement multiple_observe mode.

This check performs parallel (32) pattern simulation using the simulated values of the SHIFT procedure, and places the chain outputs to random values, MISR cells to 0, XTOL_ENABLE cells to their on state, and XTOLPRPG outputs to select the multiple observe mode.

Data inputs of MISR cells which are not at their expected value for all of the simulated parallel patterns cause a violation of this rule. The expected value is calculated by exclusively ORing the values for all the chain outputs that are connected to a MISR cell input with consideration given to their precalculated inversion parity. Only one violation is issued for a given MISR.

You can use automated violation analysis to debug this problem. This analysis displays the first failing cell for the failing MISR. Gates in the paths between XTOLPRPG outputs used to select the full observe mode and the failing MISR cell are also displayed.

You can use the `-cells` option of the `report_compressors` command to report the XTOL_ENABLE or MISR cells.

You can use the `-lfsr_connections` option of the `report_compressors` command to report the XTOLPRPG output values.

You can use the `-mode_data` option of the `report_compressors` command to report the XTOLPRPG output values for all observe modes.

You can downgrade this rule violation to a warning. But this is not recommended since it will likely cause incorrect BIST pattern intervals.

DRC Rule K12

Message Text

```
DFTMAX LBIST Rule K12: Invalid MISR capture shift operation
```

```
N MISR cells (input I of P/G) in compressor S had invalid shift values  
during X procedure.
```

```
Severity:Error
```

Severity

Error

Description

Note the following:

`N` indicates number of failing MISR cells

`P` indicates position of first failing cell in the MISR

`G` indicates gate ID of first failing cell in the MISR

`S` indicates name of DFTMAX CODEC associated with failing MISR

`F` indicates fraction of scancells observed by `multiple_observe` mode

`X` indicates name of procedure

When the `XTOL_enable` and `Power_enable` cells are set to their off state, the MISR cells are checked to ensure that they are successfully capturing data as expected for full observe mode during the `SHIFT` and `RESEED_OVERLAP_SHIFT` procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the chain outputs to random values, `XTOL_enable` and `Power_enable` cells to their off state, and all other register types to random values. MISR cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected value is calculated by exclusiving-ORing the values for all the chain outputs that are connected to a MISR cell input with consideration given to their precalculated inversion parity and also for MISR tapping. Only one violation is issued for a given MISR.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that MISR cells are properly capturing data in full observe mode as controlled directly by the XTOL_enable cells during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing MISR. If the failure occurred on a clock/set/reset input, then the pindata is the simulated value for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the paths between connecting chains and the failing MISR cell are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable, Power_enable, or MISR cells.

The `-lfsr_connections` option of the `report_compressors` command can be used to report the connections between chains and the MISR.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns which exercise this MISR capture shift operation to be incorrect.

DRC Rule K13

Message Text

DFTMAX LBIST Rule K13: Invalid MISR unload operation

N MISR cells (input I of P/G) in compressor S had invalid unload value during X procedure.

Severity

Error

Description

Note the following:

N indicates number of failing MISR cells

I indicates input ID of first failing cell in the MISR

P indicates position of first failing cell in the MISR

G indicates gate ID of first failing cell in the MISR

S indicates name of DFTMAX CODEC associated with failing MISR

X indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the MISR cells are checked to ensure that they are successfully unloading data during the RESEED_WITH_MISR_SHIFT procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the chain outputs to random values, XTOL_enable and Power_enable cells to their off state, and all other register types to random values. MISR cells which do not successfully unload their value for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given MISR.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that MISR cells are properly unloading data during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing MISR. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing MISR cell and the its adjacent cell are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable, Power_enable, or MISR cells

The `-lfsr_connections` option of the `report_compressors` command can be used to report the connections between chains and the MISR.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns which exercise this MISR capture shift operation to be incorrect.

DRC Rule K14

Message Text

DFTMAX LBIST Rule K14: Invalid MISR load input

N MISR cells (P/G) in compressor S had invalid load input value during X procedure.

Severity

Error

Description

Note the following:

N indicates number of failing MISR cells

P indicates position of first failing cell in the MISR

G indicates gate ID of first failing cell in the MISR

S indicates name of DFTMAX CODEC associated with failing MISR

X indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the MISR cells at the beginning of an unload segment are checked to ensure that they are successfully resetting the MISR to 0 during the RESEED_WITH_MISR_SHIFT procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the chain outputs to random values, XTOL_enable and Power_enable cells to their off state, and all other register types to random values. MISR cells at the beginning of a segment which do not successfully capture a 0 for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given MISR.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that MISR input ports are properly propagating values to the MISR during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing MISR. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing MISR input port and the its associated MISR cell are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable, Power_enable, or PRPG_shadow cells.

The `-chain_segments` option of the `report_compressors` command can be used to report the unload segments of the MISRs.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns which exercise this MISR reset operation to be incorrect.

DRC Rule K15

Message Text

DFTMAX LBIST Rule K15: Invalid MISR unload output

MISR output O value not equal MISR cell (P/G) in compressor S during X procedure

Severity

Error

Description

Note the following:

- indicates name of MISR output port
- P indicates position of first failing cell in the MISR
- G indicates gate ID of first failing cell in the MISR
- S indicates name of DFTMAX CODEC associated with failing MISR
- X indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the MISR output ports are checked to ensure that they are successfully unloading data during the RESEED_WITH_MISR_SHIFT procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the chain outputs to random values, XTOL_enable cells and Power_enable to their off state, and all other register types to random values. MISR output ports which do not successfully have the same value as their associated MISR cell for all of the simulated parallel patterns will result in a violation of this rule.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that MISR output ports are properly receiving values from the MISR during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing cell for the failing MISR. The displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing MISR output port and its associated MISR cell are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable, Power_enable, or PRPG_shadow cells

The `-chain_segments` option of the `report_compressors` command can be used to report the output ports of the MISR.

This rule violation can be downgraded to warning but it is not recommended since this likely causes patterns which exercise this MISR unload operation to be incorrect.

DRC Rule K16

Message Text

DFTMAX LBIST Rule K16: MISR cell disturb

N MISR cells (P/G) in compressor S were disturbed during X procedure.

Severity

Error

Description

Note the following:

N indicates number of failing MISR cells

P indicates position of first failing cell in the MISR

G indicates gate ID of first failing cell in the MISR

S indicates name of DFTMAX CODEC associated with failing MISR

X indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the MISR cells are checked to ensure that they hold state during the RESEED_SHIFT, SHADOW_TO_CAREPRPG, SHADOW_TO_XTOLPRPG, and OCC_SHIFT procedures and also during the capture cycles. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the chain outputs to random values, XTOL_enable and Power_enable cells to their off state, and all other register types to random values. MISR cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given MISR.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that MISR cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing MISR with all the gates in the traceback cone of the input which was believed to have caused the problem. If the failing procedure was the capture procedure, then the displayed pin data is set to one of the parallel pattern values that failed. Otherwise, the pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable, Power_enable, or PRPG_shadow cells.

This rule violation can be downgraded to warning but it is not recommended since this likely causes patterns which exercise this MISR unload operation to be incorrect.

DRC Rule K17

Message Text

DFTMAX LBIST Rule K17: Invalid PRPG_shadow load operation

N PRPG_shadow cells (input I of P/G) in compressor S had invalid value during X procedure.

Severity

Error

Description

Note the following:

N indicates number of failing PRPG_shadow cells

I indicates input ID of first failing cell in the PRPG_shadow register

P indicates position of first failing cell in the PRPG_shadow register

G indicates gate ID of first failing cell in the PRPG_shadow register

S indicates name of DFTMAX CODEC associated with the failing PRPG_shadow register

X indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the PRPG_shadow cells are checked to ensure that they are successfully loading data during the RESEED_WITH_MISR_SHIFT, RESEED_SHIFT, and RESEED_OVERLAP_SHIFT procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the PRPG_shadow input ports to random values, XTOL_enable and Power_enable cells to their off state, and all other register types to random values. Prpg_shadow cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected value for cells which connect to PRPG_shadow input ports are the values assigned to the input port (or inverted). The expected values for all other cells are the values assigned to the adjacent cells. Only one violation is issued for a given PRPG_shadow.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that PRPG_shadow cells are properly capturing data during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing PRPG_shadow register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable, Power_enable, or PRPG_shadow cells

The `-chain_segments` option of the `report_compressors` command can be used to report the unload segments of the MISRs.

This rule violation can be downgraded to warning but it is not recommended since this likely causes patterns which exercise the reseed operation to be incorrect.

DRC Rule K18

Message Text

DFTMAX LBIST Rule K18: PRPG_shadow cell disturb

N PRPG_shadow cells (P/G) in compressor S were disturbed during X procedure.

Severity

Error

Description

Note the following:

N indicates number of failing MISR cells

P indicates position of first failing cell in the MISR

G indicates gate ID of first failing cell in the MISR

S indicates name of DFTMAX CODEC associated with failing MISR

X indicates name of procedure

This rule is not currently checked.

What Next

No action required.

DRC Rule K19

Message Text

DFTMAX LBIST Rule K19: Invalid CarePRPG shift operation

N CarePRPG cells (input I of P/G) in compressor S had invalid shift values during X procedure.

Severity

Error

Description

Note the following:

N indicates number of failing CarePRPGcells

I indicates input ID of first failing cell in the CarePRPG

P indicates position of first failing cell in the CarePRPG

G indicates gate ID of first failing cell in the CarePRPG

S indicates name of DFTMAX CODEC associated with failing CarePRPG

X indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the CarePRPG cells are checked to ensure that they are successfully shifting data during the SHIFT and RESEED_OVERLAP_SHIFT procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL_enable and Power_enable cells to their off state, and all other register types to random values. CarePRPG cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected value for cells considers the PRPG definition with tapping. Only one violation is issued for a given CarePRPG.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that CarePRPG cells are properly shifting during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing CarePRPG register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the sources of its capture data are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable, Power_enable, or CarePRPG cells.

This rule violation can be downgraded to warning but it is not recommended since this will cause patterns which exercise the scan chain load operation from PRPGs to be incorrect.

DRC Rule K20

Message Text

DFTMAX LBIST Rule K20: Invalid CarePRPG reload operation

Seq compressor chain length L1 did not equal expected length L2.

Severity

Error

Description

Note the following:

N indicates number of failing CarePRPGcells

I indicates input ID of first failing cell in the CarePRPG

P indicates position of first failing cell in the CarePRPG

G indicates gate ID of first failing cell in the CarePRPG

S indicates name of DFTMAX CODEC associated with failing CarePRPG

X indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the CarePRPG cells are checked to ensure that they are successfully capturing reseed data from the PRPG_shadow during the SHADOW_TO_CAREPRPG procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL_enable and Power_enable cells to their off state, and all other register types to random values. CarePRPG cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected values for cells are determined by the values from the associated cell in the PRPG_shadow. Only one violation is issued for a given CarePRPG.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that CarePRPG cells are properly capturing data during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing CarePRPG register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable, Power_enable, or CarePRPG cells.

This rule violation can be downgraded to warning but it is not recommended since this likely causes patterns which exercise the scan chain load operation from PRPGs to be incorrect.

DRC Rule K21

Message Text

DFTMAX LBIST Rule K21: CarePRPG cell disturb

N CarePRPG cells (P/G) in compressor S were disturbed during X procedure.

Severity

Error

Description

Note the following:

N indicates number of failing CarePRPGcells

P indicates position of first failing cell in the CarePRPG

G indicates gate ID of first failing cell in the CarePRPG

S indicates name of DFTMAX CODEC associated with failing CarePRPG

X indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the CarePRPG cells are checked to ensure that they hold state during the RESEED_WITH_MISR_SHIFT, RESEED_SHIFT, and SHADOW_TO_XTOLPRPG procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the chain outputs to random values, XTOL_enable and Power_enable cells to their off state, and all other register types to random values. CarePRPG cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given CarePRPG.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that CarePRPG cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing CarePRPG with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable, Power_enable, or CarePRPG cells.

This rule violation can be downgraded to warning but it is not recommended since this likely causes patterns which exercise the scan chain load operation from PRPGs to be incorrect.

DRC Rule K22

Message Text

DFTMAX LBIST Rule K22: Invalid XTOLPRPG shift operation

(input I of P/G) in compressor S had invalid shift values during X
procedure.

Severity

Error

Description

Note the following:

N indicates number of failing XTOLPRPG

I indicates input ID of first failing cell in the XTOLPRPG

P indicates position of first failing cell in the XTOLPRPG

G indicates gate ID of first failing cell in the XTOLPRPG

S indicates name of DFTMAX CODEC associated with failing XTOLPRPG

X indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the XTOLPRPG cells are checked to ensure that they are successfully shifting data during the SHIFT and RESEED_OVERLAP_SHIFT procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL_enable and Power_enable cells to their off state, and all other register types to random values. XTOLPRPG cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected value for cells considers the PRPG definition with tapping. Only one violation is issued for a given XTOLPRPG.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that XTOLPRPG cells are properly shifting during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing XTOLPRPG register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed, and gates in the path between the failing cell and the sources of its capture data are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable, Power_enable, or XTOLPRPG cells.

This rule violation can be downgraded to warning but it is not recommended since this likely causes patterns which exercise the observe mode control during the MISR capture operation from internal chains to be incorrect.

DRC Rule K23

Message Text

Invalid XTOLPRPG reload operation

N XTOLPRPG cells (input I of P/G) in seq compressor S had invalid shift values during X procedure.

Severity

Error

Description

Note the following:

N indicates the number of failing XTOLPRPG

I indicates the input ID of first failing cell in the XTOLPRPG

P indicates the position of the first failing cell in the XTOLPRPG

G indicates the gate ID of the first failing cell in the XTOLPRPG

S indicates the name of the DFTMAX codec associated with the failing XTOLPRPG

X indicates the name of the procedure

When the XTOL_ENABLE and POWER_ENABLE cells are set to their off state, the XTOLPRPG cells are checked to ensure that they successfully capture reseed data from the PRPG_SHADOW during the SHADOW_TO_XTOLPRPG procedure.

This check performs parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL_ENABLE and POWER_ENABLE cells to their off state, and all other register types to random values.

CarePRPG cells which do not successfully capture their expected value for all of the simulated parallel patterns cause a violation of this rule. The expected values for cells are determined by the values from the associated cell in the PRPG_SHADOW. Only one violation is issued for a given XTOLPRPG.

If MASK_ENABLE cells are used, the XTOLPRPG cells are checked to ensure they successfully capture reseed data from the PRPG_SHADOW during the SHIFT procedure.

What Next

To correct this problem, you must change the netlist or the STIL protocol file so that XTOLPRPG cells properly capture data during the specified procedure.

You can use automated violation analysis to debug this problem. This analysis displays the first failing cell of the failing XTOLPRPG register.

If the failure occurs on a clock/set/reset input, the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

You can use the `-cells` option of the `report_compressors` command to report the XTOL_ENABLE, POWER_ENABLE, or XTOLPRPG cells.

You can downgrade this rule violation to a warning. But it is not recommended since this will likely cause patterns which exercise the observe mode control during the MISR capture operation from internal chains to be incorrect.

DRC Rule K24

Message Text

DFTMAX LBIST Rule K24: XTOLPRPG cell disturb

`N XTOLPRPG cells (P/G) in compressor S were disturbed during X procedure.`

Severity

Error

Description

Note the following:

`N` indicates number of failing XTOLPRPG

`P` indicates position of first failing cell in the XTOLPRPG

`G` indicates gate ID of first failing cell in the XTOLPRPG

`S` indicates name of DFTMAX CODEC associated with failing XTOLPRPG

`X` indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the XTOLPRPG cells are checked to ensure that they hold state during the RESEED_WITH_MISR_SHIFT, RESEED_SHIFT, and SHADOW_TO_CAREPRPG procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the

appropriate procedure, and placing the chain outputs to random values, XTOL_enable and Power_enable cells to their off state, and all other register types to random values. CarePRPG cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given XTOLPRPG.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that XTOLPRPG cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing XTOLPRPG with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable, Power_enable, or XTOLPRPG cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns which exercise the observe mode control during the MISR capture operation from internal chains to be incorrect.

DRC Rule K25

Message Text

N CARE_shadow cells (input I of P/G) in seq compressor S had invalid shift values during X procedure. where:

N indicates number of failing CARE_shadow cells

I indicates input ID of first failing cell in the CARE_shadow register

P indicates position of failing cell in the CARE_shadow register

G indicates gate ID of failing cell in the CARE_shadow register

S indicates name of DFTMAX Seq CODEC associated with failing CARE_shadow register

X indicates name of procedure

Severity

Error

Description

When the XTOL_enable and Power_enable cells are set to their off state, the CARE_shadow cells are checked to ensure that they are successfully shifting data during the SHIFT and RESEED_OVERLAP_SHIFT procedures. This checking is done by performing

parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL_enable and Power_enable cells to their off state, and all other register types to random values. CARE_shadow cells that do not successfully capture their expected value for all of the simulated parallel patterns results in a violation of this rule. The expected values for a CARE_shadow cell are the values assigned to the associated cell in the CarePRPG. If the CODEC has Low_Power v3, the expected values for a CARE_Shadow cell are ANDing values of the values assigned to the associated CAREPRPG and the silencer_DLAT. Only one violation is issued for a given CARE_shadow.

What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that CARE_shadow cells are properly shifting during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis displays the first failing cell of the failing CARE_shadow register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data is also displayed.

The -cells option of the report_compressors command can be used to report the XTOL_enable, Power_enable, or CARE_shadow cells.

This rule violation may be downgraded to a warning, but it is not recommended because this may cause patterns which exercise the internal chain load operation from PRPGs to be incorrect.

DRC Rule K26

Message Text

N CARE_shadow cells (P/G) in seq compressor S failed hold during X procedure.

where:

N indicates number of failing CARE_shadow cells

P indicates position of failing cell in the CARE_shadow register

G indicates gate ID of failing cell in the CARE_shadow register

S indicates name of DFTMAX Seq CODEC associated with failing CARE_shadow register

X indicates name of procedure

Severity

Error

Description

When the `Power_enable` cell is set to its on state, `silencer_DLATs` (if the CODEC has `Low_Power v3` hardware) are set to HI and the hold ability is activated for `CarePRPGs`, the `CARE_shadow` cells are checked to ensure that they are successfully holding state during the `SHIFT` and `RESEED_OVERLAP_SHIFT` procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing `Power_enable` to its on state, activating the hold ability of the `CarePRPG`, and placing all other register types to random values. `CARE_shadow` cells which do not successfully hold their value for all of the simulated parallel patterns results in a violation of this rule. Only one violation is issued for a given `CARE_shadow`.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that `CARE_shadow` cells are properly holding values during the specified procedure under hold conditioning. The automated violation analysis can be used to help debug this problem. This analysis displays the first failing cell of the failing `CARE_shadow` register. If the failure was believed to be due to a problem on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the hold control sources are also displayed. The hold control sources are the `Power_enable` cell and the cells of the `CarePRPG` which are used to create the hold signal.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_enable`, `Power_enable`, or `CARE_shadow` cells.

The `-hold_connections` option of the `report_compressors` command can be used to report the `CarePRPG` cells which are used to create the hold signal.

This rule violation can be downgraded to warning. Since the hold ability is not currently used during test generation, this does not create any problem with current patterns.

DRC Rule K27

Message Text

N `CARE_shadow` cells (input I of P/G) in seq compressor S had invalid value during X procedure.

where:

N indicates number of failing `CARE_shadow` cells

I indicates input ID of first failing cell in the CARE_shadow register

P indicates position of failing cell in the CARE_shadow register

G indicates gate ID of failing cell in the CARE_shadow register

S indicates name of DFTMAX Seq CODEC associated with failing CARE_shadow register

X indicates name of procedure

Severity

Error

Description

When the XTOL_enable and Power_enable cells are set to their off state, the CARE_shadow cells are checked to ensure that they are successfully loading data from the CarePRPG during the SHADOW_TO_CAREPRPG procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL_enable and Power_enable cells to their off state, and all other register types to random values. CARE_shadow cells which do not successfully capture their expected value for all of the simulated parallel patterns results in a violation of this rule. The expected values for a CARE_shadow cell are the value assigned to the associated cell in the CarePRPG. If the CODEC has Low_Power v3, the expected values for a CARE_Shadow cell are ANDing values of values assigned to the associated CAREPRPG and the silencer_DLAT. Only one violation is issued for a given CARE_shadow.

What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that CARE_shadow cells are properly capturing data from its CarePRPG during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis displays the first failing cell of the failing CARE_shadow register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the associated cell in the CarePRPG are also displayed.

The -cells option of the report_compressors command can be used to report the XTOL_enable, Power_enable, CarePRPG, or CARE_shadow cells.

This rule violation may be downgraded to warning but it is not recommended since this likely causes patterns which exercise the CarePRPG reseed operation to be incorrect.

DRC Rule K28

Message Text

DFTMAX LBIST Rule K2: CARE_shadow cell disturb

N CARE_shadow cells (P/G) in compressor S were disturbed during X procedure..

Severity

Error

Description

Note the following:

N indicates number of failing CARE_shadow cells

P indicates position of first failing cell in the CARE_shadow register

G indicates gate ID of first failing cell in the CARE_shadow register

S indicates name of DFTMAX CODEC associated with failing CARE_shadow register

X indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the CARE_shadow register cells are checked to ensure that they hold state during the SHADOW_TO_XTOLPRPG procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the chain outputs to random values, XTOL_enable and Power_enable cells to their off state, and all other register types to random values. CARE_shadow cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given CARE_shadow.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that CARE_shadow cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing CARE_shadow register with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report_compressors command can be used to report the XTOL_enable, Power_enable, or CARE_shadow cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns which exercise the CarePRPG reseed operation to be incorrect.

DRC Rule K29

Message Text

DFTMAX LBIST Rule K29: Invalid XTOLShadow shift operation

N XTOLShadow cells (input I of P/G) in compressor S had invalid shift values during X procedure.

Severity

Error

Description

Note the following:

N indicates number of failing XTOLShadow cells

I indicates input ID of first failing cell in the XTOLShadow register

P indicates position of first failing cell in the XTOLShadow register

G indicates gate ID of first failing cell in the XTOLShadow register

S indicates name of DFTMAX CODEC associated with failing XTOLShadow register

X indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the XTOLShadow cells are checked to ensure that they are successfully shifting data during the SHIFT and RESEED_OVERLAP_SHIFT procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL_enable and Power_enable cells to their off state, and all other register types to random values. XTOLShadow cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected values for an XTOLShadow cell are the values assigned to the associated cell in the XTOLPRPG. Only one violation is issued for a given XTOLShadow.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that CARE_shadow cells are properly shifting during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing CARE_shadow register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable, Power_enable, or XTOLShadow cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K30

Message Text

```
DFTMAX LBIST Rule K30: Invalid XTOLShadow hold operation
```

```
N XTOLShadow cells (P/G) in compressor S failed hold during X procedure.
```

Severity

Error

Description

Note the following:

`N` indicates number of failing XTOLShadow cells

`P` indicates position of first failing cell in the XTOLShadow register

`G` indicates gate ID of first failing cell in the XTOLShadow register

`S` indicates name of DFTMAX CODEC associated with failing XTOLShadow register

`X` indicates name of procedure

When the XTOL_enable cells are set to their on state and the hold ability is activated for x-tolerance control, the XTOLShadow cells are checked to ensure that they are successfully holding state during the SHIFT and RESEED_OVERLAP_SHIFT procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL_enable cells to their on state, activating the hold ability for x-tolerance control, and placing all other register types to random values. XTOLShadow cells which do not successfully hold their value for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given XTOLShadow.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that XTOLShadow cells are properly holding state during the specified procedure under hold conditioning. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing XTOLShadow register. If the failure was believed to be due to a problem on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback

cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the hold control sources are also displayed. The hold control sources are the Power_enable cell and the cells of the XTOLPRPG which are used to create the hold signal.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable, Power_enable, or XTOLShadow cells.

The `-hold_connections` option of the `report_compressors` command can be used to report the XTOLPRPG cells which are used to create the hold signal.

The `-mode_data` option of the `report_compressors` command can be used to report the XTOLPRPG output values used for the hold observe mode.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K31

Message Text

```
DFTMAX LBIST Rule K31: Invalid XTOLShadow reload operation
```

```
N XTOLShadow cells (input I of P/G) in compressor S had invalid value
during X procedure.
```

Severity

Error

Description

Note the following:

`N` indicates number of failing XTOLShadow cells

`I` indicates input ID of first failing cell in the XTOLShadow register

`P` indicates position of first failing cell in the XTOLShadow register

`G` indicates gate ID of first failing cell in the XTOLShadow register

`S` indicates name of DFTMAX CODEC associated with failing XTOLShadow register

`X` indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the XTOLShadow cells are checked to ensure that they are successfully loading data from the XTOLPRPG during the SHADOW_TO_XTOLPRPG procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL_enable and Power_enable cells to their off

state, and all other register types to random values. XTOLShadow cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected values for an XTOLShadow cell are the values assigned to the associated cell in the XTOLPRPG. Only one violation is issued for a given XTOLShadow.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that XTOLShadow cells are properly capturing data from its XTOLPRPG during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing XTOLShadow register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the associated cell in the XTOLPRPG are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable, Power_enable, or XTOLShadow cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K32

Message Text

DFTMAX LBIST Rule K32: XTOLShadow cell disturb

N XTOLShadow cells (P/G) in compressor S were disturbed during X procedure.

Severity

Error

Description

Note the following:

N indicates number of failing XTOLShadow cells

P indicates position of first failing cell in the XTOLShadow register

G indicates gate ID of first failing cell in the XTOLShadow register

S indicates name of DFTMAX CODEC associated with failing XTOLShadow register

X indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the XTOLShadow register cells are checked to ensure that they hold state during the SHADOW_TO_CAREPRPG procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the chain outputs to random values, XTOL_enable and Power_enable cells to their off state, and all other register types to random values. XTOLShadow cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given XTOLShadow.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that XTOLShadow cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing XTOLShadow register with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable, Power_enable, or XTOLShadow cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K33

Message Text

DFTMAX LBIST Rule K33: Invalid XTOL_enable reload operation

XTOL_enable cell (input I of G) in seq compressor S had invalid capture during X procedure.

Severity

Error

Description

Note the following:

I indicates input ID of failing XTOL_enable cell

G indicates gate ID of failing XTOL_enable cell

S indicates name of DFTMAX Seq CODEC associated with failing XTOL_enable cell

X indicates name of procedure

The XTOL_enable cells are checked to ensure that they successfully load data from the PRPG_shadow during the SHADOW_TO_CAREPRPG, SHADOW_TO_XTOLPRPG, and LAST_SHIFT procedures. This check is performed by running a parallel (32) pattern simulation using the simulated values of the appropriate procedure and placing XTOL_enable and Power_enable cells to their off state and all other register types to random values. XTOL_enable cells that do not successfully capture their expected value for all simulated parallel patterns violate this rule. The expected values for a XTOL_enable cell are those assigned to the associated cell in the PRPG_shadow.

What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that XTOL_enable cells properly capture data from its PRPG_shadow during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis displays the failing XTOL_enable cell. Suppose the failure occurred on a clock/set/reset input. In that case, the pindata is set to the simulated values for the indicated procedure, and all gates in the trace-back cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed, and gates in the path between the failing cell and the associated cell in the PRPG_shadow are also displayed.

To fix this problem, the netlist or STIL log file needs to be modified to allow XTOL_enable cells to properly capture data from their PRPG_shadow during the specified procedure. Automated violation analysis can be used to debug this issue. This analysis shows the faulty XTOL_enable cell. Suppose the error occurred on a clock/set/reset input; in this case, the PIN data is set to the simulated values for the specified procedure, and all tags in that input's traceback cone are also displayed. Otherwise, the displayed pin data is set to one of the failed parallel pattern values, and gates in the path between the failed cell and the associated cell in the PRPG_shadow are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable or PRPG_shadow cells.

This rule violation can be downgraded to a warning but is not recommended as it is likely to result in incorrect patterns.

DRC Rule K34

Message Text

```
DFTMAX LBIST Rule K3: XTOL_enable cell disturb
```

```
XTOL_enable cell (G) in compressor S was disturbed during X procedure.
```

Severity

Error

Description

Note the following:

`G` indicates gate ID of first failing cell in the XTOL_enable register

`S` indicates name of DFTMAX CODEC associated with failing XTOL_enable register

`X` indicates name of procedure

The XTOL_enable cells are checked to ensure that they hold state during the SHIFT and RESEED_OVERLAP_SHIFT procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the chain outputs to random values, XTOL_enable and Power_enable cells to their off state, and all other register types to random values. XTOL_enable cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that XTOL_enable cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing XTOL_enable cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K35

Message Text

```
DFTMAX LBIST Rule K35: Invalid internal chain input value
```

```
Input node G of chain C in compressor S had invalid simulated value  
during X procedure..
```

Severity

Error

Description

Note the following:

`G` indicates gate ID of chain input node

`c` indicates name of chain whose input had invalid value

`s` indicates name of DFTMAX CODEC associated with failing chain

`x` indicates name of procedure

Chain inputs are checked to ensure that they are at the correct value during the SHIFT procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL_enable and Power_enable cells to their off state, and all other register types to random values. Chain inputs which are not at their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected values for a chain input are calculated from the values assigned to the CARE_shadow cells considering the CarePRPG tapping.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that chain inputs have the correct value during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing chain input node. The displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing chain input and the connecting CARE_shadow cells are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable, Power_enable, or CARE_shadow cells.

The `-lfsr_connections` option of the `report_compressors` command can be used to report the CarePRPG tapping and the chain to CarePRPG connections.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K36

Message Text

DFTMAX LBIST Rule K36: Invalid internal chain shift operation

`N` internal chain cells (input `I` of `G`) of chain `C` had invalid value during `X` procedure.

Severity

Error

Description

Note the following:

`N` indicates number of failing internal cells

- I indicates input ID of first failing internal cell
- G indicates gate ID of first failing cell
- C indicates name of chain whose cell failed shift checking
- X indicates name of procedure

Internal cells are checked to ensure that they are correctly shifting data during the RESEED_OVERLAP_SHIFT procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing internal cells at random values, XTOL_enable and Power_enable cells to their off state, and all other register types to random values. Internal cells which do not capture their expected shift value for all of the simulated parallel patterns will result in a violation of this rule. The expected values for an internal cell are calculated from the values assigned to their adjacent cell considering cell-to-cell inversion. The expected shift values for the cell closest to the scan input are the values assigned to the chain input node. Only one violation is issued for a given chain.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that internal cells successfully capture the correct shift values during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing internal cell. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the adjacent cell (or chain input) are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable or Power_enable cells.

The `report_scan_chains` command can be used to report the chain input nodes or report the internal cells with their inversion.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K37

Message Text

DFTMAX LBIST Rule K37: Internal chain cell disturb

N internal chain cells (P/G of chain C) in seq compressor S were disturbed during X procedure, where,

N indicates number of failing internal cells in the failing chain.

P indicates position of first failing cell in the failing internal chain.

G indicates gate ID of first failing cell in the failing internal chain.

C indicates name of chain whose cells failed disturb checking.

S indicates name of DFTMAX Seq CODEC associated with failing internal chain cell.

X indicates name of procedure.

Severity

Error

Description

You can check the internal scan cells to ensure that they hold state during the RESEED_WITH_MISR_SHIFT, RESEED_SHIFT, SHADOW_TO_CAREPRPG, SHADOW_TO_XTOLPRPG, and OCC_SHIFT procedures. You can check this by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the chain outputs to random values, XTOL_enable and Power_enable cells to their off state, and all other register types to random values. Internal scan cells which do not hold state for all of the simulated parallel patterns result violate this rule.

What Next

To correct this problem, change the netlist or the STIL protocol file so that the internal scan cells hold state during the specified procedure. Use the automated violation analysis to help debug this problem. This analysis displays the failing internal scan cell with all the gates in the traceback cone of the input that was causing the problem. The pindata is set to the simulated values of the failing procedure.

Use the `-cells` option of the `report_compressors` command to report the XTOL_enable or Power_enable cells.

Use the `report_scan_chains` command to report the chain input nodes.

Use the `report_scan_cells` command to report the internal cells with their inversion.

For IDU2:

Tracing the failing internal chain cell provides some information about which DFF is causing the problem. Sometimes the culprit DFF receives incorrect value from the previous procedure simulation.

The following command displays the procedure which is simulated before the current procedure.

```
Set_drc -analyze_drc_violation {K37 gate_id}
```

This rule violation may be downgraded to warning but it is not recommended since this likely causes patterns to be incorrect.

DRC Rule K38

Message Text

DFTMAX LBIST Rule K38: Invalid Power_enable value from test_setup

Power_enable cell (G) had incorrect value (V) at end of test_setup procedure.

Severity

Error

Description

Note the following:

G indicates gate ID of failing Power_enable cell

V indicates value of Power_enable cell at end of test_setup procedure

The Power_enable cell is checked to ensure that it is successfully initialized to its off-state during the test_setup procedure. This checking is done by comparing the simulated value of the Power_enable cell at the end of the test_setup procedure and its off-state value.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that the Power_enable cell is properly initialized to its off-state in the test_setup procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing Power_enable cell. The pindata is set to the simulated values of the test_setup procedure and all gates in the traceback cone of this gate are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the Power_enable cell with its off state.

The `test_setup` option of the `set_pindata` command can be used to report the stored simulated values of the test_setup procedure.

The `-store_setup` option of the `set_drc` command can be used to store the simulated values for all patterns of the test_setup procedure. The default is to only store the final pattern value.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K39

Message Text

```
DFTMAX LBIST Rule K39: Invalid Power_enable value from low_power_setup  
  
Power_enable cell (G) had incorrect value (V) at end of low_power_setup  
procedure.
```

Severity

Error

Description

Note the following:

G indicates gate ID of failing Power_enable cell

V indicates value of Power_enable cell at end of low_power_setup procedure

The Power_enable cell is checked to ensure that it is successfully initialized to its on-state during the low_power_setup procedure. This checking is done by comparing the simulated value of the Power_enable cell at the end of the low_power_setup procedure and its on-state value.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that the Power_enable cell is properly initialized to its on-state in the low_power_setup procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing Power_enable cell. The pindata is set to the simulated values of the low_power_setup procedure and all gates in the traceback cone of this gate are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the Power_enable cell with its off state.

The `low_power_setup` option of the `set_pindata` command can be used to report the stored simulated values of the test_setup procedure.

This rule violation can be downgraded to warning. Since there is currently no support for using the low_power_setup procedure, it will have no effect until support is available for this procedure.

DRC Rule K40

Message Text

DFTMAX LBIST Rule K40: Power_enable cell disturb

Power_enable cell (G) was disturbed during X procedure.

Severity

Error

Description

Note the following:

G indicates gate ID of failing Power_enable cell

X indicates the name of the procedure

The Power_enable cell is checked to ensure that it holds state during the SHIFT, RESEED_WITH_MISR_SHIFT, RESEED_SHIFT, RESEED_OVERLAP_SHIFT, SHADOW_TO_CAREPRPG, SHADOW_TO_XTOLPRPG, OCC_SHIFT, and capture procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL_enable and Power_enable cells to their off state, and all other register types to random values. If the Power_enable cell does not hold state for all of the simulated parallel patterns, it will result in a violation of this rule.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that Power_enable cell holds state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing Power_enable cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report_compressors command can be used to report the Power_enable cell with its off state.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K41

Message Text

DFTMAX LBIST Rule K41: Invalid occ_chain_load operation

`N OCC_chain_cells (input I of P/G) had invalid value during X procedure..`

Severity

Error

Description

Note the following:

`N` indicates number of failing `occ_chain` cells

`I` indicates input ID of first failing cell in the `occ_chain`

`P` indicates position of first failing cell in the `occ_chain`

`G` indicates gate ID of first failing cell in the `occ_chain`

`X` indicates name of procedure

When the `XTOL_enable` and `Power_enable` cells are set to their off state, the `occ_chain` cells are checked to ensure that they are successfully loading data during the `OCC_SHIFT` procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the `occ_chain` input ports to random values, `XTOL_enable` and `Power_enable` cells to their off state, and all other register types to random values. `occ_chain` cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected value for cells which connect to `occ_chain` input ports are the values assigned to the input port (or inverted). The expected values for all other cells are the values assigned to the adjacent cells. Only one violation is issued for a given `occ_chain`.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that `occ_chain` cells are properly capturing data during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing `occ_chain`. If the failure occurred on a clock/set/reset input, then the pin data is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

The `-chain_segments` option of the `report_compressors` command can be used to report the input ports of the `occ_chain`.

DRC Rule K42

Message Text

```
DFTMAX LBIST Rule K4: occ_chain cell disturb  
N occ_chain cells (P/G) were disturbed during X procedure.
```

Severity

Error

Description

Note the following:

N indicates number of failing occ_chain cells

P indicates position of first failing cell in the occ_chain

G indicates gate ID of first failing cell in the occ_chain

X indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the occ_chain cells are checked to ensure that they hold state during the SHIFT, RESEED_WITH_MISR_SHIFT, RESEED_SHIFT, RESEED_OVERLAP_SHIFT, SHADOW_TO_CAREPRPG, and SHADOW_TO_XTOLPRPG procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, XTOL_enable and Power_enable cells to their off state, and all other register types to random values. occ_chain cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given occ_chain.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that occ_chain cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing occ_chain with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report_compressors command can be used to report the XTOL_enable, Power_enable, or occ_chain cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns which exercise the occ_chain load operation to be incorrect.

DRC Rule K43

Message Text

```
DFTMAX LBIST Rule K43: misr_mode cell disturb
```

```
Misr_mode cell (G) was disturbed during X procedure.
```

Severity

Error

Description

Note the following:

G indicates gate ID of failing misr_mode cell

X indicates name of procedure

The MISR cells are checked to ensure that they hold state during the SHIFT, RESEED_WITH_MISR_SHIFT, RESEED_SHIFT, RESEED_OVERLAP_SHIFT, SHADOW_TO_CAREPRPG, SHADOW_TO_XTOLPRPG, OCC_SHIFT, and capture procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL_enable and Power_enable cells to their off state, and all other register types to random values. If any misr_mode cell does not hold state for all of the simulated parallel patterns, it will result in a violation of this rule.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that the failing misr_mode cell holds state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing misr_mode cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report_compressors command can be used to report the misr_mode cells with their accumulator mode enable values.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K44

Message Text

```
DFTMAX LBIST Rule K44: Invalid misr_mode value from test_setup
```

`Misr_mode cell (G) had incorrect value V at end of test_setup procedure.`

Severity

Error

Description

Note the following:

`G` indicates gate ID of failing `misr_mode` cell

`V` indicates the incorrect value that was set by the `test_setup` procedure

At the end of the `test_setup` procedure (and extended `test_setup` if used), the `misr_mode` cells are checked to ensure that they are at the correct value as determined by the selected accumulator mode usage. If the accumulator mode is enabled, then the `misr_mode` cells must be set to their enable values. Otherwise, the `misr_mode` cells must be set to the complement of their enable values. `Misr_mode` cells which are not at their correct value at this time will result in a violation of this rule.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that `misr_mode` cells are at their intended value at the end of `test_setup`. The active accumulator mode selection may have to be changed to reflect the intended usage. The automated violation analysis can be used to help debug this problem. This analysis will display the failing `misr_mode` cell with all the gates in its traceback cone. The pindata is set to the simulated values at the end of the `test_setup` procedure(s).

The `-cells` option of the `report_compressors` command can be used to report the `misr_mode` cells with their accumulator mode enable values.

The `drc` option of the `report_settings` command can be used to report the current selection of the accumulator mode usage.

The `test_setup` option of the `set_pindata` command can be used to report the stored simulated values of the `test_setup` procedure.

The `-store_setup` option of the `set_drc` command can be used to store the simulated values for all patterns of the `test_setup` procedure.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K45

Message Text

`DFTMAX LBIST Rule K45: Invalid seed_type shift operation`

`Seed_type cell (input I of G) had invalid shift value during X procedure.`

Severity

Error

Description

Note the following:

`I` indicates input ID of failing `seed_type` cell

`G` indicates gate ID of failing `seed_type` cell

`X` indicates name of procedure

When the `XTOL_enable` and `Power_enable` cells are set to their off state, the `seed_type` cells are checked to ensure that they are successfully shifting data during the `RESEED_PARTIAL_OVERLAP_SHIFT` procedures. This checking is done by first simulating the JTAG load procedure to set values on the JTAG controller. This is followed by performing parallel (32) pattern simulation using the simulated values of the `RESEED_PARTIAL_OVERLAP_SHIFT` procedure, and placing `XTOL_enable` and `Power_enable` cells to their off state, and all other register types to random values. `Seed_type` cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. This rule is only checked if JTAG ULTRA is used.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that `seed_type` cells correctly shift during the `RESEED_PARTIAL_OVERLAP_SHIFT` procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing `seed_type` cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_enable`, `Power_enable`, or `seed_type` cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K46

Message Text

`DFTMAX LBIST Rule K4: seed_type cell disturb`

`Seed_type cell (G) was disturbed during X procedure..`

Severity

Error

Description

Note the following:

G indicates gate ID of failing seed_type cell

X indicates name of procedure

To correct this problem, the netlist or the STL procedure file must be changed so that seed_type cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing seed_type cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that seed_type cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing seed_type cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report_compressors command can be used to report the XTOL_enable, Power_enable, or seed_type cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K47

Message Text

```
DFTMAX LBIST Rule K47: Invalid capture_enable_shadow shift operation
```

```
Capture_enable_shadow cell (input I of G) had invalid shift value during  
X procedure.
```

Severity

Error

Description

Note the following:

I indicates input ID of failing capture_enable_shadow cell

G indicates gate ID of failing capture_enable_shadow cell

X indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the capture_enable_shadow cells are checked to ensure that they are successfully shifting data during the RESEED_PARTIAL_OVERLAP_SHIFT procedures. This checking is done by first simulating the JTAG_RESEED_SETUP procedure to set values on the JTAG controller. This is followed by performing parallel (32) pattern simulation using the simulated values of the RESEED_PARTIAL_OVERLAP_SHIFT procedure, and placing XTOL_enable and Power_enable cells to their off state, and all other register types to random values. Capture_enable_shadow cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. This rule is only checked if JTAG LBIST or JTAG ULTRA is used.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that capture_enable_shadow cells correctly shift during the RESEED_PARTIAL_OVERLAP_SHIFT procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing capture_enable_shadow cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report_compressors command can be used to report the XTOL_enable, Power_enable, or capture_enable_shadow cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K48

Message Text

DFTMAX LBIST Rule K48: Capture_enable_shadow cell disturb

Capture_enable_shadow cell (G) was disturbed during X procedure.

Severity

Error

Description

Note the following:

G indicates gate ID of failing capture_enable_shadow cell

x indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the capture_enable_shadow cells are checked to ensure that they hold state during selected procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, XTOL_enable and Power_enable cells to their off state, and all other register types to random values. Capture_enable_shadow cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. This rule is not currently checked for any procedure.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that capture_enable_shadow cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing capture_enable_shadow cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report_compressors command can be used to report the XTOL_enable, Power_enable, or capture_enable_shadow cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K49

Message Text

DFTMAX LBIST Rule K49: Invalid capture_enable reload operation

Capture_enable cell (input I of G) had invalid reload value during X procedure.

Severity

Error

Description

Note the following:

I indicates input ID of failing capture_enable cell

G indicates gate ID of failing capture_enable cell

X indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the capture_enable cells are checked to ensure that they successfully load data from their associated capture_enable_shadow cells during the SHADOW_TO_CAREPRPG and SHADOW_TO_XTOLPRPG procedures. This checking is done by first simulating the JTAG_RESEED_SETUP procedure to set values on the JTAG controller. This is followed by performing parallel (32) pattern simulation using the simulated values of the failing procedure, and placing XTOL_enable and Power_enable cells to their off state, and all other register types to random values. Capture_enable cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. This rule is only checked if JTAG LBIST or JTAG ULTRA is used.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that capture_enable cells capture their expected values from their associated capture_enable_shadow cells during the failing procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing capture_enable cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report_compressors command can be used to report the XTOL_enable, Power_enable, capture_enable, or capture_enable_shadow cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K50

Message Text

DFTMAX LBIST Rule K50: capture_enable cell disturb

Capture_enable cell (G) was disturbed during X procedure..

Severity

Error

Description

Note the following:

G indicates gate ID of failing capture_enable cell

x indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the capture_enable cells are checked to ensure that they hold state during the RESEED_PARTIAL_OVERLAP_SHIFT, JTAG_RESEED_SETUP, and JTAG_MISR_SHIFT procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, XTOL_enable and Power_enable cells to their off state, and all other register types to random values. Capture_enable cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. This rule is only checked if JTAG LBIST or JTAG ULTRA is used.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that capture_enable cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing capture_enable cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report_compressors command can be used to report the XTOL_enable, Power_enable, capture_enable, or capture_enable cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K51

Message Text

DFTMAX LBIST Rule K51: Invalid shift_counter_shadow shift operation

N shift_counter_shadow cells (input I of P/G) had invalid shift values during X procedure.

Severity:Error

Severity

Error

Description

Note the following:

N indicates number of failing shift_counter_shadow cells

P indicates position of first failing shift_counter_shadow cell

G indicates gate ID of first failing shift_counter_shadow cell

X indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the shift_counter_shadow cells are checked to ensure that they are successfully shifting data during the RESEED_PARTIAL_OVERLAP_SHIFT procedures. This checking is done by first simulating the JTAG_RESEED_SETUP procedure to set values on the JTAG controller. This is followed by performing parallel (32) pattern simulation using the simulated values of the RESEED_PARTIAL_OVERLAP_SHIFT procedure, and placing XTOL_enable and Power_enable cells to their off state, and all other register types to random values. Shift_counter_shadow cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. This rule is only checked if JTAG LBIST or JTAG ULTRA is used. Only one violation is issued for a given shift_counter_shadow chain.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that shift_counter_shadow cells correctly shift during the RESEED_PARTIAL_OVERLAP_SHIFT procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing shift_counter_shadow cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report_compressors command can be used to report the XTOL_enable, Power_enable, or shift_counter_shadow cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K52

Message Text

```
DFTMAX LBIST Rule K52: shift_counter_shadow cell disturb
```

```
Shift_counter_shadow cell (G) was disturbed during X procedure.
```

Severity

Error

Description

Note the following:

G indicates gate ID of shift_counter_shadow cell

x indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the shift_counter_shadow cells are checked to ensure that they hold state during the SHADOW_TO_CAREPRPG and SHADOW_TO_XTOLPRPG procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, XTOL_enable and Power_enable cells to their off state, and all other register types to random values. Shift_counter_shadow cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. This rule is only checked if JTAG LBIST or JTAG ULTRA is used.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that shift_counter_shadow cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing shift_counter_shadow cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report_compressors command can be used to report the XTOL_enable, Power_enable, or shift_counter_shadow cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K53

Message Text

DFTMAX LBIST Rule K53: Invalid shift_counter reload operation

N shift_counter cells (input I of P/G) had invalid reload values during X procedure.

Severity

Error

Description

Note the following:

N indicates number of failing shift_counter cells

I indicates input ID of failing shift_counter cellP

P indicates position of first failing shift_counter cell

G indicates gate ID of first failing shift_counter cell

x indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the shift_counter cells are checked to ensure that they successfully load data from their associated shift_counter_shadow cells during the SHADOW_TO_CAREPRPG and SHADOW_TO_XTOLPRPG procedures. This checking is done by first simulating the JTAG_RESEED_SETUP procedure to set values on the JTAG controller. This is followed by performing parallel (32) pattern simulation using the simulated values of the failing procedure, and placing XTOL_enable and Power_enable cells to their off state, and all other register types to random values. Shift_counter cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. This rule is only checked if JTAG LBIST or JTAG ULTRA is used. Only one violation is issued for a given shift_counter chain.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that shift_counter cells capture their expected values from their associated shift_counter_shadow cells during the failing procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing shift_counter cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report_compressors command can be used to report the XTOL_enable, Power_enable, or shift_counter_shadow cells.

This rule violation might be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K54

Message Text

```
DFTMAX LBIST Rule K54: Invalid shift_counter count operation
```

```
N shift_counter cells (P/G) had invalid count values during LBIST  
shifting.
```

Severity

Error

Description

Note the following:

N indicates number of failing shift_counter cells

P indicates position of first failing shift_counter cell

G indicates gate ID of first failing shift_counter cell

Shift_counter cells are checked to ensure that their count operation is correctly performed during the internal scanchain shift process. This checking is done by first initializing the state of the circuit to their values at the beginning of the SHIFT procedure. Then the capture_enable cell is set to its capture_enable_value, seed_type cell is set to its CarePRPG_select_value, shift_counter cells set to a value equal to the number shifts per load, shift_counter_shadow cells set to a value equal to the number shifts per load, and the pattern_counter cells are set to a value of 2. Finally, the SHIFT procedure is simulated a number of times equal to the number of shifts per load plus the minimum number of ATE clock cycles. The shift counter should end up at a value equal to the number of shifts per load. Shift_counter cells which are not at their correct value will result in a violation of this rule. This rule is only checked if JTAG LBIST is used. Only one violation is issued for a given shift_counter chain.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that shift_counter cells count properly during LBIST shifting. The shift counter and pattern counter values for each shift can be displayed during the rules checking by using the -compressor_debug_data option of the set_drc command. The automated violation analysis can be also used to help debug this problem. This analysis will display all of the cells of the failing shift counter with all connections between the cells. The pindata is set to the simulated values during the number_shifts_per_load + 1 shift cycle.

The -cells option of the report_compressors command can be used to report the shift_counter cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K55

Message Text

```
DFTMAX LBIST Rule K55: shift_counter cell disturb
```

```
Shift_counter cell (G) was disturbed during X procedure.
```

```
Severity:Error
```

Severity

Error

Description

Note the following:

G indicates gate ID of failing shift_counter cell

X indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the shift_counter cells are checked to ensure that they hold state during the JTAG_RESEED_SETUP and JTAG_MISR_SHIFT procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, XTOL_enable and Power_enable cells to their off state, and all other register types to random values. Shift_counter cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. This rule is only checked if JTAG LBIST or JTAG ULTRA is used.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that shift_counter cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing shift_counter cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report_compressors command can be used to report the XTOL_enable, Power_enable, or shift_counter cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K56

Message Text

DFTMAX LBIST Rule K56: Invalid pattern_counter shift operation

N pattern_counter cells (input I of P/G) had invalid shift values during X procedure.

Severity

Error

Description

Note the following:

N indicates number of failing pattern_counter cells

- I indicates input ID of failing pattern_counter cell
- P indicates position of first failing pattern_counter cell
- G indicates gate ID of first failing pattern_counter cell
- X indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the pattern_counter cells are checked to ensure that they are successfully shifting data during the PATCOUNTER_SHIFT procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the PATCOUNTER_SHIFT procedure, and placing XTOL_enable and Power_enable cells to their off state, and all other register types to random values. Pattern_counter cells which do not successfully shift their expected value for all of the simulated parallel patterns will result in a violation of this rule. This rule is only checked if JTAG LBIST is used. Only one violation is issued for a given shift_counter_shadow chain.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that pattern_counter cells correctly shift during the failing procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing pattern_counter cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable, Power_enable, or pattern_counter cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K57

Message Text

```
DFTMAX LBIST Rule K57: Invalid pattern_counter count operation  
N pattern_counter cells (P/G) had invalid count values during LBIST  
shifting.
```

Severity

Error

Description

Note the following:

N indicates number of failing shift_counter cells

P indicates position of first failing shift_counter cell

G indicates gate ID of first failing shift_counter cell

Pattern_counter cells are checked to ensure that their count operation is correctly performed during the internal scanchain shift process. This checking is done by first initializing the state of the circuit to their values at the beginning of the SHIFT procedure. Then the capture_enable cell is set to its capture_enable_value, seed_type cell is set to its CarePRPG_select_value, shift_counter cells set to a value equal to the number shifts per load, shift_counter_shadow cells set to a value equal to the number shifts per load, and the pattern_counter cells are set to a value of 2. Finally, the SHIFT procedure is simulated a number of times equal to the number of shifts per load plus the minimum number of ATE clock cycles. The pattern counter should end up at a value equal to 1. Pattern_counter cells which are not at their correct value will result in a violation of this rule. This rule is only checked if JTAG LBIST is used. Only one violation is issued for a given pattern_counter chain.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that pattern_counter cells count properly during shifting. The shift counter and pattern counter values for each shift can be displayed during the rules checking by using the -Compressor_debug_data option of the set_drc command. The automated violation analysis can be also used to help debug this problem. This analysis will display all of the cells of the failing pattern_counter with all connections between the cells. The pindata is set to the simulated values during the number_shifts_per_load + 1 shift cycle.

The -cells option of the report_compressors command can be used to report the pattern_counter cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K58

Message Text

DFTMAX LBIST Rule K58: pattern_counter cell disturb

Pattern_counter cell (G) was disturbed during X procedure.

Severity

Error

Description

Note the following:

G indicates gate ID of failing pattern_counter cell

X indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the pattern_counter cells are checked to ensure that they hold state during the RESEED_PARTIAL_OVERLAP_SHIFT, SHADOW_TO_CAREPRPG, SHADOW_TO_XTOLPRPG, JTAG_RESEED_SETUP, and JTAG_MISR_SHIFT procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, XTOL_enable and Power_enable cells to their off state, and all other register types to random values. Pattern_counter cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. This rule is only checked if JTAG LBIST or JTAG ULTRA is used.

What Next

To correct this problem, the netlist or the STL procedure file must be changed so that pattern_counter cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing pattern_counter cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report_compressors command can be used to report the XTOL_enable, Power_enable, or pattern_counter cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K59

Message Text

DFTMAX LBIST Rule K59: Scan cells with same erroneous signatures

Scan cells N1 (G1) and N2 (G2) have same signature.

Severity

Error

Description

Note the following:

N1 indicates name of scan cell

G1 indicates gate ID of scan cell

N2 indicates name of scan cell

G2 indicates gate ID of scan cell

Uniqueness of erroneous signatures is required improving mapping features back to scan cells for debugging and diagnosis. In certain cases, the MISR might fail to satisfy this condition and two or more failing scan cells might generate the same erroneous signature. In this case, the failing scan cell cannot be uniquely determined even when a single scan cell fails for a particular pattern.

What Next

Need to redesign the MISR to meet this condition.

DRC Rule K60

Message Text

DFTMAX LBIST Rule K6: chain_mask cell disturb

N chain_mask cells (P/G) in compressor S were disturbed during X procedure.

Severity

Error

Description

Note the following:

N indicates number of failing chain_mask chain

P indicates position of failing chain_mask chain

G indicates gate ID of failing cell in the chain_mask chain

S indicates name of CODEC associated with failing chain_mask chain

X indicates name of procedure

When the XTOL_enable and Power_enable cells are set to their off state, the chain_mask cells are checked to ensure that they hold state during the SHIFT, RESEED_WITH_MISR_SHIFT, RESEED_SHIFT, RESEED_OVERLAP_SHIFT,

RESEED_PARTIAL_OVERLAP_SHIFT, SHADOW_TO_CAREPRPG, SHADOW_TO_XTOLPRPG, JTAG_RESEED_SETUP, JTAG_MISR_SHIFT, RESEED_STREAMING_SHIFT, and OCC_SHIFT procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, XTOL_enable and Power_enable cells to their off state, and all other register types to random values. Chain_mask cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given chain_mask chain.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that chain_mask cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing chain_mask chain with all the gates in the fanin cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable, Power_enable, or chain_mask cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns which exercise the OCC_chain load operation to be incorrect.

DRC Rule K61

Message Text

DFTMAX LBIST Rule K61: Invalid xchains_register reload operation

N xchains_register cells (input I of P/G) in compressor S had invalid value during X procedure.

Severity

Error

Description

Note the following:

N indicates number of failing xchains_register cells

I indicates input ID of first failing cell in the xchains_register

P indicates position of failing cell in the xchains_register

G indicates gate ID of failing cell in the xchains_register

S indicates name of CODEC associated with failing cell in the xchains_register

x indicates name of procedure

When the XTOL_ENABLE and POWER_ENABLE cells are set to their off state, the xchains_register cells are checked to ensure that they are successfully capturing reseed data from the PRPG_SHADOW during the SHADOW_TO_XTOLPRPG procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL_ENABLE/POWER_ENABLE cells to their off state, and all other register types to random values. Xchains_register cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected value for cells are determined by the values from the associated cell in the PRPG_SHADOW. Only one violation is issued for a given xchains_register.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that xchains_register cells are properly capturing data during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing xchains_register register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

The -cells option of the report_compressors command can be used to report the XTOL_enable, Power_enable, or xchains_register cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns which exercise the scan chain load operation from PRPGs to be incorrect.

DRC Rule K62

Message Text

```
DFTMAX LBIST Rule K62: xchains_register cell disturb
```

```
N xchains_register cells (P/G) in compressor S were disturbed during X  
procedure.
```

Severity

Error

Description

Note the following:

N indicates number of failing xchains_register cells

P indicates position of failing cell in the xchains_register

G indicates gate ID of failing cell in the xchains_register

S indicates name of CODEC associated with failing cell in the xchains_register

X indicates name of procedure

When the XTOL_ENABLE and POWER_ENABLE cells are set to their off state and codec enables are set to their active state, the xchains_register cells are checked to ensure that they hold state during the SHIFT, SHADOW_TO_CAREPRPG, RESEED_WITH_MISR_SHIFT, RESEED_SHIFT, RESEED_PARTIAL_OVERLAP_SHIFT1, RESEED_PARTIAL_OVERLAP_SHIFT2, RESEED_LOAD, JTAG_RESEED_SETUP, JTAG_MISR_SHIFT, RESEED_OVERLAP_SHIFT, RESEED_PARTIAL_OVERLAP_SHIFT1, RESEED_PARTIAL_OVERLAP_SHIFT2, RESEED_STREAMING_SHIFT1, RESEED_STREAMING_SHIFT2, and OCC_SHIFT procedures.

When the xtol_enables are set active, POWER_ENABLE cells are set inactive, and codec enables are set active, the xchains_register cells are also checked to ensure that they hold state during the SHADOW_TO_XTOLPRPG procedure.

This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, XTOL_ENABLE/POWER_ENABLE cells to their off state, codec enables are set to their active state, and all other register types to random values. xXchains_register cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given procedure.

When xchains_registers are used, some procedures are also checked considering CODEC disabling variations. This checks all combinations of a single CODEC disabled (used as an xchain and only observable in single observe mode) and all other codecs enabled. When failures occur during checking with a codec disabled, the error message will include an extra line indicating which codec was disabled.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that xchains_register cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing xchains_register with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable, Power_enable, or xchains_register cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns which exercise the xchains_register operation to be incorrect.

DRC Rule K63

Message Text

DFTMAX LBIST Rule K6: Invalid value from jtag_reseed_setup

Register T cell (G) had incorrect value (V) at end of jtag_reseed_setup procedure.

Severity

Error

Description

Note the following:

T indicates type of failing register cell

G indicates gate ID of failing register cell

v indicates the incorrect value that was set by the jtag_reseed_setup procedure

At the end of the jtag_reseed_setup procedure, the address, broadcast, and increment_address cells are checked to ensure that they are at the correct value. The address cells must indicate a value of 0, the broadcast cell must be at the defined broadcast value, and the increment address cell must be at the complement of the increment_address_value. Cells which are not at their correct value at this time will result in a violation of this rule.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that the failing cells are at their intended value at the end of the jtag_reseed_setup procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing cell with all the gates in its traceback cone. The pindata is set to the simulated values of the jtag_reseed_setup procedure.

The `-cells` option of the `report_compressors` command can be used to report the address, broadcast (with broadcast values), or increment_address cells (with increment_address values).

The `-summary` option of the `report_compressors` command can be used to report the `codec_enables` with their associated enable values.

The `jtag_reseed_setup` option of the `set_pindata` command can be used to report the stored simulated values of the jtag_reseed_setup procedure.

This rule violation can be downgraded to warning, but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K64

Message Text

DFTMAX LBIST Rule K64: Invalid instruction_register reload operation

N instruction register cells (input I of P/G) had invalid value during X procedure.

Severity

Error

Description

Note the following:

N indicates number of failing failing instruction register cells

I indicates input ID of first failing cell in the instruction register

P indicates position of failing cell in the instruction register

G indicates gate ID of failing cell in the instruction register

X indicates name of procedure

When the xt看 enable cells are set off and POWER_ENABLE cells are set off and idu_enable cells are set active and codec enables are set active, the instruction register cells are checked to ensure that they are successfully loading reseed data during the RESEED_SHIFTA, RESEED_SHIFTB, and RESEED_PARTIAL_OVERLAP_SHIFT2 procedures. The instruction register contains the cells of the broadcast, address, seed_type, capture_enable_shadow, and shift_counter_shadow register types. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL_ENABLE/POWER_ENABLE cells to their off state, idu_enable/codec enable to their active state and all other register types to random values. Instruction register cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected value for cells are determined by the values from the associated cell in the PRPG_SHADOW. Only one violation is issued for a given procedure.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that xchains_register cells are properly capturing data during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing xchains_register register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise,

the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_ENABLE, power enable, codec_enable, and instruction register cells. The `-cells` option can also be used to report the individual register types used in the instruction register. These register types include broadcast, address, seed_type, capture_enable_shadow, and shift_counter_shadow.

The `-summary` option of the `report_compressors` command can be used to report the idu_enables with their associated enable values.

This rule violation can be downgraded to warning, but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K65

Message Text

DFTMAX LBIST Rule K65: Address cell disturb

N address cells (P/G) were disturbed during X procedure.

Severity

Error

Description

Note the following:

N indicates number of failing address cells

P indicates position of failing cell in the address register

G indicates gate ID of failing cell in the xchains_register

S indicates name of CODEC associated with failing cell in the address register

X indicates name of procedure

When the XTOL_ENABLE and POWER_ENABLE cells are set to their off state and codec_enables are set to their active state, the address cells are checked to ensure that they hold state during the SHIFT, RESEED_SHIFT, RESEED_LOAD, RESEED_PARTIAL_OVERLAP_SHIFT1, SHADOW_TO_CAREPRPG, and SHADOW_TO_XTOLPRPG procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, XTOL_ENABLE/POWER_ENABLE cells set to their off state, codec_enables set to their active state, and all other register types set to random values. Address cells which do not

hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given procedure.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that address cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing address cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_ENABLE`, `POWER_ENABLE`, `codec_enable`, or address cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K66

Message Text

DFTMAX LBIST Rule K66: Broadcast cell disturb

Broadcast cell (G) was disturbed during X procedure.

Severity

Error

Description

Note the following:

G indicates gate ID of failing broadcast cell

X indicates name of procedure

When the `XTOL_ENABLE` and `POWER_ENABLE` cells are set to their off state and `codec_enables` are set to their active state, the broadcast cell is checked to ensure that it holds state during the `SHIFT`, `RESEED_SHIFT`, `RESEED_LOAD`, `RESEED_PARTIAL_OVERLAP_SHIFT1`, `SHADOW_TO_CAREPRPG`, and `SHADOW_TO_XTOLPRPG` procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, `XTOL_ENABLE/POWER_ENABLE` cells set to their off state, `codec_enables` set to their active state, and all other register types set to random values. Broadcast cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given procedure.

What Next

To correct this problem, the netlist or the test procedure file must be changed so the broadcast cell holds state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing broadcast cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to broadcast cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K67

Message Text

DFTMAX LBIST Rule K6: Invalid increment_address set operation

Increment_address cell (G) had invalid set value (V1/V2) during X procedure.

Severity

Error

Description

Note the following:

G indicates gate ID of failing cell in the xchains_register

V1 indicates simulated value of failing increment_address cell

V2 indicates expected value of failing increment_address cell

X indicates name of procedure

At the end of an application of certain shift procedures which are intended to set the increment address cell, the increment_address cell is checked to ensure that it is at the correct value. The increment_address cell must be set to the complement of its increment_address_value for the RESEED_SHIFTA and RESEED_SHIFTB procedures. The increment_address cell must also be set to the its increment_address_value for the RESEED_SHIFTC, RESEED_PARTIAL_OVERLAP_SHIFT1, and RESEED_LOAD postamble procedures. Cells which are not at their correct value will result in a violation of this rule.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that the failing cells are at their intended value at the end of the failing procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing cell with all the gates in its traceback cone. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the `increment_address` cells (with `increment_address` values).

The `-summary` option of the `report_compressors` command can be used to report the `codec_enables` with their associated enable values.

The `RESEED_SHIFTA`, `RESEED_SHIFTB`, `RESEED_SHIFTC`, `RESEED_PARTIAL_OVERLAP_SHIFT1`, and `RESEED_LOAD` options of the `set_pindata` command can be used to report the stored simulated values of the failing procedure.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K68

Message Text

```
DFTMAX LBIST Rule K68: Increment_address cell disturb
```

```
Increment_address cell (G) was disturbed during X procedure.
```

Severity

Error

Description

Note the following:

G indicates gate ID of failing `increment_address` cell

X indicates name of procedure

When the `XTOL_ENABLE` and `POWER_ENABLE` cells are set to their off state and codec enables are set to their active state, the `increment_address` cells are checked to ensure that they hold state during the `SHIFT`, `RESEED_LOAD`, `RESEED_PARTIAL_OVERLAP_SHIFT2`, `SHADOW_TO_CAREPRPG`, and `SHADOW_TO_XTOLPRPG` procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, `XTOL_ENABLE/POWER_ENABLE` cells set to their off state, codec enables set to their

active state, and all other register types set to random values. Increment_address cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given procedure.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that increment_address cell holds state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing increment_address cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable, Power_enable, or address cells.

The `-summary` option of the `report_compressors` command can be used to report the codec_enables with their associated enable values.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K69

Message Text

```
DFTMAX LBIST Rule K69: Invalid codec_enable set decode value operation  
N codec_enable cells (input I of P/G) had invalid decode value during X  
procedure.
```

Severity

Error

Description

Note the following:

N indicates number of failing codec_enable cells

I indicates input ID of first failing cell in the codec_enable register

P indicates position of failing cell in the codec_enable register

G indicates gate ID of failing cell in the codec_enable register

X indicates name of procedure

When the XTOL_ENABLE and POWER_ENABLE cells are set off, the codec_enable cells are checked to ensure that they successfully perform the decode operation during

the RESEED_LOAD (third preamble shift) and RESEED_PARTIAL_OVERLAP_SHIFT2 procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL_ENABLE/POWER_ENABLE cells to their off state, and all other register types to random values. Codec_enable cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected values for codec_enable cells are calculated considering the values that were randomly assigned to the increment_address, address, and broadcast cells. For patterns where the increment_address is set to its increment_address_value, the expected value is calculated as follows:

The expected value for the first cell is 0.

Else if expected value of preceding codec_enable cell was 1, the expected value is 1.

Else expected value is value assigned to preceding codec_enable cell.

For patterns where the increment_address is not set to its increment_address_value, the expected value is calculated as follows:

Calculate the address and broadcast values for given pattern.

For cells whose $ID < address$, the expected value is 0.

For cells whose $ID = address$, the expected value is 1.

For cells whose $ID > address$, if $broadcast = broadcast_value$, the expected value is 1, else expected value is 0.

Only one violation is issued for a given procedure.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that the failing codec_enable cells are properly capturing data during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing codec_enable register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_ENABLE, power enable, codec_enable, increment_address, address, and broadcast cells

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K70

Message Text

```
DFTMAX LBIST Rule K70: codec_enable cell disturb  
N codec_enable cells (P/G) were disturbed during X procedure.
```

Severity

Error

Description

Note the following:

N indicates number of failing codec_enable cells

P indicates position of failing cell in codec_enable register

G indicates gate ID of failing cell in codec_enable register

X indicates name of procedure

When the XTOL_ENABLE and POWER_ENABLE cells are set to their off state, the codec_enable cells are checked to ensure that they hold state during the RESEED_SHIFTA, RESEED_SHIFTB, RESEED_SHIFTC, and RESEED_PARTIAL_OVERLAP_SHIFT1 procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, XTOL_ENABLE/POWER_ENABLE cells to their off state, and all other register types to random values. Codec_enable cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given procedure.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that codec_enable cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing codec_enable register with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_enable, Power_enable, or codec_enable cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K71

Message Text

DFTMAX LBIST Rule K71: Invalid cumulative_codec_enable set decode value operation

N cumulative_codec_enable cells (input I of P/G) had invalid decode value during X procedure.

Severity

Error

Description

Note the following:

N indicates number of failing cumulative_codec_enable cells

I indicates input ID of first failing cell in the cumulative_codec_enable

P indicates position of failing cell in the cumulative_codec_enable

G indicates gate ID of failing cell in the cumulative_codec_enable

X indicates name of procedure

When the XTOL_ENABLE and POWER_ENABLE cells are set off, the cumulative_codec_enable cells are checked to ensure that they successfully perform the decode operation during the RESEED_LOAD (third preamble shift) procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL_ENABLE/POWER_ENABLE cells to their off state, and all other register types to random values. Cumulative_codec_enable cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected values for cumulative_codec_enable cells are calculated considering the values that were randomly assigned to the increment_address, address, and broadcast cells. For patterns where the increment_address is set to its increment_address_value, the expected value is calculated as follows:

The expected value for the first cell is 0.

Else if expected value of preceding cumulative_codec_enable cell was 1, the expected value is 1.

Else expected value is value assigned to codec_enable cell whose ID is 1 less than ID of the cumulative_codec_enable cell.

For patterns where the `increment_address` is not set to its `increment_address_value`, the expected value is calculated as follows:

Calculate the address and broadcast values for given pattern.

For cells whose `ID < address`, the expected value is 0.

For cells whose `ID = address`, the expected value is 1.

For cells whose `ID > address`, if `broadcast = broadcast_value`, the expected value is 1, else expected value is 0.

Only one violation is issued for a given procedure.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that the failing `cumulative_codec_enable` cells are properly capturing data during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing `cumulative_codec_enable` register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_ENABLE`, power enable, `cumulative_codec_enable`, `increment_address`, `address`, and broadcast cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K72

Message Text

```
DFTMAX LBIST Rule K72: Invalid cumulative_codec_enable set accumulate  
value operation)
```

```
N cumulative_codec_enable cells (input I of P/G) had invalid accumulate  
value during X procedure.
```

Severity

Error

Description

Note the following:

N indicates number of failing `cumulative_codec_enable` cells

I indicates input ID of first failing cell in the `cumulative_codec_enable` register

P indicates position of failing cell in the `cumulative_codec_enable` register

G indicates gate ID of failing cell in the `cumulative_codec_enable` register

X indicates name of procedure

When the `XTOL_ENABLE` and `POWER_ENABLE` cells are set off, the `cumulative_codec_enable` cells are checked to ensure that they successfully perform the accumulate operation during the `RESEED_PARTIAL_OVERLAP_SHIFT2` procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing `XTOL_ENABLE/POWER_ENABLE` cells to their off state, and all other register types to random values. `Cumulative_codec_enable` cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected value for a `cumulative_codec_enable` cell is calculated by ORing its randomly assigned value with the randomly assigned value of its associated `codec_enable` cell. Only one violation is issued for a given procedure.

What Next

o correct this problem, the netlist or the test procedure file must be changed so that the failing `cumulative_codec_enable` cells are properly capturing data during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing `cumulative_codec_enable` register. If the failure occurred on a clock/set/reset input, then the pin data is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_ENABLE`, `power enable`, `cumulative_codec_enable`, and `codec_enable` cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K73

Message Text

```
DFTMAX LBIST Rule K73: Cumulative_codec_enable cell disturb)
```

`N cumulative_codec_enable cells (P/G) were disturbed during X procedure.`

Severity

Error

Description

Note the following:

`N` indicates number of failing `xchains_register` cells

`I` indicates input ID of first failing cell in the `xchains_register`

`P` indicates position of failing cell in the `xchains_register`

`G` indicates gate ID of failing cell in the `xchains_register`

`S` indicates name of CODEC associated with failing cell in the `xchains_register`

`X` indicates name of procedure

When the `XTOL_ENABLE` and `POWER_ENABLE` cells are set to their off state, the `cumulative_codec_enable` cells are checked to ensure that they hold state during the `RESEED_SHIFTA`, `RESEED_SHIFTB`, `RESEED_SHIFTC`, `RESEED_PARTIAL_OVERLAP_SHIFT1`, and `RESEED_LOAD` (postamble) procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, `XTOL_ENABLE/POWER_ENABLE` cells to their off state, and all other register types to random values. `Cumulative_codec_enable` cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given procedure.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that `cumulative_codec_enable` cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing `cumulative_codec_enable` register with all the gates in the traceback cone of the input which was believed to have caused the problem. The `pindata` is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_ENABLE`, `power enable`, or `cumulative_codec_enable` cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K74

Message Text

DFTMAX LBIST Rule K74: XTOL_ENABLE inversion difference

Seq compressors S1 and S2 had different inversion parity to XTOL_ENABLE port P.

Severity

Error

Description

Note the following:

S1 indicates name of first seq_compressor associated with failure

S2 indicates name of second seq_compressor associated with failure

P indicates name of port associated with XTOL_ENABLE

For all CODECs, the inversion parity between the XTOL_ENABLE port and the connecting PRPG_SHADOW register cell must be the same. The inversion parity of the first CODEC is compared to the inversion parity of all other CODECs. A violation is issued for each CODEC which has an inversion difference.

What Next

To correct this problem, the netlist must be changed so that the inversion parity is the same for all CODECs. The automated violation analysis can be used to help debug this problem. This analysis will display the PRPG_SHADOW cells associated with the XTOL_ENABLE of the two failing CODECS and their connection path to the XTOL_ENABLE port. There is no pindata used for this analysis.

The `-cells Prpg_shadow` option of the `report_compressors` command can be used to report the XTOL_ENABLE cell for a given CODEC.

The `-scan_input_ports` option of the `report_compressors` command can be used to report the XTOL_ENABLE port. This port is the first port in the port list.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K75

Message Text

DFTMAX LBIST Rule K75: Invalid pattern_start set operation

Pattern_start cell (G) had invalid set value (V1/V2) during X procedure.

Severity

Error

Description

Note the following:

G indicates gate_id of failing register cell

V1 indicates the simulated value of failing cell for failing procedure

V2 indicates the expected value of failing cell for failing procedure

X indicates name of failing procedure

At the end of the test_setup and shadow_to_careprpg (with capture_enable set to capture_enable value) procedures, the pattern_start cell is checked to ensure that it is at the pattern_start value. At the end of the jtag_reseed_setup (with pattern_start cell initialized to pattern_start value) and jtag_reseed_setup2 (with pattern_start cell initialized to complement of pattern_start value) procedures, the pattern_start cell is checked to ensure that it is at the complement of pattern_start value. Cells which are not at their correct value at the end of checked procedure will result in a violation of this rule.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that the failing cells are at their intended value at the end of the failing procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing cell with all the gates in its traceback cone. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report_compressors command can be used to report the pattern_start cell with its associated pattern_start value. The command usage is:

```
report_compressors <-all | comp_name> [-cells <PATTERN_Start>]
```

The test_setup, jtag_reseed_setup, jtag_reseed_setup2, and shadow_to_careprpg options of the set_pindata command can be used to report the stored simulated values of the failing procedure. The command usage is:

```
set_pindata [test_setup | jtag_reseed_setup | jtag_reseed_setup2 |  
shadow_to_careprpg]
```

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K76

Message Text

DFTMAX LBIST Rule K76: Pattern_start cell disturb

Pattern_start cell (G) was disturbed during X procedure.

Severity

Error

Description

Note the following:

G indicates gate_id of failing increment_address cell

X indicates the name of the procedure

When the XTOL_ENABLE and POWER_ENABLE cells are set to their off state and codec enables are set to their active state, the pattern_start cell is checked to ensure that it holds state during the SHIFT, RESEED_SHIFTA, RESEED_SHIFTB, RESEED_SHIFTC, RESEED_SHIFTD, RESEED_SHIFTE, RESEED_LOAD, RESEED_PARTIAL_OVERLAP_SHIFT1, RESEED_PARTIAL_OVERLAP_SHIFT2, SHADOW_TO_CAREPRPG (with capture_enable set to complement of capture_enable value), JTAG_MISR_SHIFT, OCC_SHIFT, and capture procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, XTOL_ENABLE/POWER_ENABLE cells set to their off state, codec enables set to their active state, and all other register types set to random values. Pattern_start cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that increment_address cell holds state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing increment_address cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report_compressors command can be used to report the XTOL_ENABLE, power enable, or pattern_start cells. The command usage is:

```
report_compressors <-all | comp_name> [-cells <-All | comp_name> [-Cells  
  <XTOL_Enable | PWR_Enable | PATTERN_Start>]]
```

The `-summary` option of the `report_compressors` command can be used to report the `codec_enables` with their associated enable values. The command usage is:

```
report_compressors -summary
```

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K77

Message Text

```
DFTMAX LBIST Rule K77: TE last cell of chain connected to TE MISR  
TE last cell (G) of chain C was connected to TE MISR in compressor S.
```

Severity

Error

Description

Note the following:

G indicates gate_id of failing register cell

C indicates the name of the failing chain

S indicates the name of compressor associated with failing chain

If the MISR captures data from the scan chains on the trailing edge of MISR clock, then the scan cells closest to each chain output are checked to ensure they capture data on the leading edge of the scan cell shift clocks. If a chain output cell captures on its trailing edge when the MISRs capture on its trailing edge, a violation is issued.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that either the MISR captures on the leading edge or the failing scan cells capture data on the leading edge. The automated violation analysis can be used to help debug this problem. This analysis will display the failing cell with pin data set to the simulated values of the SHIFT procedure.

The `-cells` option of the `report_compressors` command can be used to report the MISR cells. The command usage is:

```
report_compressors <-all | comp_name> [-cells < Misr>]
```


The shift option of the `set_pindata` command can be used to display the simulated values of the SHIFT procedure. The command usage is:

```
set_pindata [shift]
```

This rule violation can be downgraded to warning, but it is not recommended since this will likely cause the ATPG patterns to be incorrect due to simulation assuming MISRs capture the newly captured values for all shift cycles.

DRC Rule K78

Message Text

DFTMAX LBIST Rule K78: *T* cell (*G*) using cycle *C* failed load operation from port *P*.

Severity

Error

Description

Note the following:

T indicates the register type of the failing cell. It may be a PRPG shadow input, OCC SERDES cell, MISR_shadow SERDES (serializer/deserializer) cell, or Pattern_counter SERDES cell)

G indicates the gate ID of the failing register cell

C indicates the name of the failing chain

P indicates the name of the port that loads the data for the failing cell

When `external_cycles_per_shift` is set to a nonzero number, the SERDES circuitry (which includes pipeline and serializer cells) associated with several register types is checked to ensure that it is loaded correctly from its port inputs. The checking inserts data on the inputs of the SERDES circuitry, simulates the associated shift procedure and the adequate number of cycles, and then ensures that the SERDES outputs achieve the intended values at the right time. The following SERDES register types are checked:

PRPG_shadow register – Uses the `RESEED_PARTIAL_OVERLAP_SHIFT1` procedure and can store patterns in the `SERDES_PRPG_INPUT_SHIFT`

OCC register – Uses the `OCC_SHIFT` procedure and can store patterns in the `SERDES_OCC_INPUT_SHIFT`

MISR_shadow register – Uses the `MISR_SHADOW_SHIFT` procedure and can store patterns in the `SERDES_MISR_SHADOW_INPUT_SHIFT`

Pattern_counter register – Uses the PAT_COUNTER_SHIFT procedure and can store patterns in the SERDES_PAT_COUNTER_INPUT_SHIFT

The inputs of the SERDES circuitry are stored in the associated port list and are reported using the `report_compressors -scan_input_ports` command. The outputs of the SERDES circuitry are the segment outputs associated with the register types and can be displayed using the `report_compressors -chain_segments` command.

Simulation is performed by first setting all gates to the values at the beginning of the shift procedure associated with the selected register type. This shift procedure is then simulated for three slow cycles ($3 * N$ applications). Selected binary data is placed on the SERDES inputs at the beginning of each fast shift cycle. After the first three slow cycles, the SERDES inputs are set to X.

The simulation of the shift procedure is repeated until all loaded binary values are fully propagated to the SERDES outputs and compared with expected values. The SERDES outputs that do not achieve the intended value at the appropriate time cause a violation of the K78 rule. The patterns that were simulated during this checking can be stored during DRC using the following command:

```
set_drc -store_full_cycles pattern_type_for_selected_register
```

This data is then available for display using the following command:

```
set_pindata -serdes_full_cycle_period_display begin_period end_period
```

What Next

To correct this problem, you need to change the netlist or the test procedure file so that the failing SERDES load operation operates correctly. You can use automated violation analysis to help debug this problem. This analysis displays the SERDES input and output gates associated with the failure with pin data set to the simulated values of the associated SHIFT procedure used for the simulation.

Since `external_cycles_per_shift` is set to a nonzero number, this simulation is for a full slow cycle of N applications of the shift procedure, but the displayed data only includes three time periods: the initial value of the first application of the shift procedure, the clock-on time of the application of the shift procedure expected to capture, and the final time period of the last application of the shift procedure. All slow cycle simulation data used during the analysis can be stored and displayed using the `-store_full_cycles` option of the `set_drc` command and the `-serdes_full_cycle_period_display` option of the `set_pindata` command. The syntax for these options is as follows:

```
set_drc -store_full_cycles <serdes_prpg_input_shift |  
serdes_occ_input_shift | serdes_patcounter_input_shift |  
serdes_misr_shadow_input_shift>
```

```
set_pindata -serdes_full_cycle_period_display begin_period end_period
```

This rule violation can be downgraded to a warning but it is not recommended since this will likely cause corrupted patterns due to invalid loading of data through the SERDES circuitry.

DRC Rule K79

Message Text

DFTMAX LBIST Rule K79: SERDES MISR cell (G) using cycle C failed unload operation to port P.

Severity

Error

Description

Note the following:

G indicates gate ID of the failing register cell

C indicates the name of the failing chain

P indicates the name of the port that loads the data for the failing cell

When `external_cycles_per_shift` is set to a nonzero number, the SERDES circuitry (which includes pipeline and serializer cells) associated with several register types is checked to ensure that it is loaded correctly from its port inputs. The checking inserts data on the inputs of the SERDES circuitry, simulates the associated shift procedure and an adequate number of cycles, and then ensures that the SERDES outputs achieve the intended values at the right time.

Simulation is performed by first setting all gates to the values at the beginning of the shift procedure associated with the selected register type. This shift procedure is then simulated for three slow cycles ($3 * N$ applications). Selected binary data is placed on the SERDES inputs at the beginning of each fast shift cycle. After the first three slow cycles, the SERDES inputs are set to X.

The simulation of the shift procedure is repeated until all loaded binary values are fully propagated to the SERDES outputs and compared with expected values. The SERDES outputs that do not achieve the intended value at the appropriate time cause a violation of the K78 rule. The patterns that were simulated during this checking can be stored during DRC using the following command:

```
set_drc -store_full_cycles serdess_output_shift
```

This data is then available for display using the following command:

```
set_pindata -serdes_full_cycle_period_display begin_period end_period
```

What Next

To correct this problem, you need to change the netlist or the test procedure file so that the failing SERDES MISR load operation operates correctly. You can use automated violation analysis to help debug this problem. This analysis displays the SERDES input and output gates associated with the failure that is the pin data set to the simulated values of the associated MISR_SHIFT procedure used for the simulation.

Since `external_cycles_per_shift` is set to a nonzero number, this simulation is for a full slow cycle of N applications of the shift procedure, but the displayed data only includes three time periods: the initial value of the first application of the shift procedure, the clock-on time of the application of the shift procedure expected to capture, and the final time period of the last application of the shift procedure. All slow cycle simulation data used during the analysis can be stored and displayed using the `-store_full_cycles` option of the `set_drc` command and the `-serdes_full_cycle_period_display` option of the `set_pindata` command. The syntax for these options is as follows:

```
set_drc -store_full_cycles serdes_output_shift
```

```
set_pindata -serdes_full_cycle_period_display begin_periodend_period
```

This rule violation can be downgraded to a warning but it is not recommended since this will likely cause corrupted patterns due to invalid loading of data through the SERDES circuitry.

DRC Rule K80

Message Text

DFTMAX LBIST Rule K80: Extended test_setup simulation mismatch

T cell (*G*) in position *P* has extended test_setup simulation mismatch (*V1/V2*).

T indicates register type of failing cell.

G indicates gate_id of failing cell.

P indicates position of failing cell in the CODEC DRC chain.

V1 indicates simulated value of the failing cell.

V2 indicates expected value of the failing cell.

Severity

Error

Description

After simulating the extended test setup using the CODEC-DRC load and shift procedures, certain CODEC-DRC cells are checked to ensure they have attained their intended values. Cells which do not have the intended value that violate this rule.

What Next

To correct this problem, the netlist or the test procedure file must be changed to ensure that the `CODEC_DRC` load process is working correctly. Use the automated violation analysis to debug this problem. This analysis displays the failing cell and all gates in its traceback cone with pindata set to the simulated values of the `CODEC_DRC LOAD` procedure.

More debug info for For IDU2

The following command can be used to display more debugging information:

```
Set_drc -analyze_drc_violation {K80 gate_id first_shift last_shift}
```

It displays the total number of `codec_drc_shifts` simulated for K80 check. It displays the `Payload_cycles_counter` value, and the `Local_cycles_counter` value for every `Seq_compressor_IDU` just after decoding the `Codec_drc` procedure. It also displays the `codec_drc_shift` values (`codec_drc_init_values` are the intended values) for all the shifts, and for all the scan-ins.

This displays the value of the gate (`gate_id` provided as an argument for this command) whenever the value changes, and it also displays the `shift_number` at which the value changes. It also stores the `codec_drc_shift` simulation cycles(shift cycles between the `first_shift` and `last_shift`) under `CODEC_DRC_SHIFTA` pattern. These `first_shift` and `last_shift` are provided as arguments for this command.

`Codec_drc_init_values` has lot of X's as there are several registers whose value at the end of `extended_test_setup` can be don't care. With X's loading into the chain, it's difficult to know at which position on the `codec_drc_chain`, the `codec_drc_shifting` is not correct. The following command can be used to use binary values as the intended values.

```
Set_drc -binary_K80_check
```

This gives the nearest position where the `codec_drc_shifting` is not correct. This option is just for analyzing K80(and not be used during regular DRC checking). To reset these options, you can use the following command:

```
Set_drc -noanalyze_drc_violation -nobinary_K80_check
```

This rule violation may be downgraded to warning but it is not recommended since this likely causes patterns to be incorrect due to invalid loading of the CODEC DRC chain which is a critical part of the state initialization process.

DRC Rule K81

Message Text

DFTMAX LBIST Rule K81: *N* misr_shadow cells (input *I* of *P/G*) had invalid update values during *X* procedure.

Severity

Error

Description

Note the following:

N indicates the number of failing misr_shadow cells in the failing MISR (multiple-input signature register) shadow register

I indicates the input that caused problem of first failing cell

P indicates the position of the first failing cell in the MISR_shadow register

G indicates the gate ID of the failing register cell

X indicates the name of the procedure

The misr_shadow cells are checked to ensure that they successfully capture data from the MISR register during the MISR_SHIFT procedure. This check performs parallel pattern simulation using the simulated values of the MISR_SHIFT procedure and placing random values on the MISR cells. Misr_shadow cells that do not successfully capture the expected value for all of the parallel patterns simulated cause a violation of this rule. The expected values for a misr_shadow cell are the values assigned to the associated cell in the MISR register. Only one violation is issued for each misr_shadow register.

What Next

To correct this problem, you need to change the netlist or the test procedure file so that either the MISR captures on the leading edge or the failing scancells capture data on the leading edge. You can use automated violation analysis to debug this problem. This analysis displays the failing cell with the pin data set to the simulated values of the SHIFT procedure.

You can use the `-cells` option of the `report_compressors` command to report the MISR shadow cells. The syntax is as follows:

```
report_compressors <-all | comp_name> [-cells misr_shadow]
```

You can downgrade this rule violation to a warning, but it is not recommended since this will likely cause the patterns to be incorrect due to invalid measures associated with unloading misr_shadow_status_cells.

DRC Rule K82

Message Text

DFTMAX LBIST Rule K82: *N* misr_shadow cells (*P/G*) in compressor *S* were disturbed during *X* procedure.

Severity

Error

Description

Note the following:

N indicates the number of failing misr_shadow cells in the failing MISR_shadow register

P indicates the position of the first failing cell in the MISR_shadow register

G indicates gate ID of the failing MISR_shadow register cell

S indicates the name of the sequential compressor associated with the failing misr_shadow cell

X indicates the name of the procedure

The misr_shadow register cells are checked to ensure that they hold state during the MISR_UNLOAD procedure and all other procedures in which the MISR (multiple-input signature register) must hold state that are not associated with MISR_SHADOW and MISR_SHADOW_STATUS operations. This check is done by performing parallel pattern simulation using the simulated values of the appropriate procedure. The misr_shadow cells that do not hold state for all of the simulated parallel patterns cause a violation of this rule. Only one violation is issued for a given misr_shadow register.

What Next

To correct this problem, you need to change the netlist or the test procedure file so that misr_shadow cells hold state during the specified procedure. You can use automated violation analysis to debug this problem. This analysis displays the failing cell for the failing misr_shadow register with the pin data set to the simulated values of the failing procedure.

You can use the `-cells` option of the `report_compressors` command to report the MISR shadow cells. The syntax is as follows:

```
report_compressors <-all | comp_name> [-cells misr_shadow]
```

You can downgrade this rule violation to a warning, but it is not recommended since this will likely cause the patterns to be incorrect due to invalid measures associated with unloading misr_shadow_status_cells.

DRC Rule K83

Message Text

DFTMAX LBIST Rule K83: misr_shadow_status cell (input *I* of *G*) had invalid update value during *X* procedure.

Severity

Error

Description

Note the following:

I indicates the input that caused the problem of the first failing cell

G indicates the gate_id of the failing MISR_shadow_status cell

X indicates the name of the procedure

The misr_shadow_status cells are checked to ensure that they are successfully capturing data from the misr_shadow register during the last clock pulse of the MISR_SHADOW_STATUS_UPDATE procedure. This checking is done by performing parallel pattern simulation using the simulated values of the last clock pulse in the MISR_SHADOW_STATUS_UPDATE procedure and placing random values on the MISR_SHADOW cells. One pattern of the 32 random values assigned to the misr_shadow cells is adjusted to be all zeros. The expected value for a misr_shadow_status cell of a CODEC is the result of ORing all the values that were assigned to the misr_shadow register cells of that CODEC. The misr_shadow_status cells that do not successfully capture their expected value for all of the parallel patterns simulated causes a violation of this rule.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that the misr_shadow_status cells are properly updating data from the associated misr_shadow cells during the MISR_SHADOW_STATUS_UPDATE procedure. You can use automated violation analysis to debug this problem. This analysis displays the failing misr_shadow_status cell with the pin data set to the simulated values of the failing procedure.

You can use the `-cells` option of the `report_compressors` command to report the misr_shadow_status cells. The syntax is as follows:

```
report_compressors <-all | comp_name> [-cells misr_shadow_status_cells]
```

This rule violation can be downgraded to a warning, but it is not recommended since this will likely cause the patterns to be incorrect due to invalid measures associated with unloading misr_shadow_status_cells.

DRC Rule K84

Message Text

DFTMAX LBIST Rule K84: Misr_shadow_status cell (G) in seq compressor C was disturbed during X procedure.

Severity

Error

Description

Note the following:

G indicates gate_id of failing MISR_shadow_status cell

C indicates the name of the compressor containing failing MISR_shadow_status cell

X indicates name of procedure

The misr_shadow_status register cells are checked to ensure that they hold state during the MISR_SHIFT procedure and all other procedures where the MISR must hold state that are not associated with MISR_SHADOW and MISR_SHADOW_STATUS operations. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure. Misr_shadow_status cells which do not hold state for all of the parallel patterns simulated will result in a violation of this rule.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that misr_shadow_status cells hold state during the specified procedure. Automated violation analysis can be used to help debug this problem. This analysis will display the failing misr_shadow_status cell with pindata set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the misr_shadow_status cells. The command usage is:

```
report_compressors <-all | comp_name > [-cells misr_shadow_status ]
```

This rule violation can be downgraded to warning, but it is not recommended since this will likely cause the patterns to be incorrect due to invalid measures associated with unloading misr_shadow_status_cells.

DRC Rule K85

Message Text

DFTMAX LBIST Rule K85: Invalid idu_enable value during reseed_load preamble

IDU enable (G) had invalid value (V1/V2) during reseed_load preamble.

Severity

Error

Description

Note the following:

G indicates gate_id of failing register cell

V1 indicates simulated value of the failing cell

V2 indicates expected value of the failing cell

During the simulation of the clock-on time of the the last cycle of the preamble of the RESEED_LOAD procedure, the IDU enable gate is checked to ensure it is at the idu_enable active value. If it is not at the expected value it will result in a violation of this rule.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that the IDU enable is at the correct value. The automated violation analysis can be used to help debug this problem. This analysis will display the IDU enable gate with pindata set to the RESEED_LOAD procedure.

The `-cells` option of the `report_compressors` command can be used to report the MISR cells. The command usage is:

```
report_compressors <-all | comp_name> [-cells < Misr>]
```

This rule violation can be downgraded to warning, but it is not recommended since this will likely cause the ATPG patterns to be incorrect due to simulation assuming MISRs capture the newly captured values for all shift cycles.

DRC Rule K86

Message Text

DFTMAX LBIST Rule K86 (CODEC DRC chain cell disturb)

CODEC DRC chain cell G was disturbed during T time D of X procedure.

N DRC chain cells (P/G2) in seq compressor S were disturbed during X procedure.

Severity

Warning

Description

Note the following:

G indicates the `gate_id` of the failing CODEC_DRC chain cell

T indicates the time during which the failure occurred (preamble or postamble)

X indicates the name of the procedure

N indicates the number of the failing cells in failing PRPG_SHADOW register

P indicates the position of the first failing cell in the failing PRPG_SHADOW register

G2 indicates the `gate_id` of the first failing cell

S indicates the name of the sequential compressor associated with the failing cell

There are two distinct variations of this rule:

The slow CODEC DRC chain cells are checked to ensure that they hold state during the preamble cycles and non-shifting postamble cycles of the CODEC_DRC_LOAD_UNLOAD procedure. This checking is done using the pre-calculated simulated values of the CODEC_DRC_LOAD_UNLOAD procedure. Slow CODEC DRC chain cells which do not hold state for all of the checked time periods result in a violation of this rule.

The PRPG_SHADOW cells are checked to ensure that they hold state during the SHADOW_TO_CAREPRPG and SHADOW_TO_XTOLPRPG procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the selected procedure. PRPG_SHADOW cells which do not hold state for all of the parallel patterns simulated will result in a violation of this rule. Only one violation is issued for a given PRPG_SHADOW register.

What Next

If K-86 DRC violation is present, `codec_drc` self-test patterns may fail VCS simulation. To correct this problem, the netlist or the test procedure file must be changed so that failing cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis displays the failing cell with pin data set to the simulated values of the failing procedure.

DRC Rule K87

Message Text

DFTMAX LBIST Rule K87: invalid fast shifting

Misr_shadow_status cell (G) in seq compressor C was disturbed during X procedure.

Scan cell (G) in chain C for seq compressor S had invalid fast shifting during X procedure.

Severity

Error

Description

Note the following:

G indicates the gate_id of failing internal scan cell

C indicates the name of the failing chain

S indicates the name of compressor associated with failing chain

X indicates the name of the procedure

When external_cycles_per_shift (N) is set to a nonzero number, all scan cells are checked to identify how many times they are pulsed during N applications of the SHIFT procedure. Scan cells which are pulsed more than once will result in a violation of this rule. Only one violation is issued for an internal scan chain.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that internal scan cells shift once per N applications of the SHIFT procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing cell with pindata set to the normally stored simulated values of the SHIFT procedure. When external_cycles_per_shift is set to a nonzero number, this data only includes 3 time periods which are the initial value of the first application of the shift procedure, the clock-on time of the application of the shift procedure expected to capture, and the final time period of the last application of the shift procedure. To view all time periods of all N cycles, the -store_full_cycles option of the set_drc command can be used prior to running DRC to collect this simulation data. These values can then be displayed using the -serdes_full_cycle_period_display option of the set_pindata command. The command usages are:

```
set_drc -store_full_cycles serdes_internal_shift
```

```
set_pindata -serdes_full_cycle_period_display begin_period end_period
```

This rule violation may be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect due to invalid loading and unloading of data into and out of the internal scan chains.

DRC Rule K88

Message Text

```
DFTMAX LBIST Rule K88: MISR_SHADOW_STATUS cell disturb
```

```
MISR_SHADOW_STATUS cell G was disturbed during T time D of X procedure.
```

Severity

Error

Description

Note the following:

G indicates gate_id of failing MISR_SHADOW_STATUS cell

T indicates time during which failure occurred (preamble or postamble)

X indicates the name of the procedure

The slow MISR_SHADOW_STATUS cells are checked to ensure that they hold state during the preamble cycles and nonshifting postamble cycles of the MISR_SHADOW_STATUS_UNLOAD procedure. This checking is done using the precalculated simulated values of the MISR_SHADOW_STATUS_UNLOAD procedure. Slow MISR_SHADOW_STATUS cells which do not hold state for all of the checked time periods will result in a violation of this rule.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that failing cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing cell with pindata set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the MISR_SHADOW_STATUS cells. The command usage is:

```
report_compressors -cells misr_shadow_status
```

This rule violation may be downgraded to warning but it is not recommended since this will likely cause the IP test patterns to be incorrect.

DRC Rule K89

Message Text

DFTMAX LBIST Rule K89: Invalid ShadowMISR shift operation

N ShadowMISR cells (input I of P/G) in seq compressor S had invalid shift values during X procedure.

Severity

Error

Description

Note the following:

N indicates the number of failing ShadowMISR cells

I indicates the input ID of the first failing cell in the ShadowMISR register

P indicates the position of the failing cell in the ShadowMISR register

G indicates the gate ID of the failing cell in the ShadowMISR register

S indicates the name of the DFTMAX Seq CODEC associated with the failing ShadowMISR register

X indicates the name of the procedure

The misr_shadow_load procedure is used to ensure that ShadowMISR cells are shifting data successfully.

What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that ShadowMISR cells are properly shifting during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing ShadowMISR register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

This rule violation may be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

DRC Rule K90

Message Text

DFTMAX LBIST Rule K90: Codec "*name1*" differs from MasterCODECAddress *n*("name2") by condition (*n1*/*n2*)

Severity

Error

Description

When MasterCODECAddress keywords are used to identify an expected relationship between two codecs, DRC validates the required structural relationships between the two codecs. If the structural requirements are not satisfied the MasterCODEC relationship will not be supportable. The list of checked conditions are below. The error message also identifies the difference in value between the first code (the "slave", reported as value *n1*) and the second codec (the "master" reported as value *n2*).

The condition is one of the following:

number of multiple observe modes

full observe mode value

full observe mode mask

number of full-observable scanchains

the length of scan chains on connection *n* (chains "*ch_name1*"/"*ch_name2*")

the parity of scan chains on connection *n* (chains "*ch_name1*"/"*ch_name2*")

the number of taps on connection *n*

the tap positions on connection *n*, index *n*

What Next

To correct this problem, the codecs in the netlist and the SPF description must be aligned, or the MasterCODECAddress reference in the SPF can be removed to not match these two codecs as similar.

This rule violation can be downgraded to warning, but it is not recommended as this will likely impact performance and QoR as the ATPG flow will attempt to generate patterns to apply to both codes and will not be successful.

DRC Rule K91

Message Text

DFTMAX LBIST Rule 91: Invalid initialization of MISRs

N misr (M) cells (C/G) in seq compressor S had incorrect value at end of test_setup procedure.

Severity

Error

Description

Note the following:

N indicates the number of failing cells in the failing MISR

M indicates the name of the failing MISR (only used if multiple MISRs per CODEC)

C indicates the cell ID of the first failing cell in MISR

G indicates the gate ID of the first failing cell in MISR

S indicates the name of the compressor associated with the failing chain

When the MASK_ALL_CHAINS cell is defined and extended test setup is not used, all MISRs are checked to ensure they are initialized to 0 at the end of the test setup procedure. MISR cells which are not at 0 will result in a violation of this rule. Only one violation is issued for a MISR.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that MISR cells are set to 0 at the end of the TEST_SETUP procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing MISR cell with pindata set to the stored simulated values of the TEST_SETUP procedure. This problem can also be corrected by supporting the usage of the extended test setup procedure in the test procedure file.

This rule violation may be downgraded to warning but it is not recommended since this will likely cause the first BIST pattern interval to be incorrect.

DRC Rule K92

Message Text

DFTMAX LBIST Rule K92: Invalid initialization of MASK_ALL_CHAINS cell

`MASK_ALL_CHAINS` cell (G) had incorrect value (V) at end of the `test_setup` procedure.

Severity

Error

Description

Note the following:

G indicates gate ID of failing cell

V indicates the gate ID of the failing chain

When the `MASK_ALL_CHAINS` cell is defined and extended test setup is not used, the `MASK_ALL_CHAINS` cell is checked to ensure it is initialized to the complement of the `mask_all_chains_value` at the end of the test setup procedure. An incorrect value will result in a violation of this rule.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that the `MASK_ALL_CHAINS` cell is set to the `mask_all_chains_value` at the end of the `TEST_SETUP` procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the `MASK_ALL_CHAINS` cell with pindata set to the stored simulated values of the `TEST_SETUP` procedure. This problem can also be corrected by supporting the usage of the extended test setup procedure in the test procedure file.

This rule violation may be downgraded to warning but it is not recommended since this will likely cause the first SEQ pattern to be incorrect.

DRC Rule K93

Message Text

DFTMAX LBIST Rule K93: Invalid setting of `MASK_ALL_CHAINS` cell

`MASK_ALL_CHAINS` cell (G) had incorrect set value (V) at end of P procedure.

Severity

Error

Description

Note the following:

G indicates the `gate_id` of failing cell

V indicates the incorrect value at end of procedure

P indicates the name of the failing procedure

When the MASK_ALL_CHAINS cell is defined, the MASK_ALL_CHAINS cell is checked to ensure it is set to the mask_all_chains_value at the end of the PATCOUNTER_LOAD procedure and is set to its complement value at the end of the LOAD procedure. An incorrect value will result in a violation of this rule.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that the MASK_ALL_CHAINS cell is set to the correct value at the end of the PATCOUNTER_LOAD and the LOAD procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the MASK_ALL_CHAINS cell with pindata set to the stored simulated values of the failing procedure.

This rule violation may be downgraded to warning but it is not recommended since this will likely cause BIST pattern intervals to be incorrect.

DRC Rule K94

Message Text

DFTMAX LBIST Rule K94: Invalid MISR hold for first unload

N misr (M) cells (C/G) in seq compressor S had incorrect value at end of test_setup procedure.

N misr (M) cells (C/%G) in seq compressor S did not hold value during first unload.

Severity

Error

Description

Note the following:

N indicates the number of failing cells in failing MISR

M indicates the name of the failing MISR (only used if multiple MISRs per CODEC)

C indicates the cell ID of first failing cell in MISR

G indicates the gate ID of first failing cell in MISR

S indicates the name of the seq compressor associated with the failing MISR

When the MASK_ALL_CHAINS cell is defined, all MISRs are checked to ensure they hold their value during the first scanchain unload of a BIST pattern interval. MISR cells which do not hold will result in a violation of this rule. Only one violation is issued for a MISR.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that MISR cells hold their value during the first scanchain unload of a BIST pattern interval. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing MISR cell with pindata set to the stored simulated values of the TEST_SETUP procedure.

This rule violation may be downgraded to a warning but it is not recommended since this will likely cause the BIST pattern intervals to be incorrect.

DRC Rule K95

Message Text

Invalid setting of FIRST_SHIFT cell.

FIRST_SHIFT cell (G) had incorrect set value (V) during P procedure.

Severity

Error

Description

Note:

- G indicates the gate ID of the failing cell
- V indicates incorrect set value during the failing procedure
- P indicates the name of the failing procedure

If the FIRST_SHIFT cell is defined, the FIRST_SHIFT cell is checked to ensure that it is set to the first_shift value at the beginning of the LOAD_UNLOAD procedure and to its complement value at the end of the SHIFT procedure. An incorrect value results in a violation of this rule.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that the FIRST_SHIFT cell is set to the correct value at the beginning of the LOAD_UNLOAD procedure and the end of the SHIFT procedure. You can use the automated violation analysis to debug this problem. This analysis displays the trace backward gates of the FIRST_SHIFT cell with pin data set to the stored simulated values of the failing procedure.

DRC Rule K96

Message Text

Invalid capture of XTOL_ENABLE cell for MASK_ENABLE full_observe-on/off mode.

XTOL_ENABLE cell (input I of G) in seq compressor S had invalid capture for MASK_ENABLE full_observe_on mode.

XTOL_ENABLE cell (input I of G) in SEQ compressor S had invalid capture for MASK_ENABLE full_observe_off mode.

Severity

Error

Description

Note:

- `I` indicates the input ID of failing XTOL_ENABLE cell (0-set, 1-reset, 2-clock , 3-D pin)
- `G` indicates the gate ID of the failing XTOL_ENABLE cell
- `S` indicates name of the DFTMAX Seq CODEC associated with failing XTOL_ENABLE cell

This check is only performed if MASK_ENABLE cells are used. This checks both the full_observe_on and full_observe_off observe modes.

When the XTOL_ENABLE cells are set to their off state, the XTOLPRPG outputs are set to select the full_observe_on mode and mask_enable cells are set to their on state, the XTOL_ENABLE cells are checked to ensure that they successfully capture the on state for XTOL_ENABLES. This checking is done by performing parallel pattern simulation using the simulated values of the SHIFT procedure and placing MASK_ENABLES on, XTOL_ENABLE cells to their off state, and XTOLPRPG outputs to select the full_observe_on mode. XTOL_ENABLE cells that do not capture the correct value for all of the simulated parallel patterns results in a violation of this rule.

When the XTOL_ENABLE cells are set to their on state and the XTOLPRPG outputs are set to select the full_observe_off mode and mask_enable cells are set to their on state, the XTOL_ENABLE cells are checked to ensure that they successfully capture the off state for XTOL_ENABLES. This checking is done by performing parallel pattern simulation using the simulated values of the SHIFT procedure and placing MASK_ENABLES on, XTOL_ENABLE cells to their on state, and XTOLPRPG outputs to select the full_observe_off mode. XTOL_ENABLE cells which do not capture the correct value for all of the simulated parallel patterns results in a violation of this rule.

What Next

To correct this problem, you must change the netlist or STIL protocol file so that the failing XTOL_ENABLE cell captures the correct value in full_observe_on and full_observe_off modes.

You can use automated violation analysis to debug this problem. This analysis displays the failing XTOL_ENABLE cell, XTOLPRPG output, XTOL_ENABLE values, Mask_ALL_CHAINS values, XCHAIN_REGISTERS_CELL, and MASK_ENABLE_CELLS. The displayed pin data is set to the simulated values of the SHIFT procedure and setting all the conditions used for this checking.

If the failing ID is 2(clock input), then clock on shift pattern values are being displayed.

If the failing ID is 2(clock input), the `-shift` option of the `set_pindata` command helps to identify the problem with the clock input.

Use the `-cells` option of the `report_compressors` command to report the XTOL_ENABLE, MASK_ENABLE, or MISR cells.

Use the `-lfsr_connections` option of the `report_compressors` command to report the XTOLPRPG output values.

Use the `-mode_data` option of the `report_compressors` command to report the XTOLPRPG output values for all observe modes.

You can downgrade this rule violation to a warning. However, it is not recommended since this is likely to cause incorrect patterns that exercise the full_observe_on and full_observe_off modes.

DRC Rule K97

Message Text

```
Invalid MISR capture value for MASK_ENABLE full_observe-on/off mode.
```

```
N MISR cells (P/G) in seq compressor S had invalid capture data for  
full_observe_on mode.
```

```
N MISR cells (P/G) in seq compressor S had invalid capture data for  
full_observe_off mode.
```

Severity

Error

Description

Note the following:

N indicates the number of failing MISR cells

P indicates the position of the first failing cell in the MISR

G indicates the gate ID of the first failing cell in the MISR

s indicates the name of the DFTMAX Seq CODEC associated with the failing MISR

This check ensures that the data inputs of MISR cells successfully capture full_observe mode values without xchain credit.

Checking is done by performing parallel pattern simulation using the simulated values of the SHIFT procedure

The data inputs of MISR cells are first checked to ensure that they successfully capture full_observe mode values without xchain credit under the following conditions:

- MISR cells are set to 0
- The XTOL_ENABLE cells are set to the off state
- The XTOLPRPG outputs are set to select the full_observe_on mode
- The MASK_ENABLE cells are set to their on state

The data inputs of MISR cells are next checked to ensure that they successfully capture full_observe mode values without xchain credit under the following conditions:

- MISR cells are set to 0
- The XTOL_ENABLE cells are set to the on state
- The XTOLPRPG outputs are set to select the full_observe_off mode
- The MASK_ENABLE cells are set to their on state

Data inputs of MISR cells which are not at the correct full_observe value for all of the simulated parallel patterns cause a violation of the K97 rule. Only one violation is issued for a given MISR.

What Next

To correct this problem, you must change the netlist or STIL procedure file so that the data input of the failing MISR cell is set to the correct value in the full_observe_on and full_observe_off modes.

You can use automated violation analysis to debug this problem. This analysis displays the first failing cell for the failing MISR. The displayed pin data is set to the simulated values of the SHIFT procedure and sets all the conditions used for this checking. Also displayed are

the gates in the paths between the XTOLPRPG outputs used to select the full_observe_on and full_observe_off modes and the failing MISR cell.

Use the `-cells` option of the `report_compressors` command to report the XTOL_ENABLE, MASK_ENABLE, or MISR cells.

Use the `-lfsr_connections` option of the `report_compressors` command to report the XTOLPRPG output values.

Use the `-mode_data` option of the `report_compressors` command to report the XTOLPRPG output values for all observe modes.

You can downgrade this rule violation to a warning. But it is not recommended since this likely causes patterns which exercise the full_observe_on and full_observe_off modes to be incorrect.

DRC Rule K98

Message Text

```
Invalid MASK_ENABLE reload operation
```

```
Mask_enable cell (input I of G) in seq compressor S had invalid value  
during P procedure.
```

Severity

Error

Description

Note the following:

I indicates the input ID of first failing MASK_ENABLE cell

G indicates the gate ID of failing MASK_ENABLE cell

S indicates the name of seq compressor associated with failing MASK_ENABLE cell

P indicates the name of procedure

When the XTOL_ENABLE and POWER_ENABLE cells are set to their off state, this check ensures that the MASK_ENABLE cells successfully capture reseed data from the PRPG_SHADOW during the SHADOW_TO_XTOLPRPG procedure.

This check performs parallel pattern simulation using the simulated values of the appropriate procedure, and places the XTOL_ENABLE and POWER_ENABLE cells to their off state and all other register types to random values.

All MASK_ENABLE cells which do not successfully capture their expected value for all of the simulated parallel patterns cause a violation of the K98 rule. The expected values for the cells are determined by the values from the associated cells in the PRPG_SHADOW.

What Next

To correct this problem, you must change the netlist or the STIL procedure file so the MASK_ENABLE cells are properly capturing data during the specified procedure.

You can use automated violation analysis to debug this problem. This analysis displays the failing MASK_ENABLE cell.

If the failure occurs on a clock, set, or reset input, the pin data is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are displayed. Otherwise, the pin data displays one of the parallel pattern values that failed and the gates in the path between the failing cell and the source of its capture data are also displayed.

Use the `-cells` option of the `report_compressors` command to report the XTOL_ENABLE, POWER_ENABLE, or MASK_ENABLE cells.

You can downgrade this rule violation to a warning. But it is not recommended since this will likely cause the BIST pattern intervals to be incorrect.

DRC Rule K99

Message Text

```
mask_enable cell disturb.
```

```
mask_enable cell (G) in seq compressor S was disturbed during P  
procedure.
```

Severity

Error

Description

Note the following:

G indicates the gate ID of failing mask_enable cell

S indicates the name of seq compressor associated with failing mask_enable cell

P indicates the name of procedure

This check ensures the mask_enable cells hold state during the SHIFT, SHADOW_TO_CAREPRPG, RESEED_WITH_MISR_SHIFT, RESEED_SHIFT, and RESEED_PARTIAL_OVERLAP_SHIFT1 procedures.

When the `xtol_enable` cells are set to active, the `power_enable` cells are set to inactive, and the codec enables are set active, the `mask_enable` cells are also checked to ensure that they hold state during the `SHADOW_TO_XTOLPRPG` procedure.

When `xchains_registers` are used, some procedures are also checked to consider CODEC disabling variations. This checks all combinations with a single disabled codec (it is used as an X-chain and is only observable in single observe mode) and all other codecs enabled. When failures occur during checking with a codec disabled, an error message indicates which codec was disabled.

What Next

To correct this problem, you must change the netlist or the STIL procedure file so the `mask_enable` cells hold state during the specified procedure.

You can use automated violation analysis to debug this problem. This analysis displays all the gates in the traceback cone of the input which appear to cause the problem. The `pindata` is set to the simulated values of the failing procedure.

Use the `-cells` option of the `report_compressors` command to report the `xtol_enable`, `power_enable`, or `mask_enable` cells.

Use the `-summary` option of the `report_compressors` command to report the `codec_enable` cells with their associated enable values.

You can downgrade this rule violation to a warning. But it is not recommended since this will likely cause the pattern intervals to be incorrect.

DRC Rule K100

Message Text

`Expected seven random bits cells, found N`

This error is reported when 'N' is more than seven.

Severity

Error

Description

Random-bits registers are used for randomly selecting different sets of clocks for XLBIST. Current architecture supports up to seven registers which supply random values to clock the weight decoder logic.

What Next

To solve this problem, the netlist and the test procedure file must be changed, and random-bits registers count should be made seven or less.

This rule violation cannot be downgraded to warning.

DRC Rule K101

Message Text

N random bits cells (P/G) were disturbed during X procedure, where
N indicates the number of failing random-bits cells
P indicates the position of the failing cell in the random-bits register
G indicates the gate ID of the failing cell in the random-bits register
X indicates the name of the procedure

Severity

Error

Description

When the `xtol_enable` and `power_enable` cells are set to their off state, and codec is set to their active state, and the random-bits cell is checked to ensure that it holds state during the `SHIFT`, `RESEED_SHIFTA`, `RESEED_SHIFTB`, `RESEED_SHIFTC`, `RESEED_SHIFTD`, `RESEED_SHIFTE`, `RESEED_LOAD`, `RESEED_PARTIAL_OVERLAP_SHIFT1`, `RESEED_PARTIAL_OVERLAP_SHIFT2`, `SHADOW_TO_CAREPRPG` (with `capture_enable` set to complement of `capture_enable` value), `JTAG_MISR_SHIFT`, `OCC_SHIFT`, and capture procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, `xtol_enable/power_enable` cells set to their off state, codec enables set to their active state, and all other register types set to random values. Random-bits cells that do not hold state for all of the simulated parallel patterns result in a violation of this rule. Random-bits registers are used for randomly selecting different sets of clocks for XLBIST.

What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that random-bits cell holds state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis displays the failing random-bits cell with all the gates in the traceback cone of the input that were causing the problem. The `pindata` is set to the simulated values of the failing procedure.

This rule violation may be downgraded to a warning but it is not recommended since this likely causes patterns to be incorrect.

DRC Rule K102

Message Text

```
Set/Reset is not OFF for clock control cell 'S'.  
Clock control cell 'S' has invalid capture  
Where 'S' is the name of the BistWeightEnable gate.
```

Severity

Warning

Description

Clock weighted XLBIST functionality depends upon proper functioning of the registers specified in protocol file with tag 'BistWeightEnable'. If clocking or other asynchronous inputs of these registers are not functioning properly, clock weights mechanism would not work as expected.

What Next

To correct this problem trace back from the clock pin of the reported BistWeightEnable gate and look for the cause of clock not pulsing. You can use automated violation analysis to debug this problem. This analysis displays the failing 'BistWeightEnable' cell. The displayed pin data is set to the simulated values of the SHIFT procedure and sets all the conditions used for this checking.

DRC Rule K103

Message Text

```
Forbidden clocks 'C1' and 'C2' are grouped together.  
Where 'C1' and 'C2' are the gate-ids of clock input pin.
```

Severity

Error

Description

This message indicates clocks which are forbidden to be pulsed together are being pulsed. When clock weights are enabled for XLBIST, different sets of clocks are pulsed per pattern depending upon the randomized values of random bit registers. K103 indicates two clocks were pulsed together and these two clocks were marked as disturbed and there are too many cells that might get an 'X' during capture if these clocks are pulsed together.

Pulsing these two clocks would cause huge number of cells to capture Xs and this would cause most/all chains being masked by XLBIST. This would cause very low test coverage.

What Next

Use the `report_clocks -matrix` command and see the report about which clocks are groupable. You may have to redo DFT with proper clock grouping specifications.

You may downgrade this error to warning, but it would cause very low test coverage.

DRC Rule K104

Message Text

Disturbed clocks 'C1' and 'C2' are grouped together.

C1 and C2 are gate-ids of clocks.

Severity

Warning

Description

When clock weights are enabled for XLBIST, different sets of clocks are pulsed per pattern depending upon the randomized values of random bit registers. K104 indicates two clocks were pulsed together and these two clocks were marked as disturbed and there are cells that might get an 'X' during capture if these clocks are pulsed together. Pulsing these two clocks would cause cells to capture Xs and this would cause chains being masked by XLBIST. This would cause low test coverage.

What Next

Use the `report_clocks -matrix` command and see the report about which clocks are groupable. You may have to redo DFT with proper clock grouping specifications. You may ignore this warning, but it would cause low test coverage.

DRC Rule K105

Message Text

Clock 'C' never applied.

Where 'C' is the gate-id of the reported clock.

Severity

Warning

Description

In clock weighted XLBIST, different set of clocks are chosen at random for each pattern. Total percentage of time a clock is chosen is determined by user specification during DFT. This warning indicates that some clocks were never pulsed. This could cause some clock domain not being tested at all, causing overall low test coverage.

What Next

This might be an issue with DFT. You may check what clock specifications were there during DFT.

DRC Rule K106

Message Text

Groupable clocks 'C1' and 'C2' are not applied.

'C1' and 'C2' are clock gate-ids.

Severity

Warning

Description

Clocks which are groupable should be pulsed together in some or other pattern. Pulsing all possible clocks together gives better coverage with lesser patterns. This warning indicates clocks which were found to be safe to be pulsed together (allowed clock groups) were not pulsed in any pattern. It could be an issue with how DFT grouped the clocks with clock-weights.

What Next

This might be an issue with DFT. You may check what clock specifications were there during DFT.

DRC Rule K107

Message Text

Expected seven random bits cells, found only N

'N' is the number of random bit register.

Severity

Warning

Description

Clock weighted XLBIIST supports maximum seven random bit registers. In this case less number of registers were used. Using less number of registers restricts randomization.

What Next

Check whether user specified clocks and percentages in DFT can be handled by less than seven random bits.

DRC Rule K108

Message Text

NEW_ADDRESS/NEW_ADDRESS_SHADOW cell disturb.

NEW_ADDRESS/NEW_ADDRESS_SHADOW cell (G) of IDU S was disturbed during X procedure.

Severity

Error

Description

Note:

- G indicates the gate ID of failing NEW_ADDRESS/NEW_ADDRESS_SHADOW cell
- S indicates the IDU name
- X indicates the name of a procedure

When the XTOL_ENABLE and POWER_ENABLE cells are set to their off state and CODEC_ENABLES are set to their active state, the NEW_ADDRESS/NEW_ADDRESS_SHADOW cell is checked to ensure that it holds state during the SHIFT, RESEED_SHIFTB, RESEED_SHIFTD, RESEED_LOAD, SHADOW_TO_CAREPRPG, and SHADOW_TO_XTOLPRPG.

This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, XTOL_ENABLE/POWER_ENABLE cells set to their off state, CODEC_ENABLES set to their active state, and all other register types set to random values. NEW_ADDRESS/NEW_ADDRESS_SHADOW cells which do not hold state for all of the simulated parallel patterns results in a violation of this rule.

What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that NEW_ADDRESS/NEW_ADDRESS_SHADOW cell holds state during the specified

procedure. You can use the automated violation analysis to debug this problem. This analysis displays the failing NEW_ADDRESS/NEW_ADDRESS_SHADOW cell with all the gates in the trace-back cone of the input which was believed to have caused the problem. The pin data is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_ENABLE, power enable shadow cells.

The `-cells` option of the `report_idus` command can be used to report the NEW_ADDRESS or NEW_ADDRESS_SHADOW cells and CODEC_ENABLES with their associated enable values.

This rule violation may be downgraded to warning but it is not recommended since this likely causes patterns to be incorrect.

DRC Rule K109

Message Text

```
FINAL_SEGMENT cell disturb
```

```
FINAL_SEGMENT cell (G) of IDU S was disturbed during X procedure.
```

Severity

Error

Description

Note:

- G indicates gate ID of failing FINAL_SEGMENT cell
- S indicates the IDU name
- X indicates the name of a procedure

When the XTOL_ENABLE and POWER_ENABLE cells are set to their off state and CODEC_ENABLES are set to their active state, the FINAL_SEGMENT cell is checked to ensure that it holds state during the SHIFT, RESEED_SHIFTB, RESEED_SHIFTD, RESEED_LOAD, SHADOW_TO_CAREPRPG, and SHADOW_TO_XTOLPRPG procedures.

This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, XTOL_ENABLE and POWER_ENABLE cells set to their off state, CODEC_ENABLES set to their active state, and all other register types set to random values. FINAL_SEGMENT cells which do not hold state for all of the simulated parallel patterns result in a violation of this rule.

What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that NEW_ADDRESS/NEW_ADDRESS_SHADOW cell holds state during the specified procedure. You can use the automated violation analysis to debug this problem. This analysis displays the failing NEW_ADDRESS/NEW_ADDRESS_SHADOW cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pin data is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_ENABLE, power enable shadow cells.

The `-cells` option of the `report_idus` command can be used to report the NEW_ADDRESS or NEW_ADDRESS_SHADOW cells and CODEC_ENABLES with their associated enable values.

This rule violation may be downgraded to warning but it is not recommended since this likely causes patterns to be incorrect.

DRC Rule K110

Message Text

IDU cells disturb during RESEED_LOAD

N IDU register X cells (P/G) in seq compressor IDU S were disturbed during RESEED_LOAD procedure.

Severity

Error

Description

Note:

- `G` indicates the gate ID of the failing cell in the IDU
- `S` indicates the seq compressor IDU name
- `X` indicates the IDU register name
- `N` indicates the number of failing IDU register X cells
- `P` indicates the position of failing cell in the IDU register

When the XTOL_ENABLE and POWER_ENABLE cells are set to their off state and CODEC_ENABLES are set to their active state, the IDU cells are checked to ensure that they hold state during the RESEED_LOAD procedure.

This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and XTOL_ENABLE and POWER_ENABLE cells set to their off state, CODEC_ENABLES set to their active state, and all other register types set to random values. The IDU cells which do not hold state for all of the simulated parallel patterns result in a violation of this rule.

What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that IDU cells holds state during the RESEED_LOAD procedure. You can use the automated violation analysis to debug this problem. This analysis displays the failing IDU register cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_ENABLE, and POWER_ENABLE.

The `-cells` option of the `report_idus` command can be used to report the failings register and CODEC_ENABLES with their associated enable values.

DRC Rule K111

Message Text

IDU cells disturb during MISR_UNLOAD

N IDU register X cells (P/G) in seq compressor IDU S were disturbed during MISR_UNLOAD procedure.

Severity

Error

Description

Note:

- G indicates the gate ID of the failing cell in the IDU
- S indicates the seq compressor IDU name
- X indicates the IDU register name
- N indicates the number of failing IDU register X cells
- P indicates the position of failing cell in the IDU register

When the XTOL_ENABLE and POWER_ENABLE cells are set to their off state and CODEC_ENABLES are set to their active state, the IDU cells are checked to ensure that they hold state during the MISR_UNLOAD procedure.

This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and XTOL_ENABLE and POWER_ENABLE cells set to their off state, CODEC_ENABLES set to their active state, and all other register types set to random values. The IDU cells which do not hold state for all of the simulated parallel patterns result in a violation of this rule.

What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that IDU cells holds state during the MISR_UNLOAD procedure. You can use the automated violation analysis to debug this problem. This analysis displays the failing IDU register cell with all the gates in the trace-back cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL_ENABLE, and POWER_ENABLE.

The `-cells` option of the `report_idus` command can be used to report the failings register and CODEC_ENABLES with their associated enable values.

DRC Rule K112

Message Text

Incorrect value of Payload_cycles_counter register or
Local_cycles_counter register

N Payload(Local)_cycles_counter_reg cells(*P/G*) of IDU *S* had incorrect
values (*g/e*) during procedure *Z*, where,

N indicates number of failing Payload(Local)_cycles_counter register
cells

P indicates position of first failing Payload(Local)_cycles_counter cell

G indicates gate ID of first failing Payload(Local)_cycles_counter cell

g indicates the value of Payload(Local)_cycles_counter register

e indicates the expected value

S is the Seq compressor IDU name

Z indicates the procedure

Severity

Error

Description

Right after IDU decodes a procedure, `Payload_cycles_counter` register gets loaded from the corresponding `cycles_reg` register as follows:

Procedure	Corresponding <code>cycles_reg</code>
Reseed_load	NewAddressCyclesReg if <code>new_address_shadow</code> cell value is 1. <code>IncrAddressCyclesReg</code> if <code>new_address_shadow</code> cell value is 0.
Occ_load	OCCCyclesReg
Misr_load	MISRLoadCyclesReg
Misr_unload	MISRCyclesReg
Misr_shadow_status_unload	ShadowUnloadCyclesReg
MISR_status_unload	StatusUnloadCyclesReg
Codec_drc_load_unload	-

`Payload_cycles_counter` cells are checked to ensure that they are loaded with correct value at the end of procedure decoding for a given procedure, and their count operation is correct(should get decremented by 1 for a shift) during shift procedures. This checking is done by first initializing the state of the circuit to the previous procedure simulation values at the beginning of a particular procedure. At the end of procedure decoding, the `Payload_cycles_counter` register should have a value which is equal to the value in the corresponding `cycles_reg` of a procedure if procedure is decoded correctly. For the shift procedure, the `Payload_cycles_counter` register should have value 1 less than the value it had before shifting. `Payload_cycles_counter` register cells which are not at their correct value results in a violation of this rule.

Check for `codec_drc_load_unload` Procedure

For `codec_drc_load_unload` procedure check `Payload_cycles_counter` cell. It must be non-zero at the end of procedure decoding and zero during procedure decoding.

Similarly this check is done for `Local_cycles_counter` register also which is shown in the following table:

Procedure	Corresponding <code>local_cycles_reg</code>
Occ_load	OCCLocalCyclesReg
Misr_shadow_load	ShadowLoadLocalCycles

What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that `Payload(Local)_cycles_counter` register has a correct value after procedure decoding. The automated violation analysis can be used to help debug this problem. This analysis displays the first failing `Payload(Local)_cycles_counter` cell and the corresponding `cycles_reg` cell. The pindata is set to the simulated values of the failing procedure. Tracing the `Payload(Local)_cycles_counter` cell further gives you information as to why it is not loaded correctly. Many a times state machine flops get X during the earlier procedure simulation which causes incorrect decoding of the current procedure. The following command can be used to know the details about previous procedure simulation data:

```
Set_drc -analyze_drc_violation {K112 gate_id}
```

It displays the gate (provided by `gate_id` argument) value at the end of every procedure that is simulated, and it also stores the simulation data under pindata "`k112_debug_data`". The following option can be used to see the simulation data of a particular procedure:

```
Set_pindata -k112_debug_data {start_cycle end_cycle}
```

`Start_cycles` and `end_cycles` of procedures are printed when "`set_drc -analyze_drc_violation`" command is enabled for K112.

The `-cells` option of the `report_idus` command can be used to report the `Payload(Local)_cycles_counter` cells and various `cycles_registers`.

Hint to fix Issue With `codec_drc_load_unload` Procedure

Even if you load correct value in `Payload_cycles_counter` CDR bits, you might still get K112 error. One of the reasons could be value of `InternalLoadDelay` present in protocol file and netlist are not same.

For example:

Case 1 : If you get the following error

```
Error: payload_cycles_counter_reg cells(2/65131) of IDU <idu_name> had
incorrect
values(got=<non_zero_value>/exp=0) at delay_cycle = 2 during procedure
codec_drc_load_unload. (K112-1)
```

It is highly possible that

```
InternalLoadDelay on netlist < InternalLoadDelay on spf
```

Case 2 : If you get the following error an error:

```
Error: payload_cycles_counter_reg cells(1/65131) of IDU <idu_name> had
incorrect
```

```
values (got=0/exp=Non-zero positive value) at delay_cycle = 1 during  
procedure  
codec_drc_load_unload. (K112-1)
```

It is highly possible that

```
InternalLoadDelay on netlist > InternalLoadDelay on spf
```

DRC Rule K113

Message Text

Incorrect New(or Increment)_address_cycles_reg value

NewAddressCycles register value (d1) of IDU S is not equal to the number of cycles required to load PRPG and IR (d2).

(OR)

IncrAddressCycles register value (d3) of IDU S is not equal to the number of cycles required to load PRPG and Partial_IR (i.e. only x,b bits) d4.

Severity

Error

Description

Note:

- d1 indicates the value in the NewAddrssCycles register
- d2 indicates seq compressor IDU name
- d3 indicates the value in the IncrAddrssCycles register
- d4 indicates the number of cycles required to load PRPG and Partial_IR (i.e. only x, b bits)
- s indicates Seq compressor IDU name

At the end of test_setup or extended_test_setup (if present), the NewAddressCyclesReg and IncrAddressCyclesReg registers are checked to ensure that they are loaded with the values d1 and d2 respectively, where d1 is the number of cycles required to load the PRPG and IR, d2 is the number of cycles required to load the PRPG and Partial_IR (i.e. only x,b bits). The NewAddressCyclesReg or IncrAddressCyclesReg which are not at their correct value results in a violation of this rule.

What Next

To correct this problem, STIL protocol file must be changed so that NewAddressCyclesReg and IncrAddressCyclesReg have correct values. You can use the automated violation analysis to check the value in the NewAddressCyclesReg and IncrAddressCyclesReg. The automated violation analysis displays the first NewAddressCyclesReg or IncrAddressCyclesReg cell. The pindata is set to the simulated values of the CODEC_DRC_SHIFT procedure.

The `-cells` option of the `report_idus` command can be used to report the NewAddressCyclesReg and IncrAddressCyclesReg.

This rule violation may be downgraded to a warning but it is not recommended since this likely to cause patterns to be incorrect.

DRC Rule K114

Message Text

Incorrect loading of new_address reg

Incorrect value at pin P of new_address_reg (Gate G of IDU S) during procedure X.

Severity

Error

Description

Note:

- `P` indicates the input pin of failing NEW_ADDRESS cell
- `G` indicates the gate ID of failing NEW_ADDRESS cell
- `S` indicates the name of IDU
- `X` indicates the name of procedure

The NEW_ADDRESS cells are checked to ensure that they are successfully loading data from the NEW_ADDRESS_SHADOW during RESEED_SHIFTA procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing PAYLOAD_CYCLES_COUNTER to the value of 2. NEW_ADDRESS cells which do not successfully capture their expected value for all of the simulated parallel patterns result in a violation of this rule. The expected values for a NEW_ADDRESS cell are the values assigned to the associated NEW_ADDRESS_SHADOW cell.

What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that NEW_ADDRESS cells are properly capturing data from its NEW_ADDRESS_SHADOW during the specified procedure. You can use the automated violation analysis to debug this problem. This analysis displays the failing NEW_ADDRESS cell. If the failure occurred on a clock/set/reset input, then the pin data is set to the simulated values for the indicated procedure and all gates in the trace-back cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the associated cell in the NEW_ADDRESS_SHADOW are also displayed.

The `-cells` option of the `report_idus` command can be used to report the NEW_ADDRESS or NEW_ADDRESS_SHADOW cells.

This rule violation may be downgraded to a warning but it is not recommended since this is likely to cause patterns to be incorrect.

DRC Rule K115

Message Text

Invalid misr_status update operation

`misr_status cell (input I of G) had invalid update value during X procedure.`

Severity

Error

Description

Note:

- `I` indicates the input which caused problem of first failing cell
- `G` indicates the gate ID of failing MISR_STATUS cell
- `X` indicates the name of procedure

The MISR_STATUS cells are checked to ensure that they are successfully capturing data from the MISR register during the last clock pulse of the MISR_SHADOW_STATUS_UPDATE procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the last clock pulse in the MISR_SHADOW_STATUS_UPDATE procedure and placing random values on the MISR cells. The expected value for a MISR_STATUS cell of a CODEC is the result of ORing all the values that were assigned to the MISR register cells of that CODEC. MISR_STATUS

cells which do not successfully capture their expected value for all of the parallel patterns simulated results in a violation of this rule.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that MISR_STATUS cells are properly updating data from the associated MISR cells during the MISR_SHADOW_STATUS_UPDATE procedure. The automated violation analysis can be used to help debug this problem. This analysis displays the failing MISR_STATUS cell with pindata set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the MISR_STATUS cells.

The command usage is: `Report Compressors <-All | comp_name> [-Cells <MISR_STATUS>]`

This rule violation may be downgraded to a warning but it is not recommended since this is likely to cause patterns to be incorrect due to invalid measures associated with unloading MISR_STATUS_CELLS.

DRC Rule K116

Message Text

`MISR_STATUS cell disturb`

`MISR_STATUS cell (G) in seq compressor C was disturbed during X procedure.`

Severity

Error

Description

Note:

- `G` indicates the gate ID of failing MISR_STATUS cell
- `C` indicates the name of compressor containing failing MISR_STATUS cell
- `X` indicates the name of procedure

The MISR_STATUS register cells are checked to ensure that they hold state during the MISR_LOAD_SHIFT procedure and all other procedures where the MISR must hold state that are not associated with MISR_STATUS operations. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure. MISR_STATUS cells which do not hold state for all of the parallel patterns simulated results in a violation of this rule.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that MISR_STATUS cells hold state during the specified procedure. You can use the automated violation analysis to debug this problem. This analysis displays the failing MISR_STATUS cell with pin data set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the MISR_STATUS cells.

The command usage is: `Report Compressors <-All | comp_name> [-Cells <MISR_STATUS>]`

This rule violation may be downgraded to a warning but it is not recommended since this is likely to cause patterns to be incorrect due to invalid measures associated with unloading MISR_STATUS_CELLS.

DRC Rule K117

Message Text

`MISR_STATUS cell disturb`

`MISR_STATUS cell G was disturbed during T time %d of X procedure.`

Severity

Warning

Description

Note:

- `G` indicates the gate ID of failing MISR_STATUS cell
- `T` indicates the time during which failure occurred (preamble or postamble)
- `X` indicates the name of procedure

The slow MISR_STATUS cells are checked to ensure that they hold state during the preamble cycles and non-shifting postamble cycles of the MISR_STATUS_UNLOAD procedure. This checking is done using the precalculated simulated values of the MISR_STATUS_UNLOAD procedure. Slow MISR_STATUS cells which do not hold state for all of the checked time periods result in a violation of this rule.

What Next

To correct this problem, the netlist or the test procedure file must be changed so that failing cells hold state during the specified procedure. You can use the automated violation

analysis to debug this problem. This analysis displays the failing cell with pindata set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the `MISR_STATUS` cells.

The command usage is: `<-All | comp_name> [-Cells <MISR_STATUS>]`

This rule violation may be downgraded to a warning but it is not recommended since this is likely to cause the IP test patterns to be incorrect.

DRC Rule K118

Message Text

```
Invalid shift_counter_length
```

```
Shift Counter Length L1 is not sufficient to accommodate scan_chain X of  
length L2.
```

Severity

Error

Description

Note:

- `L1` indicates the length of the Shift Counter register
- `L2` indicates the length of the `scan_chain X`
- `x` indicates the name of the `scan_chain`

Shift Counter is loaded with the value equal to the number of shifts required to load the scan chains. Shift Counter of length L can be loaded with maximum 2^L . If the `scan_chain` length is greater than 2^L , it cannot be fully loaded using shift counter of length L .

What Next

To correct this problem, the netlist must be changed to have a shift counter of sufficient length.

DRC Rule K119

Message Text

```
N silencer_DLATs (P/G) in seq compressor S failed hold during X  
procedure.
```

where:

N indicates number of failing `silencer_DLATs`

P indicates position of the `care_shadow` cell(of the corresponding `silencer_DLAT`) in the `CARE_shadow` register

G indicates gate ID of failing `silencer_DLAT`

S indicates name of DFTMAX Seq CODEC associated with failing DLAT

X indicates name of procedure

Severity

Error

Description

`Silencer_DLATs` are checked to ensure that they are successfully holding state during the `SHIFT`, `RESEED_SHIFT`, `JTAG_RESEED_SETUP` and `MISR_UNLOAD` procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedures. `Silencer_DLATs` which do not successfully hold their value for all the simulated parallel patterns results in a violation of this rule. Only one violation is issued for all the `DLATs`.

What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that `silencer_DLATs` are holding values during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis displays the first failing `silencer_DLAT` of the `silencer_DLATs` of the associated `CARE_shadow` register. If the failure was due to a problem on a clock input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed.

The `-cells` option of the `report_compressors` command can be used to report the `CARE_Shadow` cells.

This rule violation can be downgraded to a warning but it is not recommended since this likely causes patterns which exercise the `silencer_DLAT` to be incorrect.

DRC Rule K120

Message Text

N `silencer_DLATs` (input I of P/G) in seq compressor S had invalid value during X procedure.

where:

N indicates number of failing `silencer_DLATs`

I indicates input ID of failing `silencer_DLAT`

P indicates position of the `care_shadow` cell (of the corresponding `silencer_DLAT`) in the `CARE_shadow` register

G indicates gate ID of failing `silencer_DLAT`

S indicates name of DFTMAX Seq CODEC associated with failing `DLAT`

X indicates name of procedure

Severity

Error

Description

`Silencer_DLATs` are checked to ensure that they are successfully loading data from the respective `CARE_Shadow` cells during `OCC_LOAD` procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure. `Silencer_DLATs` which do not successfully capture their expected value for all of the simulated parallel patterns results in a violation of this rule. The expected values for a `silencer_DLAT` are the values assigned to the associated `CARE_Shadow` cell in the `CARE_Shadow` register. Only one violation is issued for all the `silencer_DLATs` in a compressor.

What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that `silencer_DLATs` are properly capturing data from its `CARE_Shadow` cells during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis displays the first failing `silencer_DLAT` and the respective `CARE_Shadow` cell. If the failure occurred on a clock input, then the pin data is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing `silencer_DLAT` and the associated cell in the `CARE_Shadow` register are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the `CARE_Shadow` cells.

This rule violation may be downgraded to warning but it is not recommended since this likely causes patterns which exercise the `silencer_DLAT` to be incorrect.

Category N - Netlist Rules

See Also

- [N1 - parsing error](#)
- [N2 - unsupported construct](#)
- [N3 - maximum string size exceeded](#)
- [N4 - redefined cell](#)
- [N5 - redefined module](#)
- [N6 - duplicate definition](#)
- [N7 - missing definition](#)
- [N8 - invalid construct](#)
- [N9 - nameless instance](#)
- [N10 - ambiguous vector net direction](#)
- [N11 - redefined UDP entry](#)
- [N12 - invalid UDP entry](#)
- [N13 - X_DETECTOR found](#)
- [N14 - illegal UDP](#)
- [N15 - incomplete UDP](#)
- [N16 - overspecified UDP](#)
- [N17 - UDP failed validation](#)
- [N18 - net has name of port](#)
- [N19 - unused construct](#)
- [N20 - underspecified UDP](#)
- [N21 - unsupported UDP entry](#)
- [N22 - duplicate pin](#)
- [N23 - inconsistent UDP](#)
- [N24 - UDP cannot hold state](#)
- [N25 - redefined instantiated module](#)

- [N26 - empty model](#)
- [N27 - incomplete memory definition](#)
- [N28 - unsupported priority](#)
- [N29 - tied inout net](#)
- [N30 - inconsistent memory definition](#)
- [N31 - Name length exceeded maximum and was truncated](#)
- [N32 - Behavioral syntax detected in netlist](#)
- [N33 - Undefined port in netlist.](#)

DRC Rule N1

Message Text

Netlist Rule N1: Line *L* (*filename*), parsing error (*additional_info*).

Severity

Error or Fatal Error

Description

A syntax error was encountered while reading the netlist. Either the netlist has a syntax problem, or there is a feature of the Verilog language not supported by TestMAX ATPG.

What Next

The line number *L* in the specified *filename* identifies the area where the problem was detected. In general, the netlist file must be modified to overcome the problem.

The *additional_info* text provides clues as to the potential cause of the problem.

Sometimes the design modules contain BEHAVIORAL syntax. Only STRUCTURAL design descriptions are supported. If the syntax is correct and is structural, contact Synopsys support with a sample of the syntax you believe should be supported.

Specifying Lists in Tcl Mode

DRC Rule N2

Message Text

Netlist Rule N2: Line *L* (*filename*), unsupported entry (<additional info>).

Severity

Warning

Description

The netlist contains an unsupported construct (such as a behavioral construct).

What Next

You can ignore the violation if the information is not necessary for model build, or if the module is not referenced by your design.

DRC Rule N3

Message Text

```
Netlist Rule N3: Line L (filename), maximum size exceeded for string  
(D...).
```

Severity

Fatal Error

Description

A netlist string exceeded the maximum allowed size.

D is the first 16 characters of the string.

What Next

You can correct this by increasing the allowed size using the `set_workspace_sizes` command.

DRC Rule N4

Message Text

```
Netlist Rule N4: Line L (filename), redefined cell (<additional info>).
```

Severity

Warning

Description

An EDIF cell has already been defined in the same or another EDIF file.

What Next

Edit the EDIF files to remove the duplicate cell.

Use `set_netlist` command to define the behavior to apply when duplicate cells are encountered.

DRC Rule N5

Message Text

```
Netlist Rule N5: Line L (filename), redefined module (M).
```

Severity

Warning

Description

A module defined in any netlist should not be defined more than once. Duplicate definitions of modules might be indicative of a netlist database problem.

What Next

You can see a list of modules with multiple definitions using the `report_violations` command.

You can select that the handling of the violation use the first or the last occurrence of the definition of the module. Make the selection using the `set_netlist` command.

Edit netlist or list of files read by TestMAX ATPG to remove the redundancy if you feel the result is inaccurate.

DRC Rule N6

Message Text

```
Netlist Rule N6: Line L (filename), duplicate definition (<additional  
info>).
```

Severity

Fatal Error

Description

A module must not contain more than one entity with the same name.

What Next

This is a fatal error condition that requires changing the netlist.

DRC Rule N7

Message Text

Netlist Rule N7: Line L (filename), missing definition (M/N).

Severity

Fatal Error

Description

A module referenced an undefined entity such as an instance or a net.

M is the module name and N is the missing entity.

What Next

This is a fatal error condition that requires changing the netlist.

DRC Rule N8

Message Text

Netlist Rule N8: Line L (filename), invalid construct (<additional info>).

Severity

Fatal Error

Description

A netlist construct is used in an invalid context or with invalid arguments.

What Next

This is a fatal error condition that requires changing the netlist.

If the additional info reports "Incorrect address width of memory read port" the problem might be due to a current limitation of the TestMAX ATPG RAM syntax which does not support a RAM with an address bus of width = 1. In other words, a RAM with two addressable words. A workaround for this is to modify the RAM description to declare the address bus a vector of width 1. For example: `input [0:0] address_bus;`

If the additional info reports "Control net "N1" cannot differ from net "N2" for a RAM model, then the problem might be the order of the nets in the sensitivity list of a read or write port description. The second net, N2, which appears in the qualifying `if` clause must be the same net as the first net in the sensitivity list for *always*. For example:

```
// UNSUPPORTED sensitivity list order causes N8 Violation

always @(ADDR or READ or WRITE_OP)
  if (READ) data_out = memory[ADDR];

// After correction

always @(READ or ADDR or WRITE_OP)
  if (READ) data_out = memory[ADDR];
```

DRC Rule N9

Message Text

```
Netlist Rule N9: Line L (filename), nameless instance (<additional
info>).
```

Severity

Ignored

Description

A module instance is not given a name (this is an N1 error in netlist formats that require a name).

What Next

A unique name for the nameless instance is automatically generated. You can view all instance names with the `-verbose` option of the `report_modules` command.

DRC Rule N10

Message Text

```
Netlist Rule N10: Line L (filename), ambiguous vector net direction
(<additional info>).
```

Severity

Fatal Error

Description

Some netlists, such as Verilog, allow implicit definition of internal nets. Such nets might be parts of a vector net, whose bit direction (ascending or descending) is not known.

What Next

In most cases, the direction of the implicitly defined vector net is not required for unambiguous connectivity information and the rule violation does not occur. However, if the vector net is used as a vector entity to connect to another vector net, the direction of its bits is important and can lead to incorrect connectivity. If the rule setting is not error, a default direction is chosen and you need do nothing.

DRC Rule N11

Message Text

Netlist Rule N11: Line L (filename), redefined UDP entry (<additional info>).

Severity

Ignored

Description

A given input matches more than one UDP table entry.

What Next

The first table entry is kept and all new ones are ignored, consistent with Verilog UDP definition. In some cases, this violation might be caused by a serious UDP functionality flaw, which you should correct.

You can review the derived ATPG gate model which is used by using the `run_build_model` command to specify the module name in question. Then use the graphical schematic viewer to draw the module. Start by using SETUP to select a primitive view and then using the SHOW button and specifying a port name or the gate ID of 0. Then click the net expansion diamonds until all gates have been drawn.

See also Interpreting UDP Warning Messages in the TestMAX ATPG and TestMAX Diagnosis Messages Guide

DRC Rule N12

Message Text

Netlist Rule N12: Line L (filename), invalid UDP entry (<additional info>).

Severity

Warning

Description

A table entry is invalid.

What Next

The extracted gate representation can still be correct, but it is not correctly defined and part of the intended functionality might be missing.

You can review the derived ATPG gate model which is used by using the `run_build_model` command to specify the module name in question. Then use `add_display_gates-all`.

See also Interpreting UDP Warning Messages in the TestMAX ATPG and TestMAX Diagnosis Messages Guide

DRC Rule N13

Message Text

```
Netlist Rule N13: Line L (filename), X_DETECTOR found (<additional  
info>).
```

Severity

Warning

Description

A Boolean combination of input values in the UDP table results in the output being at a Boolean value, but changing one or more inputs to X results in the output being the opposite Boolean value. For example, in the following UDP table, the two entries, 01:0 and x1:1, describe an X-detector. This behavior cannot be modeled with Boolean gates, for which X is simply the worst case (unknown) between 0 and 1.

```
in1 in2 : out  
--- --- : ---  
0    1  : 0  
x    1  : 1
```

What Next

This behavior cannot be modeled with Boolean gates. The model is replaced with a TIEX gate.

If this module is referenced by your design you will need to come up with a suitable ATPG specific model.

DRC Rule N14

Message Text

Netlist Rule N14: Line L (filename), illegal UDP (<additional info>).

Severity

Error

Description

The UDP table contains an entry that is not allowed.

What Next

The extracted gate representation is very likely missing some of the intended UDP functionality.

You can review the derived ATPG gate model which is used by using the `run_build_model` command to specify the module name in question. Then use the graphical schematic viewer to draw the module. Start by using SETUP to select a primitive view and then using the SHOW button and specifying a port name or the gate ID of 0. Then click the net expansion diamonds until all gates have been drawn.

DRC Rule N15

Message Text

Netlist Rule N15: Line L (filename), incomplete UDP (<additional info>).

Severity

Ignored

Description

The UDP table had a missing line which the model translator was expecting. This line would have defined a non-X output value for an input combination.

Failure to satisfy this rule is not hazardous if the default `-xmodeling` option of `set_netlist` command has not been altered. However, the original Verilog UDP table should be reviewed to understand the cause of the message.

What Next

By definition, all unspecified input combinations in a UDP table result in the output being X. However, for some UDPs a boolean value can be inferred from other table entries. For example in the table below an input of "1 1" is missing and by definition means the output should be an X for this input condition. However, in translation this table into gate primitives it may be inferred that this table represent an OR function and the missing entry should indicate a 1 on the output for an input of "1 1".

```
in1 in2 : out
--- --- : ---
0    0  : 0
0    1  : 1
1    0  : 1
```

The ATPG tool issues the N15 violation for the missing "1 1" entry and based upon the `set netlist` option for X modeling, either produces an accurate (but complex) internal model which produces an X for an input of "1 1", or produces an optimized boolean gate mapping using an OR primitive.

You can review the derived ATPG gate model which is used by using the `run_build_model` command to specify the module name in question. Then use the graphical schematic viewer to draw the module. Start by using `SETUP` to select a primitive view and then using the `SHOW` button and specifying a port name or the gate ID of 0. Then click the net expansion diamonds until all gates have been drawn.

See also Interpreting UDP Warning Messages in the TestMAX ATPG and TestMAX Diagnosis Messages Guide

DRC Rule N16

Message Text

```
Netlist Rule N16: Line L (filename), overspecified UDP (<additional
info>).
```

Severity

Warning

Description

The UDP table specified an entry with a 0 or 1 as an output but the derived ATPG gate level model produces an X. The line number (L) will usually reference the "endtable" marker and not indicate the actual line number which contributed to the rule violation.

This usually is an indication of defined simulation behavior that has no exact match as a gate level equivalent. The UDP behavior for simulation has a different goal in mind than the needs of the ATPG algorithm and it is not uncommon for this condition to occur.

The extracted ATPG gate level representation is not incorrect, it is just more pessimistic than the UDP table as it predicts an X on an output where the original table produces a non-X.

Violation of this rule will cause no danger of creating bad patterns and generally the test coverage will not be reduced.

What Next

You can ignore these warnings for ATPG efforts. If you wish, you can use the `set_rules` command to change the severity level to ignore.

If you are a library developer or simulation model builder you might be interested in reviewing the original UDP table. Perhaps a few edits will eliminate the warning messages.

You can review the derived ATPG gate model which is used by using the `run_build_model` command to specify the module name in question. Then use the graphical schematic viewer to draw the module. Start by using SETUP to select a primitive view and then using the SHOW button and specifying a port name or the gate ID of 0. Then click on the net expansion diamonds until all gates have been drawn.

```
# An example which results in N16 violations.
```

```
primitive MUX ( Q, A, B, SL );
output Q;
input A, B, SL;

table
#   A B SL: Q
#   -----
    0 ? 0 : 0; # 1:
    1 ? 0 : 1; # 2:
    ? 0 1 : 0; # 3:
    ? 1 1 : 1; # 4:
    0 0 x : 0; # 5:
    1 1 x : 1; # 6:
    0 1 x : 1; # 7: causes N16
    x 1 x : 1; # 8: causes N16
endtable
endprimitive
```

See also Interpreting UDP Warning Messages in the TestMAX ATPG and TestMAX Diagnosis Messages Guide

DRC Rule N17

Message Text

Netlist Rule N17: Gate-level model of UDP failed validation.

UDP not validated (<additional info>).

Severity

Error

Description

The symbolic comparison between the circuit as defined in the UDP table and the extracted Boolean gate representation failed.

What Next

The extracted gate representation can still be correct, but too complex or not correctly defined.

You can review the derived ATPG gate model which is used by using the `run_build_model` command to specify the module name in question. Then use the graphical schematic viewer to draw the module. Start by using SETUP to select a primitive view and then using the SHOW button and specifying a port name or the gate ID of 0. Then click the net expansion diamonds until all gates have been drawn.

See also Interpreting UDP Warning Messages in the TestMAX ATPG and TestMAX Diagnosis Messages Guide

DRC Rule N18

Message Text

Netlist Rule N18: Line L (*filename*), Internal net and port have the same name (*additional_info*).

Severity

Warning

Description

An internal net has the same name as an external port of the same module.

What Next

If the setting is not error, the internal net is renamed by suffixing "_intern". If a net by this name already exists, a unique number is also appended to the name.

Note that renaming the net name due to this violation might cause the following issues:

Bridging fault annotation might not match.

Milkyway database might not match netlist in diagnostics.

DRC Rule N19

Message Text

Netlist Rule N19: Line L (filename), unused construct (<additional info>).

Severity

Ignored

Description

The netlist contains a construct not used for model build (such as a timing construct).

What Next

TestMAX ATPG ignores all timing, so no user action is required.

DRC Rule N20

Message Text

Netlist Rule N20: Line L (filename), underspecified UDP (Expected *string* got *string*).

Severity

Warning

Description

The UDP table defined an input combination for which the output produced is X but the ATPG derived gate-level model for the table predicts a non-X value. The line number (L) will reference the "endtable" line from the UDP.

The most common cause of this rule violation is a missing table entry. By definition, any input combination not explicitly given in the UDP table is assumed to produce an X for an output. When a violation occurs the derived ATPG gate-level model predicts a non-X output where the UDP table predicts an X either by an explicit line or by omitting a line.

Violation of this rule will not affect test coverage but can result in patterns created which fail in simulation. When the ATPG model predicts a non-X but the simulation model

predicts X there is a potential for a mismatch. The patterns can actually work fine on real silicon but this is hard to validate if the simulation is failing.

What Next

Usually the derived gate-level is correct with respect to silicon and the UDP table has omitted some entries either intentionally or unintentionally that are needed to reduce modeling pessimism.

It is rare that patterns will fail simulation. You should generate some test patterns to check the outcome. If simulation of patterns is failing, discuss with your model supplier the particular UDPs which experience these N20 violations and see if some of the pessimism can't be removed.

If you are a library developer or simulation model builder, you might be interested in reviewing the original UDP table. Perhaps a few edits will eliminate the warning messages.

You can review the derived ATPG gate model which is used by using the `run_build_model` command to specify the module name in question. Then use the graphical schematic viewer to draw the module. Start by using `SETUP` to select a primitive view and then using the `SHOW` button and specifying a port name or the gate ID of 0. Then click on the net expansion diamonds until all gates have been drawn.

An example which results in N20 violations.

```
primitive MUX ( Q, A, B, SL );
output Q;
input A, B, SL;
table
#   A B SL: Q
# -----
    0 ? 0 : 0;
    1 ? 0 : 1;
    ? 0 1 : 0;
    ? 1 1 : 1;
```

omitting the following lines produces N20 violations of:
underspecified UDP (Expected "??x:t" got "x"). (N20-1)

```
# 0 0 x : 0;
# 1 1 x : 1;
endtable
endprimitive
```

A second example which results in N20 violations where the intended functionality was a 2-input majority logic gate but a number of entries to reduce pessimism were missing.

```
primitive MAJORITY (Z, A, B, C);
output Z;
input A, B, C;
```

```

table
# A B C : Z
# -- -- -- : ----
    0 0 0 : 0 ;
    0 0 1 : 0 ;
    0 1 0 : 0 ;
    0 1 1 : 1 ;
    1 0 0 : 0 ;
    1 0 1 : 1 ;
    1 1 0 : 1 ;
    1 1 1 : 1 ;
endtable
endprimitive

# The revised model to remove pessimism and eliminate N20.

primitive MAJORITY (Z, A, B, C);
output Z;
input A, B, C;
table
# A B C : Z
# -- -- -- : ----
    0 0 ? : 0 ;
    0 ? 0 : 0 ;
    ? 0 0 : 0 ;
    1 1 ? : 1 ;
    1 ? 1 : 1 ;
    ? 1 1 : 1 ;
endtable
endprimitive

```

See also Interpreting UDP Warning Messages in the TestMAX ATPG and TestMAX Diagnosis Messages Guide

DRC Rule N21

Message Text

Netlist Rule N21: Line L (filename), unsupported UDP entry (Entry *string*)

Severity

Warning

Description

A UDP table entry is unsupported and is ignored when deriving an ATPG gate-level representation of the UDP. The line number (L) indicates the line in the file of the table entry that is unsupported.

This violation occurs most frequently when a non-functional table entry is detected. This is common when the "notify register" technique is used for UDP table definitions. The notify register is a dedicated input which is encoded such that any event on this input causes the output to change state. This input is usually triggered by a setup/hold or other timing check that sees a violation and wishes the output to change to X. There is no gate level representation which would provide this functionality but that is OK as this input never changes in an ATPG environment and is considered non-functional.

Violation of this rule will not affect test coverage or result in patterns which fail simulation.

What Next

In most cases, no corrective action is required. However, if you are a purist, you will review the original UDP table to make sure the notify register technique exists. If it is not there, there is some other sort of problem which must be investigated.

You can eliminate these warnings by using the `set_rules` command to change the severity level to ignore.

You can review the derived ATPG gate model which is used by using the `run_build_model` command to specify the module name in question. Then use `add_display_gates-all`.

```
# An example that produces an N21 violation
primitive FLOP ( Q, D, CK, NR );
output Q;
input D, CK, NR;
reg Q;
  table
    # D CK NR : Q- : Q+
    #-----
    0 (01) ? : ? : 0;
    1 (01) ? : ? : 1;
    ? 0 ? : ? : -;
    0 (0x) ? : 0 : 0;
    1 (0x) ? : 1 : 1;
    ? ? * : ? : x; # this is unsupported, N21
  endtable
endprimitive
```

DRC Rule N22

Message Text

Netlist Rule N22: Line L (filename), duplicate pin (<additional info>).

Severity

Warning

Description

A netlist module has two or more external pins (such as two bidirectionals) with the same name.

What Next

If this is intended functionality and not a typographical error, no action is required. All but the first pin is renamed to create unique names, and all these pins are connected together bidirectionally.

DRC Rule N23

Message Text

```
Netlist Rule N23: Line L (filename), inconsistent UDP entry (<additional info>).
```

Severity

Warning

Description

An entry in the UDP table conflicts with clock information derived from previous entries. This entry is ignored in deriving the ATPG gate-level model. The line number (L) indicates the line in the file on which the violation occurred.

A common cause of this violation is an entry which is inconsistent with prior entries defining clock or asynchronous set/reset behavior. For example, from previous entries it has been inferred that input *s* is the asynchronous set of a latch and that input *r* is its asynchronous reset. If there exists an entry that shows the state of the latch being unchanged when *s* and *r* are simultaneously active then this is considered inconsistent with prior inferred behavior. This entry would not be considered inconsistent if a previous entry in the UDP table had explicitly defined an input combination with *s* and *r* simultaneously on.

A violation of this rule will not affect test coverage but might be an indication that patterns generated could fail simulation. This is possible whenever the derived ATPG gate-level model does not match the simulation model.

What Next

The extracted gate representation can still be correct. However, the inconsistent entry represents a potential problem and can indicate that the UDP is flawed. Generate some sample ATPG patterns and then simulate them to see if there is a problem. If so, consult with your library supplier on the UDP in question.

You can review the derived ATPG gate model which is used by using the `run_build_model` command to specify the module name in question. Then use the graphical schematic viewer to draw the module. Start by using SETUP to select a primitive view and then using the SHOW button and specifying a port name or the gate ID of 0. Then click the net expansion diamonds until all gates have been drawn.

```
# An example which produces an N23 violation of:
# (Entry ?1*?:- conflicts with clocks offstate)

primitive N23_SAMPLE ( Q, D, CK, V );
input D, CK, V;
output Q;
reg Q ;
table
# D    CK    V    : Q- : Q+
#-----
  (?0) 1    ?    : ?   : 0 ; # CK=1, latch open
  (?1) 1    ?    : ?   : 1 ;

    *    0    ?    : ?   : - ; # CK=0, hold

    0 (?1)  ?    : ?   : 0 ; # open latch
    1 (?1)  ?    : ?   : 1 ;

    ? (?0)  0    : ?   : - ; # close latch, hold

    0 *      0    : 0   : 0 ; # reducing pessimism
    1 *      0    : 1   : 1 ;

    ? 1      *    : ?   : - ; # N23, logic flaw??
endtable
endprimitive
```

See also Interpreting UDP Warning Messages in the TestMAX ATPG and TestMAX Diagnosis Messages Guide

DRC Rule N24

Message Text

Netlist Rule N24: Line L (filename), UDP cannot hold state (<additional info>).

Severity

Warning

Description

A sequential UDP's table has no entry that holds state. Thus, the UDP does not represent a state element (latch or flip-flop) with optional combinational gates around it. The gate representation extracted does not include state elements, but can include feedback paths.

What Next

In many cases, such UDP represents a pulse processor in the clock logic. In these cases, the extracted representation is adequate for ATPG. If, however, the UDP intends to represent a state element, particularly a scan cell, the table is erroneous and the extracted gate representation will fail scan trace rules.

You can have to create your own ATPG specific model for this library cell.

DRC Rule N25

Message Text

```
Netlist Rule N25: Line L (filename), Module (M) has been previously  
defined and instantiated.
```

Severity

Error

Description

This violation is raised when all of the following conditions are met:

module A has been defined and has at least one vector port

module B instantiates module A by name

module A is redefined after it has been defined and instantiated

the setting of `set_netlist` is `-redefined_modulelast`.

The reason for the violation is that the new definition of module A might redefine some vector port differently than its first definition, and the first definition has already been used to create the connectivity of module B.

By default, the severity is error and the first definition is not replaced.

What Next

Any one of the following resolves the error:

Remove all but one of the definitions for module A and reread the netlist.

Use the command `set_netlist-redefined_module first`.

If the vector ports are the same in the new and old definitions or if module A is not used in the simulation model, set rule N25 to warning or ignored.

DRC Rule N26

Message Text

Netlist Rule N26: Line L (filename), empty model (<additional info>).

Severity

Warning

Description

Reading and analysis of a library module resulted in an empty ATPG model. If the module is referenced by the design an empty model will produce other problems such as TIEX nets, undriven nets, and often scan chain blockages.

An empty model can result from such items as:

- violation of a rule with severity of error

- violation of rule N13 (XDETECTOR), independent of the severity of N13

- violation of rule N15 (incomplete UDP) and a combination model.

What Next

Review the model and original source module to see if a change to the source might be helpful. You might be required to write your own ATPG model to overcome this problem.

In rare UDP cases, some benefit can occur by setting rule N15 to ignore.

DRC Rule N27

Message Text

Netlist Rule N27: Line L (filename), incomplete memory definition (<additional info>).

Severity

Warning

Description

This violation occurs when a behavioral Verilog memory model is recognized as a TestMAX ATPG supported model, but it misses some features or has some unexpected or unsupported syntax.

The occurrence message of this message is detailed in specifying the cause of the violation (for example, old memory models did not include the write-to-read update notification done by signaling an event). The occurrence message is displayed with the `report_violation` command.

It is also possible for this message to occur because of unrecognized or unexpected syntax. In these cases, the detailed text might be inaccurate and indicate the problem is a "Net XXX should be declared as REG" when in fact some other problem is the real cause. In these situations, you should view carefully near the line numbers of the file indicated in the N27 violation. Compare the syntax near that line with the supported syntax for memory modeling.

In some cases, non-TestMAX ATPG memory models can raise some N27 violations before they are recognized as non-TestMAX ATPG models. After they are identified as non-TestMAX ATPG models, you should see N2 violations but no further N27 violations.

What Next

Review the original library module definition and consider making the changes suggested.

DRC Rule N28

Message Text

```
Netlist Rule N28: Line L (filename), unsupported priority (<additional info>).
```

Severity

Warning

Description

This violation occurs when a Verilog UDP model describes behavior in which the simultaneous application of set and reset, or set and clock, or reset and clock produces known results. The TestMAX ATPG primitive for a DLAT does not fully support this priority behavior for all ATPG algorithms.

What Next

This message is mainly information of the current limitations of the ATPG tool for modeling set/reset priorities and other types of priorities. There is really very little you can do. The pessimistic ATPG model can result in a lower test coverage, but does not present a danger of creating patterns that fail in simulation.

Most common forms of set/rst/clk prioritization are supported for fault grading of functional patterns. This warning can be ignored as its effect has a potential lowering of test coverage.

See also Interpreting UDP Warning Messages in the TestMAX ATPG and TestMAX Diagnosis Messages Guide

DRC Rule N29

Message Text

```
Netlist Rule N29: Line L (filename), inout net tied to supply  
(<additional info>).
```

Severity

Error

Description

This violation occurs when module port defined to be a bidirectional port is connected to a strong driver such as a supply0 or supply1 net. This usually indicates a netlist error.

What Next

After reviewing the source of the error you might want to modify the netlist. If you wish to ignore this error and continue, then change the severity of this rule to either warning or ignore using the `set_rules` command and retry the `run_build_model` command.

DRC Rule N30

Message Text

```
Netlist Rule N30: Inconsistent memory definition
```

Severity

Error

Description

TestMAX ATPG has limited support for a single model that supports both simulation and ATPG by ignoring the #nnn delays when they appear. In order to get the simulation of a RAM model to work, it is necessary to have non-zero delay between the write port clock and the data appearing at the read port outputs. This is accomplished by putting "#nnn" delays into the model. TestMAX ATPG can limitly support a single model that supports both simulation and ATPG by ignoring the #nnn delays when they appear.

What Next

Examples and explanations:

```
"Edge-triggered read port", clk->name(), "cannot capture new memory data"
"Previous edge-triggered read port", rconn->net->name(), "cannot capture new memory data"
```

Edge-triggered ports *always* capture old data; that's the nature of edge-triggered.

You need to modify your memory model.

```
"Write port delays reflect dominance, not X-behavior"
```

Mismatch between the read/write delays and the explicitly declared behavior (``define`). You need to modify your memory model.

```
"LS read always reads new data and its delay should be > write port delay %d", maxWdelay
"LS read always reads new data and its delay %d should be > write port delay %d", minLSRdelay, maxWdelay
"Edge-read delay should be > write delay %d for read_write=new", minWdelay
"Edge-read delay %d should be > write delay %d for read_write=new", maxERdelay, minWdelay
"Write delay should be > edge-read delay %d for read_write=mixed", minERdelay
"Write delay %d should be > edge-read delay %d for read_write=mixed", maxWdelay, minERdelay
```

Definition of the error is provided in the statements above.

DRC Rule N31

Message Text

Netlist Rule N31: Line *L* (*filename*), Name length exceeded maximum and was truncated (the *name_length* is longer than the *supported_length*).

Severity

Warning

Description

The length of a netlist module instance name is longer than the length supported for internal names. Long instance names are truncated because they consume significant memory and affect all TestMAX ATPG operations before reading the netlist. The name-truncation process assigns the instance a different name from the original name in the design.

What Next

The line number (*L*) in the file name (*filename*) identifies the location of the detected problem. Modify the netlist to reduce the length of the instance name and all references to that name.

DRC Rule N32

Message Text

```
Netlist Rule N32: Line L (filename), Behavioral syntax detected in  
netlist (additional information).
```

Severity

Warning

Description

The netlist contains behavioral syntax/constructs (complex assign statements). Other behavioral constructs are flagged by DRC Rule N2.

What Next

You can ignore the violation if the information is not necessary for model build, or if the module is not referenced by your design.

DRC Rule N33

Message Text

```
Netlist Rule N33: Undefined port in netlist.
```

Severity

Warning

Description

In the netlist module, report a warning message for pins that are not connected with the net name. Use `report_violations N33` to get the pin names.

What Next

You can eliminate these warnings by using the `set_rules` command to change the severity level to ignore.

An example that produces a N33 violation is as follows:

```
=====
module CORE ( CLK, SE, SI, DIN, DOUT );
  input CLK, SE, SI, DIN;
  output DOUT;
  SDFQX1 REG_1 ( .CK(CLK), .SE(SE), .SI(SI), .D(DIN), .Q(DOUT) );
endmodule
module TOP ( CLK, SE, SI, DIN, DOUT );
  input CLK, SE, SI, DIN;
  output DOUT;
  CORE
  ( .CLK(CLK), .SE(SE), .SI(SI), .DIN(DIN), .DOUT(DOUT), .N33_PORT() );
Endmodule
=====
```

In the CORE module, N33_PORT() is not connected to any net name.

Category R - Compressor Rules

See Also

- [R1 - Compressor references an undefined chain.](#)
- [R2 - Chain input not connected to a load compressor.](#)
- [R3 - Chain output not connected to an unload compressor.](#)
- [R4 - Load compressor U_decompressor does not have an internal connection for output X during mode Y.](#)
- [R5 - Missing load compressor input connection to a port.](#)
- [R6 - Missing unload compressor output connection to a port.](#)
- [R7 - Load compressor U_decompressor failed propagation checking for chain X during mode Y.](#)
- [R8 - Incorrect unload compressor operation](#)
- [R9 - Two chains always have the same load value.](#)
- [R10 - Two chains always have the same unload signature](#)
- [R11 - X on chain affects observe ability of other chains.](#)
- [R12 - Simultaneously satisfiable modes for a compressor.](#)
- [R13 - Single observe mode connection for chain to output gate of unload compressor can be affected by X on other chains.](#)

- R14 - X source T gate (G1) is propagatable to N scancells.
- R15 - X source T gate (G1) is unblockable to N scancells.
- R16 - Unidentified unload compressor input.
- R17 - Unidentified unload compressor output.
- R18 - Load compressor pipeline cell not holding state.
- R19 - Missing unload compressor pipeline lockup latch.
- R20 - Missing load compressor pipeline lockup latch.
- R21 - X-tolerant chain connection to output port for unload mode with load_mode was not found.
- R22 - Chain chain_name has no x-tolerant observation.
- R23 - Unload mode enable <mode_name> at 0 does not condition full compression on output port of port_name.
- R24 - Unable to infer output of chain C for unload compressor
- R25 - No X-tolerant observation for chain during a load mode
- R26 - Invalid PLL clock chain connections for load compressor
- R27 - Load compressor pipeline cell not holding or shifting
- R28 - Clock chain connected to multiple ports
- R29 - Clock chain port used for X-tolerance control
- R30 - Two nonclock chains not independent ignoring clock chain inputs
- R31 - Incorrect defined multi chain X-tolerant connection to output N of unload compressor X using mode L/U
- R32 - Unload compressor had invalid capturing edge on cell from load compressor cell
- R33 - Unable to find output of unload compressor
- R34 - Multiple candidates for output of unload compressor
- R35 - Unable to find input of load compressor
- R36 - Multiple candidates for input of load compressor
- R37 - Scan cell was clocked during serializer nonshifting cycle
- R38 - Scan cell was not clocked during serializer shifting cycle

- R39 - Unload compressor decompressor failed propagation checking for chain in unload mode
- R40 - Could not find flop at index of Load Serializer
- R41 - Incorrect single fanout connection for a chain
- R42 - Load compressor pipeline cell ID disturbed at time T of P procedure
- R43 - Load compressor pipeline cell ID disturbed during serializer initial shift cycles at time T of P procedure
- R44 - Pipeline cell ID between load serializer and load compressor does not hold state during capture
- R45 - Load compressor pipeline cell ID must not be constrained on PI_NAME
- R46 - Serializer output compressor pipeline discrepancy
- R47 - Nonscan cell cannot control PLL internal clocking
- R48 - Load pipeline path cannot have multiple outputs with inconsistent inversions on different paths
- R49 - Load compressor is driven by a scan input with only dynamic bits
- R50 - Incorrect SPC chain structure
- R51 - Incorrect SPC update latch asynchronous control
- R52 - Incorrect SPC load compressor propagation
- R53 - SPC control register not holding state
- R54 - SPC control register is not capturing a constant value
- R55 - SPC chain is not in one to one mode
- R56 - Decompressor or compressor has different tail or pipeline stages
- R57 - Compressor scan assignment is missing in the load_unload procedure shift vector
- R58 - ScanEnable not constrained to LOGICAL LOW
- R59 - Failed to extract SPC register due to incorrect number of pipeline stages
- R60 - Unload compressor output pipeline cell name on output_signal does not hold during capture
- R61 - Invalid SPC register connection to cell_name
- R62 - Ultra register Cell-name (Cell-id) disturbed during pre-shift of load/unload.

- [R63 - Two chains always have the same unload signature was violated N times](#)
- [R64 - LE to TE transition from serial flop to the first scan flop.](#)

DRC Rule R1

Message Text

Compressor Rule R1: Compressor references an undefined chain.

Severity

Fatal Error

Description

Indicates an Error in protocol file. A chain referred to in CompressorStructures section is not defined as scan chain in the ScanChain section

What Next

If the protocol file is generated by TestMAX DFT, please submit a testcase.

DRC Rule R2

Message Text

Compressor Rule R2: Chain input not connected to a load compressor.

Severity

Fatal Error

Description

Indicates an error in the netlist file. An internal chain input is not connected to the decompressor.

What Next

If the netlist is generated by the TestMAX DFT, please submit a testcase.

DRC Rule R3

Message Text

Compressor Rule R3: Chain output not connected to an unload compressor.

Severity

Fatal Error

Description

Indicates an error in the netlist file. An internal chain output is not connected to the compressor.

What Next

If the netlist is generated by the TestMAX DFT, please submit a testcase.

DRC Rule R4

Message Text

Compressor Rule R4: Load compressor U_decompressor does not have an internal connection for output X during mode Y.

Unload compressor U_compressor does not have an internal connection for output X.

Severity

Fatal Error

Description

The first version of this violation indicates that DRC failed for decompressor (also called the load compressor). This failure indicates that the decompressor internal connections in the netlist do not match what is specified in the STIL procedure file.

The second version of this violation indicates that DRC failed for compressor named U_compressor (also called the unload compressor). This failure indicates that the compressor internal connections in the netlist do not match what is specified in the STIL procedure file.

There is no GUI support for an R4 violation.

What Next

This error occurs when you use the `-hookup` option of the `set_scan_signal` DFT Compiler command during implementation, but do not sensitize the path between scan-input pin and hookup path properly with the constraints. If you manually modified the netlist or the STIL procedure file, recheck your changes.

This violation is related to an issue in TestMAX DFT. See the TestMAX DFT documentation for more details.

DRC Rule R5

Message Text

Compressor Rule R5: Missing load compressor input connection to a port.

Severity

Fatal Error

Description

An R5 violation indicates that the decompressor input specified in the protocol file is not connected to any port.

What Next

If you used DFT compiler to generate the netlist and protocol file, please submit a testcase to Synopsys.

DRC Rule R6

Message Text

Compressor Rule R6: Missing unload compressor output connection to a port.

Severity

Fatal Error

Description

This violation indicates that the compressor output specified in the protocol file is not connected to any port.

What Next

If you used DFT compiler to generate the netlist and protocol file, please submit a testcase to Synopsys.

DRC Rule R7

Message Text

Compressor Rule R7: Load compressor U_decompressor failed propagation checking for chain X during mode Y.

Severity

Fatal Error

Description

An R7 violation indicates that DRC failed while verifying internal connections of the decompressor or the load compressor. There is no GUI support for this violation.

What Next

If M624 warnings are generated, you should directly identify the combination of problem gates and problem states for the mode. This tracing methodology is explained in detail in the description of the M624 message in TestMAX ATPG Help.

Alternatively, specify the following commands:

```
set_rules r7 warning  
  
set_pindata -shift  
  
run_drc
```

This methodology enables you to continue the DRC process and access TEST mode. From TEST mode, list the R7 violations using the `report_violations r7` command, then debug one of the violations. You can report the contents of the chain violations using the `report_scan_cells X` command, where “X” is the chain listed as violated in the R7 violation report.

Since the R7 violations highlight a discrepancy between a chain and the load-compressor definition, identify the chain “X” cell with the most significant scan cell order number. This is the cell closest to the scan-in side (at the bottom of the list of cells reported by `report_scan_cells`). Open the GSV and add the scan cell to the display using the `add_display_gates C` command, where “C” is the name of the scan cell closest to scan in.

Click on the data input to the flip-flop. Make sure it connects to a scan input port and the mode select pins in the STIL procedure file.

Open the STIL procedure file and search for the unload compressor name, “U_decompressor,” listed in the R7 rule violation. It might look like this:

```
CompressorStructures {  
    Compressor  
    "DFT__des_chip_core_U_decompressor_ScanCompression_mode" {  
        ModeGroup mode_group;  
        LoadGroup load_group;  
        CoreGroup core_group;  
        Modes 3;  
        Mode 0 {  
            ModeControls {
```

```

        "ci_i_rdata_de[7]" = 0;
        "spare_test_si" = 0;
    }
    Connection 0 "2" "9" "15" "20" "26" "32" "38" "45" "51" "56"
    "63"
        "69" "74" "80" "85" "91" "96";
    ...

```

Search for chain “X” in the list of connections listed under the mode “Y” specified in the R7 rule violation. The mode control pins are listed in the `ModeControls` block.

Connection 0 (as in the example, above) refers to “sccompin0” which points to the pin “ci_i_rdata_de[1]”, as shown below:

```

ScanChain "sccompin0" {
    ScanIn "ci_i_rdata_de[1]";
}

```

In the example, “Connection 0” would list the chain “X” called out by the R7 violation. The “0” of “Connection 0” is the same as the “0” of “sccompin0”. Make sure you use the Connection and Mode with chain “X” listed in it, and called out by the R7 violation.

DRC Rule R8

Message Text

Compressor Rule R8: Incorrect unload compressor operation Default
Severity: Error

Severity

Error

Description

This violation indicates that the compressor connections specified in protocol file do not match the connections in the netlist.

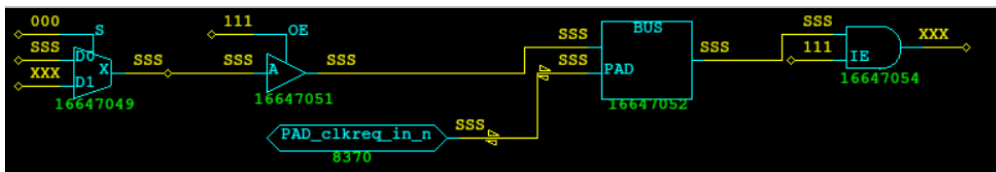
What Next

If you used DFT compiler to add compressors and generate the netlist, submit a test case to Synopsys. For more information, see the `-set_unload_mode_ports_to_x` option of the `set_drc` command.

For R8 violation, the `analyze_violation` command cannot give clear information for debugging in the following case.

One possible cause may be associated with the bus device, which is a gate inserted by TestMAX ATPG during ATPG design building to resolve multidriver nets. Both potential sources for the bus inputs appear to be driving, and both have values of X. One of

the sources that has an X value is the `PAD_clkreq_in_n` bidirectional port. You can correct this by driving the port to a Z state. You edit the STIL file to add the declaration `PAD_clkreq_in_n = Z` to one of the `V{..}` vectors at the start of the `load_unload` procedure.



DRC Rule R9

Message Text

Compressor Rule R9: Two chains always have the same load value

Severity

Warning

Description

This message indicates that the current chain count exceeds the maximum load-independence threshold. This threshold is the maximum number of chains that can be used when creating a load decompressor in which any two chains can be load-independent values (for example, any two chains can load 00, 01, 10, or 11, as needed).

The following table shows the threshold levels for the number of scan inputs and the maximum number of any two load-independent chains for a high X-tolerance codec:

Number of Scan Inputs*	Maximum Number of Any Two Load-Independent Chains
2	1
3	2
4	4
5	16
6	81
7	256
8	625
9	1296

Number of Scan Inputs*	Maximum Number of Any Two Load-Independent Chains
10	2401
11	4096
12	6561
13	10000
14	14641
15	20736
20	83521
25	234256
30	531441

* In a serializer flow, the first column ("Number of Scan Inputs") refers to the number of combinational decompressor scan inputs driven by the deserializer, as specified by the `-inputs` option of the `set_scan_compression_configuration` command in DFT Compiler.

What Next

Either increase the number of scan input pins or decrease the number of chains in the load decompressor.

DRC Rule R10

Message Text

Compressor Rule R10: Two chains always have the same unload signature was violated X times.

Severity

Warning

Description

This message indicates that the threshold was exceeded for the maximum number of chains that can be used to create an unload compressor without aliasing (no two chains have errors that cancel out each other). This violation is dependent on the compressor

architecture and not the design; it might reduce diagnostic effectiveness and lower diagnostic precision.

The following table shows the thresholds for the number of scan outputs and the maximum number of chains for creating an unload compressor with no aliasing:

Number of Scan Outputs*	Maximum Number of Chains With No Aliasing
2	3
3	7
4	15
5	31
6	63
7	127
8	255
9	510
10	1012
11	1980
12	3796
13	7098
14	12910
15	22818
16 and higher	32000

* In a serializer flow, the first column ("Number of Scan Outputs") refers to the number of combinational compressor scan outputs that drive the serializer, as specified by the `-outputs` option of the `set_scan_compression_configuration` command in DFT Compiler.

What Next

An R10 violation is usually caused by a high compression ratio. This violation also might affect diagnostic accuracy. To eliminate this violation, you can either increase the number of scan output pins or decrease the number of affected chains in the unload compressor.

DRC Rule R11

Message Text

Compressor Rule R11: X on chain affects observability of other chains

Severity

Warning

Description

This message indicates that the threshold was exceeded for the maximum number of chains that can be used to create an unload compressor in which a single X does not mask observation on any other chain in a cycle (that is, no two chains can have errors that cancel out each other).

This condition might affect coverage in the default X-tolerant flow, and might affect the compression capability in any X-tolerant flow.

The following table shows the threshold levels for the number of scan output pins and the maximum number of chains that can be used to create an unload compressor in which a single X does not mask observation on any other chain in a cycle:

Number of Scan Outputs*	Maximum Number of Chains With Single X-Tolerance
2	2
3	3
4	6
5	10
6	20
7	35
8	56
9	84
10	120
11	165
12	220
13	286

Number of Scan Outputs*	Maximum Number of Chains With Single X-Tolerance
14	364
15	455
16	560
17	680
18	816
19	969
20	1140
21	1330
22	1540
23	1771
24	2024
25	2300
26	2600
27	2925
28	3276
29	3654
30	4060

Number of
Scan Outputs*

* In a serializer flow, the first column ("Number of Scan Outputs") refers to the number of combinational compressor scan outputs that drive the serializer, as specified by the `-outputs` option of the `set_scan_compression_configuration` command in DFT Compiler.

What Next

Either increase the number of scan output pins or decrease the number of affected chains in the unload compressor.

DRC Rule R12

Message Text

Compressor Rule R12

Simultaneously satisfiable modes for a compressor

Severity

Fatal Error

Description

The load compressor modes must be mutually exclusive; that is, only one mode is active at the time. A violation of this rule typically indicates an error in the STL procedure file. For example:

```
Mode 0 {
  ModeControls {
    "test_si14" =0;
    "test_si15" =0;
  }
Mode 1 {
  ModeControls {
    "test_si14" =0; # same as in Mode 0 !!
  }
}
```

What Next

You need to correct the STL procedure file (and the netlist if needed). Keep in mind that the STL procedure file must match the netlist, otherwise other rule violations will occur.

DRC Rule R13

Message Text

Compressor Rule R13: Single observe mode connection for chain
<chain_name> to output gate <gate_id> of unload compressor
<compressor_name> using mode <load_mode>/<unload_mode> can be affected by
X on other chains.

Severity

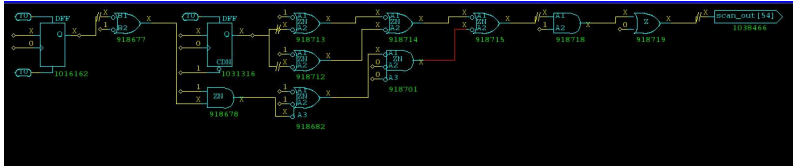
Error

Description

One-to-one modes are X-tolerant compression modes in which each scan output port is connected to only one compressed scan chain. A violation of this rule means that the

path from the last scan register of the compressed scan chain to the scan output port is not sensitized at each logic level. This is often caused by reconvergent logic on another compressed scan chain's path through the compressor, such that the one-to-one mode is sensitized when the other scan chain output is either 0 or 1, but the simulation resolves to X when it is X.

Example



The figure shows an R13 violation on the one-to-one mode path from the DFF on the left side to the scan output port on the right side. Data from the DFF in the center fans out to satisfy different compressor modes, but then reconverges to block the one-to-one mode path at the gate driven by the highlighted red net. The data on this DFF can be X for a number of reasons including capturing data from nonscan cells or from timing exception paths.

What Next

The reconvergent logic is not in the original compressor design, but is added by synthesis optimizations. The compressor should be inserted again, with synthesis optimizations disabled for it.

Downgrading this violation will result in Verilog simulation mismatches, but only when serial simulation is performed. (Parallel simulation does not simulate the data path through the compressor). These mismatches will occur when the specified one-to-one mode is selected and the other compressed scan chain value is at X during the same shift cycle.

DRC Rule R14

Message Text

Compressor Rule R14: X source T gate (G1) is propagatable to N scancells (G2) .

Severity

Warning

Description

This message indicates that the potential for unknown values (Xs) to propagate to scan cells exists. Xs can cause lower test coverage, increased pattern count, and lower diagnostic accuracy.

T is the gate type of the X-source gate, G1 is its gate id, N is the number of scan cells to which it can propagate, and G2 is the gate id of one of the scan cell which can receive the X value.

The check is done assuming Basic-Scan algorithm, so some X sources (such as memories) might be controllable with a different algorithm. Xs might be due to incomplete design data (blackboxes), constraints, floating buses or uninitialized state. Buses that can have contention are not considered X sources because ATPG patterns will attempt to avoid bus contentions (by default).

What Next

No action is required on your part if the coverage is satisfactory, or high diagnostic accuracy is not required.

The `analyze_violation` command can be used to understand the violation. The schematic display will show all gates between the X source and the identified scan cell, with pin data set to be constrain values.

To correct this problem, you might consider defining blackboxed modules, updating constraints, or enhancing the design to reduce Xs.

DRC Rule R15

Message Text

```
Compressor Rule R15: X source T gate (G1) is unblockable to scancells (G2) .
```

Severity

Warning

Description

This message indicates that it is not possible to block propagation of an unknown value (X) from an X source to a scan cell. Xs can cause lower test coverage, increased pattern count and lower diagnostic accuracy.

T is the gate type of the X source gate, G1 is its gate id, and G2 is the gate id of one of the scan cell which can receive the unblockable X value.

This check works best if all signal values required for capture mode operation are fully specified by PI constraints or in the STL procedure file. For example, scan-enable should be constrained to the capture value; if scan-enable is not constrained, this DRC check is free to assume the shift mode value to avoid X capture.

The check is done assuming Basic-Scan algorithm, so some X sources might be controllable with a different algorithm. Xs might be due to incomplete design data

(blackboxes), constraints, floating buses or uninitialized state. Buses that can have contention are not considered X sources because ATPG patterns will attempt to avoid bus contentions (by default).

What Next

No action is required on your part if the coverage is satisfactory, or high diagnostic accuracy is not required.

The `analyze_violation` command can be used to understand the violation. The schematic display will show all gates between the X source and the identified scan cell, with pin data set to be constrain values.

To correct this problem, you might consider defining black boxed modules, updating constraints, or enhancing the design to reduce Xs or allowing blocking of Xs by ATPG.

DRC Rule R16

Message Text

Compressor Rule R16

Unidentified unload compressor pipeline input. Unable to identify unload compressor pipeline input for port Y.

Severity

Fatal Error

Description

During the process of tracing back from the defined output port of the unload compressor, the tool was unable to uniquely identify the corresponding input of the pipeline stage.

What Next

This error is likely due to a wrong definition of the compressor ports (in the STL procedure file) or the number of pipeline stages. Before making any changes to your design or the SPF, you can try using the `-max_trace_gates` option of the `set_workspace_sizes` command to increase the gate-tracing limit and determine if this change removes the R16 error.

DRC Rule R17

Message Text

Compressor Rule R17

Unidentified load compressor pipeline output. Unable to identify load compressor pipeline output for port Y.

Severity

Fatal Error

Description

During the process of tracing forward from the defined input port of the load compressor, the tool was unable to uniquely identify the corresponding output of the pipeline stage.

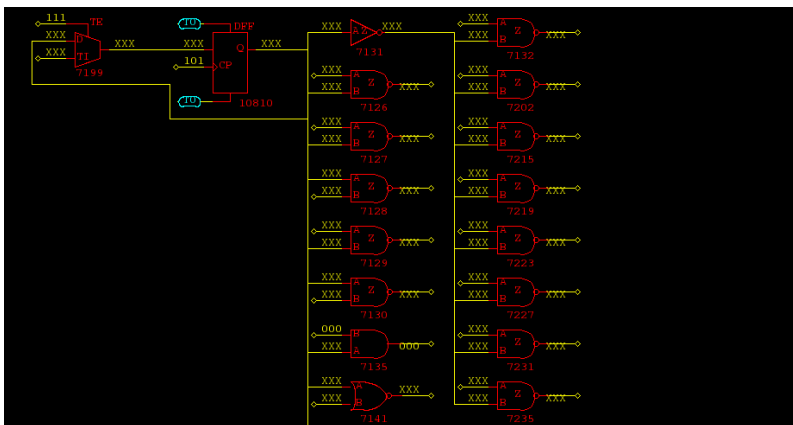
What Next

This error is likely due to a wrong definition of the compressor ports (in the SPF) or the number of pipeline stages. Before making any changes to your design or the SPF, you can try using the `-max_trace_gates` option of the `set_workspace_sizes` command to increase the gate-tracing limit and determine if this change removes the R17 error. You can use the `analyze_violation` command to display the failing path.

Example

```
>> analyze_violation R17-1
Error: Unable to identify load compressor pipeline output for port
test_si6. (R17-1)
The gate_report data is now set to "shift".
```

Open the GSV and gates at which the TestMAX ATPG blocks are displayed.



DRC Rule R18

Message Text

Compressor Rule R18

Load compressor pipeline cell not holding state. Load compressor pipeline cell Y does not hold state during capture.

Severity

Warning

Description

The load pipeline cell must hold state during capture for optimal results.

What Next

If the load pipeline cell captures its own data during capture cycles, the violation can be corrected by constraining scan_enable to its capture value. If the load pipeline cell has a separate clock, the violation can be corrected by constraining the clock off.

The presence of an R18 violation results in a small loss of coverage or increased pattern count. However, there is no danger of generating patterns that could fail simulation.

See Also

- [R27 and M674](#)

DRC Rule R19

Message Text

Compressor Rule R19 : Missing unload compressor pipeline lockup latch. Lockup latch is missing between chain Y and unload pipeline.

Severity

Error

Description

This rule is violated if the last scan cell of a chain shifts on the leading edge, and the first stage of an unload pipeline cell shifts on the trailing edge, and there is no lockup latch in between; as a result, the pipeline cell could capture the new value of the scan cell.

What Next

The simulator will assume the old value is captured, so this rule should be downgraded to warning only if enough data delay or enough clock skew exists from the last scan cell to the first unload pipeline stage to ensure that the old value is captured.

DRC Rule R20

Message Text

Compressor Rule R20 : Missing load compressor pipeline lockup latch.
Lockup latch is missing between load pipeline and chain Y.

Severity

Error

Description

This rule is violated if the first scan cell of a chain shifts on the trailing edge, and the last stage of a load pipeline cell shifts on the leading edge, and there is no lockup latch in between; as a result, the scan cell could capture the new value of the pipeline cell.

What Next

The simulator will assume the old value is captured, so this rule should be downgraded to warning only if enough data delay or enough clock skew exists from the last load pipeline stage to the first scan cell to ensure that the old value is captured.

DRC Rule R21

Message Text

Compressor Rule R21: X-tolerant chain connection to output port *port_name* for unload mode <mode_name1> with load_mode <mode_name2> was not found.

Severity

Warning

Description

This violation appears when, for a given load mode, some combination of inputs does not select an X-tolerant mode or a single-fanout XOR mode. Typically, R21 violations have no implication on QoR, although in some cases a small increase in pattern count might result.

What Next

In general, R21 violations indicate that the compressor could be enhanced to provide more opportunities for observability.

You can enhance the compressor to avoid R21 violations by using all unused unload modes to select either a direct observation mode or a single-fanout XOR mode. TestMAX DFT and the `analyze_compressors` command can be used for this purpose. However,

depending on the configuration (#inputs, #outputs, #chains), it might not be possible to eliminate all R21s.

Annotated schematic data for the first R21 violation (`report_violationR21-1`) can be viewed in the schematic viewer by setting: `set_pindata-drcdata {unload_mode <unload_mode>}`

In this case, the schematic is annotated with `unload_mode` data, that is, shift data plus unload mode pins for the unload mode given as an argument (`<unload_mode>`).

This data is for the load-mode and unload-mode specified in `report_violationR21-1`.

The same data is viewed by selecting: `analyze_violationR21-1`.

DRC Rule R22

Message Text

Compressor Rule R22 : Chain *chain_name* has no x-tolerant observation.

Severity

Error

Description

A violation of this rule indicates that the specified chain cannot be uniquely (non-XORed) observed in any unload mode. Downgrading this rule to pass DRC might result in loss of observability on the chain(s) that failed, and possibly loss of test coverage.

What Next

Regenerate the unload compressor using more input/output pins or fewer internal chains to ensure all chains have X-tolerant observation and thus no R22 message.

When running DRC (before ATPG) on circuits which include blocks that have both default and high X-tolerant architectures, specify the following `set_rules` command:

```
set rules R22 -warn
```

This will downgrade a check done for fully X-tolerant designs built by DFT MAX which were actually built with blocks that include both default X-tolerant architectures and high X-tolerant architecture

DRC Rule R23

Message Text

Compressor Rule R23 : Unload mode enable *port_name* at off_state does not condition full compression on output port of *port_name* (description)

Note that the description can be one of the following:

1. No sensitized path from chain *chain_name*.
2. No sensitized XOR path from chain *chain_name*.
3. Unexpected sensitized path from chain *chain_name* blocked by X value.
4. Blocked by X value.

Severity

Error

Description

DFTMAX compression with enhanced X-tolerance may operate in two modes:

The default mode is Full XOR. In this mode, the DFTMAX circuitry does not use any additional X masking and operates as if classic DFTMAX is being used.

In selective mask mode, DFTMAX compression enables additional X masking. Most shift cycles do not require a selective mode for masking and could achieve higher observability in full compression (full XOR) mode. The unload mode enable should create the full compression condition on a per-shift basis.

Note that downgrading this rule to pass DRC might result in increased pattern count

In the description #1 version of the message, the value placed on the named chain has NO effect on the named output. Only the first problem chain is listed.

In the description #2 version of the message, the value placed on the named chain has an effect on the named output, but the effect is not that of an XOR. Only the first problem chain is listed.

In the description #3 version of the message, the value placed on the named chain has an effect on the named output, but no connection was expected based on the protocol file description of the compressor. Only the first problem chain is listed.

In the description #4 version of the message, the named output is blocked by an X value, so it does not reflect the XOR of the chain values.

What Next

Specify the following:

```
set_pindata -drcdata unload_enable
```

Browse in the GSV and see what happens during shift with `unload_enable` at `off_state`: it should be full-XOR mode. Follow the shift path (X) from the DFF on the output of chains to the output ports; or back from the output ports to the DFF.

Note that the schematic is annotated with a value per gate derived from the shift procedure, plus the `unload_enable` primary inputs at their `off_state`. The output gates of internal scan chains are placed at X, so that you can trace the sensitized path in the GSV -- except in the case of "Blocked by X value." In this case, the output gates of internal scan chains are placed at random binary values -- thus enabling you to use the GSV to trace X-blockage.

For more information, see the `-set_unload_mode_ports_to_x` option of the `set_drc` command.

DRC Rule R24

Message Text

```
Compressor Rule R24 : Unable to infer output of chain C for unload  
compressor
```

Severity

Fatal Error

Description

TestMAX ATPG DRC initially tries to infer the connection between scan chains and the unload compressor. It traces from the scanout ports back through the unload compressor until it reaches a scan flip-flop. Since each scanout port is fed by multiple scan chains, tracing back from a scanout port creates a collection of scan chains that drive that particular scanout port. From this information, DRC infers the scanout ports driven by each scan chain. The `CompressorStructures` block in the STL procedure file used for DRC contains similar information. When TestMAX ATPG infers the scanout ports driven by a scan chain, the DRC process does not match the same ports as specified in the protocol file, and an R24 message is printed.

An R24 error can also be reported in the following cases while running DFTMAX Ultra compression:

- The deserializer chain was incorrectly traced.
- A scan chain tracing failure occurred because of incomplete shift DRC tracing.

What Next

1. Specify the following:

```
set_drc -compressor_debug_data
```

2. Execute the `run_drc` command to produce verbose output for debugging.
3. Add the missing element to the graphical schematic viewer (GSV).
4. Set the pindata to shift.
5. Analyze the violation.

If the ScanDataOut pin for a ScanChain is specified in a design's ScanStructures section of the protocol, make sure that the Place-and-Route tool does not make any changes from that pin. Failure to do this can result in pattern mismatches in simulation and on the ATE.

```
Begin compressor chain I/O checking: #undefined_io=6576 ...
```

```
Warning: #candidate chains (3287) and #connecting chains (3288)  
for unload compressor SUB_AudioVideo_U_compressor are different. (M620)
```

```
Error: Unable to infer output of chain 1877 for unload compressor.  
(R24-1)
```

```
Begin R24 debug data: Chain=1877, connection_ports = test_so54 test_so91
```

```
-----  
output_id  instance_name of cell on chain output  
connections  
-----  
-----
```

```
-----  
364962  
c8compol_7100_top_0/i_compo_gen/i2_vp_mdl/i_vp_reg_mdl/vp_reg_coeff1r_re  
gx17x test_so1 test_so23
```

There are many lines similar to the previous example. In the previous example, gate id 364962 has been identified as the output of an internal chain that connects, through the unload compressor, to POs test_so1 test_so23; this chain is then matched to a chain defined in the STL procedure file that connects to the same outputs.

To see the connections, run the `set_pindata shift` command and trace through the GSV.

DRC Rule R25

Message Text

Compressor Rule R25 : Chain A has no x-tolerant observation for load mode B

Severity

Warning

Description

This violation appears when the number of unload mode internal signals (= $\log(\text{\#chains}/\text{\#outputs})$) is larger than the number of load decompressor scan inputs (= total inputs - unload_enable_pin - load_mode_pins). The presence of R25 violations might result in increased pattern count.

This violation is similar to [R22](#), except that it refers to a specific load mode. The chain considered might have X-tolerant observation in other load modes. If it has no X-tolerant observation in any load mode, then [R22](#) is also violated. Violation of R25 might result in test coverage loss because it increases the probability that a fault cannot be observed in the load mode in which it was sensitized.

DRC Rule R26

Message Text

Compressor Rule R26 : PLL clock chain A with control cell B has invalid connections for load mode C of compressor D

Severity

Error

Description

This message checks that the cells that control PLL internal clocks do NOT share scanin data with any other scan cells.

The clock control chain is all care bits all the time, so no compression of its data is possible.

What Next

Downgrading an R26 message allows the generation of correct patterns; the simulator rejects bad patterns, if any. However, loss of coverage or increased pattern count might result.

DRC Rule R27

Message Text

Compressor Rule R27 : Load compressor pipeline cell *name* neither holds or shifts during capture.

Severity

Error

Description

The load pipeline cell must hold state during capture for optimal results (see the description of the [R18](#) violation). Alternately, the load pipeline cell can shift during capture, or it can either shift or hold during capture, depending on the value of some input (e.g., `scan_enable`). The `report_compressors-pipeline` command identifies the three types of load pipeline cells as "hold", "shift," or "both," respectively.

Load pipeline cells in the "hold" category pass both [R18](#) and R27.

Load pipeline cells in the "shift" or "both" categories fail [R18](#), but pass R27; this can result in a small loss of coverage and/or increased pattern count (see [R18](#)).

An R27 violation indicates that a load pipeline cell is in neither "hold", "shift," or "both".

Note that all chains must go through the compressors. Even if a chain (such as the clock chain) is not compressed, it is still recognized by TestMAX ATPG as going through the compressors, albeit with a 1:1 "compression." All compressor inputs must have the same pipeline depth and all compressor outputs must have the same pipeline depth.

What Next

If possible, use the `add_pi_constraints` command to add constraints to pass without an R27 violation. Otherwise, you must modify the design, including balancing the pipeline depth on all chains.

To verify if the compressors have the same pipeline depths for inputs and output, temporarily downgrade the R27 message to a warning. Then run the `report_compressors-pipeline -verbose` command so you can view additional information on the pipeline registers that might help identify the cause of the violation.

Note: Downgrading an R27 violation from an error to a warning to pass DRC can result in patterns that will fail simulation and on the tester.

DRC Rule R28

Message Text

Compressor Rule R28: Clock chain connected to multiple ports

Severity

Error

Description

This message indicates that the clock chain (that is, a scan chain that contains one or more scan cells used to control internal clocking) must connect to the same scan input and must be independent of the load decompressor mode. Note that the clock cells are always care bits, so they must be set to 0 or 1 in every pattern. If this requirement is not met, there might be a loss of test coverage and pattern inflation because of the additional load decompressor dependencies between the clock cells and other scan cells.

Because of the potential for test coverage degradation and pattern inflation, you should not downgrade this rule to pass DRC. Instead, you should change the DFT implementation. In this case, DFTMAX constructs and connects a compressor to prevent this rule violation -- but only if it is specified with a clock chain.

A common error in the OCC controller recognition flow is to omit the `set_scan_path -class occ` command to force the DFT architect to recognize the clock chain. It is important to specify this command at the hierarchical level in which the OCC controller is first declared. In a hierarchical flow, this problem might occur only at a higher level.

In all cases, the `preview_dft` command report should show that the clock chain is a segment with the "occ" property. For example:

```
Scan chain '1' contains 4 cells
Active in modes: ScanCompression_mode :
CORE/clock_chain (s) (o) (m) (CORE/duto/clk, 45.0, rising)
```

The string "(o)" refers to a clock chain. All clock chain segments should have this tag. If it is missing, then DFTMAX compression might stitch the clock chain to cause a rule violation.

DRC Rule R29

Message Text

Compressor Rule R29 : Clock chain port used for X-tolerance control

Severity

Error

Description

This message indicates that the chain (that is, a scan chain that contains one or more scan cells used to control internal clocking) must connect to a scan input which is not used to control X-tolerance. Note that the clock cells are always care bits, so they must be set to 0 or 1 in every pattern. If this requirement is not met, there might be a loss of test coverage and pattern inflation because of the additional load decompressor dependencies between clock cells and X-tolerance controls.

What Next

Because of the potential for test coverage degradation and pattern inflation, you should not downgrade this rule to pass DRC. Instead, you should change the DFT implementation. In this case, DFTMAX constructs and connects a compressor to prevent this rule violation -- but only if it is specified with a clock chain.

A common error in the OCC controller recognition flow is to omit the `set_scan_path -class occ` command to force the DFT architect to recognize the clock chain. It is important to specify this command at the hierarchical level in which the OCC controller is first declared; in a hierarchical flow, the problem might only occur at a higher level.

In all cases, the `preview_dft` command report should show that the clock chain is a segment with the "occ" property. For example:

```
Scan chain '1' contains 4 cells
Active in modes: ScanCompression_mode :
CORE/clock_chain (s) (o) (m) (CORE/duto/clk, 45.0, rising)
```

The string "(o)" refers to a clock chain. All clock chain segments should have this tag. If it is missing, then DFTMAX compression might stitch the clock chain to cause a rule violation.

DRC Rule R30

Message Text

```
Compressor Rule R30 : Nonclock chains <name1> and <name2> are not
independent when clock chain inputs are ignored.
```

Severity

Warning

Description

The load decompressor must be designed to minimize dependencies between scan chains. At a minimum, any two non-clock chains must be independent in at least one load mode. Because clock cells are always "care bits" (they must be set to 0 or 1 in every pattern), non-clock chains should be independent of each other and of clock chain scan

inputs. Failure to satisfy this requirement will result in additional load decompressor dependencies between non-clock chains. The clock chain bits is set properly, so this rule violation does not indicate that bad patterns are generated.

What Next

If this rule fails, during ATPG the results might be a loss of test coverage and pattern inflation due to the additional load decompressor dependencies.

DRC Rule R31

Message Text

Compressor Rule R31

Incorrect defined multi chain X-tolerant connection to output N of unload compressor X using mode L/U.

Incorrect defined X-tolerant connection for chain C to output N of unload compressor X using mode L/U.

Severity

Error

Description

This rule checks the correctness of the defined connections (if they are defined) between chain outputs and the outputs of an unload compressor for a given X-tolerant mode, as specified by a load and unload mode value.

There are two variations of this rule: the first is for modes with a single chain connection to a compressor output; the second is for modes with multiple chain connections to a chain output.

Note the following:

C is the name of the chain

N is the ID number of the output of the unload compressor

X is the name of the unload compressor

L is the load mode value

U is the unload mode value

This rule is checked by simulating the conditions set up by the shift procedure augmented by the conditioning of a given load and unload mode (including the X-tolerant enable pin set to its active value) and with the chain outputs set to random values. The simulated

value of an output of an unload compressor is compared with its expected value as determined by its defined connections. If this rule fails, the connection definition might be incorrect or the conditioning set up by the shift procedure might be incomplete. There is no automated violation analysis available for this violation to assist in debug.

This rule can be downgraded to a warning to pass DRC, however there is a high risk that the patterns might not work at the tester.

What Next

To debug a selected violation, specify the following command before `run_drc`:

```
set_drc -store_unload_mode_data L
```

In this case, data for up to 32 unload modes is stored for the selected load mode, L. Next, specify the following:

```
set_pindata -drcdata {unload_mode U}
```

At this point, you need to open the GSV and trace back the sensitized (X) path from the output N of the unload compressor to determine why it does not connect to the expected scan chain output(s).

For more information, see the `-set_unload_mode_ports_to_x` option of the `set_drc` command.

DRC Rule R32

Message Text

Compressor Rule R32

```
Unload compressor <unl_comp_name> had invalid capturing edge on cell  
<unl_gate_id> from load compressor cell <load_gate_id>.
```

Severity

Error

Description

This rule checks the edges of load and unload compressor pipelines for X-tolerant compressors. A violation occurs if a load compressor cell, indicated by `<load_gate_id>`, is a leading-edge (LE) cell and an unload compressor cell, indicated by `<unl_gate_id>`, is a trailing-edge (TE) cell. The LE-TE data transfer could occur in a single cycle, thus patterns might fail.

Downgrading this error to pass DRC could result in patterns that fail simulation and on the tester

What Next

The edge of the pipeline cells must be changed, or lockup latches must be inserted to eliminate all LE-TE paths.

DRC Rule R33

Message Text

Compressor Rule R33

Unable to find output *output_id* of unload compressor *name*.

Severity

Fatal Error

Description

When unload serializers are used, random pattern simulation of the events in the load and shift procedures is performed to identify unload compressor output nodes. This violation occurs when there is a failure to identify a DFF (D flip-flop) gate containing data input with the expected value at the appropriate cycle.

What Next

This violation is a fatal error, and must be corrected to pass DRC. There is no automated violation analysis for this rule. You can use the `report_serializers -all` command to display the serializer data and identify unload serializer outputs. Normally, the problem is due to an incorrect netlist or test procedure file.

This error can occur if the top-level serializer protocol is generated by the `spfgn` utility or manually created, and the `ScanEnable` signal is not constrained. In this case, you must constrain the `ScanEnable` signal to 0 using the `add_pi_constraints 0` command.

If both R33 and [R35](#) errors are reported, you should investigate all [R35](#) errors first.

If you encounter only R33 errors, and no [R35](#) errors, you should perform the following steps:

Specify the `set_drc -store_initial_shifts` command.

Specify the `set_pindata -shift` command

Execute the `run_drc` command.

Perform the following examinations:

Use the GSV to make sure the clock behavior is correct by checking clock pins of unload serializer registers.

Make sure no unexpected blockage occurs on unload serializer registers that traverse data paths to scan output ports.

To understand the pindata values, refer to the “Debugging TestMAX ATPG Serializer DRC Errors” section in the *Adaptive Scan with Serializer User Guide*.

If there are no problems on the signals, there might be problems on the internal scan chains or inside the compressor logic.

The following steps enable TestMAX ATPG to recognize user-provided unload serializer registers without undergoing the identification process during DRC. This allows you to move on to subsequent DRC processes.

Specify unload serializer register names in the STIL procedure file. For more information, refer to the “DFTMAX Compression with Serializer” section in the *TestMAX DFT, DFTMAX, and DFTMAX Ultra User Guide*.

Reset the `set_drc -nostore_initial_shifts` command.

Specify the `set_drc -nocheck_user_serializer_bits` command.

Execute the `run_drc` command.

By applying these steps, you can move on to subsequent DRC processes in which S and R violations (other than R33 to R38) might appear. You can usually debug these violations using the GSV. This enables you to skip serializer-specific DRC and move on to scan DRC and DFTMAX DRC. You will most likely encounter S or R violations (other than R33 to R38), and you should debug them.

DRC Rule R34

Message Text

Compressor Rule R34

Multiple candidates `gate_id1`, `gate_id2` for output `output_id` of unload compressor `name`.

Severity

Error

Description

When unload serializers are used, the unload compressor output nodes are identified via random pattern simulation of the events in the load and shift procedures. This violation occurs when more than one D flip-flop (DFF) gate is identified containing a data input with the expected value at the appropriate cycle.

What Next

You can downgrade the severity of this violation and continue the DRC process. If multiple candidates are identified for an unload compressor output, the first candidate identified is selected. Additional DRC failures will occur if you downgrade the severity, and the first candidate is incorrect.

If you encounter an R34 error, you can edit the STIL procedure file to specify the correct unload serializer registers, as shown in the following example:

```
UserKeywords SerializerStructures CompressorStructures;
SerializerStructures {
    InternalShiftStart 6;
    UnloadDataStart 7;
    ExternalCyclesPerShift 5;
    LoadSerializer "abc_top_U_deserializer_abc" {
        Length 5;
        ActiveScanChains load_group;
    }
    UnloadSerializer "abc_top_U_serializer_abc" {
        Length 5;
        ActiveScanChains unload_group;
        ParallelInputs {
            0 "abc_top_U_compressor_abc/serial_reg_0_" no
            1 "abc_top_U_compressor_abc/serial_reg_1_" no
            2 "abc_top_U_compressor_abc/serial_reg_2_" no
            3 "abc_top_U_compressor_abc/serial_reg_3_" no
            4 "abc_top_U_compressor_abc/serial_reg_4_" no
        }
    }
}
```

For more information, refer to the “DFTMAX Compression with Serializer” section in the *TestMAX DFT, DFTMAX, and DFTMAX Ultra User Guide*.

DRC Rule R35

Message Text

Compressor Rule R35

Unable to find input *input_id* of load compressor *name*.

Unable to find xtol-enable input of unload compressor *name*.

Unable to find mode-port input with serializer connection
port_nameposition.

Severity

Fatal Error

Description

When load serializers are used, random pattern simulation of the events in the load and shift procedures is performed to identify the load compressor input nodes, xtol-enable input nodes, and mode-port input nodes. Failure to identify any D flip-flop (DFF) gates with an output containing the expected value at the appropriate cycle will result in this violation.

What Next

This violation is a fatal error, and you must fix it to pass DRC. There is no automated violation analysis for this rule. You can use the `report_serializers -all` command to display serializer data and identify load serializer inputs.

If you encounter any R35 errors, you should perform the following steps:

Specify the `set_drc -store_initial_shifts` command.

Specify the `set_pindata -shift` command

Execute the `run_drc` command.

Perform the following examinations:

Use the GSV to make sure the clock behavior is correct by checking clock pins of the load serializer registers and update stage registers (if they are used).

Make sure no unexpected blockage occurs on load serializer registers that traverse data paths to scan input ports.

To understand the pindata values, refer to the “DFTMAX Compression with Serializer” section in the *TestMAX DFT, DFTMAX, and DFTMAX Ultra User Guide*.

After you finish the debugging process, make sure you reset the `set_drc -nostore_initial_shifts` command.

DRC Rule R36

Message Text

Compressor Rule R36

Multiple candidates *gate_id*, *gate_id* for input *input_id* of load compressor *name*.

Multiple candidates *gate_id*, *gate_id* for xtol-enable input of unload compressor *name*.

Multiple candidates *gate_id*, *gate_id* for mode-port input with serializer connection *port_nameposition*.

Severity

Error

Description

When load serializers are used, random pattern simulation of the events in the load and shift procedures is performed to identify load compressor input nodes, xtol-enable input nodes, and mode-port input nodes. This violation occurs when more than one D flip-flop (DFF) gate is identified with an output that has an expected value at the appropriate cycle.

What Next

You can reduce the severity of this violation and continue the DRC process. If multiple candidates are identified for a load-compressor, xtol-enable, or mode-port input, the first candidate identified is selected. If you reduce the severity and this candidate is not the correct choice, then other DRC failures will occur.

If you encounter an R36 error, you can edit the STIL procedure file to specify the correct load serializer registers, as shown in the following example:

```
UserKeywords SerializerStructures CompressorStructures;
SerializerStructures {
    InternalShiftStart 6;
    UnloadDataStart 7;
    ExternalCyclesPerShift 5;
    LoadSerializer "abc_top_U_deserializer_abc" {
        Length 5;
        ActiveScanChains load_group;
        ParallelOutputs {
            0 "abc_top_U_compressor_abc/serial_reg_0_" no
            1 "abc_top_U_compressor_abc/serial_reg_1_" no
            2 "abc_top_U_compressor_abc/serial_reg_2_" no
            3 "abc_top_U_compressor_abc/serial_reg_3_" no
            4 "abc_top_U_compressor_abc/serial_reg_4_" no
        }
    }
    UnloadSerializer "abc_top_U_serializer_abc" {
        Length 5;
        ActiveScanChains unload_group;
    }
}
```

For more information, refer to the “DFTMAX Compression with Serializer” section in the *TestMAX DFT, DFTMAX, and DFTMAX Ultra User Guide*.

DRC Rule R37

Message Text

Compressor Rule R37: Scan cell `gate_id` was clocked during serializer nonshifting cycle.

Severity

Error

Description

When serializers are used, random pattern simulation of the events in the load and shift procedures is performed to verify that D flip-flops (DFFs) are not clocked in the same nonshift cycle. The pattern simulation is done at the same time the load compressor inputs and unload compressor outputs are identified. An R37 error is issued if the DFFs are clocked during a nonshift cycle. Later in the DRC process, after the scan cells have been identified, the scan cell master cells are checked to make sure they are not clocked during nonshift cycles.

When using the `set_drc -store_initial_shifts` command, the first InternalShiftStart shift cycles are stored and can be inspected when the pindata is set to shift. However, this applies strictly to situations when you are debugging R violations that occur before tracing the internal scan chains (since storing all cycles in the shift will not allow the tracing of internal scan chains). An M464 warning message is issued in this case.

What Next

You can downgrade the severity of this violation, and continue the DRC process. If you reduce the severity, the patterns generated during ATPG will not work at the tester. Normally, the source of the problem is an incorrect netlist or test procedure file.

You can debug an R37 error by performing the following steps:

Specify the `set_drc -store_initial_shifts` command.

Specify the `set_pindata -shift` command

Execute the `run_drc` command.

Use the GSV to display the gate ID reported with the R37 violation, and examine the pindata on the clock line.

Trace back the clock signal to determine why it is clocked in a nonshift cycle, and identify any differences from other scan cells without the R37 violations.

To understand the pindata values, refer to the “DFTMAX Compression with Serializer” section in the *TestMAX DFT, DFTMAX, and DFTMAX Ultra User Guide*.

After you finish the debugging process, make sure you reset the `set_drc -nostore_initial_shifts` command.

DRC Rule R38

Message Text

Compressor Rule R38

Scan cell `gate_id` was not clocked during serializer shifting cycle.

Severity

Error

Description

When serializers are used, random pattern simulation of the events in the load and shift procedures is performed to verify that the D flip-flops (DFFs) are clocked in a shift cycle. This pattern simulation is performed at the same time the load compressor inputs and unload compressor outputs are identified. An R38 violation is issued if the DFFs are not clocked in a shift cycle. Later in the DRC process, after the scan cells have been identified, the scan cell master cells are checked to make sure they are clocked during shift cycles.

When using the `set_drc -store_initial_shifts` command, the first InternalShiftStart shift cycles are stored and can be inspected when the pindata is set to shift. However, this applies strictly to situations when you are debugging R violations that occur before tracing the internal scan chains (since storing all cycles in the shift will not allow the tracing of internal scan chains). An M464 warning message is issued in this case.

What Next

You can downgrade the severity of this violation, and continue the DRC process. If you reduce the severity, the patterns generated during ATPG will not work at the tester. Normally, the problem is due to an incorrect netlist or test procedure file.

You can debug an R38 error by performing the following steps:

Specify the `set_drc -store_initial_shifts` command.

Specify the `set_pindata -shift` command

Execute the `run_drc` command.

Use the GSV to display the gate ID reported with the R38 violation, and examine the pindata on the clock line.

Trace back the clock signal to determine why it is not clocked in a shift cycle, and identify any differences from other scan cells without R38 violations.

To understand the pindata values, refer to the “DFTMAX Compression with Serializer” section in the *TestMAX DFT, DFTMAX, and DFTMAX Ultra User Guide*.

After you finish the debugging process, make sure you reset the `set_drc -nostore_initial_shifts` command.

DRC Rule R39

Message Text

Compressor Rule R39

Unload compressor *decompressor* failed propagation checking for chain *n* in unload mode *M*.

Severity

Error

Description

During tracing from the ScanOut port, the expected mask bits were not found for the identified unload mode.

What Next

Specify the following:

Execute the `run_drc` command to produce verbose output for debugging.

DRC Rule R40

Message Text

Compressor Rule R40

Could not find flop at index (*N*) of Load Serializer (*M*)

Severity

Error

Description

Deserializer chain tracing was incomplete. The possible causes are as follows:

1. Clock not pulsing or netlist/STL procedure file was manually edited.
2. When serializer is implemented in the design, staggered scan clocks have timing that either occur later than trailing edge of the deserializer register clocks or earlier than leading edge of the serializer register clocks.

What Next

1. Specify the following:

```
set_drc -compressor_debug_data
```

2. Execute the `run_drc` command to produce verbose output for debugging.
3. Add the missing gate to the graphical schematic viewer (GSV).
4. Set the pindata to shift.
5. Analyze the violation.
6. If R40 is reported in a design where serializer is implemented, then you must ensure that timing of the scan cells capturing data from deserializer and update registers must not occur later than the trailing edge of the deserializer clocks. Similarly, the internal scan cells that launch the data through compressor to serializer registers must not occur earlier than leading edge of the serializer register clock.

Refer to "Staggered Scan Clocks" in the TestMAX DFT User Guide in for more details.

DRC Rule R41

Message Text

```
Compressor Rule R41
```

```
Error: Unload compressor (M): Single fanout connection error for chain  
(C). (R41-3)
```

Severity

Error

Description

R41 indicates a wrong single fanout connection for a chain (C). Chains are assigned a single fanout output based on the order of appearance of the chains in the core group

definition. A chain that is at the first position in the core group should have zero as the chain single fanout output.

What Next

R41 might be caused due to the wrong netlist or wrong SPF. First, check if the SPF is wrong. If the order of chains in the core group is different from the ordering of chains in unload 'Mode 0', then it is an SPF issue. If the ordering is correct, then the error could be due to issues in the netlist. If the ordering is different, you can modify the chain ordering in the core group definition.

DRC Rule R42

Message Text

Compressor Rule R42

Load compressor pipeline cell ID disturbed at time T of P procedure.

Severity

Error

Description

The load pipeline cells must be in a hold state during certain procedures, such as load_unload preamble.

A violation of this rule indicates that the load compressor pipeline cell ID is not in a hold state (it was disturbed) at time T of the procedure P .

Rules [R18](#) and [R27](#) check load pipeline behavior during capture. However, unlike these rules, R42 checks other procedures, such as the load_unload preamble.

You can downgrade the R42 error to a warning to pass DRC. However, this can result in patterns that will fail simulation and on the tester.

What Next

You can analyze this violation in the graphical schematic viewer (GSV) by displaying the pipeline cell ID with the pin data set to the offending procedure P . You should correct the protocol or the netlist to pass this rule.

DRC Rule R43

Message Text

Compressor Rule R43

Load compressor pipeline cell ID disturbed during serializer initial shift cycles at time T of P procedure.

Severity

Error

Description

When serializers are used, the first internal shift clock pulse occurs after a certain number of serializer shift clock pulses. For example, for the normal serializer duty cycle, the first internal shift clock pulse occurs at the InternalShiftStart cycle.

The load pipeline cells must be in a hold state during initial shift cycles of the serializer Shift procedure.

A violation of this rule indicates that the load compressor pipeline cell ID is not in a hold state (it was disturbed) at time T of the Shift procedure.

Rules [R18](#) and [R27](#) check the load pipeline behavior during capture. However, unlike these rules, [R42](#) and [R43](#) check the load pipeline behavior during scan shift. [R42](#) checks the load pipeline behavior during the load_unload preamble, while [R43](#) checks the initial shift cycles for the Shift procedure.

You can downgrade the [R43](#) error to a warning to pass DRC. However, this can result in patterns that will fail simulation and on the tester.

What Next

You can debug an [R43](#) violation by performing the following steps:

Specify the `set_drc -store_initial_shifts` command.

Specify the `set_pindata -shift` command

Execute the `run_drc` command.

Use the GSV to display the pipeline cell ID reported with the [R43](#) violation, and examine the pindata on the clock line.

Determine why the load pipeline is clocked during the initial shift cycles before the first internal shift.

Correct the netlist to pass DRC.

To understand the pindata values, refer to the “DFTMAX Compression with Serializer” section in the *TestMAX DFT, DFTMAX, and DFTMAX Ultra User Guide*.

After you finish the debugging process, make sure you reset the `set_drc -nostore_initial_shifts` command.

DRC Rule R44

Message Text

Compressor Rule R44

Pipeline cell ID between load serializer and load compressor does not hold state during capture.

Severity

Error

Description

When serializers are used, pipeline cells between load serializer and load compressor must be in a hold state during capture. Failure to satisfy this rule results in patterns that fail simulation.

What Next

You can add a self loop around the pipeline cell by using a multiplexer with the select line connected to the scan enable so the data is capture during the capture cycles.

Alternatively, if the load pipeline cell has a separate clock, the violation can be corrected by constraining the clock off.

See Also

- [R27](#)
- [R42](#)
- [R43](#)

DRC Rule R45

Message Text

Compressor Rule R45

Load compressor pipeline cell *cell_ID* must not be constrained on *PI_NAME*

Severity

Error

Description

The load pipeline cell (`cell_ID`) does not hold its state during capture, although it might be a shift pipeline. In this context, the primary input (`PI_NAME`) driving this pipeline must not be constrained. When the primary input is constrained, TestMAX ATPG generates patterns that fail the simulation and tester operation. This occurs because dependencies exist between the pattern requirements and the ability to control the state on the primary input.

What Next

Remove the primary input constraint from the specified input and make sure this change passes DRC. Note that primary input constraints might mask subsequent [R27](#) issues on these pipeline cells. These issues must be resolved separately.

See Also

- [R18](#)
- [R27](#)

DRC Rule R46

Message Text

Compressor Rule R46

Serializer output compressor pipeline discrepancy, should be *number* more elements deep.

Severity

Warning

Description

The number of shifts performed in the `load_unload` procedure does not match the data length generated for the serializer context. This indicates the serializer output pipelines were incorrectly specified in the `SerializerOutputPipelineStages` statement in the SPF. Patterns are generated in this context, however the input data for the `load_unload` procedure will exceed the data limit for the `load_unload` constructs, and may affect test translation flows.

What Next

Remove the primary input constraint from the specified input and make sure the input passes DRC. The primary input constraints might mask subsequent [R27](#) issues on these pipeline cells, so you must resolve these issues separately.

See Also

- [R18](#)
- [R27](#)

DRC Rule R47

Message Text

Compressor Rule: R47:

Nonscancell *ID* used to control PLL internal clocking.

Severity

Error

Description

This message is issued when the *ID* nonscan cell controls PLL internal clocking. However, nonscan cells cannot be controlled through scan load. This means the ability to control clocks might be severely degraded, which causes poor test coverage.

This message is also issued for an incorrectly configured compressed (DFTMAX) design if pipelining is used and the clock chain does not have pipelines.

All chains must go through the compressors. Even if a chain is not compressed, such as the clock chain, it is still recognized by TestMAX ATPG as such (it is "compressed" at a 1:1 ratio). Also, all compressor inputs must share the same pipeline depth, and all compressor outputs must share the same pipeline depth.

For example, in a design with two stages of output pipeline on the unload compressor, the two cells on the output side of the clock chain are considered pipeline cells (see the `report_compressors-pipeline` command). These cells are not part of the chain because they are nonscan cells. This situation causes the R47 violation. The expected usage is to add two additional DFFs on the output of the clock chain.

What Next

You must modify the design so that all chains, including non-compressed chains, have the same input and the same output pipeline depth.

DRC Rule R48

Message Text

Compressor Rule R48

Load pipeline path from *name_1* (*cell_ID*) has [inverted | non-inverted] polarity and is connected to cell *name_2* (*cell_ID*) with [non-inverted | inverted] polarity.

Severity

Error

Description

The load pipeline path from the *name_1* cell has multiple outputs (a forked output path in the pipeline) with inconsistent inversions on the different paths. Some of these paths might not be primary pipeline paths. However, the cells on these paths have relationships with each other; for example, as master-slave combinations. Therefore, dependencies exist between the cells. TestMAX ATPG does not support complex pipeline paths with path-specific inversions.

What Next

Eliminate the inversion differences in the pipeline fork so the paths generate properly. You may reduce this violation to a warning. However, the generated patterns contain incorrect states for one of the paths in this context and might fail during the `run_simulation` command or application at test. For information on analyzing a violation in the graphical schematic viewer (GSV), see "Analyzing DRC Violations in the GSV."

DRC Rule R49

Message Text

Compressor Rule R49

Load compressor *decompressor_name* is driven by scan-input *scan_input* (*gate_id*) with only dynamic bits. Re-run DFT with a less number of inputs at top-level.

Severity

Error

Description

This error occurs when the number of inputs required to drive a load compressor exceeds the available number of dedicated top-level inputs. When scan inputs are dedicated for a DFTMAX Ultra load compressor (also called a decompressor), each input consists of data and control flip-flops. Data bit values are stored inside the scan chains as part of the scan load. These bits are also referred to as dynamic bits because data streams through the bits to the scan chain when a pattern is loaded through the load compressor. Control bits are stored in a cache at the end of the pattern and are static since they do not change

for a pattern. If a decompressor requires more inputs than are available, DFTMAX Ultra compression separates the data and control bits, and TestMAX ATPG issues an R49 error.

What Next

Rerun DFTMAX Ultra compression using a decompressor that uses no more than the available number of top-level inputs.

DRC Rule R50

Message Text

Compressor Rule R50

Ignore: Incorrect SPC chain structure at *SPC_serial_reg_instance* (*gate_id*).

Severity

Ignore

Description

TestMAX ATPG expects a value of 0 in the shift power controller (SPC) register in order to turn off a group of chains. This rule violation indicates inversions leading up to the SPC control register. As a result, the register receives a value of 1 instead of 0.

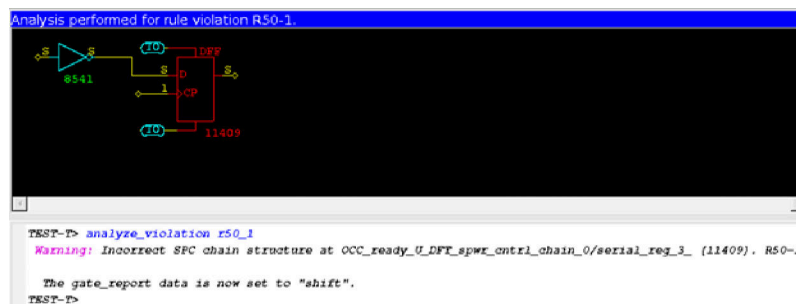
What Next

This is an informational message only. If the violation is not corrected, you can run ATPG and generate patterns as normal.

Example

```
TEST-T> report_violation r50_1
Ignore: Incorrect SPC chain structure at
OCC_ready_U_DFT_spwr_cntrl_chain_0/serial_reg_4_ (11408). R50-1
```

You can use the `analyze_violation` command to highlight the SPC register that violated the R50 rule, as shown in the following figure.



DRC Rule R51

Message Text

Compressor Rule R51

Error: Incorrect SPC update latch asynchronous control in
SPC_update_latch_instance (gate_id).

Severity

Error

Description

This rule checks the clock/set/reset inputs of the shift power controller (SPC) update latches. A violation indicates any of the following:

The set input is not off

The reset input is not off

The clock/enable input is not OFF during shift procedures

The clock/enable input is not ON during capture

What Next

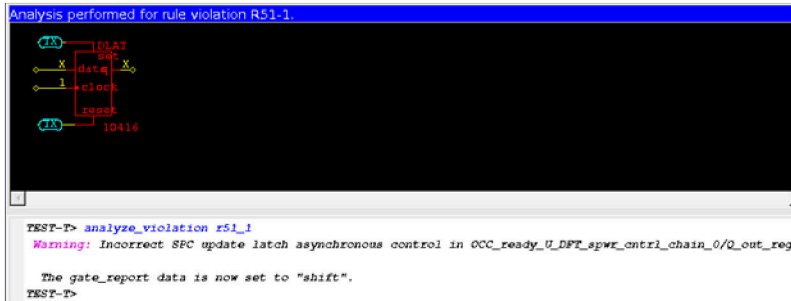
This rule can be downgraded to expedite debugging. If the violation is not corrected, you can run ATPG generate patterns as normal. However, there is a high probability that the resulting patterns will fail the Verilog resimulation.

Shift pindata is selected by the `analyze_violation` command so that violations can be debugged during the scan shifting operation (when the latch must hold state). However, it is possible that the actual violation is in capture mode (when the latch must accept new data). If the shift pindata looks correct, use the Setup dialog box to change the pindata to "Constrain Value". The latch enable must be constrained to the state that makes the latch transparent, and the latch data input must be shown as X, which means that ATPG can set it to any value.

Example

```
TEST-T> report_violation r51_1
Error: Incorrect SPC update latch asynchronous control in
OCC_ready_U_DFT_spwr_cntrl_chain_0/Q_out_reg_2_ (10416). R51-1
```

Use the `analyze_violation` command to highlight the SPC update latch that is violating the rule, as shown in the following figure.



DRC Rule R52

Message Text

Compressor Rule R52

Ignore: Incorrect SPC load compressor propagation from SPC control register *SPC_serial_reg_instance* (*gate_id*) to *scan_cell_instance* (*gate_id*).

Severity

Ignore

Description

This rule violation indicates that a value of 0 in the shift power controller (SPC) control register cannot propagate to the internal chain. The value might be blocked due to various reasons based on the gate types in the path from the SPC register to the internal chain. This violation implies that the SPC register is unable to turn off the chain.

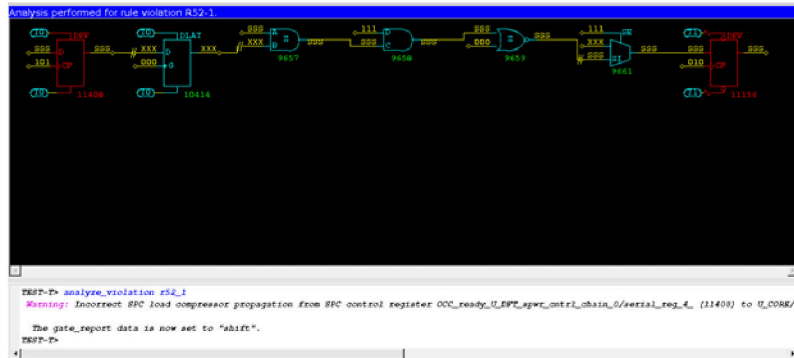
What Next

No action is required. ATPG takes the circuit structure into account when generating and formatting patterns.

Example

```
TEST-T> report_violation r52_1
Ignore: Incorrect SPC load compressor propagation from SPC control
register OCC_ready_U_DFT_spwr_ctrl_chain_0/serial_reg_4_ (11408) to
U_CORE/de_encrypt/kdin_reg_20_ (11156) R52-1
```

You can use the `analyze_violation` command to display the entire path from the SPC register to the first cell of the internal chain, that is violating the rule. The SPC register and the first cell of the internal chain will be highlighted, as shown in the following figure.



DRC Rule R53

Message Text

Compressor Rule R53

Error: SPC control register *SPC_control_reg_instance (gate_id)* not holding state.

Severity

Error

Description

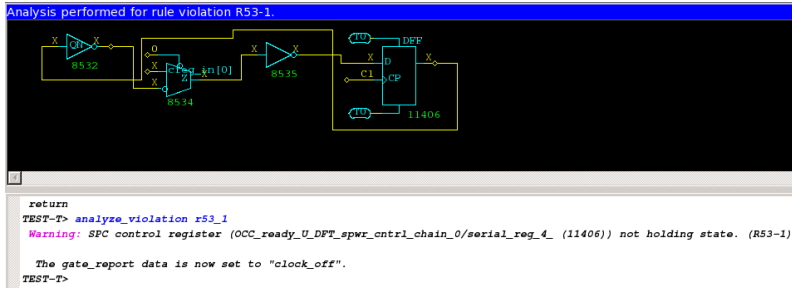
This rule can be downgraded to expedite debugging. If the violation is not corrected, you can run ATPG generate patterns as normal. However, there is a high probability the resulting patterns will fail Verilog resimulation. This rule indicates that the SPC register does not hold state during capture. This could be caused by inversions in the path that connects the SPC register output to its own input.

What Next

You can use the `analyze_violation` command to display and highlight the SPC register that violates the rule.

Example

```
DRC-T> report_violation r53
Error: SPC control register
(OCC_ready_U_DFT_spwr_cntrl_chain_0/serial_reg_4_ (11406)) not holding
state. (R53-1)
```



DRC Rule R54

Message Text

SPC control register is not capturing a constant value.

Severity

Error

Description

This rule violation applies to Serializer DFTMAX architectures when the shift power controller (SPC) cell architecture is modified for Serializer DFTMAX. The modification causes the update stage to become a delay flip-flop (DFF), and the control register captures a constant value (TIE0, TIE1 or scan enable). This modification mitigates pattern inflation caused by having the SPC control chain as an internal chain with its output connected to the unload compressor.

A violation of this rule indicates that the SPC update stage is a DFF and the SPC control register is not capturing a constant value. Downgrading this error to warning can result in pattern inflation.

What Next

Modify your design so that the SPC registers capture a constant value in capture mode.

DRC Rule R55

Message Text

SPC chain is not in one to one mode

Severity

Error

Description

This rule applies to Serializer DFTMAX architectures when the shift power controller (SPC) control chain is an internal chain, with the output of this chain connected to the unload compressor. It is expected that the input of the chain is always in one to one mode. Downgrading this rule to warning can result in patterns that will fail simulation and on the tester.

What Next

You must modify the design so the decompressor or compressor have the same head or tail pipeline stages.

DRC Rule R56

Message Text

Compressor Rule R56: [Decompressor | Compressor] *name* has different [head | tail] pipeline stages *pipeline_length* and *seg_pipeline_length* for its [inputs | outputs].

Severity

Error

Description

Indicates that a particular decompressor or compressor (*name*) contains different head or tail pipeline stage lengths for its inputs or outputs. The pipeline length and segment pipeline length are indicated by the *pipeline_length* and *seg_pipeline_length* parameters.

For example, if a decompressor has two inputs, the head pipeline stages for both inputs should be the same, but different decompressors can have different head pipeline stage lengths. Compressors can also have different tail pipeline stage lengths.

What Next

Modify your design so the decompressor or compressor has the same head or tail pipeline stage lengths.

DRC Rule R57

Message Text

Compressor Rule R57: Compressor Scan <Input | Output> *signal_name*=#
assignment is missing in the load_unload procedure Shift Vector.

Severity

Error

Description

Indicates the parameters specified in the `load_unload` procedure are incomplete and cannot support a call from the patterns. The output pattern data references the *signal_name*. However, the pattern data cannot pass into the test without a matching reference to the indicated signal in the `load_unload` procedure.

What Next

Review the set of signals in the Shift block vector of the `load_unload` procedure. Typically, these signals are referenced by the “_si” or “_so” group. In this case, a group is likely missing the indicated signal. If this signal is not intended to be included in the passed data, you can use the `set_rules` command to downgrade this rule violation to a warning to pass DRC.

DRC Rule R58

Message Text

Compressor Rule R58: ScanEnable name not constrained to LOGICAL LOW.

Severity

Warning

Description

If the ScanEnable `name` is not constrained to "LOGICAL LOW," serial simulation might fail.

What Next

If this warning is unnecessary, you can remove it by adding the `add_pi_constraints` command to the script, as shown in the following example:

```
add_pi_constraints 0 scan_en
```

DRC Rule R59

Message Text

Failed to extract SPC register due to incorrect number of pipeline stages.

Severity

Error

Description

If the number of pipeline stages in the shift power controller (SPC) control chain is different than the number of internal chains, the SPC registers could be misidentified and incorrectly classified as pipeline and SPC control registers during scan chain extraction. This can lead to a crash or incorrect behavior during DRC.

What Next

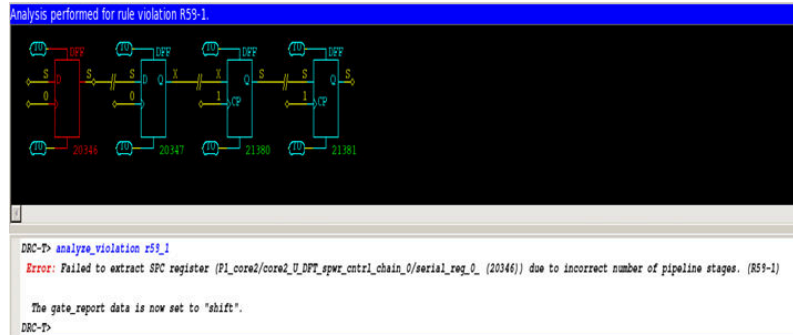
Make sure the number of pipeline stages in the SPC control chain matches the number of internal chains.

GSV Support

Use the `analyze_violation` command to display and highlight the SPC register that violates the R59 rule.

Example

The example below shows a design in which SPC register 20346 is followed by three tail pipeline registers: 20347, 21380 and 21381. Note that the clock input of 21380 is inverted with respect to 20347, which causes the scan chain extraction to consider the pair as master and slave. This creates a mismatch between the number of pipeline stages and the number of internal chains. As a result, SPC register 20346 is classified as a pipeline register to meet the expectation of three tail pipeline registers.



DRC Rule R60

Message Text

Unload compressor output pipeline cell name (`cell_ID`) on `output_signal` does not hold during capture.

Severity

Error

Description

Output pipeline cells must be hold pipelines when the `set_drc -nouse_pipelines` command is applied. Hold pipelines must maintain their current state (they cannot update) during capture operations. You can define hold pipelines by suppressing the clock to the pipeline cell during the capture operation or by recirculating the state of the element during capture.

The R60 rule is only checked then the `-nouse_pipelines` option of the `set_drc` command is applied. Rule R60 is not a restriction for any other flow. While you can downgrade this rule, all violations will fail DRC when the `set_drc -nouse_pipelines` command is set.

What Next

Modify the pipelines to behave as hold pipelines during the capture operations, or remove the `set_drc -nouse_pipelines` command to generate patterns when the pipeline behavior is not required to hold.

DRC Rule R61

Message Text

```
Invalid SPC register(cell_ID) connection to cell_name(cell_ID).
```

Severity

Error

Description

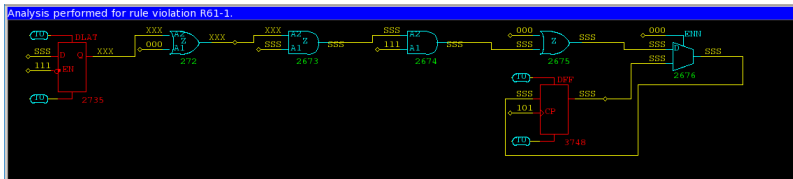
The purpose of Shift Power Controller (SPC) is to manage shift power by loading constant values directly from the SPC update registers into the scan chain inputs. Therefore, in shift mode, SPC update registers should only be connected to the first scan cell of a chain. This check is performed by tracing the SPC update registers forward in shift mode and generating an error any time the trace reaches a primary output or a register that is not the first scan cell of a chain.

What Next

The `analyze_violation` command can be used to understand this violation. The schematic display will show the connection between the SPC registers and cell that is not part of the an internal chain with pin data set to be shift.

This rule can be downgraded to expedite debugging with `set_rule R61 warning`. If the violation is not corrected, you can run ATPG generate patterns as normal. However, there is a high probability the resulting patterns fail Verilog resimulation.

To prevent simulation failures it's recommended to fix the netlist.



DRC Rule R62

Message Text

Compressor Rule R62

Compressor register Cell-name (Cell-id) disturbed during pre-shift of load_unload.

Severity

Error

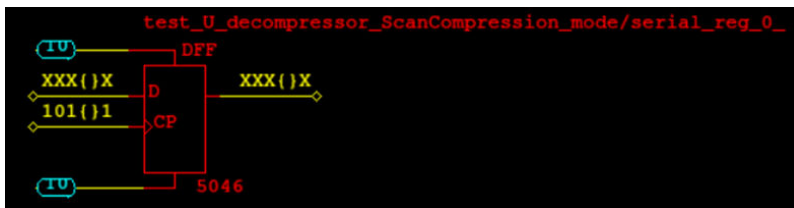
Description

DFTMAX Ultra serial registers cannot be disturbed during the pre-shift or post-shift procedures. Any disturbance of these registers would cause wrong pattern generation that leads to simulation failures.

What Next

The `analyze_violation` command can be used to understand this violation. The schematic displays the cell with the R62 violation, pin-data is set to `load_unload`.

For example, in the following figure, it is evident that `serial_reg_0` has the clock pulsing in pre-shift. Clock pins of serial registers should not be pulsed (disturbed) during pre-shift. Try to resolve the error by preventing the disturb from happening by editing the protocol file or fixing the implementation. If this error is downgraded without fixing it would lead to simulation failures.



DRC Rule R63

Message Text

Compressor Rule R63: Two chains always have the same unload signature that was violated N times.

Severity

Warning

Description

This indicates that the N scan chain pairs will alias for a sub-set of its scan cells. It gives an early indication for potential deficiency in diagnosis.

What Next

Alter the connectivity of the scan chain outputs for each pair of such scan chains. Increase the number of serializer flops or number of serializer segments to resolve this violation.

DRC Rule R64

Message Text

Compressor Rule R64: LE to TE transition from the load serial flop `instance_name` to the first scan flop `scan_instance_name` of chain `chain_name` may result in simulation mismatches.

Compressor Rule R64: LE to TE transition from the last scan flop `scan_instance_name` of chain `chain_name` to the unload serial flop `instance_name` may result in simulation mismatches.

Severity

Warning

Description

This violation can occur in both load and unload side.

When there is a transition from positive edge (LE) load decompressor serial register to a scan chain whose first scan flop is negative edge (TE), it creates a manager-worker condition. This results in missing the first shift cycle and cause simulation mismatches.

Similar issue can occur in unload side when there is a transition from LE last scan flop to TE unload side compressor serial register.

What Next

Check if the re-synchronizer flop was disabled or scan segment bypass erroneously created negative edge head element.

DRC Rule R65

Message Text

Compressor Rule R65: Inversion in between serial reg reg_instance_name1 and reg_instance_name2 can caused simulation fail.

Severity

Error

Description

Inversion between two serial registers is not supported.

What Next

Remove the inverter that causes the error.

Category S - Scan Chain Rules

See Also

- [S1 - scan chain blockage](#)
- [S2 - no scan cells in scan chain](#)
- [S3 - invalid scan chain input](#)
- [S4 - adjacent latches active at same time](#)
- [S5 - gate traced twice during chain trace](#)
- [S6 - invalid force scan input time](#)
- [S7 - invalid chain input propagation](#)
- [S8 - invalid measure scan output time](#)
- [S9 - missing master_observe procedure](#)
- [S10 - unused master_observe procedure](#)
- [S11 - unobservable master](#)

- S12 - unused shadow_observe procedure
- S13 - mismatch of entered and actual #shifts
- S14 - excessive state elements in scan cell
- S15 - invalid scan cell constraint
- S16 - multiple constraints on a scan cell
- S17 - invalid single shift chain length
- S18 - scan cell disturb
- S19 - nonscan cell disturb
- S20 - invalid user #shifts
- S21 - too many apply single shifts
- S22 - multiple clocked scan chain
- S23 - unobservable potential TLA
- S24 - clock connected to data input of potential TLA
- S25 - hot set/reset line on potential TLA
- S26 - no usable clock port for potential TLA
- S27 - clock_po connection for potential TLA
- S28 - fail TE port connection for potential TLA
- S29 - invalid dependent slave operation
- S30 - unstable RAM during test procedure operation
- S32 - Scancell set to fixed binary value after load was violated
- S34 - Specified SPC chain *spc_chain* is invalid. (S34-1)
- S35 - Specified chain C contains non-hold pipeline cells in position P (I).
- S36 - Scancell G shifts data on trailing edge.
- S37 - Scancell G captures data from shift path.

DRC Rule S1

Message Text

Scan Chain Rule S1: Chain C blocked at T gate I (G) after tracing N cells.

Severity

Fatal Error

Description

The scan chain path must be sensitized by the conditions that exist during the application of the shift procedure.

C is the name of the scan chain; T is the type of gate; I is the instance pathname of the gate; G is its corresponding gate ID; and N is the number of scan chain cells traversed before the blockage occurred. Counting starts with zero at the scan output port and increases as the analysis moves toward the scan input port.

This is a fatal error condition that must be corrected by changing the shift procedure or changing other procedures or settings that condition the shift procedure such as test_setup, PI constraints, or restricted clocks using set_drc-clock.

This check is performed by placing an "X" on the scan input and simulating the clocking of the shift procedure until all logic stabilizes. Then, using the event sequence of the simulated shift procedure, the scan output is traced backward through combinational and sequential gates that have a sensitized path (value=X) until the scan input is reached.

A rule violation occurs if the path cannot be determined or cannot pass through a sequential element.

What Next

A violation can be analyzed using the GSV by selecting its violation ID number from the ANALYZE button. This changes the pin data reporting to the simulated shift procedure data and displays all gates in the backtrace cone of the blocked gate. By reviewing the shift data near the blockage point, you might be able to determine the cause of the problem.

If the scan blockage occurs after tracing 0 cells a common problem is that a tristate enable is off for tristateable or bidirectional scan outputs. For bidirectional ports used as scan outputs you should also explicitly force a Z on this port in the load_unload procedure.

Be aware that the TestMAX ATPG scan chain trace algorithm places an "X" value on the scan data path. In this case an "X" is a good thing! TestMAX ATPG works from the scan output pin back to the scan input pin. It follows the location of the "X" to help it figure out the scan data path. When the data path is a constant value of 1 or 0 instead of X this

means there is some PI constraint or other constant value which is interfering with the scan data path by forcing the net to a constant 1 or 0.

Look for clock pins which are either X or are not being clocked. This might be because the proper port is not assigned a "port=P;" in the Shift procedure, the port is not defined as a clock, or because the clock path is blocked or uncertain as it passes through MUXes or other gates. If a clock passes through a MUX, the select line of the MUX must be constrained to a constant value either directly from a top level port, or indirectly from nonscan logic which is stable during ATPG.

Look for asynchronous set and reset pins which are either "X" or being pulsed. Either of these conditions will result in the scan blockage.

Look for data paths which are incorrect because the scan enable control is either at the wrong state, or connected with the wrong polarity.

When you run DRC, do you get a S1 violation that appears as if the clock is not clocking? Are you using the Quick STIL tab in the DRC dialog box to add clocks and scan chain/enable information to create a TestMAX ATPG STIL procedure file?

You can have used the add clock command or GSV to add your clocks, but could have forgotten to specify that clocks should be used as a SHIFT clock. In doing so, the generated STIL procedure file is a missing "clk"=P; statement in the shift procedure.

You must use the add clock `-shift` option to specify a clock that is used for shift on the command line; for example,

```
add_clocks 0 clk -shift -timing 100 50 80 40
```

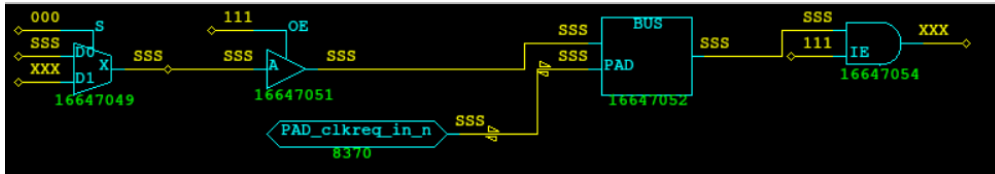
When using the GSV Edit Clocks dialog box, make sure to select the Shift checkbox to the right of the port name declaration. This port name you specify is used as a shift clock. Now the STIL procedure file will have the needed information and the DRC will pass.

A correct shift procedure could look similar to this:

```
Shift {  
  V { _si=\r1 #; _so=\r1 #;  
    "clk"=P; } # clk should be pulsed during shift  
}
```

For S1 violation, the `analyze_violation` command cannot give clear information for debugging in the following case.

One possible cause may be associated with the bus device, which is a gate inserted by TestMAX ATPG during ATPG design building to resolve multidriver nets. Both potential sources for the bus inputs appear to be driving, and both have values of X. One of the sources that has an X value is the `PAD_clkreq_in_n` bidirectional port. You can correct this by driving the port to a Z state. Edit the STIL file to add the declaration `PAD_clkreq_in_n = Z` to one of the `V{..}` vectors at the start of the `load_unload` procedure.



DRC Rule S2

Message Text

Scan Chain Rule S2: No scan cells identified in scan chain C.

Severity

Fatal Error

Description

A scan chain must contain at least one scan cell.

What Next

This is a fatal error condition that requires removing the scan chain from the scan chain list defined in the DRC file.

You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the shift procedure simulated data and displays all gates in the path from the defined scan input to the output.

The `report_scan_chains` command can be used to list the scan chains along with their input and output ports.

DRC Rule S3

Message Text

Scan Chain Rule S3: Traced input P1 of chain C does not match entered value P2.

Severity

Fatal Error

Description

The traced scan chain input pin must be the same as the defined input pin of the chain.

P1 is the port which was found to be the scan input by the simulation of the shift procedure. P2 is the port defined in the DRC file to be the scan input, and C is the scan chain name.

What Next

You can analyze a violation in the schematic viewer by selecting its violation ID number using the `ANALYZE` button. This changes the pin data reporting to the simulated shift procedure data and displays all gates from the traced scan in port to the adjacent scan cell. By reviewing this circuitry, you might be able to determine the cause of the problem.

If the correct scan-in port is the port identified by the analysis, edit the test protocol file and rerun DRC.

If the displayed port is not the correct scan input, then the port assignments in the load_unload procedure before the shift procedure should be reviewed and possibly changed to sensitize the path to the desired scan input instead of the one identified by TestMAX ATPG.

You can also analyze an S3 violation by using the `set_drc -chain_trace chain_name` command and rerunning DRC to trace the shift data propagation on the affected scan chain.

DRC Rule S4

Message Text

Scan Chain Rule S4: Adjacent latches I1 (G1) and I2 (G2) in chain C load data at same time.

Severity

Warning

Description

Adjacent latches in a scan chain path must not be active at the same time during the application of the shift procedure.

This check is performed during the tracing of the scan chain through a sensitized path using the simulated values of each time period of the shift procedure.

What Next

You can use the graphical schematic viewer (GSV) to analyze this violation by selecting its violation ID number and clicking the `ANALYZE` button. This sets the gate reporting to the shift procedure simulated data and displays all gates between the two latches. In some situations, DRC will not report adjacent latches. However, you can use the `report_lockup_latches` command to list adjacent latches with a M832 message.

DRC Rule S5

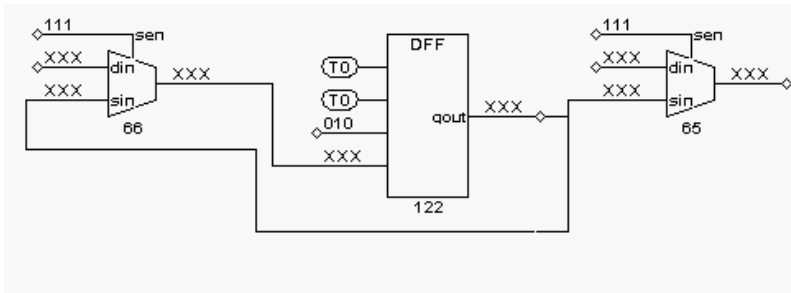
Message Text

Scan Chain Rule S5: N (G) in chain C previously used in chain trace.

Severity

Fatal Error

One example, many variations are possible:



Description

A latch or flip-flop gate can only be traced once during the scan chain tracing process. The indicated cell was either traced twice for the same scan chain or is part of a previously traced scan chain and marked as part of the other scan chain already.

What Next

This is a fatal error condition that requires either removing the scan chain from the chain list or changing the chain conditioning. You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the shift procedure simulated data and displays as a schematic the multiply traced gate.

DRC Rule S6

Message Text

Scan Chain Rule S6: Input force of chain C must be loaded by first clocking of N (G) .

Severity

Error

Description

The forcing of the scan chain inputs in the shift procedure must occur so it can be loaded by the first clocking of the scan cell connected to the chain input.

What Next

You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the shift procedure simulated data and displays all gates from the traced scan-in pin to the adjacent scan cell.

Also, you can check the WaveformTable and make sure the bidirectional delay is 0.

DRC Rule S7

Message Text

Scan Chain Rule S7: Invalid propagation of input of chain C to scan cell N (G) .

Severity

Error

Description

The shift data path between the scan chain input and the first scan chain cell (state element) must be sensitized (able to propagate data) at the time of the shift clock. In addition, a scan chain output must be measured before the shift clock changes that value.

What Next

You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the shift procedure simulated data and displays all gates from the traced scan-in pin to the adjacent scan cell.

One commonly occurring cause of this violation is attempting to define an end-of-cycle protocol which does a pre-measure of "_so=#" before the Shift procedure without defining a timing block that calls for a measure time on outputs that comes after the clock. Double check the defined timing to make sure the measure time for outputs occurs after clocks.

DRC Rule S8

Message Text

Scan Chain Rule S8: Scan cell I (G) of chain C can change before output measure time.

Severity

Error

Description

The measure of scan chain outputs by the Shift procedure must occur before the scan cell connected to the chain output is clocked again.

`I` is the instance pathname of the scan cell in violation, `G` is the corresponding gate_ID number, and `C` is the name of the scan chain.

Failure to satisfy this rule results in the scan chain being unusable for ATPG.

What Next

A common cause of this violation is an incorrectly defined clock shifting sequence in the Shift procedure. The event order of the Shift sequence is controlled at two levels: at a macro level it is controlled by the number of `V{ . . . }`, vector statements used in the Shift procedure. At the micro level the event order within each vector statement is controlled by the defined pin timing found in the Timing block. Both areas should be reviewed to check the event order of the Shift procedure.

Another possible cause of this violation is forgetting to have the scan out pre-measure required before the Shift procedure when the end-of-cycle measure protocol is used. When the measure times defined in the Timing definition of the DRC procedure file occur after clock pulses, then this implies the end-of-cycle protocol is to be used. For more information and examples of this protocol see the "Creating End-of-Cycle Measures in ATPG Patterns" topic.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to the simulated shift procedure data and displays gates between the scan output port and the scan cell with the problem. By reviewing the clocking patterns of the scan cell, you might be able to determine the problem.

DRC Rule S9

Message Text

```
Scan Chain Rule S9: Unobservable master (G) in chain C due to missing
master_observe procedure.
```

Severity

Error

Description

A `master_observe` procedure must be defined if a MASTER cannot be observed by the `load_unload` procedure.

`g` is the `gate_ID` of the scan cell considered to be a MASTER, and `c` is the name of the scan chain.

What Next

Try defining a `master_observe` procedure in the STIL procedure files and rerunning DRC. Your goal is to capture or "observe" the value in the MASTER cell identified into some other scan chain cell by performing one or more clocking operations.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to the simulated shift procedure data and displays all gates in the chain path between the failing MASTER gate and the scan cell output gate.

DRC Rule S10

Message Text

```
Scan Chain Rule S10: No master gates identified using master_observe
procedure.
```

Severity

Warning

Description

A `master_observe` procedure was defined but was not necessary. To be necessary at least one MASTER gate must be unobservable without the `master_observe` procedure.

What Next

This violation has no automated schematic viewing analysis. If all master gates are observable, the `master_observe` procedure is redundant and you should remove it from the test protocol file. If some master gates were found to be unobservable (rule S11), analyze those violations. This might identify a problem with the content of the `master_observe` procedure.

DRC Rule S11

Message Text

```
Scan Chain Rule S11: I (G) not successfully observed by master_observe
procedure.
```

Severity

Error

Description

A master_observe procedure must successfully observe master gates that cannot be observed by the load-unload procedure.

`I` is the instance pathname of the scan cell in violation, and `G` is the corresponding gate_ID number.

What Next

You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the master_observe procedure simulated data and displays all gates in the chain path from the failing master gate to the scan cell output gate.

DRC Rule S12

Message Text

```
Scan Chain Rule S12: No observable shadows identified using  
shadow_observe procedure.
```

Severity

Warning

Description

A shadow_observe procedure was defined but no SHADOW devices were observed by it. A shadow_observe procedure must observe at least one shadow gate or it is unneeded.

What Next

This violation has no automated schematic viewing analysis. If there are no shadow gates, the shadow_observe procedure is redundant and you should remove it from the test protocol file. If there are shadow gates that were intended to be observable, perform an analysis by displaying that gate with pin data selected to be "shadow_observe". Navigate through circuitry searching for why the shadow gate doesn't propagate to the output of its associated scan cell. This can identify a problem with the content of the shadow_observe procedure.

DRC Rule S13

Message Text

```
Scan Chain Rule S13: Traced number shifts (N1) for group G does not match  
entered value (N2).
```

Severity

Ignored

Description

The traced number of shifts must equal the entered number of shifts for a scan chain group. If a mismatch occurs, the traced number of shifts is used.

What Next

This violation has no automated schematic viewing analysis. A violation only indicates that the calculated number of shifts to load the scan chain did not match the number of applied shifts entered in the test protocol file. Normally, this is not a problem and the calculated number of shifts is used.

DRC Rule S14

Message Text

Scan Chain Rule S14: Excessive scan cell states due to *cell_type* *instance_name* (*gate_id*) in chain *scan_chain*.

Severity

Fatal Error

Description

A scan cell cannot contain state elements other than a master (with an optional dependent slave), a slave (with an optional dependent slave), and any number of shadows.

The *cell_type* is the cell type of the DFF or DLAT. The *instance_name* is the instance pathname of the affected scan cell. The *gate_id* is the corresponding gate ID, and the *scan_chain* is the name of the scan chain.

What Next

This is a fatal error condition that requires you to either remove the scan chain from the chain list, or change the chain shifting or conditioning, or modify the design.

You can analyze an S14 violation using schematic analysis by selecting its violation ID number. This sets the gate reporting to the shift procedure simulated data and displays the failing gate.

When this DRC violation occurs, all subsequent DRC checks are halted. You should specify the `report_rules-fail` command to list other DRC problems. You can obtain additional information for understanding a S14 violation by reviewing related DRC violations.

One common cause for this violation occurs when there are two or more latches in a series separated by combinational gates, and the controlling gates for each latch are simultaneously active. This is a timing hazard. In this case, you should use the ANALYZE button to select the S14 violation, then start your analysis on the gate farthest away from the input and analyze backward in the logic cloud for other latches.

Since an S14 violation is a fatal error, only the first violation is reported. If more than one S14 violation exists in a design, the debugging process can be intensive. If you suspect there are several S14 violations, specify the `set_rules S14 -verbose` command before the `run_drc` command. This will print all the S14 violations. You can then analyze these violations using the `analyze_violation` command.

DRC Rule S15

Message Text

Scan Chain Rule S15: Invalid cell constraint defined for N.

Severity

Fatal Error

Description

You can assign a cell constraint only to a valid scan cell.

What Next

This is a fatal error that requires you to either remove or correct the cell constraint.

To obtain a list of legal scan cell names:

Remove all cell constraints so the `run_drc` command processes completely.

Use the `report_scan_cells` command to list either the scan cells for a specific scan chain or all scan chains.

An ambiguous instance pathname usually occurs when the library cell contains more than one sequential element (such as an LSSD scan style), or when multiple scan cells are defined under a single module with a ``celldefine` construct. In this case, you should specify the gate ID of the scan cell on which to place the cell constraint, or specify the scan chain name and scan cell bit position. The gate ID can change if the design changes or if a flattened design environment changes (such as a gate optimization change).

Other options to eliminate the S15 in this context include:

Remove the ``celldefine` construct for the module.

Do not use the `-lib` option of the `read_netlist` command to maintain the internal scan cell names in the module.

If the violation message identifies a gate ID, use the `analyze_violation` command to graphically display the violation. This command sets the gate reporting to "none" and displays the invalid gate. If the cell constraint is applied to a nonscan device, you can also use the `add_capture_masks` command.

DRC Rule S16

Message Text

Scan Chain Rule S16: Multiple cell constraints defined for N.

Severity

Fatal Error

Description

No more than one cell constraint can be placed on the same scan cell.

This is a fatal error condition that requires either removing or correcting the cell constraint.

What Next

Enter the following report violations command to obtain details of all s16 violations:

```
TEST> report_violations s16
```

A violation due to a cell constraint identified by an invalid gate can be analyzed using the schematic analysis by selecting its violation ID number. This sets the gate reporting to "none" and displays the invalid gate.

DRC Rule S17

Message Text

Scan Chain Rule S17: Chain C length (L1) using single shift not equal to chain length (L2).

Severity

Fatal Error

Description

The chain length calculated for a trace of a single shift application must be the same length as that calculated for the general shift application.

What Next

This is a fatal error condition that requires either removing the scan chain from the chain list or changing the chain conditioning in the shift or load_unload procedures.

DRC Rule S18

Message Text

Scan Chain Rule S18: Scan C I (G) disturbed during time T of P procedure.

Scan Chain Rule S18: Non-state holding pipeline cell C I (G) disturbed during time T of P procedure.

Severity

Error

Description

The values loaded into scan cells by the `load_unload` or shift procedure, or captured in scan cells by the capture procedures, must not be disturbed by the other procedures.

`C` is the cell type of DFF or DLAT, `I` is the instance pathname of the affected scan cell, `G` is its corresponding `gate_ID`, `P` is the name of the procedure, and `T` is the time within the procedure when the cell was disturbed.

Failure to satisfy this rule can result in inaccurate simulation results and the generation of ATPG patterns which fail in simulation, and also unreliable coverage results and fault classifications.

This check is performed by using the simulated values of each time period of the test procedures. The rule violation occurs if any clock input of any scan cell state element allows data to be captured at an inappropriate time.

The second variation of the S18 violation message occurs only for non-state holding pipeline cells with uncompressed design. In this case, the spf include this keyword **"PipelineStateHolding "False" "**.

What Next

A common cause of this problem is often an inappropriately applied clock pulse or active asynchronous set/reset within a procedure.

You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the failing procedure pattern data and displays all gates in the back trace cone of the disturbed gate. You might need to toggle between Load pindata and Shift pindata to see the whole load operation, in case an extra clock edge is caused by a change in the clock-off state.

It is common for this violation to occur on designs using JTAG/TAP controllers. Particularly in the area of the TDO output register, as it tends to be clocked on the trailing edge of TCK while the rest of the scan chain is clocked on the rising edge.

Sometimes the source of the violation is at a sequential cell identified as a shadow register to a scan cell. If this is the case you can see some benefit in using the `-serial_shadows_only` or `-noshadows` option of the `set_drc` command to change shadow register recognition. You need to rerun DRC after making this change.

Use the `-mask` option of the `set_rules` command to compensate for the DRC violation. If you set the rule severity to warning and enable the `-mask` option, the ATPG algorithm masks off the control and observe values to ensure that any ATPG pattern generated passes simulation. The tradeoff in doing this is that there is a reduction of test coverage.

Sometimes a clock chain for an on-chip clock controller has S18 violations, especially if the ATE clock is used as the clock for the clock chain. These cells must be masked using the command `add_cell_constraints OX` to allow them to control the on-chip clock controller without their final value being strobed. This does not result in any coverage loss because the clock chain does not capture new data during capture cycles.

If you see second variation of S18 rule violation message, you can safely downgrade the violation using `set_rules S18 warning` to pass DRC and VCS. You must mask non-state holding pipeline cells by default to pass VCS.

DRC Rule S19

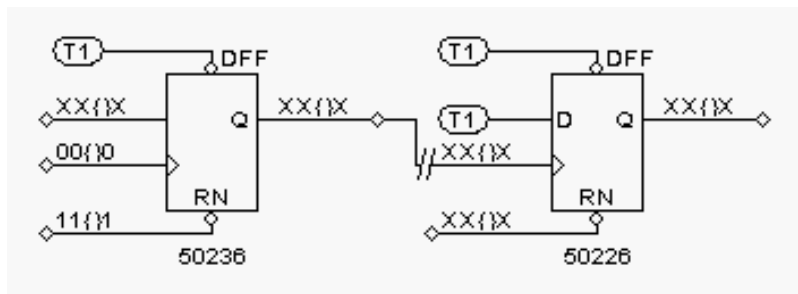
Message Text

Scan Chain Rule S19: Nonscan Cell I (G) disturbed at time T of P procedure.

Severity

Warning

One example, many variations are possible:



Description

Nonscan cells must hold their state during the application of the load_unload procedure.

`C` is the cell type of DFF or DLAT, `I` is the instance pathname of the affected scan cell, `G` is its corresponding gate_ID, `P` is the name of the procedure, and `T` is the time within the procedure when the nonscan cell was disturbed.

Passing this rule allows the fault simulator and test generator to use nonscan cell values that exist before loading and unloading scan chains to advantage and can increase test coverage. Failing this rule means the nonscan cells are treated as containing X values, and can't be used to advantage when creating ATPG patterns. There is no danger of generating patterns which fail simulation.

This check is performed using the simulated values of each time period of the load_unload test procedure. A rule violation occurs if any nonscan cell clock/set/reset line becomes potentially active. For a given nonscan cell, only the first occurrence of a violation is reported.

What Next

It is quite common for designs which contain both scan and nonscan sequential cells to have the nonscan cells disturbed. This DRC violation should only be of concern if you had expected the nonscan cells to be stable by design. If this is the case, then further investigation should be performed.

One common cause of this S19 violation is the DFF devices which form the output registers on synchronous RAMs. As TestMAX ATPG creates a derived ATPG model for the RAM it inserts DFF devices for the RAM data output ports. These devices are nonscan and can trigger the S19 violation.

You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the failing procedure pattern data and displays all gates in the back trace cone of the disturbed gate.

It's important to note that the loadable nonscan cells feature is limited in its effectiveness. It will not be of much benefit if the load_unstable cells form a deep sequential circuit, have sequential loops, or are fed by X-generators. Since this feature has runtime penalties in DRC, ATPG, and simulation, you should make sure you are getting a net benefit when using it. You can check its effectiveness by comparing the number of S19 violations to the following line of `run_drc` output (which is only printed when the `set_drc -load_nonscan_cells` command is specified):

```
Nonscan load cell identification completed: #load_nonscan_cells=6  
(nonx=4)
```

Note that the loadable nonscan cells feature is beneficial if the `#load_nonscan_cells` value is large and/or this value comprises a large percentage of S19s violations.

For more information, see the following:

Using Loadable Nonscan Cells in TestMAX ATPG

```
set_drc -load_nonscan_cells
```

```
set_simulation -shift_cycles
```

DRC Rule S20

Message Text

Scan Chain Rule S20: Selected number of shifts L1 is less than chain C length L2.

Severity

Error

Description

The user-selected number of shifts for a group of scan chains must not be less than the length of the longest scan chain of the group. If the severity is not error, the number of shifts is set to the length of the largest scan chain.

What Next

This violation has no automated schematic viewing analysis. A violation indicates that the calculated number of shifts to load the scan chain was less than the user selected number of shifts. A scan chain must be capable of being fully loaded during a load operation, and requires at least a number of shifts equal to the length of the longest scan chain. Either change the selected number of shifts to meet this requirement or change the severity to warning or ignore, which results in using the calculated number of shifts.

DRC Rule S21

Message Text

Scan Chain Rule S21: Number of single shifts (L1) is not less than chain C length (L2).

Severity

Fatal Error

Description

For a chain group, the number of single shift applications must be less than the length of the longest scan chain.

What Next

This is a fatal error condition that requires the removal of some single shift applications by adjusting procedures.

DRC Rule S22

Message Text

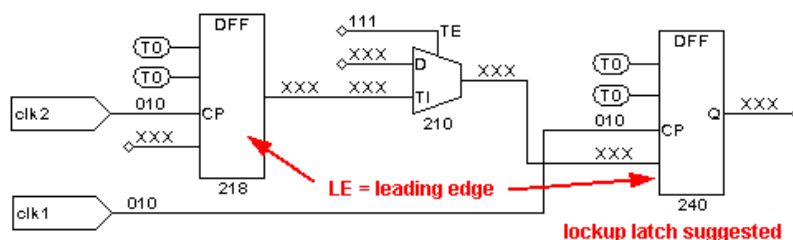
Scan Chain Rule S22: Multiple clocks (*S2C2*) were used to shift scan chain *chain_name*.

Severity

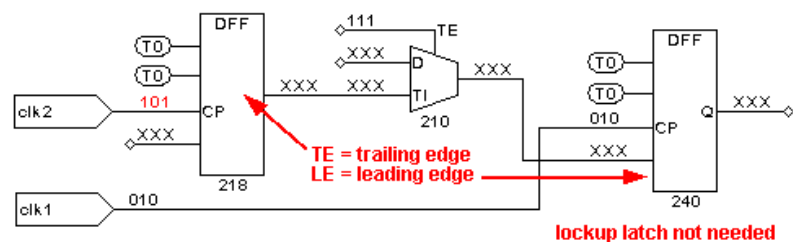
Warning

Examples

First example, LE to LE devices, LOCKUP latch suggested:



Second example, TE to LE devices, ignore S22 warning:



Description

All master gates of scan chain cells in the same scan chain should use a single shift clock. This rule is a safeguard to try to avoid a situation where chip tester clock skew results in tester pattern failures during the shifting of the scan chains. If a single clock is used then no clock skew due to the tester can occur. If more than one clock is used then any skew can result in the rejection at the tester of potentially good devices.

S2 and *C2* are the clock port names involved, and *chain_name* is the symbolic name of the scan chain as defined in the DRC procedure file used with the `run_drc` command.

This check is performed by inspecting the clocks sources of the state elements for all scan cells of a scan chain without regard to the polarity of the clocks or the phase relationship between them.

A rule violation occurs when a scan chain uses more than one clock to shift data. A potential problem can exist, but you will need to further investigate the affected clock polarities. Only the first occurrence of an S22 violation on each scan chain is identified, so investigating one violation does not mean there are not others on the same scan chain. You can specify the `set_rules s22 -verbose` command to print all instances and clocks of the violating transition.

Note that the S22 message is not reported on clock domain crossings that are identified with a DSLAVE lockup latch. This is only reported on clock domain crossings that have direct paths between two scan Master flops. If this checking is not sufficient, you can use the `report_lockup_latches` command.

What Next

You can use the `report_violationss22` command to list the name of the scan chain and the clocks that have been identified as shifting the scan chain.

A violation can also be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This changes the pin data reporting to "shift" and displays two scan elements from the same scan chain along with their different clock sources.

When the clocks of the two scan chain elements involved are of the same phase and polarity (both devices are LE) you should consider adding a lockup latch between the two scan cells, using the inverted clock of the first element to enable the latch. This will protect the scan chain from shift failures due to tester skew of the clocks involved.

When the clocks of the two scan chain elements are of opposite phase so that a TE to LE condition exists, you might investigate whether the clock of the first device might be defined with the opposite polarity. This can often have a beneficial effect on test coverage. If this is not possible, then it is generally safe to ignore the S22 violation for this specific clocking arrangement.

If the clocks can be safely pulsed at the same time for both scan chain shifting and normal operation, you might want to declare the clocks as equivalent using the `add_pi_equivalences` command. This will result in both clocks always being operated simultaneously and will eliminate the S22 violation due to these clocks. It can also have a benefit in reducing total pattern count. However, before declaring two or more clocks as being equivalent the clocks should have identical timing and you should be sure that these clocks can be safely operated together under normal design operation as well as scan data shifting.

See Also `report_lockup_latches` in the TestMAX ATPG and TestMAX Diagnosis Commands Guide

DRC Rule S23

Message Text

Scan Chain Rule S23: Latch I (G) not transparent due to not observable.

Severity

Warning

Description

A potential transparent latch gate must have a path to a primary output, a scan cell, another transparent latch, or a sequential-scan cell to be observable.

I is the instance pathname of the DLAT primitive and **G** is its corresponding gate_ID.

Failure to satisfy this rule results in the data through the latch not being observable. Consequently fault sites involved with this path cannot be detected and a reduction of test coverage can occur. There is no danger of generating ATPG patterns which fail in simulation.

What Next

If a significant number of faults associated with the latch are untestable, it might be worthwhile to add an internal scan cell to observe the value coming out of the latch.

You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the clocks-off simulated data and displays the forward trace cone of the failing gate.

DRC Rule S24

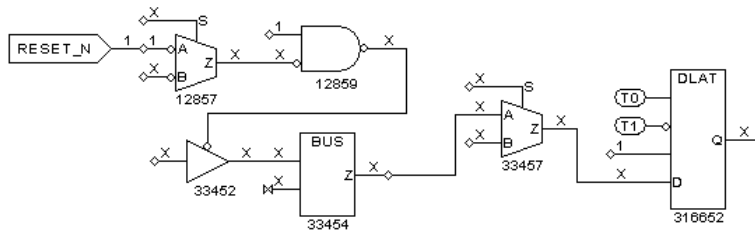
Message Text

Scan Chain Rule S24: Latch I (G) not transparent due to data input (N) connected to clock.

Severity

Warning

One example, many variations are possible:



Description

A potential transparent latch gate must not have a path from any of its data inputs to a clock. Failure to satisfy this rule prevents the latch from being used as a transparent latch and can lead to some reduction of test coverage.

I is the instance pathname of the DLAT primitive, G is its corresponding gate_ID, and N is the DLAT primitive input pin number (0=set, 1=reset, 2=clock, 3=data, and so forth.).

What Next

You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the clocks-off simulated data and displays the backward trace cone of the failing gate to clock pins.

DRC Rule S25

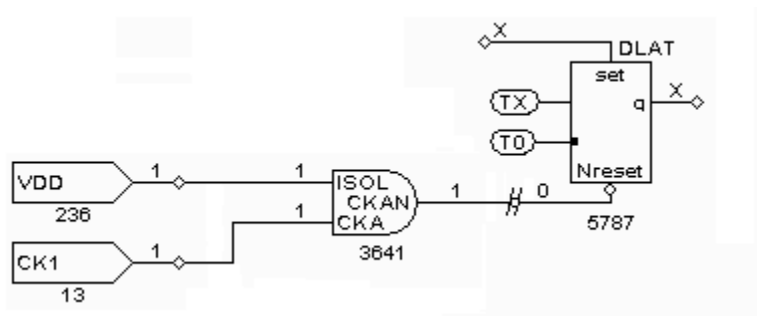
Message Text

Scan Chain Rule S25: Latch N (G) not transparent due to hot SET/RESET line.

Severity

Warning

One example, many variations are possible:



Description

A potential transparent latch gate must not have an active set or reset input when all defined clocks are off.

Failure to satisfy this rule keeps the latch from being used in a transparent mode.

A violation of this rule introduces no danger of generating bad patterns. The latch cannot be made transparent and is treated as a constant X value. This in turn, can cause a reduction of test coverage.

What Next

If the number of untestable faults associated with the latch is significant, you might want to look at design changes to eliminate the active set/reset during ATPG test mode.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to "clock off" simulated data and displays the backward trace cones of the set/reset input of the gate with the violation.

DRC Rule S26

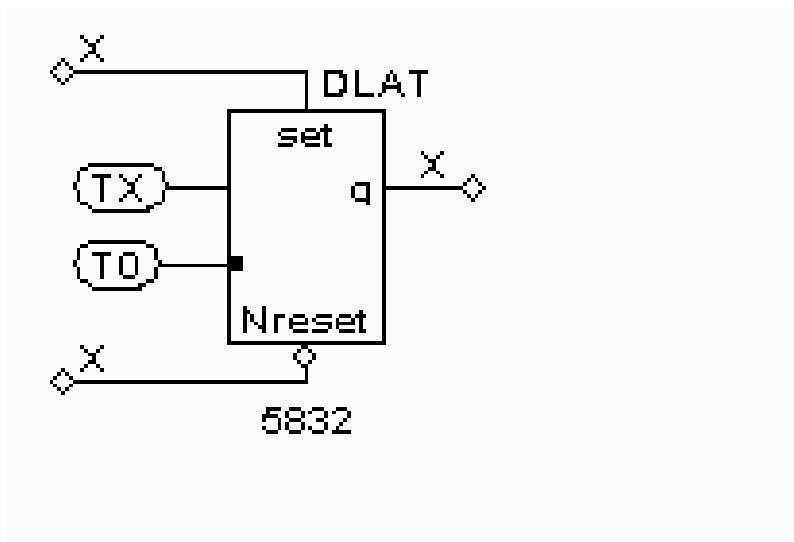
Message Text

```
Scan Chain Rule S26: Latch N (G) not transparent due to no usable clock ports.
```

Severity

Warning

One example, many variations are possible:



Description

A latch gate must have at least one clock port that can be made active when all defined clocks are off to be considered a transparent latch.

Failure to satisfy this rule keeps the latch from being used in a transparent mode.

A violation of this rule introduces no danger of generating bad patterns. The latch cannot be made transparent and is treated as a constant X value. This in turn, can cause a reduction of test coverage.

What Next

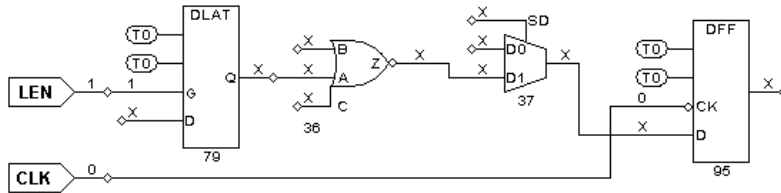
A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This changes the pin data reporting to "clock off" simulated data and displays the backward trace cones of all clock inputs of the gate with the violation.

You should also investigate the derived ATPG gate level model for the latch by selecting the latch in question in the graphical view and picking "Go Down Hierarchy" from the RMB menu. Sometimes the functional model for a latch produces a derived model where the SET and RESET of the DLAT primitive are used but the clock and data inputs are not. If you find this to be the case but your original vendor library module did not intend this functionality then you might need to create a replacement ATPG model to use.

Severity

Ignored

One example, many variations are possible:



Description

A potential transparent latch gate must not have a path to a trailing edge (TE) cell that can capture new data.

A violation of this rule introduces no danger of generating bad patterns but is a warning that a potential reduction of test coverage can result because the latch cannot be made transparent.

What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This changes the pin data reporting to "none" and displays the gates in the path between the failing gate and a failing trailing edge cell.

DRC Rule S29

Message Text

Scan Chain Rule S29: Dependent slave I1 (G1) cannot hold same value as master I2 (G2).

Severity

Warning

Description

A dependent slave device must always carry the same value as its master device. I1 and I2 are instance pathnames. G1 and G2 are the corresponding gate ID numbers.

If this rule is violated, the dependent slave device might have a value different than its master device either at the end of the shift procedure or after a capture clock. This might lead to the generation of ATPG patterns that fail during simulation. However, TestMAX ATPG takes steps to avoid a potentially bad patterns as described in the What Next section.

TestMAX ATPG performs this check using shift data. It considers the first device to be loaded during shift as the master and the second device loaded during shift as the dependent slave.

What Next

You can continue by changing the severity of this rule to either warning or ignore using the `set_rules` command and then repeating the `run_drc` command. The ATPG algorithm marks the affected master or slave device as an observe X device to avoid generating patterns that fail in simulation. However, this observe X condition might result in lower test coverage.

To analyze a violation using the schematic viewer, select its violation ID number from the ANALYZE button. This changes the pin data reporting to shift data and displays the gates in the path from the master device to the dependent slave device.

The following are common causes of the S29 scan chain rule violation:

A dependent slave device does not have exactly the same asynchronous set and reset controls as its master device. If the dependent slave has different values than its master, erroneous simulation data is created.

You are using the wrong polarity for clocks. TestMAX ATPG classifies a device as master or dependent slave (DSLAVE) based on which device captures first during a shift procedure. If you define clocks with the wrong polarity, TestMAX ATPG classifies a DSLAVE device as a MASTER device and associates it with a downstream sequential device instead of the upstream device. You should reverse the polarity of all clocks associated with the scan chain on which the S29 violation occurs.

A dependent slave device has a different clock than its master device with the opposite off state (phase). When this occurs, you might be able to redefine the polarity of a single clock for one of the devices (usually the dependent slave). When both devices use the same phase clock you eliminate the dependent slave/master relationship and have two masters instead. Your scan chain length increases and S22 violations (multiple clocked scan chain) occur. An S22 violation can also result in patterns that fail simulation, unless you use techniques such as lockup latches to prevent failures due to clock skew.

You can use the `-mask` option of the `set_rules` command to compensate for the DRC violation. If you set the severity to Warning and enable the `-mask` option, the ATPG algorithm masks the control or observe functions (or both) to ensure that any ATPG pattern generated passes simulation. However, it reduces test coverage.

DRC Rule S30

Message Text

Scan Chain Rule S30: RAM I (G) unstable during time T of P procedure.

Severity

Warning

Description

A RAM must hold state during all test procedure operations, except for the test setup procedure, if the RAM is to be used in read-only mode. "I" is the instance name of the RAM, "G" is its corresponding gate ID, "T" is an event time in test procedure "P".

This violation is detected during the simulation of the test procedure file. It indicates one of the write port controls or perhaps an optional asynchronous set or reset is not at an off state.

A violation of this rule introduces no danger of generating bad patterns but is a warning that a reduction of test coverage can result. The RAM cannot be used in read-only mode and its contents is set to all X's. This cannot be important if the RAM had no initialization file, the initialization file contained all X's, or the RAM is still controllable during Fast-Sequential pattern generation.

What Next

In general, it is better to have a RAM for which Fast-Sequential patterns can be generated than it is to have a RAM that operates in read-only mode (behaves like a ROM). If your RAM can be used during Fast-Sequential pattern generation, then you can change this rule to warning or ignore and continue.

The `report_memory` and `report_primitives` commands can be used to review whether the RAM is considered controllable during Fast-Sequential ATPG. Check that the RAM is has a stable clock and is stable during scan chain loads. If the RAM is considered usable during Fast-Sequential ATPG pattern generation, then test coverage cannot suffer at all, and in fact might be greater than treating the RAM as a read-only device.

This violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to "pattern data" and displays the gates in the backward trace cone of the RAM's write, set, or reset control input for which the violation was reported.

Based upon interactive investigation, the solution to eliminating the violation can involve changing one of the test procedures such as the `load_unload`, or `shift`, or changing logic, or perhaps finding a control input that can be constrained to avoid the RAM instability.

If no changes to the design or to test procedures are desired you can continue by using the `set_rules` command to change the severity of the rule to warning or ignore and the repeating the `run_drc` command.

DRC Rule S32

Message Text

Scan Chain Rule S32: Scancell set to fixed binary value after load was violated *number* times

Severity

Error

Description

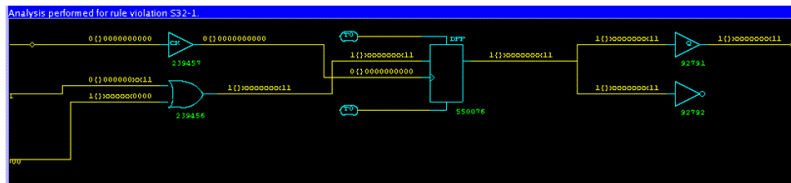
PLL shift clocks are internal nodes used for shift. These clocks might need to be defined when scan cells are set to binary values during various test procedures. This might result in incorrect identification of load0, load1 cells, which might lead to a loss of coverage.

The S32 rule checks for unconstrained scan cells identified as load0, load1. If DRC passes without defined PLL shift clocks, it implies that the definition of PLL shift clocks is not necessary.

When analyzing an S32 violation, both the value at the end of load_unload and the constrain value must be inspected for the failing scan cell. An S32 failure will occur if the value at the end of load is non-X (that is, 0 or 1), and the cause can be traced back in the GSV with pin data set to load_unload. An S32 failure can also occur when the value at the end of load is X — if the constrain value is non-X. For example, a D flip-flop (DFF) might have its reset line constrained active, so the loaded value is immediately replaced at the beginning of the capture cycle. In this case, you should set the pin data to constrain the value to trace the source of the problem in the GSV.

The load_unload pindata might show a non-X value at the end of load even though no clock pulse is displayed. When internal clocks are used, this situation can be caused by a sensitized path from a PLL clock source. The PLL clock pulses are simulated, but the GSV does not display them. To verify this, you can identify the clock used for the DFF using the `report_scan_cells -verbose` command, then use the `report_clocks -intclock` command to find the corresponding PLL source. The `analyze_violation s32-1` command displays the DFF with the load pindata. You can then view the PLL source separately, and use the GSV to see how they're connected and determine if the path is sensitized.

The following figure shows an example of this situation:



What Next

S32 violations must be analyzed and fixed. If you downgrade an S32 violation to pass DRC, a loss of test coverage might result, including increased test data and time. This might also result in generation of ATPG patterns that fail VCS simulation.

If the S32 violation is caused by PLL clocks, then removing all PLL clock and PLLSource definitions will prevent the undesired pulses. TestMAX ATPG DRC only requires these constructs for OCC controllers that require PLL pulses to enter shift mode. For other types of OCC controllers, it is safe to remove them.

DRC Rule S34

Message Text

Scan Chain Rule S34: Specified SPC chain *spc_chain* is invalid. (S34-1)

Severity

Error

Description

The chain name specified by the `set_drc -spc_chain chain_name` command does not exist in the SPF.

The chain name specified by the `set_drc -spc_chain chain_name` command exists in the SPF, but it is not an SPC chain.

The design is using the DFTMAX Ultra and the SPC bit is disabled to bypass the SPC extraction, but `set_drc -spc_chain chain_name` was still set before `set_drc`.

What Next

A violation of this rule is a fatal error and its severity cannot be downgraded. Use the `set_drc -spc_chain chain_name` command to specify the correct SPC chain or a chain that exists in the SPF and as an SPC chain.

If using DFTMAX Ultra and the SPC bit/port is being disabled, then do not specify the `set_drc -spc_chain chain_name` before running DRC.

DRC Rule S35

Message Text

Specified chain C contains non-hold pipeline cells in position P (I).

Severity

Error

Description

The pipeline cell must hold state during capture for optimal results in pipeline independent flow.

C is the name of the scan chain; P is the type of gate; I is the instance pathname of the gate;

DRC Rule S36

Message Text

Scan Chain Rule S34: Scancell G shifts data on trailing edge. G indicates the gate ID of the failing scancell.

Severity

Error

Description

When the *CAREShadow* shifts data on the leading edge (Message during DRC: *CAREShadows* is treated as capturing the leading edge). Scancells(closer to *CAREShadow*) are checked to ensure that they shift the data on the leading edge to make sure that *CAREShadow* and scancells do not shift the data in the same cycle during the `SHIFT_PROC` procedure.

What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that failing scancells shift the data on the leading edge. Use the command `set_pindata -shift` can be used to check on which edge *CAREShadow*/scancells shifts the data. This rule violation may be set to warning but it is not recommended since this could cause patterns to be incorrect.

DRC Rule S37

Message Text

Scan Chain Rule S37: Scancell G captures data from shift path. G indicates the gate ID of the failing scancell.

Severity

Warning

Description

Scancells closer to *CAREShadow* are checked if they capture data from the shift path. When scancell captures data from shift path, `noseq_comp_first_reseed_start` switch of the `set_patterns` command cannot be used. Using `noseq_comp_first_reseed_start` command may cause patterns to fail simulation as it changes the capture value of the scancell.

What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that failing scancell does not capture the data from `shift_path`.

The command `set_pindata -shift` can be used to check if the scancell can capture data from shift path.

Category V - Vector Rules

See Also

- [V1 - Parsing error](#)
- [V2 - Redefined item](#)
- [V3 - Superfluous block](#)
- [V4 - Missing definition](#)
- [V5 - Unresolved reference](#)
- [V6 - Unused item](#)
- [V7 - Unsupported construct](#)
- [V8 - Illegal context](#)
- [V9 - Invalid time](#)
- [V10 - Miscounted items](#)
- [V11 - Repeated entry](#)
- [V12 - Unexpected item](#)
- [V13 - Invalid state](#)
- [V14 - Missing state](#)
- [V15 - Illegal write patterns](#)

- [V16 - Miscounted arguments](#)
- [V17 - Clock forced on](#)
- [V18 - Mixed timing aggregate](#)
- [V19 - Missing clock definition](#)
- [V20 - Unsupported event](#)
- [V21 - Unsupported old syntax](#)
- [V22 - Invalid waveform](#)
- [V23 - Strobe conflict](#)
- [V24 - Unsupported pattern event](#)
- [V25 - Unload mode enable not defined](#)
- [V26 - Unload mode enable port incorrectly set](#)
- [V27 - Unload mode control port incorrectly set](#)
- [V28 - Pipelining not supported](#)
- [V29 - Too many events or too small a resolution in procedure](#)
- [V31 - Waveform simulation events](#)
- [V32 - Incorrect event sequence](#)
- [V33 - Missing constrained signal](#)
- [V34 - Invalid internal clock timing](#)
- [V35 - Invalid clock constraint](#)
- [V36 - Failed to map external chain](#)
- [V37 - External chain used default mapping to a codec](#)
- [V39 - Line L \(filename\), missing state in load_unload](#)

DRC Rule V1

Message Text

Vector Rule V1: Line L (filename), parsing error (<additional info>).

Severity

Fatal Error

Description

Parsing rules were not adhered to in the STIL procedure file. The problem was first identified on line 1 but issues might have occurred before this line. Note that any missing quotation marks (reported with the message `expected ""`) might not be identified until the end of the file. To find the origination of the syntax error, you might need to search the entire text of the file before the reported location.

What Next

Identify the location of the error, correct it, and try again.

DRC Rule V2

Message Text

Vector Rule V2: Line L (filename), redefined item (<additional info>).

Severity

Error

Description

Redefined items would have replaced the original definition.

What Next

Remove the duplicate definition and try again.

One possible condition which creates this violation is a change to the defined clock list and/or the PI equivalences list. Suppose you had three clocks A, B, and C and created a DRC file which had three corresponding capture procedures "capture_A", "capture_B", and "capture_C". Later you decide that you can declare these clocks as equivalent using the `add_pi_equivalences` command. Your next attempt at running DRC checks will produce a V2 violation with a message similar to:

```
redefined item (Procedure "capture_A" redefined)
```

The solution to this problem is to adjust the DRC file to contain only a single "capture_A" procedure which pulses all three clocks. Eliminate the other two capture procedures.

You can change the severity of this rule to a warning or ignore using the `set_rules` command but it is highly recommended that you attempt to eliminate the violation by identifying the cause and adjusting the DRC file so that the violation is eliminated.

DRC Rule V3

Message Text

Vector Rule V3: Line L (filename), superfluous block (<additional info>).

Severity

Error

Description

You must not redefine clocks that are already correctly defined and linked. The superfluous definition is ignored.

What Next

Remove the duplicate definition and try again or change the severity of this rule to warning or ignore using `set_rules` and try again.

DRC Rule V4

Message Text

Vector Rule V4: missing definition (*additional information*).

Severity

Fatal Error

Description

A required item is missing and must be defined.

What Next

Provide the missing item or remove the reference to it. Then specify the `run_drc` command again.

One possible cause for this violation is that a bidirectional scan input or output was referenced by name. If so, and you use the "actual_bidi_port_name"=#; syntax, then TestMAX ATPG cannot determine if the BIDI port should be forced or measured. Instead, try using predefined signal groups: `_si` for forcing scan inputs or `_so` for measuring scan outputs. For more information, see "Predefined Signal Groups in STIL."

Another possibility is that you defined a PI Constraint with a value of X. The default STIL Timing definition cannot define the `N` character. The `N` character is used in STIL to define an X value as input (the X character means don't measure an output). You can fix this by editing the Timing block of STIL to define the `N` character for inputs. The following example shows a typical timing definition with the additional `N` character definition line highlighted.

```
Timing {
  WaveformTable "_default_WFT_" {
    Period '100ns';
    Waveforms {
      "_default_In_Timing_" { 0 { '0ns' D; } }
      "_default_In_Timing_" { 1 { '0ns' U; } }
      "_default_In_Timing_" { Z { '0ns' Z; } }
      "_default_In_Timing_" { N { '0ns' N; } }
      "_default_Clk0_Timing_" { P { '0ns' D; '50ns' U; '80ns' D; } }
      "_default_Clk1_Timing_" { P { '0ns' U; '50ns' D; '80ns' U; } }
      "_default_Out_Timing_" { X { '0ns' X; } }
      "_default_Out_Timing_" { H { '0ns' X; '40ns' H; } }
      "_default_Out_Timing_" { T { '0ns' X; '40ns' T; } }
      "_default_Out_Timing_" { L { '0ns' X; '40ns' L; } }
    }
  }
}
```

You can also specify the `report_violations` command and review any [V16](#) messages in more detail.

DRC Rule V5

Message Text

Vector Rule V5: Line L (filename), unresolved reference (<additional info>).

Severity

Fatal Error

Description

With a few exceptions, all items must be defined before they are referenced.

One commonly occurring cause of this violation is the use of "clock=P" in the DRC file without either having a Waveform definition block or defining the port as a clock with the `add_clock` command.

Another possibility is that you have a bidirectional clock or asynchronous set/reset port. In the case of a bidirectional port which has been defined as a clock using `add_clock` command you must also provide the timing definition for this port in the Waveform definition block of the DRC file.

What Next

Provide the missing definition or remove the reference to it, then run DRC again.

DRC Rule V6

Message Text

Vector Rule V6: Line L (filename), unused item (<additional info>).

Severity

Warning

Description

There was an item in the DRC procedure file which was not used or ignored. The line number, L, will indicate on which line in the file the item occurred. The additional info will suggest a reason.

What Next

This is a warning that can be corrected by adjusting the line indicated.

The message:

```
Unused event "P" defined for signal(s)  port_name
```

will occur if a port or group of ports has timing defined which includes the ForcePrior or "P" waveform character. This "P" character is often necessary for proper WGL pattern output. However, the algorithm does not actually create patterns based on the "prior" signal value and this V6 violation is a warning to this difference between what is requested in the STIL timing and what the tool uses. The "P" is processed when creating the WGL pattern output file so if you are trying to generate WGL, then you should leave the DRC file alone and perhaps set this rule to ignore using the `set_rules` command.

The message:

```
unused item (Measures not allowed in test_setup; signal "XYZ")
```

will occur if you are trying to initialize a port to an X. The STIL character for forcing an undefined input is N. An X is considered an output measure that is masked. TestMAX ATPG does not support any measures of outputs in the `test_setup` procedure. Check your `test_setup` and change all X's to N's.

DRC Rule V7

Message Text

Vector Rule V7: Line L (filename), unsupported construct (<additional info>).

Severity

Warning

Description

Only a subset of the constructs in the vector format language can be used. The unsupported construct is ignored.

What Next

No action is necessary if the ignored construct will not negatively impact the usefulness of the data. Otherwise, restructuring the input file might be necessary.

DRC Rule V8

Message Text

Vector Rule V8: illegal context (<additional info>).

Severity

Fatal Error

Description

A construct can only be used in certain contexts. For example, STIL vectors cannot mix "#" and other waveform characters in the same statement.

The illegal construct is ignored and this can quickly lead to other syntax errors.

What Next

This violation is a fatal error, so it must be corrected to pass DRC.

You can specify the `report_violationsV8` command to view the full text of each reported violation. If there are many violations, or to see a specific violation only, use the following syntax:

```
report_violations V8-N
```

where "N" is the number of a specific violation to review.

The `report_violations` command will report additional detail about each violation that you can use to identify the situation to be addressed -- especially if the rule is a warning or has been downgraded to a warning.

The V8 violation can sometimes occur when a bidirectional scan input or output is used. If so, and you use the `"bidi_port"=#;` syntax then TestMAX ATPG cannot tell if the bidi port is to be forced or measured. Instead, try the predefined symbols `_si` for forcing or `_so` for measuring.

This violation can also occur when the defined timing for forcing inputs, measuring outputs, and pulsing clocks produces an event sequence that is not supportable by the current ATPG algorithm. Common causes of this are:

Attempting to use end-of-cycle measure timing when the scan out premeasure (`_so=#`) does not immediately precede the `load_unload` procedure just before the Shift macro. In this case, the waveform definition calls for a measure of scan output after the shift clock pulse, but the initial `scan_out` premeasure that TestMAX ATPG requires to support end-of-cycle is missing. See also: End-of-Cycle Measures.

Attempting to use the preferred preclock measure timing, but with an extra scan-out pre-measure (`_so#`). In this case, you have mixed the requirements of end-of-cycle with preclock. Remove the scan out premeasure and measure scan outs only within the Shift macro, or if your intent was to support end-of-cycle measure, you still need to adjust the measure time in the waveform definitions.

Messages containing references to “force-all-clocks” refer to multiple-cycle generic capture procedures. For more information, see “Creating Generic Capture Procedures” in the *TestMAX ATPG User Guide*.

An example message is as follows: `Error: illegal context (Procedure "multiclock_capture" is missing a force-all-clocks parameter). (V8-1)`

As is the case with standard capture procedures, the `multiclock_capture` procedure can consist of multiple vectors. In this case, you need to specify an additional argument to hold the variable clock-pulse information, as shown in the following example:

```
Procedures { "multiclock_capture" { // 2-cycle W "TS1"; C { "_po"=\r9
X ; } V { "_pi"=\r11 # ; "_po"=\r9 # ; } C { "_po"=\r9 X ; } V {"_clks"=
###; } } "multiclock_capture" { // 3-cycle W "TS1"; C { "_po"=\r9 X ; }
V { "_pi"=\r11 # ; } V { "_po"=\r9 # ; } C { "_po"=\r9 X ; } V {"_clks"=
###; } } }
```

You should review the pin timing of each waveform definition, and the event order of every procedure, including "capture" procedures.

Th V8 violation can also occur if the `-bidi_control_pin` option of the `set_drc` command has been used and the DRC file does not contain any capture procedures. If this is the case, use the `write_drc_file` command to create a file which defines the capture clock procedures in the LSI reflect IO template and then copy them into your original DRC file and try the `run drc` command again. See also: Creating LSI Compatible WGL

The V8 violation can occur when reading Extended VCD pattern files if the state codes in the data are inconsistent with the port direction in the top module. For example, if the top-level port "A" is an input, then TestMAX ATPG would not expect to see any of the state codes {L,H,X,T,I,h} which are for outputs. Likewise, if the top-level port "B" is an output, then TestMAX ATPG would not expect to see the state codes {D,U,N,Z,d,u} which are for inputs.

DRC Rule V9

Message Text

Vector Rule V9: Line L (filename), invalid time (<additional info>).

Severity

Fatal Error

Description

A valid time value (such as for defining signal timing) must be within certain limits.

What Next

Correct the error and run DRC again.

DRC Rule V10

Message Text

Vector Rule V10: Line L (filename), miscounted items (<additional info>).

Severity

Fatal Error

Description

The number of items in a list must correspond to previous definitions of that list.

One frequently occurring cause for this message is a mismatch in the number of '#' characters used for the scan in and scan out sections of the Shift procedure.

What Next

Extra items are automatically ignored but missing items must be supplied. Correct the error in the DRC procedure file and run DRC again.

DRC Rule V11

Message Text

Vector Rule V11: Line L (filename), repeated entry (<additional info>).

Severity

Warning

Description

An item is repeated on line *L*.

Frequently, this occurs during the assignment of values to ports because a port is assigned as part of a symbolic group and also assigned explicitly by name. An example would be:

```
V { _io=\r32 Z; bidi2 = 1; }
```

If the port "bidi2" is also a member of the symbolic group "_io" then a conflict in assigned values exists. The IEEE STIL specification indicates that the order from left-to-right of assignments within the vector should not affect the outcome. Rather than stop with an error, TestMAX ATPG uses the final value encountered reading from left to right for it's simulation and analysis and issues a V11 warning.

Note: For some forms of patterns generated, both values is assigned to the same pin at the same simulation time, to honor what is requested by the line in the procedure file. For some simulators, this will cause a glitch on the affected pin.

What Next

It is highly suggested that you adjust the indicated line in the procedure file to eliminate the duplicate assignment. This corrects the potential ambiguity of what value should be applied and also avoids the potential for a glitch during simulation.

Don't forget that the symbolic group "_pi" contains all inputs, including bidirectional pins and clocks.

You can find it convenient to define a custom symbolic group by copying an existing definition such as `_io` or `_pi`, deleting a few ports that you will explicitly set differently, and then using this new symbolic name instead of the original. An example would be:

```
V { MY_CUSTOM_LIST=\r31 Z; bidi2 = 1; }
```

This would correspond to the example earlier in which the symbolic group of `_io` was copied and the port `bidi2` was eliminated from this list. Now the line assigns 31 values to `MY_CUSTOM_LIST` and a single value to `bidi2`, without causing a multiple assignment. An example of using a symbolic group or `SignalGroup` can be found in Controlling Bidirectional Ports in STIL.

DRC Rule V12

Message Text

Vector Rule V12: Line *L* (*filename*), unexpected item (*additional info*).

Severity

Warning

Description

An unexpected definition was encountered, for example a waveform character 'Q'. Other possibilities include definition of waveform characters of {0,1,Z,N} which are input characters, for ports that are pure outputs. Or the reverse situation of defining output waveform characters {H/L/T/X} for ports which are pure inputs.

What Next

Note the following scenarios:

The V12 message reports: "Scanport names need to be changed for VCS simulation; use `-top_scanports` for VCS simulation."

In this case, the models used for the scan cell description do not allow direct parallel simulation because they are Verilog primitives. You should consider adding ``celldefine` references to your modeling flow. If modeling constructs are required, you need to write a special pattern file for simulation using the `-top_scanports` option of the `write_patterns` command.

The V12 message reports: "Scanport names changed for VCS simulation; use `-notop_scanports` for pattern read-back flows."

In this case, the pattern file is usable for simulation but cannot be read back into TestMAX ATPG, (for example, to support diagnostic flows with the pattern set).

To correct the violation indicated in the message, edit the STIL procedure file and repeat the `run_drc` command until the violation no longer appears. Failure to fix this violation can lead to unpredictable results later in the flow, including problems trying to write or read patterns.

If the message contains the text "STIL scan pad", see STIL Scan Padding Behavior.

DRC Rule V13

Message Text

Vector Rule V13: Line L (filename), invalid state (<additional info>).

Severity

Fatal Error

Description

An invalid state was found, for example a clock that should be at its off state at the end of a procedure was left on.

What Next

Correct the error and run DRC again.

DRC Rule V14

Message Text

Vector Rule V14: Line L (filename), missing state (<additional info>).

Severity

Warning

Description

A missing state was found, for example a constrained pin was not set to its constrained value in the test_setup macro.

If you are reading in functional patterns with the `set_patterns` command you can have forgotten to specify the `-strobe` option to be applied to the Extended VCD patterns.

If you are using the `-strobe` option, the port name you specified cannot be found in the design or in the Extended VCD pattern file. Double check spelling and case of the port name.

To review the context of a V14 message, specify the command `report_violations V14`. If TestMAX ATPG displays the following violation message:

```
(Missing alias in load_unload for <signal> in chain name,
group name; cannot write proper STIL for scangroups)
```

Then, adjust the scan references in the Vector of the Shift block pertaining to the group stated in the warning message. This adjustment is only required if you are generating STIL pattern data for this test. See "Multiple Scan Groups" in the *TestMAX ATPG User's Guide* for more details.

What Next

If this violation occurs during `run_drc` then the missing state is automatically added. You can write out the corrected DRC file using the `write_drc_file` command and compare it to the original.

If this violation occurs for the set patterns command, retry the command and specify one or more `-strobe` options.

If the message contains the text `STIL scan pad`, see [STIL Scan Padding Behavior](#).

DRC Rule V15

Message Text

Vector Rule V15: No pin name for parallel load.

Parallel STIL format doesn't support multi cycle Shift for `scan_compression`.

Severity

Error

Description

TestMAX ATPG cannot write patterns in the selected format using the specified pattern options.

What Next

Try a different format, or if you were trying a parallel form, try the serial form instead.

If you are attempting to use both forms by writing STIL patterns (`-format stil`) after modifying procedures for STIL99 (`-format stil99`), first write all the STIL patterns you need and then write the STIL99 patterns.

If you are attempting to write parallel Verilog patterns when the message occurred and the violation message suggested that "N+1" shifts were required where "N" is the length of the scan chain the most likely your design contains a multi bit library cell with no accessible internal instance names. The presence of a ``celldefine` directive around the definition of the multi bit module or use of the `-library` option of the `read_netlist` command restricts access and use of the internal instance names necessary for parallel simulation. You might be able to workaround the issue by remodeling the multi bit cell to remove the ``celldefine` and make the internal instance names available for simulation.

If you are attempting to write WGL patterns for LSSD-style designs, the problem might be due to a multi cycle Shift procedure. WGL only supports a single cycle shift procedure. Make sure you have a Timing block defined in the STIL Procedure file. Review the timing of the master and slave clocks to ensure that they can be applied successfully in a single tester cycle. Then adjust your Shift procedure to use a single test cycle. Pulse both master and slave clocks in this cycle.

Note that WGL patterns are not supported for a serialized compressed core without a serializer clock controller.

This problem might also occur because you specified the `set_scan_ability` command earlier in your flow. You cannot write patterns to a file if any of the patterns were generated

with instances on the scan ability list. Any such patterns would be invalid for the current design.

DRC Rule V16

Message Text

Vector Rule V16: Line L (filename), miscounted arguments (<additional info>).

Severity

Warning

Description

A procedure definition uses an incorrect number of arguments.

This frequently occurs because the number of "#s" used in the reference to scan inputs or scan outputs does not match the number of scan chains defined. It can also occur in various capture procedures if the number of #'s for force PI or measure PO does not match the number of inputs or outputs in the design.

What Next

If the rule severity is not set to error, then the correct number is substituted. You can write out the corrected DRC file using the `write_drc_file` command and compare it to the original.

DRC Rule V17

Message Text

Vector Rule V17: Line L (filename), clock *name* should be pulsed, not forced on.

Severity

Error

Description

DRC procedures must pulse clocks by assigning them the value "P" or hold the clocks off by assigning them their off state.

TestMAX ATPG does not support holding a clock to its on state for one or more tester cycles. The reason is that this would require a tester channel definition that would be RZ(RO) on one cycle, but then NRZ(NRO) on the next. Very few ASIC vendors or testers like to have the signal type switch dynamically from cycle to cycle.

What Next

You can downgrade the severity of the violation to warning or ignore to continue, but when you write patterns you will see inconsistencies across the various pattern formats. Some formats (Verilog, VHDL, STIL) will support the force "on" across multiple test cycles while others WGL, FTDL, TDL91, TSTL2 will convert the force on into a pulse in each tester cycle. The suggested solution is to change the clock forced to its on state of 1/0, into a force to "P" or pulse, or a force to its off state if that is acceptable. Following are some additional ideas you might want to consider:

Does the clock or asynchronous set/reset need to be applied? Many users feel they should apply a global reset for good measure or apply at least one clock in the test_setup procedure but ATPG generally doesn't care if the design is initialized or not unless nonscan cells need to be initialized to enable other ATPG operations. So you might be able to remove the reset or clock event. Try replacing the on value with an off value and see if the scan chain check of "run drc" still passes and what happens to the various contention checks such as Z4.

Another item to review is whether the set, reset, or clock is really used outside of the test_setup procedure? If you have a PI constraint on the set/rst/clock pin so that it is always off (except for test_setup), then there is really no reason to declare this pin as a clock and you can remove it from the defined clock list. After this is done, you are then free to force the value either to 1 or 0 in test_setup, as long as you restore the PI constraint value by the end of test_setup.

If you have a JTAG design, or for some other reason you are using the `set_drc-clock <some_pin>`, then you have the same condition as where a reset declared as a clock will not be used for patterns. You can then remove the pin from the clock list, and add it to a PI Constraint list and are then free to force it 0 or 1 in test_setup.

The option `set_drc-allow_unstable_set_resets` supports asynchronous set/rst pins without defining them as clocks. By selecting this option you can omit 'rst' from the clock list. TestMAX ATPG then creates patterns using this pin as an NRZ/NRO pin and you are free to force it to 0 or 1 in test_setup.

If the reset is actually used for pattern generation, then another alternative is to use two STIL timing definitions, one for test_setup and another for all other patterns. Suppose the standard test cycle is 100ns but you wish to initialize reset for 2000ns. Then define a second timing definition that is used in the test_setup procedure in which the cycle period is 2100ns and the reset is pulsed from 50ns to 1950ns. *Note* that many vendors using WGL as a pattern format support only one timing definition.

```
#
# Example using two timing definitions to achieve long reset
#
Timing {
  WaveformTable "NORMAL" {
    Period '100ns';
```

```

Waveforms {
  "grp1" { 01NZ { '0ns' D/U/N/Z; } }
  "grp2" { 01NZ { '5ns' D/U/N/Z; } }
  "clk" { P { '0ns' D; '45ns' U; '55ns' D; } }
  "rst" { P { '0ns' U; '45ns' D; '55ns' U; } }
  "outpins" { LHXT { '0ns' X; '95ns' L/H/X/T; } }
}
}
WaveformTable "LONG" {
  Period '2100ns';
  Waveforms {
    "grp1" { 01NZ { '0ns' D/U/N/Z; } }
    "grp2" { 01NZ { '5ns' D/U/N/Z; } }
    "clk" { P { '0ns' D; '50ns' U; '2050ns' D; } }
    "rst" { P { '0ns' U; '50ns' D; '2050ns' U; } }
    "outpins" { LHXT { '0ns' X; '2095ns' L/H/X/T; } }
  }
}
:
:

MacroDefs {
  "test_setup" {
    W "LONG"; # use 2100ns cycles
    V { clk=0; rst=P; } # apply long reset pulse

    W "NORMAL"; # switch to normal 100ns cycles
    V { clk=P; rst=1; in=0; oe=1; } # normal 'clk' pulse
    V { clk=0; }
  }
}

```

set_drc, set_rules in the TestMAX ATPG and TestMAX Diagnosis Commands Guide

DRC Rule V18

Message Text

Vector Rule V18: Mixed timing (<additional info>).

Severity

Warning

Description

In the WaveformTable section of the STIL procedures where port timing is defined it is possible to have as many different timing combinations as there are ports. Groups of ports are defined to have different timing offsets for force and/or for measure. The V18 violation

is issued when the ports do not all have a common force time and a common measure time.

The violation is due to the fact that the ATPG algorithm uses only one event sequence for its internal algorithm, despite the differing defined times that can occur for input, output, and bidirectional ports. This fundamental ATPG event sequence is:

- 1) force inputs
- 2) measure outputs
- 3) pulse capture clock or shift clock

Because of this algorithm and despite the defined timing, at least internally all of the non-clock inputs is forced at the same time, all of the measures will occur at the same time, and all of the clocks will occur at the same time.

Failure to satisfy this rule will have no affect on test coverage but there is a slight chance that patterns generated will fail simulation.

What Next

When the rule severity is not set to error the rules checking will continue and will use for an event time the largest timing value for the group of pins in question: forcing inputs, measuring outputs, pulsing clocks.

When patterns are written, the timing as originally defined is used so you have a condition where the pattern generation algorithm used a timing different than that used in the final patterns.

For test purists, you might want to adjust the defined timing so that the events are all common. This will ensure the timing in the patterns matches the timing assumed by the pattern generation algorithm. For the rest of the world, the patterns will generally work. If simulation passes, great! If simulation fails, try adjusting the timing to eliminate V18.

DRC Rule V19

Message Text

```
Vector Rule V19: Line L (filename), clock <port> undefined in test  
protocol file.
```

Severity

Warning

Description

The indicated port was previously defined as a clock, but was not defined as a clock in the test procedure file used during DRC.

If the severity is ignore or warning (the default), then the port is treated as a clock and DRC processing continues. If the severity has been changed to error, then processing stops.

What Next

Adjust the procedure file or your command sequence so that the inconsistency is removed by either removing the port from the clock list or by defining the port as a clock in the procedure file.

DRC Rule V20

Message Text

Vector Rule V20: unsupported event (more info here).

Severity

Error

Description

The defined order of events in a capture procedure is not supported or there are extra events in the procedure.

Failure to pass this rule check indicates that a procedure has defined events in an order which is not supported by the Basic-Scan or Fast-Sequential ATPG algorithms. Capture procedures must define events in the following order:

force values on inputs

measure values on outputs

pulse one defined clock and all equivalent ports (optional)

This rule violation occurs if the events have been defined in an order which does not match this sequence or if extra events (such as explicitly setting a bidirectional control pin) have been defined which are not expected.

The order of events is defined both by the number of "V{...}" structures used as well as the port timing defined in the timing section. For forces and measures within the same "V", the port timing determines the order of events.

Use of Full-Sequential ATPG provides more flexibility in the event order of capture procedures. When using the Full-Sequential ATPG algorithm you can make use of the Sequential Capture Procedure.

What Next

Review the capture procedures indicated and consider changing the order of events or the defined timing to match the sequence above, or considering use of Full-Sequential and the 'sequential_capture' procedure.

If you wish to proceed without changes, you can adjust the severity of this rule from an error to a warning using the `set_rules` command. However, be advised that you do this at your own risk and that the unsupported events or extra events are not part of the ATPG algorithm and are not considered when checking contention and performing other DRC checks. Patterns can be generated with undetected contention or can fail in simulation.

With V20 set to warning or ignore TestMAX ATPG will generate patterns using the following environment for Basic-Scan or Fast-Sequential patterns:

All data change events will occur at the same time.

Next, all data measure events will occur at the same time.

Next, all clock events will occur at the same time. So multiple clocks defined as "PI Equiv" with different timing will pulse simultaneously, but not with the unique timing the user has asked for.

Then, when patterns are written to a specific output format, the user desired timing is used to create the patterns. So the patterns are written with a different timing than what was actually used to generate those patterns and this could result in patterns which fail in simulation.

DRC Rule V21

Message Text

Vector Rule V21: unsupported old syntax (more info here).

Severity

Error

Description

You are using old syntax which is no longer supported.

What Next

You will need to identify and modify the old syntax to proceed.

If the parse error is reported in the DRC file, one common possibility is the file contains the old syntax for defining the Equivalence relationships. This old syntax was not IEEE 1450.0 or P1450.1 compliant and should now be changed to be P1450.1 compliant:

Old non-compliant syntax:

```
E { "POS"=0; "NEG"=1; } # inverting relation

E { "POS"=1; "NEG"=0; }

E { "P1"=0; "P2"=0; "P3"=0; } # non-inverting relation

E { "P1"=1; "P2"=1; "P3"=1; }
```

New STIL P1450.1 syntax:

```
E "POS" \m "NEG" ; # inverting relation

E "P1" "P2" "P3" ; # non-inverting relation
```

Perhaps the easiest way to make these changes is to use command line options to define PI equivalences, comment out all the old `E` lines in the DRC file, run `drc`, and then use the `write_drc_file` command to write a new file. This new file will use the new P1450.1 syntax for the equivalence statements.

DRC Rule V22

Message Text

Vector Rule V22: Invalid waveform (more info here).

Severity

Error

Description

There has been an inconsistency or illegal waveform definition attempt in the DRC/STL procedure file procedure file. This can occur if input WaveformChars {0,1,Z,N} are defined for output pins, or output WaveformChars {L,H,T,X} are defined for input pins, or from other causes.

What Next

You can downgrade the severity of this violation to a warning. However, this is not advised because the affect upon writing patterns cannot be predicted. In some cases, pattern syntax might be corrupted and you can have either bad patterns or patterns which will not read back in.

It is advisable to correct the cause of the V22 before continuing. Use the `report_violations` command to get explicit information for a particular V22 violation. The additional info text in the message should assist in identifying which signal definition has the problem and which line in the file the problem is near. Double check consistency between defined pin directions and the WaveformChars definitions, which are listed.

DRC Rule V23

Message Text

```
Vector Rule V23: Line L (file), strobe conflict (Measure point coincides  
with events at time = T). (V23-1)
```

Severity

Warning

Description

It is always best to try to measure circuit outputs at times when the circuit is stable and there are no inputs or outputs changing. TestMAX ATPG generates this violation when a calculated measure strobe occurs at the same time as an Extended VCD event.

For example, if the Extended VCD file contains an event at '#12500' and you specified a strobe period when reading the file of 500 nS, then an integer multiple of strobe time occurs exactly at time 12500 (500*25). If the event at time #12500 updates the circuits inputs *and* or *or* changes the expected value of the circuits outputs then TestMAX ATPG has no way of knowing whether it should make the measure using the previous values before time #12500, or process the new input changes and output expect values and then use these new values to make a measure.

In this situation TestMAX ATPG will always insert a measurePO event using the circuit state before any changes described by the new event. Since this cannot be the correct assumption for the patterns, a violation message is issued. The violation message indicates the line number `L` in the VCDE file at which strobe conflict occurs as well as the event time `T`.

What Next

If the behavior of inserting a measurePO using values before the event is acceptable, then you can ignore the violation.

You can also try to adjust either the strobe period or the strobe offset to try to shift the points in time at which measure strobos occur. You might be able to move the strobe time by a few time units and avoid the ambiguity of the strobe time collision. For example, using a `-strobe offset 490 ns` in conjunction with a strobe period of 500 ns causes the first strobe at offset 490 nS instead of 500. Subsequent strobos will occur at times of $T = 490 + (500 * N)$ and the strobe that was originally at 12500 will shift to 12490.

DRC Rule V24

Message Text

Vector Rule V24: Time T, PO measures with clocks active not supported for pattern mapping. (V24)

Time T, PI changes with clocks active not supported for pattern mapping. (V24)

Time T, Staggered clocks not supported for pattern mapping. (V24)

Time T, Overlapped clocks not supported for pattern mapping. (V24)

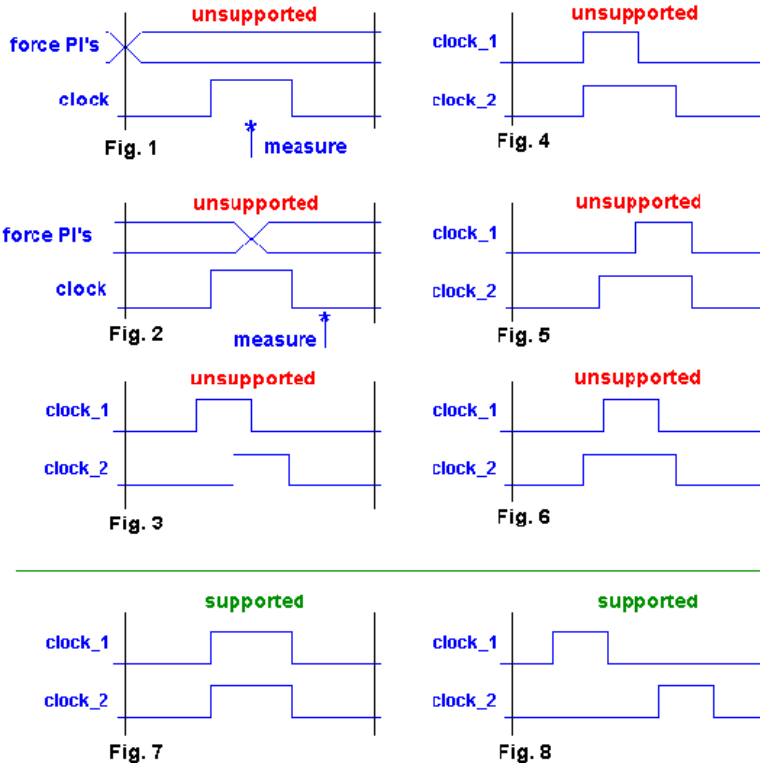
Severity

Error

Description

Pattern mapping requires certain restrictions on the input patterns in order for the mapping algorithm to have the best chance of succeeding. Certain pattern event ordering is not currently supported:

- output measures with clocks active (fig. 1)
- inputs changing with clocks active (fig. 2)
- staggered clocks (fig 3., fig 5.)
- coincident clocks with different trailing edges (fig 4.)
- overlapped clocks with different durations (fig. 6)



When an unsupported sequence is detected the time, T , near where the problem is detected is extracted from the Extended VCDE file and reported. Depending upon the type of problem, this reported time might be at or just near the problem spot in the incoming patterns.

Pattern mapping effort is aborted when a V24 is detected.

What Next

The patterns must be modified to match supported input styles.

For an output measure occurring with an active clock it might be possible to shift the measure time by adjusting the strobe options used with the run mapping command.

For inputs that change with an active clock, or the variations of staggered and overlapping clocks that do not have identical edges and durations, the patterns must be changed before they can be successfully processed into internal forms and mapping attempted.

DRC Rule V25

Message Text

Vector Rule V25 : Line L (filename), unload mode enable not defined.

Severity

Error

Description

A per_load mode is active and an unload mode enable port has not been defined. (An unload mode enable port is not required in per_shift mode.)

What Next

Downgrading this rule to pass DRC and run_atpg can result in increased pattern count for the same test coverage.

DRC Rule V26

Message Text

Vector Rule V26 : Line L (filename), unload mode enable port *port_name* incorrectly set.

Severity

Error

Description

Incorrect patterns might be generated because the unload mode enable port is incorrectly set.

What Next

In per_shift mode, the unload mode enable port (if defined) must be set to 1 in the load_unload preamble.

In per_load mode, the unload mode enable port (which must be defined, see [V25](#)), must be set to 0 in the load_unload preamble. Failure to do so can result in incorrect patterns when per-shift data for the unload mode enable is not part of the pattern.

DRC Rule V27

Message Text

Vector Rule V27: Line L (filename), unload mode control port *port_name* incorrectly set.

Severity

Error

Description

Incorrect patterns might be generated because the unload mode control port is incorrectly set.

What Next

In per_shift mode, the unload mode control ports must be set to 0 in the load_unload preamble. Failure to do so can result in incorrect patterns when per-shift data for the unload mode controls is not part of the pattern.

In per_load mode, the unload mode control ports must not be set, or else the patterns will be incorrect.

DRC Rule V28

Message Text

```
Vector Rule V28 : Line L (filename), pipelining (load_depth=<depth>,
unload_depth=<depth>) not supported.
```

Severity

Error

Description

The specified pipelining per-load patterns are not supported.

What Next

See limitations on incompatibility of pipelining with per-load patterns.

DRC Rule V29

Message Text

```
Vector Rule V29
```

```
("resolution loss"): Too many events or too small a resolution in
procedure name.
```

Severity

Warning

Description

This message appears when the timing information in the STL procedure file, STIL, or Verilog file exceeds the available integer precision (for example, 32 bits). In most cases,

this violation does not affect DRC or ATPG since only the ordering of the timing is affected and not the magnitude. However, a V29 message is issued when scaling causes a loss of resolution. If this message is generated when procedure definitions are read in the STL procedure file, the procedure definitions might be incomplete.

Overflow cases are handled as follows:

32-bit time delays are supported.

33-bit and higher delays are supported on 64-bit platforms. For 32-bit platforms, the delay is computed based on procedure information, which might cause a loss of resolution. All times are divided by 10 as often as needed to adjust them to a 32-bit value.

123000000000000 is transformed into a 32-bit value with no loss of resolution while using the appropriate timescale.

1230000000000001 causes a V29 warning.

What Next

Review the context of the V29 message and change the timescale if this prevents the time values from exceeding the 32-bit value. If this message is generated when reading a procedure (such as the test_setup procedure), the DRC and ATPG issues might be caused by truncated procedure events.

If the events in the test_setup procedure exceed the 32-bit time value limit, all subsequent TestMAX ATPG operations (such as run-time and memory requirements) are affected. This condition is sometimes caused by loops with a large number of events intended for locking PLLs. In this situation, you can insert the `DontSimulate ATPGDRC` construct before each loop, and insert the `UserKeywords DontSimulate` construct before the `MacroDefs` block. These constructs prevent the loop events from affecting the TestMAX ATPG operations and preserve the loops throughout the flow. Loops are written in the patterns even though the loop events bypass other TestMAX ATPG operations. PLL models are black boxes in TestMAX ATPG, and events sent exclusively to these models are not pertinent to TestMAX ATPG operations.

DRC Rule V31

Message Text

Vector Rule V31:

```
Waveform simulation events (H and L Waveforms across Timing have a
different number of leading Z events(n,n); check consistency) Waveform
simulation events (Both multiple and single-cycle capture procedures are
defined; (name) is [single|multi]-cycle. Affects bidirectional handling
in external simulation Waveform simulation events (Waveforms [H,L,T,X]
need Z event in single-cycle procedure (name) Waveform simulation events
```

(Waveforms [H,L,T,X] must not have a Z event in multiple cycle procedure (name))

Severity

Warning

Description

V31 rule violations flag potential issues with external simulation of the STIL data, with the events defined in the waveforms associated with bidirectional signals in the design. These are warnings only; there might be environments where the waveforms cannot be specified for these simulation behaviors, and these warnings will not affect TestMAX ATPG operation. However it is important to note these warnings if simulation of the STIL data is to be performed.

All of these rules are related to the effect of an initial drive-off event in measure waveforms on bidirectional signals, as the first event. This is typically expressed as `0ns:Z` in the waveform definition. For proper simulation, this event is required for capture procedures that contain 1 cycle; when the capture procedure is defined as multiple cycles this event must not be present.

There are four messages generated under the V31 rule:

The message H and L Waveforms across Timing have a different number of leading Z events(n,n); check consistency identifies that there were a different number of leading Z events defined across all `H` and `L` waveforms. This might indicate a consistency problem in the waveform definitions, and should be reviewed.

The message Both multiple and single-cycle capture procedures are defined identifies that there are a mixture of capture behaviors - some capture procedures are defined with a single capture Vector, others are defined with multiple capture Vectors. This will result in additional violations if the same timing is used across the different capture procedures, and it indicates a potential problem for simulation of the bidirectionals. If possible, all capture environments should be uniform, but this is not a requirement. If each capture environment defines separate timing then each capture procedure can be unique, and this warning can be ignored.

The other two Waveforms messages indicate specific waveforms that are either missing the leading Z event which is required for simulation (for single-cycle captures), or there is a leading Z event that should not be present (for multiple-cycle captures).

What Next

If an external simulation of the STIL data is not intended, these messages can be ignored.

For the Waveforms messages, consider adding or deleting the leading Z event as specified in the message. If there are mixed single-cycle and multiple-cycle capture procedures, it might be necessary to define unique timing (WaveformTables) for each

capture procedure to eliminate the Waveforms messages (in this context, you will always get a V31 message advising that there are mixed environments for capture).

When the message Both multiple and single-cycle capture procedures are defined is generated, the procedure name referenced is the first procedure where the number of cycles changed from the prior definitions. This might not be the only procedure; all procedures defined should be reviewed to eliminate this situation if possible.

Note that the warning generated for multi cycle capture procedures and the Z event on measured outputs (H.L, and T) will only cause a problem if the patterns are written when the `write_patterns-measure_forced_bidis` option is specified. If this option is not used, then any warnings associated with these states can be ignored. However, there are no issues with correcting the waveforms in any circumstance; nor is there a need to address these warnings for all potential output formatting options.

DRC Rule V32

Message Text

Vector Rule V32:

```
Incorrect event sequence (<wft-name>; PI time (<integer>) must be less  
than PO time (<integer>)
```

```
Incorrect event sequence (<wft-name>; PO time (<integer>) must be less  
than clock LE (<integer>) on <signal-name>
```

Severity

Warning

Description

The WaveformTables used for path delay testing ("_launch_WFT_", "_capture_WFT_", and "_launch_capture_WFT_") must follow certain timing restrictions. V32 messages are generated during DRC when these restrictions are violated.

These are the same messages generated for M336; please see M336 for details (an M336 message is generated if the fault model is set to transition after DRC has run).

What Next

See M336 for details.

DRC Rule V33

Message Text

Vector Rule V33: Missing constrained signal (*name* is constrained to <0|1|X|Z> for capture, not defined in load_unload)

Severity

Warning

Description

This message is generated for each signal that has been set to a constrained value, but is never applied in the load_unload procedure. By default, these signals assume their constrained values during TestMAX ATPG application of the load_unload and shift operations, in particular during DRC analysis. This behavior might not propagate through STIL test flows unless this data is explicitly provided. In some situations, the constrained state is necessary for the function to operate properly.

What Next

If you are using WGL output, the required states are provided and this warning can be ignored. If you are using STIL data, then it is recommended that you review the missing signals *and* or *or* provide for an explicit state on these signals in the load_unload procedure.

You may use the command `set_rulesV33 -autofix` to automatically insert these missing assignments in the load_unload procedure during STIL write. This option suppresses generation of the V33 rules, so non-STIL-based flows may use this option to remove this warning. Only STIL output is affected, including STL procedure file generated from `write_drc` commands.

If you do not want the load_unload procedure to be modified, you can use the command `set_drc-noconstraints`, and check the behavior of the load_unload without this data present. This option modifies the simulation of the load_unload procedure to not apply constrained values on signals. Using this option validates if the constrained signals affect the behavior of the load_unload procedure, if the constrained signals are sufficiently modeled in the design to reflect their operational effects.

DRC Rule V34

Message Text

(Version 1) Line *N* (*filename*), Invalid internal clock timing: There is no ClockTiming block named '*string*' in ClockStructures '*string*' (V34-1)

(Version 2) Line *N* (*filename*), Invalid internal clock timing: Invalid period for "*string*" (V34-1)

(Version 3) Line *N* (*filename*), Invalid internal clock timing: Missing period for "*string*" (V34-1)

(Version 4) Line *N* (*filename*), Invalid internal clock timing: Waveform events must be contained within one period for "*string*" (V34-1)

(Version 5) Line *N* (*filename*), Invalid internal clock timing: Waveform events must be monotonically increasing for "*string*" (V34-1)

(Version 6) Line *N* (*filename*), Invalid internal clock timing: Inconsistent off_state specification for clock string (V34-1)

(Version 7) Line *N* (*filename*), Invalid internal clock timing: Waveform events must be contained within one period for "*string*" (V34-1)

(Version 8) Line *N* (*filename*), Invalid internal clock timing: Cannot slow down capture from '*string*' to '*string*' beyond 1 cycles. (V34-1)

(Version 9) Line *N* (*filename*), Invalid internal clock timing: Clock string cannot safely be synchronized in Synchronization Group string. (V34-2)

Severity

Warning

Description

This violation is reported when you make an incorrect specification in the ClockTiming block of the STIL procedure file. A clock timing definition with a V34 violation is discarded and the clock's timing relationships are not reported by the `report_clocks -capture_matrix` command. ATPG will treat the clock as asynchronous to the other clocks, which reduces the test coverage. The various versions of the V34 message are described as follows:

Version 1 – The specification made by the `set_drc -internal_clock_timing` command did not match a named ClockTiming block in the STIL procedure file.

Version 2 – The period for the "*string*" statement must include a positive integer followed (without a space) by ns or ps.

Version 3 – The period for the "*string*" statement is missing or invalid. It must include a positive integer followed (without a space) by ns or ps.

Version 4 – The argument for the Period statement is too short to contain both arguments for Waveform statement.

Version 5 – The first argument for the Waveform statement must be at an earlier time than the second argument.

Version 6 – The first argument of the Waveform statement must be equal to or greater than 0ns or 0ps.

Version 7 – The difference between the arguments of the Waveform statement must be less than the Period argument.

Version 8 – Because a MultiCyclePath is defined between the clocks, the pulses be moved farther apart. But the `set_drc -multiframe_paths` command was not set.

Version 9 – This message accompanies one of the other versions of the V34 message. Correct one or more of the other message(s) first, and this version will no longer print.

What Next

Fix the violated syntax in the STIL procedure file so ATPG can use the clock as an synchronous clock.

DRC Rule V35

Message Text

Vector Rule V35:

(Version 1) Line *N* (*filename*, Invalid clock constraint: *string* " is not a declared constrainable object" (V35-1)

(Version 2) Line *N* (*filename*, Invalid clock constraint: Non-clock entity "*string*" cannot be pulsed. (V35-1)

(Version 3) Line *N* (*filename*, Invalid clock constraint: Constraints assignment is truncated to the size of "*string*" (V35-1)

(Version 4) Line *N* (*filename*, Invalid clock constraint: Assignment to InstructionRegister "*string*" is incomplete; padding with dont_cares (V35-1)

(Version 5) Line *N* (*filename*, Invalid clock constraint "*string*" is an unusable constraints assignment (V35-1)

(Version 6) Line *N* (*filename*, *string* is not a constrainable clock (V35-1)

Severity

Warning

Description

A V35 violation is reported when you make an incorrect specification in the ClockingProcedure block of the STL procedure file. A clocking procedure with a V35 violation is discarded and is not reported by the `report_clocks -constraints` command.

If other ClockingProcedure blocks exist that don't have V35 violations and are valid, then ATPG generates patterns. But if the violated clocking procedure represents real clocking behavior, the test coverage is lower than it should be. In this case, you need to fix the violated syntax so that the clocking procedure can be used by ATPG.

The various versions of the V35 message are described as follows:

Version 1 – The clocking procedure specified a pin path name that is not specified in the ClockStructures block as either a clock or a clock instruction register element.

Version 2 – The clocking procedure specified a P in the clock instruction register values.

Version 3 – The clocking procedure specified too many bits for the clock instruction register.

Version 4 – The clocking procedure did not specify enough bits for the clock instruction register.

Version 5 – The clocking procedure specified invalid characters for a clock or in the clock instruction register.

Version 6 – A reference clock is specified in the ClockingProcedure block.

What Next

If the violated clocking procedure represents real clocking behavior, you need to fix the violated syntax so that it can be used by ATPG.

DRC Rule V36

Message Text

Vector Rule V36: Failed to map external chain *chain_name* to an existing codec.

Severity

Fatal Error

Description

This message is generated for each externally visible chain that has a codec-association constraint statement defined (Compressor, LoadPipelineStages, or UnloadPipelineStages

statements) that does not match any defined codecs in the design. DRC is halted after reading the SPF on this error, and other errors are likely.

What Next

Review the constraint definitions on this chain against the codecs defined for the design, and identify a proper matching codec to use.

DRC Rule V37

Message Text

Vector Rule V37: External chain *chain_name* used default mapping to a codec; no pipeline or compressor data.

Severity

Warning

Description

This message is generated for each externally visible chain that does not have explicit codec-association constraints present (Compressor, LoadPipelineStages, or UnloadPipelineStages statements). This warning is only generated when one or more other externally visible scan chains are defined with codec-association statements.

Mixing explicit associations and default assignments creates a non-robust environment. This condition may change default assignments and the SPF. It also might cause the scan chains without associations to change to other compressors or map to an unexpected compressor that doesn't have the correct pipeline context. You should always use explicit association statements if any external chain has an explicit association statement.

What Next

To eliminate this warning and provide consistent scan chain mapping, review the scan chain definitions and add explicit codec-association statements to the scan chain definitions identified in the message.

Chains with explicit association statements prevent codecs that match these constraints from being used as default codecs (for chains that are missing explicit association statements). A codec that has associated explicit external chains requires all associated chains to use explicit statements.

DRC Rule V39

Message Text

Vector Rule V39: Line L (filename), missing state in
load_unload(<additional info>).

Severity

Warning

Description

The indicated missing signal has a constraint value (0/1) but gets 'N' during the
load_unload procedure.

ATPG shows the following violation message:

```
Warning: Line L (filename), missing state in load_unload(Constraint pin
"<missing signal>" is not set to its constraint value in "load_unload").
(V39-1)
```

ATPG generates patterns that satisfy the defined list of PI constraints. But VCS might not
get the constraint value during the load_unload procedure if it is not specified in the STIL
procedure file. This may lead to VCS failure. V39 check is added to avoid VCS failure.

What Next

If this violation occurs during run_drc then the missing state is automatically added to
the pre_shift vector statement of the load_unload procedure. You can write out the
corrected DRC file using the write_drc_file command and compare it to the original. If
the load_unload procedure does not have a pre_shift vector, then it does not add the
missing signal.

Category X - X-State Rules

See Also

- [X1 - sensitizable feedback path](#)
- [X2 - X-source gates that propagate to a scancell](#)

DRC Rule X1

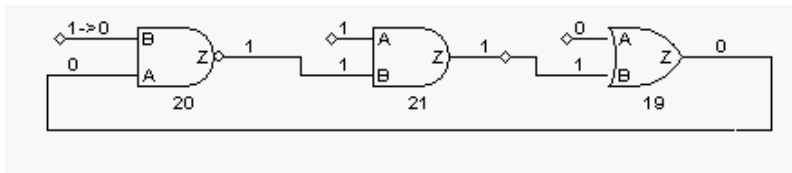
Message Text

DRC Rule X1: Feedback path network N is sensitizable through source gate G.

Severity

Warning

One example, many variations are possible:



Description

Feedback paths must not be sensitizable.

A combinational feedback path was also found to be sensitizable. In other words, a feedback loop that completes the path back on itself and results in a potential oscillation or ring driver.

Violation of this rule is only an indication of a potential sensitizable feedback path. This rule considers the effects of PI constraints and ATPG constraints created with the `-drc` option, but it does not consider the effects of nonscan cells with constant values, cell constraints, or the effects of PI equivalences. You need to investigate further to determine if there is truly a sensitizable feedback path.

Any patterns that would pose a risk of oscillation are identified and the expected values masked off so there is no danger of generating ATPG patterns that would miscompare. However, the opportunity for the design to oscillate during simulation of ATPG patterns is still present even though the expected data has been masked. This oscillation can degrade simulation performance.

In general, any feedback path (B23 warning) can cause the test generator to fail to justify all necessary test conditions, which can result in a lower test coverage. This failure to justify can happen for any feedback path but the risk is greater if the feedback path is sensitizable. The presence of B23 and X1 violations can produce degraded runtime performance.

In the violation message, `N` is the feedback path ID assigned by TestMAX ATPG when identifying combinational feedback loops. `G` is the gate ID of the ATPG primitive gate where there is the potential for oscillation to start.

What Next

You can use the `report_feedback_paths` command to review the number of gates and sources in the feedback loop. You can also use it to get a detailed list of instance pathnames of gates in the loop. The "source" gates of the feedback loop are always listed first. The feedback path analysis uses test generation, whose parameters are controlled using the `set_atpg` command. X1 analysis is affected by the `set_atpg-abort_limit` switch, since it uses ATPG to figure out if the feedback path is sensitizable. The default setting is 10 aborts before the analysis can complete.

You can analyze an X1 violation using the graphical schematic viewer by selecting the **ANALYZE** button on the left, and then choosing the specific occurrence of the X1 violation from the menu. You can also perform an analysis by manually entering the `analyze_violation` command with the `-display` option. This sets the gate reporting to the pattern data that sets up the sensitization condition and displays the gates in the failing feedback path along with the net data that enables that path.

You can find the X1 violation is nothing to worry about due to some controls not considered during the analysis such as nonscan cells that are constant or PI equivalences. You can then safely ignore the violation.

If the potential for oscillation is real, you might wish to consider altering your design to eliminate the feedback loop or at least to prevent the X1 violation, at least in ATPG mode.

You might wish to consider manually breaking the loop for ATPG efforts by disconnecting selected gates along the way. The `add_net_connections` command along with the `TIEX` and `-disconnect` options can be used to disconnect all inputs from selected gates in the feedback path. However, the use of net connections can lead to performance issues during circuit flattening, so you might want to modify the netlist (used only in the ATPG environment) to break the loop rather than use the `add_net_connections` command.

DRC Rule X2

Message Text

DRC Rule X2: X source T gate G1 is propagatable to N scan cells (G2).

Severity

Ignored

Description

Note the following definitions:

T is the gate type of the X-source gate

G1 is the `gate_id` of the X-source gate

`N` is the number of capturing scan cells of the X-source gate

`G2` is the `gate_id` of the first capturing scan cell of the X-source gate

This rule checks for gates that are potential sources of X values that can propagate to a scan cell. The following possible X-source gates are considered:

Gates with a connection to the global TIEZ gate

Gates (except BUS gates) with a connection to the global TIEZ gate

TIEZ gates

TIEZ gates, except those that connect directly to a BUS gate

BUS gates identified as controllable or that can potentially attain a Z state

TSD or SW gates that do not connect directly to BUS gates

WIRE gates identified as controllable

DLAT gates identified as non-hot transparent latches

DLAT or DFF gates associated with scan cells constrained to X

DLAT or DFF gates identified as nontransparent nonscan cells

DLAT or DFF gates with user-selected capture masking

DLAT or DFF gates with capture masking due to a failing DRC rule

RAM gates

ROM gates with memory values that are not fully defined

Feedback path source gates of sensitizable feedback paths (those that fail [Rule X1](#))

PI or PIO gates constrained to an X value

Note that constrained values from pin constraints and constant value cells are considered when determining whether an X-source gate is propagatable to a scan cell. Any X-source gate considered propagatable to at least one scan cell causes a violation of the X2 rule.

What Next

No action is required to correct this problem: ATPG can tolerate captured Xs under normal conditions. If you want to avoid Xs during ATPG, you can use automated violation analysis to help debug this problem. To enable this analysis, specify the `set_rules X2 warning` command, then run DRC. This analysis displays the failing X-source gate, the capturing scan cell gate, and all the gates in the paths between them. The pin data is set to the `constrain_value` option.

Category Z - Tristate Rules

See Also

- [Z1 - bus capable of contention](#)
- [Z2 - bus capable of holding Z state](#)
- [Z3 - wire capable of contention](#)
- [Z4 - bus contention in test procedure](#)
- [Z5 - BIDI pin not at input mode](#)
- [Z6 - ATPG constraint violation in test procedure](#)
- [Z7 - unable to prevent contention for circuit](#)
- [Z8 - unable to prevent contention for bus](#)
- [Z9 - bidi bus driver enable affected by scan cell](#)
- [Z10 - internal bus driver enable affected by scan cell](#)
- [Z11 - uncontrolled BIDI driver using BIDI control pin](#)
- [Z12 - BIDI control pin effects capture operation](#)

DRC Rule Z1

Message Text

Tristate Rule Z1: Bus gate N (G) failed contention ability check for drivers G1 and G2.

Severity

Warning

Description

A BUS primitive must not be capable of being in a contention state when PI constraints are set. A BUS primitive is automatically inserted into the design during flattening to resolve the driven value on multi driver nets. G is the gate ID of the BUS device and G1 and G2 are two drivers which could contribute to a contention situation. More might be possible but the analysis needs only two to identify a Z1 violation.

You can think of this rule check as dividing the BUS gates into two groups: Those that require further monitoring (Z4, Z7, bus analysis), and those that do not.

This violation occurs when a pattern can simultaneously enable the drivers on two or more tristate drivers to the same net, or if the analysis is terminated before completion because the ATPG abort limit was reached. Unless you select the `-nomultiple_on` option of the `set_contention` command, it also is required that the data pins be at different values to be considered a violation. BUS inputs coming from weak drive strength gates are not considered when identifying this violation.

To achieve an accurate assessment of all potential contention BUS devices this rule check is performed independently of the effects of any DRC procedures such as `test_setup` or `load_unload`. Because the analysis is independent of all procedures, the effects of nonscan cells which might be a constant by the end of `test_setup`, or might be loaded to a constant by the end of the scan chain load, are not considered.

This check does not indicate that contention is occurring, only that it is possible for it to occur and so therefore must be monitored more closely during pattern generation.

Unless finding a pattern which avoids contention cannot be found, there should be no affect on test coverage. There is no danger that patterns created can fail simulation.

What Next

If the violation was due to an aborted analysis, use the `analyze_buses` command in conjunction with the `-abort_limit` option of the `set_atpg` command to attempt to complete the analysis. After DRC, you should reduce the abort limit, otherwise your ATPG runtime is potentially much longer.

In general, BUS devices which fail Z1 and which also fail the Bus Analysis portion of DRC will require additional CPU effort during ATPG pattern generation to attempt to avoid contention. If there is an excessive number of bus contention failures as reported by the `analyze_buses` command, you might want to investigate.

The severity of this rule can be set to ignore, but be warned that this disables the BUS contention checking used by the Z4 and Z7 DRC checks and can cause those checks to fail. It is strongly recommended that you do not disable the Z1 checks.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to "pattern data" and a pattern which causes contention is selected for display on the schematic. The GSV displays the gates in the backward trace cones of the enable lines of the contending drivers. Reviewing the gates displayed can give you some insight as to whether the problem can be corrected by changing a PI constraint or clock definition or whether the problem is inherent in the design.

DRC Rule Z2

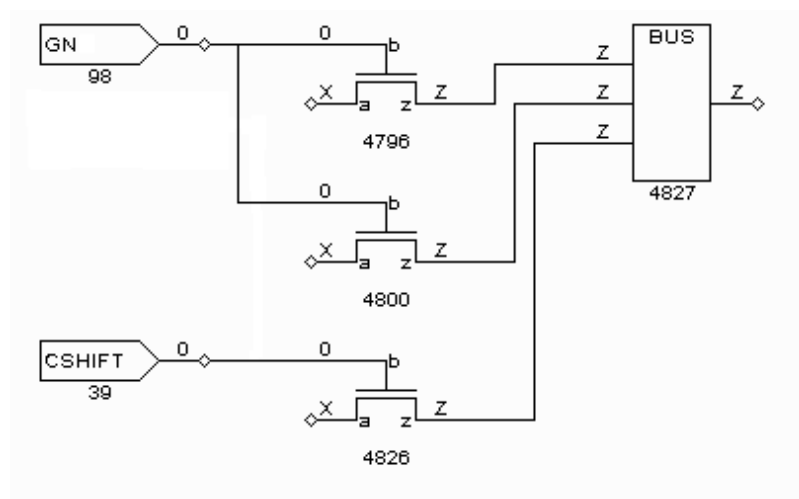
Message Text

Tristate Rule Z2: Bus gate N (G) failed Z state ability check.

Severity

Warning

One example, many variations are possible:



Description

A BUS primitive must not be capable of being in a Z state when constraints are set. A BUS primitive is automatically inserted into the design during flattening to resolve the driven value on multi driver nets.

Failure to satisfy this rule will not affect test coverage or the ability to generate patterns that pass simulation. However, the presence of internal Z states become X's as they pass through other gates and can lead to an increase in tester mask requirements if the X's make their way into scan chains or outputs.

This violation occurs when analysis aborts before completing or a pattern that can simultaneously disable all of the drivers to the same net was found.

What Next

If the violation was due to an aborted analysis, use the `analyze_buses` command to attempt to complete the analysis.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to "pattern data"

and a pattern which causes a Z state is selected for display on the schematic. The GSV displays the gates in the backward trace cones of the failing BUS primitive. Reviewing the gates displayed can give you some insight as to whether the problem can be corrected by changing a control in a test procedure or whether the problem is inherent in the design.

If this violation is acceptable, you can use the `set_rule` command to adjust its severity to ignore.

DRC Rule Z3

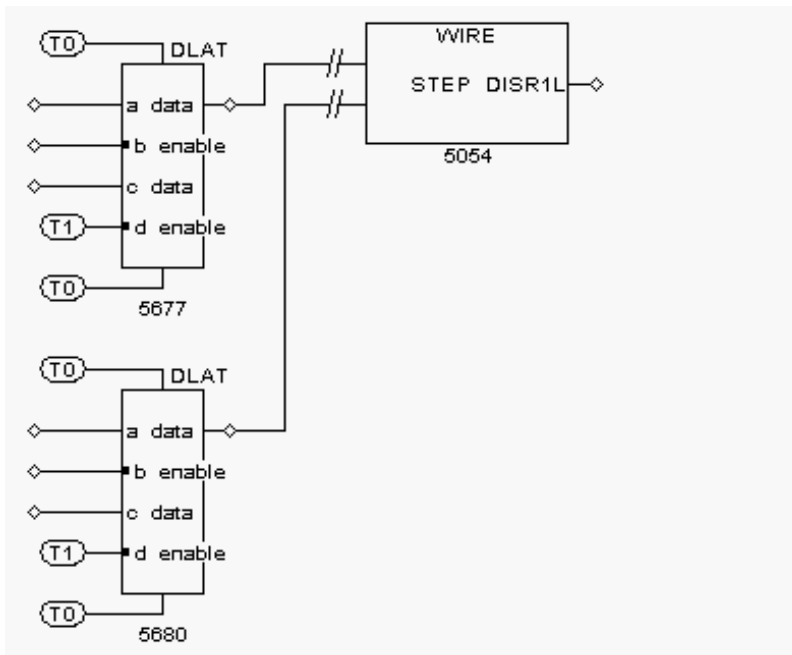
Message Text

Tristate Rule Z3: Wire gate N (G) failed contention ability check for drivers G1 and G2.

Severity

Error

One example, many variations are possible:



Description

A WIRE primitive must not be capable of being at different state when constraints are set. A WIRE primitive is automatically inserted into the design during flattening to resolve the driven value on multi driver nets that do not use tristate drivers.

Failure to satisfy this rule results in an X state on the WIRE output, which can lead to a loss of test coverage. The contending condition can also cause harmful device stress if the non tristate drivers are allowed to be in contention.

A violation occurs when a pattern can be found which simultaneously drives a different value on the outputs of two gates connected in common to a WIRE primitive. A violation can also occur if analysis is aborted before completion (low `-abort_limit` of the `set_atpg` command).

What Next

Review the `wire` option of the `set_contention` command.

If the violation was due to an aborted analysis, use the `analyze_buses` command to attempt to complete the analysis.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to "pattern data" and a pattern which causes a contention on the WIRE primitive is selected for display on the schematic. The GSV displays the gates in the backward trace cones of the contending drivers. Reviewing the gates displayed can give you some insight as to whether the problem might be overcome by changing a contention check control or applying some sort of PI constraint or whether the problem is inherent in the design.

DRC Rule Z4

Message Text

Tristate Rule Z4: Bus contention on N (G) occurred at time T of X procedure.

Severity

Warning

Description

A BUS primitive must not be in contention during the simulation of any test procedure.

`N` is the instance pathname of the BUS primitive, `G` is its `gate_ID` number, `X` is the name of the procedure, such as `shift`, `test_setup`, or `load_unload`, and `T` is the simulation time of that procedure.

Failure to satisfy this rule will not affect test coverage. However, the contention might be harmful to the device on the tester.

This violation occurs when simulation of the DRC procedures detects any BUS primitive with inputs in a contending state. A BUS primitive is automatically inserted into the design during flattening to resolve the driven value on multi driver nets.

What Next

Common causes for this violation include:

lack of control of internal tristate drivers from a top level port

forgetting to disable bidi port drive during load_unload and test_setup

forgetting to force Z on bidi ports during load_unload and test_setup

disabling Z1 analysis by setting rule Z1 to ignore

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to one of the procedures: shift, test_setup, load_unload, and so forth., and displays the gates in the backward trace cone of the failing BUS primitive.

Note that when the violation occurs in the test_setup procedure, only a single simulation value will appear on the initial schematic display. This corresponds to the time reported in the violation being analyzed. For example "... occurred at time 200 of test_setup..." would indicate the single simulation value corresponds to event time 200. You might see all the simulated events of the test_setup procedure by clicking on the `REFRESH` button. To return to a single character display, you will need to repeat the `analyze_violation` command.

Certain periods of potential contention might be allowable in the test_setup or at the beginning of the load_unload procedure. However, it is not advisable to continue with potential contention during the Shift procedure as any patterns produced can lead to device damage when applied on real silicon.

If this violation is acceptable, you can use the `set_rules` command to adjust its severity to ignore.

DRC Rule Z5

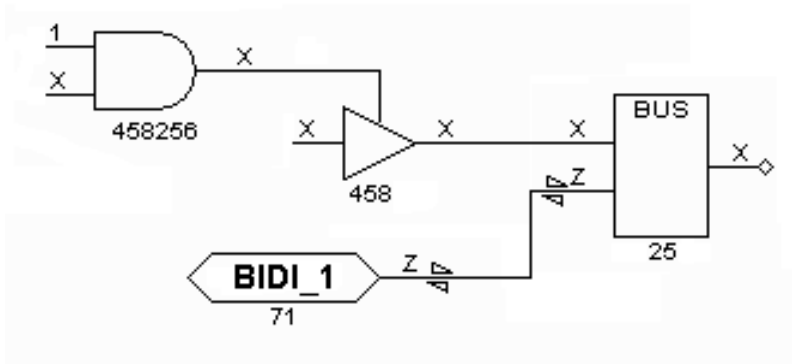
Message Text

Tristate Rule Z5: BIDI port P not set to input mode at time T of X procedure.

Severity

Ignored

One example, many variations are possible:



Description

Bidirectional ports should be in input mode during the Shift portion of the load_unload procedure and driven to a non-Z value.

Failure to satisfy this rule will have no affect on test coverage or the accuracy of any patterns created.

What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to the load_unload procedure and displays the gates in the backward trace cone of the failing PIO primitive (bidirectional port). Reviewing this displayed data can give insights in how to control the bidirectional port to satisfy the rule.

To resolve this violation investigate two areas: 1) the bidirectional enable is at an off state, 2) the bidirectional pin is forced to a 0 or 1 value in the load_unload procedure. If you are unable to guarantee the bidirectional enable is off during scan chain shifting, then this rule cannot be satisfied and you should force Z's on the bidi pins involved or a Z4 violation can occur.

Certain test methodologies insist that bidirectional ports which are not scan outputs be in a driven input mode. This is primarily intended to reduce switching noise on the tester that can occur if the bidirectional ports are allowed to change direction or state during each shift clock. Your own specific test methodology cannot have this requirement, if so you might want to set this rule's severity to ignore using the `set_rules` command.

DRC Rule Z6

Message Text

Tristate Rule Z6: ATPG constraint violation on N (G) occurred at time T of X procedure.

Severity

Warning

Description

Static ATPG constraints must be satisfied during the application of any test procedure.

What Next

You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the procedure pattern data at which the failure occurred and displays the gates in the backward trace cone of the failing ATPG constraint site.

Note that when the violation occurs in the `test_setup` procedure, only a single simulation value will appear on the initial schematic display. This corresponds to the time reported in the violation being analyzed. For example "... occurred at time 200 of test_setup..." would indicate the single simulation value corresponds to event time 200. You might see all the simulated events of the `test_setup` procedure by clicking on the `REFRESH` button. To return to a single character display, you will need to repeat the `analyze_violation` command.

DRC Rule Z7

Message Text

Tristate Rule Z7: Cannot set all buses to a noncontending state.

Severity

Error

Description

All BUS primitives must be capable of being simultaneously set to a noncontending state. A BUS primitive is automatically inserted into the design during flattening to resolve the driven value on multi driver nets.

Failure to satisfy this rule indicates that test generation cannot find a pattern that will avoid contention. Depending upon the contention checking conditions controlled by the `set_contention` command, any or all patterns generated can end up being discarded due to contention. This would then contribute to a lower test coverage and increased CPU time.

This check is performed by attempting to find a pattern which simultaneously avoids contention on all BUS devices in the design. A Z7 violation occurs if a pattern cannot be found.

Z7 checking is not performed if Z1 failures are not present. However, if there are Z1 failures, Z7 checking commences and could fail if ATPG constraints are forcing internal clocks-on in other than the first cycle.

What Next

This violation has no automated schematic viewing analysis and none is possible.

If Z1 checking has been disabled by changing the severity of Z1 to ignore, then this can contribute to Z7 violations. Use the `report_rulesz1` command to determine if Z1 is set to ignore.

If the design has Z8 violations, then investigate these violations first. A Z8 violation is issued for a `single` BUS for which no pattern can be found to avoid contention. As long as a single BUS is failing, there is no point in troubleshooting the Z7 which is a check for all BUSES.

It is important to determine if the contention prevention effort which accompanies a Z1 analysis had completed or aborted. Follow these steps:

Issue the `report_buses-summary` command to check whether any of the bus analyses were aborted (`#abort` is nonzero for contention status). If the number of aborted analyses is zero, then contention prevention completed without aborting and you can skip steps 2-5 below.

If there were aborted bus contention analysis, then determine if there were also any Z1 failures using `report_rulesz -fail`.

If there were any Z1 failures, increase the ATPG effort using the `set_atpg-abort NNN` command and re-analyze for bus contention using the `analyze_buses-all -exclusive -update` command. This rechecks all current buses capable of contention and updates their contention status.

If step #3 still shows aborted analysis, continue to increase the ATPG abort limit and try again.

If the analyze bus command completes without aborting the analysis, then retry the contention prevention check by using the `analyze_buses-prevention -all` command.

Do a `report_violationsz7`. If your Z7 is of the form Z7-nn.A, then the "A" means analysis was aborted. Increase the ATPG abort limit and try again. If you have used `set_contentionfloat` try restoring this to its default of `set_contention`.

If you observe you have zero Z1 violations and the Z7 violations cannot be resolved by increasing the ATPG abort limit or by restoring `set_contentionnofloat`, then submit a problem report with a testcase.

If the failure to satisfy contention prevention was not due to an aborted analysis (in other words, the analysis completed but could not avoid the contention), then there is no way

to avoid bus contention without modifying the design. Check for the presence of ATPG constraints using the `report_atpg_constraints` command. The presence of ATPG constraints can lead to a Z7 failure if the ATPG constraints cannot be satisfied. To continue you will either have to downgrade Z7 to a warning, or remove ATPG constraints, or disable contention checking for the design.

You can obtain a list of BUS devices contributing to the problem by using the `report_buses-contention fail` command.

If there are no Z8 violations, the inability to satisfy contention prevention is due to more than one BUS device and requires complex manual effort to find the cause. With a list of BUS devices which failed contention checks, you can use the `run_justification` command to assist in that effort.

Note: When you run the `report bus -contention fail` command, it is not uncommon to see a violation ID of the form: Z7-nn.A. the "A" at the end is an indication that analysis was aborted. You might benefit from increasing the ATPG abort limit using the `set_atpg` command.

DRC Rule Z8

Message Text

Tristate Rule Z8: Bus gate N (G) failed contention prevention ability check.

Severity

Warning

Description

A BUS primitive must be capable of being set to a non-contending state. A BUS primitive is automatically inserted into the design during flattening to resolve the driven value on multi driver nets.

Failure to satisfy this rule indicates that test generation cannot find a pattern that will avoid contention and still satisfy all other constraints and restrictions. Depending upon the contention checking controlled by the `set_contention` command, all patterns generated can end up being discarded due to contention that can contribute to a lower test coverage. If contention checking is disabled, patterns generated can fail to simulate.

This violation occurs when ATPG pattern generation can find no pattern which can simultaneously avoid contention on the BUS primitive while at the same time honoring all ATPG and other constraints.

What Next

Check the settings in use for the `set_contention` to ensure they are appropriate. This can be done using the `report_settings` command to display the current settings.

Check your define PI constraints and ATPG constraints (if any) to ensure they are appropriate.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to "none" and displays the gates in the backward trace cone of the enable lines of the BUS drivers. Reviewing this displayed data can give insights in how to control the driver enables to avoid the problem or it can indicate that a design change is required.

DRC Rule Z9

Message Text

Tristate Rule Z9: Enable of bidi bus driver N (G) connected to scan cell gate (G) .

Severity

Warning

Description

The enable line of a bidirectional pin is controlled or connected to a scan chain cell. There is a potential for driver contention between the bidirectional pin and the device tester during both scan chain shifting and after the application of a capture clock when the contents of the scan cell will change.

This rule check is performed using design connectivity plus the affects of any PI constraints, or constant load cells.

What Next

Use the `ANALYZE` button to select the violation by its violation ID and to have the violation drawn in the schematic viewer.

This rule only indicates that there are potential problems. If there are problems with scan chain shifting there will also be Z4 rule violations, which should be corrected to avoid contentions that might occur for every shift of every load.

To avoid contention due to a capture clock changing the contents of controlling scan cells, use the `-capture` option of the `set_contention` command. This causes offending patterns to be discarded if they are found to cause contention. A message is given indicating the number patterns that were rejected and there is likely some loss of test coverage due to the pattern rejection.

A general solution to avoid all problems associated with this rule is to modify the design to add circuitry which prevents the bus from attaining a contention condition in both normal operating mode as well as scan shift mode. The bus gate will now pass the Z1 rules check and will not be considered for this Z9 rule check.

DRC Rule Z10

Message Text

Tristate Rule Z10: Enable of internal bus driver N (G) connected to scan cell gate (G).

Severity

Warning

Description

The enable line of an internal three-state driver is controlled by or connected to a scan chain cell. There is a potential for driver contention between driver devices on the internal multidriver net both during scan chain shifting and after the application of a capture clock. This check is performed only for bus gates that fail the contention ability checking (rule Z1).

This rule check is performed using design connectivity plus the affects of any PI constraints, or constant load cells.

What Next

Use the schematic viewer to analyze the violation ID.

This rule only indicates that there are potential problems. If there are actual problems with scan chain shifting, Z4 rule violations will occur. All Z4 violations should be corrected, especially those that occur during the shift procedure as these might be serious.

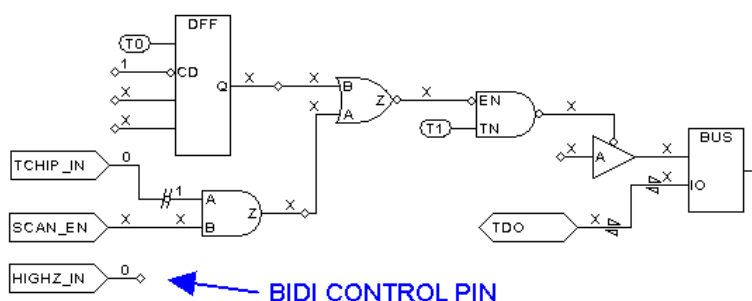
To avoid contention due to a capture clock changing the contents of controlling scan cells, use the `-capture` option of the `set_contention` command. This causes offending patterns to be discarded if they are found to cause contention or changed to a force Z if the contention occurs on a bidirectional port. A message is given indicating the number patterns that were rejected and there is likely some loss of test coverage due to the pattern rejection.

A general solution to avoid all problems associated with this rule is to modify the design to add circuitry which prevents the bus from attaining a contention condition. The bus gate will now pass the Z1 rules check and will not be considered for this Z10 rule checking.

Message Text

Severity

One example, many variations are possible:



When a bidirectional control pin (BCP) has been specified by the presence of a ReflectIO capture procedure, then ALL bidirectional pins of the design must be controlled by this BCP.

The severity of this rule is error because when the ReflectIO protocol is used the avoidance of contention cannot be guaranteed on a bidi pin if that bidi pin is not controlled by the defined BCP.

Use of a PI Constraint of Z on those bidi pins not controlled by the BCP will not provide a solution because the ReflectIO protocol reflects back as input the expected output value of the bidi pin and not the applied input value. A PI constraint affects only input value.

Use the **ANALYZE** button to select the violation by its violation ID and to have the violation drawn in the schematic viewer. This selects a pin data of `bidi_control_value` and displays the simulated values on the failing tristate driver gate along with its control logic cone and `bidi` control pin when the `bidi` control pin is at its off value.

If your design OR's the Scan Enable with the BCP control, you should try constraining the scan enable control to it's off state with an `add pi constraints` command.

Review the control logic of the tristate driver to ensure that an omission has not occurred in the defined STL procedure file procedure files. If there are no problems found in the procedure files you have a number of choices:

Change the design so that bidi control is achieved.

Look for an alternative bidi control pin.

Downgrade the severity to warning or ignore. However, it is likely that the patterns produced will have contention on the bidi pins not controlled.

Modify the procedures and command file to eliminate the use of the ReflectIO protocol. Instead, consider beginning each capture_XXX procedure with a cycle in which the PI's are forced, but the BIDI's are set to Z. Either later in the same cycle or in a separate cycle the BIDI's is forced to their intended value.

DRC Rule Z12

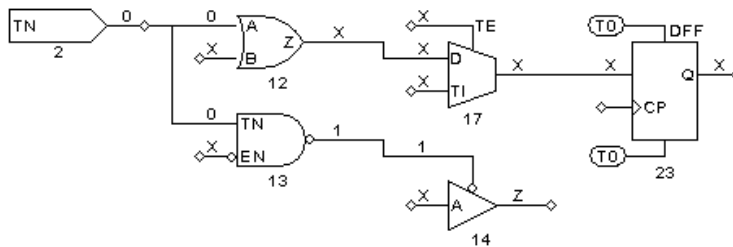
Message Text

Tristate Rule Z12: BIDI control pin P (G1) effects capture operation of <type> (G2). (Z12-1)

Severity

Error

One example, many variations are possible:



Description

When a bidirectional control pin (BCP) has been specified by either the use of the `-bidi_control_pin` option of the `set_drc` command, or by the ReflectIO capture procedure, then it must not have a path to a DFF or DLAT state element or RAM.

P is the name of the primary input which has been identified as the BCP, and G1 is its gate ID. The <type> is the type of device that can capture the BCP's value through a combinational path and is one of: DFF, DLAT, RAM. G2 is the gate ID of this device.

The severity of this rule is set at error because when using the ReflectIO protocol, any pattern developed in which the BCP is asserted is changed by the ReflectIO protocol to force BCP off during the capture clock. The ATPG pattern will have been created assuming the BCP is asserted during clock capture, but ReflectIO protocol changes this port to be off during all captures. As a result, the expected value captured into the state element is wrong and will result in a simulation mismatch reported for every pattern in which the BCP is asserted.

What Next

Use the `ANALYZE` button to select the violation by its violation ID and to have the associated gates drawn in the schematic viewer. This selects a pin data of `constrain_value` and displays the gates along the combinational gate path from the BCP to the state element which captures this value. The path drawn displays one unblocked combinational path from the BCP pin to the state element shown.

Look for a way to block the value from the BCP input point to the state element. This can be done using an `add_capture_masks` command or perhaps by locating a primary input that can be constrained with an `add_pi_constraints` command to block the path from the BCP to the state element.

Use of a PI Constraint to hold the BCP to an off state is possible, but it has a negative affect of inhibiting all bidirectional ports controlled by this BCP from ever being used in output mode. Test Coverage might be reduced by this approach.

It is highly suggested that the design be changed to eliminate the path from the BCP to the state element. If this is not possible, you might want to consider abandoning the use of the ReflectIO protocol in favor of an end-of-cycle protocol which begins each `capture_XXX` procedure with a cycle in which the PI's are forced, but the bidi's are set to Z. Either later in the same cycle or in the next cycle the bidi's is forced to their desired value (Bidi Dead-Cycle Protocol).

Category Others - All Other Rules

See Also

- [SEL_003 - Nothing implicitly matched](#)
- [SEL_004 - No objects matched](#)
- [SEL_005 - Nothing matched for %s](#)

SEL-003

Message Text

`Nothing implicitly matched '%s'`

Severity

Warning

Description

The pattern which you specified did not match any objects of the classes acceptable for this command.

What Next

Check the pattern to see if it is what you expected.

SEL-004

Message Text

`No %s objects matched '%s'`

Severity

Warning

Description

The pattern which you specified did not match any objects of the class acceptable for this command.

What Next

Check the pattern to see if it is what you expected.

SEL-005

Message Text

`Nothing matched for %s`

Severity

Error

Description

The patterns which you specified did not match any objects.

What Next

Check the values which you entered.